

This page intentionally left blank



Before You Begin

This section contains information you should review before using this book. Any updates to the information presented here will be posted at:

<http://www.deitel.com/books/jhttp10>

In addition, we provide Dive-Into® videos (which will be available in time for Fall 2014 classes) that demonstrate the instructions in this Before You Begin section.

Font and Naming Conventions

We use fonts to distinguish between on-screen components (such as menu names and menu items) and Java code or commands. Our convention is to emphasize on-screen components in a sans-serif bold **Helvetica** font (for example, **File** menu) and to emphasize Java code and commands in a sans-serif **Lucida** font (for example, `System.out.println()`).

Software Used in the Book

All the software you'll need for this book is available free for download from the web. With the exception of the examples that are specific to Java SE 8, all of the examples were tested with the Java SE 7 and Java SE 8 Java Standard Edition Development Kits (JDKs).

Java Standard Edition Development Kit 7 (JDK 7)

JDK 7 for Windows, OS X and Linux platforms is available from:

<http://www.oracle.com/technetwork/java/javase/downloads/index.html>

Java Standard Edition Development Kit (JDK) 8

At the time of this publication, the near-final version of JDK 8 for Windows, OS X and Linux platforms was available from:

<https://jdk8.java.net/download.html>

Once JDK 8 is released as final, it will be available from:

<http://www.oracle.com/technetwork/java/javase/downloads/index.html>

JDK Installation Instructions

After downloading the JDK installer, be sure to carefully follow the JDK installation instructions for your platform at:

<http://docs.oracle.com/javase/7/docs/webnotes/install/index.html>

Though these instructions are for JDK 7, they also apply to JDK 8—you'll need to update the JDK version number in any version-specific instructions.

Setting the PATH Environment Variable

The PATH environment variable on your computer designates which directories the computer searches when looking for applications, such as the applications that enable you to compile and run your Java applications (called `javac` and `java`, respectively). *Carefully follow the installation instructions for Java on your platform to ensure that you set the PATH environment variable correctly.* The steps for setting environment variables differ by operating system and sometimes by operating system version (e.g., Windows 7 vs. Windows 8). Instructions for various platforms are listed at:

<http://www.java.com/en/download/help/path.xml>

If you do not set the PATH variable correctly on Windows and some Linux installations, when you use the JDK's tools, you'll receive a message like:

'java' is not recognized as an internal or external command,
operable program or batch file.

In this case, go back to the installation instructions for setting the PATH and recheck your steps. If you've downloaded a newer version of the JDK, you may need to change the name of the JDK's installation directory in the PATH variable.

*JDK Installation Directory and the **bin** Subdirectory*

The JDK's installation directory varies by platform. The directories listed below are for Oracle's JDK 7 update 51:

- 32-bit JDK on Windows:
C:\Program Files (x86)\Java\jdk1.7.0_51
- 64-bit JDK on Windows:
C:\Program Files\Java\jdk1.7.0_51
- Mac OS X:
/Library/Java/JavaVirtualMachines/jdk1.7.0_51.jdk/Contents/Home
- Ubuntu Linux:
/usr/lib/jvm/java-7-oracle

Depending on your platform, the JDK installation folder's name might differ if you're using a different update of JDK 7 or using JDK 8. For Linux, the install location depends on the installer you use and possibly the version of Linux that you use. We used Ubuntu Linux. The PATH environment variable must point to the JDK installation directory's **bin** subdirectory.

When setting the PATH, be sure to use the proper JDK-installation-directory name for the specific version of the JDK you installed—as newer JDK releases become available, the JDK-installation-directory name changes to include an *update version number*. For example, at the time of this writing, the most recent JDK 7 release was update 51. For this version, the JDK-installation-directory name ends with "_51".

Setting the CLASSPATH Environment Variable

If you attempt to run a Java program and receive a message like

Exception in thread "main" java.lang.NoClassDefFoundError: *YourClass*

then your system has a CLASSPATH environment variable that must be modified. To fix the preceding error, follow the steps in setting the PATH environment variable, to locate the CLASSPATH variable, then edit the variable's value to include the local directory—typically represented as a dot (.). On Windows add

```
.;
```

at the beginning of the CLASSPATH's value (with no spaces before or after these characters). On other platforms, replace the semicolon with the appropriate path separator characters—typically a colon (:).

Setting the JAVA_HOME Environment Variable

The Java DB database software that you'll use in Chapter 24 and several online chapters requires you to set the JAVA_HOME environment variable to your JDK's installation directory. The same steps you used to set the PATH may also be used to set other environment variables, such as JAVA_HOME.

Java Integrated Development Environments (IDEs)

There are many Java integrated development environments that you can use for Java programming. For this reason, we used only the JDK command-line tools for most of the book's examples. We provide Dive-Into[®] videos (which will be available in time for Fall 2014 classes) that show how to download, install and use three popular IDEs—NetBeans, Eclipse and IntelliJ IDEA. We use NetBeans in Chapter 25 and several of the book's online chapters.

NetBeans Downloads

You can download the JDK/NetBeans bundle from:

```
http://www.oracle.com/technetwork/java/javase/downloads/index.html
```

The NetBeans version that's bundled with the JDK is for Java SE development. The online JavaServer Faces (JSF) chapters and web services chapter use the Java Enterprise Edition (Java EE) version of NetBeans, which you can download from:

```
https://netbeans.org/downloads/
```

This version supports both Java SE and Java EE development.

Eclipse Downloads

You can download the Eclipse IDE from:

```
https://www.eclipse.org/downloads/
```

For Java SE development choose the Eclipse IDE for Java Developers. For Java Enterprise Edition (Java EE) development (such as JSF and web services), choose the Eclipse IDE for Java EE Developers—this version supports both Java SE and Java EE development.

IntelliJ IDEA Community Edition Downloads

You can download the free IntelliJ IDEA Community Edition from:

```
http://www.jetbrains.com/idea/download/index.html
```

The free version supports only Java SE development.

Obtaining the Code Examples

The examples for *Java How to Program, 10/e* are available for download at

```
http://www.deitel.com/books/jhttp10/
```

under the heading **Download Code Examples and Other Premium Content**. The examples are also available from

```
http://www.pearsonhighered.com/deitel
```

When you download the ZIP archive file, write down the location where you choose to save it on your computer.

Extract the contents of `examples.zip` using a ZIP extraction tool such as 7-Zip (www.7-zip.org), WinZip (www.winzip.com) or the built-in capabilities of your operating system. Instructions throughout the book assume that the examples are located at:

- `C:\examples` on Windows
- your user account home folder's `examples` subfolder on Linux
- your Documents folders `examples` subfolder on Mac OS X

Java's Nimbus Look-and-Feel

Java comes bundled with a cross-platform look-and-feel known as Nimbus. For programs with Swing graphical user interfaces (e.g., Chapters 12 and 22), we configured our test computers to use Nimbus as the default look-and-feel.

To set Nimbus as the default for all Java applications, you must create a text file named `swing.properties` in the `lib` folder of both your JDK installation folder and your JRE installation folder. Place the following line of code in the file:

```
swing.defaultlaf=com.sun.java.swing.plaf.nimbus.NimbusLookAndFeel
```

For more information on locating these folders visit <http://docs.oracle.com/javase/7/docs/webnotes/install/index.html>. [Note: In addition to the standalone JRE, there's a JRE nested in your JDK's installation folder. If you're using an IDE that depends on the JDK (e.g., NetBeans), you may also need to place the `swing.properties` file in the nested `jre` folder's `lib` folder.]

You're now ready to begin your Java studies with *Java How to Program, 10/e*. We hope you enjoy the book!

Introduction to Computers, the Internet and Java

1



Man is still the most extraordinary computer of all.

—John F. Kennedy

Good design is good business.

—Thomas J. Watson, Founder of IBM

Objectives

In this chapter you'll:

- Learn about exciting recent developments in the computer field.
- Learn computer hardware, software and networking basics.
- Understand the data hierarchy.
- Understand the different types of programming languages.
- Understand the importance of Java and other leading programming languages.
- Understand object-oriented programming basics.
- Learn the importance of the Internet and the web.
- Learn a typical Java program-development environment.
- Test-drive a Java application.
- Learn some key recent software technologies.
- See how to keep up-to-date with information technologies.

- 1.1 Introduction
- 1.2 Hardware and Software
 - 1.2.1 Moore's Law
 - 1.2.2 Computer Organization
- 1.3 Data Hierarchy
- 1.4 Machine Languages, Assembly Languages and High-Level Languages
- 1.5 Introduction to Object Technology
 - 1.5.1 The Automobile as an Object
 - 1.5.2 Methods and Classes
 - 1.5.3 Instantiation
 - 1.5.4 Reuse
 - 1.5.5 Messages and Method Calls
 - 1.5.6 Attributes and Instance Variables
 - 1.5.7 Encapsulation and Information Hiding
 - 1.5.8 Inheritance
 - 1.5.9 Interfaces
 - 1.5.10 Object-Oriented Analysis and Design (OOAD)
 - 1.5.11 The UML (Unified Modeling Language)
- 1.6 Operating Systems
 - 1.6.1 Windows—A Proprietary Operating System
 - 1.6.2 Linux—An Open-Source Operating System
 - 1.6.3 Android
- 1.7 Programming Languages
- 1.8 Java
- 1.9 A Typical Java Development Environment
- 1.10 Test-Driving a Java Application
- 1.11 Internet and World Wide Web
 - 1.11.1 The Internet: A Network of Networks
 - 1.11.2 The World Wide Web: Making the Internet User-Friendly
 - 1.11.3 Web Services and Mashups
 - 1.11.4 Ajax
 - 1.11.5 The Internet of Things
- 1.12 Software Technologies
- 1.13 Keeping Up-to-Date with Information Technologies

Self-Review Exercises | Answers to Self-Review Exercises | Exercises | Making a Difference

1.1 Introduction

Welcome to Java—one of the world's most widely used computer programming languages. You're already familiar with the powerful tasks computers perform. Using this textbook, you'll write instructions commanding computers to perform those tasks. **Software** (i.e., the instructions you write) controls **hardware** (i.e., computers).

You'll learn *object-oriented programming*—today's key programming methodology. You'll create and work with many *software objects*.

For many organizations, the preferred language for meeting their enterprise programming needs is Java. Java is also widely used for implementing Internet-based applications and software for devices that communicate over a network.

Forrester Research predicts more than two billion PCs will be in use by 2015.¹ According to Oracle, 97% of enterprise desktops, 89% of PC desktops, three billion devices (Fig. 1.1) and 100% of all Blu-ray Disc™ players run Java, and there are over 9 million Java developers.²

According to a study by Gartner, mobile devices will continue to outpace PCs as users' primary computing devices; an estimated 1.96 billion smartphones and 388 million tablets will be shipped in 2015—8.7 times the number of PCs.³ By 2018, the mobile applications

1. <http://www.worldometers.info/computers>.

2. <http://www.oracle.com/technetwork/articles/java/javaone12review-1863742.html>.

3. <http://www.gartner.com/newsroom/id/2645115>.

Devices		
Airplane systems	ATMs	Automobile infotainment systems
Blu-ray Disc™ players	Cable boxes	Copiers
Credit cards	CT scanners	Desktop computers
e-Readers	Game consoles	GPS navigation systems
Home appliances	Home security systems	Light switches
Lottery terminals	Medical devices	Mobile phones
MRIs	Parking payment stations	Printers
Transportation passes	Robots	Routers
Smart cards	Smart meters	Smartpens
Smartphones	Tablets	Televisions
TV set-top boxes	Thermostats	Vehicle diagnostic systems

Fig. 1.1 | Some devices that use Java.

(apps) market is expected to reach \$92 billion.⁴ This is creating significant career opportunities for people who program mobile applications, many of which are programmed in Java (see Section 1.6.3).

Java Standard Edition

Java has evolved so rapidly that this tenth edition of *Java How to Program*—based on **Java Standard Edition 7 (Java SE 7)** and **Java Standard Edition 8 (Java SE 8)**—was published just 17 years after the first edition. Java Standard Edition contains the capabilities needed to develop desktop and server applications. The book can be used with *either* Java SE 7 *or* Java SE 8 (released just after this book was published). All of the Java SE 8 features are discussed in modular, easy-to-include-or-omit sections throughout the book.

Prior to Java SE 8, Java supported three programming paradigms—*procedural programming*, *object-oriented programming* and *generic programming*. Java SE 8 adds *functional programming*. In Chapter 17, we'll show how to use functional programming to write programs faster, more concisely, with fewer bugs and that are easier to *parallelize* (i.e., perform multiple calculations simultaneously) to take advantage of today's multi-core hardware architectures to enhance application performance.

Java Enterprise Edition

Java is used in such a broad spectrum of applications that it has two other editions. The **Java Enterprise Edition (Java EE)** is geared toward developing large-scale, distributed networking applications and web-based applications. In the past, most computer applications ran on “standalone” computers (computers that were not networked together). Today's applications can be written with the aim of communicating among the world's computers via the Internet and the web. Later in this book we discuss how to build such web-based applications with Java.

4. <https://www.abiresearch.com/press/tablets-will-generate-35-of-this-years-25-billion->

Java Micro Edition

The **Java Micro Edition (Java ME)**—a subset of Java SE—is geared toward developing applications for resource-constrained embedded devices, such as smartwatches, MP3 players, television set-top boxes, smart meters (for monitoring electric energy usage) and more.

1.2 Hardware and Software

Computers can perform calculations and make logical decisions phenomenally faster than human beings can. Many of today’s personal computers can perform billions of calculations in one second—more than a human can perform in a lifetime. *Supercomputers* are already performing *thousands of trillions (quadrillions)* of instructions per second! China’s National University of Defense Technology’s Tianhe-2 supercomputer can perform over 33 quadrillion calculations per second (33.86 *petaflops*)!⁵ To put that in perspective, *the Tianhe-2 supercomputer can perform in one second about 3 million calculations for every person on the planet!* And—these supercomputing “upper limits” are growing quickly.

Computers process data under the control of sequences of instructions called **computer programs**. These software programs guide the computer through ordered actions specified by people called computer **programmers**. In this book, you’ll learn a key programming methodology that’s enhancing programmer productivity, thereby reducing software development costs—*object-oriented programming*.

A computer consists of various devices referred to as hardware (e.g., the keyboard, screen, mouse, hard disks, memory, DVD drives and processing units). Computing costs are *dropping dramatically*, owing to rapid developments in hardware and software technologies. Computers that might have filled large rooms and cost millions of dollars decades ago are now inscribed on silicon chips smaller than a fingernail, costing perhaps a few dollars each. Ironically, silicon is one of the most abundant materials on Earth—it’s an ingredient in common sand. Silicon-chip technology has made computing so economical that computers have become a commodity.

1.2.1 Moore’s Law

Every year, you probably expect to pay at least a little more for most products and services. The opposite has been the case in the computer and communications fields, especially with regard to the hardware supporting these technologies. For many decades, hardware costs have fallen rapidly.

Every year or two, the capacities of computers have approximately *doubled* inexpensively. This remarkable trend often is called **Moore’s Law**, named for the person who identified it in the 1960s, Gordon Moore, co-founder of Intel—the leading manufacturer of the processors in today’s computers and embedded systems. Moore’s Law and related observations apply especially to the amount of memory that computers have for programs, the amount of secondary storage (such as disk storage) they have to hold programs and data over longer periods of time, and their processor speeds—the speeds at which they *execute* their programs (i.e., do their work).

5. <http://www.top500.org/>.

Similar growth has occurred in the communications field—costs have plummeted as enormous demand for communications *bandwidth* (i.e., information-carrying capacity) has attracted intense competition. We know of no other fields in which technology improves so quickly and costs fall so rapidly. Such phenomenal improvement is truly fostering the *Information Revolution*.

1.2.2 Computer Organization

Regardless of differences in *physical* appearance, computers can be envisioned as divided into various **logical units** or sections (Fig. 1.2).

Logical unit	Description
Input unit	This “receiving” section obtains information (data and computer programs) from input devices and places it at the disposal of the other units for processing. Most user input is entered into computers through keyboards, touch screens and mouse devices. Other forms of input include receiving voice commands, scanning images and barcodes, reading from secondary storage devices (such as hard drives, DVD drives, Blu-ray Disc™ drives and USB flash drives—also called “thumb drives” or “memory sticks”), receiving video from a webcam and having your computer receive information from the Internet (such as when you stream videos from YouTube® or download e-books from Amazon). Newer forms of input include position data from a GPS device, and motion and orientation information from an <i>accelerometer</i> (a device that responds to up/down, left/right and forward/backward acceleration) in a smartphone or game controller (such as Microsoft® Kinect® and Xbox®, Wii™ Remote and Sony® PlayStation® Move).
Output unit	This “shipping” section takes information the computer has processed and places it on various output devices to make it available for use outside the computer. Most information that’s output from computers today is displayed on screens (including touch screens), printed on paper (“going green” discourages this), played as audio or video on PCs and media players (such as Apple’s iPods) and giant screens in sports stadiums, transmitted over the Internet or used to control other devices, such as robots and “intelligent” appliances. Information is also commonly output to secondary storage devices, such as hard drives, DVD drives and USB flash drives. A popular recent form of output is smartphone vibration.
Memory unit	This rapid-access, relatively low-capacity “warehouse” section retains information that has been entered through the input unit, making it immediately available for processing when needed. The memory unit also retains processed information until it can be placed on output devices by the output unit. Information in the memory unit is <i>volatile</i> —it’s typically lost when the computer’s power is turned off. The memory unit is often called either memory , primary memory or RAM (Random Access Memory). Main memories on desktop and notebook computers contain as much as 128 GB of RAM. GB stands for gigabytes; a gigabyte is approximately one billion bytes. A byte is eight bits. A bit is either a 0 or a 1.

Fig. 1.2 | Logical units of a computer. (Part 1 of 2.)

Logical unit	Description
Arithmetic and logic unit (ALU)	This “manufacturing” section performs <i>calculations</i> , such as addition, subtraction, multiplication and division. It also contains the <i>decision</i> mechanisms that allow the computer, for example, to compare two items from the memory unit to determine whether they’re equal. In today’s systems, the ALU is implemented as part of the next logical unit, the CPU.
Central processing unit (CPU)	This “administrative” section coordinates and supervises the operation of the other sections. The CPU tells the input unit when information should be read into the memory unit, tells the ALU when information from the memory unit should be used in calculations and tells the output unit when to send information from the memory unit to certain output devices. Many of today’s computers have multiple CPUs and, hence, can perform many operations simultaneously. A multi-core processor implements multiple processors on a single integrated-circuit chip—a <i>dual-core processor</i> has two CPUs and a <i>quad-core processor</i> has four CPUs. Today’s desktop computers have processors that can execute billions of instructions per second.
Secondary storage unit	This is the long-term, high-capacity “warehousing” section. Programs or data not actively being used by the other units normally are placed on secondary storage devices (e.g., your <i>hard drive</i>) until they’re again needed, possibly hours, days, months or even years later. Information on secondary storage devices is <i>persistent</i> —it’s preserved even when the computer’s power is turned off. Secondary storage information takes much longer to access than information in primary memory, but its cost per unit is much less. Examples of secondary storage devices include hard drives, DVD drives and USB flash drives, some of which can hold over 2 TB (TB stands for tera-bytes; a terabyte is approximately one trillion bytes). Typical hard drives on desktop and notebook computers hold up to 2 TB, and some desktop hard drives can hold up to 4 TB.

Fig. 1.2 | Logical units of a computer. (Part 2 of 2.)

1.3 Data Hierarchy

Data items processed by computers form a **data hierarchy** that becomes larger and more complex in structure as we progress from the simplest data items (called “bits”) to richer ones, such as characters and fields. Figure 1.3 illustrates a portion of the data hierarchy.

Bits

The smallest data item in a computer can assume the value 0 or the value 1. It’s called a **bit** (short for “binary digit”—a digit that can assume one of *two* values). Remarkably, the impressive functions performed by computers involve only the simplest manipulations of 0s and 1s—*examining a bit’s value*, *setting a bit’s value* and *reversing a bit’s value* (from 1 to 0 or from 0 to 1).

Characters

It’s tedious for people to work with data in the low-level form of bits. Instead, they prefer to work with *decimal digits* (0–9), *letters* (A–Z and a–z), and *special symbols* (e.g., \$, @, %,

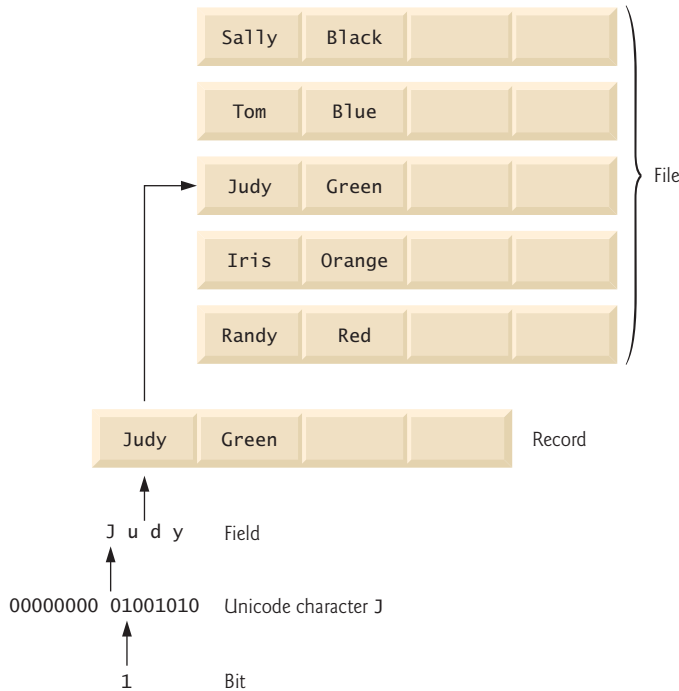


Fig. 1.3 | Data hierarchy.

&, *, (,), -, +, ", :, ? and /). Digits, letters and special symbols are known as **characters**. The computer's **character set** is the set of all the characters used to write programs and represent data items. Computers process only 1s and 0s, so a computer's character set represents every character as a pattern of 1s and 0s. Java uses **Unicode**[®] characters that are composed of one, two or four bytes (8, 16 or 32 bits). Unicode contains characters for many of the world's languages. See Appendix H for more information on Unicode. See Appendix B for more information on the **ASCII (American Standard Code for Information Interchange)** character set—the popular subset of Unicode that represents uppercase and lowercase letters, digits and some common special characters.

Fields

Just as characters are composed of bits, **fields** are composed of characters or bytes. A field is a group of characters or bytes that conveys meaning. For example, a field consisting of uppercase and lowercase letters can be used to represent a person's name, and a field consisting of decimal digits could represent a person's age.

Records

Several related fields can be used to compose a **record** (implemented as a `class` in Java). In a payroll system, for example, the record for an employee might consist of the following fields (possible types for these fields are shown in parentheses):

- Employee identification number (a whole number)
- Name (a string of characters)
- Address (a string of characters)
- Hourly pay rate (a number with a decimal point)
- Year-to-date earnings (a number with a decimal point)
- Amount of taxes withheld (a number with a decimal point)

Thus, a record is a group of related fields. In the preceding example, all the fields belong to the *same* employee. A company might have many employees and a payroll record for each.

Files

A **file** is a group of related records. [Note: More generally, a file contains arbitrary data in arbitrary formats. In some operating systems, a file is viewed simply as a *sequence of bytes*—any organization of the bytes in a file, such as organizing the data into records, is a view created by the application programmer. You'll see how to do that in Chapter 15.] It's not unusual for an organization to have many files, some containing billions, or even trillions, of characters of information.

Database

A **database** is a collection of data organized for easy access and manipulation. The most popular model is the *relational database*, in which data is stored in simple *tables*. A table includes *records* and *fields*. For example, a table of students might include first name, last name, major, year, student ID number and grade point average fields. The data for each student is a record, and the individual pieces of information in each record are the fields. You can *search*, *sort* and otherwise manipulate the data based on its relationship to multiple tables or databases. For example, a university might use data from the student database in combination with data from databases of courses, on-campus housing, meal plans, etc. We discuss databases in Chapter 24.

Big Data

The amount of data being produced worldwide is enormous and growing quickly. According to IBM, approximately 2.5 quintillion bytes (2.5 *exabytes*) of data are created daily and 90% of the world's data was created in just the past two years!⁶ According to a Digital Universe study, the global data supply reached 2.8 *zettabytes* (equal to 2.8 trillion gigabytes) in 2012.⁷ Figure 1.4 shows some common byte measurements. **Big data** applications deal with such massive amounts of data and this field is growing quickly, creating lots of opportunity for software developers. According to a study by Gartner Group, over 4 million IT jobs globally will support big data by 2015.⁸

6. <http://www-01.ibm.com/software/data/bigdata/>.

7. <http://www.guardian.co.uk/news/datablog/2012/dec/19/big-data-study-digital-universe-global-volume>.

8. <http://tech.fortune.cnn.com/2013/09/04/big-data-employment-boom/>.

Unit	Bytes	Which is approximately
1 kilobyte (KB)	1024 bytes	10^3 (1024 bytes exactly)
1 megabyte (MB)	1024 kilobytes	10^6 (1,000,000 bytes)
1 gigabyte (GB)	1024 megabytes	10^9 (1,000,000,000 bytes)
1 terabyte (TB)	1024 gigabytes	10^{12} (1,000,000,000,000 bytes)
1 petabyte (PB)	1024 terabytes	10^{15} (1,000,000,000,000,000 bytes)
1 exabyte (EB)	1024 petabytes	10^{18} (1,000,000,000,000,000,000 bytes)
1 zettabyte (ZB)	1024 exabytes	10^{21} (1,000,000,000,000,000,000,000 bytes)

Fig. 1.4 | Byte measurements.

1.4 Machine Languages, Assembly Languages and High-Level Languages

Programmers write instructions in various programming languages, some directly understandable by computers and others requiring intermediate *translation* steps. Hundreds of such languages are in use today. These may be divided into three general types:

1. Machine languages
2. Assembly languages
3. High-level languages

Machine Languages

Any computer can directly understand only its own **machine language**, defined by its hardware design. Machine languages generally consist of strings of numbers (ultimately reduced to 1s and 0s) that instruct computers to perform their most elementary operations one at a time. Machine languages are *machine dependent* (a particular machine language can be used on only one type of computer). Such languages are cumbersome for humans. For example, here's a section of an early machine-language payroll program that adds overtime pay to base pay and stores the result in gross pay:

```
+1300042774
+1400593419
+1200274027
```

Assembly Languages and Assemblers

Programming in machine language was simply too slow and tedious for most programmers. Instead of using the strings of numbers that computers could directly understand, programmers began using English-like abbreviations to represent elementary operations. These abbreviations formed the basis of **assembly languages**. *Translator programs* called **assemblers** were developed to convert early assembly-language programs to machine language at computer speeds. The following section of an assembly-language payroll program also adds overtime pay to base pay and stores the result in gross pay:

```
load    basepay
add     overpay
store   grosspay
```

Although such code is clearer to humans, it's incomprehensible to computers until translated to machine language.

High-Level Languages and Compilers

With the advent of assembly languages, computer usage increased rapidly, but programmers still had to use numerous instructions to accomplish even the simplest tasks. To speed the programming process, **high-level languages** were developed in which single statements could be written to accomplish substantial tasks. Translator programs called **compilers** convert high-level language programs into machine language. High-level languages allow you to write instructions that look almost like everyday English and contain commonly used mathematical notations. A payroll program written in a high-level language might contain a *single* statement such as

```
grossPay = basePay + overTimePay
```

From the programmer's standpoint, high-level languages are preferable to machine and assembly languages. Java is one of the most widely used high-level programming languages.

Interpreters

Compiling a large high-level language program into machine language can take considerable computer time. *Interpreter* programs, developed to execute high-level language programs directly, avoid the delay of compilation, although they run slower than compiled programs. We'll say more about interpreters in Section 1.9, where you'll learn that Java uses a clever performance-tuned mixture of compilation and interpretation to run programs.

1.5 Introduction to Object Technology

Today, as demands for new and more powerful software are soaring, building software quickly, correctly and economically remains an elusive goal. *Objects*, or more precisely, the *classes* objects come from, are essentially *reusable* software components. There are date objects, time objects, audio objects, video objects, automobile objects, people objects, etc. Almost any *noun* can be reasonably represented as a software object in terms of *attributes* (e.g., name, color and size) and *behaviors* (e.g., calculating, moving and communicating). Software-development groups can use a modular, object-oriented design-and-implementation approach to be much more productive than with earlier popular techniques like "structured programming"—object-oriented programs are often easier to understand, correct and modify.

1.5.1 The Automobile as an Object

To help you understand objects and their contents, let's begin with a simple analogy. Suppose you want to *drive a car and make it go faster by pressing its accelerator pedal*. What must happen before you can do this? Well, before you can drive a car, someone has to *design* it. A car typically begins as engineering drawings, similar to the *blueprints* that describe the design of a house. These drawings include the design for an accelerator pedal. The pedal *hides* from the driver the complex mechanisms that actually make the car go faster, just as the brake pedal "hides" the mechanisms that slow the car, and the steering wheel "hides" the mechanisms that turn the car. This enables people with little or no knowledge of how engines, braking and steering mechanisms work to drive a car easily.

Just as you cannot cook meals in the kitchen of a blueprint, you cannot drive a car's engineering drawings. Before you can drive a car, it must be *built* from the engineering drawings that describe it. A completed car has an *actual* accelerator pedal to make it go faster, but even that's not enough—the car won't accelerate on its own (hopefully!), so the driver must *press* the pedal to accelerate the car.

1.5.2 Methods and Classes

Let's use our car example to introduce some key object-oriented programming concepts. Performing a task in a program requires a **method**. The method houses the program statements that actually perform its tasks. The method hides these statements from its user, just as the accelerator pedal of a car hides from the driver the mechanisms of making the car go faster. In Java, we create a program unit called a **class** to house the set of methods that perform the class's tasks. For example, a class that represents a bank account might contain one method to *deposit* money to an account, another to *withdraw* money from an account and a third to *inquire* what the account's current balance is. A class is similar in concept to a car's engineering drawings, which house the design of an accelerator pedal, steering wheel, and so on.

1.5.3 Instantiation

Just as someone has to *build a car* from its engineering drawings before you can actually drive a car, you must *build an object* of a class before a program can perform the tasks that the class's methods define. The process of doing this is called *instantiation*. An object is then referred to as an **instance** of its class.

1.5.4 Reuse

Just as a car's engineering drawings can be *reused* many times to build many cars, you can *reuse* a class many times to build many objects. Reuse of existing classes when building new classes and programs saves time and effort. Reuse also helps you build more reliable and effective systems, because existing classes and components often have undergone extensive *testing*, *debugging* and *performance* tuning. Just as the notion of *interchangeable parts* was crucial to the Industrial Revolution, reusable classes are crucial to the software revolution that has been spurred by object technology.



Software Engineering Observation 1.1

Use a building-block approach to creating your programs. Avoid reinventing the wheel—use existing high-quality pieces wherever possible. This software reuse is a key benefit of object-oriented programming.

1.5.5 Messages and Method Calls

When you drive a car, pressing its gas pedal sends a *message* to the car to perform a task—that is, to go faster. Similarly, you *send messages to an object*. Each message is implemented as a **method call** that tells a method of the object to perform its task. For example, a program might call a bank-account object's *deposit* method to increase the account's balance.

1.5.6 Attributes and Instance Variables

A car, besides having capabilities to accomplish tasks, also has *attributes*, such as its color, its number of doors, the amount of gas in its tank, its current speed and its record of total

miles driven (i.e., its odometer reading). Like its capabilities, the car's attributes are represented as part of its design in its engineering diagrams (which, for example, include an odometer and a fuel gauge). As you drive an actual car, these attributes are carried along with the car. Every car maintains its *own* attributes. For example, each car knows how much gas is in its own gas tank, but *not* how much is in the tanks of *other* cars.

An object, similarly, has attributes that it carries along as it's used in a program. These attributes are specified as part of the object's class. For example, a bank-account object has a *balance attribute* that represents the amount of money in the account. Each bank-account object knows the balance in the account it represents, but *not* the balances of the *other* accounts in the bank. Attributes are specified by the class's **instance variables**.

1.5.7 Encapsulation and Information Hiding

Classes (and their objects) **encapsulate**, i.e., encase, their attributes and methods. A class's (and its object's) attributes and methods are intimately related. Objects may communicate with one another, but they're normally not allowed to know how other objects are implemented—implementation details are *hidden* within the objects themselves. This **information hiding**, as we'll see, is crucial to good software engineering.

1.5.8 Inheritance

A new class of objects can be created conveniently by **inheritance**—the new class (called the **subclass**) starts with the characteristics of an existing class (called the **superclass**), possibly customizing them and adding unique characteristics of its own. In our car analogy, an object of class “convertible” certainly *is an* object of the more *general* class “automobile,” but more *specifically*, the roof can be raised or lowered.

1.5.9 Interfaces

Java also supports **interfaces**—collections of related methods that typically enable you to tell objects *what* to do, but not *how* to do it (we'll see an exception to this in Java SE 8). In the car analogy, a “basic-driving-capabilities” interface consisting of a steering wheel, an accelerator pedal and a brake pedal would enable a driver to tell the car *what* to do. Once you know how to use this interface for turning, accelerating and braking, you can drive many types of cars, even though manufacturers may *implement* these systems *differently*.

A class **implements** zero or more interfaces, each of which can have one or more methods, just as a car implements separate interfaces for basic driving functions, controlling the radio, controlling the heating and air conditioning systems, and the like. Just as car manufacturers implement capabilities *differently*, classes may implement an interface's methods *differently*. For example a software system may include a “backup” interface that offers the methods *save* and *restore*. Classes may implement those methods differently, depending on the types of things being backed up, such as programs, text, audios, videos, etc., and the types of devices where these items will be stored.

1.5.10 Object-Oriented Analysis and Design (OOAD)

Soon you'll be writing programs in Java. How will you create the **code** (i.e., the program instructions) for your programs? Perhaps, like many programmers, you'll simply turn on

your computer and start typing. This approach may work for small programs (like the ones we present in the early chapters of the book), but what if you were asked to create a software system to control thousands of automated teller machines for a major bank? Or suppose you were asked to work on a team of 1,000 software developers building the next generation of the U.S. air traffic control system? For projects so large and complex, you should not simply sit down and start writing programs.

To create the best solutions, you should follow a detailed **analysis** process for determining your project's **requirements** (i.e., defining *what* the system is supposed to do) and developing a **design** that satisfies them (i.e., specifying *how* the system should do it). Ideally, you'd go through this process and carefully review the design (and have your design reviewed by other software professionals) before writing any code. If this process involves analyzing and designing your system from an object-oriented point of view, it's called an **object-oriented analysis-and-design (OOAD) process**. Languages like Java are object oriented. Programming in such a language, called **object-oriented programming (OOP)**, allows you to implement an object-oriented design as a working system.

1.5.11 The UML (Unified Modeling Language)

Although many different OOAD processes exist, a single graphical language for communicating the results of *any* OOAD process has come into wide use. The Unified Modeling Language (UML) is now the most widely used graphical scheme for modeling object-oriented systems. We present our first UML diagrams in Chapters 3 and 4, then use them in our deeper treatment of object-oriented programming through Chapter 11. In our *optional* online ATM Software Engineering Case Study in Chapters 33–34 we present a simple subset of the UML's features as we guide you through an object-oriented design experience.

1.6 Operating Systems

Operating systems are software systems that make using computers more convenient for users, application developers and system administrators. They provide services that allow each application to execute safely, efficiently and *concurrently* (i.e., in parallel) with other applications. The software that contains the core components of the operating system is the **kernel**. Popular desktop operating systems include Linux, Windows and Mac OS X. Popular mobile operating systems used in smartphones and tablets include Google's Android, Apple's iOS (for its iPhone, iPad and iPod Touch devices), Windows Phone 8 and BlackBerry OS.

1.6.1 Windows—A Proprietary Operating System

In the mid-1980s, Microsoft developed the **Windows operating system**, consisting of a graphical user interface built on top of DOS—an enormously popular personal-computer operating system that users interacted with by typing commands. Windows borrowed many concepts (such as icons, menus and windows) popularized by early Apple Macintosh operating systems and originally developed by Xerox PARC. Windows 8 is Microsoft's latest operating system—its features include PC and tablet support, a tiles-based user interface, security enhancements, touch-screen and multi-touch support, and more. Windows is a *proprietary* operating system—it's controlled by Microsoft exclusively. It's by far the world's most widely used operating system.

1.6.2 Linux—An Open-Source Operating System

The **Linux** operating system—which is popular in servers, personal computers and embedded systems—is perhaps the greatest success of the *open-source* movement. The **open-source software** development style departs from the *proprietary* development style (used, for example, with Microsoft’s Windows and Apple’s Mac OS X). With open-source development, individuals and companies—often worldwide—contribute their efforts in developing, maintaining and evolving software. Anyone can use and customize it for their own purposes, typically at no charge. The Java Development Kit and many related Java technologies are now open source.

Some organizations in the open-source community are the *Eclipse Foundation* (the *Eclipse Integrated Development Environment* helps Java programmers conveniently develop software), the *Mozilla Foundation* (creators of the *Firefox web browser*), the *Apache Software Foundation* (creators of the *Apache web server* that *delivers web pages over the Internet* in response to web-browser requests) and *GitHub* and *SourceForge* (which provide the *tools for managing open-source projects*).

Rapid improvements to computing and communications, decreasing costs and open-source software have made it easier and more economical to create software-based businesses now than just a few decades ago. Facebook, which was launched from a college dorm room, was built with open-source software.⁹

A variety of issues—such as Microsoft’s market power, the relatively small number of user-friendly Linux applications and the diversity of Linux distributions (Red Hat Linux, Ubuntu Linux and many others)—have prevented widespread Linux use on desktop computers. But Linux has become extremely popular on servers and in embedded systems, such as Google’s Android-based smartphones.

1.6.3 Android

Android—the fastest-growing mobile and smartphone operating system—is based on the Linux kernel and uses Java. Experienced Java programmers can quickly dive into Android development. One benefit of developing Android apps is the openness of the platform. The operating system is open source and free.

The Android operating system was developed by Android, Inc., which was acquired by Google in 2005. In 2007, the Open Handset Alliance™—which now has 87 company members worldwide (http://www.openhandsetalliance.com/oha_members.html)—was formed to develop, maintain and evolve Android, driving innovation in mobile technology and improving the user experience while reducing costs. As of April 2013, more than 1.5 million Android devices (smartphones, tablets, etc.) were being activated *daily*.¹⁰ By October 2013, a Strategy Analytics report showed that Android had 81.3% of the global *smartphone* market share, compared to 13.4% for Apple, 4.1% for Microsoft and 1% for BlackBerry.¹¹ Android devices now include smartphones, tablets, e-readers, robots, jet engines, NASA satellites, game consoles, refrigerators, televisions, cameras, health-care

9. <http://developers.facebook.com/opensource>.

10. <http://www.technobuffalo.com/2013/04/16/google-daily-android-activations-1-5-million/>.

11. <http://blogs.strategyanalytics.com/WSS/post/2013/10/31/Android-Captures-Record-81-Percent-Share-of-Global-Smartphone-Shipments-in-Q3-2013.aspx>.

devices, smartwatches, automobile in-vehicle infotainment systems (for controlling the radio, GPS, phone calls, thermostat, etc.) and more.¹²

Android smartphones include the functionality of a mobile phone, Internet client (for web browsing and Internet communication), MP3 player, gaming console, digital camera and more. These handheld devices feature full-color *multitouch screens* which allow you to control the device with *gestures* involving one touch or multiple simultaneous touches. You can download apps directly onto your Android device through Google Play and other app marketplaces. At the time of this writing, there were over one million apps in **Google Play**, and the number is growing quickly.¹³

We present an introduction to Android app development in our textbook, *Android How to Program, Second Edition*, and in our professional book, *Android for Programmers: An App-Driven Approach, Second Edition*. After you learn Java, you'll find it straightforward to begin developing and running Android apps. You can place your apps on Google Play (play.google.com), and if they're successful, you may even be able to launch a business. Just remember—Facebook, Microsoft and Dell were all launched from college dorm rooms.

1.7 Programming Languages

In this section, we comment briefly on several popular programming languages (Fig. 1.5). In the next section, we introduce Java.

Programming language	Description
Fortran	Fortran (FORMula TRANslator) was developed by IBM Corporation in the mid-1950s for scientific and engineering applications that require complex mathematical computations. It's still widely used, and its latest versions support object-oriented programming.
COBOL	COBOL (Common Business Oriented Language) was developed in the late 1950s by computer manufacturers, the U.S. government and industrial computer users based on a language developed by Grace Hopper, a U.S. Navy Rear Admiral and computer scientist who also advocated for the international standardization of programming languages. COBOL is still widely used for commercial applications that require precise and efficient manipulation of large amounts of data. Its latest version supports object-oriented programming.
Pascal	Research in the 1960s resulted in <i>structured programming</i> —a disciplined approach to writing programs that are clearer, easier to test and debug and easier to modify than large programs produced with previous techniques. One result of this research was the development in 1971 of the Pascal programming language, which was designed for teaching structured programming and was popular in college courses for several decades.

Fig. 1.5 | Some other programming languages. (Part 1 of 3.)

12. <http://www.businessweek.com/articles/2013-05-29/behind-the-internet-of-things-is-android-and-its-everywhere>.

13. http://en.wikipedia.org/wiki/Google_Play.

Programming language	Description
Ada	Ada, based on Pascal, was developed under the sponsorship of the U.S. Department of Defense (DOD) during the 1970s and early 1980s. The DOD wanted a single language that would fill most of its needs. The Ada language was named after Lady Ada Lovelace, daughter of the poet Lord Byron. She's credited with writing the world's first computer program in the early 1800s (for the Analytical Engine mechanical computing device designed by Charles Babbage). Ada also supports object-oriented programming.
Basic	Basic was developed in the 1960s at Dartmouth College to familiarize novices with programming techniques. Many of its latest versions are object oriented.
C	C was developed in the early 1970s by Dennis Ritchie at Bell Laboratories. It initially became widely known as the UNIX operating system's development language. Today, most of the code for general-purpose operating systems is written in C or C++.
C++	C++, which is based on C, was developed by Bjarne Stroustrup in the early 1980s at Bell Laboratories. C++ provides several features that "spruce up" the C language, but more important, it provides capabilities for object-oriented programming.
Objective-C	Objective-C is another object-oriented language based on C. It was developed in the early 1980s and later acquired by NeXT, which in turn was acquired by Apple. It has become the key programming language for the OS X operating system and all iOS-powered devices (such as iPods, iPhones and iPads).
Visual Basic	Microsoft's Visual Basic language was introduced in the early 1990s to simplify the development of Microsoft Windows applications. Its latest versions support object-oriented programming.
Visual C#	Microsoft's three object-oriented primary programming languages are Visual Basic (based on the original Basic), Visual C++ (based on C++) and Visual C# (based on C++ and Java, and developed for integrating the Internet and the web into computer applications).
PHP	PHP, an object-oriented, open-source scripting language supported by a community of users and developers, is used by millions of websites. PHP is platform independent—implementations exist for all major UNIX, Linux, Mac and Windows operating systems. PHP also supports many databases, including the popular open-source MySQL.
Perl	Perl (Practical Extraction and Report Language), one of the most widely used object-oriented scripting languages for web programming, was developed in 1987 by Larry Wall. It features rich text-processing capabilities.
Python	Python, another object-oriented scripting language, was released publicly in 1991. Developed by Guido van Rossum of the National Research Institute for Mathematics and Computer Science in Amsterdam (CWI), Python draws heavily from Modula-3—a systems programming language. Python is "extensible"—it can be extended through classes and programming interfaces.
JavaScript	JavaScript is the most widely used scripting language. It's primarily used to add dynamic behavior to web pages—for example, animations and improved interactivity with the user. It's provided with all major web browsers.

Fig. 1.5 | Some other programming languages. (Part 2 of 3.)

Programming language	Description
Ruby on Rails	Ruby, created in the mid-1990s, is an open-source, object-oriented programming language with a simple syntax that's similar to Python. Ruby on Rails combines the scripting language Ruby with the Rails web application framework developed by 37Signals. Their book, <i>Getting Real</i> (gettingreal.37signals.com/toc.php), is a must read for web developers. Many Ruby on Rails developers have reported productivity gains over other languages when developing database-intensive web applications.

Fig. 1.5 | Some other programming languages. (Part 3 of 3.)

1.8 Java

The microprocessor revolution's most important contribution to date is that it enabled the development of personal computers. Microprocessors have had a profound impact in intelligent consumer-electronic devices. Recognizing this, Sun Microsystems in 1991 funded an internal corporate research project led by James Gosling, which resulted in a C++-based object-oriented programming language that Sun called Java.

A key goal of Java is to be able to write programs that will run on a great variety of computer systems and computer-controlled devices. This is sometimes called “write once, run anywhere.”

The web exploded in popularity in 1993, and Sun saw the potential of using Java to add *dynamic content*, such as interactivity and animations, to web pages. Java drew the attention of the business community because of the phenomenal interest in the web. Java is now used to develop large-scale enterprise applications, to enhance the functionality of web servers (the computers that provide the content we see in our web browsers), to provide applications for consumer devices (cell phones, smartphones, television set-top boxes and more) and for many other purposes. Java is also the key language for developing Android smartphone and tablet apps. Sun Microsystems was acquired by Oracle in 2010.

Java Class Libraries

You can create each class and method you need to form your Java programs. However, most Java programmers take advantage of the rich collections of existing classes and methods in the **Java class libraries**, also known as the **Java APIs (Application Programming Interfaces)**.



Performance Tip 1.1

Using Java API classes and methods instead of writing your own versions can improve program performance, because they're carefully written to perform efficiently. This also shortens program development time.

1.9 A Typical Java Development Environment

We now explain the steps to create and execute a Java application. Normally there are five phases—edit, compile, load, verify and execute. We discuss them in the context of the Java

SE 8 Development Kit (JDK). See the *Before You Begin* section for information on downloading and installing the JDK on Windows, Linux and OS X.

Phase 1: Creating a Program

Phase 1 consists of editing a file with an *editor program*, normally known simply as an *editor* (Fig. 1.6). Using the editor, you type a Java program (typically referred to as **source code**), make any necessary corrections and save it on a secondary storage device, such as your hard drive. Java source code files are given a name ending with the **.java extension**, indicating that the file contains Java source code.

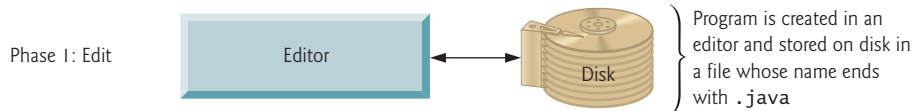


Fig. 1.6 | Typical Java development environment—editing phase.

Two editors widely used on Linux systems are **vi** and **emacs**. Windows provides **Notepad**. OS X provides **TextEdit**. Many freeware and shareware editors are also available online, including Notepad++ (notepad-plus-plus.org), EditPlus (www.editplus.com), TextPad (www.textpad.com) and jEdit (www.jedit.org).

Integrated development environments (IDEs) provide tools that support the software development process, such as editors, debuggers for locating **logic errors** (errors that cause programs to execute incorrectly) and more. There are many popular Java IDEs, including:

- Eclipse (www.eclipse.org)
- NetBeans (www.netbeans.org)
- IntelliJ IDEA (www.jetbrains.com)

On the book's website at

www.deitel.com/books/jhttp10

we provide Dive-Into® videos that show you how to execute this book's Java applications and how to develop new Java applications with Eclipse, NetBeans and IntelliJ IDEA.

Phase 2: Compiling a Java Program into Bytecodes

In Phase 2, you use the command **javac** (the **Java compiler**) to **compile** a program (Fig. 1.7). For example, to compile a program called `Welcome.java`, you'd type

```
javac Welcome.java
```

in your system's command window (i.e., the **Command Prompt** in Windows, the **Terminal** application in OS X) or a Linux shell (also called **Terminal** in some versions of Linux). If the program compiles, the compiler produces a **.class** file called `Welcome.class` that contains the compiled version. IDEs typically provide a menu item, such as **Build** or **Make**, that invokes the `javac` command for you. If the compiler detects errors, you'll need to go back to Phase 1 and correct them. In Chapter 2, we'll say more about the kinds of errors the compiler can detect.

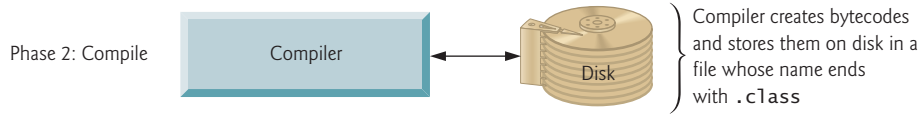


Fig. 1.7 | Typical Java development environment—compilation phase.

The Java compiler translates Java source code into **bytecodes** that represent the tasks to execute in the execution phase (Phase 5). The **Java Virtual Machine (JVM)**—a part of the JDK and the foundation of the Java platform—executes bytecodes. A **virtual machine (VM)** is a software application that simulates a computer but hides the underlying operating system and hardware from the programs that interact with it. If the same VM is implemented on many computer platforms, applications written for that type of VM can be used on all those platforms. The JVM is one of the most widely used virtual machines. Microsoft's .NET uses a similar virtual-machine architecture.

Unlike machine-language instructions, which are *platform dependent* (that is, dependent on specific computer hardware), bytecode instructions are *platform independent*. So, Java's bytecodes are **portable**—without recompiling the source code, the same bytecode instructions can execute on any platform containing a JVM that understands the version of Java in which the bytecodes were compiled. The JVM is invoked by the **java** command. For example, to execute a Java application called `Welcome`, you'd type the command

```
java Welcome
```

in a command window to invoke the JVM, which would then initiate the steps necessary to execute the application. This begins Phase 3. IDEs typically provide a menu item, such as **Run**, that invokes the `java` command for you.

Phase 3: Loading a Program into Memory

In Phase 3, the JVM places the program in memory to execute it—this is known as **loading** (Fig. 1.8). The JVM's **class loader** takes the `.class` files containing the program's bytecodes and transfers them to primary memory. It also loads any of the `.class` files provided by Java that your program uses. The `.class` files can be loaded from a disk on your system or over a network (e.g., your local college or company network, or the Internet).

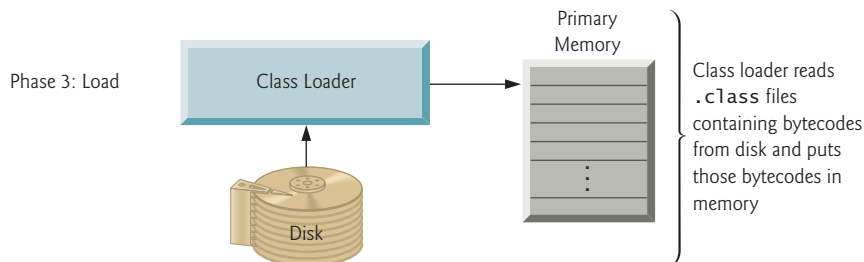


Fig. 1.8 | Typical Java development environment—loading phase.

Phase 4: Bytecode Verification

In Phase 4, as the classes are loaded, the **bytecode verifier** examines their bytecodes to ensure that they're valid and do not violate Java's security restrictions (Fig. 1.9). Java enforces strong security to make sure that Java programs arriving over the network do not damage your files or your system (as computer viruses and worms might).

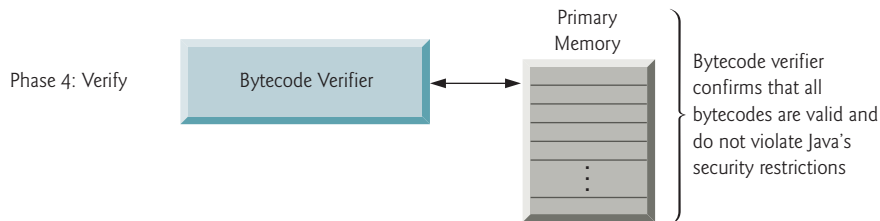


Fig. 1.9 | Typical Java development environment—verification phase.

Phase 5: Execution

In Phase 5, the JVM **executes** the program's bytecodes, thus performing the actions specified by the program (Fig. 1.10). In early Java versions, the JVM was simply an *interpreter* for Java bytecodes. Most Java programs would execute slowly, because the JVM would interpret and execute one bytecode at a time. Some modern computer architectures can execute several instructions in parallel. Today's JVMs typically execute bytecodes using a combination of interpretation and so-called **just-in-time (JIT) compilation**. In this process, the JVM analyzes the bytecodes as they're interpreted, searching for *hot spots*—parts of the bytecodes that execute frequently. For these parts, a **just-in-time (JIT) compiler**, such as Oracle's **Java HotSpot™ compiler**, translates the bytecodes into the underlying computer's machine language. When the JVM encounters these compiled parts again, the faster machine-language code executes. Thus Java programs actually go through *two* compilation phases—one in which source code is translated into bytecodes (for portability across JVMs on different computer platforms) and a second in which, during execution, the *bytecodes* are translated into *machine language* for the actual computer on which the program executes.

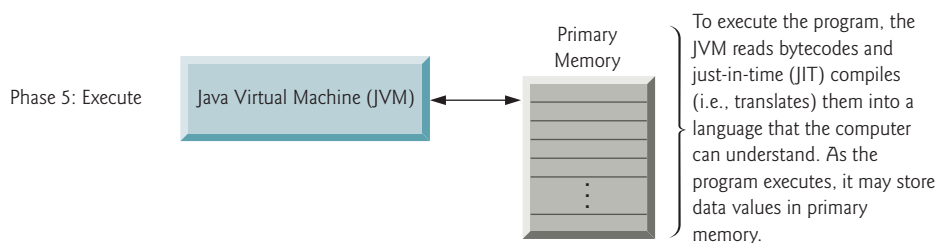


Fig. 1.10 | Typical Java development environment—execution phase.

Problems That May Occur at Execution Time

Programs might not work on the first try. Each of the preceding phases can fail because of various errors that we'll discuss throughout this book. For example, an executing program

might try to divide by zero (an illegal operation for whole-number arithmetic in Java). This would cause the Java program to display an error message. If this occurred, you'd have to return to the edit phase, make the necessary corrections and proceed through the remaining phases again to determine that the corrections fixed the problem(s). [Note: Most programs in Java input or output data. When we say that a program displays a message, we normally mean that it displays that message on your computer's screen. Messages and other data may be output to other devices, such as disks and hardcopy printers, or even to a network for transmission to other computers.]



Common Programming Error 1.1

*Errors such as division by zero occur as a program runs, so they're called **runtime errors** or **execution-time errors**. **Fatal runtime errors** cause programs to terminate immediately without having successfully performed their jobs. **Nonfatal runtime errors** allow programs to run to completion, often producing incorrect results.*

1.10 Test-Driving a Java Application

In this section, you'll run and interact with your first Java application. The **Painter** application, which you'll build over the course of several exercises, allows you to drag the mouse to "paint." The elements and functionality you see here are typical of what you'll learn to program in this book. Using the **Painter's** graphical user interface (GUI), you can control the drawing color, the shape to draw (line, rectangle or oval) and whether the shape is filled with the drawing color. You can also undo the last shape you added to the drawing or clear the entire drawing. [Note: We use fonts to distinguish between features. Our convention is to emphasize screen features like titles and menus (e.g., the **File** menu) in a semibold sans-serif Helvetica font and to emphasize nonscreen elements, such as file names, program code or input (e.g., `ProgramName.java`), in a sans-serif Lucida font.]

The steps in this section show you how to execute the **Painter** app from a **Command Prompt** (Windows), **Terminal** (OS X) or shell (Linux) window on your system. Throughout the book, we'll refer to these windows simply as *command windows*. Perform the following steps to use the **Painter** application to draw a smiley face:

1. *Checking your setup.* Read the Before You Begin section to confirm that you've set up Java properly on your computer, that you've copied the book's examples to your hard drive and that you know how to open a command window on your system.
2. *Changing to the completed application's directory.* Open a command window and use the `cd` command to change to the directory (also called a *folder*) for the **Painter** application. We assume that the book's examples are located in `C:\examples` on Windows or in your user account's `Documents/examples` folder on Linux/OS X. On Windows type `cd C:\examples\ch01\painter`, then press *Enter*. On Linux/OS X, type `cd ~/Documents/examples/ch01/painter`, then press *Enter*.
3. *Running the Painter application.* Recall that the `java` command, followed by the name of the application's `.class` file (in this case, `Painter`), executes the application. Type the command `java Painter` and press *Enter* to execute the app. Figure 1.11 shows the app running on Windows, Linux and OS X, respectively—we shortened the windows to save space.

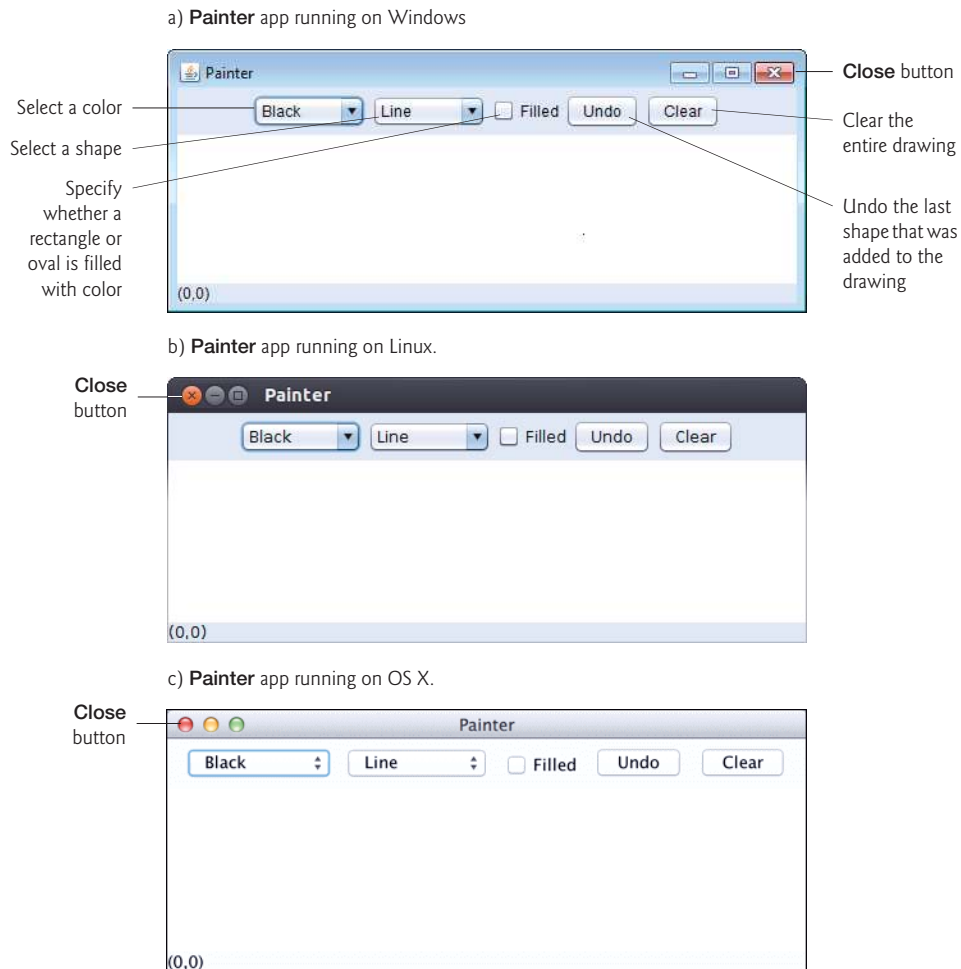


Fig. 1.11 | **Painter** app executing in Windows 7, Linux and OS X.

[*Note:* Java commands are *case sensitive*—that is, uppercase letters are different from lowercase letters. It's important to type the name of this application as **Painter** with a capital P. Otherwise, the application will *not* execute. Specifying the `.class` extension when using the `java` command results in an error. Also, if you receive the error message, "Exception in thread "main" java.lang.NoClassDefFoundError: Painter," your system has a CLASSPATH problem. Please refer to the Before You Begin section for instructions to help you fix this problem.]

4. **Drawing a filled yellow oval for the face.** Select **Yellow** as the drawing color, **Oval** as the shape and check the **Filled** checkbox, then drag the mouse to draw a large oval (Fig. 1.12).

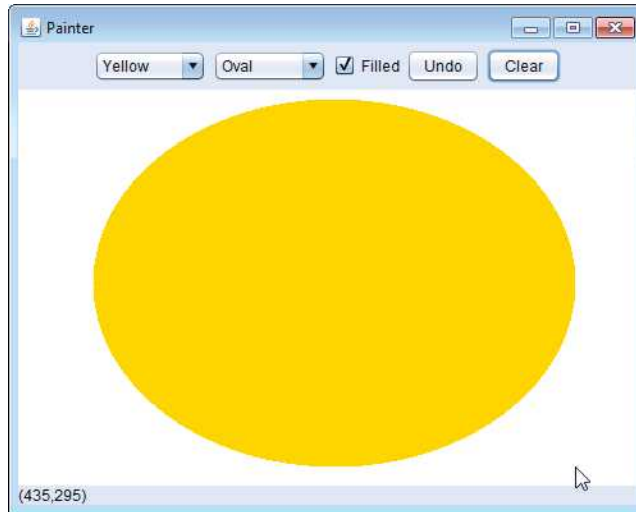


Fig. 1.12 | Drawing a filled yellow oval for the face.

5. *Drawing blue eyes.* Select **Blue** as the drawing color, then draw two small ovals as the eyes (Fig. 1.13).

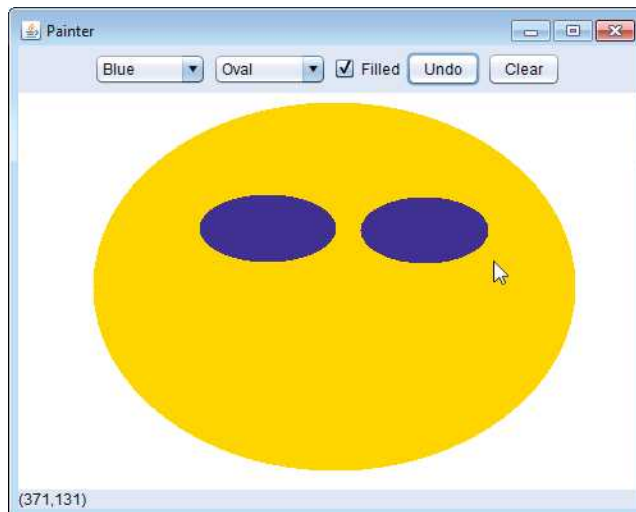


Fig. 1.13 | Drawing blue eyes.

6. *Drawing black eyebrows and a nose.* Select **Black** as the drawing color and **Line** as the shape, then draw eyebrows and a nose (Fig. 1.14). Lines do not have fill, so leaving the **Filled** checkbox checked has no effect when drawing lines.

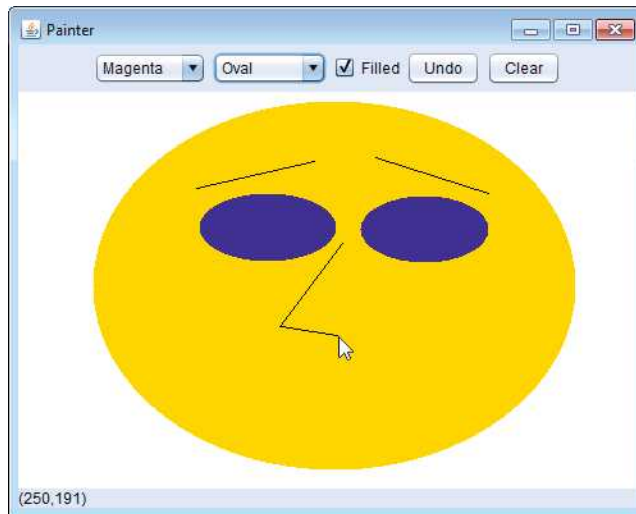


Fig. 1.14 | Drawing black eyebrows and a nose.

7. *Drawing a magenta mouth.* Select **Magenta** as the drawing color and **Oval** as the shape, then draw a mouth (Fig. 1.15).

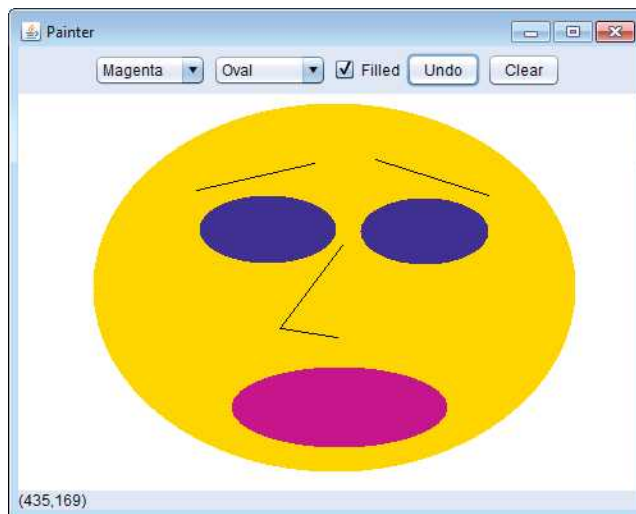


Fig. 1.15 | Drawing a magenta mouth.

8. *Drawing a yellow oval on the mouth to make a smile.* Select **Yellow** as the drawing color, then draw an oval to change the magenta oval into a smile (Fig. 1.16).

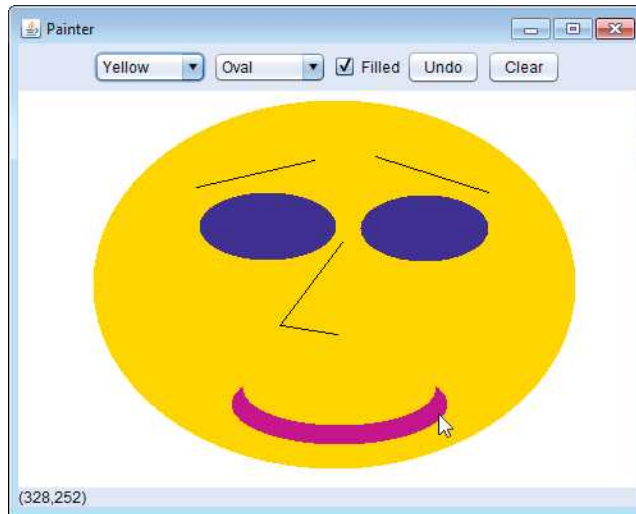


Fig. 1.16 | Drawing a yellow oval on the mouth to make a smile.

9. *Exiting the Painter application.* To exit the **Painter** application, click the **Close** button (in the window's upper-right corner on Windows and the upper-left corner on Linux and OS X). Closing the window causes the **Painter** application to terminate.

1.11 Internet and World Wide Web

In the late 1960s, ARPA—the Advanced Research Projects Agency of the United States Department of Defense—rolled out plans for networking the main computer systems of approximately a dozen ARPA-funded universities and research institutions. The computers were to be connected with communications lines operating at speeds on the order of 50,000 bits per second, a stunning rate at a time when most people (of the few who even had networking access) were connecting over telephone lines to computers at a rate of 110 bits per second. Academic research was about to take a giant leap forward. ARPA proceeded to implement what quickly became known as the ARPANET, the precursor to today's **Internet**. Today's fastest Internet speeds are on the order of billions of bits per second with trillion-bit-per-second speeds on the horizon!

Things worked out differently from the original plan. Although the ARPANET enabled researchers to network their computers, its main benefit proved to be the capability for quick and easy communication via what came to be known as electronic mail (e-mail). This is true even on today's Internet, with e-mail, instant messaging, file transfer and social media such as Facebook and Twitter enabling billions of people worldwide to communicate quickly and easily.

The protocol (set of rules) for communicating over the ARPANET became known as the **Transmission Control Protocol (TCP)**. TCP ensured that messages, consisting of sequentially numbered pieces called *packets*, were properly routed from sender to receiver, arrived intact and were assembled in the correct order.

1.11.1 The Internet: A Network of Networks

In parallel with the early evolution of the Internet, organizations worldwide were implementing their own networks for both intraorganization (that is, within an organization) and interorganization (that is, between organizations) communication. A huge variety of networking hardware and software appeared. One challenge was to enable these different networks to communicate with each other. ARPA accomplished this by developing the Internet Protocol (IP), which created a true “network of networks,” the current architecture of the Internet. The combined set of protocols is now called **TCP/IP**.

Businesses rapidly realized that by using the Internet, they could improve their operations and offer new and better services to their clients. Companies started spending large amounts of money to develop and enhance their Internet presence. This generated fierce competition among communications carriers and hardware and software suppliers to meet the increased infrastructure demand. As a result, **bandwidth**—the information-carrying capacity of communications lines—on the Internet has increased tremendously, while hardware costs have plummeted.

1.11.2 The World Wide Web: Making the Internet User-Friendly

The **World Wide Web** (simply called “the web”) is a collection of hardware and software associated with the Internet that allows computer users to locate and view multimedia-based documents (documents with various combinations of text, graphics, animations, audios and videos) on almost any subject. The introduction of the web was a relatively recent event. In 1989, Tim Berners-Lee of CERN (the European Organization for Nuclear Research) began to develop a technology for sharing information via “hyperlinked” text documents. Berners-Lee called his invention the **HyperText Markup Language (HTML)**. He also wrote communication protocols such as **HyperText Transfer Protocol (HTTP)** to form the backbone of his new hypertext information system, which he referred to as the World Wide Web.

In 1994, Berners-Lee founded an organization, called the **World Wide Web Consortium (W3C)**, www.w3.org, devoted to developing web technologies. One of the W3C’s primary goals is to make the web universally accessible to everyone regardless of disabilities, language or culture. In this book, you’ll use Java to build web-based applications.

1.11.3 Web Services and Mashups

In online Chapter 32, we include a substantial treatment of web services (Fig. 1.17). The applications-development methodology of *mashups* enables you to rapidly develop powerful software applications by combining (often free) complementary web services and other forms of information feeds. One of the first mashups combined the real-estate listings provided by www.craigslist.org with the mapping capabilities of *Google Maps* to offer maps that showed the locations of homes for sale or rent in a given area.

Web services source	How it’s used
Google Maps	Mapping services
Twitter	Microblogging

Fig. 1.17 | Some popular web services (www.programmableweb.com/apis/directory/1?sort=mashups). (Part 1 of 2.)

Web services source	How it's used
YouTube	Video search
Facebook	Social networking
Instagram	Photo sharing
Foursquare	Mobile check-in
LinkedIn	Social networking for business
Groupon	Social commerce
Netflix	Movie rentals
eBay	Internet auctions
Wikipedia	Collaborative encyclopedia
PayPal	Payments
Last.fm	Internet radio
Amazon eCommerce	Shopping for books and many other products
Salesforce.com	Customer Relationship Management (CRM)
Skype	Internet telephony
Microsoft Bing	Search
Flickr	Photo sharing
Zillow	Real-estate pricing
Yahoo Search	Search
WeatherBug	Weather

Fig. 1.17 | Some popular web services (www.programmableweb.com/apis/directory/1?sort=mashups). (Part 2 of 2.)

1.11.4 Ajax

Ajax helps Internet-based applications perform like desktop applications—a difficult task, given that such applications suffer transmission delays as data is shuttled back and forth between your computer and server computers on the Internet. Using Ajax, applications like Google Maps have achieved excellent performance and approach the look-and-feel of desktop applications. Although we don't discuss "raw" Ajax programming (which is quite complex) in this text, we do show in online Chapter 31 how to build Ajax-enabled applications using JavaServer Faces (JSF) Ajax-enabled components.

1.11.5 The Internet of Things

The Internet is no longer just a network of computers—it's an **Internet of Things**. A *thing* is any object with an IP address and the ability to send data automatically over a network—e.g., a car with a transponder for paying tolls, a heart monitor implanted in a human, a smart meter that reports energy usage, mobile apps that can track your movement and location, and smart thermostats that adjust room temperatures based on weather forecasts and activity in the home. You'll use IP addresses to build networked applications in online Chapter 28.

1.12 Software Technologies

Figure 1.18 lists a number of buzzwords that you'll hear in the software development community. We've created Resource Centers on most of these topics, with more on the way.

Technology	Description
Agile software development	Agile software development is a set of methodologies that try to get software implemented faster and using fewer resources. Check out the Agile Alliance (www.agilealliance.org) and the Agile Manifesto (www.agilemanifesto.org).
Refactoring	Refactoring involves reworking programs to make them clearer and easier to maintain while preserving their correctness and functionality. It's widely employed with agile development methodologies. Many IDEs contain built-in <i>refactoring tools</i> to do major portions of the reworking automatically.
Design patterns	Design patterns are proven architectures for constructing flexible and maintainable object-oriented software. The field of design patterns tries to enumerate those recurring patterns, encouraging software designers to <i>reuse</i> them to develop better-quality software using less time, money and effort. We discuss Java design patterns in the online Appendix N.
LAMP	LAMP is an acronym for the open-source technologies that many developers use to build web applications—it stands for <i>Linux</i> , <i>Apache</i> , <i>MySQL</i> and <i>PHP</i> (or <i>Perl</i> or <i>Python</i> —two other scripting languages). MySQL is an open-source database management system. PHP is the most popular open-source server-side “scripting” language for developing web applications. Apache is the most popular web server software. The equivalent for Windows development is WAMP— <i>Windows</i> , <i>Apache</i> , <i>MySQL</i> and <i>PHP</i> .
Software as a Service (SaaS)	Software has generally been viewed as a product; most software still is offered this way. If you want to run an application, you buy a software package from a software vendor—often a CD, DVD or web download. You then install that software on your computer and run it as needed. As new versions appear, you upgrade your software, often at considerable cost in time and money. This process can become cumbersome for organizations that must maintain tens of thousands of systems on a diverse array of computer equipment. With Software as a Service (SaaS) , the software runs on servers elsewhere on the Internet. When that server is updated, all clients worldwide see the new capabilities—no local installation is needed. You access the service through a browser. Browsers are quite portable, so you can run the same applications on a wide variety of computers from anywhere in the world. Salesforce.com, Google, and Microsoft's Office Live and Windows Live all offer SaaS.
Platform as a Service (PaaS)	Platform as a Service (PaaS) provides a computing platform for developing and running applications as a service over the web, rather than installing the tools on your computer. Some PaaS providers are Google App Engine, Amazon EC2 and Windows Azure™.

Fig. 1.18 | Software technologies. (Part 1 of 2.)

Technology	Description
Cloud computing	SaaS and PaaS are examples of cloud computing . You can use software and data stored in the “cloud”—i.e., accessed on remote computers (or servers) via the Internet and available on demand—rather than having it stored on your desktop, notebook computer or mobile device. This allows you to increase or decrease computing resources to meet your needs at any given time, which is more cost effective than purchasing hardware to provide enough storage and processing power to meet occasional peak demands. Cloud computing also saves money by shifting the burden of managing these apps to the service provider.
Software Development Kit (SDK)	Software Development Kits (SDKs) include the tools and documentation developers use to program applications. For example, you’ll use the Java Development Kit (JDK) to build and run Java applications.

Fig. 1.18 | Software technologies. (Part 2 of 2.)

Software is complex. Large, real-world software applications can take many months or even years to design and implement. When large software products are under development, they typically are made available to the user communities as a series of releases, each more complete and polished than the last (Fig. 1.19).

Version	Description
Alpha	<i>Alpha</i> software is the earliest release of a software product that’s still under active development. Alpha versions are often buggy, incomplete and unstable and are released to a relatively small number of developers for testing new features, getting early feedback, etc.
Beta	<i>Beta</i> versions are released to a larger number of developers later in the development process after most major bugs have been fixed and new features are nearly complete. Beta software is more stable, but still subject to change.
Release candidates	<i>Release candidates</i> are generally <i>feature complete</i> , (mostly) bug free and ready for use by the community, which provides a diverse testing environment—the software is used on different systems, with varying constraints and for a variety of purposes.
Final release	Any bugs that appear in the release candidate are corrected, and eventually the final product is released to the general public. Software companies often distribute incremental updates over the Internet.
Continuous beta	Software that’s developed using this approach (for example, Google search or Gmail) generally does not have version numbers. It’s hosted in the <i>cloud</i> (not installed on your computer) and is constantly evolving so that users always have the latest version.

Fig. 1.19 | Software product-release terminology.

1.13 Keeping Up-to-Date with Information Technologies

Figure 1.20 lists key technical and business publications that will help you stay up-to-date with the latest news and trends and technology. You can also find a growing list of Internet- and web-related Resource Centers at www.deitel.com/ResourceCenters.html.

Publication	URL
AllThingsD	allthingsd.com
Bloomberg BusinessWeek	www.businessweek.com
CNET	news.cnet.com
Communications of the ACM	cacm.acm.org
Computerworld	www.computerworld.com
Engadget	www.engadget.com
eWeek	www.eweek.com
Fast Company	www.fastcompany.com/
Fortune	money.cnn.com/magazines/fortune
GigaOM	gigaom.com
Hacker News	news.ycombinator.com
IEEE Computer Magazine	www.computer.org/portal/web/computingnow/computer
InfoWorld	www.infoworld.com
Mashable	mashable.com
PCWorld	www.pcworld.com
SD Times	www.sdtimes.com
Slashdot	slashdot.org/
Technology Review	technologyreview.com
Techcrunch	techcrunch.com
The Next Web	thenextweb.com
The Verge	www.theverge.com
Wired	www.wired.com

Fig. 1.20 | Technical and business publications.

Self-Review Exercises

- 1.1** Fill in the blanks in each of the following statements:
- Computers process data under the control of sets of instructions called _____.
 - The key logical units of the computer are the _____, _____, _____, _____, _____ and _____.
 - The three types of languages discussed in the chapter are _____, _____ and _____.
 - The programs that translate high-level language programs into machine language are called _____.

- e) _____ is an operating system for mobile devices based on the Linux kernel and Java.
- f) _____ software is generally feature complete, (supposedly) bug free and ready for use by the community.
- g) The Wii Remote, as well as many smartphones, use a(n) _____ which allows the device to respond to motion.

1.2 Fill in the blanks in each of the following sentences about the Java environment:

- a) The _____ command from the JDK executes a Java application.
- b) The _____ command from the JDK compiles a Java program.
- c) A Java source code file must end with the _____ file extension.
- d) When a Java program is compiled, the file produced by the compiler ends with the _____ file extension.
- e) The file produced by the Java compiler contains _____ that are executed by the Java Virtual Machine.

1.3 Fill in the blanks in each of the following statements (based on Section 1.5):

- a) Objects enable the design practice of _____—although they may know how to communicate with one another across well-defined interfaces, they normally are not allowed to know how other objects are implemented.
- b) Java programmers concentrate on creating _____, which contain fields and the set of methods that manipulate those fields and provide services to clients.
- c) The process of analyzing and designing a system from an object-oriented point of view is called _____.
- d) A new class of objects can be created conveniently by _____—the new class (called the subclass) starts with the characteristics of an existing class (called the superclass), possibly customizing them and adding unique characteristics of its own.
- e) _____ is a graphical language that allows people who design software systems to use an industry-standard notation to represent them.
- f) The size, shape, color and weight of an object are considered _____ of the object's class.

Answers to Self-Review Exercises

1.1 a) programs. b) input unit, output unit, memory unit, central processing unit, arithmetic and logic unit, secondary storage unit. c) machine languages, assembly languages, high-level languages. d) compilers. e) Android. f) Release candidate. g) accelerometer.

1.2 a) java. b) javac. c) .java. d) .class. e) bytecodes.

1.3 a) information hiding. b) classes. c) object-oriented analysis and design (OOAD). d) inheritance. e) The Unified Modeling Language (UML). f) attributes.

Exercises

1.4 Fill in the blanks in each of the following statements:

- a) The logical unit that receives information from outside the computer for use by the computer is the _____.
- b) The process of instructing the computer to solve a problem is called _____.
- c) _____ is a type of computer language that uses Englishlike abbreviations for machine-language instructions.
- d) _____ is a logical unit that sends information which has already been processed by the computer to various devices so that it may be used outside the computer.
- e) _____ and _____ are logical units of the computer that retain information.
- f) _____ is a logical unit of the computer that performs calculations.
- g) _____ is a logical unit of the computer that makes logical decisions.

- h) _____ languages are most convenient to the programmer for writing programs quickly and easily.
- i) The only language a computer can directly understand is that computer's _____.
- j) _____ is a logical unit of the computer that coordinates the activities of all the other logical units.

1.5 Fill in the blanks in each of the following statements:

- a) The _____ programming language is now used to develop large-scale enterprise applications, to enhance the functionality of web servers, to provide applications for consumer devices and for many other purposes.
- b) _____ initially became widely known as the development language of the UNIX operating system.
- c) The _____ ensures that messages, consisting of sequentially numbered pieces called bytes, were properly routed from sender to receiver, arrived intact and were assembled in the correct order.
- d) The _____ programming language was developed by Bjarne Stroustrup in the early 1980s at Bell Laboratories.

1.6 Fill in the blanks in each of the following statements:

- a) Java programs normally go through five phases—_____, _____, _____, _____ and _____.
- b) A(n) _____ provides many tools that support the software development process, such as editors for writing and editing programs, debuggers for locating logic errors in programs, and many other features.
- c) The command java invokes the _____, which executes Java programs.
- d) A(n) _____ is a software application that simulates a computer, but hides the underlying operating system and hardware from the programs that interact with it.
- e) The _____ takes the .class files containing the program's bytecodes and transfers them to primary memory.
- f) The _____ examines bytecodes to ensure that they're valid.

1.7 Explain the two compilation phases of Java programs.

1.8 One of the world's most common objects is a wrist watch. Discuss how each of the following terms and concepts applies to the notion of a watch: object, attributes, behaviors, class, inheritance (consider, for example, an alarm clock), modeling, messages, encapsulation, interface and information hiding.

Making a Difference

Throughout the book we've included Making a Difference exercises in which you'll be asked to work on problems that really matter to individuals, communities, countries and the world. For more information about worldwide organizations working to make a difference, and for related programming project ideas, visit our Making a Difference Resource Center at www.deitel.com/makingadifference.

1.9 (*Test-Drive: Carbon Footprint Calculator*) Some scientists believe that carbon emissions, especially from the burning of fossil fuels, contribute significantly to global warming and that this can be combated if individuals take steps to limit their use of carbon-based fuels. Organizations and individuals are increasingly concerned about their "carbon footprints." Websites such as TerraPass

<http://www.terrapass.com/carbon-footprint-calculator/>

and Carbon Footprint

<http://www.carbonfootprint.com/calculator.aspx>

provide carbon-footprint calculators. Test-drive these calculators to determine your carbon footprint. Exercises in later chapters will ask you to program your own carbon-footprint calculator. To prepare for this, use the web to research the formulas for calculating carbon footprints.

1.10 (*Test-Drive: Body Mass Index Calculator*) Obesity causes significant increases in illnesses such as diabetes and heart disease. To determine whether a person is overweight or obese, you can use a measure called the body mass index (BMI). The United States Department of Health and Human Services provides a BMI calculator at <http://www.nhlbi.nih.gov/guidelines/obesity/BMI/bmicalc.htm>. Use it to calculate your own BMI. A forthcoming exercise will ask you to program your own BMI calculator. To prepare for this, use the web to research the formulas for calculating BMI.

1.11 (*Attributes of Hybrid Vehicles*) In this chapter you learned some basics of classes. Now you'll "flesh out" aspects of a class called "Hybrid Vehicle." Hybrid vehicles are becoming increasingly popular, because they often get much better mileage than purely gasoline-powered vehicles. Browse the web and study the features of four or five of today's popular hybrid cars, then list as many of their hybrid-related attributes as you can. Some common attributes include city-miles-per-gallon and highway-miles-per-gallon. Also list the attributes of the batteries (type, weight, etc.).

1.12 (*Gender Neutrality*) Many people want to eliminate sexism in all forms of communication. You've been asked to create a program that can process a paragraph of text and replace gender-specific words with gender-neutral ones. Assuming that you've been given a list of gender-specific words and their gender-neutral replacements (e.g., replace both "wife" and "husband" with "spouse," "man" and "woman" with "person," "daughter" and "son" with "child"), explain the procedure you'd use to read through a paragraph of text and manually perform these replacements. How might your procedure generate a strange term like "woperchild?" You'll soon learn that a more formal term for "procedure" is "algorithm," and that an algorithm specifies the steps to be performed and the order in which to perform them. We'll show how to develop algorithms then convert them to Java programs which can be run on computers.

2

Introduction to Java Applications; Input/Output and Operators

*What's in a name?
That which we call a rose
By any other name would
smell as sweet.*

—William Shakespeare

*The chief merit of language
is clearness.*

—Galen

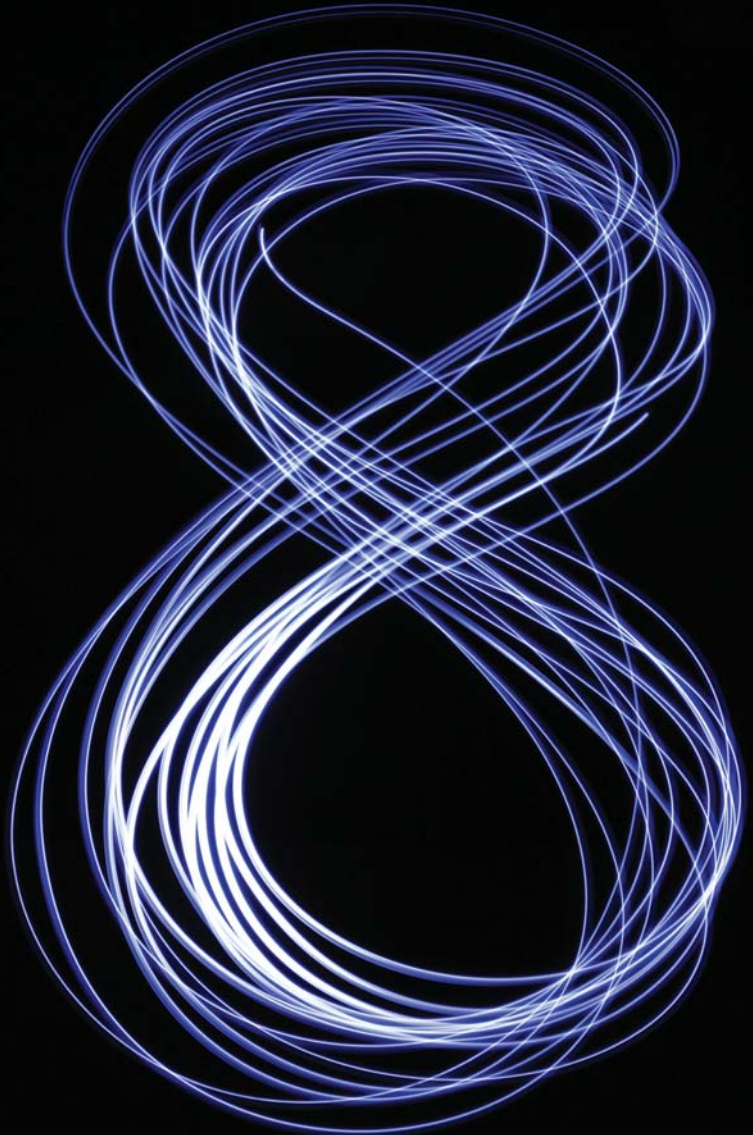
*One person can make a
difference and every person
should try.*

—John F. Kennedy

Objectives

In this chapter you'll:

- Write simple Java applications.
- Use input and output statements.
- Learn about Java's primitive types.
- Understand basic memory concepts.
- Use arithmetic operators.
- Learn the precedence of arithmetic operators.
- Write decision-making statements.
- Use relational and equality operators.



- 2.1 Introduction
- 2.2 Your First Program in Java: Printing a Line of Text
- 2.3 Modifying Your First Java Program
- 2.4 Displaying Text with `printf`
- 2.5 Another Application: Adding Integers
 - 2.5.1 `import` Declarations
 - 2.5.2 Declaring Class `Addition`
 - 2.5.3 Declaring and Creating a `Scanner` to Obtain User Input from the Keyboard
 - 2.5.4 Declaring Variables to Store Integers
 - 2.5.5 Prompting the User for Input
 - 2.5.6 Obtaining an `int` as Input from the User
 - 2.5.7 Prompting for and Inputting a Second `int`
 - 2.5.8 Using Variables in a Calculation
 - 2.5.9 Displaying the Result of the Calculation
 - 2.5.10 Java API Documentation
- 2.6 Memory Concepts
- 2.7 Arithmetic
- 2.8 Decision Making: Equality and Relational Operators
- 2.9 Wrap-Up

Summary | Self-Review Exercises | Answers to Self-Review Exercises | Exercises | Making a Difference

2.1 Introduction

This chapter introduces Java application programming. We begin with examples of programs that display (output) messages on the screen. We then present a program that obtains (inputs) two numbers from a user, calculates their sum and displays the result. You'll learn how to instruct the computer to perform arithmetic calculations and save their results for later use. The last example demonstrates how to make decisions. The application compares two numbers, then displays messages that show the comparison results. You'll use the JDK command-line tools to compile and run this chapter's programs. If you prefer to use an integrated development environment (IDE), we've also posted Dive Into[®] videos at <http://www.deitel.com/books/jhttp10/> for Eclipse, NetBeans and IntelliJ IDEA.

2.2 Your First Program in Java: Printing a Line of Text

A Java **application** is a computer program that executes when you use the **java** command to launch the Java Virtual Machine (JVM). Later in this section we'll discuss how to compile and run a Java application. First we consider a simple application that displays a line of text. Figure 2.1 shows the program followed by a box that displays its output.

```

1 // Fig. 2.1: Welcome1.java
2 // Text-printing program.
3
4 public class Welcome1
5 {
6     // main method begins execution of Java application
7     public static void main(String[] args)
8     {
9         System.out.println("Welcome to Java Programming!");
10    } // end method main
11 } // end class Welcome1

```

Fig. 2.1 | Text-printing program. (Part 1 of 2.)

```
Welcome to Java Programming!
```

Fig. 2.1 | Text-printing program. (Part 2 of 2.)

The program includes line numbers. We've added these for instructional purposes—they're *not* part of a Java program. This example illustrates several important Java features. We'll see that line 9 does the work—displaying the phrase `Welcome to Java Programming!` on the screen.

Commenting Your Programs

We insert **comments** to **document programs** and improve their readability. The Java compiler *ignores* comments, so they do *not* cause the computer to perform any action when the program is run.

By convention, we begin every program with a comment indicating the figure number and filename. The comment in line 1

```
// Fig. 2.1: Welcome1.java
```

begins with `//`, indicating that it's an **end-of-line comment**—it terminates at the end of the line on which the `//` appears. An end-of-line comment need not begin a line; it also can begin in the middle of a line and continue until the end (as in lines 6, 10 and 11). Line 2

```
// Text-printing program.
```

by our convention, is a comment that describes the purpose of the program.

Java also has **traditional comments**, which can be spread over several lines as in

```
/* This is a traditional comment. It
   can be split over multiple lines */
```

These begin and end with delimiters, `/*` and `*/`. The compiler ignores all text between the delimiters. Java incorporated traditional comments and end-of-line comments from the C and C++ programming languages, respectively. We prefer using `//` comments.

Java provides comments of a third type—**Javadoc comments**. These are delimited by `/**` and `*/`. The compiler ignores all text between the delimiters. Javadoc comments enable you to embed program documentation directly in your programs. Such comments are the preferred Java documenting format in industry. The **javadoc utility program** (part of the JDK) reads Javadoc comments and uses them to prepare program documentation in HTML format. We demonstrate Javadoc comments and the javadoc utility in online Appendix G, Creating Documentation with javadoc.



Common Programming Error 2.1

*Forgetting one of the delimiters of a traditional or Javadoc comment is a syntax error. A **syntax error** occurs when the compiler encounters code that violates Java's language rules (i.e., its syntax). These rules are similar to a natural language's grammar rules specifying sentence structure. Syntax errors are also called **compiler errors**, **compile-time errors** or **compilation errors**, because the compiler detects them when compiling the program. When a syntax error is encountered, the compiler issues an error message. You must eliminate all compilation errors before your program will compile properly.*



Good Programming Practice 2.1

Some organizations require that every program begin with a comment that states the purpose of the program and the author, date and time when the program was last modified.



Error-Prevention Tip 2.1

As you write new programs or modify existing ones, keep your comments up-to-date with the code. Programmers will often need to make changes to existing code to fix errors or to enhance capabilities. Updating your comments helps ensure that they accurately reflect what the code does. This will make your programs easier to understand and modify in the future. Programmers using or updating code with out-of-date comments might make incorrect assumptions about the code that could lead to errors or even security breaches.

Using Blank Lines

Line 3 is a blank line. Blank lines, space characters and tabs make programs easier to read. Together, they're known as **white space** (or whitespace). The compiler ignores white space.



Good Programming Practice 2.2

Use blank lines and spaces to enhance program readability.

Declaring a Class

Line 4

```
public class Welcome1
```

begins a **class declaration** for class `Welcome1`. Every Java program consists of at least one class that you (the programmer) define. The **class keyword** introduces a class declaration and is immediately followed by the **class name** (`Welcome1`). **Keywords** (sometimes called **reserved words**) are reserved for use by Java and are always spelled with all lowercase letters. The complete list of keywords is shown in Appendix C.

In Chapters 2–7, every class we define begins with the **public** keyword. For now, we simply require `public`. You'll learn more about `public` and non-`public` classes in Chapter 8.

Filename for a public Class

A `public` class *must* be placed in a file that has a filename of the form *ClassName.java*, so class `Welcome1` is stored in the file `Welcome1.java`.



Common Programming Error 2.2

A compilation error occurs if a `public` class's filename is not exactly same name as the class (in terms of both spelling and capitalization) followed by the `.java` extension.

Class Names and Identifiers

By convention, class names begin with a capital letter and capitalize the first letter of each word they include (e.g., `SampleClassName`). A class name is an **identifier**—a series of characters consisting of letters, digits, underscores (`_`) and dollar signs (`$`) that does *not* begin with a digit and does *not* contain spaces. Some valid identifiers are `Welcome1`, `$value`, `_value`, `m_inputField1` and `button7`. The name `7button` is *not* a valid identifier because it begins with a digit, and the name `input field` is *not* a valid identifier because it contains a space. Normally, an identifier that does not begin with a capital letter is not a class name.

Java is **case sensitive**—uppercase and lowercase letters are distinct—so `value` and `Value` are different (but both valid) identifiers.

Class Body

A **left brace** (as in line 5), `{`, begins the **body** of every class declaration. A corresponding **right brace** (at line 11), `}`, must end each class declaration. Lines 6–10 are indented.



Good Programming Practice 2.3

Indent the entire body of each class declaration one “level” between the left brace and the right brace that delimit the body of the class. This format emphasizes the class declaration’s structure and makes it easier to read. We use three spaces to form a level of indent—many programmers prefer two or four spaces. Whatever you choose, use it consistently.



Error-Prevention Tip 2.2

When you type an opening left brace, `{`, immediately type the closing right brace, `}`, then reposition the cursor between the braces and indent to begin typing the body. This practice helps prevent errors due to missing braces. Many IDEs insert the closing right brace for you when you type the opening left brace.



Common Programming Error 2.3

It’s a syntax error if braces do not occur in matching pairs.



Good Programming Practice 2.4

IDEs typically indent code for you. The Tab key may also be used to indent code. You can configure each IDE to specify the number of spaces inserted when you press Tab.

Declaring a Method

Line 6

```
// main method begins execution of Java application
```

is an end-of-line comment indicating the purpose of lines 7–10 of the program. Line 7

```
public static void main(String[] args)
```

is the starting point of every Java application. The **parentheses** after the identifier `main` indicate that it’s a program building block called a **method**. Java class declarations normally contain one or more methods. For a Java application, one of the methods *must* be called `main` and must be defined as shown in line 7; otherwise, the Java Virtual Machine (JVM) will not execute the application. Methods perform tasks and can return information when they complete their tasks. We’ll explain the purpose of keyword `static` in Section 3.2.5. Keyword **`void`** indicates that this method will *not* return any information. Later, we’ll see how a method can return information. For now, simply mimic `main`’s first line in your Java applications. In line 7, the `String[] args` in parentheses is a required part of the method `main`’s declaration—we discuss this in Chapter 7.

The left brace in line 8 begins the **body of the method declaration**. A corresponding right brace must end it (line 10). Line 9 in the method body is indented between the braces.



Good Programming Practice 2.5

Indent the entire body of each method declaration one “level” between the braces that define the body of the method. This makes the structure of the method stand out and makes the method declaration easier to read.

Performing Output with `System.out.println`

Line 9

```
System.out.println("Welcome to Java Programming!");
```

instructs the computer to perform an action—namely, to display the characters contained between the double quotation marks (the quotation marks themselves are *not* displayed). Together, the quotation marks and the characters between them are a **string**—also known as a **character string** or a **string literal**. White-space characters in strings are *not* ignored by the compiler. Strings *cannot* span multiple lines of code.

The **System.out** object—which is predefined for you—is known as the **standard output object**. It allows a Java application to display information in the **command window** from which it executes. In recent versions of Microsoft Windows, the command window is the **Command Prompt**. In UNIX/Linux/Mac OS X, the command window is called a **terminal window** or a **shell**. Many programmers call it simply the **command line**.

Method **System.out.println** displays (or prints) a line of text in the command window. The string in the parentheses in line 9 is the **argument** to the method. When `System.out.println` completes its task, it positions the output cursor (the location where the next character will be displayed) at the beginning of the next line in the command window. This is similar to what happens when you press the *Enter* key while typing in a text editor—the cursor appears at the beginning of the next line in the document.

The entire line 9, including `System.out.println`, the argument `"Welcome to Java Programming!"` in the parentheses and the **semicolon** (`;`), is called a **statement**. A method typically contains one or more statements that perform its task. Most statements end with a semicolon. When the statement in line 9 executes, it displays `Welcome to Java Programming!` in the command window.

When learning how to program, sometimes it's helpful to “break” a working program so you can familiarize yourself with the compiler's syntax-error messages. These messages do not always state the exact problem in the code. When you encounter an error, it will give you an idea of what caused it. [Try removing a semicolon or brace from the program of Fig. 2.1, then recompile the program to see the error messages generated by the omission.]



Error-Prevention Tip 2.3

When the compiler reports a syntax error, it may not be on the line that the error message indicates. First, check the line for which the error was reported. If you don't find an error on that line, check several preceding lines.

Using End-of-Line Comments on Right Braces for Readability

As an aid to programming novices, we include an end-of-line comment after a closing brace that ends a method declaration and after a closing brace that ends a class declaration. For example, line 10

```
} // end method main
```

indicates the closing brace of method `main`, and line 11

```
} // end class Welcome1
```

indicates the closing brace of class `Welcome1`. Each comment indicates the method or class that the right brace terminates. We'll omit such ending comments after this chapter.

Compiling Your First Java Application

We're now ready to compile and execute our program. We assume you're using the Java Development Kit's command-line tools, not an IDE. To help you compile and run your programs in an IDE, we provide online Dive Into® videos for the popular IDEs Eclipse, NetBeans and IntelliJ IDEA. These are located on the book's website:

```
http://www.deitel.com/books/jhtp10
```

To prepare to compile the program, open a command window and change to the directory where the program is stored. Many operating systems use the command `cd` to change directories. On Windows, for example,

```
cd c:\examples\ch02\fig02_01
```

changes to the `fig02_01` directory. On UNIX/Linux/Max OS X, the command

```
cd ~/examples/ch02/fig02_01
```

changes to the `fig02_01` directory. To compile the program, type

```
javac Welcome1.java
```

If the program contains no compilation errors, this command creates a new file called `Welcome1.class` (known as the **class file** for `Welcome1`) containing the platform-independent Java bytecodes that represent our application. When we use the `java` command to execute the application on a given platform, the JVM will translate these bytecodes into instructions that are understood by the underlying operating system and hardware.



Common Programming Error 2.4

*When using `javac`, if you receive a message such as “bad command or filename,” “`javac: command not found`” or “`javac` is not recognized as an internal or external command, operable program or batch file,” then your Java software installation was not completed properly. This indicates that the system's `PATH` environment variable was not set properly. Carefully review the installation instructions in the *Before You Begin* section of this book. On some systems, after correcting the `PATH`, you may need to reboot your computer or open a new command window for these settings to take effect.*

Each syntax-error message contains the filename and line number where the error occurred. For example, `Welcome1.java:6` indicates that an error occurred at line 6 in `Welcome1.java`. The rest of the message provides information about the syntax error.



Common Programming Error 2.5

The compiler error message “`class Welcome1 is public, should be declared in a file named Welcome1.java`” indicates that the filename does not match the name of the public class in the file or that you typed the class name incorrectly when compiling the class.

Executing the Welcome1 Application

The following instructions assume that the book's examples are located in C:\examples on Windows or in your user account's Documents/examples folder on Linux/OS X. To execute this program in a command window, change to the directory containing Welcome1.java—C:\examples\ch02\fig02_01 on Microsoft Windows or ~/Documents/examples/ch02/fig02_01 on Linux/OS X. Next, type

```
java Welcome1
```

and press *Enter*. This command launches the JVM, which loads the Welcome1.class file. The command *omits* the .class file-name extension; otherwise, the JVM will *not* execute the program. The JVM calls class Welcome1's main method. Next, the statement at line 9 of main displays "Welcome to Java Programming!". Figure 2.2 shows the program executing in a Microsoft Windows **Command Prompt** window. [Note: Many environments show command windows with black backgrounds and white text. We adjusted these settings to make our screen captures more readable.]



Error-Prevention Tip 2.4

When attempting to run a Java program, if you receive a message such as "Exception in thread "main" java.lang.NoClassDefFoundError: Welcome1," your CLASSPATH environment variable has not been set properly. Please carefully review the installation instructions in the *Before You Begin* section of this book. On some systems, you may need to reboot your computer or open a new command window after configuring the CLASSPATH.

You type this command to execute the application

The program outputs to the screen
Welcome to Java Programming!

Fig. 2.2 | Executing Welcome1 from the Command Prompt.

2.3 Modifying Your First Java Program

In this section, we modify the example in Fig. 2.1 to print text on one line by using multiple statements and to print text on several lines by using a single statement.

Displaying a Single Line of Text with Multiple Statements

Welcome to Java Programming! can be displayed several ways. Class Welcome2, shown in Fig. 2.3, uses two statements (lines 9–10) to produce the output shown in Fig. 2.1. [Note: From this point forward, we highlight with a yellow background the new and key features in each code listing, as we've done for lines 9–10.]

The program is similar to Fig. 2.1, so we discuss only the changes here. Line 2

```
// Printing a line of text with multiple statements.
```

```

1 // Fig. 2.3: Welcome2.java
2 // Printing a line of text with multiple statements.
3
4 public class Welcome2
5 {
6     // main method begins execution of Java application
7     public static void main(String[] args)
8     {
9         System.out.print("Welcome to ");
10        System.out.println("Java Programming!");
11    } // end method main
12 } // end class Welcome2

```

```
Welcome to Java Programming!
```

Fig. 2.3 | Printing a line of text with multiple statements.

is an end-of-line comment stating the purpose of the program. Line 4 begins the `Welcome2` class declaration. Lines 9–10 of method `main`

```

System.out.print("Welcome to ");
System.out.println("Java Programming!");

```

display one line of text. The first statement uses `System.out`'s method `print` to display a string. Each `print` or `println` statement resumes displaying characters from where the last `print` or `println` statement stopped displaying characters. Unlike `println`, after displaying its argument, `print` does *not* position the output cursor at the beginning of the next line in the command window—the next character the program displays will appear *immediately after* the last character that `print` displays. So, line 10 positions the first character in its argument (the letter “J”) immediately after the last character that line 9 displays (the *space character* before the string's closing double-quote character).

Displaying Multiple Lines of Text with a Single Statement

A single statement can display multiple lines by using **newline characters**, which indicate to `System.out`'s `print` and `println` methods when to position the output cursor at the beginning of the next line in the command window. Like blank lines, space characters and tab characters, newline characters are whitespace characters. The program in Fig. 2.4 outputs four lines of text, using newline characters to determine when to begin each new line. Most of the program is identical to those in Figs. 2.1 and 2.3.

```

1 // Fig. 2.4: Welcome3.java
2 // Printing multiple lines of text with a single statement.
3
4 public class Welcome3
5 {
6     // main method begins execution of Java application
7     public static void main(String[] args)
8     {

```

Fig. 2.4 | Printing multiple lines of text with a single statement. (Part I of 2.)

```

9      System.out.println("Welcome\nto\nJava\nProgramming!");
10     } // end method main
11 } // end class Welcome3

```

```

Welcome
to
Java
Programming!

```

Fig. 2.4 | Printing multiple lines of text with a single statement. (Part 2 of 2.)

Line 9

```
System.out.println("Welcome\nto\nJava\nProgramming!");
```

displays four lines of text in the command window. Normally, the characters in a string are displayed *exactly* as they appear in the double quotes. However, the paired characters \ and n (repeated three times in the statement) do *not* appear on the screen. The **backslash** (\) is an **escape character**, which has special meaning to System.out's print and println methods. When a backslash appears in a string, Java combines it with the next character to form an **escape sequence**—\n represents the newline character. When a newline character appears in a string being output with System.out, the newline character causes the screen's output cursor to move to the beginning of the next line in the command window.

Figure 2.5 lists several common escape sequences and describes how they affect the display of characters in the command window. For the complete list of escape sequences, visit

```
http://docs.oracle.com/javase/specs/jls/se7/html/
jls-3.html#jls-3.10.6
```

Escape sequence	Description
\n	Newline. Position the screen cursor at the beginning of the <i>next</i> line.
\t	Horizontal tab. Move the screen cursor to the next tab stop.
\r	Carriage return. Position the screen cursor at the beginning of the <i>current</i> line—do <i>not</i> advance to the next line. Any characters output after the carriage return <i>overwrite</i> the characters previously output on that line.
\\	Backslash. Used to print a backslash character.
\"	Double quote. Used to print a double-quote character. For example, System.out.println("\"in quotes\""); displays "in quotes".

Fig. 2.5 | Some common escape sequences.

2.4 Displaying Text with printf

The **System.out.printf** method (f means “formatted”) displays *formatted* data. Figure 2.6 uses this method to output on two lines the strings “Welcome to” and “Java Programming!”.

```

1 // Fig. 2.6: Welcome4.java
2 // Displaying multiple lines with method System.out.printf.
3
4 public class Welcome4
5 {
6     // main method begins execution of Java application
7     public static void main(String[] args)
8     {
9         System.out.printf("%s\n%s\n",
10             "Welcome to", "Java Programming!");
11     } // end method main
12 } // end class Welcome4

```

```

Welcome to
Java Programming!

```

Fig. 2.6 | Displaying multiple lines with method `System.out.printf`.

Lines 9–10

```

System.out.printf("%s\n%s\n",
    "Welcome to", "Java Programming!");

```

call method `System.out.printf` to display the program's output. The method call specifies three arguments. When a method requires multiple arguments, they're placed in a **comma-separated list**. Calling a method is also referred to as **invoking** a method.



Good Programming Practice 2.6

Place a space after each comma (,) in an argument list to make programs more readable.

Lines 9–10 represent only *one* statement. Java allows large statements to be split over many lines. We indent line 10 to indicate that it's a *continuation* of line 9.



Common Programming Error 2.6

Splitting a statement in the middle of an identifier or a string is a syntax error.

Method `printf`'s first argument is a **format string** that may consist of **fixed text** and **format specifiers**. Fixed text is output by `printf` just as it would be by `print` or `println`. Each format specifier is a *placeholder* for a value and specifies the *type of data* to output. Format specifiers also may include optional formatting information.

Format specifiers begin with a percent sign (%) followed by a character that represents the *data type*. For example, the format specifier **%s** is a placeholder for a string. The format string in line 9 specifies that `printf` should output two strings, each followed by a newline character. At the first format specifier's position, `printf` substitutes the value of the first argument after the format string. At each subsequent format specifier's position, `printf` substitutes the value of the next argument. So this example substitutes "Welcome to" for the first %s and "Java Programming!" for the second %s. The output shows that two lines of text are displayed on two lines.

Notice that instead of using the escape sequence `\n`, we used the **%n** format specifier, which is a line separator that's *portable* across operating systems. You cannot use %n in the

argument to `System.out.print` or `System.out.println`; however, the line separator output by `System.out.println` *after* it displays its argument *is* portable across operating systems. Online Appendix I presents more details of formatting output with `printf`.

2.5 Another Application: Adding Integers

Our next application reads (or inputs) two **integers** (whole numbers, such as -22, 7, 0 and 1024) typed by a user at the keyboard, computes their sum and displays it. This program must keep track of the numbers supplied by the user for the calculation later in the program. Programs remember numbers and other data in the computer's memory and access that data through program elements called **variables**. The program of Fig. 2.7 demonstrates these concepts. In the sample output, we use bold text to identify the user's input (i.e., **45** and **72**). As in prior programs, Lines 1–2 state the figure number, filename and purpose of the program.

```

1  // Fig. 2.7: Addition.java
2  // Addition program that inputs two numbers then displays their sum.
3  import java.util.Scanner; // program uses class Scanner
4
5  public class Addition
6  {
7      // main method begins execution of Java application
8      public static void main(String[] args)
9      {
10         // create a Scanner to obtain input from the command window
11         Scanner input = new Scanner(System.in);
12
13         int number1; // first number to add
14         int number2; // second number to add
15         int sum; // sum of number1 and number2
16
17         System.out.print("Enter first integer: "); // prompt
18         number1 = input.nextInt(); // read first number from user
19
20         System.out.print("Enter second integer: "); // prompt
21         number2 = input.nextInt(); // read second number from user
22
23         sum = number1 + number2; // add numbers, then store total in sum
24
25         System.out.printf("Sum is %d\n", sum); // display sum
26     } // end method main
27 } // end class Addition

```

```

Enter first integer: 45
Enter second integer: 72
Sum is 117

```

Fig. 2.7 | Addition program that inputs two numbers then displays their sum.

2.5.1 import Declarations

A great strength of Java is its rich set of predefined classes that you can *reuse* rather than “reinventing the wheel.” These classes are grouped into **packages**—*named groups of related*

classes—and are collectively referred to as the **Java class library**, or the **Java Application Programming Interface (Java API)**. Line 3

```
import java.util.Scanner; // program uses class Scanner
```

is an **import declaration** that helps the compiler locate a class that's used in this program. It indicates that the program uses the predefined Scanner class (discussed shortly) from the package named **java.util**. The compiler then ensures that you use the class correctly.



Common Programming Error 2.7

All import declarations must appear before the first class declaration in the file. Placing an import declaration inside or after a class declaration is a syntax error.



Common Programming Error 2.8

Forgetting to include an import declaration for a class that must be imported results in a compilation error containing a message such as “cannot find symbol.” When this occurs, check that you provided the proper import declarations and that the names in them are correct, including proper capitalization.



Software Engineering Observation 2.1

*In each new Java version, the APIs typically contain new capabilities that fix bugs, improve performance or offer better means for accomplishing tasks. The corresponding older versions are no longer needed and should not be used. Such APIs are said to be **deprecated** and might be removed from later Java versions.*

You'll often encounter deprecated APIs when browsing the online API documentation. The compiler will warn you when you compile code that uses deprecated APIs. If you compile your code with javac using the command-line argument `-deprecation`, the compiler will tell you which deprecated features you're using. For each one, the online documentation (<http://docs.oracle.com/javase/7/docs/api/>) indicates and typically links to the new feature that replaces the deprecated one.

2.5.2 Declaring Class Addition

Line 5

```
public class Addition
```

begins the declaration of class Addition. The filename for this public class must be Addition.java. Remember that the body of each class declaration starts with an opening left brace (line 6) and ends with a closing right brace (line 27).

The application begins execution with the main method (lines 8–26). The left brace (line 9) marks the beginning of method main's body, and the corresponding right brace (line 26) marks its end. Method main is indented one level in the body of class Addition, and the code in the body of main is indented another level for readability.

2.5.3 Declaring and Creating a Scanner to Obtain User Input from the Keyboard

A **variable** is a location in the computer's memory where a value can be stored for use later in a program. All Java variables *must* be declared with a **name** and a **type** before they can be used. A variable's *name* enables the program to access the *value* of the variable in memory. A

variable's name can be any valid identifier—again, a series of characters consisting of letters, digits, underscores (_) and dollar signs (\$) that does *not* begin with a digit and does *not* contain spaces. A variable's *type* specifies what kind of information is stored at that location in memory. Like other statements, declaration statements end with a semicolon (;).

Line 11

```
Scanner input = new Scanner(System.in);
```

is a **variable declaration statement** that specifies the *name* (input) and *type* (Scanner) of a variable that's used in this program. A **Scanner** enables a program to read data (e.g., numbers and strings) for use in a program. The data can come from many sources, such as the user at the keyboard or a file on disk. Before using a Scanner, you must create it and specify the *source* of the data.

The = in line 11 indicates that Scanner variable input should be **initialized** (i.e., prepared for use in the program) in its declaration with the result of the expression to the right of the equals sign—new Scanner(System.in). This expression uses the **new** keyword to create a Scanner object that reads characters typed by the user at the keyboard. The **standard input object, System.in**, enables applications to read *bytes* of data typed by the user. The Scanner translates these bytes into types (like ints) that can be used in a program.

2.5.4 Declaring Variables to Store Integers

The variable declaration statements in lines 13–15

```
int number1; // first number to add
int number2; // second number to add
int sum; // sum of number1 and number2
```

declare that variables number1, number2 and sum hold data of type **int**—they can hold *integer* values (whole numbers such as 72, -1127 and 0). These variables are *not* yet initialized. The range of values for an int is -2,147,483,648 to +2,147,483,647. [Note: The int values you use in a program may not contain commas.]

Some other types of data are **float** and **double**, for holding real numbers, and **char**, for holding character data. Real numbers contain decimal points, such as in 3.4, 0.0 and -11.19. Variables of type char represent individual characters, such as an uppercase letter (e.g., A), a digit (e.g., 7), a special character (e.g., * or %) or an escape sequence (e.g., the tab character, \t). The types int, float, double and char are called **primitive types**. Primitive-type names are keywords and must appear in all lowercase letters. Appendix D summarizes the characteristics of the eight primitive types (boolean, byte, char, short, int, long, float and double).

Several variables of the same type may be declared in a single declaration with the variable names separated by commas (i.e., a comma-separated list of variable names). For example, lines 13–15 can also be written as:

```
int number1, // first number to add
    number2, // second number to add
    sum; // sum of number1 and number2
```



Good Programming Practice 2.7

Declare each variable in its own declaration. This format allows a descriptive comment to be inserted next to each variable being declared.



Good Programming Practice 2.8

Choosing meaningful variable names helps a program to be self-documenting (i.e., one can understand the program simply by reading it rather than by reading associated documentation or creating and viewing an excessive number of comments).



Good Programming Practice 2.9

*By convention, variable-name identifiers begin with a lowercase letter, and every word in the name after the first word begins with a capital letter. For example, variable-name identifier `firstNumber` starts its second word, `Number`, with a capital N. This naming convention is known as *camel case*, because the uppercase letters stand out like a camel's humps.*

2.5.5 Prompting the User for Input

Line 17

```
System.out.print("Enter first integer: "); // prompt
```

uses `System.out.print` to display the message "Enter first integer: ". This message is called a **prompt** because it directs the user to take a specific action. We use method `print` here rather than `println` so that the user's input appears on the same line as the prompt. Recall from Section 2.2 that identifiers starting with capital letters typically represent class names. Class `System` is part of package `java.lang`. Notice that class `System` is *not* imported with an `import` declaration at the beginning of the program.



Software Engineering Observation 2.2

By default, package `java.lang` is imported in every Java program; thus, classes in `java.lang` are the only ones in the Java API that do not require an `import` declaration.

2.5.6 Obtaining an `int` as Input from the User

Line 18

```
number1 = input.nextInt(); // read first number from user
```

uses `Scanner` object `input`'s `nextInt` method to obtain an integer from the user at the keyboard. At this point the program *waits* for the user to type the number and press the *Enter* key to submit the number to the program.

Our program assumes that the user enters a valid integer value. If not, a runtime logic error will occur and the program will terminate. Chapter 11, *Exception Handling: A Deeper Look*, discusses how to make your programs more robust by enabling them to handle such errors. This is also known as making your program *fault tolerant*.

In line 18, we place the result of the call to method `nextInt` (an `int` value) in variable `number1` by using the **assignment operator**, `=`. The statement is read as "number1 gets the value of `input.nextInt()`." Operator `=` is called a **binary operator**, because it has *two operands*—`number1` and the result of the method call `input.nextInt()`. This statement is called an *assignment statement*, because it *assigns* a value to a variable. Everything to the *right* of the assignment operator, `=`, is always evaluated *before* the assignment is performed.



Good Programming Practice 2.10

Place spaces on either side of a binary operator for readability.

2.5.7 Prompting for and Inputting a Second int

Line 20

```
System.out.print("Enter second integer: "); // prompt
```

prompts the user to enter the second integer. Line 21

```
number2 = input.nextInt(); // read second number from user
```

reads the second integer and assigns it to variable `number2`.

2.5.8 Using Variables in a Calculation

Line 23

```
sum = number1 + number2; // add numbers then store total in sum
```

is an assignment statement that calculates the sum of the variables `number1` and `number2` then assigns the result to variable `sum` by using the assignment operator, `=`. The statement is read as “`sum` gets the value of `number1 + number2`.” When the program encounters the addition operation, it performs the calculation using the values stored in the variables `number1` and `number2`. In the preceding statement, the addition operator is a *binary operator*—its *two* operands are the variables `number1` and `number2`. Portions of statements that contain calculations are called **expressions**. In fact, an expression is any portion of a statement that has a *value* associated with it. For example, the value of the expression `number1 + number2` is the *sum* of the numbers. Similarly, the value of the expression `input.nextInt()` is the integer typed by the user.

2.5.9 Displaying the Result of the Calculation

After the calculation has been performed, line 25

```
System.out.printf("Sum is %d\n", sum); // display sum
```

uses method `System.out.printf` to display the sum. The format specifier **%d** is a *placeholder* for an `int` value (in this case the value of `sum`)—the letter `d` stands for “decimal integer.” The remaining characters in the format string are all fixed text. So, method `printf` displays “Sum is ”, followed by the value of `sum` (in the position of the `%d` format specifier) and a newline.

Calculations can also be performed *inside* `printf` statements. We could have combined the statements at lines 23 and 25 into the statement

```
System.out.printf("Sum is %d\n", (number1 + number2));
```

The parentheses around the expression `number1 + number2` are optional—they’re included to emphasize that the value of the *entire* expression is output in the position of the `%d` format specifier. Such parentheses are said to be **redundant**.

2.5.10 Java API Documentation

For each new Java API class we use, we indicate the package in which it’s located. This information helps you locate descriptions of each package and class in the Java API documentation. A web-based version of this documentation can be found at

```
http://docs.oracle.com/javase/7/docs/api/index.html
```

You can download it from the Additional Resources section at

<http://www.oracle.com/technetwork/java/javase/downloads/index.html>

Appendix F shows how to use this documentation.

2.6 Memory Concepts

Variable names such as `number1`, `number2` and `sum` actually correspond to *locations* in the computer's memory. Every variable has a **name**, a **type**, a **size** (in bytes) and a **value**.

In the addition program of Fig. 2.7, when the following statement (line 18) executes:

```
number1 = input.nextInt(); // read first number from user
```

the number typed by the user is placed into a memory location corresponding to the name `number1`. Suppose that the user enters 45. The computer places that integer value into location `number1` (Fig. 2.8), replacing the previous value (if any) in that location. The previous value is lost, so this process is said to be *destructive*.

number1	45
---------	----

Fig. 2.8 | Memory location showing the name and value of variable `number1`.

When the statement (line 21)

```
number2 = input.nextInt(); // read second number from user
```

executes, suppose that the user enters 72. The computer places that integer value into location `number2`. The memory now appears as shown in Fig. 2.9.

number1	45
---------	----

number2	72
---------	----

Fig. 2.9 | Memory locations after storing values for `number1` and `number2`.

After the program of Fig. 2.7 obtains values for `number1` and `number2`, it adds the values and places the total into variable `sum`. The statement (line 23)

```
sum = number1 + number2; // add numbers, then store total in sum
```

performs the addition, then replaces any previous value in `sum`. After `sum` has been calculated, memory appears as shown in Fig. 2.10. The values of `number1` and `number2` appear exactly as they did before they were used in the calculation of `sum`. These values were used, but *not* destroyed, as the computer performed the calculation. When a value is read from a memory location, the process is *nondestructive*.

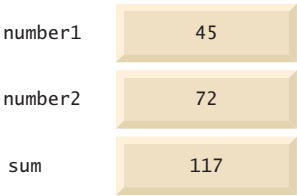


Fig. 2.10 | Memory locations after storing the sum of number1 and number2.

2.7 Arithmetic

Most programs perform arithmetic calculations. The **arithmetic operators** are summarized in Fig. 2.11. Note the use of various special symbols not used in algebra. The **asterisk** (*) indicates multiplication, and the percent sign (%) is the **remainder operator**, which we’ll discuss shortly. The arithmetic operators in Fig. 2.11 are *binary* operators, because each operates on *two* operands. For example, the expression `f + 7` contains the binary operator `+` and the two operands `f` and `7`.

Java operation	Operator	Algebraic expression	Java expression
Addition	<code>+</code>	$f + 7$	<code>f + 7</code>
Subtraction	<code>-</code>	$p - c$	<code>p - c</code>
Multiplication	<code>*</code>	bm	<code>b * m</code>
Division	<code>/</code>	x / y or $\frac{x}{y}$ or $x \div y$	<code>x / y</code>
Remainder	<code>%</code>	$r \bmod s$	<code>r % s</code>

Fig. 2.11 | Arithmetic operators.

Integer division yields an integer quotient. For example, the expression `7 / 4` evaluates to 1, and the expression `17 / 5` evaluates to 3. Any fractional part in integer division is simply *truncated* (i.e., *discarded*)—no *rounding* occurs. Java provides the remainder operator, `%`, which yields the remainder after division. The expression `x % y` yields the remainder after `x` is divided by `y`. Thus, `7 % 4` yields 3, and `17 % 5` yields 2. This operator is most commonly used with integer operands but it can also be used with other arithmetic types. In this chapter’s exercises and in later chapters, we consider several interesting applications of the remainder operator, such as determining whether one number is a multiple of another.

Arithmetic Expressions in Straight-Line Form

Arithmetic expressions in Java must be written in **straight-line form** to facilitate entering programs into the computer. Thus, expressions such as “a divided by b” must be written as `a / b`, so that all constants, variables and operators appear in a straight line. The following algebraic notation is generally not acceptable to compilers:

$\frac{a}{b}$

Parentheses for Grouping Subexpressions

Parentheses are used to group terms in Java expressions in the same manner as in algebraic expressions. For example, to multiply *a* times the quantity *b + c*, we write

```
a * (b + c)
```

If an expression contains **nested parentheses**, such as

```
((a + b) * c)
```

the expression in the *innermost* set of parentheses (*a + b* in this case) is evaluated *first*.

Rules of Operator Precedence

Java applies the operators in arithmetic expressions in a precise sequence determined by the **rules of operator precedence**, which are generally the same as those followed in algebra:

1. Multiplication, division and remainder operations are applied first. If an expression contains several such operations, they're applied from left to right. Multiplication, division and remainder operators have the same level of precedence.
2. Addition and subtraction operations are applied next. If an expression contains several such operations, the operators are applied from left to right. Addition and subtraction operators have the same level of precedence.

These rules enable Java to apply operators in the correct *order*.¹ When we say that operators are applied from left to right, we're referring to their **associativity**. Some operators associate from right to left. Figure 2.12 summarizes these rules of operator precedence. A complete precedence chart is included in Appendix A.

Operator(s)	Operation(s)	Order of evaluation (precedence)
* / %	Multiplication Division Remainder	Evaluated first. If there are several operators of this type, they're evaluated from <i>left to right</i> .
+ -	Addition Subtraction	Evaluated next. If there are several operators of this type, they're evaluated from <i>left to right</i> .
=	Assignment	Evaluated last.

Fig. 2.12 | Precedence of arithmetic operators.

Sample Algebraic and Java Expressions

Now let's consider several expressions in light of the rules of operator precedence. Each example lists an algebraic expression and its Java equivalent. The following is an example of an arithmetic mean (average) of five terms:

1. We use simple examples to explain the *order of evaluation* of expressions. Subtle issues occur in the more complex expressions you'll encounter later in the book. For more information on order of evaluation, see Chapter 15 of *The Java™ Language Specification* (<http://docs.oracle.com/javase/specs/jls/se7/html/index.html>).

Algebra: $m = \frac{a + b + c + d + e}{5}$
 Java: `m = (a + b + c + d + e) / 5;`

The parentheses are required because division has higher precedence than addition. The entire quantity $(a + b + c + d + e)$ is to be divided by 5. If the parentheses are erroneously omitted, we obtain $a + b + c + d + e / 5$, which evaluates as

$$a + b + c + d + \frac{e}{5}$$

Here's an example of the equation of a straight line:

Algebra: $y = mx + b$
 Java: `y = m * x + b;`

No parentheses are required. The multiplication operator is applied first because multiplication has a higher precedence than addition. The assignment occurs last because it has a lower precedence than multiplication or addition.

The following example contains remainder (%), multiplication, division, addition and subtraction operations:

Algebra: $z = pr \% q + w / x - y$
 Java: `z = p * r \% q + w / x - y;`

The circled numbers under the statement indicate the *order* in which Java applies the operators. The *, % and / operations are evaluated first in *left-to-right* order (i.e., they associate from left to right), because they have higher precedence than + and -. The + and - operations are evaluated next. These operations are also applied from *left to right*. The assignment (=) operation is evaluated last.

Evaluation of a Second-Degree Polynomial

To develop a better understanding of the rules of operator precedence, consider the evaluation of an assignment expression that includes a second-degree polynomial $ax^2 + bx + c$:

`y = a * x * x + b * x + c;`

The multiplication operations are evaluated first in left-to-right order (i.e., they associate from left to right), because they have higher precedence than addition. (Java has no arithmetic operator for exponentiation, so x^2 is represented as $x * x$. Section 5.4 shows an alternative for performing exponentiation.) The addition operations are evaluated next from *left to right*. Suppose that a , b , c and x are initialized (given values) as follows: $a = 2$, $b = 3$, $c = 7$ and $x = 5$. Figure 2.13 illustrates the order in which the operators are applied.

You can use *redundant parentheses* (unnecessary parentheses) to make an expression clearer. For example, the preceding statement might be parenthesized as follows:

`y = (a * x * x) + (b * x) + c;`

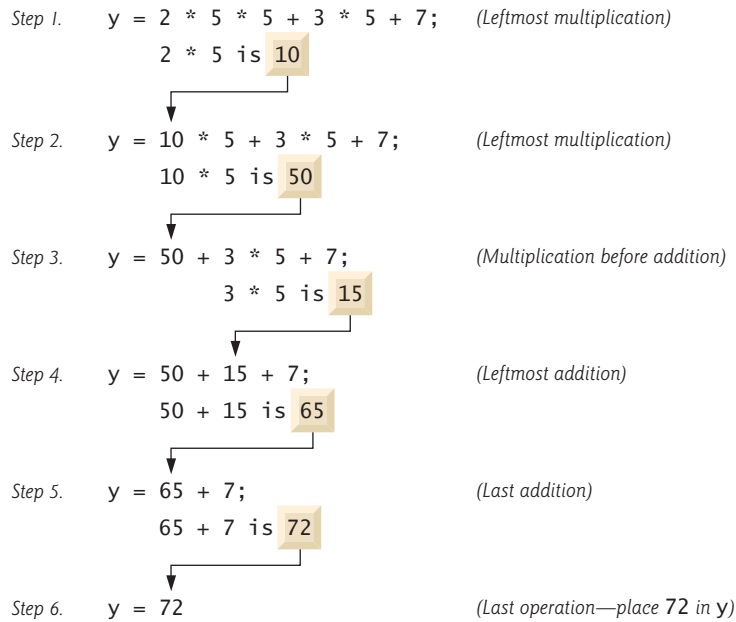


Fig. 2.13 | Order in which a second-degree polynomial is evaluated.

2.8 Decision Making: Equality and Relational Operators

A **condition** is an expression that can be **true** or **false**. This section introduces Java’s **if selection statement**, which allows a program to make a **decision** based on a condition’s value. For example, the condition “grade is greater than or equal to 60” determines whether a student passed a test. If the condition in an **if** statement is *true*, the body of the **if** statement executes. If the condition is *false*, the body does not execute. We’ll see an example shortly.

Conditions in **if** statements can be formed by using the **equality operators** (`==` and `!=`) and **relational operators** (`>`, `<`, `>=` and `<=`) summarized in Fig. 2.14. Both equality operators have the same level of precedence, which is *lower* than that of the relational operators. The equality operators associate from *left to right*. The relational operators all have the same level of precedence and also associate from *left to right*.

Algebraic operator	Java equality or relational operator	Sample Java condition	Meaning of Java condition
<i>Equality operators</i>			
<code>=</code>	<code>==</code>	<code>x == y</code>	x is equal to y
<code>1</code>	<code>!=</code>	<code>x != y</code>	x is not equal to y

Fig. 2.14 | Equality and relational operators. (Part I of 2.)

Algebraic operator	Java equality or relational operator	Sample Java condition	Meaning of Java condition
<i>Relational operators</i>			
>	>	x > y	x is greater than y
<	<	x < y	x is less than y
≥	>=	x >= y	x is greater than or equal to y
≤	<=	x <= y	x is less than or equal to y

Fig. 2.14 | Equality and relational operators. (Part 2 of 2.)

Figure 2.15 uses six `if` statements to compare two integers input by the user. If the condition in any of these `if` statements is *true*, the statement associated with that `if` statement executes; otherwise, the statement is skipped. We use a `Scanner` to input the integers from the user and store them in variables `number1` and `number2`. The program *compares* the numbers and displays the results of the comparisons that are true.

```

1 // Fig. 2.15: Comparison.java
2 // Compare integers using if statements, relational operators
3 // and equality operators.
4 import java.util.Scanner; // program uses class Scanner
5
6 public class Comparison
7 {
8     // main method begins execution of Java application
9     public static void main(String[] args)
10    {
11        // create Scanner to obtain input from command line
12        Scanner input = new Scanner(System.in);
13
14        int number1; // first number to compare
15        int number2; // second number to compare
16
17        System.out.print("Enter first integer: "); // prompt
18        number1 = input.nextInt(); // read first number from user
19
20        System.out.print("Enter second integer: "); // prompt
21        number2 = input.nextInt(); // read second number from user
22
23        if (number1 == number2)
24            System.out.printf("%d == %d\n", number1, number2);
25
26        if (number1 != number2)
27            System.out.printf("%d != %d\n", number1, number2);
28
29        if (number1 < number2)
30            System.out.printf("%d < %d\n", number1, number2);

```

Fig. 2.15 | Compare integers using `if` statements, relational operators and equality operators. (Part 1 of 2.)

```

31
32     if (number1 > number2)
33         System.out.printf("%d > %d\n", number1, number2);
34
35     if (number1 <= number2)
36         System.out.printf("%d <= %d\n", number1, number2);
37
38     if (number1 >= number2)
39         System.out.printf("%d >= %d\n", number1, number2);
40 } // end method main
41 } // end class Comparison

```

```

Enter first integer: 777
Enter second integer: 777
777 == 777
777 <= 777
777 >= 777

```

```

Enter first integer: 1000
Enter second integer: 2000
1000 != 2000
1000 < 2000
1000 <= 2000

```

```

Enter first integer: 2000
Enter second integer: 1000
2000 != 1000
2000 > 1000
2000 >= 1000

```

Fig. 2.15 | Compare integers using if statements, relational operators and equality operators.
(Part 2 of 2.)

The declaration of class Comparison begins at line 6

```
public class Comparison
```

The class's main method (lines 9–40) begins the execution of the program. Line 12

```
Scanner input = new Scanner(System.in);
```

declares Scanner variable input and assigns it a Scanner that inputs data from the standard input (i.e., the keyboard).

Lines 14–15

```
int number1; // first number to compare
int number2; // second number to compare
```

declare the int variables used to store the values input from the user.

Lines 17–18

```
System.out.print("Enter first integer: "); // prompt
number1 = input.nextInt(); // read first number from user

```

prompt the user to enter the first integer and input the value, respectively. The value is stored in variable `number1`.

Lines 20–21

```
System.out.print("Enter second integer: "); // prompt
number2 = input.nextInt(); // read second number from user
```

prompt the user to enter the second integer and input the value, respectively. The value is stored in variable `number2`.

Lines 23–24

```
if (number1 == number2)
    System.out.printf("%d == %d\n", number1, number2);
```

compare the values of `number1` and `number2` to determine whether they're equal. An `if` statement always begins with keyword `if`, followed by a condition in parentheses. An `if` statement expects one statement in its body, but may contain multiple statements if they're enclosed in a set of braces (`{}`). The indentation of the body statement shown here is not required, but it improves the program's readability by emphasizing that the statement in line 24 *is part of* the `if` statement that begins at line 23. Line 24 executes only if the numbers stored in variables `number1` and `number2` are equal (i.e., the condition is *true*). The `if` statements in lines 26–27, 29–30, 32–33, 35–36 and 38–39 compare `number1` and `number2` using the operators `!=`, `<`, `>`, `<=` and `>=`, respectively. If the condition in one or more of the `if` statements is *true*, the corresponding body statement executes.



Common Programming Error 2.9

Confusing the equality operator, `==`, with the assignment operator, `=`, can cause a logic error or a compilation error. The equality operator should be read as “is equal to” and the assignment operator as “gets” or “gets the value of.” To avoid confusion, some people read the equality operator as “double equals” or “equals equals.”



Good Programming Practice 2.11

Place only one statement per line in a program for readability.

There's no semicolon (`;`) at the end of the first line of each `if` statement. Such a semicolon would result in a logic error at execution time. For example,

```
if (number1 == number2); // logic error
    System.out.printf("%d == %d\n", number1, number2);
```

would actually be interpreted by Java as

```
if (number1 == number2)
    ; // empty statement
    System.out.printf("%d == %d\n", number1, number2);
```

where the semicolon on the line by itself—called the **empty statement**—is the statement to execute if the condition in the `if` statement is *true*. When the empty statement executes, no task is performed. The program then continues with the output statement, which *always* executes, *regardless* of whether the condition is true or false, because the output statement is *not* part of the `if` statement.

White space

Note the use of white space in Fig. 2.15. Recall that the compiler normally ignores white space. So, statements may be split over several lines and may be spaced according to your preferences without affecting a program's meaning. It's incorrect to split identifiers and strings. Ideally, statements should be kept small, but this is not always possible.



Error-Prevention Tip 2.5

A lengthy statement can be spread over several lines. If a single statement must be split across lines, choose natural breaking points, such as after a comma in a comma-separated list, or after an operator in a lengthy expression. If a statement is split across two or more lines, indent all subsequent lines until the end of the statement.

Operators Discussed So Far

Figure 2.16 shows the operators discussed so far in decreasing order of precedence. All but the assignment operator, `=`, associate from *left to right*. The assignment operator, `=`, associates from *right to left*. An assignment expression's value is whatever was assigned to the variable on the `=` operator's left side—for example, the value of the expression `x = 7` is 7. So an expression like `x = y = 0` is evaluated as if it had been written as `x = (y = 0)`, which first assigns the value 0 to variable `y`, then assigns the result of that assignment, 0, to `x`.

Operators	Associativity	Type
* / %	left to right	multiplicative
+ -	left to right	additive
< <= > >=	left to right	relational
== !=	left to right	equality
=	right to left	assignment

Fig. 2.16 | Precedence and associativity of operators discussed.



Good Programming Practice 2.12

When writing expressions containing many operators, refer to the operator precedence chart (Appendix A). Confirm that the operations in the expression are performed in the order you expect. If, in a complex expression, you're uncertain about the order of evaluation, use parentheses to force the order, exactly as you'd do in algebraic expressions.

2.9 Wrap-Up

In this chapter, you learned many important features of Java, including displaying data on the screen in a **Command Prompt**, inputting data from the keyboard, performing calculations and making decisions. The applications presented here introduced you to many basic programming concepts. As you'll see in Chapter 3, Java applications typically contain just a few lines of code in method `main`—these statements normally create the objects that perform the work of the application. In Chapter 3, you'll learn how to implement your own classes and use objects of those classes in applications.

Summary

Section 2.2 Your First Program in Java: Printing a Line of Text

- A Java application (p. 35) executes when you use the `java` command to launch the JVM.
- Comments (p. 36) document programs and improve their readability. The compiler ignores them.
- A comment that begins with `//` is an end-of-line comment—it terminates at the end of the line on which it appears.
- Traditional comments (p. 36) can be spread over several lines and are delimited by `/*` and `*/`.
- Javadoc comments (p. 36), delimited by `/**` and `*/`, enable you to embed program documentation in your code. The javadoc utility program generates HTML pages based on these comments.
- A syntax error (p. 36; also called a compiler error, compile-time error or compilation error) occurs when the compiler encounters code that violates Java’s language rules. It’s similar to a grammar error in a natural language.
- Blank lines, space characters and tab characters are known as white space (p. 37). White space makes programs easier to read and is ignored by the compiler.
- Keywords (p. 37) are reserved for use by Java and are always spelled with all lowercase letters.
- Keyword `class` (p. 37) introduces a class declaration.
- By convention, all class names in Java begin with a capital letter and capitalize the first letter of each word they include (e.g., `SampleClassName`).
- A Java class name is an identifier—a series of characters consisting of letters, digits, underscores (`_`) and dollar signs (`$`) that does not begin with a digit and does not contain spaces.
- Java is case sensitive (p. 38)—that is, uppercase and lowercase letters are distinct.
- The body of every class declaration (p. 38) is delimited by braces, `{` and `}`.
- A `public` (p. 37) class declaration must be saved in a file with the same name as the class followed by the `“.java”` file-name extension.
- Method `main` (p. 38) is the starting point of every Java application and must begin with

```
public static void main(String[] args)
```

otherwise, the JVM will not execute the application.

- Methods perform tasks and return information when they complete them. Keyword `void` (p. 38) indicates that a method will perform a task but return no information.
- Statements instruct the computer to perform actions.
- A string (p. 39) in double quotes is sometimes called a character string or a string literal.
- The standard output object (`System.out`; p. 39) displays characters in the command window.
- Method `System.out.println` (p. 39) displays its argument (p. 39) in the command window followed by a newline character to position the output cursor to the beginning of the next line.
- You compile a program with the command `javac`. If the program contains no syntax errors, a class file (p. 40) containing the Java bytecodes that represent the application is created. These bytecodes are interpreted by the JVM when you execute the program.
- To run an application, type `java` followed by the name of the class that contains the `main` method.

Section 2.3 Modifying Your First Java Program

- `System.out.print` (p. 42) displays its argument and positions the output cursor immediately after the last character displayed.
- A backslash (`\`) in a string is an escape character (p. 43). Java combines it with the next character to form an escape sequence (p. 43). The escape sequence `\n` (p. 43) represents the newline character.

Section 2.4 Displaying Text with `printf`

- `System.out.printf` method (p. 43; `f` means “formatted”) displays formatted data.
- Method `printf`’s first argument is a format string (p. 44) containing fixed text and/or format specifiers. Each format specifier (p. 44) indicates the type of data to output and is a placeholder for a corresponding argument that appears after the format string.
- Format specifiers begin with a percent sign (%) and are followed by a character that represents the data type. The format specifier `%s` (p. 44) is a placeholder for a string.
- The `%n` format specifier (p. 44) is a portable line separator. You cannot use `%n` in the argument to `System.out.print` or `System.out.println`; however, the line separator output by `System.out.println` after it displays its argument is portable across operating systems.

Section 2.5 Another Application: Adding Integers

- An `import` declaration (p. 46) helps the compiler locate a class that’s used in a program.
- Java’s rich set of predefined classes are grouped into packages (p. 45)—named groups of classes. These are referred to as the Java class library (p. 46), or the Java Application Programming Interface (Java API).
- A variable (p. 46) is a location in the computer’s memory where a value can be stored for use later in a program. All variables must be declared with a name and a type before they can be used.
- A variable’s name enables the program to access the variable’s value in memory.
- A `Scanner` (package `java.util`; p. 47) enables a program to read data that the program will use. Before a `Scanner` can be used, the program must create it and specify the source of the data.
- Variables should be initialized (p. 47) to prepare them for use in a program.
- The expression `new Scanner(System.in)` creates a `Scanner` that reads from the standard input object (`System.in`; p. 47)—normally the keyboard.
- Data type `int` (p. 47) is used to declare variables that will hold integer values. The range of values for an `int` is $-2,147,483,648$ to $+2,147,483,647$.
- Types `float` and `double` (p. 47) specify real numbers with decimal points, such as `3.4` and `-11.19`.
- Variables of type `char` (p. 47) represent individual characters, such as an uppercase letter (e.g., `A`), a digit (e.g., `7`), a special character (e.g., `*` or `%`) or an escape sequence (e.g., `tab`, `\t`).
- Types such as `int`, `float`, `double` and `char` are primitive types (p. 47). Primitive-type names are keywords; thus, they must appear in all lowercase letters.
- A prompt (p. 48) directs the user to take a specific action.
- `Scanner` method `nextInt` obtains an integer for use in a program.
- The assignment operator, `=` (p. 48), enables the program to give a value to a variable. It’s called a binary operator (p. 48) because it has two operands.
- Portions of statements that have values are called expressions (p. 49).
- The format specifier `%d` (p. 49) is a placeholder for an `int` value.

Section 2.6 Memory Concepts

- Variable names (p. 50) correspond to locations in the computer’s memory. Every variable has a name, a type, a size and a value.
- A value that’s placed in a memory location replaces the location’s previous value, which is lost.

Section 2.7 Arithmetic

- The arithmetic operators (p. 51) are `+` (addition), `-` (subtraction), `*` (multiplication), `/` (division) and `%` (remainder).

- Integer division (p. 51) yields an integer quotient.
- The remainder operator, % (p. 51), yields the remainder after division.
- Arithmetic expressions must be written in straight-line form (p. 51).
- If an expression contains nested parentheses (p. 52), the innermost set is evaluated first.
- Java applies the operators in arithmetic expressions in a precise sequence determined by the rules of operator precedence (p. 52).
- When we say that operators are applied from left to right, we're referring to their associativity (p. 52). Some operators associate from right to left.
- Redundant parentheses (p. 53) can make an expression clearer.

Section 2.8 Decision Making: Equality and Relational Operators

- The if statement (p. 54) makes a decision based on a condition's value (true or false).
- Conditions in if statements can be formed by using the equality (== and !=) and relational (>, <, >= and <=) operators (p. 54).
- An if statement begins with keyword if followed by a condition in parentheses and expects one statement in its body.
- The empty statement (p. 57) is a statement that does not perform a task.

Self-Review Exercises

2.1 Fill in the blanks in each of the following statements:

- A(n) _____ begins the body of every method, and a(n) _____ ends the body of every method.
- You can use the _____ statement to make decisions.
- _____ begins an end-of-line comment.
- _____, _____ and _____ are called white space.
- _____ are reserved for use by Java.
- Java applications begin execution at method _____.
- Methods _____, _____ and _____ display information in a command window.

2.2 State whether each of the following is *true* or *false*. If *false*, explain why.

- Comments cause the computer to print the text after the // on the screen when the program executes.
- All variables must be given a type when they're declared.
- Java considers the variables number and NuMbEr to be identical.
- The remainder operator (%) can be used only with integer operands.
- The arithmetic operators *, /, %, + and - all have the same level of precedence.

2.3 Write statements to accomplish each of the following tasks:

- Declare variables c, thisIsAVariable, q76354 and number to be of type int.
- Prompt the user to enter an integer.
- Input an integer and assign the result to int variable value. Assume Scanner variable input can be used to read a value from the keyboard.
- Print "This is a Java program" on one line in the command window. Use method System.out.println.
- Print "This is a Java program" on two lines in the command window. The first line should end with Java. Use method System.out.printf and two %s format specifiers.
- If the variable number is not equal to 7, display "The variable number is not equal to 7".

- 2.4** Identify and correct the errors in each of the following statements:
- a) `if (c < 7);`
`System.out.println("c is less than 7");`
 - b) `if (c => 7)`
`System.out.println("c is equal to or greater than 7");`
- 2.5** Write declarations, statements or comments that accomplish each of the following tasks:
- a) State that a program will calculate the product of three integers.
 - b) Create a Scanner called `input` that reads values from the standard input.
 - c) Declare the variables `x`, `y`, `z` and `result` to be of type `int`.
 - d) Prompt the user to enter the first integer.
 - e) Read the first integer from the user and store it in the variable `x`.
 - f) Prompt the user to enter the second integer.
 - g) Read the second integer from the user and store it in the variable `y`.
 - h) Prompt the user to enter the third integer.
 - i) Read the third integer from the user and store it in the variable `z`.
 - j) Compute the product of the three integers contained in variables `x`, `y` and `z`, and assign the result to the variable `result`.
 - k) Use `System.out.printf` to display the message "Product is" followed by the value of the variable `result`.
- 2.6** Using the statements you wrote in Exercise 2.5, write a complete program that calculates and prints the product of three integers.

Answers to Self-Review Exercises

- 2.1** a) left brace (`{`), right brace (`}`). b) `if`. c) `//`. d) Space characters, newlines and tabs. e) Keywords. f) `main`. g) `System.out.print`, `System.out.println` and `System.out.printf`.
- 2.2** a) False. Comments do not cause any action to be performed when the program executes. They're used to document programs and improve their readability.
 b) True.
 c) False. Java is case sensitive, so these variables are distinct.
 d) False. The remainder operator can also be used with noninteger operands in Java.
 e) False. The operators `*`, `/` and `%` are higher precedence than operators `+` and `-`.
- 2.3** a) `int c, thisIsAVariable, q76354, number;`
 or
`int c;`
`int thisIsAVariable;`
`int q76354;`
`int number;`
 b) `System.out.print("Enter an integer: ");`
 c) `value = input.nextInt();`
 d) `System.out.println("This is a Java program");`
 e) `System.out.printf("%s\n%s\n", "This is a Java", "program");`
 f) `if (number != 7)`
`System.out.println("The variable number is not equal to 7");`
- 2.4** a) Error: Semicolon after the right parenthesis of the condition (`c < 7`) in the `if`.
 Correction: Remove the semicolon after the right parenthesis. [Note: As a result, the output statement will execute regardless of whether the condition in the `if` is true.]
 b) Error: The relational operator `=>` is incorrect. Correction: Change `=>` to `>=`.

- 2.5**
- a) `// Calculate the product of three integers`
 - b) `Scanner input = new Scanner(System.in);`
 - c) `int x, y, z, result;`
or
`int x;`
`int y;`
`int z;`
`int result;`
 - d) `System.out.print("Enter first integer: ");`
 - e) `x = input.nextInt();`
 - f) `System.out.print("Enter second integer: ");`
 - g) `y = input.nextInt();`
 - h) `System.out.print("Enter third integer: ");`
 - i) `z = input.nextInt();`
 - j) `result = x * y * z;`
 - k) `System.out.printf("Product is %d\n", result);`

2.6 The solution to Self-Review Exercise 2.6 is as follows:

```

1  // Ex. 2.6: Product.java
2  // Calculate the product of three integers.
3  import java.util.Scanner; // program uses Scanner
4
5  public class Product
6  {
7      public static void main(String[] args)
8      {
9          // create Scanner to obtain input from command window
10         Scanner input = new Scanner(System.in);
11
12         int x; // first number input by user
13         int y; // second number input by user
14         int z; // third number input by user
15         int result; // product of numbers
16
17         System.out.print("Enter first integer: "); // prompt for input
18         x = input.nextInt(); // read first integer
19
20         System.out.print("Enter second integer: "); // prompt for input
21         y = input.nextInt(); // read second integer
22
23         System.out.print("Enter third integer: "); // prompt for input
24         z = input.nextInt(); // read third integer
25
26         result = x * y * z; // calculate product of numbers
27
28         System.out.printf("Product is %d\n", result);
29     } // end method main
30 } // end class Product

```

```

Enter first integer: 10
Enter second integer: 20
Enter third integer: 30
Product is 6000

```

Exercises

- 2.7** Fill in the blanks in each of the following statements:
- _____ are used to document a program and improve its readability.
 - A decision can be made in a Java program with a(n) _____.
 - Calculations are normally performed by _____ statements.
 - The arithmetic operators with the same precedence as multiplication are _____ and _____.
 - When parentheses in an arithmetic expression are nested, the _____ set of parentheses is evaluated first.
 - A location in the computer's memory that may contain different values at various times throughout the execution of a program is called a(n) _____.
- 2.8** Write Java statements that accomplish each of the following tasks:
- Display the message "Enter an integer: ", leaving the cursor on the same line.
 - Assign the product of variables *b* and *c* to variable *a*.
 - Use a comment to state that a program performs a sample payroll calculation.
- 2.9** State whether each of the following is *true* or *false*. If *false*, explain why.
- Java operators are evaluated from left to right.
 - The following are all valid variable names: `_under_bar_`, `m928134`, `t5`, `j7`, `her_sales$`, `his_$account_total`, `a`, `b$`, `c`, `z` and `z2`.
 - A valid Java arithmetic expression with no parentheses is evaluated from left to right.
 - The following are all invalid variable names: `3g`, `87`, `67h2`, `h22` and `2h`.
- 2.10** Assuming that $x = 2$ and $y = 3$, what does each of the following statements display?
- `System.out.printf("x = %d\n", x);`
 - `System.out.printf("Value of %d + %d is %d\n", x, x, (x + x));`
 - `System.out.printf("x =");`
 - `System.out.printf("%d = %d\n", (x + y), (y + x));`
- 2.11** Which of the following Java statements contain variables whose values are modified?
- `p = i + j + k + 7;`
 - `System.out.println("variables whose values are modified");`
 - `System.out.println("a = 5");`
 - `value = input.nextInt();`
- 2.12** Given that $y = ax^3 + 7$, which of the following are correct Java statements for this equation?
- `y = a * x * x * x + 7;`
 - `y = a * x * x * (x + 7);`
 - `y = (a * x) * x * (x + 7);`
 - `y = (a * x) * x * x + 7;`
 - `y = a * (x * x * x) + 7;`
 - `y = a * x * (x * x + 7);`
- 2.13** State the order of evaluation of the operators in each of the following Java statements, and show the value of *x* after each statement is performed:
- `x = 7 + 3 * 6 / 2 - 1;`
 - `x = 2 % 2 + 2 * 2 - 2 / 2;`
 - `x = (3 * 9 * (3 + (9 * 3 / (3))));`
- 2.14** Write an application that displays the numbers 1 to 4 on the same line, with each pair of adjacent numbers separated by one space. Use the following techniques:
- Use one `System.out.println` statement.
 - Use four `System.out.print` statements.
 - Use one `System.out.printf` statement.

2.15 (*Arithmetic*) Write an application that asks the user to enter two integers, obtains them from the user and prints their sum, product, difference and quotient (division). Use the techniques shown in Fig. 2.7.

2.16 (*Comparing Integers*) Write an application that asks the user to enter two integers, obtains them from the user and displays the larger number followed by the words "is larger". If the numbers are equal, print the message "These numbers are equal". Use the techniques shown in Fig. 2.15.

2.17 (*Arithmetic, Smallest and Largest*) Write an application that inputs three integers from the user and displays the sum, average, product, smallest and largest of the numbers. Use the techniques shown in Fig. 2.15. [Note: The calculation of the average in this exercise should result in an integer representation of the average. So, if the sum of the values is 7, the average should be 2, not 2.3333....]

2.18 (*Displaying Shapes with Asterisks*) Write an application that displays a box, an oval, an arrow and a diamond using asterisks (*), as follows:

```

*****      ***      *      *
*          *  *      *      ***      *  *
*          *  *      *      *****      *  *
*          *  *      *      *      *      *
*          *  *      *      *      *      *
*          *  *      *      *      *      *
*          *  *      *      *      *      *
*          *  *      *      *      *      *
*****      ***      *      *

```

2.19 What does the following code print?

```
System.out.printf("%n**%n***%n****%n*****%n");
```

2.20 What does the following code print?

```
System.out.println("*");
System.out.println("***");
System.out.println("*****");
System.out.println("*****");
System.out.println("**");
```

2.21 What does the following code print?

```
System.out.print("*");
System.out.print("***");
System.out.print("*****");
System.out.print("*****");
System.out.println("**");
```

2.22 What does the following code print?

```
System.out.print("*");
System.out.println("***");
System.out.println("*****");
System.out.print("*****");
System.out.println("**");
```

2.23 What does the following code print?

```
System.out.printf("%s\n%s\n%s\n", "*", "***", "*****");
```

2.24 (*Largest and Smallest Integers*) Write an application that reads five integers and determines and prints the largest and smallest integers in the group. Use only the programming techniques you learned in this chapter.

2.25 (Odd or Even) Write an application that reads an integer and determines and prints whether it's odd or even. [Hint: Use the remainder operator. An even number is a multiple of 2. Any multiple of 2 leaves a remainder of 0 when divided by 2.]

2.26 (Multiples) Write an application that reads two integers, determines whether the first is a multiple of the second and prints the result. [Hint: Use the remainder operator.]

2.27 (Checkerboard Pattern of Asterisks) Write an application that displays a checkerboard pattern, as follows:

```
* * * * *
 * * * * *
* * * * *
 * * * * *
* * * * *
 * * * * *
* * * * *
 * * * * *
```

2.28 (Diameter, Circumference and Area of a Circle) Here's a peek ahead. In this chapter, you learned about integers and the type `int`. Java can also represent floating-point numbers that contain decimal points, such as 3.14159. Write an application that inputs from the user the radius of a circle as an integer and prints the circle's diameter, circumference and area using the floating-point value 3.14159 for π . Use the techniques shown in Fig. 2.7. [Note: You may also use the predefined constant `Math.PI` for the value of π . This constant is more precise than the value 3.14159. Class `Math` is defined in package `java.lang`. Classes in that package are imported automatically, so you do not need to `import class Math` to use it.] Use the following formulas (r is the radius):

$$\begin{aligned} \text{diameter} &= 2r \\ \text{circumference} &= 2\pi r \\ \text{area} &= \pi r^2 \end{aligned}$$

Do not store the results of each calculation in a variable. Rather, specify each calculation as the value that will be output in a `System.out.printf` statement. The values produced by the circumference and area calculations are floating-point numbers. Such values can be output with the format specifier `%f` in a `System.out.printf` statement. You'll learn more about floating-point numbers in Chapter 3.

2.29 (Integer Value of a Character) Here's another peek ahead. In this chapter, you learned about integers and the type `int`. Java can also represent uppercase letters, lowercase letters and a considerable variety of special symbols. Every character has a corresponding integer representation. The set of characters a computer uses together with the corresponding integer representations for those characters is called that computer's character set. You can indicate a character value in a program simply by enclosing that character in single quotes, as in `'A'`.

You can determine a character's integer equivalent by preceding that character with `(int)`, as in

```
(int) 'A'
```

An operator of this form is called a cast operator. (You'll learn about cast operators in Chapter 4.) The following statement outputs a character and its integer equivalent:

```
System.out.printf("The character %c has the value %d\n", 'A', ((int) 'A'));
```

When the preceding statement executes, it displays the character `A` and the value 65 (from the Unicode® character set) as part of the string. The format specifier `%c` is a placeholder for a character (in this case, the character `'A'`).

Using statements similar to the one shown earlier in this exercise, write an application that displays the integer equivalents of some uppercase letters, lowercase letters, digits and special symbols. Display the integer equivalents of the following: `A B C a b c 0 1 2 $ * + /` and the blank character.

2.30 (*Separating the Digits in an Integer*) Write an application that inputs one number consisting of five digits from the user, separates the number into its individual digits and prints the digits separated from one another by three spaces each. For example, if the user types in the number 42339, the program should print

```
4   2   3   3   9
```

Assume that the user enters the correct number of digits. What happens when you enter a number with more than five digits? What happens when you enter a number with fewer than five digits? [Hint: It's possible to do this exercise with the techniques you learned in this chapter. You'll need to use both division and remainder operations to "pick off" each digit.]

2.31 (*Table of Squares and Cubes*) Using only the programming techniques you learned in this chapter, write an application that calculates the squares and cubes of the numbers from 0 to 10 and prints the resulting values in table format, as shown below.

number	square	cube
0	0	0
1	1	1
2	4	8
3	9	27
4	16	64
5	25	125
6	36	216
7	49	343
8	64	512
9	81	729
10	100	1000

2.32 (*Negative, Positive and Zero Values*) Write a program that inputs five numbers and determines and prints the number of negative numbers input, the number of positive numbers input and the number of zeros input.

Making a Difference

2.33 (*Body Mass Index Calculator*) We introduced the body mass index (BMI) calculator in Exercise 1.10. The formulas for calculating BMI are

$$BMI = \frac{weightInPounds \times 703}{heightInInches \times heightInInches}$$

or

$$BMI = \frac{weightInKilograms}{heightInMeters \times heightInMeters}$$

Create a BMI calculator that reads the user's weight in pounds and height in inches (or, if you prefer, the user's weight in kilograms and height in meters), then calculates and displays the user's body mass index. Also, display the following information from the Department of Health and Human Services/National Institutes of Health so the user can evaluate his/her BMI:

```
BMI VALUES
Underweight: less than 18.5
Normal:      between 18.5 and 24.9
Overweight:  between 25 and 29.9
Obese:       30 or greater
```

[*Note:* In this chapter, you learned to use the `int` type to represent whole numbers. The BMI calculations when done with `int` values will both produce whole-number results. In Chapter 3 you'll learn to use the `double` type to represent numbers with decimal points. When the BMI calculations are performed with `doubles`, they'll both produce numbers with decimal points—these are called “floating-point” numbers.]

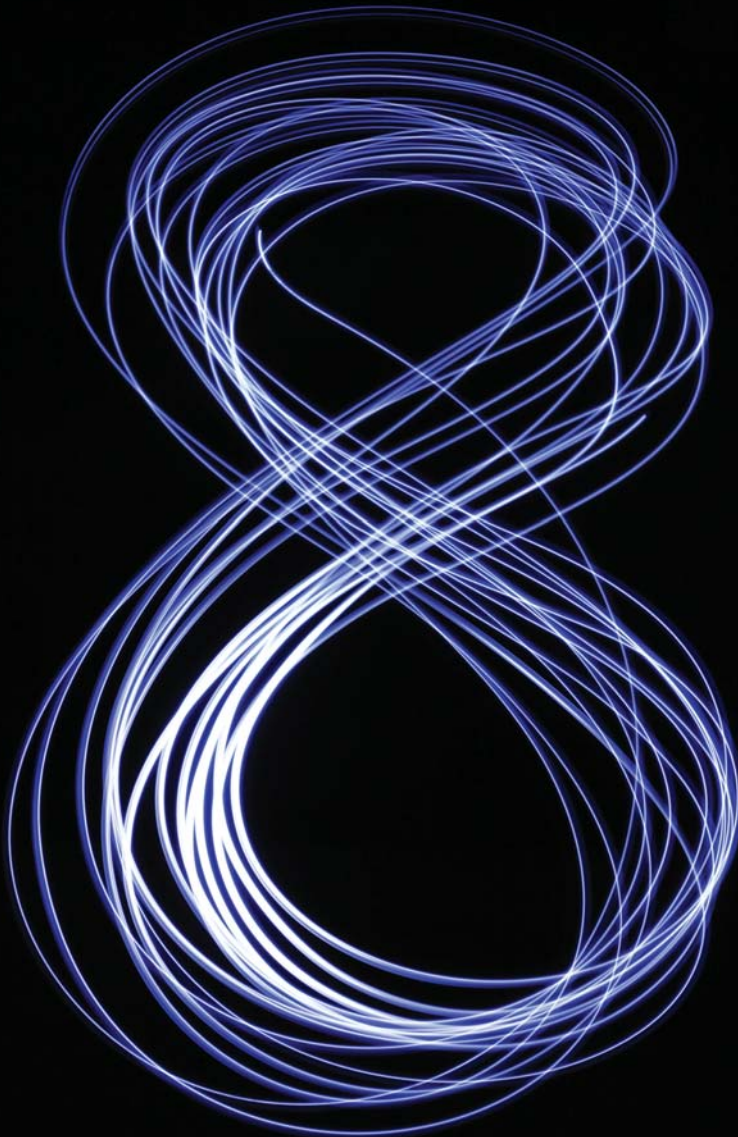
2.34 (*World Population Growth Calculator*) Use the web to determine the current world population and the annual world population growth rate. Write an application that inputs these values, then displays the estimated world population after one, two, three, four and five years.

2.35 (*Car-Pool Savings Calculator*) Research several car-pooling websites. Create an application that calculates your daily driving cost, so that you can estimate how much money could be saved by car pooling, which also has other advantages such as reducing carbon emissions and reducing traffic congestion. The application should input the following information and display the user's cost per day of driving to work:

- a) Total miles driven per day.
- b) Cost per gallon of gasoline.
- c) Average miles per gallon.
- d) Parking fees per day.
- e) Tolls per day.

Introduction to Classes, Objects, Methods and Strings

3



Your public servants serve you right.

—Adlai E. Stevenson

Nothing can have value without being an object of utility.

—Karl Marx

Objectives

In this chapter you'll learn:

- How to declare a class and use it to create an object.
- How to implement a class's behaviors as methods.
- How to implement a class's attributes as instance variables.
- How to call an object's methods to make them perform their tasks.
- What local variables of a method are and how they differ from instance variables.
- What primitive types and reference types are.
- How to use a constructor to initialize an object's data.
- How to represent and use numbers containing decimal points.

- 3.1 Introduction
- 3.2 Instance Variables, *set* Methods and *get* Methods
 - 3.2.1 Account Class with an Instance Variable, a *set* Method and a *get* Method
 - 3.2.2 AccountTest Class That Creates and Uses an Object of Class Account
 - 3.2.3 Compiling and Executing an App with Multiple Classes
 - 3.2.4 Account UML Class Diagram with an Instance Variable and *set* and *get* Methods
 - 3.2.5 Additional Notes on Class AccountTest
 - 3.2.6 Software Engineering with *private* Instance Variables and *public set* and *get* Methods
- 3.3 Primitive Types vs. Reference Types
- 3.4 Account Class: Initializing Objects with Constructors
 - 3.4.1 Declaring an Account Constructor for Custom Object Initialization
 - 3.4.2 Class AccountTest: Initializing Account Objects When They're Created
- 3.5 Account Class with a Balance; Floating-Point Numbers
 - 3.5.1 Account Class with a *balance* Instance Variable of Type `double`
 - 3.5.2 AccountTest Class to Use Class Account
- 3.6 (Optional) GUI and Graphics Case Study: Using Dialog Boxes
- 3.7 Wrap-Up

Summary | Self-Review Exercises | Answers to Self-Review Exercises | Exercises | Making a Difference

3.1 Introduction

[*Note:* This chapter depends on the terminology and concepts of object-oriented programming introduced in Section 1.5, Introduction to Object Technology.]

In Chapter 2, you worked with *existing* classes, objects and methods. You used the *pre-defined* standard output object `System.out`, *invoking* its methods `print`, `println` and `printf` to display information on the screen. You used the *existing* `Scanner` class to create an object that reads into memory integer data typed by the user at the keyboard. Throughout the book, you'll use many more *preexisting* classes and objects—this is one of the great strengths of Java as an object-oriented programming language.

In this chapter, you'll learn how to create your own classes and methods. Each new *class* you create becomes a new *type* that can be used to declare variables and create objects. You can declare new classes as needed; this is one reason why Java is known as an *extensible* language.

We present a case study on creating and using a simple, real-world bank account class—`Account`. Such a class should maintain as *instance variables* attributes such as its name and balance, and provide *methods* for tasks such as querying the balance (`getBalance`), making deposits that increase the balance (`deposit`) and making withdrawals that decrease the balance (`withdraw`). We'll build the `getBalance` and `deposit` methods into the class in the chapter's examples and you'll add the `withdraw` method in the exercises.

In Chapter 2 we used the data type `int` to represent integers. In this chapter, we introduce data type `double` to represent an account balance as a number that can contain a *decimal point*—such numbers are called *floating-point numbers*. [In Chapter 8, when we get a bit deeper into object technology, we'll begin representing monetary amounts precisely with class `BigDecimal` (package `java.math`) as you should do when writing industrial-strength monetary applications.]

Typically, the apps you develop in this book will consist of two or more classes. If you become part of a development team in industry, you might work on apps that contain hundreds, or even thousands, of classes.

3.2 Instance Variables, set Methods and get Methods

In this section, you'll create two classes—Account (Fig. 3.1) and AccountTest (Fig. 3.2). Class AccountTest is an *application class* in which the main method will create and use an Account object to demonstrate class Account's capabilities.

3.2.1 Account Class with an Instance Variable, a set Method and a get Method

Different accounts typically have different names. For this reason, class Account (Fig. 3.1) contains a name *instance variable*. A class's instance variables maintain data for each object (that is, each instance) of the class. Later in the chapter we'll add an instance variable named balance so we can keep track of how much money is in the account. Class Account contains two methods—method setName stores a name in an Account object and method getName obtains a name from an Account object.

```

1  // Fig. 3.1: Account.java
2  // Account class that contains a name instance variable
3  // and methods to set and get its value.
4
5  public class Account
6  {
7      private String name; // instance variable
8
9      // method to set the name in the object
10     public void setName(String name)
11     {
12         this.name = name; // store the name
13     }
14
15     // method to retrieve the name from the object
16     public String getName()
17     {
18         return name; // return value of name to caller
19     }
20 } // end class Account

```

Fig. 3.1 | Account class that contains a name instance variable and methods to set and get its value.

Class Declaration

The *class declaration* begins in line 5:

```
public class Account
```

The keyword `public` (which Chapter 8 explains in detail) is an **access modifier**. For now, we'll simply declare every class `public`. Each `public` class declaration must be stored in a file having the *same* name as the class and ending with the `.java` filename extension; otherwise,

a compilation error will occur. Thus, `public` classes `Account` and `AccountTest` (Fig. 3.2) *must* be declared in the *separate* files `Account.java` and `AccountTest.java`, respectively.

Every class declaration contains the keyword `class` followed immediately by the class's name—in this case, `Account`. Every class's body is enclosed in a pair of left and right braces as in lines 6 and 20 of Fig. 3.1.

Identifiers and Camel Case Naming

Class names, method names and variable names are all *identifiers* and by convention all use the same *camel case* naming scheme we discussed in Chapter 2. Also by convention, class names begin with an initial *uppercase* letter, and method names and variable names begin with an initial *lowercase* letter.

Instance Variable name

Recall from Section 1.5 that an object has attributes, implemented as instance variables and carried with it throughout its lifetime. Instance variables exist before methods are called on an object, while the methods are executing and after the methods complete execution. Each object (instance) of the class has its *own* copy of the class's instance variables. A class normally contains one or more methods that manipulate the instance variables belonging to particular objects of the class.

Instance variables are declared *inside* a class declaration but *outside* the bodies of the class's methods. Line 7

```
private String name; // instance variable
```

declares instance variable `name` of type `String` *outside* the bodies of methods `setName` (lines 10–13) and `getName` (lines 16–19). `String` variables can hold character string values such as "Jane Green". If there are many `Account` objects, each has its own `name`. Because `name` is an instance variable, it can be manipulated by each of the class's methods.



Good Programming Practice 3.1

We prefer to list a class's instance variables first in the class's body, so that you see the names and types of the variables before they're used in the class's methods. You can list the class's instance variables anywhere in the class outside its method declarations, but scattering the instance variables can lead to hard-to-read code.

Access Modifiers `public` and `private`

Most instance-variable declarations are preceded with the keyword `private` (as in line 7). Like `public`, **`private`** is an *access modifier*. Variables or methods declared with access modifier `private` are accessible only to methods of the class in which they're declared. So, the variable name can be used only in each `Account` object's methods (`setName` and `getName` in this case). You'll soon see that this presents powerful software engineering opportunities.

`setName` Method of Class `Account`

Let's walk through the code of `setName`'s method declaration (lines 10–13):

```
public void setName(String name) — This line is the method header
{
    this.name = name; // store the name
}
```

We refer to the first line of each method declaration (line 10 in this case) as the *method header*. The method's **return type** (which appears before the method name) specifies the type of data the method returns to its *caller* after performing its task. The return type `void` (line 10) indicates that `setName` will perform a task but will *not* return (i.e., give back) any information to its caller. In Chapter 2, you used methods that return information—for example, you used `Scanner` method `nextInt` to input an integer typed by the user at the keyboard. When `nextInt` reads a value from the user, it *returns* that value for use in the program. As you'll soon see, `Account` method `getName` returns a value.

Method `setName` receives *parameter* name of type `String`—which represents the name that will be passed to the method as an *argument*. You'll see how parameters and arguments work together when we discuss the method call in line 21 of Fig. 3.2.

Parameters are declared in a **parameter list**, which is located inside the parentheses that follow the method name in the method header. When there are multiple parameters, each is separated from the next by a comma. Each parameter *must* specify a type (in this case, `String`) followed by a variable name (in this case, `name`).

Parameters Are Local Variables

In Chapter 2, we declared all of an app's variables in the `main` method. Variables declared in a particular method's body (such as `main`) are **local variables** which can be used *only* in that method. Each method can access only its own local variables, not those of other methods. When a method terminates, the values of its local variables are *lost*. A method's parameters also are local variables of the method.

setName Method Body

Every *method body* is delimited by a pair of *braces* (as in lines 11 and 13 of Fig. 3.1) containing one or more statements that perform the method's task(s). In this case, the method body contains a single statement (line 12) that assigns the value of the *name parameter* (a `String`) to the class's *name instance variable*, thus storing the account name in the object.

If a method contains a local variable with the *same* name as an instance variable (as in lines 10 and 7, respectively), that method's body will refer to the local variable rather than the instance variable. In this case, the local variable is said to *shadow* the instance variable in the method's body. The method's body can use the keyword **this** to refer to the shadowed instance variable explicitly, as shown on the left side of the assignment in line 12.



Good Programming Practice 3.2

We could have avoided the need for keyword `this` here by choosing a different name for the parameter in line 10, but using the `this` keyword as shown in line 12 is a widely accepted practice to minimize the proliferation of identifier names.

After line 12 executes, the method has completed its task, so it returns to its *caller*. As you'll soon see, the statement in line 21 of `main` (Fig. 3.2) calls method `setName`.

getName Method of Class Account

Method `getName` (lines 16–19)

```
public String getName()
{
    return name; // return value of name to caller
}
```

Keyword `return` passes the `String` name back to the method's caller

returns a particular `Account` object's name to the caller. The method has an *empty* parameter list, so it does *not* require additional information to perform its task. The method returns a `String`. When a method that specifies a return type *other* than `void` is called and completes its task, it *must* return a result to its caller. A statement that calls method `getName` on an `Account` object (such as the ones in lines 16 and 26 of Fig. 3.2) expects to receive the `Account`'s name—a `String`, as specified in the method declaration's *return type*.

The **return** statement in line 18 of Fig. 3.1 passes the `String` value of instance variable `name` back to the caller. For example, when the value is returned to the statement in lines 25–26 of Fig. 3.2, the statement uses that value to output the name.

3.2.2 `AccountTest` Class That Creates and Uses an Object of Class `Account`

Next, we'd like to use class `Account` in an app and *call* each of its methods. A class that contains a `main` method begins the execution of a Java app. Class `Account` *cannot* execute by itself because it does *not* contain a `main` method—if you type `java Account` in the command window, you'll get an error indicating “Main method not found in class `Account`.” To fix this problem, you must either declare a *separate* class that contains a `main` method or place a `main` method in class `Account`.

Driver Class `AccountTest`

To help you prepare for the larger programs you'll encounter later in this book and in industry, we use a separate class `AccountTest` (Fig. 3.2) containing method `main` to test class `Account`. Once `main` begins executing, it may call other methods in this and other classes; those may, in turn, call other methods, and so on. Class `AccountTest`'s `main` method creates one `Account` object and calls its `getName` and `setName` methods. Such a class is sometimes called a *driver class*—just as a `Person` object drives a `Car` object by telling it what to do (go faster, go slower, turn left, turn right, etc.), class `AccountTest` drives an `Account` object, telling it what to do by calling its methods.

```

1 // Fig. 3.2: AccountTest.java
2 // Creating and manipulating an Account object.
3 import java.util.Scanner;
4
5 public class AccountTest
6 {
7     public static void main(String[] args)
8     {
9         // create a Scanner object to obtain input from the command window
10        Scanner input = new Scanner(System.in);
11
12        // create an Account object and assign it to myAccount
13        Account myAccount = new Account();
14
15        // display initial value of name (null)
16        System.out.printf("Initial name is: %s\n\n", myAccount.getName());
17    }

```

Fig. 3.2 | Creating and manipulating an `Account` object. (Part I of 2.)

```

18     // prompt for and read name
19     System.out.println("Please enter the name:");
20     String theName = input.nextLine(); // read a line of text
21     myAccount.setName(theName); // put theName in myAccount
22     System.out.println(); // outputs a blank line
23
24     // display the name stored in object myAccount
25     System.out.printf("Name in object myAccount is:%n%s%n",
26         myAccount.getName());
27 }
28 } // end class AccountTest

```

Initial name is: null

Please enter the name:
Jane Green

Name in object myAccount is:
Jane Green

Fig. 3.2 | Creating and manipulating an Account object. (Part 2 of 2.)

Scanner Object for Receiving Input from the User

Line 10 creates a Scanner object named `input` for inputting the name from the user. Line 19 prompts the user to enter a name. Line 20 uses the Scanner object's `nextLine` method to read the name from the user and assign it to the *local* variable `theName`. You type the name and press *Enter* to submit it to the program. Pressing *Enter* inserts a newline character after the characters you typed. Method `nextLine` reads characters (including white-space characters, such as the blank in "Jane Green") until it encounters the newline, then returns a String containing the characters up to, but *not* including, the newline, which is *discarded*.

Class Scanner provides various other input methods, as you'll see throughout the book. A method similar to `nextLine`—named `next`—reads the *next word*. When you press *Enter* after typing some text, method `next` reads characters until it encounters a *white-space character* (such as a space, tab or newline), then returns a String containing the characters up to, but *not* including, the white-space character, which is *discarded*. All information after the first white-space character is *not lost*—it can be read by subsequent statements that call the Scanner's methods later in the program.

Instantiating an Object—Keyword new and Constructors

Line 13 creates an Account object and assigns it to variable `myAccount` of type Account. Variable `myAccount` is initialized with the result of the **class instance creation expression** `new Account()`. Keyword **new** creates a new object of the specified class—in this case, Account. The parentheses to the right of Account are *required*. As you'll learn in Section 3.4, those parentheses in combination with a class name represent a call to a **constructor**, which is *similar* to a method but is called implicitly by the new operator to *initialize* an object's instance variables when the object is *created*. In Section 3.4, you'll see how to place an *argument* in the parentheses to specify an *initial value* for an Account object's name instance variable—you'll enhance class Account to enable this. For now, we simply leave the parentheses *empty*. Line 10 contains a class instance creation expression for a Scanner object—

the expression initializes the Scanner with `System.in`, which tells the Scanner where to read the input from (i.e., the keyboard).

Calling Class Account's getName Method

Line 16 displays the *initial* name, which is obtained by calling the object's `getName` method. Just as we can use object `System.out` to call its methods `print`, `printf` and `println`, we can use object `myAccount` to call its methods `getName` and `setName`. Line 16 calls `getName` using the `myAccount` object created in line 13, followed by a **dot separator** (`.`), then the method name `getName` and an *empty* set of parentheses because no arguments are being passed. When `getName` is called:

1. The app transfers program execution from the call (line 16 in `main`) to method `getName`'s declaration (lines 16–19 of Fig. 3.1). Because `getName` was called via the `myAccount` object, `getName` “knows” which object's instance variable to manipulate.
2. Next, method `getName` performs its task—that is, it *returns* the name (line 18 of Fig. 3.1). When the return statement executes, program execution continues where `getName` was called (line 16 in Fig. 3.2).
3. `System.out.printf` displays the `String` returned by method `getName`, then the program continues executing at line 19 in `main`.



Error-Prevention Tip 3.1

Never use as a format-control a string that was input from the user. When method `System.out.printf` evaluates the format-control string in its first argument, the method performs tasks based on the conversion specifier(s) in that string. If the format-control string were obtained from the user, a malicious user could supply conversion specifiers that would be executed by `System.out.printf`, possibly causing a security breach.

null—the Default Initial Value for String Variables

The first line of the output shows the name “`null`.” Unlike local variables, which are *not* automatically initialized, *every instance variable has a default initial value*—a value provided by Java when you do *not* specify the instance variable's initial value. Thus, *instance variables are not* required to be explicitly initialized before they're used in a program—unless they must be initialized to values *other than* their default values. The default value for an instance variable of type `String` (like `name` in this example) is `null`, which we discuss further in Section 3.3 when we consider *reference types*.

Calling Class Account's setName Method

Line 21 calls `myAccount`'s `setName` method. A method call can supply *arguments* whose *values* are assigned to the corresponding method parameters. In this case, the value of `main`'s local variable `theName` in parentheses is the *argument* that's passed to `setName` so that the method can perform its task. When `setName` is called:

1. The app transfers program execution from line 21 in `main` to `setName` method's declaration (lines 10–13 of Fig. 3.1), and the *argument value* in the call's parentheses (`theName`) is assigned to the corresponding *parameter* (`name`) in the method header (line 10 of Fig. 3.1). Because `setName` was called via the `myAccount` object, `setName` “knows” which object's instance variable to manipulate.

2. Next, method `setName` performs its task—that is, it assigns the `name` parameter’s value to instance variable `name` (line 12 of Fig. 3.1).
3. When program execution reaches `setName`’s closing right brace, it returns to where `setName` was called (line 21 of Fig. 3.2), then continues at line 22 of Fig. 3.2.

The number of *arguments* in a method call *must match* the number of *parameters* in the method declaration’s parameter list. Also, the argument types in the method call must be *consistent* with the types of the corresponding parameters in the method’s declaration. (As you’ll learn in Chapter 6, an argument’s type and its corresponding parameter’s type are *not* required to be *identical*.) In our example, the method call passes one argument of type `String` (`theName`)—and the method declaration specifies one parameter of type `String` (`name`, declared in line 10 of Fig. 3.1). So in this example, the type of the argument in the method call *exactly* matches the type of the parameter in the method header.

Displaying the Name That Was Entered by the User

Line 22 of Fig. 3.2 outputs a blank line. When the second call to method `getName` (line 26) executes, the name entered by the user in line 20 is displayed. When the statement at lines 25–26 completes execution, the end of method `main` is reached, so the program terminates.

3.2.3 Compiling and Executing an App with Multiple Classes

You must compile the classes in Figs. 3.1 and 3.2 before you can *execute* the app. This is the first time you’ve created an app with *multiple* classes. Class `AccountTest` has a `main` method; class `Account` does not. To compile this app, first change to the directory that contains the app’s source-code files. Next, type the command

```
javac Account.java AccountTest.java
```

to compile *both* classes at once. If the directory containing the app includes *only* this app’s files, you can compile both classes with the command

```
javac *.java
```

The asterisk (*) in `*.java` indicates that *all* files in the *current* directory ending with the filename extension “`.java`” should be compiled. If both classes compile correctly—that is, no compilation errors are displayed—you can then run the app with the command

```
java AccountTest
```

3.2.4 Account UML Class Diagram with an Instance Variable and set and get Methods

We’ll often use UML class diagrams to summarize a class’s *attributes* and *operations*. In industry, UML diagrams help systems designers specify a system in a concise, graphical, programming-language-independent manner, before programmers implement the system in a specific programming language. Figure 3.3 presents a **UML class diagram** for class `Account` of Fig. 3.1.

Top Compartment

In the UML, each class is modeled in a class diagram as a rectangle with three compartments. In this diagram the *top* compartment contains the *class name* `Account` centered horizontally in boldface type.

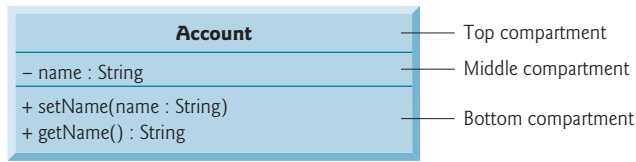


Fig. 3.3 | UML class diagram for class Account of Fig. 3.1.

Middle Compartment

The *middle* compartment contains the *class's attribute* name, which corresponds to the instance variable of the same name in Java. Instance variable name is *private* in Java, so the UML class diagram lists a *minus sign* (–) *access modifier* before the attribute name. Following the attribute name are a *colon* and the *attribute type*, in this case *String*.

Bottom Compartment

The *bottom* compartment contains the class's **operations**, *setName* and *getName*, which correspond to the methods of the same names in Java. The UML models operations by listing the operation name preceded by an *access modifier*, in this case + *getName*. This plus sign (+) indicates that *getName* is a *public* operation in the UML (because it's a *public* method in Java). Operation *getName* does *not* have any parameters, so the parentheses following the operation name in the class diagram are *empty*, just as they are in the method's declaration in line 16 of Fig. 3.1. Operation *setName*, also a *public* operation, has a *String* parameter called *name*.

Return Types

The UML indicates the *return type* of an operation by placing a colon and the return type *after* the parentheses following the operation name. Account method *getName* (Fig. 3.1) has a *String* return type. Method *setName* *does not* return a value (because it returns *void* in Java), so the UML class diagram *does not* specify a return type after the parentheses of this operation.

Parameters

The UML models a parameter a bit differently from Java by listing the parameter name, followed by a colon and the parameter type in the parentheses after the operation name. The UML has its own data types similar to those of Java, but for simplicity, we'll use the Java data types. Account method *setName* (Fig. 3.1) has a *String* parameter named *name*, so Fig. 3.3 lists *name : String* between the parentheses following the method name.

3.2.5 Additional Notes on Class AccountTest

static Method main

In Chapter 2, each class we declared had one method named *main*. Recall that *main* is a special method that's *always* called automatically by the Java Virtual Machine (JVM) when you execute an app. You must call most other methods *explicitly* to tell them to perform their tasks.

Lines 7–27 of Fig. 3.2 declare method *main*. A key part of enabling the JVM to locate and call method *main* to begin the app's execution is the *static* keyword (line 7), which

indicates that `main` is a static method. A static method is special, because you can call it *without first creating an object of the class in which the method is declared*—in this case class `AccountTest`. We discuss static methods in detail in Chapter 6.

Notes on import Declarations

Notice the `import` declaration in Fig. 3.2 (line 3), which indicates to the compiler that the program uses class `Scanner`. As you learned in Chapter 2, classes `System` and `String` are in package `java.lang`, which is *implicitly* imported into *every* Java program, so all programs can use that package's classes *without* explicitly importing them. *Most* other classes you'll use in Java programs *must* be imported *explicitly*.

There's a special relationship between classes that are compiled in the *same* directory, like classes `Account` and `AccountTest`. By default, such classes are considered to be in the *same* package—known as the **default package**. Classes in the *same* package are *implicitly imported* into the source-code files of other classes in that package. Thus, an `import` declaration is *not* required when one class in a package uses another in the *same* package—such as when class `AccountTest` uses class `Account`.

The `import` declaration in line 3 is *not* required if we refer to class `Scanner` throughout this file as `java.util.Scanner`, which includes the *full package name and class name*. This is known as the class's **fully qualified class name**. For example, line 10 of Fig. 3.2 also could be written as

```
java.util.Scanner input = new java.util.Scanner(System.in);
```



Software Engineering Observation 3.1

The Java compiler does not require `import` declarations in a Java source-code file if the fully qualified class name is specified every time a class name is used. Most Java programmers prefer the more concise programming style enabled by `import` declarations.

3.2.6 Software Engineering with private Instance Variables and public set and get Methods

As you'll see, through the use of *set* and *get* methods, you can *validate* attempted modifications to private data and control how that data is presented to the caller—these are compelling software engineering benefits. We'll discuss this in more detail in Section 3.5.

If the instance variable were `public`, any **client** of the class—that is, any other class that calls the class's methods—could see the data and do whatever it wanted with it, including setting it to an *invalid* value.

You might think that even though a client of the class cannot directly access a private instance variable, the client can do whatever it wants with the variable through `public set` and `get` methods. You would think that you could peek at the private data any time with the `public get` method and that you could modify the private data at will through the `public set` method. But *set* methods can be programmed to *validate* their arguments and reject any attempts to *set* the data to bad values, such as a negative body temperature, a day in March out of the range 1 through 31, a product code not in the company's product catalog, etc. And a *get* method can present the data in a different form. For example, a `Grade` class might store a grade as an `int` between 0 and 100, but a `getGrade` method might return a letter grade as a `String`, such as "A" for grades between 90 and 100, "B" for grades between 80 and 89, etc. Tightly controlling the access to and presentation of

private data can greatly reduce errors, while increasing the robustness and security of your programs.

Declaring instance variables with access modifier `private` is known as *data hiding* or *information hiding*. When a program creates (instantiates) an object of class `Account`, variable name is *encapsulated* (hidden) in the object and can be accessed only by methods of the object's class.



Software Engineering Observation 3.2

Precede each instance variable and method declaration with an access modifier. Generally, instance variables should be declared `private` and methods `public`. Later in the book, we'll discuss why you might want to declare a method `private`.

Conceptual View of an `Account` Object with Encapsulated Data

You can think of an `Account` object as shown in Fig. 3.4. The private instance variable name is *hidden inside* the object (represented by the inner circle containing name) and *protected by an outer layer* of `public` methods (represented by the outer circle containing `getName` and `setName`). Any client code that needs to interact with the `Account` object can do so *only* by calling the `public` methods of the protective outer layer.

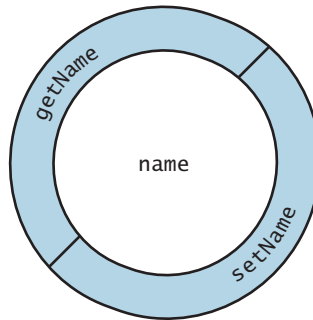


Fig. 3.4 | Conceptual view of an `Account` object with its encapsulated private instance variable `name` and protective layer of `public` methods.

3.3 Primitive Types vs. Reference Types

Java's types are divided into primitive types and **reference types**. In Chapter 2, you worked with variables of type `int`—one of the primitive types. The other primitive types are `boolean`, `byte`, `char`, `short`, `long`, `float` and `double`, each of which we discuss in this book—these are summarized in Appendix D. All nonprimitive types are *reference types*, so classes, which specify the types of objects, are reference types.

A primitive-type variable can hold exactly *one* value of its declared type at a time. For example, an `int` variable can store one integer at a time. When another value is assigned to that variable, the new value replaces the previous one—which is *lost*.

Recall that local variables are *not* initialized by default. Primitive-type instance variables *are* initialized by default—instance variables of types `byte`, `char`, `short`, `int`, `long`, `float` and `double` are initialized to 0, and variables of type `boolean` are initialized to

false. You can specify your own initial value for a primitive-type variable by assigning the variable a value in its declaration, as in

```
private int numberOfStudents = 10;
```

Programs use variables of reference types (normally called **references**) to store the *addresses* of objects in the computer's memory. Such a variable is said to **refer to an object** in the program. *Objects* that are referenced may each contain *many* instance variables. Line 10 of Fig. 3.2:

```
Scanner input = new Scanner(System.in);
```

creates an object of class `Scanner`, then assigns to the variable `input` a *reference* to that `Scanner` object. Line 13 of Fig. 3.2:

```
Account myAccount = new Account();
```

creates an object of class `Account`, then assigns to the variable `myAccount` a *reference* to that `Account` object. *Reference-type instance variables, if not explicitly initialized, are initialized by default to the value null*—which represents a “reference to nothing.” That’s why the first call to `getName` in line 16 of Fig. 3.2 returns `null`—the value of `name` has *not* yet been set, so the *default initial value* `null` is returned.

To call methods on an object, you need a reference to the object. In Fig. 3.2, the statements in method `main` use the variable `myAccount` to call methods `getName` (lines 16 and 26) and `setName` (line 21) to interact with the `Account` object. Primitive-type variables do *not* refer to objects, so such variables *cannot* be used to call methods.

3.4 Account Class: Initializing Objects with Constructors

As mentioned in Section 3.2, when an object of class `Account` (Fig. 3.1) is created, its `String` instance variable `name` is initialized to `null` by *default*. But what if you want to provide a name when you *create* an `Account` object?

Each class you declare can optionally provide a *constructor* with parameters that can be used to initialize an object of a class when the object is created. Java *requires* a constructor call for *every* object that’s created, so this is the ideal point to initialize an object’s instance variables. The next example enhances class `Account` (Fig. 3.5) with a constructor that can receive a name and use it to initialize instance variable `name` when an `Account` object is created (Fig. 3.6).

3.4.1 Declaring an Account Constructor for Custom Object Initialization

When you declare a class, you can provide your own constructor to specify *custom initialization* for objects of your class. For example, you might want to specify a name for an `Account` object when the object is created, as in line 10 of Fig. 3.6:

```
Account account1 = new Account("Jane Green");
```

In this case, the `String` argument `"Jane Green"` is passed to the `Account` object’s constructor and used to initialize the `name` instance variable. The preceding statement requires that the class provide a constructor that takes only a `String` parameter. Figure 3.5 contains a modified `Account` class with such a constructor.

```

1 // Fig. 3.5: Account.java
2 // Account class with a constructor that initializes the name.
3
4 public class Account
5 {
6     private String name; // instance variable
7
8     // constructor initializes name with parameter name
9     public Account(String name) // constructor name is class name
10    {
11        this.name = name;
12    }
13
14    // method to set the name
15    public void setName(String name)
16    {
17        this.name = name;
18    }
19
20    // method to retrieve the name
21    public String getName()
22    {
23        return name;
24    }
25 } // end class Account

```

Fig. 3.5 | Account class with a constructor that initializes the name.

Account Constructor Declaration

Lines 9–12 of Fig. 3.5 declare Account’s constructor. A constructor *must* have the *same name* as the class. A constructor’s *parameter list* specifies that the constructor requires one or more pieces of data to perform its task. Line 9 indicates that the constructor has a String parameter called name. When you create a new Account object (as you’ll see in Fig. 3.6), you’ll pass a person’s name to the constructor, which will receive that name in the parameter name. The constructor will then assign name to *instance variable* name in line 11.



Error-Prevention Tip 3.2

Even though it’s possible to do so, do not call methods from constructors. We’ll explain this in Chapter 10, Object-Oriented Programming: Polymorphism and Interfaces.

Parameter name of Class Account’s Constructor and Method setName

Recall from Section 3.2.1 that method parameters are local variables. In Fig. 3.5, the constructor and method setName both have a parameter called name. Although these parameters have the same identifier (name), the parameter in line 9 is a local variable of the constructor that’s *not* visible to method setName, and the one in line 15 is a local variable of setName that’s *not* visible to the constructor.

3.4.2 Class AccountTest: Initializing Account Objects When They’re Created

The AccountTest program (Fig. 3.6) initializes two Account objects using the constructor. Line 10 creates and initializes the Account object account1. Keyword new requests

memory from the system to store the `Account` object, then implicitly calls the class's constructor to *initialize* the object. The call is indicated by the parentheses after the class name, which contain the *argument* "Jane Green" that's used to initialize the new object's name. The class instance creation expression in line 10 returns a *reference* to the new object, which is assigned to the variable `account1`. Line 11 repeats this process, passing the argument "John Blue" to initialize the name for `account2`. Lines 14–15 use each object's `getName` method to obtain the names and show that they were indeed initialized when the objects were *created*. The output shows *different* names, confirming that each `Account` maintains its *own copy* of instance variable name.

```

1 // Fig. 3.6: AccountTest.java
2 // Using the Account constructor to initialize the name instance
3 // variable at the time each Account object is created.
4
5 public class AccountTest
6 {
7     public static void main(String[] args)
8     {
9         // create two Account objects
10        Account account1 = new Account("Jane Green");
11        Account account2 = new Account("John Blue");
12
13        // display initial value of name for each Account
14        System.out.printf("account1 name is: %s\n", account1.getName());
15        System.out.printf("account2 name is: %s\n", account2.getName());
16    }
17 } // end class AccountTest

```

```

account1 name is: Jane Green
account2 name is: John Blue

```

Fig. 3.6 | Using the `Account` constructor to initialize the name instance variable at the time each `Account` object is created.

Constructors Cannot Return Values

An important difference between constructors and methods is that *constructors cannot return values*, so they *cannot* specify a return type (not even `void`). Normally, constructors are declared `public`—later in the book we'll explain when to use private constructors.

Default Constructor

Recall that line 13 of Fig. 3.2

```
Account myAccount = new Account();
```

used `new` to create an `Account` object. The *empty* parentheses after "new `Account`" indicate a call to the class's **default constructor**—in any class that does *not* explicitly declare a constructor, the compiler provides a default constructor (which always has no parameters). When a class has only the default constructor, the class's instance variables are initialized to their *default values*. In Section 8.5, you'll learn that classes can have multiple constructors.

There's No Default Constructor in a Class That Declares a Constructor

If you declare a constructor for a class, the compiler will *not* create a *default constructor* for that class. In that case, you will not be able to create an `Account` object with the class instance creation expression `new Account()` as we did in Fig. 3.2—unless the custom constructor you declare takes *no* parameters.

**Software Engineering Observation 3.3**

Unless default initialization of your class's instance variables is acceptable, provide a custom constructor to ensure that your instance variables are properly initialized with meaningful values when each new object of your class is created.

Adding the Constructor to Class Account's UML Class Diagram

The UML class diagram of Fig. 3.7 models class `Account` of Fig. 3.5, which has a constructor with a `String` `name` parameter. Like operations, the UML models constructors in the *third* compartment of a class diagram. To distinguish a constructor from the class's operations, the UML requires that the word “constructor” be enclosed in **guillemets** (« and ») and placed before the constructor's name. It's customary to list constructors *before* other operations in the third compartment.

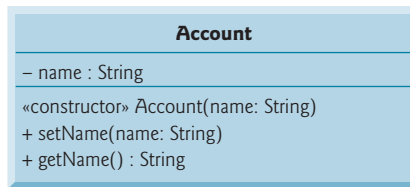


Fig. 3.7 | UML class diagram for `Account` class of Fig. 3.5.

3.5 Account Class with a Balance; Floating-Point Numbers

We now declare an `Account` class that maintains the *balance* of a bank account in addition to the name. Most account balances are not integers. So, class `Account` represents the account balance as a **floating-point number**—a number with a *decimal point*, such as 43.95, 0.0, −129.8873. [In Chapter 8, we'll begin representing monetary amounts precisely with class `BigDecimal` as you should do when writing industrial-strength monetary applications.]

Java provides two primitive types for storing floating-point numbers in memory—`float` and `double`. Variables of type **float** represent **single-precision floating-point numbers** and can hold up to *seven significant digits*. Variables of type **double** represent **double-precision floating-point numbers**. These require *twice* as much memory as `float` variables and can hold up to *15 significant digits*—about *double* the precision of `float` variables.

Most programmers represent floating-point numbers with type `double`. In fact, Java treats all floating-point numbers you type in a program's source code (such as 7.33 and 0.0975) as `double` values by default. Such values in the source code are known as **floating-point literals**. See Appendix D, Primitive Types, for the precise ranges of values for `float`s and `double`s.

3.5.1 Account Class with a balance Instance Variable of Type double

Our next app contains a version of class `Account` (Fig. 3.8) that maintains as instance variables the name *and* the balance of a bank account. A typical bank services *many* accounts, each with its *own* balance, so line 8 declares an instance variable `balance` of type `double`. Every instance (i.e., object) of class `Account` contains its *own* copies of *both* the name and the balance.

```

1  // Fig. 3.8: Account.java
2  // Account class with a double instance variable balance and a constructor
3  // and deposit method that perform validation.
4
5  public class Account
6  {
7      private String name; // instance variable
8      private double balance; // instance variable
9
10     // Account constructor that receives two parameters
11     public Account(String name, double balance)
12     {
13         this.name = name; // assign name to instance variable name
14
15         // validate that the balance is greater than 0.0; if it's not,
16         // instance variable balance keeps its default initial value of 0.0
17         if (balance > 0.0) // if the balance is valid
18             this.balance = balance; // assign it to instance variable balance
19     }
20
21     // method that deposits (adds) only a valid amount to the balance
22     public void deposit(double depositAmount)
23     {
24         if (depositAmount > 0.0) // if the depositAmount is valid
25             balance = balance + depositAmount; // add it to the balance
26     }
27
28     // method returns the account balance
29     public double getBalance()
30     {
31         return balance;
32     }
33
34     // method that sets the name
35     public void setName(String name)
36     {
37         this.name = name;
38     }
39
40     // method that returns the name
41     public String getName()
42     {

```

Fig. 3.8 | Account class with a `double` instance variable `balance` and a constructor and deposit method that perform validation. (Part 1 of 2.)

```

43         return name; // give value of name back to caller
44     } // end method getName
45 } // end class Account

```

Fig. 3.8 | Account class with a `double` instance variable `balance` and a constructor and `deposit` method that perform validation. (Part 2 of 2.)

Account Class Two-Parameter Constructor

The class has a *constructor* and four *methods*. It's common for someone opening an account to deposit money immediately, so the constructor (lines 11–19) now receives a second parameter—`initialBalance` of type `double` that represents the *starting balance*. Lines 17–18 ensure that `initialBalance` is greater than 0.0. If so, `initialBalance`'s value is assigned to instance variable `balance`. Otherwise, `balance` remains at 0.0—its *default initial value*.

Account Class deposit Method

Method `deposit` (lines 22–26) does *not* return any data when it completes its task, so its return type is `void`. The method receives one parameter named `depositAmount`—a `double` value that's *added* to the balance *only* if the parameter value is *valid* (i.e., greater than zero). Line 25 first adds the current balance and `depositAmount`, forming a *temporary* sum which is *then* assigned to `balance`, *replacing* its prior value (recall that addition has a *higher* precedence than assignment). It's important to understand that the calculation on the right side of the assignment operator in line 25 does *not* modify the balance—that's why the assignment is necessary.

Account Class getBalance Method

Method `getBalance` (lines 29–32) allows *clients* of the class (i.e., other classes whose methods call the methods of this class) to obtain the value of a particular `Account` object's `balance`. The method specifies return type `double` and an *empty* parameter list.

Account's Methods Can All Use balance

Once again, the statements in lines 18, 25 and 31 use the variable `balance` even though it was *not* declared in *any* of the methods. We can use `balance` in these methods because it's an *instance variable* of the class.

3.5.2 AccountTest Class to Use Class Account

Class `AccountTest` (Fig. 3.9) creates two `Account` objects (lines 9–10) and initializes them with a *valid* balance of 50.00 and an *invalid* balance of -7.53, respectively—for the purpose of our examples, we assume that balances must be greater than or equal to zero. The calls to method `System.out.printf` in lines 13–16 output the account names and balances, which are obtained by calling each `Account`'s `getName` and `getBalance` methods.

```

1 // Fig. 3.9: AccountTest.java
2 // Inputting and outputting floating-point numbers with Account objects.
3 import java.util.Scanner;

```

Fig. 3.9 | Inputting and outputting floating-point numbers with `Account` objects. (Part 1 of 3.)


```

4
5 public class AccountTest
6 {
7     public static void main(String[] args)
8     {
9         Account account1 = new Account("Jane Green", 50.00);
10        Account account2 = new Account("John Blue", -7.53);
11
12        // display initial balance of each object
13        System.out.printf("%s balance: %.2f%n",
14            account1.getName(), account1.getBalance());
15        System.out.printf("%s balance: %.2f%n%n",
16            account2.getName(), account2.getBalance());
17
18        // create a Scanner to obtain input from the command window
19        Scanner input = new Scanner(System.in);
20
21        System.out.print("Enter deposit amount for account1: "); // prompt
22        double depositAmount = input.nextDouble(); // obtain user input
23        System.out.printf("%nadding %.2f to account1 balance%n%n",
24            depositAmount);
25        account1.deposit(depositAmount); // add to account1's balance
26
27        // display balances
28        System.out.printf("%s balance: %.2f%n",
29            account1.getName(), account1.getBalance());
30        System.out.printf("%s balance: %.2f%n%n",
31            account2.getName(), account2.getBalance());
32
33        System.out.print("Enter deposit amount for account2: "); // prompt
34        depositAmount = input.nextDouble(); // obtain user input
35        System.out.printf("%nadding %.2f to account2 balance%n%n",
36            depositAmount);
37        account2.deposit(depositAmount); // add to account2 balance
38
39        // display balances
40        System.out.printf("%s balance: %.2f%n",
41            account1.getName(), account1.getBalance());
42        System.out.printf("%s balance: %.2f%n%n",
43            account2.getName(), account2.getBalance());
44    } // end main
45 } // end class AccountTest

```

```

Jane Green balance: $50.00
John Blue balance: $0.00

Enter deposit amount for account1: 25.53

adding 25.53 to account1 balance

Jane Green balance: $75.53
John Blue balance: $0.00

```

Fig. 3.9 | Inputting and outputting floating-point numbers with Account objects. (Part 2 of 3.)

```

Enter deposit amount for account2: 123.45

adding 123.45 to account2 balance

Jane Green balance: $75.53
John Blue balance: $123.45

```

Fig. 3.9 | Inputting and outputting floating-point numbers with Account objects. (Part 3 of 3.)

Displaying the Account Objects' Initial Balances

When method `getBalance` is called for `account1` from line 14, the value of `account1`'s balance is returned from line 31 of Fig. 3.8 and displayed by the `System.out.printf` statement (Fig. 3.9, lines 13–14). Similarly, when method `getBalance` is called for `account2` from line 16, the value of the `account2`'s balance is returned from line 31 of Fig. 3.8 and displayed by the `System.out.printf` statement (Fig. 3.9, lines 15–16). The balance of `account2` is initially 0.00, because the constructor rejected the attempt to start `account2` with a *negative* balance, so the balance retains its default initial value.

Formatting Floating-Point Numbers for Display

Each of the balances is output by `printf` with the format specifier `%.2f`. The **%f format specifier** is used to output values of type `float` or `double`. The `.2` between `%` and `f` represents the number of *decimal places* (2) that should be output to the *right* of the decimal point in the floating-point number—also known as the number's **precision**. Any floating-point value output with `%.2f` will be *rounded* to the *hundredths position*—for example, 123.457 would be rounded to 123.46 and 27.33379 would be rounded to 27.33.

Reading a Floating-Point Value from the User and Making a Deposit

Line 21 (Fig. 3.9) prompts the user to enter a deposit amount for `account1`. Line 22 declares *local* variable `depositAmount` to store each deposit amount entered by the user. Unlike *instance* variables (such as `name` and `balance` in class `Account`), *local* variables (like `depositAmount` in `main`) are *not* initialized by default, so they normally must be initialized explicitly. As you'll learn momentarily, variable `depositAmount`'s initial value will be determined by the user's input.



Common Programming Error 3.1

The Java compiler will issue a compilation error if you attempt to use the value of an uninitialized local variable. This helps you avoid dangerous execution-time logic errors. It's always better to get the errors out of your programs at compilation time rather than execution time.

Line 22 obtains the input from the user by calling `Scanner` object `input`'s **nextDouble** method, which returns a `double` value entered by the user. Lines 23–24 display the `depositAmount`. Line 25 calls object `account1`'s `deposit` method with the `depositAmount` as the method's *argument*. When the method is called, the argument's value is assigned to the parameter `depositAmount` of method `deposit` (line 22 of Fig. 3.8); then method `deposit` adds that value to the balance. Lines 28–31 (Fig. 3.9) output the names and balances of both `Accounts` *again* to show that *only* `account1`'s balance has changed.

Line 33 prompts the user to enter a deposit amount for `account2`. Line 34 obtains the input from the user by calling `Scanner` object `input`'s `nextDouble` method. Lines 35–36 display the `depositAmount`. Line 37 calls object `account2`'s `deposit` method with `depositAmount` as the method's *argument*; then method `deposit` adds that value to the balance. Finally, lines 40–43 output the names and balances of both `Accounts` *again* to show that *only* `account2`'s balance has changed.

Duplicated Code in Method main

The six statements at lines 13–14, 15–16, 28–29, 30–31, 40–41 and 42–43 are almost *identical*—they each output an `Account`'s name and balance. They differ only in the name of the `Account` object—`account1` or `account2`. Duplicate code like this can create *code maintenance problems* when that code needs to be updated—if *six* copies of the same code all have the same error or update to be made, you must make that change *six* times, *without making errors*. Exercise 3.15 asks you to modify Fig. 3.9 to include a `displayAccount` method that takes as a parameter an `Account` object and outputs the object's name and balance. You'll then replace `main`'s duplicated statements with six calls to `displayAccount`, thus reducing the size of your program and improving its maintainability by having *one* copy of the code that displays an `Account`'s name and balance.



Software Engineering Observation 3.4

Replacing duplicated code with calls to a method that contains one copy of that code can reduce the size of your program and improve its maintainability.

UML Class Diagram for Class Account

The UML class diagram in Fig. 3.10 concisely models class `Account` of Fig. 3.8. The diagram models in its *second* compartment the private attributes `name` of type `String` and `balance` of type `double`.

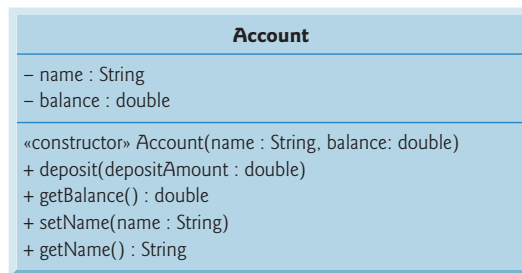


Fig. 3.10 | UML class diagram for `Account` class of Fig. 3.8.

Class `Account`'s *constructor* is modeled in the *third* compartment with parameters `name` of type `String` and `initialBalance` of type `double`. The class's four public methods also are modeled in the *third* compartment—operation `deposit` with a `depositAmount` parameter of type `double`, operation `getBalance` with a return type of `double`, operation `setName` with a `name` parameter of type `String` and operation `getName` with a return type of `String`.

3.6 (Optional) GUI and Graphics Case Study: Using Dialog Boxes

This optional case study is designed for those who want to begin learning Java’s powerful capabilities for creating graphical user interfaces (GUIs) and graphics early in the book, before the deeper discussions of these topics later in the book. This case study features Java’s mature Swing technology, which as of this writing is still a bit more popular than the newer JavaFX technology presented in later chapters.

This GUI and Graphics Case Study appears in 10 brief sections (see Fig. 3.11). Each section introduces new concepts and provides examples with screen captures that show sample interactions and results. In the first few sections, you’ll create your first graphical apps. In subsequent sections, you’ll use object-oriented programming concepts to create an app that draws a variety of shapes. When we formally introduce GUIs in Chapter 12, we use the mouse to choose exactly which shapes to draw and where to draw them. In Chapter 13, we add capabilities of the Java 2D graphics API to draw the shapes with different line thicknesses and fills. We hope you find this case study informative and entertaining.

Location	Title—Exercise(s)
Section 3.6	Using Dialog Boxes—Basic input and output with dialog boxes
Section 4.15	Creating Simple Drawings—Displaying and drawing lines on the screen
Section 5.11	Drawing Rectangles and Ovals—Using shapes to represent data
Section 6.13	Colors and Filled Shapes—Drawing a bull’s-eye and random graphics
Section 7.17	Drawing Arcs—Drawing spirals with arcs
Section 8.16	Using Objects with Graphics—Storing shapes as objects
Section 9.7	Displaying Text and Images Using Labels—Providing status information
Section 10.10	Drawing with Polymorphism—Identifying the similarities between shapes
Exercise 12.17	Expanding the Interface—Using GUI components and event handling
Exercise 13.31	Adding Java 2D—Using the Java 2D API to enhance drawings

Fig. 3.11 | Summary of the GUI and Graphics Case Study in each chapter.

Displaying Text in a Dialog Box

The programs presented thus far display output in the *command window*. Many apps use windows or **dialog boxes** (also called **dialogs**) to display output. Web browsers such as Chrome, Firefox, Internet Explorer, Safari and Opera display web pages in their own windows. E-mail programs allow you to type and read messages in a window. Typically, dialog boxes are windows in which programs display important messages to users. Class **JOptionPane** provides prebuilt dialog boxes that enable programs to display windows containing messages—such windows are called **message dialogs**. Figure 3.12 displays the String “Welcome to Java” in a message dialog.

```

1 // Fig. 3.12: Dialog1.java
2 // Using JOptionPane to display multiple lines in a dialog box.
3 import javax.swing.JOptionPane;
4
5 public class Dialog1
6 {
7     public static void main(String[] args)
8     {
9         // display a dialog with a message
10        JOptionPane.showMessageDialog(null, "Welcome to Java");
11    }
12 } // end class Dialog1

```



Fig. 3.12 | Using `JOptionPane` to display multiple lines in a dialog box.

JOptionPane Class static Method showMessageDialog

Line 3 indicates that the program uses class `JOptionPane` from package `javax.swing`. This package contains many classes that help you create **graphical user interfaces (GUIs)**. **GUI components** facilitate data entry by a program's user and presentation of outputs to the user. Line 10 calls `JOptionPane` method `showMessageDialog` to display a dialog box containing a message. The method requires two arguments. The first helps the Java app determine where to position the dialog box. A dialog is typically displayed from a GUI app with its own window. The first argument refers to that window (known as the *parent window*) and causes the dialog to appear centered over the app's window. If the first argument is `null`, the dialog box is displayed at the center of your screen. The second argument is the `String` to display in the dialog box.

Introducing static Methods

`JOptionPane` method `showMessageDialog` is a so-called **static method**. Such methods often define frequently used tasks. For example, many programs display dialog boxes, and the code to do this is the same each time. Rather than requiring you to “reinvent the wheel” and create code to display a dialog, the designers of class `JOptionPane` declared a static method that performs this task for you. A static method is called by using its class name followed by a dot (.) and the method name, as in

```
ClassName.methodName(arguments)
```

Notice that you do *not* create an object of class `JOptionPane` to use its static method `showMessageDialog`. We discuss static methods in more detail in Chapter 6.

Entering Text in a Dialog

Figure 3.13 uses another predefined `JOptionPane` dialog called an **input dialog** that allows the user to *enter data* into a program. The program asks for the user's name and responds with a message dialog containing a greeting and the name that the user entered.

```

1 // Fig. 3.13: NameDialog.java
2 // Obtaining user input from a dialog.
3 import javax.swing.JOptionPane;
4
5 public class NameDialog
6 {
7     public static void main(String[] args)
8     {
9         // prompt user to enter name
10        String name = JOptionPane.showInputDialog("What is your name?");
11
12        // create the message
13        String message =
14            String.format("Welcome, %s, to Java Programming!", name);
15
16        // display the message to welcome the user by name
17        JOptionPane.showMessageDialog(null, message);
18    } // end main
19 } // end class NameDialog

```

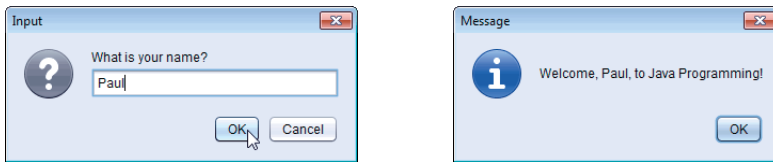


Fig. 3.13 | Obtaining user input from a dialog.

JOptionPane Class static Method showInputDialog

Line 10 uses `JOptionPane` method **showInputDialog** to display an input dialog containing a prompt and a *field* (known as a **text field**) in which the user can enter text. Method `showInputDialog`'s argument is the prompt that indicates what the user should enter. The user types characters in the text field, then clicks the **OK** button or presses the *Enter* key to return the `String` to the program. Method `showInputDialog` returns a `String` containing the characters typed by the user. We store the `String` in variable `name`. If you press the dialog's **Cancel** button or press the *Esc* key, the method returns `null` and the program displays the word "null" as the name.

String Class static Method format

Lines 13–14 use static `String` method **format** to return a `String` containing a greeting with the user's name. Method `format` works like method `System.out.printf`, except that `format` returns the formatted `String` rather than displaying it in a command window. Line 17 displays the greeting in a message dialog, just as we did in Fig. 3.12.

GUI and Graphics Case Study Exercise

3.1 Modify the addition program in Fig. 2.7 to use dialog-based input and output with the methods of class `JOptionPane`. Since method `showInputDialog` returns a `String`, you must convert the `String` the user enters to an `int` for use in calculations. The static method `parseInt` of class `Integer` (package `java.lang`) takes a `String` argument representing an integer and returns the value as an `int`. If the `String` does not contain a valid integer, the program will terminate with an error.

3.7 Wrap-Up

In this chapter, you learned how to create your own classes and methods, create objects of those classes and call methods of those objects to perform useful actions. You declared instance variables of a class to maintain data for each object of the class, and you declared your own methods to operate on that data. You learned how to call a method to tell it to perform its task and how to pass information to a method as arguments whose values are assigned to the method's parameters. You learned the difference between a local variable of a method and an instance variable of a class, and that only instance variables are initialized automatically. You also learned how to use a class's constructor to specify the initial values for an object's instance variables. You saw how to create UML class diagrams that model the methods, attributes and constructors of classes. Finally, you learned about floating-point numbers (numbers with decimal points)—how to store them with variables of primitive type `double`, how to input them with a `Scanner` object and how to format them with `printf` and format specifier `%f` for display purposes. [In Chapter 8, we'll begin representing monetary amounts precisely with class `BigDecimal`.] You may have also begun the optional GUI and Graphics case study, learning how to write your first GUI applications. In the next chapter we begin our introduction to control statements, which specify the order in which a program's actions are performed. You'll use these in your methods to specify how they should order their tasks.

Summary

Section 3.2 Instance Variables, set Methods and get Methods

- Each class you create becomes a new type that can be used to declare variables and create objects.
- You can declare new classes as needed; this is one reason Java is known as an extensible language.

Section 3.2.1 Account Class with an Instance Variable, a set Method and a get Method

- Each class declaration that begins with the access modifier (p. 71) `public` must be stored in a file that has the same name as the class and ends with the `.java` filename extension.
- Every class declaration contains keyword `class` followed immediately by the class's name.
- Class, method and variable names are identifiers. By convention all use camel case names. Class names begin with an uppercase letter, and method and variable names begin with a lowercase letter.
- An object has attributes that are implemented as instance variables (p. 72) and carried with it throughout its lifetime.
- Instance variables exist before methods are called on an object, while the methods are executing and after the methods complete execution.
- A class normally contains one or more methods that manipulate the instance variables that belong to particular objects of the class.
- Instance variables are declared inside a class declaration but outside the bodies of the class's method declarations.
- Each object (instance) of the class has its own copy of each of the class's instance variables.
- Most instance-variable declarations are preceded with the keyword `private` (p. 72), which is an access modifier. Variables or methods declared with access modifier `private` are accessible only to methods of the class in which they're declared.

- Parameters are declared in a comma-separated parameter list (p. 73), which is located inside the parentheses that follow the method name in the method declaration. Multiple parameters are separated by commas. Each parameter must specify a type followed by a variable name.
- Variables declared in the body of a particular method are local variables and can be used only in that method. When a method terminates, the values of its local variables are lost. A method's parameters are local variables of the method.
- Every method's body is delimited by left and right braces (`{` and `}`).
- Each method's body contains one or more statements that perform the method's task(s).
- The method's return type specifies the type of data returned to a method's caller. Keyword `void` indicates that a method will perform a task but will not return any information.
- Empty parentheses following a method name indicate that the method does not require any parameters to perform its task.
- When a method that specifies a return type (p. 73) other than `void` is called and completes its task, the method must return a result to its calling method.
- The return statement (p. 74) passes a value from a called method back to its caller.
- Classes often provide `public` methods to allow the class's clients to *set* or *get* private instance variables. The names of these methods need not begin with *set* or *get*, but this naming convention is recommended.

Section 3.2.2 AccountTest Class That Creates and Uses an Object of Class Account

- A class that creates an object of another class, then calls the object's methods, is a driver class.
- Scanner method `nextLine` (p. 75) reads characters until a newline character is encountered, then returns the characters as a `String`.
- Scanner method `next` (p. 75) reads characters until any white-space character is encountered, then returns the characters as a `String`.
- A class instance creation expression (p. 75) begins with keyword `new` and creates a new object.
- A constructor is similar to a method but is called implicitly by the `new` operator to initialize an object's instance variables at the time the object is created.
- To call a method of an object, follow the object name with a dot separator (p. 76), the method name and a set of parentheses containing the method's arguments.
- Local variables are not automatically initialized. Every instance variable has a default initial value—a value provided by Java when you do not specify the instance variable's initial value.
- The default value for an instance variable of type `String` is `null`.
- A method call supplies values—known as arguments—for each of the method's parameters. Each argument's value is assigned to the corresponding parameter in the method header.
- The number of arguments in a method call must match the number of parameters in the method declaration's parameter list.
- The argument types in the method call must be consistent with the types of the corresponding parameters in the method's declaration.

Section 3.2.3 Compiling and Executing an App with Multiple Classes

- The `javac` command can compile multiple classes at once. Simply list the source-code filenames after the command with each filename separated by a space from the next. If the directory containing the app includes only one app's files, you can compile all of its classes with the command `javac *.java`. The asterisk (*) in `*.java` indicates that all files in the current directory ending with the filename extension `".java"` should be compiled.

Section 3.2.4 Account UML Class Diagram with an Instance Variable and set and get Methods

- In the UML, each class is modeled in a class diagram (p. 77) as a rectangle with three compartments. The top one contains the class's name centered horizontally in boldface. The middle one contains the class's attributes, which correspond to instance variables in Java. The bottom one contains the class's operations (p. 78), which correspond to methods and constructors in Java.
- The UML represents instance variables as an attribute name, followed by a colon and the type.
- Private attributes are preceded by a minus sign (-) in the UML.
- The UML models operations by listing the operation name followed by a set of parentheses. A plus sign (+) in front of the operation name indicates that the operation is a public one in the UML (i.e., a `public` method in Java).
- The UML models a parameter of an operation by listing the parameter name, followed by a colon and the parameter type between the parentheses after the operation name.
- The UML indicates an operation's return type by placing a colon and the return type after the parentheses following the operation name.
- UML class diagrams do not specify return types for operations that do not return values.
- Declaring instance variables private is known as data hiding or information hiding.

Section 3.2.5 Additional Notes on Class `AccountTest`

- You must call most methods other than `main` explicitly to tell them to perform their tasks.
- A key part of enabling the JVM to locate and call method `main` to begin the app's execution is the `static` keyword, which indicates that `main` is a `static` method that can be called without first creating an object of the class in which the method is declared.
- Most classes you'll use in Java programs must be imported explicitly. There's a special relationship between classes that are compiled in the same directory. By default, such classes are considered to be in the same package—known as the default package. Classes in the same package are implicitly imported into the source-code files of other classes in that package. An `import` declaration is not required when one class in a package uses another in the same package.
- An `import` declaration is not required if you always refer to a class with its fully qualified class name, which includes its package name and class name.

Section 3.2.6 Software Engineering with **private** Instance Variables and **public** set and get Methods

- Declaring instance variables private is known as data hiding or information hiding.

Section 3.3 Primitive Types vs. Reference Types

- Types in Java are divided into two categories—primitive types and reference types. The primitive types are `boolean`, `byte`, `char`, `short`, `int`, `long`, `float` and `double`. All other types are reference types, so classes, which specify the types of objects, are reference types.
- A primitive-type variable can store exactly one value of its declared type at a time.
- Primitive-type instance variables are initialized by default. Variables of types `byte`, `char`, `short`, `int`, `long`, `float` and `double` are initialized to 0. Variables of type `boolean` are initialized to `false`.
- Reference-type variables (called references; p. 81) store the location of an object in the computer's memory. Such variables refer to objects in the program. The object that's referenced may contain many instance variables and methods.
- Reference-type instance variables are initialized by default to the value `null`.

- A reference to an object (p. 81) is required to invoke an object's methods. A primitive-type variable does not refer to an object and therefore cannot be used to invoke a method.

Section 3.4 Account Class: Initializing Objects with Constructors

- Each class you declare can optionally provide a constructor with parameters that can be used to initialize an object of a class when the object is created.
- Java requires a constructor call for every object that's created.
- Constructors can specify parameters but not return types.
- If a class does not define constructors, the compiler provides a default constructor (p. 83) with no parameters, and the class's instance variables are initialized to their default values.
- If you declare a constructor for a class, the compiler will *not* create a *default constructor* for that class.
- The UML models constructors in the third compartment of a class diagram. To distinguish a constructor from a class's operations, the UML places the word "constructor" between guillemets (« and »; p. 84) before the constructor's name.

Section 3.5 Account Class with a Balance; Floating-Point Numbers and Type `double`

- A floating-point number (p. 84) is a number with a decimal point. Java provides two primitive types for storing floating-point numbers in memory—`float` and `double` (p. 84).
- Variables of type `float` represent single-precision floating-point numbers and have seven significant digits. Variables of type `double` represent double-precision floating-point numbers. These require twice as much memory as `float` variables and provide 15 significant digits—approximately double the precision of `float` variables.
- Floating-point literals (p. 84) are of type `double` by default.
- Scanner method `nextDouble` (p. 88) returns a `double` value.
- The format specifier `%f` (p. 88) is used to output values of type `float` or `double`. The format specifier `%.2f` specifies that two digits of precision (p. 88) should be output to the right of the decimal point in the floating-point number.
- The default value for an instance variable of type `double` is 0.0, and the default value for an instance variable of type `int` is 0.

Self-Review Exercises

3.1 Fill in the blanks in each of the following:

- Each class declaration that begins with keyword _____ must be stored in a file that has exactly the same name as the class and ends with the `.java` filename extension.
- Keyword _____ in a class declaration is followed immediately by the class's name.
- Keyword _____ requests memory from the system to store an object, then calls the corresponding class's constructor to initialize the object.
- Each parameter must specify both a(n) _____ and a(n) _____.
- By default, classes that are compiled in the same directory are considered to be in the same package, known as the _____.
- Java provides two primitive types for storing floating-point numbers in memory: _____ and _____.
- Variables of type `double` represent _____ floating-point numbers.
- Scanner method _____ returns a `double` value.
- Keyword `public` is an access _____.

- j) Return type _____ indicates that a method will not return a value.
- k) Scanner method _____ reads characters until it encounters a newline character, then returns those characters as a `String`.
- l) Class `String` is in package _____.
- m) `A(n)` _____ is not required if you always refer to a class with its fully qualified class name.
- n) `A(n)` _____ is a number with a decimal point, such as 7.33, 0.0975 or 1000.12345.
- o) Variables of type `float` represent _____-precision floating-point numbers.
- p) The format specifier _____ is used to output values of type `float` or `double`.
- q) Types in Java are divided into two categories—_____ types and _____ types.

3.2 State whether each of the following is *true* or *false*. If *false*, explain why.

- a) By convention, method names begin with an uppercase first letter, and all subsequent words in the name begin with a capital first letter.
- b) An `import` declaration is not required when one class in a package uses another in the same package.
- c) Empty parentheses following a method name in a method declaration indicate that the method does not require any parameters to perform its task.
- d) A primitive-type variable can be used to invoke a method.
- e) Variables declared in the body of a particular method are known as instance variables and can be used in all methods of the class.
- f) Every method's body is delimited by left and right braces (`{` and `}`).
- g) Primitive-type local variables are initialized by default.
- h) Reference-type instance variables are initialized by default to the value `null`.
- i) Any class that contains `public static void main(String[] args)` can be used to execute an app.
- j) The number of arguments in the method call must match the number of parameters in the method declaration's parameter list.
- k) Floating-point values that appear in source code are known as floating-point literals and are type `float` by default.

3.3 What is the difference between a local variable and an instance variable?

3.4 Explain the purpose of a method parameter. What is the difference between a parameter and an argument?

Answers to Self-Review Exercises

3.1 a) `public`. b) `class`. c) `new`. d) type, name. e) default package. f) `float`, `double`. g) double-precision. h) `nextDouble`. i) modifier. j) `void`. k) `nextLine`. l) `java.lang`. m) `import` declaration. n) floating-point number. o) single. p) `%f`. q) primitive, reference.

3.2 a) False. By convention, method names begin with a lowercase first letter and all subsequent words in the name begin with a capital first letter. b) True. c) True. d) False. A primitive-type variable cannot be used to invoke a method—a reference to an object is required to invoke the object's methods. e) False. Such variables are called local variables and can be used only in the method in which they're declared. f) True. g) False. Primitive-type instance variables are initialized by default. Each local variable must explicitly be assigned a value. h) True. i) True. j) True. k) False. Such literals are of type `double` by default.

3.3 A local variable is declared in the body of a method and can be used only from the point at which it's declared through the end of the method declaration. An instance variable is declared in a class, but not in the body of any of the class's methods. Also, instance variables are accessible to all methods of the class. (We'll see an exception to this in Chapter 8.)

3.4 A parameter represents additional information that a method requires to perform its task. Each parameter required by a method is specified in the method's declaration. An argument is the actual value for a method parameter. When a method is called, the argument values are passed to the corresponding parameters of the method so that it can perform its task.

Exercises

3.5 (*Keyword new*) What's the purpose of keyword `new`? Explain what happens when you use it.

3.6 (*Default Constructors*) What is a default constructor? How are an object's instance variables initialized if a class has only a default constructor?

3.7 (*Instance Variables*) Explain the purpose of an instance variable.

3.8 (*Using Classes without Importing Them*) Most classes need to be imported before they can be used in an app. Why is every app allowed to use classes `System` and `String` without first importing them?

3.9 (*Using a Class without Importing It*) Explain how a program could use class `Scanner` without importing it.

3.10 (*set and get Methods*) Explain why a class might provide a *set* method and a *get* method for an instance variable.

3.11 (*Modified Account Class*) Modify class `Account` (Fig. 3.8) to provide a method called `withdraw` that withdraws money from an `Account`. Ensure that the withdrawal amount does not exceed the `Account`'s balance. If it does, the balance should be left unchanged and the method should print a message indicating "Withdrawal amount exceeded account balance." Modify class `AccountTest` (Fig. 3.9) to test method `withdraw`.

3.12 (*Invoice Class*) Create a class called `Invoice` that a hardware store might use to represent an invoice for an item sold at the store. An `Invoice` should include four pieces of information as instance variables—a part number (type `String`), a part description (type `String`), a quantity of the item being purchased (type `int`) and a price per item (type `double`). Your class should have a constructor that initializes the four instance variables. Provide a *set* and a *get* method for each instance variable. In addition, provide a method named `getInvoiceAmount` that calculates the invoice amount (i.e., multiplies the quantity by the price per item), then returns the amount as a `double` value. If the quantity is not positive, it should be set to 0. If the price per item is not positive, it should be set to 0.0. Write a test app named `InvoiceTest` that demonstrates class `Invoice`'s capabilities.

3.13 (*Employee Class*) Create a class called `Employee` that includes three instance variables—a first name (type `String`), a last name (type `String`) and a monthly salary (type `double`). Provide a constructor that initializes the three instance variables. Provide a *set* and a *get* method for each instance variable. If the monthly salary is not positive, do not set its value. Write a test app named `EmployeeTest` that demonstrates class `Employee`'s capabilities. Create two `Employee` objects and display each object's *yearly* salary. Then give each `Employee` a 10% raise and display each `Employee`'s *yearly* salary again.

3.14 (*Date Class*) Create a class called `Date` that includes three instance variables—a month (type `int`), a day (type `int`) and a year (type `int`). Provide a constructor that initializes the three instance variables and assumes that the values provided are correct. Provide a *set* and a *get* method for each instance variable. Provide a method `displayDate` that displays the month, day and year separated by forward slashes (/). Write a test app named `DateTest` that demonstrates class `Date`'s capabilities.

3.15 (*Removing Duplicated Code in Method main*) In the `AccountTest` class of Fig. 3.9, method `main` contains six statements (lines 13–14, 15–16, 28–29, 30–31, 40–41 and 42–43) that each display an `Account` object's `name` and `balance`. Study these statements and you'll notice that they differ

only in the `Account` object being manipulated—`account1` or `account2`. In this exercise, you'll define a new `displayAccount` method that contains *one* copy of that output statement. The method's parameter will be an `Account` object and the method will output the object's name and balance. You'll then replace the six duplicated statements in `main` with calls to `displayAccount`, passing as an argument the specific `Account` object to output.

Modify class `AccountTest` class of Fig. 3.9 to declare the following `displayAccount` method *after* the closing right brace of `main` and *before* the closing right brace of class `AccountTest`:

```
public static void displayAccount(Account accountToDisplay)
{
    // place the statement that displays
    // accountToDisplay's name and balance here
}
```

Replace the comment in the method's body with a statement that displays `accountToDisplay`'s name and balance.

Recall that `main` is a static method, so it can be called without first creating an object of the class in which `main` is declared. We also declared method `displayAccount` as a static method. When `main` needs to call another method in the same class without first creating an object of that class, the other method *also* must be declared static.

Once you've completed `displayAccount`'s declaration, modify `main` to replace the statements that display each `Account`'s name and balance with calls to `displayAccount`—each receiving as its argument the `account1` or `account2` object, as appropriate. Then, test the updated `AccountTest` class to ensure that it produces the same output as shown in Fig. 3.9.

Making a Difference

3.16 (Target-Heart-Rate Calculator) While exercising, you can use a heart-rate monitor to see that your heart rate stays within a safe range suggested by your trainers and doctors. According to the American Heart Association (AHA) (www.americanheart.org/presenter.jhtml?identifier=4736), the formula for calculating your *maximum heart rate* in beats per minute is 220 minus your age in years. Your *target heart rate* is a range that's 50–85% of your maximum heart rate. [Note: These formulas are estimates provided by the AHA. Maximum and target heart rates may vary based on the health, fitness and gender of the individual. **Always consult a physician or qualified health-care professional before beginning or modifying an exercise program.**] Create a class called `HeartRates`. The class attributes should include the person's first name, last name and date of birth (consisting of separate attributes for the month, day and year of birth). Your class should have a constructor that receives this data as parameters. For each attribute provide *set* and *get* methods. The class also should include a method that calculates and returns the person's age (in years), a method that calculates and returns the person's maximum heart rate and a method that calculates and returns the person's target heart rate. Write a Java app that prompts for the person's information, instantiates an object of class `HeartRates` and prints the information from that object—including the person's first name, last name and date of birth—then calculates and prints the person's age in (years), maximum heart rate and target-heart-rate range.

3.17 (Computerization of Health Records) A health-care issue that has been in the news lately is the computerization of health records. This possibility is being approached cautiously because of sensitive privacy and security concerns, among others. [We address such concerns in later exercises.] Computerizing health records could make it easier for patients to share their health profiles and histories among their various health-care professionals. This could improve the quality of health care, help avoid drug conflicts and erroneous drug prescriptions, reduce costs and, in emergencies, could save lives. In this exercise, you'll design a "starter" `HealthProfile` class for a person. The class attributes should include the person's first name, last name, gender, date of birth (consisting of separate

attributes for the month, day and year of birth), height (in inches) and weight (in pounds). Your class should have a constructor that receives this data. For each attribute, provide *set* and *get* methods. The class also should include methods that calculate and return the user's age in years, maximum heart rate and target-heart-rate range (see Exercise 3.16), and body mass index (BMI; see Exercise 2.33). Write a Java app that prompts for the person's information, instantiates an object of class `HealthProfile` for that person and prints the information from that object—including the person's first name, last name, gender, date of birth, height and weight—then calculates and prints the person's age in years, BMI, maximum heart rate and target-heart-rate range. It should also display the BMI values chart from Exercise 2.33.

Control Statements: Part I; Assignment, ++ and -- Operators

4



Let's all move one place on.
—Lewis Carroll

*How many apples fell on
Newton's head before he took the
hint!*
—Robert Frost

Objectives

In this chapter you'll:

- Learn basic problem-solving techniques.
- Develop algorithms through the process of top-down, stepwise refinement.
- Use the `if` and `if...else` selection statements to choose between alternative actions.
- Use the `while` repetition statement to execute statements in a program repeatedly.
- Use counter-controlled repetition and sentinel-controlled repetition.
- Use the compound assignment operator, and the increment and decrement operators.
- Learn about the portability of primitive data types.

- | | |
|---|---|
| 4.1 Introduction | 4.10 Formulating Algorithms: Sentinel-Controlled Repetition |
| 4.2 Algorithms | 4.11 Formulating Algorithms: Nested Control Statements |
| 4.3 Pseudocode | 4.12 Compound Assignment Operators |
| 4.4 Control Structures | 4.13 Increment and Decrement Operators |
| 4.5 if Single-Selection Statement | 4.14 Primitive Types |
| 4.6 if...else Double-Selection Statement | 4.15 (Optional) GUI and Graphics Case Study: Creating Simple Drawings |
| 4.7 Student Class: Nested if...else Statements | 4.16 Wrap-Up |
| 4.8 while Repetition Statement | |
| 4.9 Formulating Algorithms: Counter-Controlled Repetition | |

Summary | Self-Review Exercises | Answers to Self-Review Exercises | Exercises | Making a Difference

4.1 Introduction

Before writing a program to solve a problem, you should have a thorough understanding of the problem and a carefully planned approach to solving it. When writing a program, you also should understand the available building blocks and employ proven program-construction techniques. In this chapter and the next, we discuss these issues in presenting the theory and principles of structured programming. The concepts presented here are crucial in building classes and manipulating objects. We discuss Java's `if` statement in additional detail, and introduce the `if...else` and `while` statements—all of these building blocks allow you to specify the logic required for methods to perform their tasks. We also introduce the compound assignment operator and the increment and decrement operators. Finally, we consider the portability of Java's primitive types.

4.2 Algorithms

Any computing problem can be solved by executing a series of actions in a specific order. A *procedure* for solving a problem in terms of

1. the **actions** to execute and
2. the **order** in which these actions execute

is called an **algorithm**. The following example demonstrates that correctly specifying the order in which the actions execute is important.

Consider the “rise-and-shine algorithm” followed by one executive for getting out of bed and going to work: (1) Get out of bed; (2) take off pajamas; (3) take a shower; (4) get dressed; (5) eat breakfast; (6) carpool to work. This routine gets the executive to work well prepared to make critical decisions. Suppose that the same steps are performed in a slightly different order: (1) Get out of bed; (2) take off pajamas; (3) get dressed; (4) take a shower; (5) eat breakfast; (6) carpool to work. In this case, our executive shows up for work soaking wet. Specifying the order in which statements (actions) execute in a program is called **program control**. This chapter investigates program control using Java's **control statements**.

4.3 Pseudocode

Pseudocode is an informal language that helps you develop algorithms without having to worry about the strict details of Java language syntax. The pseudocode we present is particularly useful for developing algorithms that will be converted to structured portions of Java programs. The pseudocode we use in this book is similar to everyday English—it’s convenient and user friendly, but it’s not an actual computer programming language. You’ll see an algorithm written in pseudocode in Fig. 4.7. You may, of course, use your own native language(s) to develop your own pseudocode.

Pseudocode does not execute on computers. Rather, it helps you “think out” a program before attempting to write it in a programming language, such as Java. This chapter provides several examples of using pseudocode to develop Java programs.

The style of pseudocode we present consists purely of characters, so you can type pseudocode conveniently, using any text-editor program. A carefully prepared pseudocode program can easily be converted to a corresponding Java program.

Pseudocode normally describes only statements representing the *actions* that occur after you convert a program from pseudocode to Java and the program is run on a computer. Such actions might include *input*, *output* or *calculations*. In our pseudocode, we typically do *not* include variable declarations, but some programmers choose to list variables and mention their purposes.

4.4 Control Structures

Normally, statements in a program are executed one after the other in the order in which they’re written. This process is called **sequential execution**. Various Java statements, which we’ll soon discuss, enable you to specify that the next statement to execute is *not* necessarily the *next* one in sequence. This is called **transfer of control**.

During the 1960s, it became clear that the indiscriminate use of transfers of control was the root of much difficulty experienced by software development groups. The blame was pointed at the **goto statement** (used in most programming languages of the time), which allows you to specify a transfer of control to one of a wide range of destinations in a program. [Note: Java does *not* have a goto statement; however, the word goto is *reserved* by Java and should *not* be used as an identifier in programs.]

The research of Bohm and Jacopini¹ had demonstrated that programs could be written *without* any goto statements. The challenge of the era for programmers was to shift their styles to “goto-less programming.” The term **structured programming** became almost synonymous with “goto elimination.” Not until the 1970s did most programmers start taking structured programming seriously. The results were impressive. Software development groups reported shorter development times, more frequent on-time delivery of systems and more frequent within-budget completion of software projects. The key to these successes was that structured programs were clearer, easier to debug and modify, and more likely to be bug free in the first place.

Bohm and Jacopini’s work demonstrated that all programs could be written in terms of only three control structures—the **sequence structure**, the **selection structure** and the

1. C. Bohm, and G. Jacopini, “Flow Diagrams, Turing Machines, and Languages with Only Two Formation Rules,” *Communications of the ACM*, Vol. 9, No. 5, May 1966, pp. 336–371.

repetition structure. When we introduce Java’s control-structure implementations, we’ll refer to them in the terminology of the *Java Language Specification* as “control statements.”

Sequence Structure in Java

The sequence structure is built into Java. Unless directed otherwise, the computer executes Java statements one after the other in the order in which they’re written—that is, in sequence. The **activity diagram** in Fig. 4.1 illustrates a typical sequence structure in which two calculations are performed in order. Java lets you have as many actions as you want in a sequence structure. As we’ll soon see, anywhere a single action may be placed, we may place several actions in sequence.

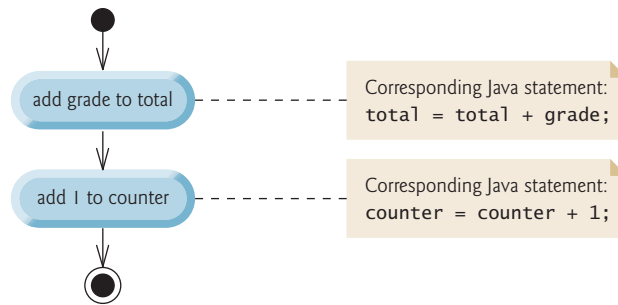


Fig. 4.1 | Sequence-structure activity diagram.

A UML activity diagram models the **workflow** (also called the **activity**) of a portion of a software system. Such workflows may include a portion of an algorithm, like the sequence structure in Fig. 4.1. Activity diagrams are composed of symbols, such as **action-state symbols** (rectangles with their left and right sides replaced with outward arcs), **diamonds** and **small circles**. These symbols are connected by **transition arrows**, which represent the *flow of the activity*—that is, the *order* in which the actions should occur.

Like pseudocode, activity diagrams help you develop and represent algorithms. Activity diagrams clearly show how control structures operate. We use the UML in this chapter and Chapter 5 to show control flow in control statements. Online Chapters 33–34 use the UML in a real-world automated-teller-machine case study.

Consider the sequence-structure activity diagram in Fig. 4.1. It contains two **action states**, each containing an **action expression**—for example, “add grade to total” or “add 1 to counter”—that specifies a particular action to perform. Other actions might include calculations or input/output operations. The arrows in the activity diagram represent **transitions**, which indicate the *order* in which the actions represented by the action states occur. The program that implements the activities illustrated by the diagram in Fig. 4.1 first adds grade to `total`, then adds 1 to `counter`.

The **solid circle** at the top of the activity diagram represents the **initial state**—the *beginning* of the workflow *before* the program performs the modeled actions. The **solid circle surrounded by a hollow circle** at the bottom of the diagram represents the **final state**—the *end* of the workflow *after* the program performs its actions.

Figure 4.1 also includes rectangles with the upper-right corners folded over. These are UML **notes** (like comments in Java)—explanatory remarks that describe the purpose of sym-

bols in the diagram. Figure 4.1 uses UML notes to show the Java code associated with each action state. A **dotted line** connects each note with the element it describes. Activity diagrams normally do *not* show the Java code that implements the activity. We do this here to illustrate how the diagram relates to Java code. For more information on the UML, see our optional online object-oriented design case study (Chapters 33–34) or visit www.uml.org.

Selection Statements in Java

Java has three types of **selection statements** (discussed in this chapter and Chapter 5). The *if statement* either performs (selects) an action, if a condition is *true*, or skips it, if the condition is *false*. The *if...else statement* performs an action if a condition is *true* and performs a different action if the condition is *false*. The *switch statement* (Chapter 5) performs one of *many* different actions, depending on the value of an expression.

The *if* statement is a **single-selection statement** because it selects or ignores a *single* action (or, as we'll soon see, a *single group of actions*). The *if...else* statement is called a **double-selection statement** because it selects between *two different actions* (or *groups of actions*). The *switch* statement is called a **multiple-selection statement** because it selects among *many different actions* (or *groups of actions*).

Repetition Statements in Java

Java provides three **repetition statements** (also called **iteration statements** or **looping statements**) that enable programs to perform statements repeatedly as long as a condition (called the **loop-continuation condition**) remains *true*. The repetition statements are the *while*, *do...while*, *for* and enhanced *for* statements. (Chapter 5 presents the *do...while* and *for* statements and Chapter 7 presents the enhanced *for* statement.) The *while* and *for* statements perform the action (or group of actions) in their bodies zero or more times—if the loop-continuation condition is initially *false*, the action (or group of actions) will *not* execute. The *do...while* statement performs the action (or group of actions) in its body *one or more* times. The words *if*, *else*, *switch*, *while*, *do* and *for* are Java keywords. A complete list of Java keywords appears in Appendix C.

Summary of Control Statements in Java

Java has only three kinds of control structures, which from this point forward we refer to as *control statements*: the *sequence statement*, *selection statements* (three types) and *repetition statements* (three types). Every program is formed by combining as many of these statements as is appropriate for the algorithm the program implements. We can model each control statement as an activity diagram. Like Fig. 4.1, each diagram contains an initial state and a final state that represent a control statement's entry point and exit point, respectively. **Single-entry/single-exit control statements** make it easy to build programs—we simply connect the exit point of one to the entry point of the next. We call this **control-statement stacking**. We'll learn that there's only one other way in which control statements may be connected—**control-statement nesting**—in which one control statement appears *inside* another. Thus, algorithms in Java programs are constructed from only three kinds of control statements, combined in only two ways. This is the essence of simplicity.

4.5 if Single-Selection Statement

Programs use selection statements to choose among alternative courses of action. For example, suppose that the passing grade on an exam is 60. The *pseudocode* statement

*If student's grade is greater than or equal to 60
Print "Passed"*

determines whether the *condition* “student’s grade is greater than or equal to 60” is *true*. If so, “Passed” is printed, and the next pseudocode statement in order is “performed.” (Remember, pseudocode is not a real programming language.) If the condition is *false*, the *Print* statement is ignored, and the next pseudocode statement in order is performed. The indentation of the second line of this selection statement is optional, but recommended, because it emphasizes the inherent structure of structured programs.

The preceding pseudocode *If* statement may be written in Java as

```
if (studentGrade >= 60)
    System.out.println("Passed");
```

The Java code corresponds closely to the pseudocode. This is one of the properties of pseudocode that makes it such a useful program development tool.

UML Activity Diagram for an *if* Statement

Figure 4.2 illustrates the single-selection *if* statement. This figure contains the most important symbol in an activity diagram—the *diamond*, or **decision symbol**, which indicates that a *decision* is to be made. The workflow continues along a path determined by the symbol’s associated **guard conditions**, which can be *true* or *false*. Each transition arrow emerging from a decision symbol has a guard condition (specified in *square brackets* next to the arrow). If a guard condition is *true*, the workflow enters the action state to which the transition arrow points. In Fig. 4.2, if the grade is greater than or equal to 60, the program prints “Passed,” then transitions to the activity’s final state. If the grade is less than 60, the program immediately transitions to the final state without displaying a message.

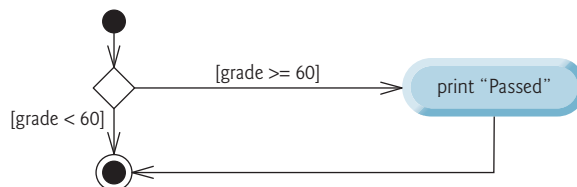


Fig. 4.2 | *if* single-selection statement UML activity diagram.

The *if* statement is a single-entry/single-exit control statement. We’ll see that the activity diagrams for the remaining control statements also contain initial states, transition arrows, action states that indicate actions to perform, decision symbols (with associated guard conditions) that indicate decisions to be made, and final states.

4.6 *if...else* Double-Selection Statement

The *if* single-selection statement performs an indicated action only when the condition is *true*; otherwise, the action is skipped. The ***if...else* double-selection statement** allows you to specify an action to perform when the condition is *true* and another action when the condition is *false*. For example, the pseudocode statement

*If student's grade is greater than or equal to 60
Print "Passed"*
*Else
Print "Failed"*

prints "Passed" if the student's grade is greater than or equal to 60, but prints "Failed" if it's less than 60. In either case, after printing occurs, the next pseudocode statement in sequence is "performed."

The preceding *If...Else* pseudocode statement can be written in Java as

```
if (grade >= 60)
    System.out.println("Passed");
else
    System.out.println("Failed");
```

The body of the `else` is also indented. Whatever indentation convention you choose should be applied consistently throughout your programs.



Good Programming Practice 4.1

Indent both body statements of an if...else statement. Many IDEs do this for you.



Good Programming Practice 4.2

If there are several levels of indentation, each level should be indented the same additional amount of space.

UML Activity Diagram for an if...else Statement

Figure 4.3 illustrates the flow of control in the `if...else` statement. Once again, the symbols in the UML activity diagram (besides the initial state, transition arrows and final state) represent action states and decisions.

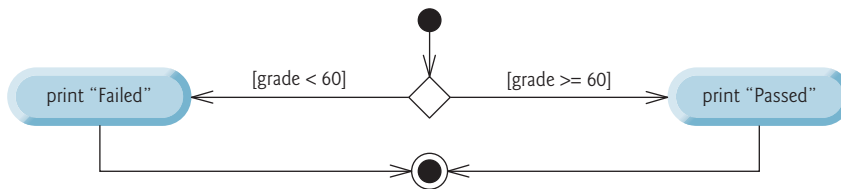


Fig. 4.3 | `if...else` double-selection statement UML activity diagram.

Nested if...else Statements

A program can test multiple cases by placing `if...else` statements inside other `if...else` statements to create **nested if...else statements**. For example, the following pseudocode represents a nested `if...else` that prints A for exam grades greater than or equal to 90, B for grades 80 to 89, C for grades 70 to 79, D for grades 60 to 69 and F for all other grades:

```

If student's grade is greater than or equal to 90
    Print "A"
else
    If student's grade is greater than or equal to 80
        Print "B"
    else
        If student's grade is greater than or equal to 70
            Print "C"
        else
            If student's grade is greater than or equal to 60
                Print "D"
            else
                Print "F"

```

This pseudocode may be written in Java as

```

if (studentGrade >= 90)
    System.out.println("A");
else
    if (studentGrade >= 80)
        System.out.println("B");
    else
        if (studentGrade >= 70)
            System.out.println("C");
        else
            if (studentGrade >= 60)
                System.out.println("D");
            else
                System.out.println("F");

```



Error-Prevention Tip 4.1

In a nested if...else statement, ensure that you test for all possible cases.

If variable `studentGrade` is greater than or equal to 90, the first four conditions in the nested if...else statement will be true, but only the statement in the if part of the first if...else statement will execute. After that statement executes, the else part of the “outermost” if...else statement is skipped. Many programmers prefer to write the preceding nested if...else statement as

```

if (studentGrade >= 90)
    System.out.println("A");
else if (studentGrade >= 80)
    System.out.println("B");
else if (studentGrade >= 70)
    System.out.println("C");
else if (studentGrade >= 60)
    System.out.println("D");
else
    System.out.println("F");

```

The two forms are identical except for the spacing and indentation, which the compiler ignores. The latter form avoids deep indentation of the code to the right. Such indentation often leaves little room on a line of source code, forcing lines to be split.

Dangling-else Problem

The Java compiler always associates an `else` with the immediately preceding `if` unless told to do otherwise by the placement of braces (`{` and `}`). This behavior can lead to what is referred to as the **dangling-else problem**. For example,

```
if (x > 5)
    if (y > 5)
        System.out.println("x and y are > 5");
else
    System.out.println("x is <= 5");
```

appears to indicate that if `x` is greater than 5, the nested `if` statement determines whether `y` is also greater than 5. If so, the string "x and y are > 5" is output. Otherwise, it appears that if `x` is not greater than 5, the `else` part of the `if...else` outputs the string "x is <= 5". Beware! This nested `if...else` statement does *not* execute as it appears. The compiler actually interprets the statement as

```
if (x > 5)
    if (y > 5)
        System.out.println("x and y are > 5");
else
    System.out.println("x is <= 5");
```

in which the body of the first `if` is a *nested* `if...else`. The outer `if` statement tests whether `x` is greater than 5. If so, execution continues by testing whether `y` is also greater than 5. If the second condition is *true*, the proper string—"x and y are > 5"—is displayed. However, if the second condition is *false*, the string "x is <= 5" is displayed, even though we know that `x` is greater than 5. Equally bad, if the outer `if` statement's condition is false, the inner `if...else` is skipped and nothing is displayed.

To force the nested `if...else` statement to execute as it was originally intended, we must write it as follows:

```
if (x > 5)
{
    if (y > 5)
        System.out.println("x and y are > 5");
}
else
    System.out.println("x is <= 5");
```

The braces indicate that the second `if` is in the body of the first and that the `else` is associated with the *first* `if`. Exercises 4.27–4.28 investigate the dangling-else problem further.

Blocks

The `if` statement normally expects only *one* statement in its body. To include *several* statements in the body of an `if` (or the body of an `else` for an `if...else` statement), enclose the statements in braces. Statements contained in a pair of braces (such as the body of a

method) form a **block**. A block can be placed anywhere in a method that a single statement can be placed.

The following example includes a block in the `else` part of an `if...else` statement:

```
if (grade >= 60)
    System.out.println("Passed");
else
{
    System.out.println("Failed");
    System.out.println("You must take this course again.");
}
```

In this case, if grade is less than 60, the program executes *both* statements in the body of the `else` and prints

```
Failed
You must take this course again.
```

Note the braces surrounding the two statements in the `else` clause. These braces are important. Without the braces, the statement

```
System.out.println("You must take this course again.");
```

would be outside the body of the `else` part of the `if...else` statement and would execute *regardless* of whether the grade was less than 60.

Syntax errors (e.g., when one brace in a block is left out of the program) are caught by the compiler. A **logic error** (e.g., when both braces in a block are left out of the program) has its effect at execution time. A **fatal logic error** causes a program to fail and terminate prematurely. A **nonfatal logic error** allows a program to continue executing but causes it to produce incorrect results.

Just as a block can be placed anywhere a single statement can be placed, it's also possible to have an empty statement. Recall from Section 2.8 that the empty statement is represented by placing a semicolon (;) where a statement would normally be.



Common Programming Error 4.1

Placing a semicolon after the condition in an `if` or `if...else` statement leads to a logic error in single-selection `if` statements and a syntax error in double-selection `if...else` statements (when the `if`-part contains an actual body statement).

Conditional Operator (?:)

Java provides the **conditional operator** (`?:`) that can be used in place of an `if...else` statement. This can make your code shorter and clearer. The conditional operator is Java's only **ternary operator** (i.e., an operator that takes *three* operands). Together, the operands and the `?:` symbol form a **conditional expression**. The first operand (to the left of the `?`) is a **boolean expression** (i.e., a *condition* that evaluates to a boolean value—**true** or **false**), the second operand (between the `?` and `:`) is the value of the conditional expression if the boolean expression is true and the third operand (to the right of the `:`) is the value of the conditional expression if the boolean expression evaluates to false. For example, the statement

```
System.out.println(studentGrade >= 60 ? "Passed" : "Failed");
```

prints the value of `println`'s conditional-expression argument. The conditional expression in this statement evaluates to the string "Passed" if the boolean expression `student-Grade >= 60` is true and to the string "Failed" if it's false. Thus, this statement with the conditional operator performs essentially the same function as the `if...else` statement shown earlier in this section. The precedence of the conditional operator is low, so the entire conditional expression is normally placed in parentheses. We'll see that conditional expressions can be used in some situations where `if...else` statements cannot.



Error-Prevention Tip 4.2

Use expressions of the same type for the second and third operands of the `?:` operator to avoid subtle errors.

4.7 Student Class: Nested `if...else` Statements

The example of Figs. 4.4–4.5 demonstrates a nested `if...else` statement that determines a student's letter grade based on the student's average in a course.

Class *Student*

Class *Student* (Fig. 4.4) has features similar to those of class *Account* (discussed in Chapter 3). Class *Student* stores a student's name and average and provides methods for manipulating these values. The class contains:

- instance variable `name` of type `String` (line 5) to store a Student's name
- instance variable `average` of type `double` (line 6) to store a Student's average in a course
- a constructor (lines 9–18) that initializes the name and average—in Section 5.9, you'll learn how to express lines 15–16 and 37–38 more concisely with logical operators that can test multiple conditions
- methods `setName` and `getName` (lines 21–30) to *set* and *get* the Student's name
- methods `setAverage` and `getAverage` (lines 33–46) to *set* and *get* the Student's average
- method `getLetterGrade` (lines 49–65), which uses *nested if...else statements* to determine the Student's *letter grade* based on the Student's average

The constructor and method `setAverage` each use *nested if statements* (lines 15–17 and 37–39) to *validate* the value used to set the average—these statements ensure that the value is greater than 0.0 *and* less than or equal to 100.0; otherwise, `average`'s value is left *unchanged*. Each `if` statement contains a *simple* condition. If the condition in line 15 is *true*, only then will the condition in line 16 be tested, and *only* if the conditions in both line 15 *and* line 16 are *true* will the statement in line 17 execute.



Software Engineering Observation 4.1

Recall from Chapter 3 that you should not call methods from constructors (we'll explain why in Chapter 10, *Object-Oriented Programming: Polymorphism and Interfaces*). For this reason, there is duplicated validation code in lines 15–17 and 37–39 of Fig. 4.4 and in subsequent examples.

```
1 // Fig. 4.4: Student.java
2 // Student class that stores a student name and average.
3 public class Student
4 {
5     private String name;
6     private double average;
7
8     // constructor initializes instance variables
9     public Student(String name, double average)
10    {
11        this.name = name;
12
13        // validate that average is > 0.0 and <= 100.0; otherwise,
14        // keep instance variable average's default value (0.0)
15        if (average > 0.0)
16            if (average <= 100.0)
17                this.average = average; // assign to instance variable
18    }
19
20    // sets the Student's name
21    public void setName(String name)
22    {
23        this.name = name;
24    }
25
26    // retrieves the Student's name
27    public String getName()
28    {
29        return name;
30    }
31
32    // sets the Student's average
33    public void setAverage(double studentAverage)
34    {
35        // validate that average is > 0.0 and <= 100.0; otherwise,
36        // keep instance variable average's current value
37        if (average > 0.0)
38            if (average <= 100.0)
39                this.average = average; // assign to instance variable
40    }
41
42    // retrieves the Student's average
43    public double getAverage()
44    {
45        return average;
46    }
47
48    // determines and returns the Student's letter grade
49    public String getLetterGrade()
50    {
51        String letterGrade = ""; // initialized to empty String
52    }
```

Fig. 4.4 | Student class that stores a student name and average. (Part I of 2.)

```

53     if (average >= 90.0)
54         letterGrade = "A";
55     else if (average >= 80.0)
56         letterGrade = "B";
57     else if (average >= 70.0)
58         letterGrade = "C";
59     else if (average >= 60.0)
60         letterGrade = "D";
61     else
62         letterGrade = "F";
63
64     return letterGrade;
65 }
66 } // end class Student

```

Fig. 4.4 | Student class that stores a student name and average. (Part 2 of 2.)

Class StudentTest

To demonstrate the nested if...else statements in class Student's getLetterGrade method, class StudentTest's main method (Fig. 4.5) creates two Student objects (lines 7–8). Next, lines 10–13 display each Student's name and letter grade by calling the objects' getName and getLetterGrade methods, respectively.

```

1 // Fig. 4.5: StudentTest.java
2 // Create and test Student objects.
3 public class StudentTest
4 {
5     public static void main(String[] args)
6     {
7         Student account1 = new Student("Jane Green", 93.5);
8         Student account2 = new Student("John Blue", 72.75);
9
10        System.out.printf("%s's letter grade is: %s\n",
11            account1.getName(), account1.getLetterGrade());
12        System.out.printf("%s's letter grade is: %s\n",
13            account2.getName(), account2.getLetterGrade());
14    }
15 } // end class StudentTest

```

```

Jane Green's letter grade is: A
John Blue's letter grade is: C

```

Fig. 4.5 | Create and test Student objects.

4.8 while Repetition Statement

A repetition statement allows you to specify that a program should repeat an action while some condition remains *true*. The pseudocode statement

*While there are more items on my shopping list
Purchase next item and cross it off my list*

describes the repetition during a shopping trip. The condition “there are more items on my shopping list” may be true or false. If it’s *true*, then the action “Purchase next item and cross it off my list” is performed. This action will be performed *repeatedly* while the condition remains *true*. The statement(s) contained in the *While* repetition statement constitute its body, which may be a single statement or a block. Eventually, the condition will become *false* (when the shopping list’s last item has been purchased and crossed off). At this point, the repetition terminates, and the first statement after the repetition statement executes.

As an example of Java’s **while repetition statement**, consider a program segment that finds the first power of 3 larger than 100. Suppose that the `int` variable `product` is initialized to 3. After the following `while` statement executes, `product` contains the result:

```
while (product <= 100)
    product = 3 * product;
```

Each iteration of the `while` statement multiplies `product` by 3, so `product` takes on the values 9, 27, 81 and 243 successively. When `product` becomes 243, `product <= 100` becomes false. This terminates the repetition, so the final value of `product` is 243. At this point, program execution continues with the next statement after the `while` statement.



Common Programming Error 4.2

*Not providing in the body of a `while` statement an action that eventually causes the condition in the `while` to become false normally results in a logic error called an **infinite loop** (the loop never terminates).*

UML Activity Diagram for a `while` Statement

The UML activity diagram in Fig. 4.6 illustrates the flow of control in the preceding `while` statement. Once again, the symbols in the diagram (besides the initial state, transition arrows, a final state and three notes) represent an action state and a decision. This diagram introduces the UML’s **merge symbol**. The UML represents both the merge symbol and the decision symbol as diamonds. The merge symbol joins two flows of activity into one. In this diagram, the merge symbol joins the transitions from the initial state and from the action state, so they both flow into the decision that determines whether the loop should begin (or continue) executing.

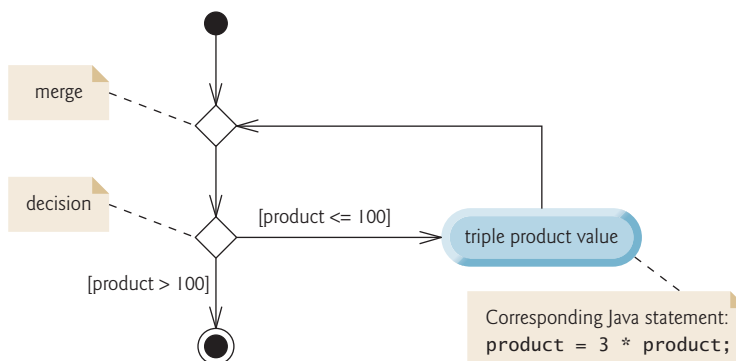


Fig. 4.6 | `while` repetition statement UML activity diagram.

The decision and merge symbols can be distinguished by the number of “incoming” and “outgoing” transition arrows. A decision symbol has one transition arrow pointing to the diamond and two or more pointing out from it to indicate possible transitions from that point. In addition, each transition arrow pointing out of a decision symbol has a guard condition next to it. A merge symbol has two or more transition arrows pointing to the diamond and only one pointing from the diamond, to indicate multiple activity flows merging to continue the activity. *None* of the transition arrows associated with a merge symbol has a guard condition.

Figure 4.6 clearly shows the repetition of the `while` statement discussed earlier in this section. The transition arrow emerging from the action state points back to the merge, from which program flow transitions back to the decision that’s tested at the beginning of each iteration of the loop. The loop continues to execute until the guard condition `product > 100` becomes true. Then the `while` statement exits (reaches its final state), and control passes to the next statement in sequence in the program.

4.9 Formulating Algorithms: Counter-Controlled Repetition

To illustrate how algorithms are developed, we solve two variations of a problem that averages student grades. Consider the following problem statement:

A class of ten students took a quiz. The grades (integers in the range 0–100) for this quiz are available to you. Determine the class average on the quiz.

The class average is equal to the sum of the grades divided by the number of students. The algorithm for solving this problem on a computer must input each grade, keep track of the total of all grades input, perform the averaging calculation and print the result.

Pseudocode Algorithm with Counter-Controlled Repetition

Let’s use pseudocode to list the actions to execute and specify the order in which they should execute. We use **counter-controlled repetition** to input the grades one at a time. This technique uses a variable called a **counter** (or **control variable**) to control the number of times a set of statements will execute. Counter-controlled repetition is often called **definite repetition**, because the number of repetitions is known *before* the loop begins executing. In this example, repetition terminates when the counter exceeds 10. This section presents a fully developed pseudocode algorithm (Fig. 4.7) and a corresponding Java program (Fig. 4.8) that implements the algorithm. In Section 4.10, we demonstrate how to use pseudocode to develop such an algorithm from scratch.

Note the references in the algorithm of Fig. 4.7 to a total and a counter. A **total** is a variable used to accumulate the sum of several values. A counter is a variable used to count—in this case, the grade counter indicates which of the 10 grades is about to be entered by the user. Variables used to store totals are normally initialized to zero before being used in a program.



Software Engineering Observation 4.2

Experience has shown that the most difficult part of solving a problem on a computer is developing the algorithm for the solution. Once a correct algorithm has been specified, producing a working Java program from it is usually straightforward.

```

1  Set total to zero
2  Set grade counter to one
3
4  While grade counter is less than or equal to ten
5      Prompt the user to enter the next grade
6      Input the next grade
7      Add the grade into the total
8      Add one to the grade counter
9
10 Set the class average to the total divided by ten
11 Print the class average

```

Fig. 4.7 | Pseudocode algorithm that uses counter-controlled repetition to solve the class-average problem.

Implementing Counter-Controlled Repetition

In Fig. 4.8, class `ClassAverage`'s main method (lines 7–31) implements the class-averaging algorithm described by the pseudocode in Fig. 4.7—it allows the user to enter 10 grades, then calculates and displays the average.

```

1  // Fig. 4.8: ClassAverage.java
2  // Solving the class-average problem using counter-controlled repetition.
3  import java.util.Scanner; // program uses class Scanner
4
5  public class ClassAverage
6  {
7      public static void main(String[] args)
8      {
9          // create Scanner to obtain input from command window
10         Scanner input = new Scanner(System.in);
11
12         // initialization phase
13         int total = 0; // initialize sum of grades entered by the user
14         int gradeCounter = 1; // initialize # of grade to be entered next
15
16         // processing phase uses counter-controlled repetition
17         while (gradeCounter <= 10) // loop 10 times
18         {
19             System.out.print("Enter grade: "); // prompt
20             int grade = input.nextInt(); // input next grade
21             total = total + grade; // add grade to total
22             gradeCounter = gradeCounter + 1; // increment counter by 1
23         }
24
25         // termination phase
26         int average = total / 10; // integer division yields integer result
27
28         // display total and average of grades
29         System.out.printf("\nTotal of all 10 grades is %d\n", total);

```

Fig. 4.8 | Solving the class-average problem using counter-controlled repetition. (Part I of 2.)


```
30         System.out.printf("Class average is %d\n", average);
31     }
32 } // end class ClassAverage
```

```
Enter grade: 67
Enter grade: 78
Enter grade: 89
Enter grade: 67
Enter grade: 87
Enter grade: 98
Enter grade: 93
Enter grade: 85
Enter grade: 82
Enter grade: 100
```

```
Total of all 10 grades is 846
Class average is 84
```

Fig. 4.8 | Solving the class-average problem using counter-controlled repetition. (Part 2 of 2.)

Local Variables in Method main

Line 10 declares and initializes Scanner variable `input`, which is used to read values entered by the user. Lines 13, 14, 20 and 26 declare local variables `total`, `gradeCounter`, `grade` and `average`, respectively, to be of type `int`. Variable `grade` stores the user input.

These declarations appear in the body of method `main`. Recall that variables declared in a method body are local variables and can be used only from the line of their declaration to the closing right brace of the method declaration. A local variable's declaration must appear *before* the variable is used in that method. A local variable cannot be accessed outside the method in which it's declared. Variable `grade`, declared in the body of the `while` loop, can be used only in that block.

Initialization Phase: Initializing Variables `total` and `gradeCounter`

The assignments (in lines 13–14) initialize `total` to 0 and `gradeCounter` to 1. These initializations occur *before* the variables are used in calculations.



Common Programming Error 4.3

Using the value of a local variable before it's initialized results in a compilation error. All local variables must be initialized before their values are used in expressions.



Error-Prevention Tip 4.3

Initialize each total and counter, either in its declaration or in an assignment statement. Totals are normally initialized to 0. Counters are normally initialized to 0 or 1, depending on how they're used (we'll show examples of when to use 0 and when to use 1).

Processing Phase: Reading 10 Grades from the User

Line 17 indicates that the `while` statement should continue looping (also called **iterating**) as long as `gradeCounter`'s value is less than or equal to 10. While this condition remains *true*, the `while` statement repeatedly executes the statements between the braces that delimit its body (lines 18–23).

Line 19 displays the prompt "Enter grade: ". Line 20 reads the grade entered by the user and assigns it to variable `grade`. Then line 21 adds the new grade entered by the user to the `total` and assigns the result to `total`, which replaces its previous value.

Line 22 adds 1 to `gradeCounter` to indicate that the program has processed a grade and is ready to input the next grade from the user. Incrementing `gradeCounter` eventually causes it to exceed 10. Then the loop terminates, because its condition (line 17) becomes *false*.

Termination Phase: Calculating and Displaying the Class Average

When the loop terminates, line 26 performs the averaging calculation and assigns its result to the variable `average`. Line 29 uses `System.out`'s `printf` method to display the text "Total of all 10 grades is " followed by variable `total`'s value. Line 30 then uses `printf` to display the text "Class average is " followed by variable `average`'s value. When execution reaches line 31, the program terminates.

Notice that this example contains only one class, with method `main` performing all the work. In this chapter and in Chapter 3, you've seen examples consisting of two classes—one containing instance variables and methods that perform tasks using those variables and one containing method `main`, which creates an object of the other class and calls its methods. Occasionally, when it does not make sense to try to create a reusable class to demonstrate a concept, we'll place the program's statements entirely within a single class's `main` method.

Notes on Integer Division and Truncation

The averaging calculation performed by method `main` produces an integer result. The program's output indicates that the sum of the grade values in the sample execution is 846, which, when divided by 10, should yield the floating-point number 84.6. However, the result of the calculation `total / 10` (line 26 of Fig. 4.8) is the integer 84, because `total` and 10 are both integers. Dividing two integers results in **integer division**—any fractional part of the calculation is **truncated** (i.e., *lost*). In the next section we'll see how to obtain a floating-point result from the averaging calculation.



Common Programming Error 4.4

Assuming that integer division rounds (rather than truncates) can lead to incorrect results. For example, $7 \div 4$, which yields 1.75 in conventional arithmetic, truncates to 1 in integer arithmetic, rather than rounding to 2.

A Note About Arithmetic Overflow

In Fig. 4.8, line 21

```
total = total + grade; // add grade to total
```

added each grade entered by the user to the `total`. Even this simple statement has a *potential* problem—adding the integers could result in a value that's *too large* to store in an `int` variable. This is known as **arithmetic overflow** and causes *undefined behavior*, which can lead to unintended results (http://en.wikipedia.org/wiki/Integer_overflow#Security_ramifications). Figure 2.7's Addition program had the same issue in line 23, which calculated the sum of two `int` values entered by the user:

```
sum = number1 + number2; // add numbers, then store total in sum
```

The maximum and minimum values that can be stored in an `int` variable are represented by the constants `MIN_VALUE` and `MAX_VALUE`, respectively, which are defined in class

Integer. There are similar constants for the other integral types and for floating-point types. Each primitive type has a corresponding class type in package `java.lang`. You can see the values of these constants in each class's online documentation. The online documentation for class Integer is located at:

```
http://docs.oracle.com/javase/7/docs/api/java/lang/Integer.html
```

It's considered a good practice to ensure, *before* you perform arithmetic calculations like those in line 21 of Fig. 4.8 and line 23 of Fig. 2.7, that they will *not* overflow. The code for doing this is shown on the CERT website www.securecoding.cert.org—just search for guideline “NUM00-J.” The code uses the `&&` (logical AND) and `||` (logical OR) operators, which are introduced in Chapter 5. In industrial-strength code, you should perform checks like these for *all* calculations.

A Deeper Look at Receiving User Input

Any time a program receives input from the user, various problems might occur. For example, in line 20 of Fig. 4.8

```
int grade = input.nextInt(); // input next grade
```

we assume that the user will enter an integer grade in the range 0 to 100. However, the person entering a grade could enter an integer less than 0, an integer greater than 100, an integer outside the range of values that can be stored in an `int` variable, a number containing a decimal point or a value containing letters or special symbols that's not even an integer.

To ensure that inputs are valid, industrial-strength programs must test for all possible erroneous cases. A program that inputs grades should **validate** the grades by using **range checking** to ensure that they are values from 0 to 100. You can then ask the user to reenter any value that's out of range. If a program requires inputs from a specific set of values (e.g., nonsequential product codes), you can ensure that each input matches a value in the set.

4.10 Formulating Algorithms: Sentinel-Controlled Repetition

Let's generalize Section 4.9's class-average problem. Consider the following problem:

Develop a class-averaging program that processes grades for an arbitrary number of students each time it's run.

In the previous class-average example, the problem statement specified the number of students, so the number of grades (10) was known in advance. In this example, no indication is given of how many grades the user will enter during the program's execution. The program must process an arbitrary number of grades. How can it determine when to stop the input of grades? How will it know when to calculate and print the class average?

One way to solve this problem is to use a special value called a **sentinel value** (also called a **signal value**, a **dummy value** or a **flag value**) to indicate “end of data entry.” The user enters grades until all legitimate grades have been entered. The user then types the sentinel value to indicate that no more grades will be entered. **Sentinel-controlled repetition** is often called **indefinite repetition** because the number of repetitions is *not* known before the loop begins executing.

Clearly, a sentinel value must be chosen that cannot be confused with an acceptable input value. Grades on a quiz are nonnegative integers, so -1 is an acceptable sentinel value for this problem. Thus, a run of the class-average program might process a stream of inputs such as 95, 96, 75, 74, 89 and -1 . The program would then compute and print the class average for the grades 95, 96, 75, 74 and 89; since -1 is the sentinel value, it should *not* enter into the averaging calculation.

Developing the Pseudocode Algorithm with Top-Down, Stepwise Refinement: The Top and First Refinement

We approach this class-average program with a technique called **top-down, stepwise refinement**, which is essential to the development of well-structured programs. We begin with a pseudocode representation of the **top**—a single statement that conveys the overall function of the program:

Determine the class average for the quiz

The top is, in effect, a *complete* representation of a program. Unfortunately, the top rarely conveys sufficient detail from which to write a Java program. So we now begin the refinement process. We divide the top into a series of smaller tasks and list these in the order in which they'll be performed. This results in the following **first refinement**:

*Initialize variables
Input, sum and count the quiz grades
Calculate and print the class average*

This refinement uses only the *sequence structure*—the steps listed should execute in order, one after the other.



Software Engineering Observation 4.3

Each refinement, as well as the top itself, is a complete specification of the algorithm—only the level of detail varies.



Software Engineering Observation 4.4

Many programs can be divided logically into three phases: an initialization phase that initializes the variables; a processing phase that inputs data values and adjusts program variables accordingly; and a termination phase that calculates and outputs the final results.

Proceeding to the Second Refinement

The preceding Software Engineering Observation is often all you need for the first refinement in the top-down process. To proceed to the next level of refinement—that is, the **second refinement**—we commit to specific variables. In this example, we need a running total of the numbers, a count of how many numbers have been processed, a variable to receive the value of each grade as it's input by the user and a variable to hold the calculated average. The pseudocode statement

Initialize variables

can be refined as follows:

*Initialize total to zero
Initialize counter to zero*

Only the variables *total* and *counter* need to be initialized before they're used. The variables *average* and *grade* (for the calculated average and the user input, respectively) need not be initialized, because their values will be replaced as they're calculated or input.

The pseudocode statement

Input, sum and count the quiz grades

requires *repetition* to successively input each grade. We do not know in advance how many grades will be entered, so we'll use sentinel-controlled repetition. The user enters grades one at a time. After entering the last grade, the user enters the sentinel value. The program tests for the sentinel value after each grade is input and terminates the loop when the user enters the sentinel value. The second refinement of the preceding pseudocode statement is then

Prompt the user to enter the first grade
Input the first grade (possibly the sentinel)
While the user has not yet entered the sentinel
 Add this grade into the running total
 Add one to the grade counter
 Prompt the user to enter the next grade
 Input the next grade (possibly the sentinel)

In pseudocode, we do *not* use braces around the statements that form the body of the *While* structure. We simply indent the statements under the *While* to show that they belong to the *While*. Again, pseudocode is only an informal program development aid.

The pseudocode statement

Calculate and print the class average

can be refined as follows:

If the counter is not equal to zero
 Set the average to the total divided by the counter
 Print the average
else
 Print "No grades were entered"

We're careful here to test for the possibility of *division by zero*—a *logic error* that, if undetected, would cause the program to fail or produce invalid output. The complete second refinement of the pseudocode for the class-average problem is shown in Fig. 4.9.



Error-Prevention Tip 4.4

When performing division (/) or remainder (%) calculations in which the right operand could be zero, test for this and handle it (e.g., display an error message) rather than allowing the error to occur.

- 1 *Initialize total to zero*
- 2 *Initialize counter to zero*
- 3

Fig. 4.9 | Class-average pseudocode algorithm with sentinel-controlled repetition. (Part 1 of 2.)

```

4  Prompt the user to enter the first grade
5  Input the first grade (possibly the sentinel)
6
7  While the user has not yet entered the sentinel
8      Add this grade into the running total
9      Add one to the grade counter
10 Prompt the user to enter the next grade
11 Input the next grade (possibly the sentinel)
12
13 If the counter is not equal to zero
14     Set the average to the total divided by the counter
15     Print the average
16 else
17     Print "No grades were entered"

```

Fig. 4.9 | Class-average pseudocode algorithm with sentinel-controlled repetition. (Part 2 of 2.)

In Fig. 4.7 and Fig. 4.9, we included blank lines and indentation in the pseudocode to make it more readable. The blank lines separate the algorithms into their phases and set off control statements; the indentation emphasizes the bodies of the control statements.

The pseudocode algorithm in Fig. 4.9 solves the more general class-average problem. This algorithm was developed after two refinements. Sometimes more are needed.



Software Engineering Observation 4.5

Terminate the top-down, stepwise refinement process when you've specified the pseudocode algorithm in sufficient detail for you to convert the pseudocode to Java. Normally, implementing the Java program is then straightforward.



Software Engineering Observation 4.6

Some programmers do not use program development tools like pseudocode. They feel that their ultimate goal is to solve the problem on a computer and that writing pseudocode merely delays the production of final outputs. Although this may work for simple and familiar problems, it can lead to serious errors and delays in large, complex projects.

Implementing Sentinel-Controlled Repetition

In Fig. 4.10, method `main` (lines 7–46) implements the pseudocode algorithm of Fig. 4.9. Although each grade is an integer, the averaging calculation is likely to produce a number with a *decimal point*—in other words, a real (floating-point) number. The type `int` cannot represent such a number, so this class uses type `double` to do so. You'll also see that control statements may be *stacked* on top of one another (in sequence). The `while` statement (lines 22–30) is followed in sequence by an `if...else` statement (lines 34–45). Much of the code in this program is identical to that in Fig. 4.8, so we concentrate on the new concepts.

```

1  // Fig. 4.10: ClassAverage.java
2  // Solving the class-average problem using sentinel-controlled repetition.
3  import java.util.Scanner; // program uses class Scanner

```

Fig. 4.10 | Solving the class-average problem using sentinel-controlled repetition. (Part 1 of 2.)

```

4
5 public class ClassAverage
6 {
7     public static void main(String[] args)
8     {
9         // create Scanner to obtain input from command window
10        Scanner input = new Scanner(System.in);
11
12        // initialization phase
13        int total = 0; // initialize sum of grades
14        int gradeCounter = 0; // initialize # of grades entered so far
15
16        // processing phase
17        // prompt for input and read grade from user
18        System.out.print("Enter grade or -1 to quit: ");
19        int grade = input.nextInt();
20
21        // loop until sentinel value read from user
22        while (grade != -1)
23        {
24            total = total + grade; // add grade to total
25            gradeCounter = gradeCounter + 1; // increment counter
26
27            // prompt for input and read next grade from user
28            System.out.print("Enter grade or -1 to quit: ");
29            grade = input.nextInt();
30        }
31
32        // termination phase
33        // if user entered at least one grade...
34        if (gradeCounter != 0)
35        {
36            // use number with decimal point to calculate average of grades
37            double average = (double) total / gradeCounter;
38
39            // display total and average (with two digits of precision)
40            System.out.printf("%nTotal of the %d grades entered is %d%n",
41                             gradeCounter, total);
42            System.out.printf("Class average is %.2f%n", average);
43        }
44        else // no grades were entered, so output appropriate message
45            System.out.println("No grades were entered");
46    }
47 } // end class ClassAverage

```

```

Enter grade or -1 to quit: 97
Enter grade or -1 to quit: 88
Enter grade or -1 to quit: 72
Enter grade or -1 to quit: -1

```

```

Total of the 3 grades entered is 257
Class average is 85.67

```

Fig. 4.10 | Solving the class-average problem using sentinel-controlled repetition. (Part 2 of 2.)

Program Logic for Sentinel-Controlled Repetition vs. Counter-Controlled Repetition

Line 37 declares `double` variable `average`, which allows us to store the class average as a floating-point number. Line 14 initializes `gradeCounter` to 0, because no grades have been entered yet. Remember that this program uses *sentinel-controlled repetition* to input the grades. To keep an accurate record of the number of grades entered, the program increments `gradeCounter` only when the user enters a valid grade.

Compare the program logic for sentinel-controlled repetition in this application with that for counter-controlled repetition in Fig. 4.8. In counter-controlled repetition, each iteration of the `while` statement (lines 17–23 of Fig. 4.8) reads a value from the user, for the specified number of iterations. In sentinel-controlled repetition, the program reads the first value (lines 18–19 of Fig. 4.10) before reaching the `while`. This value determines whether the program's flow of control should enter the body of the `while`. If the condition of the `while` is false, the user entered the sentinel value, so the body of the `while` does not execute (i.e., no grades were entered). If, on the other hand, the condition is *true*, the body begins execution, and the loop adds the grade value to the `total` and increments the `gradeCounter` (lines 24–25). Then lines 28–29 in the loop body input the next value from the user. Next, program control reaches the closing right brace of the loop body at line 30, so execution continues with the test of the `while`'s condition (line 22). The condition uses the most recent grade input by the user to determine whether the loop body should execute again. The value of variable `grade` is always input from the user immediately before the program tests the `while` condition. This allows the program to determine whether the value just input is the sentinel value *before* the program processes that value (i.e., adds it to the `total`). If the sentinel value is input, the loop terminates, and the program does not add -1 to the `total`.

**Good Programming Practice 4.3**

In a sentinel-controlled loop, prompts should remind the user of the sentinel.

After the loop terminates, the `if...else` statement at lines 34–45 executes. The condition at line 34 determines whether any grades were input. If none were input, the `else` part (lines 44–45) of the `if...else` statement executes and displays the message "No grades were entered" and the method returns control to the calling method.

Braces in a while statement

Notice the `while` statement's *block* in Fig. 4.10 (lines 23–30). Without the braces, the loop would consider its body to be only the first statement, which adds the grade to the `total`. The last three statements in the block would fall outside the loop body, causing the computer to interpret the code incorrectly as follows:

```
while (grade != -1)
    total = total + grade; // add grade to total
    gradeCounter = gradeCounter + 1; // increment counter
    // prompt for input and read next grade from user
    System.out.print("Enter grade or -1 to quit: ");
    grade = input.nextInt();
```

The preceding code would cause an *infinite loop* in the program if the user did not input the sentinel -1 at line 19 (before the `while` statement).



Common Programming Error 4.5

Omitting the braces that delimit a block can lead to logic errors, such as infinite loops. To prevent this problem, some programmers enclose the body of every control statement in braces, even if the body contains only a single statement.

Explicitly and Implicitly Converting Between Primitive Types

If at least one grade was entered, line 37 of Fig. 4.10 calculates the average of the grades. Recall from Fig. 4.8 that integer division yields an integer result. Even though variable `average` is declared as a `double`, if we had written the averaging calculation as

```
double average = total / gradeCounter;
```

it would lose the fractional part of the quotient *before* the result of the division is assigned to `average`. This occurs because `total` and `gradeCounter` are *both* integers, and integer division yields an integer result.

Most averages are not whole numbers (e.g., 0, -22 and 1024). For this reason, we calculate the class average in this example as a floating-point number. To perform a floating-point calculation with integer values, we must *temporarily* treat these values as floating-point numbers for use in the calculation. Java provides the **unary cast operator** to accomplish this task. Line 37 of Fig. 4.10 uses the **(double)** cast operator—a unary operator—to create a *temporary* floating-point copy of its operand `total` (which appears to the right of the operator). Using a cast operator in this manner is called **explicit conversion** or **type casting**. The value stored in `total` is still an integer.

The calculation now consists of a floating-point value (the temporary `double` copy of `total`) divided by the integer `gradeCounter`. Java can evaluate only arithmetic expressions in which the operands' types are *identical*. To ensure this, Java performs an operation called **promotion** (or **implicit conversion**) on selected operands. For example, in an expression containing `int` and `double` values, the `int` values are promoted to `double` values for use in the expression. In this example, the value of `gradeCounter` is promoted to type `double`, then floating-point division is performed and the result of the calculation is assigned to `average`. As long as the **(double)** cast operator is applied to *any* variable in the calculation, the calculation will yield a `double` result. Later in this chapter, we discuss all the primitive types. You'll learn more about the promotion rules in Section 6.7.



Common Programming Error 4.6

*A cast operator can be used to convert between primitive numeric types, such as `int` and `double`, and between related reference types (as we discuss in Chapter 10, *Object-Oriented Programming: Polymorphism and Interfaces*). Casting to the wrong type may cause compilation errors or runtime errors.*

A cast operator is formed by placing parentheses around any type's name. The operator is a **unary operator** (i.e., an operator that takes only one operand). Java also supports unary versions of the plus (+) and minus (-) operators, so you can write expressions like -7 or +5. Cast operators associate from *right to left* and have the same precedence as other unary operators, such as unary + and unary -. This precedence is one level higher than that of the **multiplicative operators** *, / and %. (See the operator precedence chart in Appendix A.) We indicate the cast operator with the notation *(type)* in our precedence charts, to indicate that any type name can be used to form a cast operator.

Line 42 displays the class average. In this example, we display the class average *rounded* to the nearest hundredth. The format specifier `%.2f` in `printf`'s format control string indicates that variable `average`'s value should be displayed with two digits of precision to the right of the decimal point—indicated by `.2` in the format specifier. The three grades entered during the sample execution (Fig. 4.10) total 257, which yields the average 85.666666.... Method `printf` uses the precision in the format specifier to round the value to the specified number of digits. In this program, the average is rounded to the hundredths position and is displayed as 85.67.

Floating-Point Number Precision

Floating-point numbers are not always 100% precise, but they have numerous applications. For example, when we speak of a “normal” body temperature of 98.6, we do not need to be precise to a large number of digits. When we read the temperature on a thermometer as 98.6, it may actually be 98.5999473210643. Calling this number simply 98.6 is fine for most applications involving body temperatures.

Floating-point numbers often arise as a result of division, such as in this example's class-average calculation. In conventional arithmetic, when we divide 10 by 3, the result is 3.333333..., with the sequence of 3s repeating infinitely. The computer allocates only a fixed amount of space to hold such a value, so clearly the stored floating-point value can be only an approximation.

Owing to the imprecise nature of floating-point numbers, type `double` is preferred over type `float`, because `double` variables can represent floating-point numbers more accurately. For this reason, we primarily use type `double` throughout the book. In some applications, the precision of `float` and `double` variables will be inadequate. For precise floating-point numbers (such as those required by monetary calculations), Java provides class `BigDecimal` (package `java.math`), which we'll discuss in Chapter 8.



Common Programming Error 4.7

Using floating-point numbers in a manner that assumes they're represented precisely can lead to incorrect results.

4.11 Formulating Algorithms: Nested Control Statements

For the next example, we once again formulate an algorithm by using pseudocode and top-down, stepwise refinement, and write a corresponding Java program. We've seen that control statements can be stacked on top of one another (in sequence). In this case study, we examine the only other structured way control statements can be connected—namely, by **nesting** one control statement within another.

Consider the following problem statement:

A college offers a course that prepares students for the state licensing exam for real-estate brokers. Last year, ten of the students who completed this course took the exam. The college wants to know how well its students did on the exam. You've been asked to write a program to summarize the results. You've been given a list of these 10 students. Next to each name is written a 1 if the student passed the exam or a 2 if the student failed.

Your program should analyze the results of the exam as follows:

1. *Input each test result (i.e., a 1 or a 2). Display the message “Enter result” on the screen each time the program requests another test result.*

2. Count the number of test results of each type.
3. Display a summary of the test results, indicating the number of students who passed and the number who failed.
4. If more than eight students passed the exam, print “Bonus to instructor!”

After reading the problem statement carefully, we make the following observations:

1. The program must process test results for 10 students. A counter-controlled loop can be used, because the number of test results is known in advance.
2. Each test result has a numeric value—either a 1 or a 2. Each time it reads a test result, the program must determine whether it’s a 1 or a 2. We test for a 1 in our algorithm. If the number is not a 1, we assume that it’s a 2. (Exercise 4.24 considers the consequences of this assumption.)
3. Two counters are used to keep track of the exam results—one to count the number of students who passed the exam and one to count the number who failed.
4. After the program has processed all the results, it must decide whether more than eight students passed the exam.

Let’s proceed with top-down, stepwise refinement. We begin with a pseudocode representation of the top:

Analyze exam results and decide whether a bonus should be paid

Once again, the top is a *complete* representation of the program, but several refinements are likely to be needed before the pseudocode can evolve naturally into a Java program.

Our first refinement is

Initialize variables

Input the 10 exam results, and count passes and failures

Print a summary of the exam results and decide whether a bonus should be paid

Here, too, even though we have a *complete* representation of the entire program, further refinement is necessary. We now commit to specific variables. Counters are needed to record the passes and failures, a counter will be used to control the looping process and a variable is needed to store the user input. The variable in which the user input will be stored is *not* initialized at the start of the algorithm, because its value is read from the user during each iteration of the loop.

The pseudocode statement

Initialize variables

can be refined as follows:

Initialize passes to zero

Initialize failures to zero

Initialize student counter to one

Notice that only the counters are initialized at the start of the algorithm.

The pseudocode statement

Input the 10 exam results, and count passes and failures

requires a loop that successively inputs the result of each exam. We know in advance that there are precisely 10 exam results, so counter-controlled looping is appropriate. Inside the loop (i.e., *nested* within the loop), a double-selection structure will determine whether each exam result is a pass or a failure and will increment the appropriate counter. The refinement of the preceding pseudocode statement is then

```

While student counter is less than or equal to 10
    Prompt the user to enter the next exam result
    Input the next exam result
    If the student passed
        Add one to passes
    Else
        Add one to failures
    Add one to student counter
  
```

We use blank lines to isolate the *If...Else* control structure, which improves readability. The pseudocode statement

```

Print a summary of the exam results and decide whether a bonus should be paid
  
```

can be refined as follows:

```

Print the number of passes
Print the number of failures
If more than eight students passed
    Print "Bonus to instructor!"
  
```

Complete Second Refinement of Pseudocode and Conversion to Class Analysis

The complete second refinement appears in Fig. 4.11. Notice that blank lines are also used to set off the *While* structure for program readability. This pseudocode is now sufficiently refined for conversion to Java.

```

1  Initialize passes to zero
2  Initialize failures to zero
3  Initialize student counter to one
4
5  While student counter is less than or equal to 10
6      Prompt the user to enter the next exam result
7      Input the next exam result
8
9      If the student passed
10         Add one to passes
11      Else
12         Add one to failures
13
14     Add one to student counter
  
```

Fig. 4.11 | Pseudocode for examination-results problem. (Part I of 2.)

```

15
16 Print the number of passes
17 Print the number of failures
18
19 If more than eight students passed
20 Print "Bonus to instructor!"

```

Fig. 4.11 | Pseudocode for examination-results problem. (Part 2 of 2.)

The Java class that implements the pseudocode algorithm and two sample executions are shown in Fig. 4.12. Lines 13, 14, 15 and 22 of main declare the variables that are used to process the examination results.



Error-Prevention Tip 4.5

Initializing local variables when they're declared helps you avoid any compilation errors that might arise from attempts to use uninitialized variables. While Java does not require that local-variable initializations be incorporated into declarations, it does require that local variables be initialized before their values are used in an expression.

```

1  // Fig. 4.12: Analysis.java
2  // Analysis of examination results using nested control statements.
3  import java.util.Scanner; // class uses class Scanner
4
5  public class Analysis
6  {
7      public static void main(String[] args)
8      {
9          // create Scanner to obtain input from command window
10         Scanner input = new Scanner(System.in);
11
12         // initializing variables in declarations
13         int passes = 0;
14         int failures = 0;
15         int studentCounter = 1;
16
17         // process 10 students using counter-controlled loop
18         while (studentCounter <= 10)
19         {
20             // prompt user for input and obtain value from user
21             System.out.print("Enter result (1 = pass, 2 = fail): ");
22             int result = input.nextInt();
23
24             // if...else is nested in the while statement
25             if (result == 1)
26                 passes = passes + 1;
27             else
28                 failures = failures + 1;
29

```

Fig. 4.12 | Analysis of examination results using nested control statements. (Part 1 of 2.)

```

30         // increment studentCounter so loop eventually terminates
31         studentCounter = studentCounter + 1;
32     }
33
34     // termination phase; prepare and display results
35     System.out.printf("Passed: %d\nFailed: %d\n", passes, failures);
36
37     // determine whether more than 8 students passed
38     if (passes > 8)
39         System.out.println("Bonus to instructor!");
40     }
41 } // end class Analysis

```

```

Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 2
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Passed: 9
Failed: 1
Bonus to instructor!

```

```

Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 2
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 2
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 2
Enter result (1 = pass, 2 = fail): 2
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Enter result (1 = pass, 2 = fail): 1
Passed: 6
Failed: 4

```

Fig. 4.12 | Analysis of examination results using nested control statements. (Part 2 of 2.)

The `while` statement (lines 18–32) loops 10 times. During each iteration, the loop inputs and processes one exam result. Notice that the `if...else` statement (lines 25–28) for processing each result is *nested* in the `while` statement. If the result is 1, the `if...else` statement increments `passes`; otherwise, it assumes the result is 2 and increments `failures`. Line 31 increments `studentCounter` before the loop condition is tested again at line 18. After 10 values have been input, the loop terminates and line 35 displays the number of passes and failures. The `if` statement at lines 38–39 determines whether more than eight students passed the exam and, if so, outputs the message "Bonus to instructor!".

Figure 4.12 shows the input and output from two sample executions of the program. During the first, the condition at line 38 of method `main` is `true`—more than eight students passed the exam, so the program outputs a message to bonus the instructor.

4.12 Compound Assignment Operators

The **compound assignment operators** abbreviate assignment expressions. Statements like

```
variable = variable operator expression;
```

where *operator* is one of the binary operators `+`, `-`, `*`, `/` or `%` (or others we discuss later in the text) can be written in the form

```
variable operator = expression;
```

For example, you can abbreviate the statement

```
c = c + 3;
```

with the **addition compound assignment operator**, `+=`, as

```
c += 3;
```

The `+=` operator adds the value of the expression on its right to the value of the variable on its left and stores the result in the variable on the left of the operator. Thus, the assignment expression `c += 3` adds 3 to `c`. Figure 4.13 shows the arithmetic compound assignment operators, sample expressions using the operators and explanations of what the operators do.

Assignment operator	Sample expression	Explanation	Assigns
<i>Assume: int c = 3, d = 5, e = 4, f = 6, g = 12;</i>			
<code>+=</code>	<code>c += 7</code>	<code>c = c + 7</code>	10 to c
<code>-=</code>	<code>d -= 4</code>	<code>d = d - 4</code>	1 to d
<code>*=</code>	<code>e *= 5</code>	<code>e = e * 5</code>	20 to e
<code>/=</code>	<code>f /= 3</code>	<code>f = f / 3</code>	2 to f
<code>%=</code>	<code>g %= 9</code>	<code>g = g % 9</code>	3 to g

Fig. 4.13 | Arithmetic compound assignment operators.

4.13 Increment and Decrement Operators

Java provides two unary operators (summarized in Fig. 4.14) for adding 1 to or subtracting 1 from the value of a numeric variable. These are the unary **increment operator**, `++`, and the unary **decrement operator**, `--`. A program can increment by 1 the value of a variable called `c` using the increment operator, `++`, rather than the expression `c = c + 1` or `c += 1`. An increment or decrement operator that's prefixed to (placed before) a variable is referred to as the **prefix increment** or **prefix decrement operator**, respectively. An increment or decrement operator that's postfix to (placed after) a variable is referred to as the **postfix increment** or **postfix decrement operator**, respectively.

Operator	Operator name	Sample expression	Explanation
++	prefix increment	++a	Increment a by 1, then use the new value of a in the expression in which a resides.
++	postfix increment	a++	Use the current value of a in the expression in which a resides, then increment a by 1.
--	prefix decrement	--b	Decrement b by 1, then use the new value of b in the expression in which b resides.
--	postfix decrement	b--	Use the current value of b in the expression in which b resides, then decrement b by 1.

Fig. 4.14 | Increment and decrement operators.

Using the prefix increment (or decrement) operator to add 1 to (or subtract 1 from) a variable is known as **preincrementing** (or **predecrementing**). This causes the variable to be incremented (decremented) by 1; then the new value of the variable is used in the expression in which it appears. Using the postfix increment (or decrement) operator to add 1 to (or subtract 1 from) a variable is known as **postincrementing** (or **postdecrementing**). This causes the current value of the variable to be used in the expression in which it appears; then the variable's value is incremented (decremented) by 1.



Good Programming Practice 4.4

Unlike binary operators, the unary increment and decrement operators should be placed next to their operands, with no intervening spaces.

Difference Between Prefix Increment and Postfix Increment Operators

Figure 4.15 demonstrates the difference between the prefix increment and postfix increment versions of the ++ increment operator. The decrement operator (--) works similarly.

Line 9 initializes the variable c to 5, and line 10 outputs c's initial value. Line 11 outputs the value of the expression c++. This expression postincrements the variable c, so c's *original* value (5) is output, then c's value is incremented (to 6). Thus, line 11 outputs c's initial value (5) again. Line 12 outputs c's new value (6) to prove that the variable's value was indeed incremented in line 11.

Line 17 resets c's value to 5, and line 18 outputs c's value. Line 19 outputs the value of the expression ++c. This expression preincrements c, so its value is incremented; then the *new* value (6) is output. Line 20 outputs c's value again to show that the value of c is still 6 after line 19 executes.

```

1  // Fig. 4.15: Increment.java
2  // Prefix increment and postfix increment operators.
3
4  public class Increment
5  {

```

Fig. 4.15 | Prefix increment and postfix increment operators. (Part I of 2.)


```

6   public static void main(String[] args)
7   {
8       // demonstrate postfix increment operator
9       int c = 5;
10      System.out.printf("c before postincrement: %d\n", c); // prints 5
11      System.out.printf("    postincrementing c: %d\n", c++); // prints 5
12      System.out.printf("  c after postincrement: %d\n", c); // prints 6
13
14      System.out.println(); // skip a line
15
16      // demonstrate prefix increment operator
17      c = 5;
18      System.out.printf(" c before preincrement: %d\n", c); // prints 5
19      System.out.printf("    preincrementing c: %d\n", ++c); // prints 6
20      System.out.printf("  c after preincrement: %d\n", c); // prints 6
21  }
22 } // end class Increment

```

```

c before postincrement: 5
  postincrementing c: 5
c after postincrement: 6

c before preincrement: 5
  preincrementing c: 6
c after preincrement: 6

```

Fig. 4.15 | Prefix increment and postfix increment operators. (Part 2 of 2.)

Simplifying Statements with the Arithmetic Compound Assignment, Increment and Decrement Operators

The arithmetic compound assignment operators and the increment and decrement operators can be used to simplify program statements. For example, the three assignment statements in Fig. 4.12 (lines 26, 28 and 31)

```

passes = passes + 1;
failures = failures + 1;
studentCounter = studentCounter + 1;

```

can be written more concisely with compound assignment operators as

```

passes += 1;
failures += 1;
studentCounter += 1;

```

with prefix increment operators as

```

++passes;
++failures;
++studentCounter;

```

or with postfix increment operators as

```

passes++;
failures++;
studentCounter++;

```

When incrementing or decrementing a variable in a statement by itself, the prefix increment and postfix increment forms have the *same* effect, and the prefix decrement and postfix decrement forms have the *same* effect. It's only when a variable appears in the context of a larger expression that preincrementing and postincrementing the variable have different effects (and similarly for predecrementing and postdecrementing).



Common Programming Error 4.8

Attempting to use the increment or decrement operator on an expression other than one to which a value can be assigned is a syntax error. For example, writing `++(x + 1)` is a syntax error, because `(x + 1)` is not a variable.

Operator Precedence and Associativity

Figure 4.16 shows the precedence and associativity of the operators we've introduced. They're shown from top to bottom in decreasing order of precedence. The second column describes the associativity of the operators at each level of precedence. The conditional operator (`?:`); the unary operators increment (`++`), decrement (`--`), plus (`+`) and minus (`-`); the cast operators and the assignment operators `=`, `+=`, `-=`, `*=`, `/=` and `%=` associate from *right to left*. All the other operators in the operator precedence chart in Fig. 4.16 associate from left to right. The third column lists the type of each group of operators.



Good Programming Practice 4.5

Refer to the operator precedence and associativity chart (Appendix A) when writing expressions containing many operators. Confirm that the operators in the expression are performed in the order you expect. If you're uncertain about the order of evaluation in a complex expression, break the expression into smaller statements or use parentheses to force the order of evaluation, exactly as you'd do in an algebraic expression. Be sure to observe that some operators such as assignment (`=`) associate right to left rather than left to right.

Operators	Associativity	Type
<code>++</code> <code>--</code>	right to left	unary postfix
<code>++</code> <code>--</code> <code>+</code> <code>-</code> (<i>type</i>)	right to left	unary prefix
<code>*</code> <code>/</code> <code>%</code>	left to right	multiplicative
<code>+</code> <code>-</code>	left to right	additive
<code><</code> <code><=</code> <code>></code> <code>>=</code>	left to right	relational
<code>==</code> <code>!=</code>	left to right	equality
<code>?:</code>	right to left	conditional
<code>=</code> <code>+=</code> <code>-=</code> <code>*=</code> <code>/=</code> <code>%=</code>	right to left	assignment

Fig. 4.16 | Precedence and associativity of the operators discussed so far.

4.14 Primitive Types

The table in Appendix D lists the eight primitive types in Java. Like its predecessor languages C and C++, Java requires all variables to have a type. For this reason, Java is referred to as a **strongly typed language**.

In C and C++, programmers frequently have to write separate versions of programs to support different computer platforms, because the primitive types are not guaranteed to be identical from computer to computer. For example, an `int` on one machine might be represented by 16 bits (2 bytes) of memory, on a second machine by 32 bits (4 bytes), and on another machine by 64 bits (8 bytes). In Java, `int` values are always 32 bits (4 bytes).



Portability Tip 4.1

The primitive types in Java are portable across all computer platforms that support Java.

Each type in Appendix D is listed with its size in bits (there are eight bits to a byte) and its range of values. Because the designers of Java want to ensure portability, they use internationally recognized standards for character formats (Unicode; for more information, visit www.unicode.org) and floating-point numbers (IEEE 754; for more information, visit grouper.ieee.org/groups/754/).

Recall from Section 3.2 that variables of primitive types declared outside of a method as instance variables of a class are *automatically assigned default values unless explicitly initialized*. Instance variables of types `char`, `byte`, `short`, `int`, `long`, `float` and `double` are all given the value 0 by default. Instance variables of type `boolean` are given the value `false` by default. Reference-type instance variables are initialized by default to the value `null`.

4.15 (Optional) GUI and Graphics Case Study: Creating Simple Drawings

An appealing feature of Java is its graphics support, which enables you to visually enhance your applications. We now introduce one of Java's graphical capabilities—drawing lines. It also covers the basics of creating a window to display a drawing on the computer screen.

Java's Coordinate System

To draw in Java, you must understand Java's **coordinate system** (Fig. 4.17), a scheme for identifying points on the screen. By default, the upper-left corner of a GUI component has the coordinates (0, 0). A coordinate pair is composed of an **x-coordinate** (the **horizontal coordinate**) and a **y-coordinate** (the **vertical coordinate**). The **x-coordinate** is the horizontal location moving from *left to right*. The **y-coordinate** is the vertical location moving from *top to bottom*. The **x-axis** describes every horizontal coordinate, and the **y-axis** every vertical coordinate. Coordinates indicate where graphics should be displayed on a screen. Coordinate units are measured in **pixels**. The term pixel stands for "picture element." A pixel is a display monitor's smallest unit of resolution.

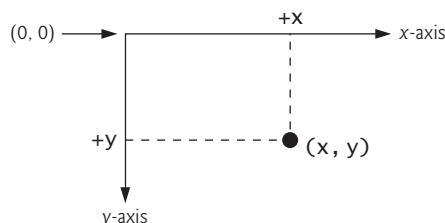


Fig. 4.17 | Java coordinate system. Units are measured in pixels.

First Drawing Application

Our first drawing application simply draws two lines. Class `DrawPanel` (Fig. 4.18) performs the actual drawing, while class `DrawPanelTest` (Fig. 4.19) creates a window to display the drawing. In class `DrawPanel`, the import statements in lines 3–4 allow us to use class **Graphics** (from package `java.awt`), which provides various methods for drawing text and shapes onto the screen, and class **JPanel** (from package `javax.swing`), which provides an area on which we can draw.

```

1 // Fig. 4.18: DrawPanel.java
2 // Using drawLine to connect the corners of a panel.
3 import java.awt.Graphics;
4 import javax.swing.JPanel;
5
6 public class DrawPanel extends JPanel
7 {
8     // draws an X from the corners of the panel
9     public void paintComponent(Graphics g)
10    {
11        // call paintComponent to ensure the panel displays correctly
12        super.paintComponent(g);
13
14        int width = getWidth(); // total width
15        int height = getHeight(); // total height
16
17        // draw a line from the upper-left to the lower-right
18        g.drawLine(0, 0, width, height);
19
20        // draw a line from the lower-left to the upper-right
21        g.drawLine(0, height, width, 0);
22    }
23 } // end class DrawPanel

```

Fig. 4.18 | Using `drawLine` to connect the corners of a panel.

```

1 // Fig. 4.19: DrawPanelTest.java
2 // Creating JFrame to display DrawPanel.
3 import javax.swing.JFrame;
4
5 public class DrawPanelTest
6 {
7     public static void main(String[] args)
8     {
9         // create a panel that contains our drawing
10        DrawPanel panel = new DrawPanel();
11
12        // create a new frame to hold the panel
13        JFrame application = new JFrame();
14
15        // set the frame to exit when it is closed
16        application.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

```

Fig. 4.19 | Creating `JFrame` to display `DrawPanel`. (Part I of 2.)

```
17  
18     application.add(panel); // add the panel to the frame  
19     application.setSize(250, 250); // set the size of the frame  
20     application.setVisible(true); // make the frame visible  
21 }  
22 } // end class DrawPanelTest
```

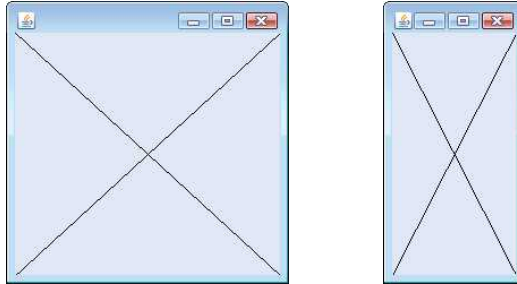


Fig. 4.19 | Creating JFrame to display DrawPanel. (Part 2 of 2.)

Line 6 uses the keyword **extends** to indicate that class `DrawPanel` is an enhanced type of `JPanel`. The keyword **extends** represents a so-called *inheritance* relationship in which our new class `DrawPanel` begins with the existing members (data and methods) from class `JPanel`. The class from which `DrawPanel` **inherits**, `JPanel`, appears to the right of keyword **extends**. In this inheritance relationship, `JPanel` is called the **superclass** and `DrawPanel` is called the **subclass**. This results in a `DrawPanel` class that has the attributes (data) and behaviors (methods) of class `JPanel` as well as the new features we're adding in our `DrawPanel` class declaration—specifically, the ability to draw two lines along the diagonals of the panel. Inheritance is explained in detail in Chapter 9. For now, you should mimic our `DrawPanel` class when creating your own graphics programs.

Method `paintComponent`

Every `JPanel`, including our `DrawPanel`, has a **`paintComponent`** method (lines 9–22), which the system automatically calls every time it needs to display the `DrawPanel`. Method `paintComponent` must be declared as shown in line 9—otherwise, the system will not call it. This method is called when a `JPanel` is first displayed on the screen, when it's *covered* then *uncovered* by a window on the screen, and when the window in which it appears is *resized*. Method `paintComponent` requires one argument, a `Graphics` object, that's provided by the system when it calls `paintComponent`. This `Graphics` object is used to draw lines, rectangles, ovals and other graphics.

The first statement in every `paintComponent` method you create should always be

```
super.paintComponent(g);
```

which ensures that the panel is properly rendered before we begin drawing on it. Next, lines 14–15 call methods that class `DrawPanel` inherits from `JPanel`. Because `DrawPanel` extends `JPanel`, `DrawPanel` can use any public methods of `JPanel`. Methods **`getWidth`** and **`getHeight`** return the `JPanel`'s width and height, respectively. Lines 14–15 store these values in the local variables `width` and `height`. Finally, lines 18 and 21 use the `Graphics` variable `g` to call method **`drawLine`** to draw the two lines. Method `drawLine` draws a line

between two points represented by its four arguments. The first two arguments are the x - and y -coordinates for one endpoint, and the last two arguments are the coordinates for the other endpoint. If you *resize* the window, the lines will *scale* accordingly, because the arguments are based on the width and height of the panel. Resizing the window in this application causes the system to call `paintComponent` to *redraw* the `DrawPanel`'s contents.

Class `DrawPanelTest`

To display the `DrawPanel` on the screen, you must place it in a window. You create a window with an object of class `JFrame`. In `DrawPanelTest.java` (Fig. 4.19), line 3 imports class `JFrame` from package `javax.swing`. Line 10 in `main` creates a `DrawPanel` object, which contains our drawing, and line 13 creates a new `JFrame` that can hold and display our panel. Line 16 calls `JFrame` method `setDefaultCloseOperation` with the argument `JFrame.EXIT_ON_CLOSE` to indicate that the application should terminate when the user closes the window. Line 18 uses class `JFrame`'s `add` method to *attach* the `DrawPanel` to the `JFrame`. Line 19 sets the *size* of the `JFrame`. Method `setSize` takes two parameters that represent the width and height of the `JFrame`, respectively. Finally, line 20 *displays* the `JFrame` by calling its `setVisible` method with the argument `true`. When the `JFrame` is displayed, the `DrawPanel`'s `paintComponent` method (lines 9–22 of Fig. 4.18) is implicitly called, and the two lines are drawn (see the sample outputs in Fig. 4.19). Try resizing the window to see that the lines always draw based on the window's current width and height.

GUI and Graphics Case Study Exercises

- 4.1 Using loops and control statements to draw lines can lead to many interesting designs.
- Create the design in the left screen capture of Fig. 4.20. This design draws lines from the top-left corner, fanning them out until they cover the upper-left half of the panel. One approach is to divide the width and height into an equal number of steps (we found 15 steps worked well). The first endpoint of a line will always be in the top-left corner (0, 0). The second endpoint can be found by starting at the bottom-left corner and moving up one vertical step and right one horizontal step. Draw a line between the two endpoints. Continue moving up and to the right one step to find each successive endpoint. The figure should scale accordingly as you resize the window.

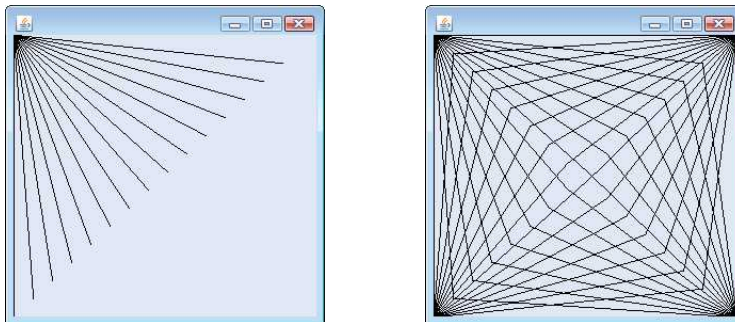


Fig. 4.20 | Lines fanning from a corner.

- b) Modify part (a) to have lines fan out from all four corners, as shown in the right screen capture of Fig. 4.20. Lines from opposite corners should intersect along the middle.
- 4.2** Figure 4.21 displays two additional designs created using `while` loops and `drawLine`.
- a) Create the design in the left screen capture of Fig. 4.21. Begin by dividing each edge into an equal number of increments (we chose 15 again). The first line starts in the top-left corner and ends one step right on the bottom edge. For each successive line, move down one increment on the left edge and right one increment on the bottom edge. Continue drawing lines until you reach the bottom-right corner. The figure should scale as you resize the window so that the endpoints always touch the edges.
- b) Modify your answer in part (a) to mirror the design in all four corners, as shown in the right screen capture of Fig. 4.21.

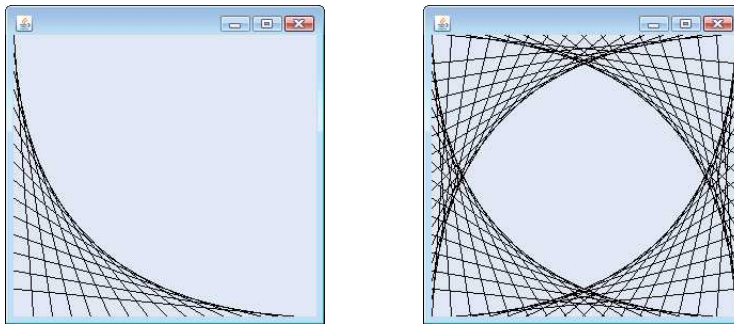


Fig. 4.21 | Line art with loops and `drawLine`.

4.16 Wrap-Up

This chapter presented basic problem solving for building classes and developing methods for these classes. We demonstrated how to construct an algorithm (i.e., an approach to solving a problem), then how to refine the algorithm through several phases of pseudocode development, resulting in Java code that can be executed as part of a method. The chapter showed how to use top-down, stepwise refinement to plan out the specific actions that a method must perform and the order in which the method must perform these actions.

Only three types of control structures—sequence, selection and repetition—are needed to develop any problem-solving algorithm. Specifically, this chapter demonstrated the `if` single-selection statement, the `if...else` double-selection statement and the `while` repetition statement. These are some of the building blocks used to construct solutions to many problems. We used control-statement stacking to total and compute the average of a set of student grades with counter- and sentinel-controlled repetition, and we used control-statement nesting to analyze and make decisions based on a set of exam results. We introduced Java's compound assignment operators and its increment and decrement operators. Finally, we discussed Java's primitive types. In Chapter 5, we continue our discussion of control statements, introducing the `for`, `do...while` and `switch` statements.

Summary

Section 4.1 Introduction

- Before writing a program to solve a problem, you must have a thorough understanding of the problem and a carefully planned approach to solving it. You must also understand the building blocks that are available and employ proven program-construction techniques.

Section 4.2 Algorithms

- Any computing problem can be solved by executing a series of actions (p. 102) in a specific order.
- A procedure for solving a problem in terms of the actions to execute and the order in which they execute is called an algorithm (p. 102).
- Specifying the order in which statements execute in a program is called program control (p. 102).

Section 4.3 Pseudocode

- Pseudocode (p. 103) is an informal language that helps you develop algorithms without having to worry about the strict details of Java language syntax.
- The pseudocode we use in this book is similar to everyday English—it's convenient and user friendly, but it's not an actual computer programming language. You may, of course, use your own native language(s) to develop your own pseudocode.
- Pseudocode helps you “think out” a program before attempting to write it in a programming language, such as Java.
- Carefully prepared pseudocode can easily be converted to a corresponding Java program.

Section 4.4 Control Structures

- Normally, statements in a program are executed one after the other in the order in which they're written. This process is called sequential execution (p. 103).
- Various Java statements enable you to specify that the next statement to execute is not necessarily the next one in sequence. This is called transfer of control (p. 103).
- Bohm and Jacopini demonstrated that all programs could be written in terms of only three control structures (p. 103)—the sequence structure, the selection structure and the repetition structure.
- The term “control structures” comes from the field of computer science. The *Java Language Specification* refers to “control structures” as “control statements” (p. 104).
- The sequence structure is built into Java. Unless directed otherwise, the computer executes Java statements one after the other in the order in which they're written—that is, in sequence.
- Anywhere a single action may be placed, several actions may be placed in sequence.
- Activity diagrams (p. 104) are part of the UML. An activity diagram models the workflow (p. 104; also called the activity) of a portion of a software system.
- Activity diagrams are composed of symbols (p. 104)—such as action-state symbols, diamonds and small circles—that are connected by transition arrows, which represent the flow of the activity.
- Action states (p. 104) contain action expressions that specify particular actions to perform.
- The arrows in an activity diagram represent transitions, which indicate the order in which the actions represented by the action states occur.
- The solid circle located at the top of an activity diagram represents the activity's initial state (p. 104)—the beginning of the workflow before the program performs the modeled actions.
- The solid circle surrounded by a hollow circle that appears at the bottom of the diagram represents the final state (p. 104)—the end of the workflow after the program performs its actions.

- Rectangles with their upper-right corners folded over are UML notes (p. 104)—explanatory remarks that describe the purpose of symbols in the diagram.
- Java has three types of selection statements (p. 105).
- The `if` single-selection statement (p. 105) selects or ignores one or more actions.
- The `if...else` double-selection statement selects between two actions or groups of actions.
- The `switch` statement is called a multiple-selection statement (p. 105) because it selects among many different actions or groups of actions.
- Java provides the `while`, `do...while` and `for` repetition (also called iteration or looping) statements that enable programs to perform statements repeatedly as long as a loop-continuation condition remains true.
- The `while` and `for` statements perform the action(s) in their bodies zero or more times—if the loop-continuation condition (p. 105) is initially false, the action(s) will not execute. The `do...while` statement performs the action(s) in its body one or more times.
- The words `if`, `else`, `switch`, `while`, `do` and `for` are Java keywords. Keywords cannot be used as identifiers, such as variable names.
- Every program is formed by combining as many sequence, selection and repetition statements (p. 105) as is appropriate for the algorithm the program implements.
- Single-entry/single-exit control statements (p. 105) are attached to one another by connecting the exit point of one to the entry point of the next. This is known as control-statement stacking.
- A control statement may also be nested (p. 105) inside another control statement.

Section 4.5 *if* Single-Selection Statement

- Programs use selection statements to choose among alternative courses of action.
- The single-selection `if` statement's activity diagram contains the diamond symbol, which indicates that a decision is to be made. The workflow follows a path determined by the symbol's associated guard conditions (p. 106). If a guard condition is true, the workflow enters the action state to which the corresponding transition arrow points.
- The `if` statement is a single-entry/single-exit control statement.

Section 4.6 *if...else* Double-Selection Statement

- The `if` single-selection statement performs an indicated action only when the condition is true.
- The `if...else` double-selection (p. 105) statement performs one action when the condition is true and another action when the condition is false.
- A program can test multiple cases with nested `if...else` statements (p. 107).
- The conditional operator (p. 110; `?:`) is Java's only ternary operator—it takes three operands. Together, the operands and the `?:` symbol form a conditional expression (p. 110).
- The Java compiler associates an `else` with the immediately preceding `if` unless told to do otherwise by the placement of braces.
- The `if` statement expects one statement in its body. To include several statements in the body of an `if` (or the body of an `else` for an `if...else` statement), enclose the statements in braces.
- A block (p. 110) of statements can be placed anywhere that a single statement can be placed.
- A logic error (p. 110) has its effect at execution time. A fatal logic error (p. 110) causes a program to fail and terminate prematurely. A nonfatal logic error (p. 110) allows a program to continue executing, but causes it to produce incorrect results.

- Just as a block can be placed anywhere a single statement can be placed, you can also use an empty statement, represented by placing a semicolon (;) where a statement would normally be.

Section 4.8 *while Repetition Statement*

- The `while` repetition statement (p. 114) allows you to specify that a program should repeat an action while some condition remains true.
- The UML's merge (p. 114) symbol joins two flows of activity into one.
- The decision and merge symbols can be distinguished by the number of incoming and outgoing transition arrows. A decision symbol has one transition arrow pointing to the diamond and two or more transition arrows pointing out from the diamond to indicate possible transitions from that point. Each transition arrow pointing out of a decision symbol has a guard condition. A merge symbol has two or more transition arrows pointing to the diamond and only one transition arrow pointing from the diamond, to indicate multiple activity flows merging to continue the activity. None of the transition arrows associated with a merge symbol has a guard condition.

Section 4.9 *Formulating Algorithms: Counter-Controlled Repetition*

- Counter-controlled repetition (p. 115) uses a variable called a counter (or control variable) to control the number of times a set of statements execute.
- Counter-controlled repetition is often called definite repetition (p. 115), because the number of repetitions is known before the loop begins executing.
- A total (p. 115) is a variable used to accumulate the sum of several values. Variables used to store totals are normally initialized to zero before being used in a program.
- A local variable's declaration must appear before the variable is used in that method. A local variable cannot be accessed outside the method in which it's declared.
- Dividing two integers results in integer division—the calculation's fractional part is truncated.

Section 4.10 *Formulating Algorithms: Sentinel-Controlled Repetition*

- In sentinel-controlled repetition (p. 119), a special value called a sentinel value (also called a signal value, a dummy value or a flag value) is used to indicate "end of data entry."
- A sentinel value must be chosen that cannot be confused with an acceptable input value.
- Top-down, stepwise refinement (p. 120) is essential to the development of well-structured programs.
- Division by zero is a logic error.
- To perform a floating-point calculation with integer values, cast one of the integers to type `double`.
- Java knows how to evaluate only arithmetic expressions in which the operands' types are identical. To ensure this, Java performs an operation called promotion on selected operands.
- The unary cast operator is formed by placing parentheses around the name of a type.

Section 4.12 *Compound Assignment Operators*

- The compound assignment operators (p. 131) abbreviate assignment expressions. Statements of the form

variable = *variable* *operator* *expression*;

where *operator* is one of the binary operators `+`, `-`, `*`, `/` or `%`, can be written in the form

variable *operator* = *expression*;

- The `+=` operator adds the value of the expression on the right of the operator to the value of the variable on the left of the operator and stores the result in the variable on the left of the operator.

Section 4.13 Increment and Decrement Operators

- The unary increment operator, ++, and the unary decrement operator, --, add 1 to or subtract 1 from the value of a numeric variable (p. 131).
- An increment or decrement operator that's prefixed (p. 131) to a variable is the prefix increment or prefix decrement operator, respectively. An increment or decrement operator that's postfix (p. 131) to a variable is the postfix increment or postfix decrement operator, respectively.
- Using the prefix increment or decrement operator to add or subtract 1 is known as preincrementing or predecrementing, respectively.
- Preincrementing or predecrementing a variable causes the variable to be incremented or decremented by 1; then the new value of the variable is used in the expression in which it appears.
- Using the postfix increment or decrement operator to add or subtract 1 is known as postincrementing or postdecrementing, respectively.
- Postincrementing or postdecrementing the variable causes its value to be used in the expression in which it appears; then the variable's value is incremented or decremented by 1.
- When incrementing or decrementing a variable in a statement by itself, the prefix and postfix increment have the same effect, and the prefix and postfix decrement have the same effect.

Section 4.14 Primitive Types

- Java requires all variables to have a type. Thus, Java is referred to as a strongly typed language (p. 134).
- Java uses Unicode characters and IEEE 754 floating-point numbers.

Self-Review Exercises

- 4.1** Fill in the blanks in each of the following statements:
- All programs can be written in terms of three types of control structures: _____, _____ and _____.
 - The _____ statement is used to execute one action when a condition is true and another when that condition is false.
 - Repeating a set of instructions a specific number of times is called _____ repetition.
 - When it's not known in advance how many times a set of statements will be repeated, a(n) _____ value can be used to terminate the repetition.
 - The _____ structure is built into Java; by default, statements execute in the order they appear.
 - Instance variables of types `char`, `byte`, `short`, `int`, `long`, `float` and `double` are all given the value _____ by default.
 - Java is a(n) _____ language; it requires all variables to have a type.
 - If the increment operator is _____ to a variable, first the variable is incremented by 1, then its new value is used in the expression.
- 4.2** State whether each of the following is *true* or *false*. If *false*, explain why.
- An algorithm is a procedure for solving a problem in terms of the actions to execute and the order in which they execute.
 - A set of statements contained within a pair of parentheses is called a block.
 - A selection statement specifies that an action is to be repeated while some condition remains true.
 - A nested control statement appears in the body of another control statement.
 - Java provides the arithmetic compound assignment operators `+=`, `-=`, `*=`, `/=` and `%=` for abbreviating assignment expressions.

- f) The primitive types (`boolean`, `char`, `byte`, `short`, `int`, `long`, `float` and `double`) are portable across only Windows platforms.
- g) Specifying the order in which statements execute in a program is called program control.
- h) The unary cast operator (`double`) creates a temporary integer copy of its operand.
- i) Instance variables of type `boolean` are given the value `true` by default.
- j) Pseudocode helps you think out a program before attempting to write it in a programming language.

4.3 Write four different Java statements that each add 1 to integer variable `x`.

4.4 Write Java statements to accomplish each of the following tasks:

- a) Use one statement to assign the sum of `x` and `y` to `z`, then increment `x` by 1.
- b) Test whether variable `count` is greater than 10. If it is, print "Count is greater than 10".
- c) Use one statement to decrement the variable `x` by 1, then subtract it from variable `total` and store the result in variable `total`.
- d) Calculate the remainder after `q` is divided by `divisor`, and assign the result to `q`. Write this statement in two different ways.

4.5 Write a Java statement to accomplish each of the following tasks:

- a) Declare variables `sum` of type `int` and initialize it to 0.
- b) Declare variables `x` of type `int` and initialize it to 1.
- c) Add variable `x` to variable `sum`, and assign the result to variable `sum`.
- d) Print "The sum is: ", followed by the value of variable `sum`.

4.6 Combine the statements that you wrote in Exercise 4.5 into a Java application that calculates and prints the sum of the integers from 1 to 10. Use a `while` statement to loop through the calculation and increment statements. The loop should terminate when the value of `x` becomes 11.

4.7 Determine the value of the variables in the statement `product *= x++`; after the calculation is performed. Assume that all variables are type `int` and initially have the value 5.

4.8 Identify and correct the errors in each of the following sets of code:

- a)

```
while (c <= 5)
{
    product *= c;
    ++c;
}
```
- b)

```
if (gender == 1)
    System.out.println("Woman");
else;
    System.out.println("Man");
```

4.9 What is wrong with the following `while` statement?

```
while (z >= 0)
    sum += z;
```

Answers to Self-Review Exercises

4.1 a) sequence, selection, repetition. b) `if...else`. c) counter-controlled (or definite). d) sentinel, signal, flag or dummy. e) sequence. f) 0 (zero). g) strongly typed. h) prefixed.

4.2 a) True. b) False. A set of statements contained within a pair of braces (`{` and `}`) is called a block. c) False. A repetition statement specifies that an action is to be repeated while some condition remains true. d) True. e) True. f) False. The primitive types (`boolean`, `char`, `byte`, `short`, `int`, `long`, `float` and `double`) are portable across all computer platforms that support Java. g) True. h) False. The unary cast operator (`double`) creates a temporary floating-point copy of its operand. i) False. Instance variables of type `boolean` are given the value `false` by default. j) True.

- 4.3** `x = x + 1;`
 `x += 1;`
 `++x;`
 `x++;`
- 4.4** a) `z = x++ + y;`
 b) `if (count > 10)`
 `System.out.println("Count is greater than 10");`
 c) `total -= --x;`
 d) `q %= divisor;`
 `q = q % divisor;`
- 4.5** a) `int sum = 0;`
 b) `int x = 1;`
 c) `sum += x; or sum = sum + x;`
 d) `System.out.printf("The sum is: %d\n", sum);`
- 4.6** The program is as follows:

```

1  // Exercise 4.6: Calculate.java
2  // Calculate the sum of the integers from 1 to 10
3  public class Calculate
4  {
5      public static void main(String[] args)
6      {
7          int sum = 0;
8          int x = 1;
9
10         while (x <= 10) // while x is less than or equal to 10
11         {
12             sum += x; // add x to sum
13             ++x; // increment x
14         }
15
16         System.out.printf("The sum is: %d\n", sum);
17     }
18 } // end class Calculate

```

The sum is: 55

- 4.7** `product = 25, x = 6`
- 4.8** a) Error: The closing right brace of the `while` statement's body is missing.
 Correction: Add a closing right brace after the statement `++c;`.
 b) Error: The semicolon after `else` results in a logic error. The second output statement will always be executed.
 Correction: Remove the semicolon after `else`.
- 4.9** The value of the variable `z` is never changed in the `while` statement. Therefore, if the loop-continuation condition (`z >= 0`) is true, an infinite loop is created. To prevent an infinite loop from occurring, `z` must be decremented so that it eventually becomes less than 0.

Exercises

- 4.10** Compare and contrast the `if` single-selection statement and the `while` repetition statement. How are these two statements similar? How are they different?

4.11 Explain what happens when a Java program attempts to divide one integer by another. What happens to the fractional part of the calculation? How can you avoid that outcome?

4.12 Describe the two ways in which control statements can be combined.

4.13 What type of repetition would be appropriate for calculating the sum of the first 100 positive integers? What type would be appropriate for calculating the sum of an arbitrary number of positive integers? Briefly describe how each of these tasks could be performed.

4.14 What is the difference between preincrementing and postincrementing a variable?

4.15 Identify and correct the errors in each of the following pieces of code. [Note: There may be more than one error in each piece of code.]

- a)

```
if (age >= 65);
    System.out.println("Age is greater than or equal to 65");
else
    System.out.println("Age is less than 65");
```
- b)

```
int x = 1, total;
while (x <= 10)
{
    total += x;
    ++x;
}
```
- c)

```
while (x <= 100)
    total += x;
    ++x;
```
- d)

```
while (y > 0)
{
    System.out.println(y);
    ++y;
```

4.16 What does the following program print?

```
1 // Exercise 4.16: Mystery.java
2 public class Mystery
3 {
4     public static void main(String[] args)
5     {
6         int x = 1;
7         int total = 0;
8
9         while (x <= 10)
10        {
11            int y = x * x;
12            System.out.println(y);
13            total += y;
14            ++x;
15        }
16
17        System.out.printf("Total is %d\n", total);
18    }
19 } // end class Mystery
```

For Exercise 4.17 through Exercise 4.20, perform each of the following steps:

- a) Read the problem statement.
- b) Formulate the algorithm using pseudocode and top-down, stepwise refinement.
- c) Write a Java program.

- d) Test, debug and execute the Java program.
- e) Process three complete sets of data.

4.17 (Gas Mileage) Drivers are concerned with the mileage their automobiles get. One driver has kept track of several trips by recording the miles driven and gallons used for each tankful. Develop a Java application that will input the miles driven and gallons used (both as integers) for each trip. The program should calculate and display the miles per gallon obtained for each trip and print the combined miles per gallon obtained for all trips up to this point. All averaging calculations should produce floating-point results. Use class `Scanner` and sentinel-controlled repetition to obtain the data from the user.

4.18 (Credit Limit Calculator) Develop a Java application that determines whether any of several department-store customers has exceeded the credit limit on a charge account. For each customer, the following facts are available:

- a) account number
- b) balance at the beginning of the month
- c) total of all items charged by the customer this month
- d) total of all credits applied to the customer's account this month
- e) allowed credit limit.

The program should input all these facts as integers, calculate the new balance ($= \text{beginning balance} + \text{charges} - \text{credits}$), display the new balance and determine whether the new balance exceeds the customer's credit limit. For those customers whose credit limit is exceeded, the program should display the message "Credit limit exceeded".

4.19 (Sales Commission Calculator) A large company pays its salespeople on a commission basis. The salespeople receive \$200 per week plus 9% of their gross sales for that week. For example, a salesperson who sells \$5,000 worth of merchandise in a week receives \$200 plus 9% of \$5000, or a total of \$650. You've been supplied with a list of the items sold by each salesperson. The values of these items are as follows:

Item	Value
1	239.99
2	129.75
3	99.95
4	350.89

Develop a Java application that inputs one salesperson's items sold for last week and calculates and displays that salesperson's earnings. There's no limit to the number of items that can be sold.

4.20 (Salary Calculator) Develop a Java application that determines the gross pay for each of three employees. The company pays straight time for the first 40 hours worked by each employee and time and a half for all hours worked in excess of 40. You're given a list of the employees, their number of hours worked last week and their hourly rates. Your program should input this information for each employee, then determine and display the employee's gross pay. Use class `Scanner` to input the data.

4.21 (Find the Largest Number) The process of finding the largest value is used frequently in computer applications. For example, a program that determines the winner of a sales contest would input the number of units sold by each salesperson. The salesperson who sells the most units wins the contest. Write a pseudocode program, then a Java application that inputs a series of 10 integers and determines and prints the largest integer. Your program should use at least the following three variables:

- a) counter: A counter to count to 10 (i.e., to keep track of how many numbers have been input and to determine when all 10 numbers have been processed).
- b) number: The integer most recently input by the user.
- c) largest: The largest number found so far.

4.22 (*Tabular Output*) Write a Java application that uses looping to print the following table of values:

N	10*N	100*N	1000*N
1	10	100	1000
2	20	200	2000
3	30	300	3000
4	40	400	4000
5	50	500	5000

4.23 (*Find the Two Largest Numbers*) Using an approach similar to that for Exercise 4.21, find the *two* largest values of the 10 values entered. [Note: You may input each number only once.]

4.24 (*Validating User Input*) Modify the program in Fig. 4.12 to validate its inputs. For any input, if the value entered is other than 1 or 2, keep looping until the user enters a correct value.

4.25 What does the following program print?

```

1  // Exercise 4.25: Mystery2.java
2  public class Mystery2
3  {
4      public static void main(String[] args)
5      {
6          int count = 1;
7
8          while (count <= 10)
9          {
10             System.out.println(count % 2 == 1 ? "****" : "+++++++");
11             ++count;
12         }
13     }
14 } // end class Mystery2

```

4.26 What does the following program print?

```

1  // Exercise 4.26: Mystery3.java
2  public class Mystery3
3  {
4      public static void main(String[] args)
5      {
6          int row = 10;
7
8          while (row >= 1)
9          {
10             int column = 1;
11
12             while (column <= 10)
13             {
14                 System.out.print(row % 2 == 1 ? "<" : ">");
15                 ++column;
16             }
17
18             --row;
19             System.out.println();
20         }
21     }
22 } // end class Mystery3

```


4.27 (Dangling-else Problem) Determine the output for each of the given sets of code when x is 9 and y is 11 and when x is 11 and y is 9. The compiler ignores the indentation in a Java program. Also, the Java compiler always associates an `else` with the immediately preceding `if` unless told to do otherwise by the placement of braces (`{}`). On first glance, you may not be sure which `if` a particular `else` matches—this situation is referred to as the “dangling-else problem.” We’ve eliminated the indentation from the following code to make the problem more challenging. [Hint: Apply the indentation conventions you’ve learned.]

- a) `if (x < 10)`
 `if (y > 10)`
 `System.out.println("*****");`
 `else`
 `System.out.println("#####");`
 `System.out.println("$$$$$");`
- b) `if (x < 10)`
 `{`
 `if (y > 10)`
 `System.out.println("*****");`
 `}`
 `else`
 `{`
 `System.out.println("#####");`
 `System.out.println("$$$$$");`
 `}`

4.28 (Another Dangling-else Problem) Modify the given code to produce the output shown in each part of the problem. Use proper indentation techniques. Make no changes other than inserting braces and changing the indentation of the code. The compiler ignores indentation in a Java program. We’ve eliminated the indentation from the given code to make the problem more challenging. [Note: It’s possible that no modification is necessary for some of the parts.]

```
if (y == 8)
if (x == 5)
System.out.println("@@@@");
else
System.out.println("#####");
System.out.println("$$$$$");
System.out.println("&&&&&");
```

- a) Assuming that $x = 5$ and $y = 8$, the following output is produced:

```
@@@@@
$$$$$
&&&&&
```

- b) Assuming that $x = 5$ and $y = 8$, the following output is produced:

```
@@@@@
```

- c) Assuming that $x = 5$ and $y = 8$, the following output is produced:

```
@@@@@
```

- d) Assuming that $x = 5$ and $y = 7$, the following output is produced. [Note: The last three output statements after the `else` are all part of a block.]

```
#####
$$$$$
&&&&&
```

4.29 (*Square of Asterisks*) Write an application that prompts the user to enter the size of the side of a square, then displays a hollow square of that size made of asterisks. Your program should work for squares of all side lengths between 1 and 20.

4.30 (*Palindromes*) A palindrome is a sequence of characters that reads the same backward as forward. For example, each of the following five-digit integers is a palindrome: 12321, 55555, 45554 and 11611. Write an application that reads in a five-digit integer and determines whether it's a palindrome. If the number is not five digits long, display an error message and allow the user to enter a new value.

4.31 (*Printing the Decimal Equivalent of a Binary Number*) Write an application that inputs an integer containing only 0s and 1s (i.e., a binary integer) and prints its decimal equivalent. [*Hint*: Use the remainder and division operators to pick off the binary number's digits one at a time, from right to left. In the decimal number system, the rightmost digit has a positional value of 1 and the next digit to the left a positional value of 10, then 100, then 1000, and so on. The decimal number 234 can be interpreted as $4 * 1 + 3 * 10 + 2 * 100$. In the binary number system, the rightmost digit has a positional value of 1, the next digit to the left a positional value of 2, then 4, then 8, and so on. The decimal equivalent of binary 1101 is $1 * 1 + 0 * 2 + 1 * 4 + 1 * 8$, or $1 + 0 + 4 + 8$ or, 13.]

4.32 (*Checkerboard Pattern of Asterisks*) Write an application that uses only the output statements

```
System.out.print("* ");
System.out.print(" ");
System.out.println();
```

to display the checkerboard pattern that follows. A `System.out.println` method call with no arguments causes the program to output a single newline character. [*Hint*: Repetition statements are required.]

```
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
```

4.33 (*Multiples of 2 with an Infinite Loop*) Write an application that keeps displaying in the command window the multiples of the integer 2—namely, 2, 4, 8, 16, 32, 64, and so on. Your loop should not terminate (i.e., it should create an infinite loop). What happens when you run this program?

4.34 (*What's Wrong with This Code?*) What is wrong with the following statement? Provide the correct statement to add one to the sum of `x` and `y`.

```
System.out.println(++(x + y));
```

4.35 (*Sides of a Triangle*) Write an application that reads three nonzero values entered by the user and determines and prints whether they could represent the sides of a triangle.

4.36 (*Sides of a Right Triangle*) Write an application that reads three nonzero integers and determines and prints whether they could represent the sides of a right triangle.

4.37 (*Factorial*) The factorial of a nonnegative integer n is written as $n!$ (pronounced “ n factorial”) and is defined as follows:

$$n! = n \cdot (n-1) \cdot (n-2) \cdot \dots \cdot 1 \quad (\text{for values of } n \text{ greater than or equal to } 1)$$

and

$$n! = 1 \quad (\text{for } n = 0)$$

For example, $5! = 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1$, which is 120.

- a) Write an application that reads a nonnegative integer and computes and prints its factorial.
- b) Write an application that estimates the value of the mathematical constant e by using the following formula. Allow the user to enter the number of terms to calculate.

$$e = 1 + \frac{1}{1!} + \frac{1}{2!} + \frac{1}{3!} + \dots$$

- c) Write an application that computes the value of e^x by using the following formula. Allow the user to enter the number of terms to calculate.

$$e^x = 1 + \frac{x}{1!} + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots$$

Making a Difference

4.38 (Enforcing Privacy with Cryptography) The explosive growth of Internet communications and data storage on Internet-connected computers has greatly increased privacy concerns. The field of cryptography is concerned with coding data to make it difficult (and hopefully—with the most advanced schemes—impossible) for unauthorized users to read. In this exercise you'll investigate a simple scheme for encrypting and decrypting data. A company that wants to send data over the Internet has asked you to write a program that will encrypt it so that it may be transmitted more securely. All the data is transmitted as four-digit integers. Your application should read a four-digit integer entered by the user and encrypt it as follows: Replace each digit with the result of adding 7 to the digit and getting the remainder after dividing the new value by 10. Then swap the first digit with the third, and swap the second digit with the fourth. Then print the encrypted integer. Write a separate application that inputs an encrypted four-digit integer and decrypts it (by reversing the encryption scheme) to form the original number. [*Optional reading project:* Research “public key cryptography” in general and the PGP (Pretty Good Privacy) specific public key scheme. You may also want to investigate the RSA scheme, which is widely used in industrial-strength applications.]

4.39 (World Population Growth) World population has grown considerably over the centuries. Continued growth could eventually challenge the limits of breathable air, drinkable water, arable cropland and other limited resources. There's evidence that growth has been slowing in recent years and that world population could peak some time this century, then start to decline.

For this exercise, research world population growth issues online. *Be sure to investigate various viewpoints.* Get estimates for the current world population and its growth rate (the percentage by which it's likely to increase this year). Write a program that calculates world population growth each year for the next 75 years, *using the simplifying assumption that the current growth rate will stay constant.* Print the results in a table. The first column should display the year from year 1 to year 75. The second column should display the anticipated world population at the end of that year. The third column should display the numerical increase in the world population that would occur that year. Using your results, determine the year in which the population would be double what it is today, if this year's growth rate were to persist.

5

Control Statements: Part 2; Logical Operators

The wheel is come full circle.
—William Shakespeare

*All the evolution we know of
proceeds from the vague to the
definite.*
—Charles Sanders Peirce

Objectives

In this chapter you'll:

- Learn the essentials of counter-controlled repetition.
- Use the `for` and `do...while` repetition statements to execute statements in a program repeatedly.
- Understand multiple selection using the `switch` selection statement.
- Use the `break` and `continue` program control statements to alter the flow of control.
- Use the logical operators to form complex conditional expressions in control statements.



5.1 Introduction	5.7 Class <code>AutoPolicy</code> Case Study: Strings in <code>switch</code> Statements
5.2 Essentials of Counter-Controlled Repetition	5.8 <code>break</code> and <code>continue</code> Statements
5.3 <code>for</code> Repetition Statement	5.9 Logical Operators
5.4 Examples Using the <code>for</code> Statement	5.10 Structured Programming Summary
5.5 <code>do...while</code> Repetition Statement	5.11 (Optional) GUI and Graphics Case Study: Drawing Rectangles and Ovals
5.6 <code>switch</code> Multiple-Selection Statement	5.12 Wrap-Up

Summary | Self-Review Exercises | Answers to Self-Review Exercises | Exercises | Making a Difference

5.1 Introduction

This chapter continues our presentation of structured programming theory and principles by introducing all but one of Java's remaining control statements. We demonstrate Java's `for`, `do...while` and `switch` statements. Through a series of short examples using `while` and `for`, we explore the essentials of counter-controlled repetition. We use a `switch` statement to count the number of A, B, C, D and F grade equivalents in a set of numeric grades entered by the user. We introduce the `break` and `continue` program-control statements. We discuss Java's logical operators, which enable you to use more complex conditional expressions in control statements. Finally, we summarize Java's control statements and the proven problem-solving techniques presented in this chapter and Chapter 4.

5.2 Essentials of Counter-Controlled Repetition

This section uses the `while` repetition statement introduced in Chapter 4 to formalize the elements required to perform counter-controlled repetition, which requires

1. a **control variable** (or loop counter)
2. the **initial value** of the control variable
3. the **increment** by which the control variable is modified each time through the loop (also known as **each iteration of the loop**)
4. the **loop-continuation condition** that determines if looping should continue.

To see these elements of counter-controlled repetition, consider the application of Fig. 5.1, which uses a loop to display the numbers from 1 through 10.

```

1 // Fig. 5.1: WhileCounter.java
2 // Counter-controlled repetition with the while repetition statement.
3
4 public class WhileCounter
5 {
6     public static void main(String[] args)
7     {
```

Fig. 5.1 | Counter-controlled repetition with the `while` repetition statement. (Part 1 of 2.)

```

8      int counter = 1; // declare and initialize control variable
9
10     while (counter <= 10) // loop-continuation condition
11     {
12         System.out.printf("%d ", counter);
13         ++counter; // increment control variable
14     }
15
16     System.out.println();
17 }
18 } // end class WhileCounter

```

```
1 2 3 4 5 6 7 8 9 10
```

Fig. 5.1 | Counter-controlled repetition with the `while` repetition statement. (Part 2 of 2.)

In Fig. 5.1, the elements of counter-controlled repetition are defined in lines 8, 10 and 13. Line 8 *declares* the control variable (`counter`) as an `int`, *reserves space* for it in memory and sets its *initial value* to 1. Variable `counter` also could have been declared and initialized with the following local-variable declaration and assignment statements:

```

int counter; // declare counter
counter = 1; // initialize counter to 1

```

Line 12 displays control variable `counter`'s value during each iteration of the loop. Line 13 *increments* the control variable by 1 for each iteration of the loop. The loop-continuation condition in the `while` (line 10) tests whether the value of the control variable is less than or equal to 10 (the final value for which the condition is true). The program performs the body of this `while` even when the control variable is 10. The loop terminates when the control variable exceeds 10 (i.e., `counter` becomes 11).



Common Programming Error 5.1

Because floating-point values may be approximate, controlling loops with floating-point variables may result in imprecise counter values and inaccurate termination tests.



Error-Prevention Tip 5.1

Use integers to control counting loops.

The program in Fig. 5.1 can be made more concise by initializing `counter` to 0 in line 8 and *preincrementing* `counter` in the `while` condition as follows:

```

while (++counter <= 10) // loop-continuation condition
    System.out.printf("%d ", counter);

```

This code saves a statement, because the `while` condition performs the increment before testing the condition. (Recall from Section 4.13 that the precedence of `++` is higher than that of `<=`.) Coding in such a condensed fashion takes practice, might make code more difficult to read, debug, modify and maintain, and typically should be avoided.



Software Engineering Observation 5.1

"Keep it simple" is good advice for most of the code you'll write.

5.3 for Repetition Statement

Section 5.2 presented the essentials of counter-controlled repetition. The `while` statement can be used to implement any counter-controlled loop. Java also provides the **for repetition statement**, which specifies the counter-controlled-repetition details in a single line of code. Figure 5.2 reimplements the application of Fig. 5.1 using `for`.

```

1  // Fig. 5.2: ForCounter.java
2  // Counter-controlled repetition with the for repetition statement.
3
4  public class ForCounter
5  {
6      public static void main(String[] args)
7      {
8          // for statement header includes initialization,
9          // loop-continuation condition and increment
10         for (int counter = 1; counter <= 10; counter++)
11             System.out.printf("%d ", counter);
12
13         System.out.println();
14     }
15 } // end class ForCounter

```

```
1 2 3 4 5 6 7 8 9 10
```

Fig. 5.2 | Counter-controlled repetition with the `for` repetition statement.

When the `for` statement (lines 10–11) begins executing, the control variable `counter` is *declared* and *initialized* to 1. (Recall from Section 5.2 that the first two elements of counter-controlled repetition are the *control variable* and its *initial value*.) Next, the program checks the *loop-continuation condition*, `counter <= 10`, which is between the two required semicolons. Because the initial value of `counter` is 1, the condition initially is true. Therefore, the body statement (line 11) displays control variable `counter`'s value, namely 1. After executing the loop's body, the program increments `counter` in the expression `counter++`, which appears to the right of the second semicolon. Then the loop-continuation test is performed again to determine whether the program should continue with the next iteration of the loop. At this point, the control variable's value is 2, so the condition is still true (the *final value* is not exceeded)—thus, the program performs the body statement again (i.e., the next iteration of the loop). This process continues until the numbers 1 through 10 have been displayed and the `counter`'s value becomes 11, causing the loop-continuation test to fail and repetition to terminate (after 10 repetitions of the loop body). Then the program performs the first statement after the `for`—in this case, line 13.

Figure 5.2 uses (in line 10) the loop-continuation condition `counter <= 10`. If you incorrectly specified `counter < 10` as the condition, the loop would iterate only nine times. This is a common *logic error* called an **off-by-one error**.



Common Programming Error 5.2

Using an incorrect relational operator or an incorrect final value of a loop counter in the loop-continuation condition of a repetition statement can cause an off-by-one error.

**Error-Prevention Tip 5.2**

Using the final value and operator `<=` in a loop's condition helps avoid off-by-one errors. For a loop that outputs 1 to 10, the loop-continuation condition should be `counter <= 10` rather than `counter < 10` (which causes an off-by-one error) or `counter < 11` (which is correct). Many programmers prefer so-called zero-based counting, in which to count 10 times, counter would be initialized to zero and the loop-continuation test would be `counter < 10`.

**Error-Prevention Tip 5.3**

As Chapter 4 mentioned, integers can overflow, causing logic errors. A loop's control variable also could overflow. Write your loop conditions carefully to prevent this.

A Closer Look at the `for` Statement's Header

Figure 5.3 takes a closer look at the `for` statement in Fig. 5.2. The first line—including the keyword `for` and everything in parentheses after `for` (line 10 in Fig. 5.2)—is sometimes called the **for statement header**. The `for` header “does it all”—it specifies each item needed for counter-controlled repetition with a control variable. If there's more than one statement in the body of the `for`, braces are required to define the body of the loop.

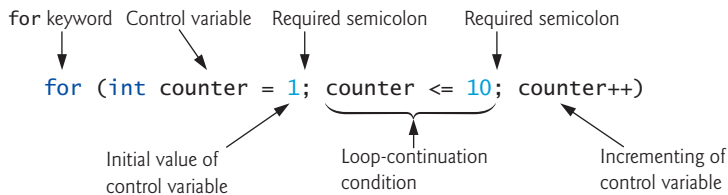


Fig. 5.3 | `for` statement header components.

General Format of a `for` Statement

The general format of the `for` statement is

```
for (initialization; loopContinuationCondition; increment)
    statement
```

where the *initialization* expression names the loop's control variable and *optionally* provides its initial value, *loopContinuationCondition* determines whether the loop should continue executing and *increment* modifies the control variable's value, so that the loop-continuation condition eventually becomes false. The two semicolons in the `for` header are required. If the loop-continuation condition is initially false, the program does *not* execute the `for` statement's body. Instead, execution proceeds with the statement following the `for`.

Representing a `for` Statement with an Equivalent `while` Statement

The `for` statement often can be represented with an equivalent `while` statement as follows:

```
initialization;
while (loopContinuationCondition)
{
    statement
    increment;
}
```


In Section 5.8, we show a case in which a `for` statement cannot be represented with an equivalent `while` statement. Typically, `for` statements are used for counter-controlled repetition and `while` statements for sentinel-controlled repetition. However, `while` and `for` can each be used for either repetition type.

Scope of a `for` Statement's Control Variable

If the *initialization* expression in the `for` header declares the control variable (i.e., the control variable's type is specified before the variable name, as in Fig. 5.2), the control variable can be used *only* in that `for` statement—it will not exist outside it. This restricted use is known as the variable's **scope**. The scope of a variable defines where it can be used in a program. For example, a *local variable* can be used *only* in the method that declares it and *only* from the point of declaration through the end of the method. Scope is discussed in detail in Chapter 6, *Methods: A Deeper Look*.



Common Programming Error 5.3

When a `for` statement's control variable is declared in the initialization section of the `for`'s header, using the control variable after the `for`'s body is a compilation error.

Expressions in a `for` Statement's Header Are Optional

All three expressions in a `for` header are optional. If the *loopContinuationCondition* is omitted, Java assumes that the loop-continuation condition is *always true*, thus creating an *infinite loop*. You might omit the *initialization* expression if the program initializes the control variable *before* the loop. You might omit the *increment* expression if the program calculates the increment with statements in the loop's body or if no increment is needed. The increment expression in a `for` acts as if it were a standalone statement at the end of the `for`'s body. Therefore, the expressions

```
counter = counter + 1
counter += 1
++counter
counter++
```

are equivalent increment expressions in a `for` statement. Many programmers prefer `counter++` because it's concise and because a `for` loop evaluates its increment expression *after* its body executes, so the postfix increment form seems more natural. In this case, the variable being incremented does not appear in a larger expression, so preincrementing and postincrementing actually have the *same* effect.



Common Programming Error 5.4

Placing a semicolon immediately to the right of the right parenthesis of a `for` header makes that `for`'s body an empty statement. This is normally a logic error.



Error-Prevention Tip 5.4

Infinite loops occur when the loop-continuation condition in a repetition statement never becomes `false`. To prevent this situation in a counter-controlled loop, ensure that the control variable is modified during each iteration of the loop so that the loop-continuation condition will eventually become `false`. In a sentinel-controlled loop, ensure that the sentinel value is able to be input.

Placing Arithmetic Expressions in a for Statement's Header

The initialization, loop-continuation condition and increment portions of a for statement can contain arithmetic expressions. For example, assume that $x = 2$ and $y = 10$. If x and y are not modified in the body of the loop, the statement

```
for (int j = x; j <= 4 * x * y; j += y / x)
```

is equivalent to the statement

```
for (int j = 2; j <= 80; j += 5)
```

The increment of a for statement may also be *negative*, in which case it's a **decrement**, and the loop counts *downward*.

Using a for Statement's Control Variable in the Statement's Body

Programs frequently display the control-variable value or use it in calculations in the loop body, but this use is *not* required. The control variable is commonly used to control repetition *without* being mentioned in the body of the for.

**Error-Prevention Tip 5.5**

Although the value of the control variable can be changed in the body of a for loop, avoid doing so, because this practice can lead to subtle errors.

UML Activity Diagram for the for Statement

The for statement's UML activity diagram is similar to that of the while statement (Fig. 4.6). Figure 5.4 shows the activity diagram of the for statement in Fig. 5.2. The diagram makes it clear that initialization occurs *once before* the loop-continuation test is evaluated the first time, and that incrementing occurs *each* time through the loop *after* the body statement executes.

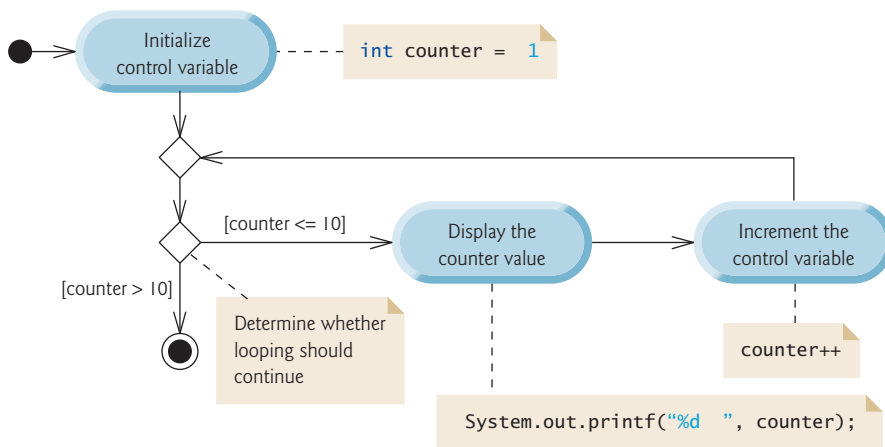


Fig. 5.4 | UML activity diagram for the for statement in Fig. 5.2.

5.4 Examples Using the for Statement

The following examples show techniques for varying the control variable in a for statement. In each case, we write *only* the appropriate for header. Note the change in the relational operator for the loops that *decrement* the control variable.

- a) Vary the control variable from 1 to 100 in increments of 1.

```
for (int i = 1; i <= 100; i++)
```

- b) Vary the control variable from 100 to 1 in *decrements* of 1.

```
for (int i = 100; i >= 1; i--)
```

- c) Vary the control variable from 7 to 77 in increments of 7.

```
for (int i = 7; i <= 77; i += 7)
```

- d) Vary the control variable from 20 to 2 in *decrements* of 2.

```
for (int i = 20; i >= 2; i -= 2)
```

- e) Vary the control variable over the values 2, 5, 8, 11, 14, 17, 20.

```
for (int i = 2; i <= 20; i += 3)
```

- f) Vary the control variable over the values 99, 88, 77, 66, 55, 44, 33, 22, 11, 0.

```
for (int i = 99; i >= 0; i -= 11)
```



Common Programming Error 5.5

Using an incorrect relational operator in the loop-continuation condition of a loop that counts downward (e.g., using `i <= 1` instead of `i >= 1` in a loop counting down to 1) is usually a logic error.



Common Programming Error 5.6

Do not use equality operators (`!=` or `==`) in a loop-continuation condition if the loop's control variable increments or decrements by more than 1. For example, consider the for statement header `for (int counter = 1; counter != 10; counter += 2)`. The loop-continuation test `counter != 10` never becomes false (resulting in an infinite loop) because `counter` increments by 2 after each iteration.

Application: Summing the Even Integers from 2 to 20

We now consider two sample applications that demonstrate simple uses of for. The application in Fig. 5.5 uses a for statement to sum the even integers from 2 to 20 and store the result in an `int` variable called `total`.

```
1 // Fig. 5.5: Sum.java
2 // Summing integers with the for statement.
3
4 public class Sum
5 {
```

Fig. 5.5 | Summing integers with the for statement. (Part I of 2.)

```

6   public static void main(String[] args)
7   {
8       int total = 0;
9
10      // total even integers from 2 through 20
11      for (int number = 2; number <= 20; number += 2)
12          total += number;
13
14      System.out.printf("Sum is %d\n", total);
15  }
16 } // end class Sum

```

Sum is 110

Fig. 5.5 | Summing integers with the for statement. (Part 2 of 2.)

The *initialization* and *increment* expressions can be comma-separated lists that enable you to use multiple initialization expressions or multiple increment expressions. For example, *although this is discouraged*, you could merge the body of the for statement in lines 11–12 of Fig. 5.5 into the increment portion of the for header by using a comma as follows:

```

for (int number = 2; number <= 20; total += number, number += 2)
    ; // empty statement

```



Good Programming Practice 5.1

For readability limit the size of control-statement headers to a single line if possible.

Application: Compound-Interest Calculations

Let's use the for statement to compute compound interest. Consider the following problem:

A person invests \$1,000 in a savings account yielding 5% interest. Assuming that all the interest is left on deposit, calculate and print the amount of money in the account at the end of each year for 10 years. Use the following formula to determine the amounts:

$$a = p(1 + r)^n$$

where

p is the original amount invested (i.e., the principal)

r is the annual interest rate (e.g., use 0.05 for 5%)

n is the number of years

a is the amount on deposit at the end of the n th year.

The solution to this problem (Fig. 5.6) involves a loop that performs the indicated calculation for each of the 10 years the money remains on deposit. Lines 8–10 in method `main` declare `double` variables `amount`, `principal` and `rate`, and initialize `principal` to 1000.0 and `rate` to 0.05. Java treats floating-point constants like 1000.0 and 0.05 as type `double`. Similarly, Java treats whole-number constants like 7 and -22 as type `int`.

```

1 // Fig. 5.6: Interest.java
2 // Compound-interest calculations with for.
3
4 public class Interest
5 {
6     public static void main(String[] args)
7     {
8         double amount; // amount on deposit at end of each year
9         double principal = 1000.0; // initial amount before interest
10        double rate = 0.05; // interest rate
11
12        // display headers
13        System.out.printf("%s%20s\n", "Year", "Amount on deposit");
14
15        // calculate amount on deposit for each of ten years
16        for (int year = 1; year <= 10; ++year)
17        {
18            // calculate new amount for specified year
19            amount = principal * Math.pow(1.0 + rate, year);
20
21            // display the year and the amount
22            System.out.printf("%4d%,20.2f\n", year, amount);
23        }
24    }
25 } // end class Interest

```

Year	Amount on deposit
1	1,050.00
2	1,102.50
3	1,157.63
4	1,215.51
5	1,276.28
6	1,340.10
7	1,407.10
8	1,477.46
9	1,551.33
10	1,628.89

Fig. 5.6 | Compound-interest calculations with for.

Formatting Strings with Field Widths and Justification

Line 13 outputs the headers for two columns of output. The first column displays the year and the second column the amount on deposit at the end of that year. We use the format specifier `%20s` to output the String "Amount on Deposit". The integer 20 between the % and the conversion character `s` indicates that the value should be displayed with a **field width** of 20—that is, `printf` displays the value with at least 20 character positions. If the value to be output is less than 20 character positions wide (17 characters in this example), the value is **right justified** in the field by default. If the year value to be output were more than four character positions wide, the field width would be extended to the right to accommodate the entire value—this would push the amount field to the right, upsetting the neat columns of our tabular output. To output values **left justified**, simply precede the field width with the **minus sign (-) formatting flag** (e.g., `%-20s`).

Performing the Interest Calculations with static Method pow of Class Math

The `for` statement (lines 16–23) executes its body 10 times, varying control variable `year` from 1 to 10 in increments of 1. This loop terminates when `year` becomes 11. (Variable `year` represents n in the problem statement.)

Classes provide methods that perform common tasks on objects. In fact, most methods must be called on a specific object. For example, to output text in Fig. 5.6, line 13 calls method `printf` on the `System.out` object. Some classes also provide methods that perform common tasks and do *not* require you to first create objects of those classes. These are called *static methods*. For example, Java does not include an exponentiation operator, so the designers of Java's `Math` class defined static method `pow` for raising a value to a power. You can call a static method by specifying the *class name* followed by a dot (`.`) and the method name, as in

```
ClassName.methodName(arguments)
```

In Chapter 6, you'll learn how to implement static methods in your own classes.

We use static method `pow` of class `Math` to perform the compound-interest calculation in Fig. 5.6. `Math.pow(x, y)` calculates the value of x raised to the y^{th} power. The method receives two `double` arguments and returns a `double` value. Line 19 performs the calculation $a = p(1 + r)^n$, where a is amount, p is principal, r is rate and n is year. Class `Math` is defined in package `java.lang`, so you do *not* need to import class `Math` to use it.

The body of the `for` statement contains the calculation `1.0 + rate`, which appears as an argument to the `Math.pow` method. In fact, this calculation produces the *same* result each time through the loop, so repeating it in every iteration of the loop is wasteful.

**Performance Tip 5.1**

In loops, avoid calculations for which the result never changes—such calculations should typically be placed before the loop. Many of today's sophisticated optimizing compilers will place such calculations outside loops in the compiled code.

Formatting Floating-Point Numbers

After each calculation, line 22 outputs the year and the amount on deposit at the end of that year. The year is output in a field width of four characters (as specified by `%4d`). The amount is output as a floating-point number with the format specifier `%,20.2f`. The **comma (,) formatting flag** indicates that the floating-point value should be output with a **grouping separator**. The actual separator used is specific to the user's locale (i.e., country). For example, in the United States, the number will be output using commas to separate every three digits and a decimal point to separate the fractional part of the number, as in 1,234.45. The number 20 in the format specification indicates that the value should be output right justified in a *field width* of 20 characters. The `.2` specifies the formatted number's *precision*—in this case, the number is *rounded* to the nearest hundredth and output with two digits to the right of the decimal point.

A Warning about Displaying Rounded Values

We declared variables `amount`, `principal` and `rate` to be of type `double` in this example. We're dealing with fractional parts of dollars and thus need a type that allows decimal points in its values. Unfortunately, floating-point numbers can cause trouble. Here's a simple explanation of what can go wrong when using `double` (or `float`) to represent dollar

amounts (assuming that dollar amounts are displayed with two digits to the right of the decimal point): Two `double` dollar amounts stored in the machine could be 14.234 (which would normally be rounded to 14.23 for display purposes) and 18.673 (which would normally be rounded to 18.67 for display purposes). When these amounts are added, they produce the internal sum 32.907, which would normally be rounded to 32.91 for display purposes. Thus, your output could appear as

```
14.23
+ 18.67
-----
32.91
```

but a person adding the individual numbers as displayed would expect the sum to be 32.90. You've been warned!



Error-Prevention Tip 5.6

Do not use variables of type `double` (or `float`) to perform precise monetary calculations. The imprecision of floating-point numbers can lead to errors. In the exercises, you'll learn how to use integers to perform precise monetary calculations—Java also provides class `java.math.BigDecimal` for this purpose, which we demonstrate in Fig. 8.16.

5.5 do...while Repetition Statement

The **do...while repetition statement** is similar to the `while` statement. In the `while`, the program tests the loop-continuation condition at the *beginning* of the loop, *before* executing the loop's body; if the condition is *false*, the body *never* executes. The `do...while` statement tests the loop-continuation condition *after* executing the loop's body; therefore, *the body always executes at least once*. When a `do...while` statement terminates, execution continues with the next statement in sequence. Figure 5.7 uses a `do...while` to output the numbers 1–10.

```
1 // Fig. 5.7: DoWhileTest.java
2 // do...while repetition statement.
3
4 public class DoWhileTest
5 {
6     public static void main(String[] args)
7     {
8         int counter = 1;
9
10        do
11        {
12            System.out.printf("%d ", counter);
13            ++counter;
14        } while (counter <= 10); // end do...while
15
16        System.out.println();
17    }
18 } // end class DoWhileTest
```

Fig. 5.7 | do...while repetition statement. (Part 1 of 2.)

```

1  2  3  4  5  6  7  8  9 10

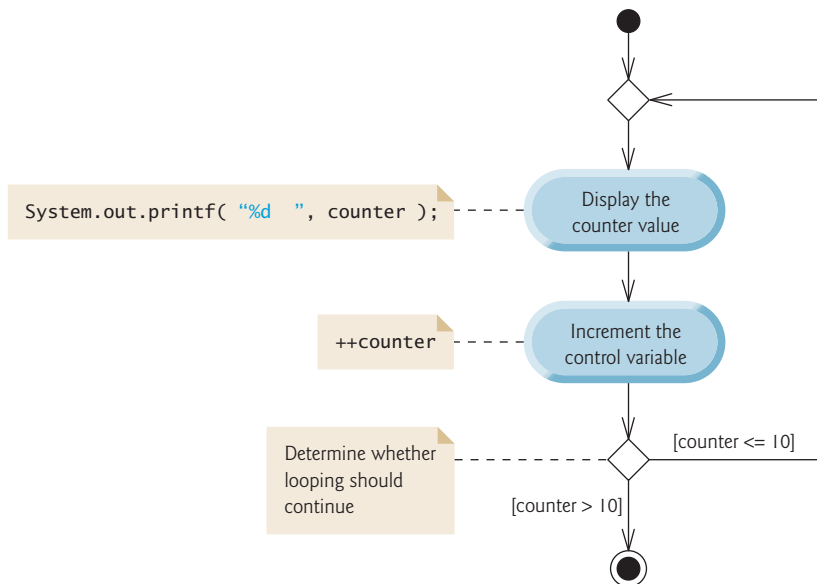
```

Fig. 5.7 | `do...while` repetition statement. (Part 2 of 2.)

Line 8 declares and initializes control variable `counter`. Upon entering the `do...while` statement, line 12 outputs `counter`'s value and line 13 increments `counter`. Then the program evaluates the loop-continuation test at the *bottom* of the loop (line 14). If the condition is *true*, the loop continues at the first body statement (line 12). If the condition is *false*, the loop terminates and the program continues at the next statement after the loop.

UML Activity Diagram for the `do...while` Repetition Statement

Figure 5.8 contains the UML activity diagram for the `do...while` statement. This diagram makes it clear that the loop-continuation condition is not evaluated until *after* the loop performs the action state *at least once*. Compare this activity diagram with that of the `while` statement (Fig. 4.6).

**Fig. 5.8** | `do...while` repetition statement UML activity diagram.

Braces in a `do...while` Repetition Statement

It isn't necessary to use braces in the `do...while` repetition statement if there's only one statement in the body. However, many programmers include the braces, to avoid confusion between the `while` and `do...while` statements. For example,

```
while (condition)
```

is normally the first line of a `while` statement. A `do...while` statement with no braces around a single-statement body appears as:


```
do
    statement
while (condition);
```

which can be confusing. A reader may misinterpret the last line—`while (condition);`—as a `while` statement containing an empty statement (the semicolon by itself). Thus, the `do...while` statement with one body statement is usually written with braces as follows:

```
do
{
    statement
} while (condition);
```



Good Programming Practice 5.2

Always include braces in a `do...while` statement. This helps eliminate ambiguity between the `while` statement and a `do...while` statement containing only one statement.

5.6 `switch` Multiple-Selection Statement

Chapter 4 discussed the `if` single-selection statement and the `if...else` double-selection statement. The **`switch` multiple-selection statement** performs different actions based on the possible values of a **constant integral expression** of type `byte`, `short`, `int` or `char`. As of Java SE 7, the expression may also be a `String`, which we discuss in Section 5.7.

Using a `switch` Statement to Count A, B, C, D and F Grades

Figure 5.9 calculates the class average of a set of numeric grades entered by the user, and uses a `switch` statement to determine whether each grade is the equivalent of an A, B, C, D or F and to increment the appropriate grade counter. The program also displays a summary of the number of students who received each grade.

Like earlier versions of the class-average program, the `main` method of class `LetterGrades` (Fig. 5.9) declares local variables `total` (line 9) and `gradeCounter` (line 10) to keep track of the sum of the grades entered by the user and the number of grades entered, respectively. Lines 11–15 declare counter variables for each grade category. Note that the variables in lines 9–15 are explicitly initialized to 0.

Method `main` has two key parts. Lines 26–56 read an arbitrary number of integer grades from the user using sentinel-controlled repetition, update instance variables `total` and `gradeCounter`, and increment an appropriate letter-grade counter for each grade entered. Lines 59–80 output a report containing the total of all grades entered, the average of the grades and the number of students who received each letter grade. Let's examine these parts in more detail.

```
1 // Fig. 5.9: LetterGrades.java
2 // LetterGrades class uses the switch statement to count letter grades.
3 import java.util.Scanner;
4
5 public class LetterGrades
6 {
7     public static void main(String[] args)
8     {
```

Fig. 5.9 | `LetterGrades` class uses the `switch` statement to count letter grades. (Part I of 3.)

```

9      int total = 0; // sum of grades
10     int gradeCounter = 0; // number of grades entered
11     int aCount = 0; // count of A grades
12     int bCount = 0; // count of B grades
13     int cCount = 0; // count of C grades
14     int dCount = 0; // count of D grades
15     int fCount = 0; // count of F grades
16
17     Scanner input = new Scanner(System.in);
18
19     System.out.printf("%s\n%s\n  %s\n  %s\n",
20         "Enter the integer grades in the range 0-100.",
21         "Type the end-of-file indicator to terminate input:",
22         "On UNIX/Linux/Mac OS X type <Ctrl> d then press Enter",
23         "On Windows type <Ctrl> z then press Enter");
24
25     // loop until user enters the end-of-file indicator
26     while (input.hasNext())
27     {
28         int grade = input.nextInt(); // read grade
29         total += grade; // add grade to total
30         ++gradeCounter; // increment number of grades
31
32         // increment appropriate letter-grade counter
33         switch (grade / 10)
34         {
35             case 9: // grade was between 90
36                 case 10: // and 100, inclusive
37                     ++aCount;
38                     break; // exits switch
39
40             case 8: // grade was between 80 and 89
41                 ++bCount;
42                 break; // exits switch
43
44             case 7: // grade was between 70 and 79
45                 ++cCount;
46                 break; // exits switch
47
48             case 6: // grade was between 60 and 69
49                 ++dCount;
50                 break; // exits switch
51
52             default: // grade was less than 60
53                 ++fCount;
54                 break; // optional; exits switch anyway
55         } // end switch
56     } // end while
57
58     // display grade report
59     System.out.printf("\nGrade Report:\n");
60

```

Fig. 5.9 | LetterGrades class uses the switch statement to count letter grades. (Part 2 of 3.)

```

61 // if user entered at least one grade...
62 if (gradeCounter != 0)
63 {
64     // calculate average of all grades entered
65     double average = (double) total / gradeCounter;
66
67     // output summary of results
68     System.out.printf("Total of the %d grades entered is %d\n",
69         gradeCounter, total);
70     System.out.printf("Class average is %.2f\n", average);
71     System.out.printf("%n%s%n%s%d%n%s%d%n%s%d%n%s%d%n%s%d\n",
72         "Number of students who received each grade:",
73         "A: ", aCount, // display number of A grades
74         "B: ", bCount, // display number of B grades
75         "C: ", cCount, // display number of C grades
76         "D: ", dCount, // display number of D grades
77         "F: ", fCount); // display number of F grades
78 } // end if
79 else // no grades were entered, so output appropriate message
80     System.out.println("No grades were entered");
81 } // end main
82 } // end class LetterGrades

```

Enter the integer grades in the range 0-100.
 Type the end-of-file indicator to terminate input:
 On UNIX/Linux/Mac OS X type <Ctrl> d then press Enter
 On Windows type <Ctrl> z then press Enter

```

99
92
45
57
63
71
76
85
90
100
^Z

```

Grade Report:
 Total of the 10 grades entered is 778
 Class average is 77.80

Number of students who received each grade:
 A: 4
 B: 1
 C: 2
 D: 1
 F: 2

Fig. 5.9 | LetterGrades class uses the switch statement to count letter grades. (Part 3 of 3.)

Reading Grades from the User

Lines 19–23 prompt the user to enter integer grades and to type the end-of-file indicator to terminate the input. The **end-of-file indicator** is a system-dependent keystroke combi-

nation which the user enters to indicate that there's *no more data to input*. In Chapter 15, Files, Streams and Object Serialization, you'll see how the end-of-file indicator is used when a program reads its input from a file.

On UNIX/Linux/Mac OS X systems, end-of-file is entered by typing the sequence

```
<Ctrl> d
```

on a line by itself. This notation means to simultaneously press both the *Ctrl* key and the *d* key. On Windows systems, end-of-file can be entered by typing

```
<Ctrl> z
```

[*Note:* On some systems, you must press *Enter* after typing the end-of-file key sequence. Also, Windows typically displays the characters ^Z on the screen when the end-of-file indicator is typed, as shown in the output of Fig. 5.9.]



Portability Tip 5.1

The keystroke combinations for entering end-of-file are system dependent.

The `while` statement (lines 26–56) obtains the user input. The condition at line 26 calls `Scanner` method `hasNext` to determine whether there's more data to input. This method returns the boolean value `true` if there's more data; otherwise, it returns `false`. The returned value is then used as the value of the condition in the `while` statement. Method `hasNext` returns `false` once the user types the end-of-file indicator.

Line 28 inputs a grade value from the user. Line 29 adds grade to `total`. Line 30 increments `gradeCounter`. These variables are used to compute the average of the grades. Lines 33–55 use a `switch` statement to increment the appropriate letter-grade counter based on the numeric grade entered.

Processing the Grades

The `switch` statement (lines 33–55) determines which counter to increment. We assume that the user enters a valid grade in the range 0–100. A grade in the range 90–100 represents A, 80–89 represents B, 70–79 represents C, 60–69 represents D and 0–59 represents F. The `switch` statement consists of a block that contains a sequence of **case labels** and an optional **default case**. These are used in this example to determine which counter to increment based on the grade.

When the flow of control reaches the `switch`, the program evaluates the expression in the parentheses (`grade / 10`) following keyword `switch`. This is the `switch`'s **controlling expression**. The program compares this expression's value (which must evaluate to an integral value of type `byte`, `char`, `short` or `int`, or to a `String`) with each case label. The controlling expression in line 33 performs integer division, which *truncates the fractional part* of the result. Thus, when we divide a value from 0 to 100 by 10, the result is always a value from 0 to 10. We use several of these values in our case labels. For example, if the user enters the integer 85, the controlling expression evaluates to 8. The `switch` compares 8 with each case label. If a match occurs (case 8: at line 40), the program executes that case's statements. For the integer 8, line 41 increments `bCount`, because a grade in the 80s is a B. The **break statement** (line 42) causes program control to proceed with the first statement after the `switch`—in this program, we reach the end of the `while` loop, so con-

control returns to the loop-continuation condition in line 26 to determine whether the loop should continue executing.

The cases in our `switch` explicitly test for the values 10, 9, 8, 7 and 6. Note the cases at lines 35–36 that test for the values 9 and 10 (both of which represent the grade A). Listing cases consecutively in this manner with no statements between them enables the cases to perform the same set of statements—when the controlling expression evaluates to 9 or 10, the statements in lines 37–38 will execute. The `switch` statement does *not* provide a mechanism for testing *ranges* of values, so *every* value you need to test must be listed in a separate case label. Each case can have multiple statements. The `switch` statement differs from other control statements in that it does *not* require braces around multiple statements in a case.

case without a break Statement

Without break statements, each time a match occurs in the `switch`, the statements for that case and subsequent cases execute until a break statement or the end of the `switch` is encountered. This is often referred to as “falling through” to the statements in subsequent cases. (This feature is perfect for writing a concise program that displays the iterative song “The Twelve Days of Christmas” in Exercise 5.29.)



Common Programming Error 5.7

Forgetting a break statement when one is needed in a switch is a logic error.

The default Case

If no match occurs between the controlling expression’s value and a case label, the default case (lines 52–54) executes. We use the default case in this example to process all controlling-expression values that are less than 6—that is, all failing grades. If no match occurs and the `switch` does not contain a default case, program control simply continues with the first statement after the `switch`.



Error-Prevention Tip 5.7

In a switch statement, ensure that you test for all possible values of the controlling expression.

Displaying the Grade Report

Lines 59–80 output a report based on the grades entered (as shown in the input/output window in Fig. 5.9). Line 62 determines whether the user entered at least one grade—this helps us avoid dividing by zero. If so, line 65 calculates the average of the grades. Lines 68–77 then output the total of all the grades, the class average and the number of students who received each letter grade. If no grades were entered, line 80 outputs an appropriate message. The output in Fig. 5.9 shows a sample grade report based on 10 grades.

switch Statement UML Activity Diagram

Figure 5.10 shows the UML activity diagram for the general `switch` statement. Most `switch` statements use a break in each case to terminate the `switch` statement after processing the case. Figure 5.10 emphasizes this by including break statements in the activity diagram. The diagram makes it clear that the break statement at the end of a case causes control to exit the `switch` statement immediately.

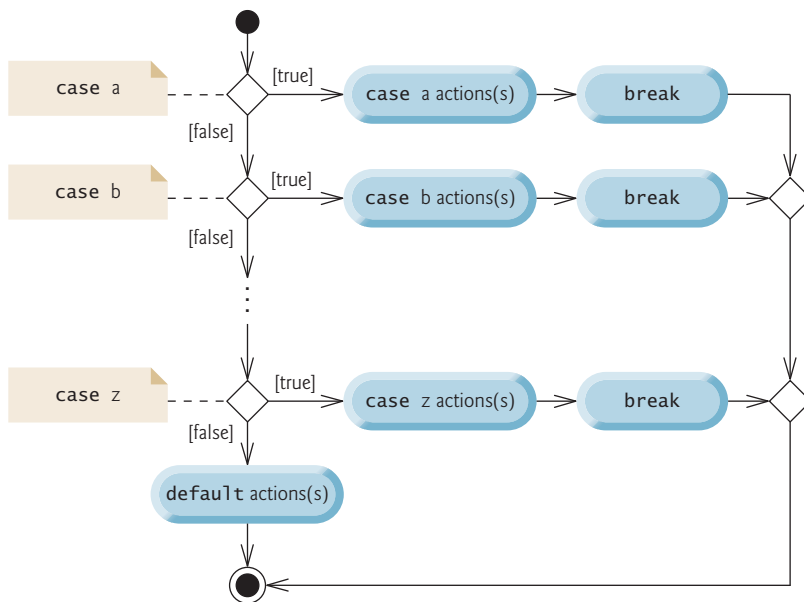


Fig. 5.10 | switch multiple-selection statement UML activity diagram with break statements.

The break statement is *not* required for the switch's last case (or the optional default case, when it appears last), because execution continues with the next statement after the switch.



Error-Prevention Tip 5.8

Provide a default case in switch statements. This focuses you on the need to process exceptional conditions.



Good Programming Practice 5.3

Although each case and the default case in a switch can occur in any order, place the default case last. When the default case is listed last, the break for that case is not required.

Notes on the Expression in Each case of a switch

When using the switch statement, remember that each case must contain a constant integral expression—that is, any combination of integer constants that evaluates to a constant integer value (e.g., `-7`, `0` or `221`)—or a `String`. An integer constant is simply an integer value. In addition, you can use **character constants**—specific characters in single quotes, such as `'A'`, `'7'` or `'$'`—which represent the integer values of characters and enum constants (introduced in Section 6.10). (Appendix B shows the integer values of the characters in the ASCII character set, which is a subset of the Unicode® character set used by Java.)

The expression in each case can also be a **constant variable**—a variable containing a value which does not change for the entire program. Such a variable is declared with keyword `final` (discussed in Chapter 6). Java has a feature called enum types, which we also present in Chapter 6—enum type constants can also be used in case labels.

In Chapter 10, Object-Oriented Programming: Polymorphism and Interfaces, we present a more elegant way to implement `switch` logic—we use a technique called *polymorphism* to create programs that are often clearer, easier to maintain and easier to extend than programs using `switch` logic.

5.7 Class `AutoPolicy` Case Study: Strings in `switch` Statements

Strings can be used as controlling expressions in `switch` statements, and `String` literals can be used in case labels. To demonstrate that `Strings` can be used as controlling expressions in `switch` statements and that `String` literals can be used in case labels, we'll implement an app that meets the following requirements:

You've been hired by an auto insurance company that serves these northeast states—Connecticut, Maine, Massachusetts, New Hampshire, New Jersey, New York, Pennsylvania, Rhode Island and Vermont. The company would like you to create a program that produces a report indicating for each of their auto insurance policies whether the policy is held in a state with “no-fault” auto insurance—Massachusetts, New Jersey, New York and Pennsylvania.

The Java app that meets these requirements contains two classes—`AutoPolicy` (Fig. 5.11) and `AutoPolicyTest` (Fig. 5.12).

Class `AutoPolicy`

Class `AutoPolicy` (Fig. 5.11) represents an auto insurance policy. The class contains:

- `int` instance variable `accountNumber` (line 5) to store the policy's account number
- `String` instance variable `makeAndModel` (line 6) to store the car's make and model (such as a "Toyota Camry")
- `String` instance variable `state` (line 7) to store a two-character state abbreviation representing the state in which the policy is held (e.g., "MA" for Massachusetts)
- a constructor (lines 10–15) that initializes the class's instance variables
- methods `setAccountNumber` and `getAccountNumber` (lines 18–27) to *set* and *get* an `AutoPolicy`'s `accountNumber` instance variable
- methods `setMakeAndModel` and `getMakeAndModel` (lines 30–39) to *set* and *get* an `AutoPolicy`'s `makeAndModel` instance variable
- methods `setState` and `getState` (lines 42–51) to *set* and *get* an `AutoPolicy`'s `state` instance variable
- method `isNoFaultState` (lines 54–70) to return a `boolean` value indicating whether the policy is held in a no-fault auto insurance state; note the method name—the naming convention for a *get* method that returns a `boolean` value is to begin the name with "is" rather than "get" (such a method is commonly called a *predicate method*).

In method `isNoFaultState`, the `switch` statement's controlling expression (line 59) is the `String` returned by `AutoPolicy` method `getState`. The `switch` statement compares the controlling expression's value with the case labels (line 61) to determine whether the policy is held in Massachusetts, New Jersey, New York or Pennsylvania (the no-fault states).

If there's a match, then line 62 sets local variable `noFaultState` to `true` and the `switch` statement terminates; otherwise, the default case sets `noFaultState` to `false` (line 65). Then method `isNoFaultState` returns local variable `noFaultState`'s value.

For simplicity, we did not validate an `AutoPolicy`'s data in the constructor or *set* methods, and we assume that state abbreviations are always two uppercase letters. In addition, a real `AutoPolicy` class would likely contain many other instance variables and methods for data such as the account holder's name, address, etc. In Exercise 5.30, you'll be asked to enhance class `AutoPolicy` by validating its state abbreviation using techniques that you'll learn in Section 5.9.

```

1  // Fig. 5.11: AutoPolicy.java
2  // Class that represents an auto insurance policy.
3  public class AutoPolicy
4  {
5      private int accountNumber; // policy account number
6      private String makeAndModel; // car that the policy applies to
7      private String state; // two-letter state abbreviation
8
9      // constructor
10     public AutoPolicy(int accountNumber, String makeAndModel, String state)
11     {
12         this.accountNumber = accountNumber;
13         this.makeAndModel = makeAndModel;
14         this.state = state;
15     }
16
17     // sets the accountNumber
18     public void setAccountNumber(int accountNumber)
19     {
20         this.accountNumber = accountNumber;
21     }
22
23     // returns the accountNumber
24     public int getAccountNumber()
25     {
26         return accountNumber;
27     }
28
29     // sets the makeAndModel
30     public void setMakeAndModel(String makeAndModel)
31     {
32         this.makeAndModel = makeAndModel;
33     }
34
35     // returns the makeAndModel
36     public String getMakeAndModel()
37     {
38         return makeAndModel;
39     }
40

```

Fig. 5.11 | Class that represents an auto insurance policy. (Part 1 of 2.)


```
41 // sets the state
42 public void setState(String state)
43 {
44     this.state = state;
45 }
46
47 // returns the state
48 public String getState()
49 {
50     return state;
51 }
52
53 // predicate method returns whether the state has no-fault insurance
54 public boolean isNoFaultState()
55 {
56     boolean noFaultState;
57
58     // determine whether state has no-fault auto insurance
59     switch (getState()) // get AutoPolicy object's state abbreviation
60     {
61         case "MA": case "NJ": case "NY": case "PA":
62             noFaultState = true;
63             break;
64         default:
65             noFaultState = false;
66             break;
67     }
68
69     return noFaultState;
70 }
71 } // end class AutoPolicy
```

Fig. 5.11 | Class that represents an auto insurance policy. (Part 2 of 2.)

Class AutoPolicyTest

Class `AutoPolicyTest` (Fig. 5.12) creates two `AutoPolicy` objects (lines 8–11 in main). Lines 14–15 pass each object to static method `policyInNoFaultState` (lines 20–28), which uses `AutoPolicy` methods to determine and display whether the object it receives represents a policy in a no-fault auto insurance state.

```
1 // Fig. 5.12: AutoPolicyTest.java
2 // Demonstrating Strings in switch.
3 public class AutoPolicyTest
4 {
5     public static void main(String[] args)
6     {
7         // create two AutoPolicy objects
8         AutoPolicy policy1 =
9             new AutoPolicy(11111111, "Toyota Camry", "NJ");
10        AutoPolicy policy2 =
11            new AutoPolicy(22222222, "Ford Fusion", "ME");
```

Fig. 5.12 | Demonstrating Strings in switch. (Part 1 of 2.)

```

12
13     // display whether each policy is in a no-fault state
14     policyInNoFaultState(policy1);
15     policyInNoFaultState(policy2);
16 }
17
18 // method that displays whether an AutoPolicy
19 // is in a state with no-fault auto insurance
20 public static void policyInNoFaultState(AutoPolicy policy)
21 {
22     System.out.println("The auto policy:");
23     System.out.printf(
24         "Account #: %d; Car: %s; State %s %s a no-fault state%n%n",
25         policy.getAccountNumber(), policy.getMakeAndModel(),
26         policy.getState(),
27         (policy.isNoFaultState() ? "is": "is not"));
28 }
29 } // end class AutoPolicyTest

```

```

The auto policy:
Account #: 11111111; Car: Toyota Camry;
State NJ is a no-fault state

```

```

The auto policy:
Account #: 22222222; Car: Ford Fusion;
State ME is not a no-fault state

```

Fig. 5.12 | Demonstrating Strings in switch. (Part 2 of 2.)

5.8 break and continue Statements

In addition to selection and repetition statements, Java provides statements **break** (which we discussed in the context of the **switch** statement) and **continue** (presented in this section and online Appendix L) to alter the flow of control. The preceding section showed how **break** can be used to terminate a **switch** statement's execution. This section discusses how to use **break** in repetition statements.

break Statement

The **break** statement, when executed in a **while**, **for**, **do...while** or **switch**, causes *immediate* exit from that statement. Execution continues with the first statement after the control statement. Common uses of the **break** statement are to escape early from a loop or to skip the remainder of a **switch** (as in Fig. 5.9). Figure 5.13 demonstrates a **break** statement exiting a **for**.

```

1 // Fig. 5.13: BreakTest.java
2 // break statement exiting a for statement.
3 public class BreakTest
4 {
5     public static void main(String[] args)
6     {

```

Fig. 5.13 | **break** statement exiting a **for** statement. (Part 1 of 2.)

```

7      int count; // control variable also used after loop terminates
8
9      for (count = 1; count <= 10; count++) // loop 10 times
10     {
11         if (count == 5)
12             break; // terminates loop if count is 5
13
14         System.out.printf("%d ", count);
15     }
16
17     System.out.printf("\nBroke out of loop at count = %d\n", count);
18 }
19 } // end class BreakTest

```

```

1 2 3 4
Broke out of loop at count = 5

```

Fig. 5.13 | break statement exiting a for statement. (Part 2 of 2.)

When the if statement nested at lines 11–12 in the for statement (lines 9–15) detects that count is 5, the break statement at line 12 executes. This terminates the for statement, and the program proceeds to line 17 (immediately after the for statement), which displays a message indicating the value of the control variable when the loop terminated. The loop fully executes its body only four times instead of 10.

continue Statement

The continue statement, when executed in a while, for or do...while, skips the remaining statements in the loop body and proceeds with the *next iteration* of the loop. In while and do...while statements, the program evaluates the loop-continuation test immediately after the continue statement executes. In a for statement, the increment expression executes, then the program evaluates the loop-continuation test.

```

1  // Fig. 5.14: ContinueTest.java
2  // continue statement terminating an iteration of a for statement.
3  public class ContinueTest
4  {
5      public static void main(String[] args)
6      {
7          for (int count = 1; count <= 10; count++) // loop 10 times
8          {
9              if (count == 5)
10                 continue; // skip remaining code in loop body if count is 5
11
12                 System.out.printf("%d ", count);
13             }
14
15             System.out.printf("\nUsed continue to skip printing 5\n");
16         }
17     } // end class ContinueTest

```

Fig. 5.14 | continue statement terminating an iteration of a for statement. (Part 1 of 2.)

```
1 2 3 4 6 7 8 9 10
Used continue to skip printing 5
```

Fig. 5.14 | `continue` statement terminating an iteration of a `for` statement. (Part 2 of 2.)

Figure 5.14 uses `continue` (line 10) to skip the statement at line 12 when the nested `if` determines that `count`'s value is 5. When the `continue` statement executes, program control continues with the increment of the control variable in the `for` statement (line 7).

In Section 5.3, we stated that `while` could be used in most cases in place of `for`. This is *not* true when the increment expression in the `while` follows a `continue` statement. In this case, the increment does *not* execute before the program evaluates the repetition-continuation condition, so the `while` does not execute in the same manner as the `for`.



Software Engineering Observation 5.2

Some programmers feel that `break` and `continue` violate structured programming. Since the same effects are achievable with structured programming techniques, these programmers do not use `break` or `continue`.



Software Engineering Observation 5.3

There's a tension between achieving quality software engineering and achieving the best-performing software. Sometimes one of these goals is achieved at the expense of the other. For all but the most performance-intensive situations, apply the following rule of thumb: First, make your code simple and correct; then make it fast and small, but only if necessary.

5.9 Logical Operators

The `if`, `if...else`, `while`, `do...while` and `for` statements each require a *condition* to determine how to continue a program's flow of control. So far, we've studied only simple conditions, such as `count <= 10`, `number != sentinelValue` and `total > 1000`. Simple conditions are expressed in terms of the relational operators `>`, `<`, `>=` and `<=` and the equality operators `==` and `!=`, and each expression tests only one condition. To test *multiple* conditions in the process of making a decision, we performed these tests in separate statements or in nested `if` or `if...else` statements. Sometimes control statements require more complex conditions to determine a program's flow of control.

Java's **logical operators** enable you to form more complex conditions by *combining* simple conditions. The logical operators are `&&` (conditional AND), `||` (conditional OR), `&` (boolean logical AND), `|` (boolean logical inclusive OR), `^` (boolean logical exclusive OR) and `!` (logical NOT). [Note: The `&`, `|` and `^` operators are also bitwise operators when they're applied to integral operands. We discuss the bitwise operators in online Appendix K.]

Conditional AND (&&) Operator

Suppose we wish to ensure at some point in a program that two conditions are *both* true before we choose a certain path of execution. In this case, we can use the **&&** (conditional AND) operator, as follows:

```
if (gender == FEMALE && age >= 65)
    ++seniorFemales;
```

This `if` statement contains two simple conditions. The condition `gender == FEMALE` compares variable `gender` to the constant `FEMALE` to determine whether a person is female. The condition `age >= 65` might be evaluated to determine whether a person is a senior citizen. The `if` statement considers the combined condition

```
gender == FEMALE && age >= 65
```

which is true if and only if *both* simple conditions are true. In this case, the `if` statement's body increments `seniorFemales` by 1. If either or both of the simple conditions are false, the program skips the increment. Some programmers find that the preceding combined condition is more readable when *redundant* parentheses are added, as in:

```
(gender == FEMALE) && (age >= 65)
```

The table in Fig. 5.15 summarizes the `&&` operator. The table shows all four possible combinations of false and true values for *expression1* and *expression2*. Such tables are called **truth tables**. Java evaluates to false or true all expressions that include relational operators, equality operators or logical operators.

expression1	expression2	expression1 && expression2
false	false	false
false	true	false
true	false	false
true	true	true

Fig. 5.15 | `&&` (conditional AND) operator truth table.

Conditional OR (`||`) Operator

Now suppose we wish to ensure that *either or both* of two conditions are true before we choose a certain path of execution. In this case, we use the `||` (**conditional OR**) operator, as in the following program segment:

```
if ((semesterAverage >= 90) || (finalExam >= 90))
    System.out.println ("Student grade is A");
```

This statement also contains two simple conditions. The condition `semesterAverage >= 90` evaluates to determine whether the student deserves an A in the course because of a solid performance throughout the semester. The condition `finalExam >= 90` evaluates to determine whether the student deserves an A in the course because of an outstanding performance on the final exam. The `if` statement then considers the combined condition

```
(semesterAverage >= 90) || (finalExam >= 90)
```

and awards the student an A if *either or both* of the simple conditions are true. The only time the message "Student grade is A" is *not* printed is when *both* of the simple conditions are *false*. Figure 5.16 is a truth table for operator conditional OR (`||`). Operator `&&` has a higher precedence than operator `||`. Both operators associate from left to right.

expression1	expression2	expression1 expression2
false	false	false
false	true	true
true	false	true
true	true	true

Fig. 5.16 | || (conditional OR) operator truth table.

Short-Circuit Evaluation of Complex Conditions

The parts of an expression containing && or || operators are evaluated *only* until it's known whether the condition is true or false. Thus, evaluation of the expression

```
(gender == FEMALE) && (age >= 65)
```

stops immediately if *gender* is *not* equal to FEMALE (i.e., the entire expression is *false*) and continues if *gender* is equal to FEMALE (i.e., the entire expression could still be *true* if the condition *age >= 65* is *true*). This feature of conditional AND and conditional OR expressions is called **short-circuit evaluation**.



Common Programming Error 5.8

In expressions using operator &&, a condition—we'll call this the dependent condition—may require another condition to be true for the evaluation of the dependent condition to be meaningful. In this case, the dependent condition should be placed after the && operator to prevent errors. Consider the expression (i != 0) && (10 / i == 2). The dependent condition (10 / i == 2) must appear after the && operator to prevent the possibility of division by zero.

Boolean Logical AND (&) and Boolean Logical Inclusive OR (|) Operators

The **boolean logical AND (&)** and **boolean logical inclusive OR (|)** operators are identical to the && and || operators, except that the & and | operators *always* evaluate *both* of their operands (i.e., they do *not* perform short-circuit evaluation). So, the expression

```
(gender == 1) & (age >= 65)
```

evaluates *age >= 65* *regardless* of whether *gender* is equal to 1. This is useful if the right operand has a required **side effect**—a modification of a variable's value. For example, the expression

```
(birthday == true) | (++age >= 65)
```

guarantees that the condition *++age >= 65* will be evaluated. Thus, the variable *age* is incremented, regardless of whether the overall expression is *true* or *false*.



Error-Prevention Tip 5.9

For clarity, avoid expressions with side effects (such as assignments) in conditions. They can make code harder to understand and can lead to subtle logic errors.



Error-Prevention Tip 5.10

Assignment (=) expressions generally should not be used in conditions. Every condition must result in a boolean value; otherwise, a compilation error occurs. In a condition, an assignment will compile only if a boolean expression is assigned to a boolean variable.

Boolean Logical Exclusive OR (^)

A simple condition containing the **boolean logical exclusive OR (^)** operator is true *if and only if one of its operands is true and the other is false*. If both are true or both are false, the entire condition is false. Figure 5.17 is a truth table for the boolean logical exclusive OR operator (^). This operator is guaranteed to evaluate *both* of its operands.

expression1	expression2	expression1 ^ expression2
false	false	false
false	true	true
true	false	true
true	true	false

Fig. 5.17 | ^ (boolean logical exclusive OR) operator truth table.

Logical Negation (!) Operator

The **!** (logical NOT, also called **logical negation** or **logical complement**) operator “reverses” the meaning of a condition. Unlike the logical operators &&, ||, &, | and ^, which are *binary* operators that combine two conditions, the logical negation operator is a *unary* operator that has only one condition as an operand. The operator is placed *before* a condition to choose a path of execution if the original condition (without the logical negation operator) is false, as in the program segment

```
if (! (grade == sentinelValue))
    System.out.printf("The next grade is %d\n", grade);
```

which executes the printf call only if grade is *not* equal to sentinelValue. The parentheses around the condition grade == sentinelValue are needed because the logical negation operator has a *higher* precedence than the equality operator.

In most cases, you can avoid using logical negation by expressing the condition differently with an appropriate relational or equality operator. For example, the previous statement may also be written as follows:

```
if (grade != sentinelValue)
    System.out.printf("The next grade is %d\n", grade);
```

This flexibility can help you express a condition in a more convenient manner. Figure 5.18 is a truth table for the logical negation operator.

expression	!expression
false	true
true	false

Fig. 5.18 | ! (logical NOT) operator truth table.

Logical Operators Example

Figure 5.19 uses logical operators to produce the truth tables discussed in this section. The output shows the boolean expression that was evaluated and its result. We used the

%b format specifier to display the word “true” or the word “false” based on a boolean expression’s value. Lines 9–13 produce the truth table for `&&`. Lines 16–20 produce the truth table for `||`. Lines 23–27 produce the truth table for `&`. Lines 30–35 produce the truth table for `|`. Lines 38–43 produce the truth table for `^`. Lines 46–47 produce the truth table for `!`.

```

1  // Fig. 5.19: LogicalOperators.java
2  // Logical operators.
3
4  public class LogicalOperators
5  {
6      public static void main(String[] args)
7      {
8          // create truth table for && (conditional AND) operator
9          System.out.printf("%s\n%s: %b\n%s: %b\n%s: %b\n%s: %b\n\n",
10              "Conditional AND (&&)", "false && false", (false && false) ,
11              "false && true", (false && true) ,
12              "true && false", (true && false) ,
13              "true && true", (true && true) );
14
15          // create truth table for || (conditional OR) operator
16          System.out.printf("%s\n%s: %b\n%s: %b\n%s: %b\n%s: %b\n\n",
17              "Conditional OR (||)", "false || false", (false || false) ,
18              "false || true", (false || true) ,
19              "true || false", (true || false) ,
20              "true || true", (true || true) );
21
22          // create truth table for & (boolean logical AND) operator
23          System.out.printf("%s\n%s: %b\n%s: %b\n%s: %b\n%s: %b\n\n",
24              "Boolean logical AND (&)", "false & false", (false & false) ,
25              "false & true", (false & true) ,
26              "true & false", (true & false) ,
27              "true & true", (true & true) );
28
29          // create truth table for | (boolean logical inclusive OR) operator
30          System.out.printf("%s\n%s: %b\n%s: %b\n%s: %b\n%s: %b\n\n",
31              "Boolean logical inclusive OR (|)",
32              "false | false", (false | false) ,
33              "false | true", (false | true) ,
34              "true | false", (true | false) ,
35              "true | true", (true | true) );
36
37          // create truth table for ^ (boolean logical exclusive OR) operator
38          System.out.printf("%s\n%s: %b\n%s: %b\n%s: %b\n%s: %b\n\n",
39              "Boolean logical exclusive OR (^)",
40              "false ^ false", (false ^ false) ,
41              "false ^ true", (false ^ true) ,
42              "true ^ false", (true ^ false) ,
43              "true ^ true", (true ^ true) );
44

```

Fig. 5.19 | Logical operators. (Part I of 2.)


```

45      // create truth table for ! (logical negation) operator
46      System.out.printf("%s\n%s: %b\n%s: %b\n", "Logical NOT (!)",
47          "false", (!false), "true", (!true));
48  }
49  } // end class LogicalOperators

```

```

Conditional AND (&&)
false && false: false
false && true: false
true && false: false
true && true: true

Conditional OR (||)
false || false: false
false || true: true
true || false: true
true || true: true

Boolean logical AND (&)
false & false: false
false & true: false
true & false: false
true & true: true

Boolean logical inclusive OR (|)
false | false: false
false | true: true
true | false: true
true | true: true

Boolean logical exclusive OR (^)
false ^ false: false
false ^ true: true
true ^ false: true
true ^ true: false

Logical NOT (!)
!false: true
!true: false

```

Fig. 5.19 | Logical operators. (Part 2 of 2.)

Precedence and Associativity of the Operators Presented So Far

Figure 5.20 shows the precedence and associativity of the Java operators introduced so far. The operators are shown from top to bottom in decreasing order of precedence.

Operators	Associativity	Type
++ --	right to left	unary postfix
++ -- + - ! (type)	right to left	unary prefix
* / %	left to right	multiplicative

Fig. 5.20 | Precedence/associativity of the operators discussed so far. (Part 1 of 2.)

Operators	Associativity	Type
+ -	left to right	additive
< <= > >=	left to right	relational
== !=	left to right	equality
&	left to right	boolean logical AND
^	left to right	boolean logical exclusive OR
	left to right	boolean logical inclusive OR
&&	left to right	conditional AND
	left to right	conditional OR
?:	right to left	conditional
= += -= *= /= %=	right to left	assignment

Fig. 5.20 | Precedence/associativity of the operators discussed so far. (Part 2 of 2.)

5.10 Structured Programming Summary

Just as architects design buildings by employing the collective wisdom of their profession, so should programmers design programs. Our field is much younger than architecture, and our collective wisdom is considerably sparser. We've learned that structured programming produces programs that are easier than unstructured programs to understand, test, debug, modify and even prove correct in a mathematical sense.

Java Control Statements Are Single-Entry/Single-Exit

Figure 5.21 uses UML activity diagrams to summarize Java's control statements. The initial and final states indicate the *single entry point* and the *single exit point* of each control statement. Arbitrarily connecting individual symbols in an activity diagram can lead to unstructured programs. Therefore, the programming profession has chosen a limited set of control statements that can be combined in only two simple ways to build structured programs.

For simplicity, Java includes only *single-entry/single-exit* control statements—there's only one way to enter and only one way to exit each control statement. Connecting control statements in sequence to form structured programs is simple. The final state of one control statement is connected to the initial state of the next—that is, the control statements are placed one after another in a program in sequence. We call this *control-statement stacking*. The rules for forming structured programs also allow for control statements to be *nested*.

Rules for Forming Structured Programs

Figure 5.22 shows the rules for forming structured programs. The rules assume that action states may be used to indicate *any* action. The rules also assume that we begin with the simplest activity diagram (Fig. 5.23) consisting of only an initial state, an action state, a final state and transition arrows.

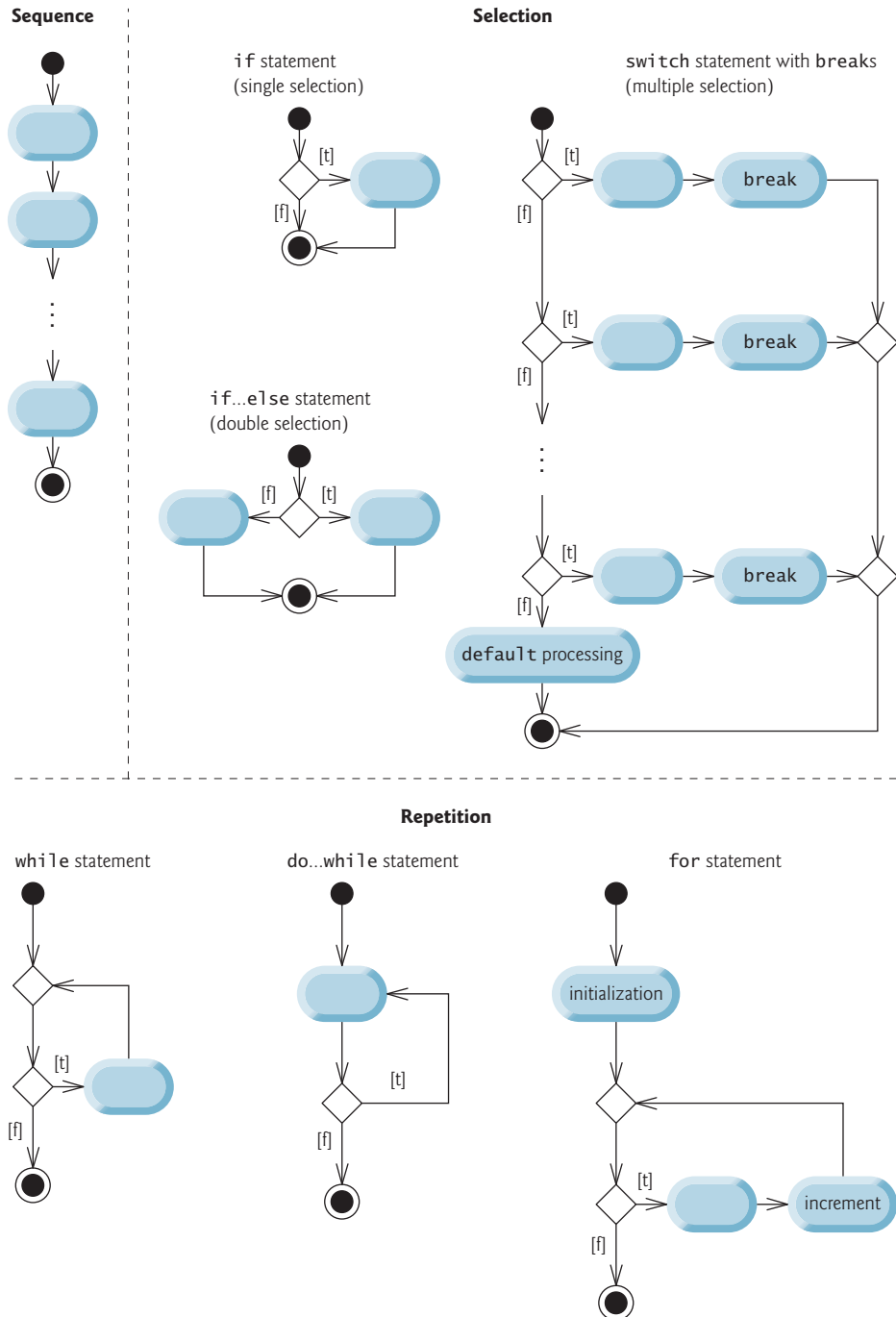
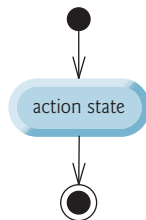


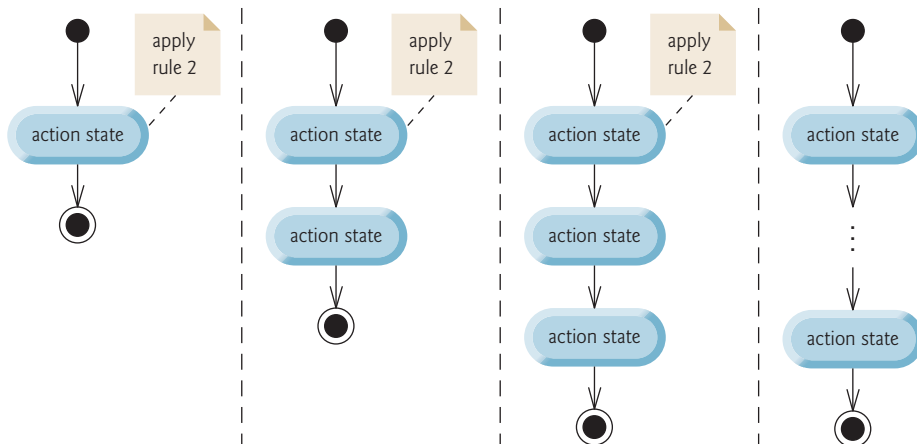
Fig. 5.21 | Java's single-entry/single-exit sequence, selection and repetition statements.

Rules for forming structured programs

1. Begin with the simplest activity diagram (Fig. 5.23).
2. Any action state can be replaced by two action states in sequence.
3. Any action state can be replaced by any control statement (sequence of action states, if, if...else, switch, while, do...while or for).
4. Rules 2 and 3 can be applied as often as you like and in any order.

Fig. 5.22 | Rules for forming structured programs.**Fig. 5.23** | Simplest activity diagram.

Applying the rules in Fig. 5.22 always results in a properly structured activity diagram with a neat, building-block appearance. For example, repeatedly applying rule 2 to the simplest activity diagram results in an activity diagram containing many action states in sequence (Fig. 5.24). Rule 2 generates a *stack* of control statements, so let's call rule 2 the **stacking rule**. The vertical dashed lines in Fig. 5.24 are not part of the UML—we use them to separate the four activity diagrams that demonstrate rule 2 of Fig. 5.22 being applied.

**Fig. 5.24** | Repeatedly applying rule 2 of Fig. 5.22 to the simplest activity diagram.

Rule 3 is called the **nesting rule**. Repeatedly applying rule 3 to the simplest activity diagram results in one with neatly *nested* control statements. For example, in Fig. 5.25, the action state in the simplest activity diagram is replaced with a double-selection (if...else) statement. Then rule 3 is applied again to the action states in the double-selection statement, replacing each with a double-selection statement. The dashed action-state symbol around each double-selection statement represents the action state that was replaced. [Note: The dashed arrows and dashed action-state symbols shown in Fig. 5.25 are not part of the UML. They're used here to illustrate that *any* action state can be replaced with a control statement.]

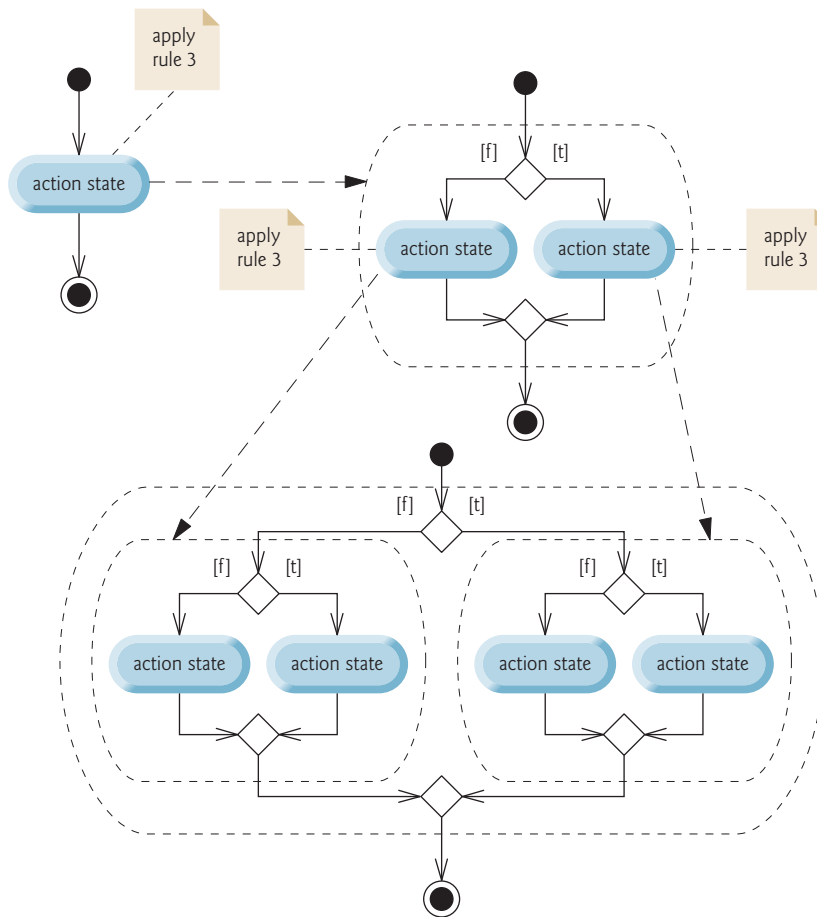


Fig. 5.25 | Repeatedly applying rule 3 of Fig. 5.22 to the simplest activity diagram.

Rule 4 generates larger, more involved and more deeply nested statements. The diagrams that emerge from applying the rules in Fig. 5.22 constitute the set of all possible structured activity diagrams and hence the set of all possible structured programs. The beauty of the structured approach is that we use *only seven* simple single-entry/single-exit control statements and assemble them in *only two* simple ways.

If the rules in Fig. 5.22 are followed, an “unstructured” activity diagram (like the one in Fig. 5.26) cannot be created. If you’re uncertain about whether a particular diagram is structured, apply the rules of Fig. 5.22 in reverse to reduce it to the simplest activity diagram. If you can reduce it, the original diagram is structured; otherwise, it’s not.

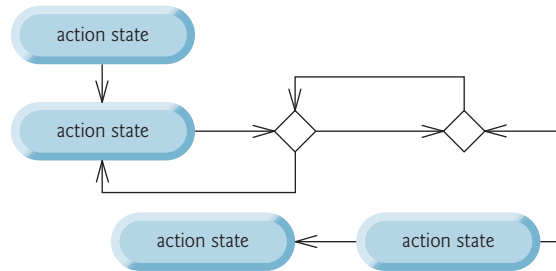


Fig. 5.26 | “Unstructured” activity diagram.

Three Forms of Control

Structured programming promotes simplicity. Only three forms of control are needed to implement an algorithm:

- sequence
- selection
- repetition

The sequence structure is trivial. Simply list the statements to execute in the order in which they should execute. Selection is implemented in one of three ways:

- `if` statement (single selection)
- `if...else` statement (double selection)
- `switch` statement (multiple selection)

In fact, it’s straightforward to prove that the simple `if` statement is sufficient to provide *any* form of selection—everything that can be done with the `if...else` statement and the `switch` statement can be implemented by combining `if` statements (although perhaps not as clearly and efficiently).

Repetition is implemented in one of three ways:

- `while` statement
- `do...while` statement
- `for` statement

[*Note:* There’s a fourth repetition statement—the *enhanced for statement*—that we discuss in Section 7.7.] It’s straightforward to prove that the `while` statement is sufficient to provide *any* form of repetition. Everything that can be done with `do...while` and `for` can be done with the `while` statement (although perhaps not as conveniently).

Combining these results illustrates that *any* form of control ever needed in a Java program can be expressed in terms of

- sequence
- if statement (selection)
- while statement (repetition)

and that these can be combined in only two ways—*stacking* and *nesting*. Indeed, structured programming is the essence of simplicity.

5.11 (Optional) GUI and Graphics Case Study: Drawing Rectangles and Ovals

This section demonstrates drawing rectangles and ovals, using the Graphics methods **drawRect** and **drawOval**, respectively. These methods are demonstrated in Fig. 5.27.

```
1 // Fig. 5.27: Shapes.java
2 // Drawing a cascade of shapes based on the user's choice.
3 import java.awt.Graphics;
4 import javax.swing.JPanel;
5
6 public class Shapes extends JPanel
7 {
8     private int choice; // user's choice of which shape to draw
9
10    // constructor sets the user's choice
11    public Shapes(int userChoice)
12    {
13        choice = userChoice;
14    }
15
16    // draws a cascade of shapes starting from the top-left corner
17    public void paintComponent(Graphics g)
18    {
19        super.paintComponent(g);
20
21        for (int i = 0; i < 10; i++)
22        {
23            // pick the shape based on the user's choice
24            switch (choice)
25            {
26                case 1: // draw rectangles
27                    g.drawRect(10 + i * 10, 10 + i * 10,
28                               50 + i * 10, 50 + i * 10);
29                    break;
30                case 2: // draw ovals
31                    g.drawOval(10 + i * 10, 10 + i * 10,
32                              50 + i * 10, 50 + i * 10);
33                    break;
34            }
35        }
36    }
37 } // end class Shapes
```

Fig. 5.27 | Drawing a cascade of shapes based on the user's choice.

Line 6 begins the class declaration for `Shapes`, which extends `JPanel`. Instance variable `choice` determines whether `paintComponent` should draw rectangles or ovals. The `Shapes` constructor initializes `choice` with the value passed in parameter `userChoice`.

Method `paintComponent` performs the actual drawing. Remember, the first statement in every `paintComponent` method should be a call to `super.paintComponent`, as in line 19. Lines 21–35 loop 10 times to draw 10 shapes. The *nested* `switch` statement (lines 24–34) chooses between drawing rectangles and drawing ovals.

If `choice` is 1, then the program draws rectangles. Lines 27–28 call `Graphics` method `drawRect`. Method `drawRect` requires four arguments. The first two represent the *x*- and *y*-coordinates of the upper-left corner of the rectangle; the next two represent the rectangle's width and height. In this example, we start at a position 10 pixels down and 10 pixels right of the top-left corner, and every iteration of the loop moves the upper-left corner another 10 pixels down and to the right. The width and the height of the rectangle start at 50 pixels and increase by 10 pixels in each iteration.

If `choice` is 2, the program draws ovals. It creates an imaginary rectangle called a **bounding rectangle** and places inside it an oval that touches the midpoints of all four sides. Method `drawOval` (lines 31–32) requires the same four arguments as method `drawRect`. The arguments specify the position and size of the bounding rectangle for the oval. The values passed to `drawOval` in this example are exactly the same as those passed to `drawRect` in lines 27–28. Since the width and height of the bounding rectangle are identical in this example, lines 27–28 draw a *circle*. As an exercise, try modifying the program to draw both rectangles and ovals to see how `drawOval` and `drawRect` are related.

Class *ShapesTest*

Figure 5.28 is responsible for handling input from the user and creating a window to display the appropriate drawing based on the user's response. Line 3 imports `JFrame` to handle the display, and line 4 imports `JOptionPane` to handle the input. Lines 11–13 prompt the user with an input dialog and store the user's response in variable `input`. Notice that when displaying multiple lines of prompt text in a `JOptionPane`, you must use `\n` to begin a new line of text, rather than `%n`. Line 15 uses `Integer` method `parseInt` to convert the `String` entered by the user to an `int` and stores the result in variable `choice`. Line 18 creates a `Shapes` object and passes the user's choice to the constructor. Lines 20–25 perform the standard operations that create and set up a window in this case study—create a *frame*, set it to exit the application when closed, add the drawing to the frame, set the frame size and make it visible.

```

1  // Fig. 5.28: ShapesTest.java
2  // Obtaining user input and creating a JFrame to display Shapes.
3  import javax.swing.JFrame; //handle the display
4  import javax.swing.JOptionPane;
5
6  public class ShapesTest
7  {
8      public static void main(String[] args)
9      {

```

Fig. 5.28 | Obtaining user input and creating a `JFrame` to display `Shapes`. (Part 1 of 2.)


```

10 // obtain user's choice
11 String input = JOptionPane.showInputDialog(
12     "Enter 1 to draw rectangles\n" +
13     "Enter 2 to draw ovals");
14
15 int choice = Integer.parseInt(input); // convert input to int
16
17 // create the panel with the user's input
18 Shapes panel = new Shapes(choice);
19
20 JFrame application = new JFrame(); // creates a new JFrame
21
22 application.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
23 application.add(panel);
24 application.setSize(300, 300);
25 application.setVisible(true);
26 }
27 } // end class ShapesTest

```

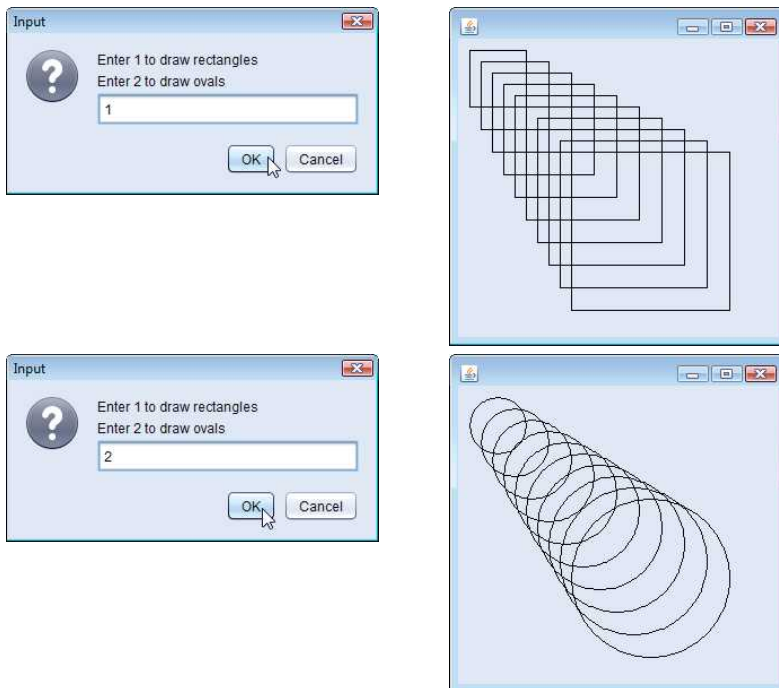


Fig. 5.28 | Obtaining user input and creating a JFrame to display Shapes. (Part 2 of 2.)

GUI and Graphics Case Study Exercises

5.1 Draw 12 concentric circles in the center of a JPanel (Fig. 5.29). The innermost circle should have a radius of 10 pixels, and each successive circle should have a radius 10 pixels larger than the previous one. Begin by finding the center of the JPanel. To get the upper-left corner of a circle, move up one radius and to the left one radius from the center. The width and height of the bounding rectangle are both the same as the circle's diameter (i.e., twice the radius).

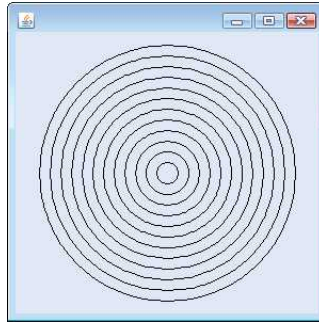


Fig. 5.29 | Drawing concentric circles.

5.2 Modify Exercise 5.16 from the end-of-chapter exercises to read input using dialogs and to display the bar chart using rectangles of varying lengths.

5.12 Wrap-Up

In this chapter, we completed our introduction to control statements, which enable you to control the flow of execution in methods. Chapter 4 discussed `if`, `if...else` and `while`. This chapter demonstrated `for`, `do...while` and `switch`. We showed that any algorithm can be developed using combinations of the sequence structure, the three types of selection statements—`if`, `if...else` and `switch`—and the three types of repetition statements—`while`, `do...while` and `for`. In this chapter and Chapter 4, we discussed how you can combine these building blocks to utilize proven program-construction and problem-solving techniques. You used the `break` statement to exit a `switch` statement and to immediately terminate a loop, and used a `continue` statement to terminate a loop's current iteration and proceed with the loop's next iteration. This chapter also introduced Java's logical operators, which enable you to use more complex conditional expressions in control statements. In Chapter 6, we examine methods in greater depth.

Summary

Section 5.2 Essentials of Counter-Controlled Repetition

- Counter-controlled repetition (p. 153) requires a control variable, the initial value of the control variable, the increment by which the control variable is modified each time through the loop (also known as each iteration of the loop) and the loop-continuation condition that determines whether looping should continue.
- You can declare a variable and initialize it in the same statement.

Section 5.3 for Repetition Statement

- The `while` statement can be used to implement any counter-controlled loop.
- The `for` statement (p. 155) specifies all the details of counter-controlled repetition in its header
- When the `for` statement begins executing, its control variable is declared and initialized. If the loop-continuation condition is initially true, the body executes. After executing the loop's body, the increment expression executes. Then the loop-continuation test is performed again to determine whether the program should continue with the next iteration of the loop.

- The general format of the `for` statement is

```
for (initialization; loopContinuationCondition; increment)
    statement
```

where the *initialization* expression names the loop's control variable and provides its initial value, *loopContinuationCondition* determines whether the loop should continue executing and *increment* modifies the control variable's value, so that the loop-continuation condition eventually becomes false. The two semicolons in the `for` header are required.

- Most `for` statements can be represented with equivalent `while` statements as follows:

```
initialization;
while (loopContinuationCondition)
{
    statement
    increment;
}
```

- Typically, `for` statements are used for counter-controlled repetition and `while` statements for sentinel-controlled repetition.
- If the *initialization* expression in the `for` header declares the control variable, the control variable can be used only in that `for` statement—it will not exist outside the `for` statement.
- The expressions in a `for` header are optional. If the *loopContinuationCondition* is omitted, Java assumes that it's always true, thus creating an infinite loop. You might omit the *initialization* expression if the control variable is initialized before the loop. You might omit the *increment* expression if the increment is calculated with statements in the loop's body or if no increment is needed.
- The increment expression in a `for` acts as if it's a standalone statement at the end of the `for`'s body.
- A `for` statement can count downward by using a negative increment—i.e., a decrement (p. 158).
- If the loop-continuation condition is initially `false`, the `for` statement's body does not execute.

Section 5.4 Examples Using the `for` Statement

- Java treats floating-point constants like `1000.0` and `0.05` as type `double`. Similarly, Java treats whole-number constants like `7` and `-22` as type `int`.
- The format specifier `%4s` outputs a `String` in a field width (p. 161) of 4—that is, `printf` displays the value with at least 4 character positions. If the value to be output is less than 4 character positions wide, the value is right justified (p. 161) in the field by default. If the value is greater than 4 character positions wide, the field width expands to accommodate the appropriate number of characters. To left justify (p. 161) the value, use a negative integer to specify the field width.
- `Math.pow(x, y)` (p. 162) calculates the value of x raised to the y^{th} power. The method receives two `double` arguments and returns a `double` value.
- The comma (,) formatting flag (p. 162) in a format specifier indicates that a floating-point value should be output with a grouping separator (p. 162). The actual separator used is specific to the user's locale (i.e., country). In the United States, the number will have commas separating every three digits and a decimal point separating the fractional part of the number, as in `1,234.45`.
- The `.` in a format specifier indicates that the integer to its right is the number's precision.

Section 5.5 `do...while` Repetition Statement

- The `do...while` statement (p. 163) is similar to the `while` statement. In the `while`, the program tests the loop-continuation condition at the beginning of the loop, before executing its body; if the condition is false, the body never executes. The `do...while` statement tests the loop-continuation condition *after* executing the loop's body; therefore, the body always executes at least once.

Section 5.6 *switch Multiple-Selection Statement*

- The `switch` statement (p. 165) performs different actions based on the possible values of a constant integral expression (a constant value of type `byte`, `short`, `int` or `char`, but not `long`), or a `String`.
- The end-of-file indicator is a system-dependent keystroke combination that terminates user input. On UNIX/Linux/Mac OS X systems, end-of-file is entered by typing the sequence `<Ctrl>d` on a line by itself. This notation means to simultaneously press both the `Ctrl` key and the `d` key. On Windows systems, enter end-of-file by typing `<Ctrl>z`.
- Scanner method `hasNext` (p. 168) determines whether there's more data to input. This method returns the `boolean` value `true` if there's more data; otherwise, it returns `false`. As long as the end-of-file indicator has not been typed, method `hasNext` will return `true`.
- The `switch` statement consists of a block that contains a sequence of case labels (p. 168) and an optional default case (p. 168).
- In a `switch`, the program evaluates the controlling expression and compares its value with each case label. If a match occurs, the program executes the statements for that case.
- Listing cases consecutively with no statements between them enables the cases to perform the same set of statements.
- Every value you wish to test in a `switch` must be listed in a separate case label.
- Each case can have multiple statements, and these need not be placed in braces.
- A case's statements typically end with a `break` statement (p. 168) that terminates the `switch`'s execution.
- Without `break` statements, each time a match occurs in the `switch`, the statements for that case and subsequent cases execute until a `break` statement or the end of the `switch` is encountered.
- If no match occurs between the controlling expression's value and a case label, the optional default case executes. If no match occurs and the `switch` does not contain a default case, program control simply continues with the first statement after the `switch`.

Section 5.7 *Class AutoPolicy Case Study: Strings in switch Statements*

- `Strings` can be used in a `switch` statement's controlling expression and case labels.

Section 5.8 *break and continue Statements*

- The `break` statement, when executed in a `while`, `for`, `do...while` or `switch`, causes immediate exit from that statement.
- The `continue` statement (p. 174), when executed in a `while`, `for` or `do...while`, skips the loop's remaining body statements and proceeds with its next iteration. In `while` and `do...while` statements, the program evaluates the loop-continuation test immediately. In a `for` statement, the increment expression executes, then the program evaluates the loop-continuation test.

Section 5.9 *Logical Operators*

- Simple conditions are expressed in terms of the relational operators `>`, `<`, `>=` and `<=` and the equality operators `==` and `!=`, and each expression tests only one condition.
- Logical operators (p. 176) enable you to form more complex conditions by combining simple conditions. The logical operators are `&&` (conditional AND), `||` (conditional OR), `&` (boolean logical AND), `|` (boolean logical inclusive OR), `^` (boolean logical exclusive OR) and `!` (logical NOT).
- To ensure that two conditions are *both* true, use the `&&` (conditional AND) operator. If either or both of the simple conditions are false, the entire expression is false.
- To ensure that either *or* both of two conditions are true, use the `||` (conditional OR) operator, which evaluates to true if either or both of its simple conditions are true.

- A condition using `&&` or `||` operators (p. 176) uses short-circuit evaluation (p. 178)—they’re evaluated only until it’s known whether the condition is true or false.
- The `&` and `|` operators (p. 178) work identically to the `&&` and `||` operators but always evaluate both operands.
- A simple condition containing the boolean logical exclusive OR (`^`; p. 179) operator is true *if and only if one of its operands is true and the other is false*. If both operands are true or both are false, the entire condition is false. This operator is also guaranteed to evaluate both of its operands.
- The unary `!` (logical NOT; p. 179) operator “reverses” the value of a condition.

Self-Review Exercises

5.1 Fill in the blanks in each of the following statements:

- Typically, _____ statements are used for counter-controlled repetition and _____ statements for sentinel-controlled repetition.
- The `do...while` statement tests the loop-continuation condition _____ executing the loop’s body; therefore, the body always executes at least once.
- The _____ statement selects among multiple actions based on the possible values of an integer variable or expression, or a `String`.
- The _____ statement, when executed in a repetition statement, skips the remaining statements in the loop body and proceeds with the next iteration of the loop.
- The _____ operator can be used to ensure that two conditions are *both* true before choosing a certain path of execution.
- If the loop-continuation condition in a `for` header is initially _____, the program does not execute the `for` statement’s body.
- Methods that perform common tasks and do not require objects are called _____ methods.

5.2 State whether each of the following is *true* or *false*. If *false*, explain why.

- The `default` case is required in the `switch` selection statement.
- The `break` statement is required in the last case of a `switch` selection statement.
- The expression `((x > y) && (a < b))` is true if either `x > y` is true or `a < b` is true.
- An expression containing the `||` operator is true if either or both of its operands are true.
- The comma `,` formatting flag in a format specifier (e.g., `%,20.2f`) indicates that a value should be output with a thousands separator.
- To test for a range of values in a `switch` statement, use a hyphen `-` between the start and end values of the range in a case label.
- Listing cases consecutively with no statements between them enables the cases to perform the same set of statements.

5.3 Write a Java statement or a set of Java statements to accomplish each of the following tasks:

- Sum the odd integers between 1 and 99, using a `for` statement. Assume that the integer variables `sum` and `count` have been declared.
- Calculate the value of 2.5 raised to the power of 3, using the `pow` method.
- Print the integers from 1 to 20, using a `while` loop and the counter variable `i`. Assume that the variable `i` has been declared, but not initialized. Print only five integers per line. [*Hint: Use the calculation `i % 5`. When the value of this expression is 0, print a newline character; otherwise, print a tab character. Assume that this code is an application. Use the `System.out.println()` method to output the newline character, and use the `System.out.print('\t')` method to output the tab character.*]
- Repeat part (c), using a `for` statement.

5.4 Find the error in each of the following code segments, and explain how to correct it:

- a) `i = 1;`
`while (i <= 10);`
`++i;`
`}`
- b) `for (k = 0.1; k != 1.0; k += 0.1)`
`System.out.println(k);`
- c) `switch (n)`
`{`
`case 1:`
`System.out.println("The number is 1");`
`case 2:`
`System.out.println("The number is 2");`
`break;`
`default:`
`System.out.println("The number is not 1 or 2");`
`break;`
`}`
- d) The following code should print the values 1 to 10:
`n = 1;`
`while (n < 10)`
`System.out.println(n++);`

Answers to Self-Review Exercises

5.1 a) for, while. b) after. c) switch. d) continue. e) && (conditional AND). f) false. g) static.

5.2 a) False. The default case is optional. If no default action is needed, then there's no need for a default case. b) False. The break statement is used to exit the switch statement. The break statement is not required for the last case in a switch statement. c) False. *Both* of the relational expressions must be true for the entire expression to be true when using the && operator. d) True. e) True. f) False. The switch statement does not provide a mechanism for testing ranges of values, so every value that must be tested should be listed in a separate case label. g) True.

- 5.3** a) `sum = 0;`
`for (count = 1; count <= 99; count += 2)`
`sum += count;`
- b) `double result = Math.pow(2.5, 3);`
- c) `i = 1;`
`while (i <= 20)`
`{`
`System.out.print(i);`
`if (i % 5 == 0)`
`System.out.println();`
`else`
`System.out.print('\t');`
`++i;`
`}`

```
d) for (i = 1; i <= 20; i++)
{
    System.out.print(i);

    if (i % 5 == 0)
        System.out.println();
    else
        System.out.print('\t');
}
```

- 5.4** a) Error: The semicolon after the `while` header causes an infinite loop, and there's a missing left brace.
Correction: Replace the semicolon by a `{`, or remove both the `;` and the `}`.
- b) Error: Using a floating-point number to control a `for` statement may not work, because floating-point numbers are represented only approximately by most computers.
Correction: Use an integer, and perform the proper calculation in order to get the values you desire:

```
for (k = 1; k != 10; k++)
    System.out.println((double) k / 10);
```

- c) Error: The missing code is the `break` statement in the statements for the first case.
Correction: Add a `break` statement at the end of the statements for the first case. This omission is not necessarily an error if you want the statement of case 2: to execute every time the case 1: statement executes.
- d) Error: An improper relational operator is used in the `while`'s continuation condition.
Correction: Use `<=` rather than `<`, or change 10 to 11.

Exercises

- 5.5** Describe the four basic elements of counter-controlled repetition.
- 5.6** Compare and contrast the `while` and `for` repetition statements.
- 5.7** Discuss a situation in which it would be more appropriate to use a `do...while` statement than a `while` statement. Explain why.
- 5.8** Compare and contrast the `break` and `continue` statements.
- 5.9** Find and correct the error(s) in each of the following segments of code:
- a)

```
For (i = 100, i >= 1, i++)
    System.out.println(i);
```
- b) The following code should print whether integer value is odd or even:
- ```
switch (value % 2)
{
 case 0:
 System.out.println("Even integer");
 case 1:
 System.out.println("Odd integer");
}
```
- c) The following code should output the odd integers from 19 to 1:
- ```
for (i = 19; i >= 1; i += 2)
    System.out.println(i);
```

d) The following code should output the even integers from 2 to 100:

```
counter = 2;

do
{
    System.out.println(counter);
    counter += 2;
} While (counter < 100);
```

5.10 What does the following program do?

```
1 // Exercise 5.10: Printing.java
2 public class Printing
3 {
4     public static void main(String[] args)
5     {
6         for (int i = 1; i <= 10; i++)
7         {
8             for (int j = 1; j <= 5; j++)
9                 System.out.print('@');
10
11             System.out.println();
12         }
13     }
14 } // end class Printing
```

5.11 (*Find the Smallest Value*) Write an application that finds the smallest of several integers. Assume that the first value read specifies the number of values to input from the user.

5.12 (*Calculating the Product of Odd Integers*) Write an application that calculates the product of the odd integers from 1 to 15.

5.13 (*Factorials*) Factorials are used frequently in probability problems. The factorial of a positive integer n (written $n!$ and pronounced “ n factorial”) is equal to the product of the positive integers from 1 to n . Write an application that calculates the factorials of 1 through 20. Use type long. Display the results in tabular format. What difficulty might prevent you from calculating the factorial of 100?

5.14 (*Modified Compound-Interest Program*) Modify the compound-interest application of Fig. 5.6 to repeat its steps for interest rates of 5%, 6%, 7%, 8%, 9% and 10%. Use a for loop to vary the interest rate.

5.15 (*Triangle Printing Program*) Write an application that displays the following patterns separately, one below the other. Use for loops to generate the patterns. All asterisks (*) should be printed by a single statement of the form `System.out.print('*')`; which causes the asterisks to print side by side. A statement of the form `System.out.println()`; can be used to move to the next line. A statement of the form `System.out.print(' ')`; can be used to display a space for the last two patterns. There should be no other output statements in the program. [Hint: The last two patterns require that each line begin with an appropriate number of blank spaces.]

(a)	(b)	(c)	(d)
*	*****	*****	*
**	*****	*****	**
***	*****	*****	***
****	*****	*****	****
*****	*****	*****	*****
*****	*****	*****	*****
*****	****	*****	*****
*****	***	****	*****
*****	**	**	*****
*****	*	*	*****

5.16 (Bar Chart Printing Program) One interesting application of computers is to display graphs and bar charts. Write an application that reads five numbers between 1 and 30. For each number that's read, your program should display the same number of adjacent asterisks. For example, if your program reads the number 7, it should display `*****`. Display the bars of asterisks *after* you read all five numbers.

5.17 (Calculating Sales) An online retailer sells five products whose retail prices are as follows: Product 1, \$2.98; product 2, \$4.50; product 3, \$9.98; product 4, \$4.49 and product 5, \$6.87. Write an application that reads a series of pairs of numbers as follows:

- a) product number
- b) quantity sold

Your program should use a switch statement to determine the retail price for each product. It should calculate and display the total retail value of all products sold. Use a sentinel-controlled loop to determine when the program should stop looping and display the final results.

5.18 (Modified Compound-Interest Program) Modify the application in Fig. 5.6 to use only integers to calculate the compound interest. [Hint: Treat all monetary amounts as integral numbers of pennies. Then break the result into its dollars and cents portions by using the division and remainder operations, respectively. Insert a period between the dollars and the cents portions.]

5.19 Assume that `i = 1`, `j = 2`, `k = 3` and `m = 2`. What does each of the following statements print?

- a) `System.out.println(i == 1);`
- b) `System.out.println(j == 3);`
- c) `System.out.println((i >= 1) && (j < 4));`
- d) `System.out.println((m <= 99) & (k < m));`
- e) `System.out.println((j >= i) || (k == m));`
- f) `System.out.println((k + m < j) | (3 - j >= k));`
- g) `System.out.println(!(k > m));`

5.20 (Calculating the Value of π) Calculate the value of π from the infinite series

$$\pi = 4 - \frac{4}{3} + \frac{4}{5} - \frac{4}{7} + \frac{4}{9} - \frac{4}{11} + \cdots$$

Print a table that shows the value of π approximated by computing the first 200,000 terms of this series. How many terms do you have to use before you first get a value that begins with 3.14159?

5.21 (Pythagorean Triples) A right triangle can have sides whose lengths are all integers. The set of three integer values for the lengths of the sides of a right triangle is called a Pythagorean triple. The lengths of the three sides must satisfy the relationship that the sum of the squares of two of the sides is equal to the square of the hypotenuse. Write an application that displays a table of the Pythagorean triples for `side1`, `side2` and the hypotenuse, all no larger than 500. Use a triple-nested for loop that tries all possibilities. This method is an example of “brute-force” computing. You’ll learn in more advanced computer science courses that for many interesting problems there’s no known algorithmic approach other than using sheer brute force.

5.22 (Modified Triangle Printing Program) Modify Exercise 5.15 to combine your code from the four separate triangles of asterisks such that all four patterns print side by side. [Hint: Make clever use of nested for loops.]

5.23 (De Morgan’s Laws) In this chapter, we discussed the logical operators `&&`, `&`, `||`, `|`, `^` and `!`. De Morgan’s laws can sometimes make it more convenient for us to express a logical expression. These laws state that the expression `!(condition1 && condition2)` is logically equivalent to the expression `!(condition1) || !(condition2)`. Also, the expression `!(condition1 || condition2)` is logically equivalent to the expression `!(condition1) && !(condition2)`. Use De Morgan’s laws to write equivalent

expressions for each of the following, then write an application to show that both the original expression and the new expression in each case produce the same value:

- a) `!(x < 5) && !(y >= 7)`
- b) `!(a == b) || !(g != 5)`
- c) `!((x <= 8) && (y > 4))`
- d) `!((i > 4) || (j <= 6))`

5.24 (*Diamond Printing Program*) Write an application that prints the following diamond shape. You may use output statements that print a single asterisk (*), a single space or a single new-line character. Maximize your use of repetition (with nested for statements), and minimize the number of output statements.

```

    *
   **
  ***
 ****
*****
*****
*****
 *****
  *****
   *****
    *

```

5.25 (*Modified Diamond Printing Program*) Modify the application you wrote in Exercise 5.24 to read an odd number in the range 1 to 19 to specify the number of rows in the diamond. Your program should then display a diamond of the appropriate size.

5.26 A criticism of the `break` statement and the `continue` statement is that each is unstructured. Actually, these statements can always be replaced by structured statements, although doing so can be awkward. Describe in general how you'd remove any `break` statement from a loop in a program and replace it with some structured equivalent. [*Hint:* The `break` statement exits a loop from the body of the loop. The other way to exit is by failing the loop-continuation test. Consider using in the loop-continuation test a second test that indicates “early exit because of a ‘break’ condition.”] Use the technique you develop here to remove the `break` statement from the application in Fig. 5.13.

5.27 What does the following program segment do?

```

for (i = 1; i <= 5; i++)
{
    for (j = 1; j <= 3; j++)
    {
        for (k = 1; k <= 4; k++)
            System.out.print('*');

        System.out.println();
    } // end inner for

    System.out.println();
} // end outer for

```

5.28 Describe in general how you'd remove any `continue` statement from a loop in a program and replace it with some structured equivalent. Use the technique you develop here to remove the `continue` statement from the program in Fig. 5.14.

5.29 (*“The Twelve Days of Christmas” Song*) Write an application that uses repetition and `switch` statements to print the song “The Twelve Days of Christmas.” One `switch` statement should be used to print the day (“first,” “second,” and so on). A separate `switch` statement should be used

to print the remainder of each verse. Visit the website [en.wikipedia.org/wiki/The_Twelve_Days_of_Christmas_\(song\)](http://en.wikipedia.org/wiki/The_Twelve_Days_of_Christmas_(song)) for the lyrics of the song.

5.30 (*Modified AutoPolicy Class*) Modify class `AutoPolicy` in Fig. 5.11 to validate the two-letter state codes for the northeast states. The codes are: CT for Connecticut, MA for Massachusetts, ME for Maine, NH for New Hampshire, NJ for New Jersey, NY for New York, PA for Pennsylvania and VT for Vermont. In `AutoPolicy` method `setState`, use the logical OR (`||`) operator (Section 5.9) to create a compound condition in an `if...else` statement that compares the method's argument with each two-letter code. If the code is incorrect, the `else` part of the `if...else` statement should display an error message. In later chapters, you'll learn how to use exception handling to indicate that a method received an invalid value.

Making a Difference

5.31 (*Global Warming Facts Quiz*) The controversial issue of global warming has been widely publicized by the film “An Inconvenient Truth,” featuring former Vice President Al Gore. Mr. Gore and a U.N. network of scientists, the Intergovernmental Panel on Climate Change, shared the 2007 Nobel Peace Prize in recognition of “their efforts to build up and disseminate greater knowledge about man-made climate change.” Research *both* sides of the global warming issue online (you might want to search for phrases like “global warming skeptics”). Create a five-question multiple-choice quiz on global warming, each question having four possible answers (numbered 1–4). Be objective and try to fairly represent both sides of the issue. Next, write an application that administers the quiz, calculates the number of correct answers (zero through five) and returns a message to the user. If the user correctly answers five questions, print “Excellent”; if four, print “Very good”; if three or fewer, print “Time to brush up on your knowledge of global warming,” and include a list of some of the websites where you found your facts.

5.32 (*Tax Plan Alternatives; The “FairTax”*) There are many proposals to make taxation fairer. Check out the FairTax initiative in the United States at www.fairtax.org. Research how the proposed FairTax works. One suggestion is to eliminate income taxes and most other taxes in favor of a 23% consumption tax on all products and services that you buy. Some FairTax opponents question the 23% figure and say that because of the way the tax is calculated, it would be more accurate to say the rate is 30%—check this carefully. Write a program that prompts the user to enter expenses in various expense categories they have (e.g., housing, food, clothing, transportation, education, health care, vacations), then prints the estimated FairTax that person would pay.

5.33 (*Facebook User Base Growth*) According to CNNMoney.com, Facebook hit one billion users in October 2012. Using the compound-growth technique you learned in Fig. 5.6 and assuming its user base grows at a rate of 4% per month, how many months will it take for Facebook to grow its user base to 1.5 billion users? How many months will it take for Facebook to grow its user base to two billion users?

6

Methods: A Deeper Look

Form ever follows function.

—Louis Henri Sullivan

E pluribus unum.
(*One composed of many.*)

—Virgil

*O! call back yesterday, bid time
return.*

—William Shakespeare

Answer me in one word.

—William Shakespeare

*There is a point at which
methods devour themselves.*

—Frantz Fanon

Objectives

In this chapter you'll learn:

- How `static` methods and fields are associated with classes rather than objects.
- How the method-call/return mechanism is supported by the method-call stack.
- About argument promotion and casting.
- How packages group related classes.
- How to use secure random-number generation to implement game-playing applications.
- How the visibility of declarations is limited to specific regions of programs.
- What method overloading is and how to create overloaded methods.



- | | |
|--|---|
| 6.1 Introduction | 6.8 Java API Packages |
| 6.2 Program Modules in Java | 6.9 Case Study: Secure Random-Number Generation |
| 6.3 <code>static</code> Methods, <code>static</code> Fields and Class <code>Math</code> | 6.10 Case Study: A Game of Chance; Introducing <code>enum</code> Types |
| 6.4 Declaring Methods with Multiple Parameters | 6.11 Scope of Declarations |
| 6.5 Notes on Declaring and Using Methods | 6.12 Method Overloading |
| 6.6 Method-Call Stack and Stack Frames | 6.13 (Optional) GUI and Graphics Case Study: Colors and Filled Shapes |
| 6.7 Argument Promotion and Casting | 6.14 Wrap-Up |

Summary | Self-Review Exercises | Answers to Self-Review Exercises | Exercises | Making a Difference

6.1 Introduction

Experience has shown that the best way to develop and maintain a large program is to construct it from small, simple pieces, or **modules**. This technique is called **divide and conquer**. Methods, which we introduced in Chapter 3, help you modularize programs. In this chapter, we study methods in more depth.

You'll learn more about `static` methods, which can be called without the need for an object of the class to exist. You'll also learn how Java is able to keep track of which method is currently executing, how local variables of methods are maintained in memory and how a method knows where to return after it completes execution.

We'll take a brief diversion into simulation techniques with random-number generation and develop a version of the dice game called craps that uses most of the programming techniques you've used to this point in the book. In addition, you'll learn how to declare constants in your programs.

Many of the classes you'll use or create while developing applications will have more than one method of the same name. This technique, called *overloading*, is used to implement methods that perform similar tasks for arguments of different types or for different numbers of arguments.

We continue our discussion of methods in Chapter 18, Recursion. Recursion provides an intriguing way of thinking about methods and algorithms.

6.2 Program Modules in Java

You write Java programs by combining new methods and classes with predefined ones available in the **Java Application Programming Interface** (also referred to as the **Java API** or **Java class library**) and in various other class libraries. Related classes are typically grouped into *packages* so that they can be *imported* into programs and *reused*. You'll learn how to group your own classes into *packages* in Section 21.4.10. The Java API provides a rich collection of predefined classes that contain methods for performing common mathematical calculations, string manipulations, character manipulations, input/output operations, database operations, networking operations, file processing, error checking and more.



Software Engineering Observation 6.1

Familiarize yourself with the rich collection of classes and methods provided by the Java API (<http://docs.oracle.com/javase/7/docs/api/>). Section 6.8 overviews several common packages. Online Appendix F explains how to navigate the API documentation. Don't reinvent the wheel. When possible, reuse Java API classes and methods. This reduces program development time and avoids introducing programming errors.

Divide and Conquer with Classes and Methods

Classes and methods help you modularize a program by separating its tasks into self-contained units. The statements in the method bodies are written only once, are hidden from other methods and can be reused from several locations in a program.

One motivation for modularizing a program into classes and methods is the *divide-and-conquer* approach, which makes program development more manageable by constructing programs from small, simple pieces. Another is **software reusability**—using existing classes and methods as building blocks to create new programs. Often, you can create programs mostly from existing classes and methods rather than by building customized code. For example, in earlier programs, we did not define how to read data from the keyboard—Java provides these capabilities in the methods of class `Scanner`. A third motivation is to *avoid repeating code*. Dividing a program into meaningful classes and methods makes the program easier to debug and maintain.



Software Engineering Observation 6.2

To promote software reusability, every method should be limited to performing a single, well-defined task, and the name of the method should express that task effectively.



Error-Prevention Tip 6.1

A method that performs one task is easier to test and debug than one that performs many tasks.



Software Engineering Observation 6.3

If you cannot choose a concise name that expresses a method's task, your method might be attempting to perform too many tasks. Break such a method into several smaller ones.

Hierarchical Relationship Between Method Calls

As you know, a method is invoked by a method call, and when the called method completes its task, it returns control and possibly a result to the caller. An analogy to this program structure is the hierarchical form of management (Fig. 6.1). A boss (the caller) asks a worker (the called method) to perform a task and report back (return) the results after completing the task. The boss method does not know how the worker method performs its designated tasks. The worker may also call other worker methods, unbeknown to the boss. This “hiding” of implementation details promotes good software engineering. Figure 6.1 shows the boss method communicating with several worker methods in a hierarchical manner. The boss method divides its responsibilities among the various worker methods. Here, worker1 acts as a “boss method” to worker4 and worker5.



Error-Prevention Tip 6.2

When you call a method that returns a value indicating whether the method performed its task successfully, be sure to check the return value of that method and, if that method was unsuccessful, deal with the issue appropriately.

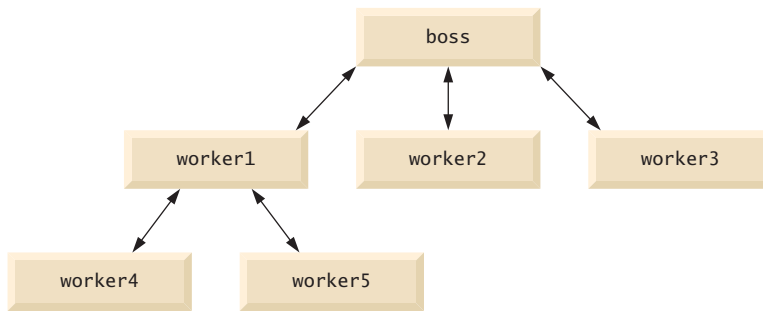


Fig. 6.1 | Hierarchical boss-method/worker-method relationship.

6.3 static Methods, static Fields and Class Math

Although most methods execute in response to method calls *on specific objects*, this is not always the case. Sometimes a method performs a task that does not depend on an object. Such a method applies to the class in which it's declared as a whole and is known as a **static method** or a **class method**.

It's common for classes to contain convenient static methods to perform common tasks. For example, recall that we used static method `pow` of class `Math` to raise a value to a power in Fig. 5.6. To declare a method as static, place the keyword `static` before the return type in the method's declaration. For any class imported into your program, you can call the class's static methods by specifying the name of the class in which the method is declared, followed by a dot (`.`) and the method name, as in

```
ClassName.methodName(arguments)
```

Math Class Methods

We use various `Math` class methods here to present the concept of static methods. Class `Math` provides a collection of methods that enable you to perform common mathematical calculations. For example, you can calculate the square root of 900.0 with the static method call

```
Math.sqrt(900.0)
```

This expression evaluates to 30.0. Method `sqrt` takes an argument of type `double` and returns a result of type `double`. To output the value of the preceding method call in the command window, you might write the statement

```
System.out.println(Math.sqrt(900.0));
```

In this statement, the value that `sqrt` returns becomes the argument to method `println`. There was no need to create a `Math` object before calling method `sqrt`. Also *all* `Math` class methods are static—therefore, each is called by preceding its name with the class name `Math` and the dot (`.`) separator.



Software Engineering Observation 6.4

Class `Math` is part of the `java.lang` package, which is implicitly imported by the compiler, so it's not necessary to import class `Math` to use its methods.

Method arguments may be constants, variables or expressions. If $c = 13.0$, $d = 3.0$ and $f = 4.0$, then the statement

```
System.out.println(Math.sqrt(c + d * f));
```

calculates and prints the square root of $13.0 + 3.0 * 4.0 = 25.0$ —namely, 5.0. Figure 6.2 summarizes several Math class methods. In the figure, x and y are of type `double`.

Method	Description	Example
<code>abs(x)</code>	absolute value of x	<code>abs(23.7)</code> is 23.7 <code>abs(0.0)</code> is 0.0 <code>abs(-23.7)</code> is 23.7
<code>ceil(x)</code>	rounds x to the smallest integer not less than x	<code>ceil(9.2)</code> is 10.0 <code>ceil(-9.8)</code> is -9.0
<code>cos(x)</code>	trigonometric cosine of x (x in radians)	<code>cos(0.0)</code> is 1.0
<code>exp(x)</code>	exponential method e^x	<code>exp(1.0)</code> is 2.71828 <code>exp(2.0)</code> is 7.38906
<code>floor(x)</code>	rounds x to the largest integer not greater than x	<code>floor(9.2)</code> is 9.0 <code>floor(-9.8)</code> is -10.0
<code>log(x)</code>	natural logarithm of x (base e)	<code>log(Math.E)</code> is 1.0 <code>log(Math.E * Math.E)</code> is 2.0
<code>max(x, y)</code>	larger value of x and y	<code>max(2.3, 12.7)</code> is 12.7 <code>max(-2.3, -12.7)</code> is -2.3
<code>min(x, y)</code>	smaller value of x and y	<code>min(2.3, 12.7)</code> is 2.3 <code>min(-2.3, -12.7)</code> is -12.7
<code>pow(x, y)</code>	x raised to the power y (i.e., x^y)	<code>pow(2.0, 7.0)</code> is 128.0 <code>pow(9.0, 0.5)</code> is 3.0
<code>sin(x)</code>	trigonometric sine of x (x in radians)	<code>sin(0.0)</code> is 0.0
<code>sqrt(x)</code>	square root of x	<code>sqrt(900.0)</code> is 30.0
<code>tan(x)</code>	trigonometric tangent of x (x in radians)	<code>tan(0.0)</code> is 0.0

Fig. 6.2 | Math class methods.

static Variables

Recall from Section 3.2 that each object of a class maintains its *own* copy of every instance variable of the class. There are variables for which each object of a class does *not* need its own separate copy (as you'll see momentarily). Such variables are declared `static` and are also known as **class variables**. When objects of a class containing static variables are created, all the objects of that class share *one* copy of those variables. Together a class's static variables and instance variables are known as its **fields**. You'll learn more about static fields in Section 8.11.

Math Class static Constants `PI` and `E`

Class Math declares two constants, `Math.PI` and `Math.E`, that represent *high-precision approximations* to commonly used mathematical constants. `Math.PI` (3.141592653589793) is the ratio of a circle's circumference to its diameter. `Math.E` (2.718281828459045) is the

base value for natural logarithms (calculated with static `Math` method `log`). These constants are declared in class `Math` with the modifiers `public`, `final` and `static`. Making them `public` allows you to use them in your own classes. Any field declared with keyword **final** is *constant*—its value cannot change after the field is initialized. Making these fields `static` allows them to be accessed via the class name `Math` and a dot (`.`) separator, just as class `Math`'s methods are.

Why Is Method `main` Declared `static`?

When you execute the Java Virtual Machine (JVM) with the `java` command, the JVM attempts to invoke the `main` method of the class you specify—at this point no objects of the class have been created. Declaring `main` as `static` allows the JVM to invoke `main` without creating an instance of the class. When you execute your application, you specify its class name as an argument to the `java` command, as in

```
java ClassName argument1 argument2 ...
```

The JVM loads the class specified by `ClassName` and uses that class name to invoke method `main`. In the preceding command, `ClassName` is a **command-line argument** to the JVM that tells it which class to execute. Following the `ClassName`, you can also specify a list of Strings (separated by spaces) as command-line arguments that the JVM will pass to your application. Such arguments might be used to specify options (e.g., a filename) to run the application. As you'll learn in Chapter 7, Arrays and ArrayLists, your application can access those command-line arguments and use them to customize the application.

6.4 Declaring Methods with Multiple Parameters

Methods often require more than one piece of information to perform their tasks. We now consider how to write your own methods with *multiple* parameters.

Figure 6.3 uses a method called `maximum` to determine and return the largest of three `double` values. In `main`, lines 14–18 prompt the user to enter three `double` values, then read them from the user. Line 21 calls method `maximum` (declared in lines 28–41) to determine the largest of the three values it receives as arguments. When method `maximum` returns the result to line 21, the program assigns `maximum`'s return value to local variable `result`. Then line 24 outputs the maximum value. At the end of this section, we'll discuss the use of operator `+` in line 24.

```

1 // Fig. 6.3: MaximumFinder.java
2 // Programmer-declared method maximum with three double parameters.
3 import java.util.Scanner;
4
5 public class MaximumFinder
6 {
7     // obtain three floating-point values and locate the maximum value
8     public static void main(String[] args)
9     {
10         // create Scanner for input from command window
11         Scanner input = new Scanner(System.in);
12

```

Fig. 6.3 | Programmer-declared method `maximum` with three `double` parameters. (Part 1 of 2.)

```

13      // prompt for and input three floating-point values
14      System.out.print(
15          "Enter three floating-point values separated by spaces: ";
16      double number1 = input.nextDouble(); // read first double
17      double number2 = input.nextDouble(); // read second double
18      double number3 = input.nextDouble(); // read third double
19
20      // determine the maximum value
21      double result = maximum(number1, number2, number3);
22
23      // display maximum value
24      System.out.println("Maximum is: " + result);
25  }
26
27      // returns the maximum of its three double parameters
28      public static double maximum(double x, double y, double z)
29      {
30          double maximumValue = x; // assume x is the largest to start
31
32          // determine whether y is greater than maximumValue
33          if (y > maximumValue)
34              maximumValue = y;
35
36          // determine whether z is greater than maximumValue
37          if (z > maximumValue)
38              maximumValue = z;
39
40          return maximumValue;
41      }
42  } // end class MaximumFinder

```

Enter three floating-point values separated by spaces: 9.35 2.74 5.1
Maximum is: 9.35

Enter three floating-point values separated by spaces: 5.8 12.45 8.32
Maximum is: 12.45

Enter three floating-point values separated by spaces: 6.46 4.12 10.54
Maximum is: 10.54

Fig. 6.3 | Programmer-declared method `maximum` with three `double` parameters. (Part 2 of 2.)

*The **public** and **static** Keywords*

Method `maximum`'s declaration begins with keyword `public` to indicate that the method is “available to the public”—it can be called from methods of other classes. The keyword `static` enables the `main` method (another static method) to call `maximum` as shown in line 21 without qualifying the method name with the class name `MaximumFinder`—static methods in the same class can call each other directly. Any other class that uses `maximum` must fully qualify the method name with the class name.

Method maximum

Consider `maximum`'s declaration (lines 28–41). Line 28 indicates that it returns a `double` value, that the method's name is `maximum` and that the method requires three `double` parameters (`x`, `y` and `z`) to accomplish its task. Multiple parameters are specified as a comma-separated list. When `maximum` is called from line 21, the parameters `x`, `y` and `z` are initialized with copies of the values of arguments `number1`, `number2` and `number3`, respectively. There must be one argument in the method call for each parameter in the method declaration. Also, each argument must be *consistent* with the type of the corresponding parameter. For example, a parameter of type `double` can receive values like 7.35, 22 or `-0.03456`, but not `Strings` like `"hello"` nor the `boolean` values `true` or `false`. Section 6.7 discusses the argument types that can be provided in a method call for each parameter of a primitive type.

To determine the maximum value, we begin with the assumption that parameter `x` contains the largest value, so line 30 declares local variable `maximumValue` and initializes it with the value of parameter `x`. Of course, it's possible that parameter `y` or `z` contains the actual largest value, so we must compare each of these values with `maximumValue`. The `if` statement at lines 33–34 determines whether `y` is greater than `maximumValue`. If so, line 34 assigns `y` to `maximumValue`. The `if` statement at lines 37–38 determines whether `z` is greater than `maximumValue`. If so, line 38 assigns `z` to `maximumValue`. At this point the largest of the three values resides in `maximumValue`, so line 40 returns that value to line 21. When program control returns to the point in the program where `maximum` was called, `maximum`'s parameters `x`, `y` and `z` no longer exist in memory.

**Software Engineering Observation 6.5**

Methods can return at most one value, but the returned value could be a reference to an object that contains many values.

**Software Engineering Observation 6.6**

Variables should be declared as fields only if they're required for use in more than one method of the class or if the program should save their values between calls to the class's methods.

**Common Programming Error 6.1**

Declaring method parameters of the same type as `float x, y` instead of `float x, float y` is a syntax error—a type is required for each parameter in the parameter list.

Implementing Method maximum by Reusing Method Math.max

The entire body of our `maximum` method could also be implemented with two calls to `Math.max`, as follows:

```
return Math.max(x, Math.max(y, z));
```

The first call to `Math.max` specifies arguments `x` and `Math.max(y, z)`. *Before* any method can be called, its arguments must be evaluated to determine their values. If an argument is a method call, the method call must be performed to determine its return value. So, in the preceding statement, `Math.max(y, z)` is evaluated to determine the maximum of `y` and `z`. Then the result is passed as the second argument to the other call to `Math.max`, which returns the larger of its two arguments. This is a good example of *software reuse*—we find the largest of three values by reusing `Math.max`, which finds the larger of two values. Note how concise this code is compared to lines 30–38 of Fig. 6.3.

Assembling Strings with String Concatenation

Java allows you to assemble `String` objects into larger strings by using operators `+` or `+=`. This is known as **string concatenation**. When both operands of operator `+` are `String` objects, operator `+` creates a new `String` object in which the characters of the right operand are placed at the end of those in the left operand—e.g., the expression `"hello " + "there"` creates the `String` `"hello there"`.

In line 24 of Fig. 6.3, the expression `"Maximum is: " + result` uses operator `+` with operands of types `String` and `double`. *Every primitive value and object in Java can be represented as a String*. When one of the `+` operator's operands is a `String`, the other is converted to a `String`, then the two are *concatenated*. In line 24, the `double` value is converted to its `String` representation and placed at the end of the `String` `"Maximum is: "`. If there are any *trailing zeros* in a `double` value, these will be *discarded* when the number is converted to a `String`—for example 9.3500 would be represented as 9.35.

Primitive values used in `String` concatenation are converted to `Strings`. A `boolean` concatenated with a `String` is converted to the `String` `"true"` or `"false"`. *All objects have a toString method that returns a String representation of the object*. (We discuss `toString` in more detail in subsequent chapters.) When an object is concatenated with a `String`, the object's `toString` method is implicitly called to obtain the `String` representation of the object. Method `toString` also can be called explicitly.

You can break large `String` literals into several smaller `Strings` and place them on multiple lines of code for readability. In this case, the `Strings` can be reassembled using concatenation. We discuss the details of `Strings` in Chapter 14.



Common Programming Error 6.2

It's a syntax error to break a `String` literal across lines. If necessary, you can split a `String` into several smaller `Strings` and use concatenation to form the desired `String`.



Common Programming Error 6.3

Confusing the `+` operator used for string concatenation with the `+` operator used for addition can lead to strange results. Java evaluates the operands of an operator from left to right. For example, if integer variable `y` has the value 5, the expression `"y + 2 = " + y + 2` results in the string `"y + 2 = 52"`, not `"y + 2 = 7"`, because first the value of `y` (5) is concatenated to the string `"y + 2 = "`, then the value 2 is concatenated to the new larger string `"y + 2 = 5"`. The expression `"y + 2 = " + (y + 2)` produces the desired result `"y + 2 = 7"`.

6.5 Notes on Declaring and Using Methods

There are three ways to call a method:

1. Using a method name by itself to call another method of the *same* class—such as `maximum(number1, number2, number3)` in line 21 of Fig. 6.3.
2. Using a variable that contains a reference to an object, followed by a dot (`.`) and the method name to call a non-static method of the referenced object—such as the method call in line 16 of Fig. 3.2, `myAccount.getName()`, which calls a method of class `Account` from the main method of `AccountTest`. Non-static methods are typically called **instance methods**.

3. Using the class name and a dot (.) to call a static method of a class—such as `Math.sqrt(900.0)` in Section 6.3.

A static method can call other static methods of the same class directly (i.e., using the method name by itself) and can manipulate static variables in the same class directly. To access the class's instance variables and instance methods, a static method must use a reference to an object of the class. Instance methods can access all fields (static variables and instance variables) and methods of the class.

Recall that static methods relate to a class as a whole, whereas instance methods are associated with a specific instance (object) of the class and may manipulate the instance variables of that object. Many objects of a class, each with its *own* copies of the instance variables, may exist at the same time. Suppose a static method were to invoke an instance method directly. How would the static method know which object's instance variables to manipulate? What would happen if *no* objects of the class existed at the time the instance method was invoked? Thus, Java does *not* allow a static method to directly access instance variables and instance methods of the same class.

There are three ways to return control to the statement that calls a method. If the method does not return a result, control returns when the program flow reaches the method-ending right brace or when the statement

```
return;
```

is executed. If the method returns a result, the statement

```
return expression;
```

evaluates the *expression*, then returns the result to the caller.



Common Programming Error 6.4

Declaring a method outside the body of a class declaration or inside the body of another method is a syntax error.



Common Programming Error 6.5

Redeclaring a parameter as a local variable in the method's body is a compilation error.



Common Programming Error 6.6

Forgetting to return a value from a method that should return a value is a compilation error. If a return type other than void is specified, the method must contain a return statement that returns a value consistent with the method's return type. Returning a value from a method whose return type has been declared void is a compilation error.

6.6 Method-Call Stack and Stack Frames

To understand how Java performs method calls, we first need to consider a data structure (i.e., collection of related data items) known as a **stack**. You can think of a stack as analogous to a pile of dishes. When a dish is placed on the pile, it's normally placed at the top (referred to as **pushing** the dish onto the stack). Similarly, when a dish is removed from the pile, it's normally removed from the top (referred to as **popping** the dish off the stack).

Stacks are known as **last-in, first-out (LIFO) data structures**—the *last* item pushed onto the stack is the *first* item popped from the stack.

When a program *calls* a method, the called method must know how to *return* to its caller, so the *return address* of the calling method is *pushed* onto the **method-call stack**. If a series of method calls occurs, the successive return addresses are pushed onto the stack in last-in, first-out order so that each method can return to its caller.

The method-call stack also contains the memory for the *local variables* (including the method parameters) used in each invocation of a method during a program's execution. This data, stored as a portion of the method-call stack, is known as the **stack frame** (or **activation record**) of the method call. When a method call is made, the stack frame for that method call is *pushed* onto the method-call stack. When the method returns to its caller, the stack frame for this method call is *popped* off the stack and those local variables are no longer known to the program. If a local variable holding a reference to an object is the only variable in the program with a reference to that object, then, when the stack frame containing that local variable is popped off the stack, the object can no longer be accessed by the program and will eventually be deleted from memory by the JVM during *garbage collection*, which we discuss in Section 8.10.

Of course, a computer's memory is finite, so only a certain amount can be used to store stack frames on the method-call stack. If more method calls occur than can have their stack frames stored, an error known as a **stack overflow** occurs—we'll talk more about this in Chapter 11, Exception Handling: A Deeper Look.

6.7 Argument Promotion and Casting

Another important feature of method calls is **argument promotion**—converting an *argument's value*, if possible, to the type that the method expects to receive in its corresponding *parameter*. For example, a program can call `Math.sqrt` with an `int` argument even though a `double` argument is expected. The statement

```
System.out.println(Math.sqrt(4));
```

correctly evaluates `Math.sqrt(4)` and prints the value `2.0`. The method declaration's parameter list causes Java to convert the `int` value `4` to the `double` value `4.0` *before* passing the value to method `sqrt`. Such conversions may lead to compilation errors if Java's **promotion rules** are not satisfied. These rules specify which conversions are allowed—that is, which ones can be performed *without losing data*. In the `sqrt` example above, an `int` is converted to a `double` without changing its value. However, converting a `double` to an `int` *truncates* the fractional part of the `double` value—thus, part of the value is lost. Converting large integer types to small integer types (e.g., `long` to `int`, or `int` to `short`) may also result in changed values.

The promotion rules apply to expressions containing values of two or more primitive types and to primitive-type values passed as arguments to methods. Each value is promoted to the “highest” type in the expression. Actually, the expression uses a *temporary copy* of each value—the types of the original values remain unchanged. Figure 6.4 lists the primitive types and the types to which each can be promoted. The valid promotions for a given type are always to a type higher in the table. For example, an `int` can be promoted to the higher types `long`, `float` and `double`.

Converting values to types lower in the table of Fig. 6.4 will result in different values if the lower type cannot represent the value of the higher type (e.g., the `int` value 2000000 cannot be represented as a `short`, and any floating-point number with digits after its decimal point cannot be represented in an integer type such as `long`, `int` or `short`). Therefore, in cases where information may be lost due to conversion, the Java compiler requires you to use a *cast operator* (introduced in Section 4.10) to explicitly force the conversion to occur—otherwise a compilation error occurs. This enables you to “take control” from the compiler. You essentially say, “I know this conversion might cause loss of information, but for my purposes here, that’s fine.” Suppose method `square` calculates the square of an integer and thus requires an `int` argument. To call `square` with a `double` argument named `doubleValue`, we would be required to write the method call as

```
square((int) doubleValue)
```

This method call explicitly casts (converts) `doubleValue`’s value to a temporary integer for use in method `square`. Thus, if `doubleValue`’s value is 4.5, the method receives the value 4 and returns 16, not 20.25.



Common Programming Error 6.7

Casting a primitive-type value to another primitive type may change the value if the new type is not a valid promotion. For example, casting a floating-point value to an integer value may introduce truncation errors (loss of the fractional part) into the result.

Type	Valid promotions
<code>double</code>	None
<code>float</code>	<code>double</code>
<code>long</code>	<code>float</code> or <code>double</code>
<code>int</code>	<code>long</code> , <code>float</code> or <code>double</code>
<code>char</code>	<code>int</code> , <code>long</code> , <code>float</code> or <code>double</code>
<code>short</code>	<code>int</code> , <code>long</code> , <code>float</code> or <code>double</code> (but not <code>char</code>)
<code>byte</code>	<code>short</code> , <code>int</code> , <code>long</code> , <code>float</code> or <code>double</code> (but not <code>char</code>)
<code>boolean</code>	None (boolean values are not considered to be numbers in Java)

Fig. 6.4 | Promotions allowed for primitive types.

6.8 Java API Packages

As you’ve seen, Java contains many *predefined* classes that are grouped into categories of related classes called *packages*. Together, these are known as the Java Application Programming Interface (Java API), or the Java class library. A great strength of Java is the Java API’s thousands of classes. Some key Java API packages that we use in this book are described in Fig. 6.5, which represents only a small portion of the *reusable components* in the Java API.

Package	Description
<code>java.awt.event</code>	The Java Abstract Window Toolkit Event Package contains classes and interfaces that enable event handling for GUI components in both the <code>java.awt</code> and <code>javax.swing</code> packages. (See Chapter 12, GUI Components: Part 1, and Chapter 22, GUI Components: Part 2.)
<code>java.awt.geom</code>	The Java 2D Shapes Package contains classes and interfaces for working with Java's advanced two-dimensional graphics capabilities. (See Chapter 13, .)
<code>java.io</code>	The Java Input/Output Package contains classes and interfaces that enable programs to input and output data. (See Chapter 15, Files, Streams and Object Serialization.)
<code>java.lang</code>	The Java Language Package contains classes and interfaces (discussed throughout the book) that are required by many Java programs. This package is imported by the compiler into all programs.
<code>java.net</code>	The Java Networking Package contains classes and interfaces that enable programs to communicate via computer networks like the Internet. (See online Chapter 28, Networking.)
<code>java.security</code>	The Java Security Package contains classes and interfaces for enhancing application security.
<code>java.sql</code>	The JDBC Package contains classes and interfaces for working with databases. (See Chapter 24, Accessing Databases with JDBC.)
<code>java.util</code>	The Java Utilities Package contains utility classes and interfaces that enable storing and processing of large amounts of data. Many of these classes and interfaces have been updated to support Java SE 8's new lambda capabilities. (See Chapter 16, Generic Collections.)
<code>java.util.concurrent</code>	The Java Concurrency Package contains utility classes and interfaces for implementing programs that can perform multiple tasks in parallel. (See Chapter 23, Concurrency.)
<code>javax.swing</code>	The Java Swing GUI Components Package contains classes and interfaces for Java's Swing GUI components that provide support for portable GUIs. This package still uses some elements of the older <code>java.awt</code> package. (See Chapter 12, GUI Components: Part 1, and Chapter 22, GUI Components: Part 2.)
<code>javax.swing.event</code>	The Java Swing Event Package contains classes and interfaces that enable event handling (e.g., responding to button clicks) for GUI components in package <code>javax.swing</code> . (See Chapter 12, GUI Components: Part 1, and Chapter 22, GUI Components: Part 2.)
<code>javax.xml.ws</code>	The JAX-WS Package contains classes and interfaces for working with web services in Java. (See online Chapter 32, REST-Based Web Services.)
<code>javafx</code> packages	JavaFX is the preferred GUI technology for the future. We discuss these packages in Chapter 25, JavaFX GUI: Part 1 and in the online JavaFX GUI and multimedia chapters.

Fig. 6.5 | Java API packages (a subset). (Part 1 of 2.)

Package	Description
<i>Some Java SE 8 Packages Used in This Book</i>	
<code>java.time</code>	The new Java SE 8 Date/Time API Package contains classes and interfaces for working with dates and times. These features are designed to replace the older date and time capabilities of package <code>java.util</code> . (See Chapter 23, Concurrency.)
<code>java.util.function</code> and <code>java.util.stream</code>	These packages contain classes and interfaces for working with Java SE 8's functional programming capabilities. (See Chapter 17, Java SE 8 Lambdas and Streams.)

Fig. 6.5 | Java API packages (a subset). (Part 2 of 2.)

The set of packages available in Java is quite large. In addition to those summarized in Fig. 6.5, Java includes packages for complex graphics, advanced graphical user interfaces, printing, advanced networking, security, database processing, multimedia, accessibility (for people with disabilities), concurrent programming, cryptography, XML processing and many other capabilities. For an overview of the packages in Java, visit

<http://docs.oracle.com/javase/7/docs/api/overview-summary.html>
<http://download.java.net/jdk8/docs/api/overview-summary.html>

You can locate additional information about a predefined Java class's methods in the Java API documentation at

<http://docs.oracle.com/javase/7/docs/api/>

When you visit this site, click the **Index** link to see an alphabetical listing of all the classes and methods in the Java API. Locate the class name and click its link to see the online description of the class. Click the **METHOD** link to see a table of the class's methods. Each static method will be listed with the word “static” preceding its return type.

6.9 Case Study: Secure Random-Number Generation

We now take a brief diversion into a popular type of programming application—simulation and game playing. In this and the next section, we develop a game-playing program with multiple methods. The program uses most of the control statements presented thus far in the book and introduces several new programming concepts.

The **element of chance** can be introduced in a program via an object of class **SecureRandom** (package `java.security`). Such objects can produce random `boolean`, `byte`, `float`, `double`, `int`, `long` and Gaussian values. In the next several examples, we use objects of class `SecureRandom` to produce random values.

Moving to Secure Random Numbers

Recent editions of this book used Java's `Random` class to obtain “random” values. This class produced *deterministic* values that could be *predicted* by malicious programmers. `SecureRandom` objects produce **nondeterministic random numbers** that *cannot* be predicted.

Deterministic random numbers have been the source of many software security breaches. Most programming languages now have library features similar to Java's `Secure-`

Random class for producing nondeterministic random numbers to help prevent such problems. From this point forward in the text, when we refer to “random numbers” we mean “secure random numbers.”

Creating a SecureRandom Object

A new secure random-number generator object can be created as follows:

```
SecureRandom randomNumbers = new SecureRandom();
```

It can then be used to generate random values—we discuss only random `int` values here. For more information on the `SecureRandom` class, see docs.oracle.com/javase/7/docs/api/java/security/SecureRandom.html.

Obtaining a Random int Value

Consider the following statement:

```
int randomValue = randomNumbers.nextInt();
```

`SecureRandom` method `nextInt` generates a random `int` value. If it truly produces values *at random*, then every value in the range should have an *equal chance* (or probability) of being chosen each time `nextInt` is called.

Changing the Range of Values Produced By nextInt

The range of values produced by method `nextInt` generally differs from the range of values required in a particular Java application. For example, a program that simulates coin tossing might require only 0 for “heads” and 1 for “tails.” A program that simulates the rolling of a six-sided die might require random integers in the range 1–6. A program that randomly predicts the next type of spaceship (out of four possibilities) that will fly across the horizon in a video game might require random integers in the range 1–4. For cases like these, class `SecureRandom` provides another version of method `nextInt` that receives an `int` argument and returns a value from 0 up to, but not including, the argument’s value. For example, for coin tossing, the following statement returns 0 or 1.

```
int randomValue = randomNumbers.nextInt(2);
```

Rolling a Six-Sided Die

To demonstrate random numbers, let’s develop a program that simulates 20 rolls of a six-sided die and displays the value of each roll. We begin by using `nextInt` to produce random values in the range 0–5, as follows:

```
int face = randomNumbers.nextInt(6);
```

The argument 6—called the **scaling factor**—represents the number of unique values that `nextInt` should produce (in this case six—0, 1, 2, 3, 4 and 5). This manipulation is called **scaling** the range of values produced by `SecureRandom` method `nextInt`.

A six-sided die has the numbers 1–6 on its faces, not 0–5. So we **shift** the range of numbers produced by adding a **shifting value**—in this case 1—to our previous result, as in

```
int face = 1 + randomNumbers.nextInt(6);
```

The shifting value (1) specifies the *first* value in the desired range of random integers. The preceding statement assigns `face` a random integer in the range 1–6.

Rolling a Six-Sided Die 20 Times

Figure 6.6 shows two sample outputs which confirm that the results of the preceding calculation are integers in the range 1–6, and that each run of the program can produce a *different* sequence of random numbers. Line 3 imports class `SecureRandom` from the `java.security` package. Line 10 creates the `SecureRandom` object `randomNumbers` to produce random values. Line 16 executes 20 times in a loop to roll the die. The `if` statement (lines 21–22) in the loop starts a new line of output after every five numbers to create a neat, five-column format.

```

1  // Fig. 6.6: RandomIntegers.java
2  // Shifted and scaled random integers.
3  import java.security.SecureRandom; // program uses class SecureRandom
4
5  public class RandomIntegers
6  {
7      public static void main(String[] args)
8      {
9          // randomNumbers object will produce secure random numbers
10         SecureRandom randomNumbers = new SecureRandom();
11
12         // loop 20 times
13         for (int counter = 1; counter <= 20; counter++)
14         {
15             // pick random integer from 1 to 6
16             int face = 1 + randomNumbers.nextInt(6);
17
18             System.out.printf("%d ", face); // display generated value
19
20             // if counter is divisible by 5, start a new line of output
21             if (counter % 5 == 0)
22                 System.out.println();
23         }
24     }
25 } // end class RandomIntegers

```

```

1 5 3 6 2
5 2 6 5 2
4 4 4 2 6
3 1 6 2 2

```

```

6 5 4 2 6
1 2 5 1 3
6 3 2 2 1
6 4 2 6 4

```

Fig. 6.6 | Shifted and scaled random integers.

Rolling a Six-Sided Die 6,000,000 Times

To show that the numbers produced by `nextInt` occur with approximately equal likelihood, let's simulate 6,000,000 rolls of a die with the application in Fig. 6.7. Each integer from 1 to 6 should appear approximately 1,000,000 times.

```

1  // Fig. 6.7: RollDie.java
2  // Roll a six-sided die 6,000,000 times.
3  import java.security.SecureRandom;
4
5  public class RollDie
6  {
7      public static void main(String[] args)
8      {
9          // randomNumbers object will produce secure random numbers
10         SecureRandom randomNumbers = new SecureRandom();
11
12         int frequency1 = 0; // count of 1s rolled
13         int frequency2 = 0; // count of 2s rolled
14         int frequency3 = 0; // count of 3s rolled
15         int frequency4 = 0; // count of 4s rolled
16         int frequency5 = 0; // count of 5s rolled
17         int frequency6 = 0; // count of 6s rolled
18
19         // tally counts for 6,000,000 rolls of a die
20         for (int roll = 1; roll <= 6000000; roll++)
21         {
22             int face = 1 + randomNumbers.nextInt(6); // number from 1 to 6
23
24             // use face value 1-6 to determine which counter to increment
25             switch (face)
26             {
27                 case 1:
28                     ++frequency1; // increment the 1s counter
29                     break;
30                 case 2:
31                     ++frequency2; // increment the 2s counter
32                     break;
33                 case 3:
34                     ++frequency3; // increment the 3s counter
35                     break;
36                 case 4:
37                     ++frequency4; // increment the 4s counter
38                     break;
39                 case 5:
40                     ++frequency5; // increment the 5s counter
41                     break;
42                 case 6:
43                     ++frequency6; // increment the 6s counter
44                     break;
45             }
46         }
47
48         System.out.println("Face\tFrequency"); // output headers
49         System.out.printf("1\t%d\n2\t%d\n3\t%d\n4\t%d\n5\t%d\n6\t%d\n",
50             frequency1, frequency2, frequency3, frequency4,
51             frequency5, frequency6);
52     }
53 } // end class RollDie

```

Fig. 6.7 | Roll a six-sided die 6,000,000 times. (Part 1 of 2.)

Face	Frequency
1	999501
2	1000412
3	998262
4	1000820
5	1002245
6	998760

Face	Frequency
1	999647
2	999557
3	999571
4	1000376
5	1000701
6	1000148

Fig. 6.7 | Roll a six-sided die 6,000,000 times. (Part 2 of 2.)

As the sample outputs show, scaling and shifting the values produced by `nextInt` enables the program to simulate rolling a six-sided die. The application uses nested control statements (the `switch` is nested inside the `for`) to determine the number of times each side of the die appears. The `for` statement (lines 20–46) iterates 6,000,000 times. During each iteration, line 22 produces a random value from 1 to 6. That value is then used as the controlling expression (line 25) of the `switch` statement (lines 25–45). Based on the face value, the `switch` statement increments one of the six counter variables during each iteration of the loop. This `switch` statement has no default case, because we have a case for every possible die value that the expression in line 22 could produce. Run the program, and observe the results. As you'll see, every time you run this program, it produces *different* results.

When we study arrays in Chapter 7, we'll show an elegant way to replace the entire `switch` statement in this program with a *single* statement. Then, when we study Java SE 8's new functional programming capabilities in Chapter 17, we'll show how to replace the loop that rolls the dice, the `switch` statement *and* the statement that displays the results with a *single* statement!

Generalized Scaling and Shifting of Random Numbers

Previously, we simulated the rolling of a six-sided die with the statement

```
int face = 1 + randomNumbers.nextInt(6);
```

This statement always assigns to variable `face` an integer in the range $1 \leq \text{face} \leq 6$. The *width* of this range (i.e., the number of consecutive integers in the range) is 6, and the *starting number* in the range is 1. In the preceding statement, the width of the range is determined by the number 6 that's passed as an argument to `SecureRandom` method `nextInt`, and the starting number of the range is the number 1 that's added to `randomNumbers.nextInt(6)`. We can generalize this result as

```
int number = shiftingValue + randomNumbers.nextInt(scalingFactor);
```

where *shiftingValue* specifies the *first number* in the desired range of consecutive integers and *scalingFactor* specifies *how many numbers* are in the range.

It's also possible to choose integers at random from sets of values other than ranges of consecutive integers. For example, to obtain a random value from the sequence 2, 5, 8, 11 and 14, you could use the statement

```
int number = 2 + 3 * randomNumbers.nextInt(5);
```

In this case, `randomNumbers.nextInt(5)` produces values in the range 0–4. Each value produced is multiplied by 3 to produce a number in the sequence 0, 3, 6, 9 and 12. We add 2 to that value to *shift* the range of values and obtain a value from the sequence 2, 5, 8, 11 and 14. We can generalize this result as

```
int number = shiftingValue +
    differenceBetweenValues * randomNumbers.nextInt(scalingFactor);
```

where *shiftingValue* specifies the first number in the desired range of values, *differenceBetweenValues* represents the *constant difference* between consecutive numbers in the sequence and *scalingFactor* specifies how many numbers are in the range.

A Note About Performance

Using `SecureRandom` instead of `Random` to achieve higher levels of security incurs a significant performance penalty. For “casual” applications, you might want to use class `Random` from package `java.util`—simply replace `SecureRandom` with `Random`.

6.10 Case Study: A Game of Chance; Introducing enum Types

A popular game of chance is a dice game known as craps, which is played in casinos and back alleys throughout the world. The rules of the game are straightforward:

You roll two dice. Each die has six faces, which contain one, two, three, four, five and six spots, respectively. After the dice have come to rest, the sum of the spots on the two upward faces is calculated. If the sum is 7 or 11 on the first throw, you win. If the sum is 2, 3 or 12 on the first throw (called “craps”), you lose (i.e., the “house” wins). If the sum is 4, 5, 6, 8, 9 or 10 on the first throw, that sum becomes your “point.” To win, you must continue rolling the dice until you “make your point” (i.e., roll that same point value). You lose by rolling a 7 before making your point.

Figure 6.8 simulates the game of craps, using methods to implement the game's logic. The main method (lines 21–65) calls the `rollDice` method (lines 68–81) as necessary to roll the dice and compute their sum. The sample outputs show winning and losing on the first roll, and winning and losing on a subsequent roll.

```
1 // Fig. 6.8: Craps.java
2 // Craps class simulates the dice game craps.
3 import java.security.SecureRandom;
4
5 public class Craps
6 {
7     // create secure random number generator for use in method rollDice
8     private static final SecureRandom randomNumbers = new SecureRandom();
```

Fig. 6.8 | Craps class simulates the dice game craps. (Part I of 3.)

```
9
10 // enum type with constants that represent the game status
11 private enum Status { CONTINUE, WON, LOST };
12
13 // constants that represent common rolls of the dice
14 private static final int SNAKE_EYES = 2;
15 private static final int TREY = 3;
16 private static final int SEVEN = 7;
17 private static final int YO_LEVEN = 11;
18 private static final int BOX_CARS = 12;
19
20 // plays one game of craps
21 public static void main(String[] args)
22 {
23     int myPoint = 0; // point if no win or loss on first roll
24     Status gameStatus; // can contain CONTINUE, WON or LOST
25
26     int sumOfDice = rollDice(); // first roll of the dice
27
28     // determine game status and point based on first roll
29     switch (sumOfDice)
30     {
31         case SEVEN: // win with 7 on first roll
32         case YO_LEVEN: // win with 11 on first roll
33             gameStatus = Status.WON;
34             break;
35         case SNAKE_EYES: // lose with 2 on first roll
36         case TREY: // lose with 3 on first roll
37         case BOX_CARS: // lose with 12 on first roll
38             gameStatus = Status.LOST;
39             break;
40         default: // did not win or lose, so remember point
41             gameStatus = Status.CONTINUE; // game is not over
42             myPoint = sumOfDice; // remember the point
43             System.out.printf("Point is %d\n", myPoint);
44             break;
45     }
46
47     // while game is not complete
48     while (gameStatus == Status.CONTINUE) // not WON or LOST
49     {
50         sumOfDice = rollDice(); // roll dice again
51
52         // determine game status
53         if (sumOfDice == myPoint) // win by making point
54             gameStatus = Status.WON;
55         else
56             if (sumOfDice == SEVEN) // lose by rolling 7 before point
57                 gameStatus = Status.LOST;
58     }
59 }
```

Fig. 6.8 | Craps class simulates the dice game craps. (Part 2 of 3.)

```

60     // display won or lost message
61     if (gameStatus == Status.WON)
62         System.out.println("Player wins");
63     else
64         System.out.println("Player loses");
65 }
66
67 // roll dice, calculate sum and display results
68 public static int rollDice()
69 {
70     // pick random die values
71     int die1 = 1 + randomNumbers.nextInt(6); // first die roll
72     int die2 = 1 + randomNumbers.nextInt(6); // second die roll
73
74     int sum = die1 + die2; // sum of die values
75
76     // display results of this roll
77     System.out.printf("Player rolled %d + %d = %d\n",
78         die1, die2, sum);
79
80     return sum;
81 }
82 } // end class Craps

```

Player rolled 5 + 6 = 11
Player wins

Player rolled 5 + 4 = 9
Point is 9
Player rolled 4 + 2 = 6
Player rolled 3 + 6 = 9
Player wins

Player rolled 1 + 2 = 3
Player loses

Player rolled 2 + 6 = 8
Point is 8
Player rolled 5 + 1 = 6
Player rolled 2 + 1 = 3
Player rolled 1 + 6 = 7
Player loses

Fig. 6.8 | Craps class simulates the dice game craps. (Part 3 of 3.)

Method rollDice

In the rules of the game, the player must roll *two* dice on the first and all subsequent rolls. We declare method `rollDice` (lines 68–81) to roll the dice and compute and print their sum. Method `rollDice` is declared once, but it's called from two places (lines 26 and 50) in `main`, which contains the logic for one complete game of craps. Method `rollDice` takes

no arguments, so it has an empty parameter list. Each time it's called, `rollDice` returns the sum of the dice, so the return type `int` is indicated in the method header (line 68). Although lines 71 and 72 look the same (except for the die names), they do not necessarily produce the same result. Each of these statements produces a *random* value in the range 1–6. Variable `randomNumbers` (used in lines 71–72) is *not* declared in the method. Instead it's declared as a `private static final` variable of the class and initialized in line 8. This enables us to create one `SecureRandom` object that's reused in each call to `rollDice`. If there were a program that contained multiple instances of class `Craps`, they'd all share this one `SecureRandom` object.

Method `main`'s Local Variables

The game is reasonably involved. The player may win or lose on the first roll, or may win or lose on any subsequent roll. Method `main` (lines 21–65) uses local variable `myPoint` (line 23) to store the “point” if the player doesn't win or lose on the first roll, local variable `gameStatus` (line 24) to keep track of the overall game status and local variable `sumOfDice` (line 26) to hold the sum of the dice for the most recent roll. Variable `myPoint` is initialized to 0 to ensure that the application will compile. If you do not initialize `myPoint`, the compiler issues an error, because `myPoint` is not assigned a value in *every* case of the `switch` statement, and thus the program could try to use `myPoint` before it's assigned a value. By contrast, `gameStatus` is assigned a value in *every* case of the `switch` statement (including the default case)—thus, it's guaranteed to be initialized before it's used, so we do not need to initialize it in line 24.

enum Type Status

Local variable `gameStatus` (line 24) is declared to be of a new type called `Status` (declared at line 11). Type `Status` is a private member of class `Craps`, because `Status` will be used only in that class. `Status` is a type called an **enum type**, which, in its simplest form, declares a set of constants represented by identifiers. An enum type is a special kind of class that's introduced by the keyword `enum` and a type name (in this case, `Status`). As with classes, braces delimit an enum declaration's body. Inside the braces is a comma-separated list of **enum constants**, each representing a unique value. The identifiers in an enum must be *unique*. You'll learn more about enum types in Chapter 8.



Good Programming Practice 6.1

Use only uppercase letters in the names of enum constants to make them stand out and remind you that they're not variables.

Variables of type `Status` can be assigned only the three constants declared in the enum (line 11) or a compilation error will occur. When the game is won, the program sets local variable `gameStatus` to `Status.WON` (lines 33 and 54). When the game is lost, the program sets local variable `gameStatus` to `Status.LOST` (lines 38 and 57). Otherwise, the program sets local variable `gameStatus` to `Status.CONTINUE` (line 41) to indicate that the game is not over and the dice must be rolled again.



Good Programming Practice 6.2

Using enum constants (like `Status.WON`, `Status.LOST` and `Status.CONTINUE`) rather than literal values (such as 0, 1 and 2) makes programs easier to read and maintain.

Logic of the main Method

Line 26 in `main` calls `rollDice`, which picks two random values from 1 to 6, displays the values of the first die, the second die and their sum, and returns the sum. Method `main` next enters the `switch` statement (lines 29–45), which uses the `sumOfDice` value from line 26 to determine whether the game has been won or lost, or should continue with another roll. The values that result in a win or loss on the first roll are declared as `private static final int` constants in lines 14–18. The identifier names use casino parlance for these sums. These constants, like `enum` constants, are declared by convention with all capital letters, to make them stand out in the program. Lines 31–34 determine whether the player won on the first roll with `SEVEN` (7) or `YO_LEVEN` (11). Lines 35–39 determine whether the player lost on the first roll with `SNAKE_EYES` (2), `TREY` (3), or `BOX_CARS` (12). After the first roll, if the game is not over, the default case (lines 40–44) sets `gameStatus` to `Status.CONTINUE`, saves `sumOfDice` in `myPoint` and displays the point.

If we’re still trying to “make our point” (i.e., the game is continuing from a prior roll), lines 48–58 execute. Line 50 rolls the dice again. If `sumOfDice` matches `myPoint` (line 53), line 54 sets `gameStatus` to `Status.WON`, then the loop terminates because the game is complete. If `sumOfDice` is `SEVEN` (line 56), line 57 sets `gameStatus` to `Status.LOST`, and the loop terminates because the game is complete. When the game completes, lines 61–64 display a message indicating whether the player won or lost, and the program terminates.

The program uses the various program-control mechanisms we’ve discussed. The `Craps` class uses two methods—`main` and `rollDice` (called twice from `main`)—and the `switch`, `while`, `if...else` and nested `if` control statements. Note also the use of multiple case labels in the `switch` statement to execute the same statements for sums of `SEVEN` and `YO_LEVEN` (lines 31–32) and for sums of `SNAKE_EYES`, `TREY` and `BOX_CARS` (lines 35–37).

Why Some Constants Are Not Defined as enum Constants

You might be wondering why we declared the sums of the dice as `private static final int` constants rather than as `enum` constants. The reason is that the program must compare the `int` variable `sumOfDice` (line 26) to these constants to determine the outcome of each roll. Suppose we declared `enum Sum` containing constants (e.g., `Sum.SNAKE_EYES`) representing the five sums used in the game, then used these constants in the `switch` statement (lines 29–45). Doing so would prevent us from using `sumOfDice` as the `switch` statement’s controlling expression, because Java does *not* allow an `int` to be compared to an `enum` constant. To achieve the same functionality as the current program, we would have to use a variable `currentSum` of type `Sum` as the `switch`’s controlling expression. Unfortunately, Java does not provide an easy way to convert an `int` value to a particular `enum` constant. This could be done with a separate `switch` statement. This would be cumbersome and would not improve the program’s readability (thus defeating the purpose of using an `enum`).

6.11 Scope of Declarations

You’ve seen declarations of various Java entities, such as classes, methods, variables and parameters. Declarations introduce names that can be used to refer to such Java entities. The **scope** of a declaration is the portion of the program that can refer to the declared entity by its name. Such an entity is said to be “in scope” for that portion of the program. This section introduces several important scope issues.

The basic scope rules are as follows:

1. The scope of a parameter declaration is the body of the method in which the declaration appears.
2. The scope of a local-variable declaration is from the point at which the declaration appears to the end of that block.
3. The scope of a local-variable declaration that appears in the initialization section of a `for` statement's header is the body of the `for` statement and the other expressions in the header.
4. A method or field's scope is the entire body of the class. This enables a class's instance methods to use the fields and other methods of the class.

Any block may contain variable declarations. If a local variable or parameter in a method has the same name as a field of the class, the field is *hidden* until the block terminates execution—this is called **shadowing**. To access a shadowed field in a block:

- If the field is an instance variable, precede its name with the `this` keyword and a dot (`.`), as in `this.x`.
- If the field is a static class variable, precede its name with the class's name and a dot (`.`), as in `ClassName.x`.

Figure 6.9 demonstrates scoping issues with fields and local variables. Line 7 declares and initializes the field `x` to 1. This field is *shadowed* in any block (or method) that declares a local variable named `x`. Method `main` (lines 11–23) declares a local variable `x` (line 13) and initializes it to 5. This local variable's value is output to show that the field `x` (whose value is 1) is *shadowed* in `main`. The program declares two other methods—`useLocalVariable` (lines 26–35) and `useField` (lines 38–45)—that each take no arguments and return no results. Method `main` calls each method twice (lines 17–20). Method `useLocalVariable` declares local variable `x` (line 28). When `useLocalVariable` is first called (line 17), it creates local variable `x` and initializes it to 25 (line 28), outputs the value of `x` (lines 30–31), increments `x` (line 32) and outputs the value of `x` again (lines 33–34). When `useLocalVariable` is called a second time (line 19), it *recreates* local variable `x` and *reinitializes* it to 25, so the output of each `useLocalVariable` call is identical.

```

1  // Fig. 6.9: Scope.java
2  // Scope class demonstrates field and local variable scopes.
3
4  public class Scope
5  {
6      // field that is accessible to all methods of this class
7      private static int x = 1;
8
9      // method main creates and initializes local variable x
10     // and calls methods useLocalVariable and useField
11     public static void main(String[] args)
12     {
13         int x = 5; // method's local variable x shadows field x
14     }

```

Fig. 6.9 | Scope class demonstrates field and local-variable scopes. (Part 1 of 2.)

```

15     System.out.printf("local x in main is %d\n", x);
16
17     useLocalVariable(); // useLocalVariable has local x
18     useField(); // useField uses class Scope's field x
19     useLocalVariable(); // useLocalVariable reinitializes local x
20     useField(); // class Scope's field x retains its value
21
22     System.out.printf("\nlocal x in main is %d\n", x);
23 }
24
25 // create and initialize local variable x during each call
26 public static void useLocalVariable()
27 {
28     int x = 25; // initialized each time useLocalVariable is called
29
30     System.out.printf(
31         "\nlocal x on entering method useLocalVariable is %d\n", x);
32     ++x; // modifies this method's local variable x
33     System.out.printf(
34         "local x before exiting method useLocalVariable is %d\n", x);
35 }
36
37 // modify class Scope's field x during each call
38 public static void useField()
39 {
40     System.out.printf(
41         "\nfield x on entering method useField is %d\n", x);
42     x *= 10; // modifies class Scope's field x
43     System.out.printf(
44         "field x before exiting method useField is %d\n", x);
45 }
46 } // end class Scope

```

```

local x in main is 5

local x on entering method useLocalVariable is 25
local x before exiting method useLocalVariable is 26

field x on entering method useField is 1
field x before exiting method useField is 10

local x on entering method useLocalVariable is 25
local x before exiting method useLocalVariable is 26

field x on entering method useField is 10
field x before exiting method useField is 100

local x in main is 5

```

Fig. 6.9 | Scope class demonstrates field and local-variable scopes. (Part 2 of 2.)

Method `useField` does not declare any local variables. Therefore, when it refers to `x`, field `x` (line 7) of the class is used. When method `useField` is first called (line 18), it outputs the value (1) of field `x` (lines 40–41), multiplies the field `x` by 10 (line 42) and outputs

the value (10) of field `x` again (lines 43–44) before returning. The next time method `useField` is called (line 20), the field has its modified value (10), so the method outputs 10, then 100. Finally, in method `main`, the program outputs the value of local variable `x` again (line 22) to show that none of the method calls modified `main`’s local variable `x`, because the methods all referred to variables named `x` in other scopes.

Principle of Least Privilege

In a general sense, “things” should have the capabilities they need to get their job done, but no more. An example is the scope of a variable. A variable should not be visible when it’s not needed.



Good Programming Practice 6.3

Declare variables as close to where they’re first used as possible.

6.12 Method Overloading

Methods of the *same* name can be declared in the same class, as long as they have *different* sets of parameters (determined by the number, types and order of the parameters)—this is called **method overloading**. When an overloaded method is called, the compiler selects the appropriate method by examining the number, types and order of the arguments in the call. Method overloading is commonly used to create several methods with the *same* name that perform the *same* or *similar* tasks, but on *different* types or *different* numbers of arguments. For example, `Math` methods `abs`, `min` and `max` (summarized in Section 6.3) are overloaded with four versions each:

1. One with two `double` parameters.
2. One with two `float` parameters.
3. One with two `int` parameters.
4. One with two `long` parameters.

Our next example demonstrates declaring and invoking overloaded methods. We demonstrate overloaded constructors in Chapter 8.

Declaring Overloaded Methods

Class `MethodOverload` (Fig. 6.10) includes two overloaded versions of method `square`—one that calculates the square of an `int` (and returns an `int`) and one that calculates the square of a `double` (and returns a `double`). Although these methods have the same name and similar parameter lists and bodies, think of them simply as *different* methods. It may help to think of the method names as “square of `int`” and “square of `double`,” respectively.

```
1 // Fig. 6.10: MethodOverload.java
2 // Overloaded method declarations.
3
4 public class MethodOverload
5 {
```

Fig. 6.10 | Overloaded method declarations. (Part 1 of 2.)

```

6 // test overloaded square methods
7 public static void main(String[] args)
8 {
9     System.out.printf("Square of integer 7 is %d\n", square(7));
10    System.out.printf("Square of double 7.5 is %f\n", square(7.5));
11 }
12
13 // square method with int argument
14 public static int square(int intValue)
15 {
16     System.out.printf("%nCalled square with int argument: %d\n",
17                       intValue);
18     return intValue * intValue;
19 }
20
21 // square method with double argument
22 public static double square(double doubleValue)
23 {
24     System.out.printf("%nCalled square with double argument: %f\n",
25                       doubleValue);
26     return doubleValue * doubleValue;
27 }
28 } // end class MethodOverload

```

```

Called square with int argument: 7
Square of integer 7 is 49

Called square with double argument: 7.500000
Square of double 7.5 is 56.250000

```

Fig. 6.10 | Overloaded method declarations. (Part 2 of 2.)

Line 9 invokes method `square` with the argument 7. Literal integer values are treated as type `int`, so the method call in line 9 invokes the version of `square` at lines 14–19 that specifies an `int` parameter. Similarly, line 10 invokes method `square` with the argument 7.5. Literal floating-point values are treated as type `double`, so the method call in line 10 invokes the version of `square` at lines 22–27 that specifies a `double` parameter. Each method first outputs a line of text to prove that the proper method was called in each case. The values in lines 10 and 24 are displayed with the format specifier `%f`. We did not specify a precision in either case. By default, floating-point values are displayed with six digits of precision if the precision is *not* specified in the format specifier.

Distinguishing Between Overloaded Methods

The compiler distinguishes overloaded methods by their **signatures**—a combination of the method's *name* and the *number*, *types* and *order* of its parameters, but *not* its return type. If the compiler looked only at method names during compilation, the code in Fig. 6.10 would be ambiguous—the compiler would not know how to distinguish between the two `square` methods (lines 14–19 and 22–27). Internally, the compiler uses longer method names that include the original method name, the types of each parameter and the exact order of the parameters to determine whether the methods in a class are *unique* in that class.

For example, in Fig. 6.10, the compiler might (internally) use the logical name “square of int” for the square method that specifies an `int` parameter and “square of double” for the square method that specifies a `double` parameter (the actual names the compiler uses are messier). If `method1`’s declaration begins as

```
void method1(int a, float b)
```

then the compiler might use the logical name “method1 of int and float.” If the parameters are specified as

```
void method1(float a, int b)
```

then the compiler might use the logical name “method1 of float and int.” The *order* of the parameter types is important—the compiler considers the preceding two `method1` headers to be *distinct*.

Return Types of Overloaded Methods

In discussing the logical names of methods used by the compiler, we did not mention the return types of the methods. *Method calls cannot be distinguished only by return type.* If you had overloaded methods that differed *only* by their return types and you called one of the methods in a standalone statement as in:

```
square(2);
```

the compiler would *not* be able to determine the version of the method to call, because the return value is *ignored*. When two methods have the *same* signature and *different* return types, the compiler issues an error message indicating that the method is already defined in the class. Overloaded methods *can* have *different* return types if the methods have *different* parameter lists. Also, overloaded methods need *not* have the same number of parameters.



Common Programming Error 6.8

Declaring overloaded methods with identical parameter lists is a compilation error regardless of whether the return types are different.

6.13 (Optional) GUI and Graphics Case Study: Colors and Filled Shapes

Although you can create many interesting designs with just lines and basic shapes, class `Graphics` provides many more capabilities. The next two features we introduce are colors and filled shapes. Adding color enriches the drawings a user sees on the computer screen. Shapes can be filled with solid colors.

Colors displayed on computer screens are defined by their *red*, *green*, and *blue* components (called **RGB values**) that have integer values from 0 to 255. The higher the value of a component color, the richer that color’s shade will be. Java uses class `Color` (package `java.awt`) to represent colors using their RGB values. For convenience, class `Color` contains various predefined static `Color` objects—`BLACK`, `BLUE`, `CYAN`, `DARK_GRAY`, `GRAY`, `GREEN`, `LIGHT_GRAY`, `MAGENTA`, `ORANGE`, `PINK`, `RED`, `WHITE` and `YELLOW`. Each can be accessed via the class name and a dot (`.`) as in `Color.RED`. You can create custom colors by passing the red-, green- and blue-component values to class `Color`’s constructor:

```
public Color(int r, int g, int b)
```

Graphics methods **fillRect** and **fillOval** draw filled rectangles and ovals, respectively. These have the same parameters as **drawRect** and **drawOval**; the first two are the coordinates for the *upper-left corner* of the shape, while the next two determine the *width* and *height*. The example in Figs. 6.11–6.12 demonstrates colors and filled shapes by drawing and displaying a yellow smiley face on the screen.

```

1  // Fig. 6.11: DrawSmiley.java
2  // Drawing a smiley face using colors and filled shapes.
3  import java.awt.Color;
4  import java.awt.Graphics;
5  import javax.swing.JPanel;
6
7  public class DrawSmiley extends JPanel
8  {
9      public void paintComponent(Graphics g)
10     {
11         super.paintComponent(g);
12
13         // draw the face
14         g.setColor(Color.YELLOW);
15         g.fillOval(10, 10, 200, 200);
16
17         // draw the eyes
18         g.setColor(Color.BLACK);
19         g.fillOval(55, 65, 30, 30);
20         g.fillOval(135, 65, 30, 30);
21
22         // draw the mouth
23         g.fillOval(50, 110, 120, 60);
24
25         // "touch up" the mouth into a smile
26         g.setColor(Color.YELLOW);
27         g.fillRect(50, 110, 120, 30);
28         g.fillOval(50, 120, 120, 40);
29     }
30 } // end class DrawSmiley

```

Fig. 6.11 | Drawing a smiley face using colors and filled shapes.

The import statements in lines 3–5 of Fig. 6.11 import classes **Color**, **Graphics** and **JPanel**. Class **DrawSmiley** (lines 7–30) uses class **Color** to specify drawing colors, and uses class **Graphics** to draw.

Class **JPanel** again provides the area in which we draw. Line 14 in method **paintComponent** uses **Graphics** method **setColor** to set the current drawing color to **Color.YELLOW**. Method **setColor** requires one argument, the **Color** to set as the drawing color. In this case, we use the predefined object **Color.YELLOW**.

Line 15 draws a circle with diameter 200 to represent the face—when the width and height arguments are identical, method **fillOval** draws a circle. Next, line 18 sets the color to **Color.Black**, and lines 19–20 draw the eyes. Line 23 draws the mouth as an oval, but this is not quite what we want.

To create a happy face, we'll touch up the mouth. Line 26 sets the color to `Color.YELLOW`, so any shapes we draw will blend in with the face. Line 27 draws a rectangle that's half the mouth's height. This erases the top half of the mouth, leaving just the bottom half. To create a better smile, line 28 draws another oval to slightly cover the upper portion of the mouth. Class `DrawSmileyTest` (Fig. 6.12) creates and displays a `JFrame` containing the drawing. When the `JFrame` is displayed, the system calls method `paintComponent` to draw the smiley face.

```

1  // Fig. 6.12: DrawSmileyTest.java
2  // Test application that displays a smiley face.
3  import javax.swing.JFrame;
4
5  public class DrawSmileyTest
6  {
7      public static void main(String[] args)
8      {
9          DrawSmiley panel = new DrawSmiley();
10         JFrame application = new JFrame();
11
12         application.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
13         application.add(panel);
14         application.setSize(230, 250);
15         application.setVisible(true);
16     }
17 } // end class DrawSmileyTest

```

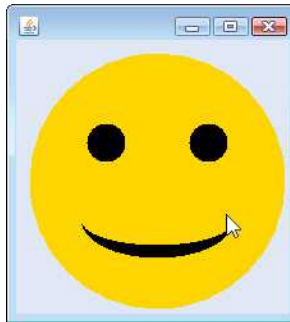


Fig. 6.12 | Test application that displays a smiley face.

GUI and Graphics Case Study Exercises

6.1 Using method `fillOval`, draw a bull's-eye that alternates between two random colors, as in Fig. 6.13. Use the constructor `Color(int r, int g, int b)` with random arguments to generate random colors.

6.2 Create a program that draws 10 random filled shapes in random colors, positions and sizes (Fig. 6.14). Method `paintComponent` should contain a loop that iterates 10 times. In each iteration, the loop should determine whether to draw a filled rectangle or an oval, create a random color and choose coordinates and dimensions at random. The coordinates should be chosen based on the panel's width and height. Lengths of sides should be limited to half the width or height of the window.

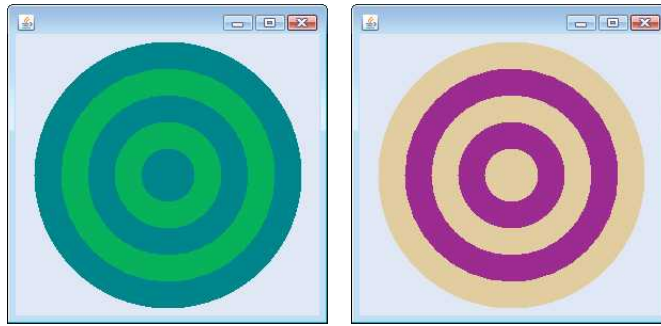


Fig. 6.13 | A bulls-eye with two alternating, random colors.

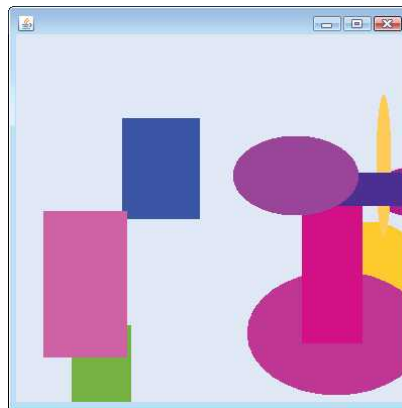


Fig. 6.14 | Randomly generated shapes.

6.14 Wrap-Up

In this chapter, you learned more about method declarations. You also learned the difference between instance methods and static methods and how to call static methods by preceding the method name with the name of the class in which it appears and the dot (.) separator. You learned how to use operators `+` and `+=` to perform string concatenations. We discussed how the method-call stack and stack frames keep track of the methods that have been called and where each method must return to when it completes its task. We also discussed Java's promotion rules for converting implicitly between primitive types and how to perform explicit conversions with cast operators. Next, you learned about some of the commonly used packages in the Java API.

You saw how to declare named constants using both `enum` types and `private static final` variables. You used class `SecureRandom` to generate random numbers for simulations. You also learned about the scope of fields and local variables in a class. Finally, you learned that multiple methods in one class can be overloaded by providing methods with the same name and different signatures. Such methods can be used to perform the same or similar tasks using different types or different numbers of parameters.

In Chapter 7, you'll learn how to maintain lists and tables of data in arrays. You'll see a more elegant implementation of the application that rolls a die 6,000,000 times. We'll present two versions of a GradeBook case study that stores sets of student grades in a GradeBook object. You'll also learn how to access an application's command-line arguments that are passed to method `main` when an application begins execution.

Summary

Section 6.1 Introduction

- Experience has shown that the best way to develop and maintain a large program is to construct it from small, simple pieces, or modules. This technique is called divide and conquer (p. 201).

Section 6.2 Program Modules in Java

- Methods are declared within classes. Classes are typically grouped into packages so they can be imported and reused.
- Methods allow you to modularize a program by separating its tasks into self-contained units. The statements in a method are written only once and hidden from other methods.
- Using existing methods as building blocks to create new programs is a form of software reusability (p. 202) that allows you to avoid repeating code within a program.

Section 6.3 static Methods, static Fields and Class Math

- A method call specifies the name of the method to call and provides the arguments that the called method requires to perform its task. When the method call completes, the method returns either a result, or simply control, to its caller.
- A class may contain static methods to perform common tasks that do not require an object of the class. Any data a static method might require to perform its tasks can be sent to the method as arguments in a method call. A static method is called by specifying the name of the class in which the method is declared followed by a dot (.) and the method name, as in

ClassName.methodName(arguments)

- Class `Math` provides static methods for performing common mathematical calculations.
- The constant `Math.PI` (p. 204; 3.141592653589793) is the ratio of a circle's circumference to its diameter. The constant `Math.E` (p. 204; 2.718281828459045) is the base value for natural logarithms (calculated with static `Math` method `log`).
- `Math.PI` and `Math.E` are declared with the modifiers `public`, `final` and `static`. Making them `public` allows you to use these fields in your own classes. A field declared with keyword `final` (p. 205) is constant—its value cannot be changed after it's initialized. Both `PI` and `E` are declared `final` because their values never change. Making these fields `static` allows them to be accessed via the class name `Math` and a dot (.) separator, just like class `Math`'s methods.
- All the objects of a class share one copy of the class's static fields. Together the class variables (p. 204) and instance variables represent the fields of a class.
- When you execute the Java Virtual Machine (JVM) with the `java` command, the JVM loads the class you specify and uses that class name to invoke method `main`. You can specify additional command-line arguments (p. 205) that the JVM will pass to your application.
- You can place a `main` method in every class you declare—only the `main` method in the class you use to execute the application will be called by the `java` command.

Section 6.4 Declaring Methods with Multiple Parameters

- When a method is called, the program makes a copy of the method's argument values and assigns them to the method's corresponding parameters. When program control returns to the point in the program where the method was called, the method's parameters are removed from memory.
- A method can return at most one value, but the returned value could be a reference to an object that contains many values.
- Variables should be declared as fields of a class only if they're required for use in more than one method of the class or if the program should save their values between calls to the class's methods.
- When a method has more than one parameter, the parameters are specified as a comma-separated list. There must be one argument in the method call for each parameter in the method declaration. Also, each argument must be consistent with the type of the corresponding parameter. If a method does not accept arguments, the parameter list is empty.
- Strings can be concatenated (p. 208) using operator `+`, which places the characters of the right operand at the end of those in the left operand.
- Every primitive value and object in Java can be represented as a `String`. When an object is concatenated with a `String`, the object is converted to a `String`, then the two `Strings` are concatenated.
- If a `boolean` is concatenated with a `String`, the word `"true"` or the word `"false"` is used to represent the `boolean` value.
- All objects in Java have a special method named `toString` that returns a `String` representation of the object's contents. When an object is concatenated with a `String`, the JVM implicitly calls the object's `toString` method to obtain the string representation of the object.
- You can break large `String` literals into several smaller `Strings` and place them on multiple lines of code for readability, then reassemble the `Strings` using concatenation.

Section 6.5 Notes on Declaring and Using Methods

- There are three ways to call a method—using a method name by itself to call another method of the same class; using a variable that contains a reference to an object, followed by a dot (`.`) and the method name to call a method of the referenced object; and using the class name and a dot (`.`) to call a static method of a class.
- There are three ways to return control to a statement that calls a method. If the method does not return a result, control returns when the program flow reaches the method-ending right brace or when the statement

```
return;
```

is executed. If the method returns a result, the statement

```
return expression;
```

evaluates the *expression*, then immediately returns the resulting value to the caller.

Section 6.6 Method-Call Stack and Stack Frames

- Stacks (p. 209) are known as last-in, first-out (LIFO) data structures—the last item pushed (inserted) onto the stack is the first item popped (removed) from the stack.
- A called method must know how to return to its caller, so the return address of the calling method is pushed onto the method-call stack when the method is called. If a series of method calls occurs, the successive return addresses are pushed onto the stack in last-in, first-out order so that the last method to execute will be the first to return to its caller.
- The method-call stack (p. 210) contains the memory for the local variables used in each invocation of a method during a program's execution. This data is known as the method call's stack

frame or activation record. When a method call is made, the stack frame for that method call is pushed onto the method-call stack. When the method returns to its caller, its stack frame call is popped off the stack and the local variables are no longer known to the program.

- If there are more method calls than can have their stack frames stored on the method-call stack, an error known as a stack overflow (p. 210) occurs. The application will compile correctly, but its execution causes a stack overflow.

Section 6.7 Argument Promotion and Casting

- Argument promotion (p. 210) converts an argument's value to the type that the method expects to receive in its corresponding parameter.
- Promotion rules (p. 210) apply to expressions containing values of two or more primitive types and to primitive-type values passed as arguments to methods. Each value is promoted to the "highest" type in the expression. In cases where information may be lost due to conversion, the Java compiler requires you to use a cast operator to explicitly force the conversion to occur.

Section 6.9 Case Study: Secure Random-Number Generation

- Objects of class `SecureRandom` (package `java.security`; p. 213) can produce nondeterministic random values.
- `SecureRandom` method `nextInt` (p. 214) generates a random `int` value.
- Class `SecureRandom` provides another version of method `nextInt` that receives an `int` argument and returns a value from 0 up to, but not including, the argument's value.
- Random numbers in a range (p. 214) can be generated with

```
int number = shiftingValue + randomNumbers.nextInt(scalingFactor);
```

where *shiftingValue* specifies the first number in the desired range of consecutive integers, and *scalingFactor* specifies how many numbers are in the range.

- Random numbers can be chosen from nonconsecutive integer ranges, as in

```
int number = shiftingValue +
    differenceBetweenValues * randomNumbers.nextInt(scalingFactor);
```

where *shiftingValue* specifies the first number in the range of values, *differenceBetweenValues* represents the difference between consecutive numbers in the sequence and *scalingFactor* specifies how many numbers are in the range.

Section 6.10 Case Study: A Game of Chance; Introducing enum Types

- An `enum` type (p. 221) is introduced by the keyword `enum` and a type name. As with any class, braces (`{` and `}`) delimit the body of an `enum` declaration. Inside the braces is a comma-separated list of `enum` constants, each representing a unique value. The identifiers in an `enum` must be unique. Variables of an `enum` type can be assigned only constants of that `enum` type.
- Constants can also be declared as `private static final` variables. Such constants are declared by convention with all capital letters to make them stand out in the program.

Section 6.11 Scope of Declarations

- Scope (p. 222) is the portion of the program in which an entity, such as a variable or a method, can be referred to by its name. Such an entity is said to be "in scope" for that portion of the program.
- The scope of a parameter declaration is the body of the method in which the declaration appears.
- The scope of a local-variable declaration is from the point at which the declaration appears to the end of that block.

- The scope of a local-variable declaration that appears in the initialization section of a `for` statement's header is the body of the `for` statement and the other expressions in the header.
- The scope of a method or field of a class is the entire body of the class. This enables a class's methods to use simple names to call the class's other methods and to access the class's fields.
- Any block may contain variable declarations. If a local variable or parameter in a method has the same name as a field, the field is shadowed (p. 223) until the block terminates execution.

Section 6.12 Method Overloading

- Java allows overloaded methods (p. 225) in a class, as long as the methods have different sets of parameters (determined by the number, order and types of the parameters).
- Overloaded methods are distinguished by their signatures (p. 226)—combinations of the methods' names and the number, types and order of their parameters, but not their return types.

Self-Review Exercises

- 6.1** Fill in the blanks in each of the following statements:
- A method is invoked with a(n) _____.
 - A variable known only within the method in which it's declared is called a(n) _____.
 - The _____ statement in a called method can be used to pass the value of an expression back to the calling method.
 - The keyword _____ indicates that a method does not return a value.
 - Data can be added or removed only from the _____ of a stack.
 - Stacks are known as _____ data structures; the last item pushed (inserted) onto the stack is the first item popped (removed) from the stack.
 - The three ways to return control from a called method to a caller are _____, _____ and _____.
 - An object of class _____ produces truly random numbers.
 - The method-call stack contains the memory for local variables on each invocation of a method during a program's execution. This data, stored as a portion of the method-call stack, is known as the _____ or _____ of the method call.
 - If there are more method calls than can be stored on the method-call stack, an error known as a(n) _____ occurs.
 - The _____ of a declaration is the portion of a program that can refer to the entity in the declaration by name.
 - It's possible to have several methods with the same name that each operate on different types or numbers of arguments. This feature is called method _____.
- 6.2** For the class `Craps` in Fig. 6.8, state the scope of each of the following entities:
- the variable `randomNumbers`.
 - the variable `die1`.
 - the method `rollDice`.
 - the method `main`.
 - the variable `sumOfDice`.
- 6.3** Write an application that tests whether the examples of the `Math` class method calls shown in Fig. 6.2 actually produce the indicated results.
- 6.4** Give the method header for each of the following methods:
- Method `hypotenuse`, which takes two double-precision, floating-point arguments `side1` and `side2` and returns a double-precision, floating-point result.
 - Method `smallest`, which takes three integers `x`, `y` and `z` and returns an integer.

- c) Method instructions, which does not take any arguments and does not return a value. [Note: Such methods are commonly used to display instructions to a user.]
- d) Method `intToFloat`, which takes integer argument `number` and returns a `float`.

6.5 Find the error in each of the following program segments. Explain how to correct the error.

- a)

```
void g()
{
    System.out.println("Inside method g");

    void h()
    {
        System.out.println("Inside method h");
    }
}
```
- b)

```
int sum(int x, int y)
{
    int result;
    result = x + y;
}
```
- c)

```
void f(float a);
{
    float a;
    System.out.println(a);
}
```
- d)

```
void product()
{
    int a = 6, b = 5, c = 4, result;
    result = a * b * c;
    System.out.printf("Result is %d\n", result);
    return result;
}
```

6.6 Declare method `sphereVolume` to calculate and return the volume of the sphere. Use the following statement to calculate the volume:

```
double volume = (4.0 / 3.0) * Math.PI * Math.pow(radius, 3)
```

Write a Java application that prompts the user for the `double` radius of a sphere, calls `sphereVolume` to calculate the volume and displays the result.

Answers to Self-Review Exercises

6.1 a) method call. b) local variable. c) return. d) void. e) top. f) last-in, first-out (LIFO). g) return; or return *expression*; or encountering the closing right brace of a method. h) `SecureRandom`. i) stack frame, activation record. j) stack overflow. k) scope. l) method overloading.

6.2 a) class body. b) block that defines method `rollDice`'s body. c) class body. d) class body. e) block that defines method `main`'s body.

6.3 The following solution demonstrates the `Math` class methods in Fig. 6.2:

```
1 // Exercise 6.3: MathTest.java
2 // Testing the Math class methods.
3 public class MathTest
4 {
5     public static void main(String[] args)
6     {
```

```

7      System.out.printf("Math.abs(23.7) = %f\n", Math.abs(23.7));
8      System.out.printf("Math.abs(0.0) = %f\n", Math.abs(0.0));
9      System.out.printf("Math.abs(-23.7) = %f\n", Math.abs(-23.7));
10     System.out.printf("Math.ceil(9.2) = %f\n", Math.ceil(9.2));
11     System.out.printf("Math.ceil(-9.8) = %f\n", Math.ceil(-9.8));
12     System.out.printf("Math.cos(0.0) = %f\n", Math.cos(0.0));
13     System.out.printf("Math.exp(1.0) = %f\n", Math.exp(1.0));
14     System.out.printf("Math.exp(2.0) = %f\n", Math.exp(2.0));
15     System.out.printf("Math.floor(9.2) = %f\n", Math.floor(9.2));
16     System.out.printf("Math.floor(-9.8) = %f\n", Math.floor(-9.8));
17     System.out.printf("Math.log(Math.E) = %f\n", Math.log(Math.E));
18     System.out.printf("Math.log(Math.E * Math.E) = %f\n",
19         Math.log(Math.E * Math.E));
20     System.out.printf("Math.max(2.3, 12.7) = %f\n", Math.max(2.3, 12.7));
21     System.out.printf("Math.max(-2.3, -12.7) = %f\n",
22         Math.max(-2.3, -12.7));
23     System.out.printf("Math.min(2.3, 12.7) = %f\n", Math.min(2.3, 12.7));
24     System.out.printf("Math.min(-2.3, -12.7) = %f\n",
25         Math.min(-2.3, -12.7));
26     System.out.printf("Math.pow(2.0, 7.0) = %f\n", Math.pow(2.0, 7.0));
27     System.out.printf("Math.pow(9.0, 0.5) = %f\n", Math.pow(9.0, 0.5));
28     System.out.printf("Math.sin(0.0) = %f\n", Math.sin(0.0));
29     System.out.printf("Math.sqrt(900.0) = %f\n", Math.sqrt(900.0));
30     System.out.printf("Math.tan(0.0) = %f\n", Math.tan(0.0));
31 } // end main
32 } // end class MathTest

```

```

Math.abs(23.7) = 23.700000
Math.abs(0.0) = 0.000000
Math.abs(-23.7) = 23.700000
Math.ceil(9.2) = 10.000000
Math.ceil(-9.8) = -9.000000
Math.cos(0.0) = 1.000000
Math.exp(1.0) = 2.718282
Math.exp(2.0) = 7.389056
Math.floor(9.2) = 9.000000
Math.floor(-9.8) = -10.000000
Math.log(Math.E) = 1.000000
Math.log(Math.E * Math.E) = 2.000000
Math.max(2.3, 12.7) = 12.700000
Math.max(-2.3, -12.7) = -2.300000
Math.min(2.3, 12.7) = 2.300000
Math.min(-2.3, -12.7) = -12.700000
Math.pow(2.0, 7.0) = 128.000000
Math.pow(9.0, 0.5) = 3.000000
Math.sin(0.0) = 0.000000
Math.sqrt(900.0) = 30.000000
Math.tan(0.0) = 0.000000

```

- 6.4 a) `double` hypotenuse(`double` side1, `double` side2)
b) `int` smallest(`int` x, `int` y, `int` z)
c) `void` instructions()
d) `float` intToFloat(`int` number)

- 6.5 a) Error: Method h is declared within method g.
Correction: Move the declaration of h outside the declaration of g.
b) Error: The method is supposed to return an integer, but does not.
Correction: Delete the variable `result`, and place the statement
`return x + y;`
in the method, or add the following statement at the end of the method body:
`return result;`

- c) Error: The semicolon after the right parenthesis of the parameter list is incorrect, and the parameter `a` should not be redeclared in the method.
Correction: Delete the semicolon after the right parenthesis of the parameter list, and delete the declaration `float a`;
- d) Error: The method returns a value when it's not supposed to.
Correction: Change the return type from `void` to `int`.

6.6 The following solution calculates the volume of a sphere, using the radius entered by the user:

```

1  // Exercise 6.6: Sphere.java
2  // Calculate the volume of a sphere.
3  import java.util.Scanner;
4
5  public class Sphere
6  {
7      // obtain radius from user and display volume of sphere
8      public static void main(String[] args)
9      {
10         Scanner input = new Scanner(System.in);
11
12         System.out.print("Enter radius of sphere: ");
13         double radius = input.nextDouble();
14
15         System.out.printf("Volume is %f\n", sphereVolume(radius));
16     } // end method determineSphereVolume
17
18     // calculate and return sphere volume
19     public static double sphereVolume(double radius)
20     {
21         double volume = (4.0 / 3.0) * Math.PI * Math.pow(radius, 3);
22         return volume;
23     } // end method sphereVolume
24 } // end class Sphere

```

```

Enter radius of sphere: 4
Volume is 268.082573

```

Exercises

6.7 What is the value of `x` after each of the following statements is executed?

- `x = Math.abs(7.5);`
- `x = Math.floor(7.5);`
- `x = Math.abs(0.0);`
- `x = Math.ceil(0.0);`
- `x = Math.abs(-6.4);`
- `x = Math.ceil(-6.4);`
- `x = Math.ceil(-Math.abs(-8 + Math.floor(-5.5)));`

6.8 (*Parking Charges*) A parking garage charges a \$2.00 minimum fee to park for up to three hours. The garage charges an additional \$0.50 per hour for each hour *or part thereof* in excess of three hours. The maximum charge for any given 24-hour period is \$10.00. Assume that no car parks for longer than 24 hours at a time. Write an application that calculates and displays the parking charges for each customer who parked in the garage yesterday. You should enter the hours parked for each customer. The program should display the charge for the current customer and should calculate and display the running total of yesterday's receipts. It should use the method `calculateCharges` to determine the charge for each customer.

6.9 (*Rounding Numbers*) `Math.floor` can be used to round values to the nearest integer—e.g.,

```
y = Math.floor(x + 0.5);
```

will round the number `x` to the nearest integer and assign the result to `y`. Write an application that reads `double` values and uses the preceding statement to round each of the numbers to the nearest integer. For each number processed, display both the original number and the rounded number.

6.10 (*Rounding Numbers*) To round numbers to specific decimal places, use a statement like

```
y = Math.floor(x * 10 + 0.5) / 10;
```

which rounds `x` to the tenths position (i.e., the first position to the right of the decimal point), or

```
y = Math.floor(x * 100 + 0.5) / 100;
```

which rounds `x` to the hundredths position (i.e., the second position to the right of the decimal point). Write an application that defines four methods for rounding a number `x` in various ways:

- a) `roundToInteger(number)`
- b) `roundToTenths(number)`
- c) `roundToHundredths(number)`
- d) `roundToThousandths(number)`

For each value read, your program should display the original value, the number rounded to the nearest integer, the number rounded to the nearest tenth, the number rounded to the nearest hundredth and the number rounded to the nearest thousandth.

6.11 Answer each of the following questions:

- a) What does it mean to choose numbers “at random”?
- b) Why is the `nextInt` method of class `SecureRandom` useful for simulating games of chance?
- c) Why is it often necessary to scale or shift the values produced by a `SecureRandom` object?
- d) Why is computerized simulation of real-world situations a useful technique?

6.12 Write statements that assign random integers to the variable `n` in the following ranges:

- a) $1 \leq n \leq 2$.
- b) $1 \leq n \leq 100$.
- c) $0 \leq n \leq 9$.
- d) $1000 \leq n \leq 1112$.
- e) $-1 \leq n \leq 1$.
- f) $-3 \leq n \leq 11$.

6.13 Write statements that will display a random number from each of the following sets:

- a) 2, 4, 6, 8, 10.
- b) 3, 5, 7, 9, 11.
- c) 6, 10, 14, 18, 22.

6.14 (*Exponentiation*) Write a method `integerPower(base, exponent)` that returns the value of $\text{base}^{\text{exponent}}$

For example, `integerPower(3, 4)` calculates 3^4 (or $3 * 3 * 3 * 3$). Assume that `exponent` is a positive, nonzero integer and that `base` is an integer. Use a `for` or `while` statement to control the calculation. Do not use any `Math` class methods. Incorporate this method into an application that reads integer values for `base` and `exponent` and performs the calculation with the `integerPower` method.

6.15 (*Hypotenuse Calculations*) Define a method `hypotenuse` that calculates the hypotenuse of a right triangle when the lengths of the other two sides are given. The method should take two arguments of type `double` and return the hypotenuse as a `double`. Incorporate this method into an

application that reads values for `side1` and `side2` and performs the calculation with the `hypotenuse` method. Use `Math` methods `pow` and `sqrt` to determine the length of the hypotenuse for each of the triangles in Fig. 6.15. [Note: Class `Math` also provides method `hypot` to perform this calculation.]

Triangle	Side 1	Side 2
1	3.0	4.0
2	5.0	12.0
3	8.0	15.0

Fig. 6.15 | Values for the sides of triangles in Exercise 6.15.

6.16 (*Multiples*) Write a method `isMultiple` that determines, for a pair of integers, whether the second integer is a multiple of the first. The method should take two integer arguments and return `true` if the second is a multiple of the first and `false` otherwise. [Hint: Use the remainder operator.] Incorporate this method into an application that inputs a series of pairs of integers (one pair at a time) and determines whether the second value in each pair is a multiple of the first.

6.17 (*Even or Odd*) Write a method `isEven` that uses the remainder operator (%) to determine whether an integer is even. The method should take an integer argument and return `true` if the integer is even and `false` otherwise. Incorporate this method into an application that inputs a sequence of integers (one at a time) and determines whether each is even or odd.

6.18 (*Displaying a Square of Asterisks*) Write a method `squareOfAsterisks` that displays a solid square (the same number of rows and columns) of asterisks whose side is specified in integer parameter side. For example, if side is 4, the method should display

```
****
****
****
****
```

Incorporate this method into an application that reads an integer value for `side` from the user and outputs the asterisks with the `squareOfAsterisks` method.

6.19 (*Displaying a Square of Any Character*) Modify the method created in Exercise 6.18 to receive a second parameter of type `char` called `fillCharacter`. Form the square using the `char` provided as an argument. Thus, if side is 5 and `fillCharacter` is `#`, the method should display

```
#####
#####
#####
#####
#####
```

Use the following statement (in which `input` is a `Scanner` object) to read a character from the user at the keyboard:

```
char fill = input.next().charAt(0);
```

6.20 (*Circle Area*) Write an application that prompts the user for the radius of a circle and uses a method called `circleArea` to calculate the area of the circle.

6.21 (*Separating Digits*) Write methods that accomplish each of the following tasks:

- Calculate the integer part of the quotient when integer `a` is divided by integer `b`.
- Calculate the integer remainder when integer `a` is divided by integer `b`.

- c) Use the methods developed in parts (a) and (b) to write a method `displayDigits` that receives an integer between 1 and 99999 and displays it as a sequence of digits, separating each pair of digits by two spaces. For example, the integer 4562 should appear as

4 5 6 2

Incorporate the methods into an application that inputs an integer and calls `displayDigits` by passing the method the integer entered. Display the results.

6.22 (*Temperature Conversions*) Implement the following integer methods:

- a) Method `celsius` returns the Celsius equivalent of a Fahrenheit temperature, using the calculation

`celsius = 5.0 / 9.0 * (fahrenheit - 32);`

- b) Method `fahrenheit` returns the Fahrenheit equivalent of a Celsius temperature, using the calculation

`fahrenheit = 9.0 / 5.0 * celsius + 32;`

- c) Use the methods from parts (a) and (b) to write an application that enables the user either to enter a Fahrenheit temperature and display the Celsius equivalent or to enter a Celsius temperature and display the Fahrenheit equivalent.

6.23 (*Find the Minimum*) Write a method `minimum3` that returns the smallest of three floating-point numbers. Use the `Math.min` method to implement `minimum3`. Incorporate the method into an application that reads three values from the user, determines the smallest value and displays the result.

6.24 (*Perfect Numbers*) An integer number is said to be a *perfect number* if its factors, including 1 (but not the number itself), sum to the number. For example, 6 is a perfect number, because $6 = 1 + 2 + 3$. Write a method `isPerfect` that determines whether parameter `number` is a perfect number. Use this method in an application that displays all the perfect numbers between 1 and 1000. Display the factors of each perfect number to confirm that the number is indeed perfect. Challenge the computing power of your computer by testing numbers much larger than 1000. Display the results.

6.25 (*Prime Numbers*) A positive integer is *prime* if it's divisible by only 1 and itself. For example, 2, 3, 5 and 7 are prime, but 4, 6, 8 and 9 are not. The number 1, by definition, is not prime.

- Write a method that determines whether a number is prime.
- Use this method in an application that determines and displays all the prime numbers less than 10,000. How many numbers up to 10,000 do you have to test to ensure that you've found all the primes?
- Initially, you might think that $n/2$ is the upper limit for which you must test to see whether a number n is prime, but you need only go as high as the square root of n . Rewrite the program, and run it both ways.

6.26 (*Reversing Digits*) Write a method that takes an integer value and returns the number with its digits reversed. For example, given the number 7631, the method should return 1367. Incorporate the method into an application that reads a value from the user and displays the result.

6.27 (*Greatest Common Divisor*) The *greatest common divisor (GCD)* of two integers is the largest integer that evenly divides each of the two numbers. Write a method `gcd` that returns the greatest common divisor of two integers. [Hint: You might want to use Euclid's algorithm. You can find information about it at en.wikipedia.org/wiki/Euclidean_algorithm.] Incorporate the method into an application that reads two values from the user and displays the result.

6.28 Write a method `qualityPoints` that inputs a student's average and returns 4 if it's 90–100, 3 if 80–89, 2 if 70–79, 1 if 60–69 and 0 if lower than 60. Incorporate the method into an application that reads a value from the user and displays the result.

6.29 (Coin Tossing) Write an application that simulates coin tossing. Let the program toss a coin each time the user chooses the “Toss Coin” menu option. Count the number of times each side of the coin appears. Display the results. The program should call a separate method `flip` that takes no arguments and returns a value from a `Coin` enum (`HEADS` and `TAILS`). [Note: If the program realistically simulates coin tossing, each side of the coin should appear approximately half the time.]

6.30 (Guess the Number) Write an application that plays “guess the number” as follows: Your program chooses the number to be guessed by selecting a random integer in the range 1 to 1000. The application displays the prompt `Guess a number between 1 and 1000`. The player inputs a first guess. If the player's guess is incorrect, your program should display `Too high. Try again.` or `Too low. Try again.` to help the player “zero in” on the correct answer. The program should prompt the user for the next guess. When the user enters the correct answer, display `Congratulations. You guessed the number!`, and allow the user to choose whether to play again. [Note: The guessing technique employed in this problem is similar to a binary search, which is discussed in Chapter 19, Searching, Sorting and Big O.]

6.31 (Guess the Number Modification) Modify the program of Exercise 6.30 to count the number of guesses the player makes. If the number is 10 or fewer, display `Either you know the secret or you got lucky!` If the player guesses the number in 10 tries, display `Aha! You know the secret!` If the player makes more than 10 guesses, display `You should be able to do better!` Why should it take no more than 10 guesses? Well, with each “good guess,” the player should be able to eliminate half of the numbers, then half of the remaining numbers, and so on.

6.32 (Distance Between Points) Write method `distance` to calculate the distance between two points $(x1, y1)$ and $(x2, y2)$. All numbers and return values should be of type `double`. Incorporate this method into an application that enables the user to enter the coordinates of the points.

6.33 (Craps Game Modification) Modify the craps program of Fig. 6.8 to allow wagering. Initialize variable `bankBalance` to 1000 dollars. Prompt the player to enter a wager. Check that `wager` is less than or equal to `bankBalance`, and if it's not, have the user reenter `wager` until a valid `wager` is entered. Then, run one game of craps. If the player wins, increase `bankBalance` by `wager` and display the new `bankBalance`. If the player loses, decrease `bankBalance` by `wager`, display the new `bankBalance`, check whether `bankBalance` has become zero and, if so, display the message `Sorry. You busted!` As the game progresses, display various messages to create some “chatter,” such as `Oh, you're going for broke, huh?` or `Aw c'mon, take a chance!` or `You're up big. Now's the time to cash in your chips!`. Implement the “chatter” as a separate method that randomly chooses the string to display.

6.34 (Table of Binary, Octal and Hexadecimal Numbers) Write an application that displays a table of the binary, octal and hexadecimal equivalents of the decimal numbers in the range 1 through 256. If you aren't familiar with these number systems, read online Appendix J first.

Making a Difference

As computer costs decline, it becomes feasible for every student, regardless of economic circumstance, to have a computer and use it in school. This creates exciting possibilities for improving the educational experience of all students worldwide, as suggested by the next five exercises. [Note: Check out initiatives such as the One Laptop Per Child Project (www.laptop.org). Also, research “green” laptops—what are some key “going green” characteristics of these devices? Look into the Electronic Product Environmental Assessment Tool (www.epeat.net), which can help you assess the “greenness” of desktops, notebooks and monitors to help you decide which products to purchase.]

6.35 (Computer-Assisted Instruction) The use of computers in education is referred to as *computer-assisted instruction (CAI)*. Write a program that will help an elementary school student learn

multiplication. Use a `SecureRandom` object to produce two positive one-digit integers. The program should then prompt the user with a question, such as

How much is 6 times 7?

The student then inputs the answer. Next, the program checks the student's answer. If it's correct, display the message "Very good!" and ask another multiplication question. If the answer is wrong, display the message "No. Please try again." and let the student try the same question repeatedly until the student finally gets it right. A separate method should be used to generate each new question. This method should be called once when the application begins execution and each time the user answers the question correctly.

6.36 (*Computer-Assisted Instruction: Reducing Student Fatigue*) One problem in CAI environments is student fatigue. This can be reduced by varying the computer's responses to hold the student's attention. Modify the program of Exercise 6.35 so that various comments are displayed for each answer as follows:

Possible responses to a correct answer:

Very good!
Excellent!
Nice work!
Keep up the good work!

Possible responses to an incorrect answer:

No. Please try again.
Wrong. Try once more.
Don't give up!
No. Keep trying.

Use random-number generation to choose a number from 1 to 4 that will be used to select one of the four appropriate responses to each correct or incorrect answer. Use a `switch` statement to issue the responses.

6.37 (*Computer-Assisted Instruction: Monitoring Student Performance*) More sophisticated computer-assisted instruction systems monitor the student's performance over a period of time. The decision to begin a new topic is often based on the student's success with previous topics. Modify the program of Exercise 6.36 to count the number of correct and incorrect responses typed by the student. After the student types 10 answers, your program should calculate the percentage that are correct. If the percentage is lower than 75%, display "Please ask your teacher for extra help.", then reset the program so another student can try it. If the percentage is 75% or higher, display "Congratulations, you are ready to go to the next level!", then reset the program so another student can try it.

6.38 (*Computer-Assisted Instruction: Difficulty Levels*) Exercises 6.35–6.37 developed a computer-assisted instruction program to help teach an elementary school student multiplication. Modify the program to allow the user to enter a difficulty level. At a difficulty level of 1, the program should use only single-digit numbers in the problems; at a difficulty level of 2, numbers as large as two digits, and so on.

6.39 (*Computer-Assisted Instruction: Varying the Types of Problems*) Modify the program of Exercise 6.38 to allow the user to pick a type of arithmetic problem to study. An option of 1 means addition problems only, 2 means subtraction problems only, 3 means multiplication problems only, 4 means division problems only and 5 means a random mixture of all these types.

Arrays and ArrayLists

7



Begin at the beginning, ... and go on till you come to the end: then stop.

—Lewis Carroll

To go beyond is as wrong as to fall short.

—Confucius

Objectives

In this chapter you'll:

- Learn what arrays are.
- Use arrays to store data in and retrieve data from lists and tables of values.
- Declare arrays, initialize arrays and refer to individual elements of arrays.
- Iterate through arrays with the enhanced `for` statement.
- Pass arrays to methods.
- Declare and manipulate multidimensional arrays.
- Use variable-length argument lists.
- Read command-line arguments into a program.
- Build an object-oriented instructor gradebook class.
- Perform common array manipulations with the methods of class `Arrays`.
- Use class `ArrayList` to manipulate a dynamically resizable arraylike data structure.

- 7.1 Introduction
- 7.2 Arrays
- 7.3 Declaring and Creating Arrays
- 7.4 Examples Using Arrays
 - 7.4.1 Creating and Initializing an Array
 - 7.4.2 Using an Array Initializer
 - 7.4.3 Calculating the Values to Store in an Array
 - 7.4.4 Summing the Elements of an Array
 - 7.4.5 Using Bar Charts to Display Array Data Graphically
 - 7.4.6 Using the Elements of an Array as Counters
 - 7.4.7 Using Arrays to Analyze Survey Results
- 7.5 Exception Handling: Processing the Incorrect Response
 - 7.5.1 The try Statement
 - 7.5.2 Executing the catch Block
 - 7.5.3 toString Method of the Exception Parameter
- 7.6 Case Study: Card Shuffling and Dealing Simulation
- 7.7 Enhanced for Statement
- 7.8 Passing Arrays to Methods
- 7.9 Pass-By-Value vs. Pass-By-Reference
- 7.10 Case Study: Class GradeBook Using an Array to Store Grades
- 7.11 Multidimensional Arrays
- 7.12 Case Study: Class GradeBook Using a Two-Dimensional Array
- 7.13 Variable-Length Argument Lists
- 7.14 Using Command-Line Arguments
- 7.15 Class Arrays
- 7.16 Introduction to Collections and Class ArrayList
- 7.17 (Optional) GUI and Graphics Case Study: Drawing Arcs
- 7.18 Wrap-Up

Summary | Self-Review Exercises | Answers to Self-Review Exercises | Exercises |
 Special Section: Building Your Own Computer | Making a Difference

7.1 Introduction

This chapter introduces **data structures**—collections of related data items. **Array** objects are data structures consisting of related data items of the same type. Arrays make it convenient to process related groups of values. Arrays remain the same length once they're created. We study data structures in depth in Chapters 16–21.

After discussing how arrays are declared, created and initialized, we present practical examples that demonstrate common array manipulations. We introduce Java's *exception-handling* mechanism and use it to allow a program to continue executing when it attempts to access an array element that does not exist. We also present a case study that examines how arrays can help simulate the shuffling and dealing of playing cards in a card-game application. We introduce Java's *enhanced for statement*, which allows a program to access the data in an array more easily than does the counter-controlled for statement presented in Section 5.3. We build two versions of an instructor GradeBook case study that use arrays to maintain sets of student grades *in memory* and analyze student grades. We show how to use *variable-length argument lists* to create methods that can be called with varying numbers of arguments, and we demonstrate how to process *command-line arguments* in method main. Next, we present some common array manipulations with static methods of class Arrays from the java.util package.

Although commonly used, arrays have limited capabilities. For instance, you must specify an array's size, and if at execution time you wish to modify it, you must do so by creating a new array. At the end of this chapter, we introduce one of Java's prebuilt data

structures from the Java API's *collection classes*. These offer greater capabilities than traditional arrays. They're reusable, reliable, powerful and efficient. We focus on the `ArrayList` collection. `ArrayLists` are similar to arrays but provide additional functionality, such as **dynamic resizing** as necessary to accommodate more or fewer elements.

Java SE 8

After reading Chapter 17, Java SE 8 Lambdas and Streams, you'll be able to reimplement many of Chapter 7's examples in a more concise and elegant manner, and in a way that makes them easier to parallelize to improve performance on today's multi-core systems.

7.2 Arrays

An array is a group of variables (called **elements** or **components**) containing values that all have the *same* type. Arrays are *objects*, so they're considered *reference types*. As you'll soon see, what we typically think of as an array is actually a *reference* to an array object in memory. The *elements* of an array can be either *primitive types* or *reference types* (including arrays, as we'll see in Section 7.11). To refer to a particular element in an array, we specify the *name* of the reference to the array and the *position number* of the element in the array. The position number of the element is called the element's **index** or **subscript**.

Logical Array Representation

Figure 7.1 shows a logical representation of an integer array called `c`. This array contains 12 *elements*. A program refers to any one of these elements with an **array-access expression** that includes the *name* of the array followed by the *index* of the particular element in **square brackets** (`[]`). The first element in every array has **index zero** and is sometimes called the **zeroth element**. Thus, the elements of array `c` are `c[0]`, `c[1]`, `c[2]` and so on. The highest index in array `c` is 11, which is 1 less than 12—the number of elements in the array. Array names follow the same conventions as other variable names.

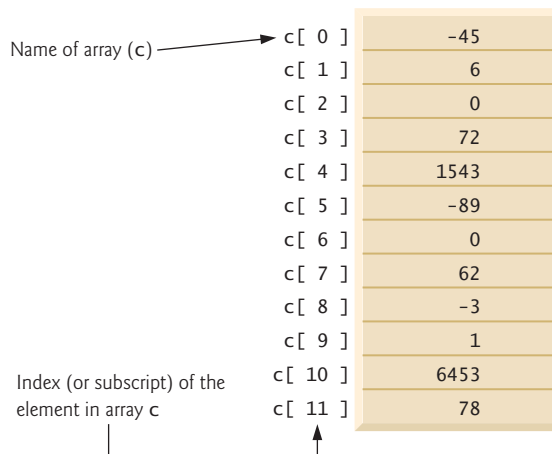


Fig. 7.1 | A 12-element array.

An index must be a *nonnegative integer*. A program can use an expression as an index. For example, if we assume that variable *a* is 5 and variable *b* is 6, then the statement

```
c[a + b] += 2;
```

adds 2 to array element *c*[11]. An indexed array name is an *array-access expression*, which can be used on the left side of an assignment to place a new value into an array element.



Common Programming Error 7.1

An index must be an int value or a value of a type that can be promoted to int—namely, byte, short or char, but not long; otherwise, a compilation error occurs.

Let's examine array *c* in Fig. 7.1 more closely. The **name** of the array is *c*. Every array object knows its own length and stores it in a **length instance variable**. The expression *c.length* returns array *c*'s length. Even though the *length* instance variable of an array is *public*, it cannot be changed because it's a *final* variable. This array's 12 elements are referred to as *c*[0], *c*[1], *c*[2], ..., *c*[11]. The value of *c*[0] is -45, the value of *c*[1] is 6, the value of *c*[2] is 0, the value of *c*[7] is 62 and the value of *c*[11] is 78. To calculate the sum of the values contained in the first three elements of array *c* and store the result in variable *sum*, we would write

```
sum = c[0] + c[1] + c[2];
```

To divide the value of *c*[6] by 2 and assign the result to the variable *x*, we would write

```
x = c[6] / 2;
```

7.3 Declaring and Creating Arrays

Array objects occupy space in memory. Like other objects, arrays are created with keyword *new*. To create an array object, you specify the type of the array elements and the number of elements as part of an **array-creation expression** that uses keyword *new*. Such an expression returns a *reference* that can be stored in an array variable. The following declaration and array-creation expression create an array object containing 12 *int* elements and store the array's reference in the array variable named *c*:

```
int[] c = new int[12];
```

This expression can be used to create the array in Fig. 7.1. When an array is created, each of its elements receives a default value—zero for the numeric primitive-type elements, *false* for *boolean* elements and *null* for references. As you'll soon see, you can provide nondefault element values when you create an array.

Creating the array in Fig. 7.1 can also be performed in two steps as follows:

```
int[] c; // declare the array variable
c = new int[12]; // create the array; assign to array variable
```

In the declaration, the *square brackets* following the type indicate that *c* is a variable that will refer to an array (i.e., the variable will store an array *reference*). In the assignment statement, the array variable *c* receives the reference to a new array of 12 *int* elements.



Common Programming Error 7.2

In an array declaration, specifying the number of elements in the square brackets of the declaration (e.g., `int[12] c;`) is a syntax error.

A program can create several arrays in a single declaration. The following declaration reserves 100 elements for `b` and 27 elements for `x`:

```
String[] b = new String[100], x = new String[27];
```

When the type of the array and the square brackets are combined at the beginning of the declaration, all the identifiers in the declaration are array variables. In this case, variables `b` and `x` refer to `String` arrays. For readability, we prefer to declare only *one* variable per declaration. The preceding declaration is equivalent to:

```
String[] b = new String[100]; // create array b
String[] x = new String[27]; // create array x
```



Good Programming Practice 7.1

For readability, declare only one variable per declaration. Keep each declaration on a separate line, and include a comment describing the variable being declared.

When only one variable is declared in each declaration, the square brackets can be placed either after the type or after the array variable name, as in:

```
String b[] = new String[100]; // create array b
String x[] = new String[27]; // create array x
```

but placing the square brackets after the type is preferred.



Common Programming Error 7.3

Declaring multiple array variables in a single declaration can lead to subtle errors. Consider the declaration `int[] a, b, c;`. If `a`, `b` and `c` should be declared as array variables, then this declaration is correct—placing square brackets directly following the type indicates that all the identifiers in the declaration are array variables. However, if only `a` is intended to be an array variable, and `b` and `c` are intended to be individual `int` variables, then this declaration is incorrect—the declaration `int a[], b, c;` would achieve the desired result.

A program can declare arrays of any type. Every element of a primitive-type array contains a value of the array's declared element type. Similarly, in an array of a reference type, every element is a reference to an object of the array's declared element type. For example, every element of an `int` array is an `int` value, and every element of a `String` array is a reference to a `String` object.

7.4 Examples Using Arrays

This section presents several examples that demonstrate declaring arrays, creating arrays, initializing arrays and manipulating array elements.

7.4.1 Creating and Initializing an Array

The application of Fig. 7.2 uses keyword `new` to create an array of 10 `int` elements, which are initially zero (the default initial value for `int` variables). Line 9 declares `array`—a reference capable of referring to an array of `int` elements—then initializes the variable with a reference to an array object containing 10 `int` elements. Line 11 outputs the column headings. The first column contains the index (0–9) of each array element, and the second column contains the default initial value (0) of each array element.

```

1 // Fig. 7.2: InitArray.java
2 // Initializing the elements of an array to default values of zero.
3
4 public class InitArray
5 {
6     public static void main(String[] args)
7     {
8         // declare variable array and initialize it with an array object
9         int[] array = new int[10]; // create the array object
10
11         System.out.printf("%s%s\n", "Index", "Value"); // column headings
12
13         // output each array element's value
14         for (int counter = 0; counter < array.length; counter++)
15             System.out.printf("%5d%8d\n", counter, array[counter]);
16     }
17 } // end class InitArray

```

Index	Value
0	0
1	0
2	0
3	0
4	0
5	0
6	0
7	0
8	0
9	0

Fig. 7.2 | Initializing the elements of an array to default values of zero.

The `for` statement (lines 14–15) outputs the index (represented by `counter`) and value of each array element (represented by `array[counter]`). Control variable `counter` is initially 0—index values start at 0, so using **zero-based counting** allows the loop to access every element of the array. The `for`'s loop-continuation condition uses the expression `array.length` (line 14) to determine the length of the array. In this example, the length of the array is 10, so the loop continues executing as long as the value of control variable `counter` is less than 10. The highest index value of a 10-element array is 9, so using the less-than operator in the loop-continuation condition guarantees that the loop does not attempt to access an element *beyond* the end of the array (i.e., during the final iteration of the loop, `counter` is 9). We'll soon see what Java does when it encounters such an *out-of-range index* at execution time.

7.4.2 Using an Array Initializer

You can create an array and initialize its elements with an **array initializer**—a comma-separated list of expressions (called an **initializer list**) enclosed in braces. In this case, the array length is determined by the number of elements in the initializer list. For example,

```
int[] n = { 10, 20, 30, 40, 50 };
```

creates a *five*-element array with index values 0–4. Element `n[0]` is initialized to 10, `n[1]` is initialized to 20, and so on. When the compiler encounters an array declaration that in-

cludes an initializer list, it *counts* the number of initializers in the list to determine the size of the array, then sets up the appropriate new operation “behind the scenes.”

The application in Fig. 7.3 initializes an integer array with 10 values (line 9) and displays the array in tabular format. The code for displaying the array elements (lines 14–15) is identical to that in Fig. 7.2 (lines 15–16).

```

1  // Fig. 7.3: InitArray.java
2  // Initializing the elements of an array with an array initializer.
3
4  public class InitArray
5  {
6      public static void main(String[] args)
7      {
8          // initializer list specifies the initial value for each element
9          int[] array = { 32, 27, 64, 18, 95, 14, 90, 70, 60, 37 };
10
11         System.out.printf("%s%s\n", "Index", "Value"); // column headings
12
13         // output each array element's value
14         for (int counter = 0; counter < array.length; counter++)
15             System.out.printf("%5d%8d\n", counter, array[counter]);
16
17     } // end class InitArray

```

Index	Value
0	32
1	27
2	64
3	18
4	95
5	14
6	90
7	70
8	60
9	37

Fig. 7.3 | Initializing the elements of an array with an array initializer.

7.4.3 Calculating the Values to Store in an Array

The application in Fig. 7.4 creates a 10-element array and assigns to each element one of the even integers from 2 to 20 (2, 4, 6, ..., 20). Then the application displays the array in tabular format. The `for` statement at lines 12–13 calculates an array element’s value by multiplying the current value of the control variable `counter` by 2, then adding 2.

```

1  // Fig. 7.4: InitArray.java
2  // Calculating the values to be placed into the elements of an array.
3
4  public class InitArray
5  {

```

Fig. 7.4 | Calculating the values to be placed into the elements of an array. (Part I of 2.)

```

6   public static void main(String[] args)
7   {
8       final int ARRAY_LENGTH = 10; // declare constant
9       int[] array = new int[ARRAY_LENGTH]; // create array
10
11      // calculate value for each array element
12      for (int counter = 0; counter < array.length; counter++)
13          array[counter] = 2 + 2 * counter;
14
15      System.out.printf("%s%s\n", "Index", "Value"); // column headings
16
17      // output each array element's value
18      for (int counter = 0; counter < array.length; counter++)
19          System.out.printf("%5d%8d\n", counter, array[counter]);
20  }
21 } // end class InitArray

```

Index	Value
0	2
1	4
2	6
3	8
4	10
5	12
6	14
7	16
8	18
9	20

Fig. 7.4 | Calculating the values to be placed into the elements of an array. (Part 2 of 2.)

Line 8 uses the modifier `final` to declare the constant variable `ARRAY_LENGTH` with the value 10. Constant variables must be initialized *before* they're used and *cannot* be modified thereafter. If you attempt to *modify* a `final` variable after it's initialized in its declaration, the compiler issues an error message like

cannot assign a value to final variable *variableName*



Good Programming Practice 7.2

Constant variables also are called *named constants*. They often make programs more readable than programs that use literal values (e.g., 10)—a named constant such as `ARRAY_LENGTH` clearly indicates its purpose, whereas a literal value could have different meanings based on its context.



Good Programming Practice 7.3

Multiword named constants should have each word separated from the next with an underscore (`_`) as in `ARRAY_LENGTH`.



Common Programming Error 7.4

Assigning a value to a `final` variable after it has been initialized is a compilation error. Similarly, attempting to access the value of a `final` variable before it's initialized results in a compilation error like, “variable *variableName* might not have been initialized.”

7.4.4 Summing the Elements of an Array

Often, the elements of an array represent a series of values to be used in a calculation. If, for example, they represent exam grades, a professor may wish to total the elements of the array and use that sum to calculate the class average for the exam. The GradeBook examples in Figs. 7.14 and 7.18 use this technique.

Figure 7.5 sums the values contained in a 10-element integer array. The program declares, creates and initializes the array at line 8. The `for` statement performs the calculations. [Note: The values supplied as array initializers are often read into a program rather than specified in an initializer list. For example, an application could input the values from a user or from a file on disk (as discussed in Chapter 15, Files, Streams and Object Serialization). Reading the data into a program (rather than “hand coding” it into the program) makes the program more reusable, because it can be used with *different* sets of data.]

```

1  // Fig. 7.5: SumArray.java
2  // Computing the sum of the elements of an array.
3
4  public class SumArray
5  {
6      public static void main(String[] args)
7      {
8          int[] array = { 87, 68, 94, 100, 83, 78, 85, 91, 76, 87 };
9          int total = 0;
10
11          // add each element's value to total
12          for (int counter = 0; counter < array.length; counter++)
13              total += array[counter];
14
15          System.out.printf("Total of array elements: %d\n", total);
16      }
17  } // end class SumArray

```

Total of array elements: 849

Fig. 7.5 | Computing the sum of the elements of an array.

7.4.5 Using Bar Charts to Display Array Data Graphically

Many programs present data to users in a graphical manner. For example, numeric values are often displayed as bars in a bar chart. In such a chart, longer bars represent proportionally larger numeric values. One simple way to display numeric data graphically is with a bar chart that shows each numeric value as a bar of asterisks (*).

Professors often like to examine the distribution of grades on an exam. A professor might graph the number of grades in each of several categories to visualize the grade distribution. Suppose the grades on an exam were 87, 68, 94, 100, 83, 78, 85, 91, 76 and 87. They include one grade of 100, two grades in the 90s, four grades in the 80s, two grades in the 70s, one grade in the 60s and no grades below 60. Our next application (Fig. 7.6) stores this grade distribution data in an array of 11 elements, each corresponding to a category of grades. For example, `array[0]` indicates the number of grades in the range 0–9, `array[7]` the number of grades in the range 70–79 and `array[10]` the number of 100

grades. The GradeBook classes later in the chapter (Figs. 7.14 and 7.18) contain code that calculates these grade frequencies based on a set of grades. For now, we manually create the array with the given grade frequencies.

```

1  // Fig. 7.6: BarChart.java
2  // Bar chart printing program.
3
4  public class BarChart
5  {
6      public static void main(String[] args)
7      {
8          int[] array = { 0, 0, 0, 0, 0, 0, 1, 2, 4, 2, 1 };
9
10         System.out.println("Grade distribution:");
11
12         // for each array element, output a bar of the chart
13         for (int counter = 0; counter < array.length; counter++)
14         {
15             // output bar label ("00-09: ", ..., "90-99: ", "100: ")
16             if (counter == 10)
17                 System.out.printf("%5d: ", 100);
18             else
19                 System.out.printf("%02d-%02d: ",
20                                     counter * 10, counter * 10 + 9);
21
22             // print bar of asterisks
23             for (int stars = 0; stars < array[counter]; stars++)
24                 System.out.print("*");
25
26             System.out.println();
27         }
28     }
29 } // end class BarChart

```

```

Grade distribution:
00-09:
10-19:
20-29:
30-39:
40-49:
50-59:
60-69: *
70-79: **
80-89: ****
90-99: **
100: *

```

Fig. 7.6 | Bar chart printing program.

The application reads the numbers from the array and graphs the information as a bar chart. It displays each grade range followed by a bar of asterisks indicating the number of grades in that range. To label each bar, lines 16–20 output a grade range (e.g., "70–79: ") based on the current value of counter. When counter is 10, line 17 outputs 100 with a

field width of 5, followed by a colon and a space, to align the label "100: " with the other bar labels. The nested `for` statement (lines 23–24) outputs the bars. Note the loop-continuation condition at line 23 (`stars < array[counter]`). Each time the program reaches the inner `for`, the loop counts from 0 up to `array[counter]`, thus using a value in `array` to determine the number of asterisks to display. In this example, *no* students received a grade below 60, so `array[0]–array[5]` contain zeroes, and *no* asterisks are displayed next to the first six grade ranges. In line 19, the format specifier `%02d` indicates that an `int` value should be formatted as a field of two digits. The **0 flag** in the format specifier displays a leading 0 for values with fewer digits than the field width (2).

7.4.6 Using the Elements of an Array as Counters

Sometimes, programs use counter variables to summarize data, such as the results of a survey. In Fig. 6.7, we used separate counters in our die-rolling program to track the number of occurrences of each side of a six-sided die as the program rolled the die 6,000,000 times. An array version of this application is shown in Fig. 7.7.

```

1  // Fig. 7.7: RollDie.java
2  // Die-rolling program using arrays instead of switch.
3  import java.security.SecureRandom;
4
5  public class RollDie
6  {
7      public static void main(String[] args)
8      {
9          SecureRandom randomNumbers = new SecureRandom();
10         int[] frequency = new int[7]; // array of frequency counters
11
12         // roll die 6,000,000 times; use die value as frequency index
13         for (int roll = 1; roll <= 6000000; roll++)
14             ++frequency[1 + randomNumbers.nextInt(6)];
15
16         System.out.printf("%s%10s%n", "Face", "Frequency");
17
18         // output each array element's value
19         for (int face = 1; face < frequency.length; face++)
20             System.out.printf("%4d%10d%n", face, frequency[face]);
21     }
22 } // end class RollDie

```

Face	Frequency
1	999690
2	999512
3	1000575
4	999815
5	999781
6	1000627

Fig. 7.7 | Die-rolling program using arrays instead of `switch`.

Figure 7.7 uses the array `frequency` (line 10) to count the occurrences of each side of the die. *The single statement in line 14 of this program replaces lines 22–45 of Fig. 6.7.* Line

14 uses the random value to determine which frequency element to increment during each iteration of the loop. The calculation in line 14 produces random numbers from 1 to 6, so the array frequency must be large enough to store six counters. However, we use a seven-element array in which we ignore frequency[0]—it’s more logical to have the face value 1 increment frequency[1] than frequency[0]. Thus, each face value is used as an index for array frequency. In line 14, the calculation inside the square brackets evaluates first to determine which element of the array to increment, then the ++ operator adds one to that element. We also replaced lines 49–51 from Fig. 6.7 by looping through array frequency to output the results (lines 19–20). When we study Java SE 8’s new functional programming capabilities in Chapter 17, we’ll show how to replace lines 13–14 and 19–20 with a *single* statement!

7.4.7 Using Arrays to Analyze Survey Results

Our next example uses arrays to summarize data collected in a survey. Consider the following problem statement:

Twenty students were asked to rate on a scale of 1 to 5 the quality of the food in the student cafeteria, with 1 being “awful” and 5 being “excellent.” Place the 20 responses in an integer array and determine the frequency of each rating.

This is a typical array-processing application (Fig. 7.8). We wish to summarize the number of responses of each type (that is, 1–5). Array responses (lines 9–10) is a 20-element integer array containing the students’ survey responses. The last value in the array *is intentionally* an incorrect response (14). When a Java program executes, array element indices are checked for validity—all indices must be greater than or equal to 0 and less than the length of the array. Any attempt to access an element outside that range of indices results in a runtime error that’s known as an `ArrayIndexOutOfBoundsException`. At the end of this section, we’ll discuss the invalid response value, demonstrate array **bounds checking** and introduce Java’s *exception-handling* mechanism, which can be used to detect and handle an `ArrayIndexOutOfBoundsException`.

```

1  // Fig. 7.8: StudentPoll.java
2  // Poll analysis program.
3
4  public class StudentPoll
5  {
6      public static void main(String[] args)
7      {
8          // student response array (more typically, input at runtime)
9          int[] responses = { 1, 2, 5, 4, 3, 5, 2, 1, 3, 3, 1, 4, 3, 3, 3,
10                           2, 3, 3, 2, 14 };
11          int[] frequency = new int[6]; // array of frequency counters
12
13          // for each answer, select responses element and use that value
14          // as frequency index to determine element to increment
15          for (int answer = 0; answer < responses.length; answer++)
16          {

```

Fig. 7.8 | Poll analysis program. (Part I of 2.)

```

17     try
18     {
19         ++frequency[responses[answer]];
20     }
21     catch (ArrayIndexOutOfBoundsException e)
22     {
23         System.out.println(e); // invokes toString method
24         System.out.printf("    responses[%d] = %d\n\n",
25             answer, responses[answer]);
26     }
27 }
28
29 System.out.printf("%s%10s\n", "Rating", "Frequency");
30
31 // output each array element's value
32 for (int rating = 1; rating < frequency.length; rating++)
33     System.out.printf("%6d%10d\n", rating, frequency[rating]);
34 }
35 } // end class StudentPoll

```

```

java.lang.ArrayIndexOutOfBoundsException: 14
    responses[19] = 14

```

Rating	Frequency
1	3
2	4
3	8
4	2
5	2

Fig. 7.8 | Poll analysis program. (Part 2 of 2.)

The frequency Array

We use the *six-element* array `frequency` (line 11) to count the number of occurrences of each response. Each element is used as a *counter* for one of the possible types of survey responses—`frequency[1]` counts the number of students who rated the food as 1, `frequency[2]` counts the number of students who rated the food as 2, and so on.

Summarizing the Results

The `for` statement (lines 15–27) reads the responses from the array `responses` one at a time and increments one of the counters `frequency[1]` to `frequency[5]`; we ignore `frequency[0]` because the survey responses are limited to the range 1–5. The key statement in the loop appears in line 19. This statement increments the appropriate frequency counter as determined by the value of `responses[answer]`.

Let's step through the first few iterations of the `for` statement:

- When the counter `answer` is 0, `responses[answer]` is the value of `responses[0]` (that is, 1—see line 9). In this case, `frequency[responses[answer]]` is interpreted as `frequency[1]`, and the counter `frequency[1]` is incremented by one. To evaluate the expression, we begin with the value in the *innermost* set of brackets (`answer`, currently 0). The value of `answer` is plugged into the expression, and the

next set of brackets (`responses[answer]`) is evaluated. That value is used as the index for the `frequency` array to determine which counter to increment (in this case, `frequency[1]`).

- The next time through the loop `answer` is 1, `responses[answer]` is the value of `responses[1]` (that is, 2—see line 9), so `frequency[responses[answer]]` is interpreted as `frequency[2]`, causing `frequency[2]` to be incremented.
- When `answer` is 2, `responses[answer]` is the value of `responses[2]` (that is, 5—see line 9), so `frequency[responses[answer]]` is interpreted as `frequency[5]`, causing `frequency[5]` to be incremented, and so on.

Regardless of the number of responses processed, only a six-element array (in which we ignore element zero) is required to summarize the results, because all the correct response values are values from 1 to 5, and the index values for a six-element array are 0–5. In the program’s output, the `Frequency` column summarizes only 19 of the 20 values in the `responses` array—the last element of the array `responses` contains an (intentionally) incorrect response that was not counted. Section 7.5 discusses what happens when the program in Fig. 7.8 encounters the invalid response (14) in the last element of array `responses`.

7.5 Exception Handling: Processing the Incorrect Response

An **exception** indicates a problem that occurs while a program executes. The name “exception” suggests that the problem occurs infrequently—if the “rule” is that a statement normally executes correctly, then the problem represents the “exception to the rule.” **Exception handling** helps you create **fault-tolerant programs** that can resolve (or handle) exceptions. In many cases, this allows a program to continue executing as if no problems were encountered. For example, the `StudentPoll` application still displays results (Fig. 7.8), even though one of the responses was out of range. More severe problems might prevent a program from continuing normal execution, instead requiring the program to notify the user of the problem, then terminate. When the JVM or a method detects a problem, such as an invalid array index or an invalid method argument, it **throws** an exception—that is, an exception occurs. Methods in your own classes can also throw exceptions, as you’ll learn in Chapter 8.

7.5.1 The try Statement

To handle an exception, place any code that might throw an exception in a **try statement** (lines 17–26). The **try block** (lines 17–20) contains the code that might *throw* an exception, and the **catch block** (lines 21–26) contains the code that *handles* the exception if one occurs. You can have *many* catch blocks to handle different *types* of exceptions that might be thrown in the corresponding try block. When line 19 correctly increments an element of the `frequency` array, lines 21–26 are ignored. The braces that delimit the bodies of the try and catch blocks are required.

7.5.2 Executing the catch Block

When the program encounters the invalid value 14 in the `responses` array, it attempts to add 1 to `frequency[14]`, which is *outside* the bounds of the array—the `frequency` array

has only six elements (with indexes 0–5). Because array bounds checking is performed at execution time, the JVM generates an *exception*—specifically line 19 throws an **ArrayIndexOutOfBoundsException** to notify the program of this problem. At this point the try block terminates and the catch block begins executing—if you declared any local variables in the try block, they’re now *out of scope* (and no longer exist), so they’re not accessible in the catch block.

The catch block declares an exception parameter (e) of type (IndexOutOfRangeException). The catch block can handle exceptions of the specified type. Inside the catch block, you can use the parameter’s identifier to interact with a caught exception object.



Error-Prevention Tip 7.1

When writing code to access an array element, ensure that the array index remains greater than or equal to 0 and less than the length of the array. This would prevent ArrayIndexOutOfBoundsExceptions if your program is correct.



Software Engineering Observation 7.1

Systems in industry that have undergone extensive testing are still likely to contain bugs. Our preference for industrial-strength systems is to catch and deal with runtime exceptions, such as ArrayIndexOutOfBoundsExceptions, to ensure that a system either stays up and running or degrades gracefully, and to inform the system’s developers of the problem.

7.5.3 toString Method of the Exception Parameter

When lines 21–26 *catch* the exception, the program displays a message indicating the problem that occurred. Line 23 *implicitly* calls the exception object’s `toString` method to get the error message that’s implicitly stored in the exception object and display it. Once the message is displayed in this example, the exception is considered *handled* and the program continues with the next statement after the catch block’s closing brace. In this example, the end of the for statement is reached (line 27), so the program continues with the increment of the control variable in line 15. We discuss exception handling again in Chapter 8, and more deeply in Chapter 11.

7.6 Case Study: Card Shuffling and Dealing Simulation

The examples in the chapter thus far have used arrays containing elements of primitive types. Recall from Section 7.2 that the elements of an array can be either primitive types or reference types. This section uses random-number generation and an array of reference-type elements, namely objects representing playing cards, to develop a class that simulates card shuffling and dealing. This class can then be used to implement applications that play specific card games. The exercises at the end of the chapter use the classes developed here to build a simple poker application.

We first develop class `Card` (Fig. 7.9), which represents a playing card that has a face (e.g., "Ace", "Deuce", "Three", ..., "Jack", "Queen", "King") and a suit (e.g., "Hearts", "Diamonds", "Clubs", "Spades"). Next, we develop the `DeckOfCards` class (Fig. 7.10), which creates a deck of 52 playing cards in which each element is a `Card` object. We then build a test application (Fig. 7.11) that demonstrates class `DeckOfCards`’s card shuffling and dealing capabilities.

Class Card

Class Card (Fig. 7.9) contains two String instance variables—face and suit—that are used to store references to the face name and suit name for a specific Card. The constructor for the class (lines 10–14) receives two Strings that it uses to initialize face and suit. Method toString (lines 17–20) creates a String consisting of the face of the card, the String " of " and the suit of the card. Card's toString method can be invoked *explicitly* to obtain a string representation of a Card object (e.g., "Ace of Spades"). The toString method of an object is called *implicitly* when the object is used where a String is expected (e.g., when printf outputs the object as a String using the %s format specifier or when the object is concatenated to a String using the + operator). For this behavior to occur, toString must be declared with the header shown in Fig. 7.9.

```

1  // Fig. 7.9: Card.java
2  // Card class represents a playing card.
3
4  public class Card
5  {
6      private final String face; // face of card ("Ace", "Deuce", ...)
7      private final String suit; // suit of card ("Hearts", "Diamonds", ...)
8
9      // two-argument constructor initializes card's face and suit
10     public Card(String cardFace, String cardSuit)
11     {
12         this.face = cardFace; // initialize face of card
13         this.suit = cardSuit; // initialize suit of card
14     }
15
16     // return String representation of Card
17     public String toString()
18     {
19         return face + " of " + suit;
20     }
21 } // end class Card

```

Fig. 7.9 | Card class represents a playing card.

Class DeckOfCards

Class DeckOfCards (Fig. 7.10) declares as an instance variable a Card array named deck (line 7). An array of a *reference* type is declared like any other array. Class DeckOfCards also declares an integer instance variable currentCard (line 8) representing the sequence number (0–51) of the next Card to be dealt from the deck array, and a named constant NUMBER_OF_CARDS (line 9) indicating the number of Cards in the deck (52).

```

1  // Fig. 7.10: DeckOfCards.java
2  // DeckOfCards class represents a deck of playing cards.
3  import java.security.SecureRandom;
4

```

Fig. 7.10 | DeckOfCards class represents a deck of playing cards. (Part I of 2.)

```

5 public class DeckOfCards
6 {
7     private Card[] deck; // array of Card objects
8     private int currentCard; // index of next Card to be dealt (0-51)
9     private static final int NUMBER_OF_CARDS = 52; // constant # of Cards
10    // random number generator
11    private static final SecureRandom randomNumbers = new SecureRandom();
12
13    // constructor fills deck of Cards
14    public DeckOfCards()
15    {
16        String[] faces = { "Ace", "Deuce", "Three", "Four", "Five", "Six",
17                           "Seven", "Eight", "Nine", "Ten", "Jack", "Queen", "King" };
18        String[] suits = { "Hearts", "Diamonds", "Clubs", "Spades" };
19
20        deck = new Card[NUMBER_OF_CARDS]; // create array of Card objects
21        currentCard = 0; // first Card dealt will be deck[0]
22
23        // populate deck with Card objects
24        for (int count = 0; count < deck.length; count++)
25            deck[count] =
26                new Card(faces[count % 13], suits[count / 13]);
27    }
28
29    // shuffle deck of Cards with one-pass algorithm
30    public void shuffle()
31    {
32        // next call to method dealCard should start at deck[0] again
33        currentCard = 0;
34
35        // for each Card, pick another random Card (0-51) and swap them
36        for (int first = 0; first < deck.length; first++)
37        {
38            // select a random number between 0 and 51
39            int second = randomNumbers.nextInt(NUMBER_OF_CARDS);
40
41            // swap current Card with randomly selected Card
42            Card temp = deck[first];
43            deck[first] = deck[second];
44            deck[second] = temp;
45        }
46    }
47
48    // deal one Card
49    public Card dealCard()
50    {
51        // determine whether Cards remain to be dealt
52        if (currentCard < deck.length)
53            return deck[currentCard++]; // return current Card in array
54        else
55            return null; // return null to indicate that all Cards were dealt
56    }
57 } // end class DeckOfCards

```

Fig. 7.10 | DeckOfCards class represents a deck of playing cards. (Part 2 of 2.)

DeckOfCards Constructor

The class's constructor instantiates the deck array (line 20) with `NUMBER_OF_CARDS` (52) elements. The elements of `deck` are `null` by default, so the constructor uses a `for` statement (lines 24–26) to fill the deck with `Cards`. The loop initializes control variable `count` to 0 and loops while `count` is less than `deck.length`, causing `count` to take on each integer value from 0 to 51 (the indices of the deck array). Each `Card` is instantiated and initialized with two `Strings`—one from the `faces` array (which contains the `Strings` "Ace" through "King") and one from the `suits` array (which contains the `Strings` "Hearts", "Diamonds", "Clubs" and "Spades"). The calculation `count % 13` always results in a value from 0 to 12 (the 13 indices of the `faces` array in lines 16–17), and the calculation `count / 13` always results in a value from 0 to 3 (the four indices of the `suits` array in line 18). When the deck array is initialized, it contains the `Cards` with faces "Ace" through "King" in order for each suit (all 13 "Hearts", then all the "Diamonds", then the "Clubs", then the "Spades"). We use arrays of `Strings` to represent the faces and suits in this example. In Exercise 7.34, we ask you to modify this example to use arrays of `enum` constants to represent the faces and suits.

DeckOfCards Method shuffle

Method `shuffle` (lines 30–46) shuffles the `Cards` in the deck. The method loops through all 52 `Cards` (array indices 0 to 51). For each `Card`, a number between 0 and 51 is picked randomly to select another `Card`. Next, the current `Card` object and the randomly selected `Card` object are swapped in the array. This exchange is performed by the three assignments in lines 42–44. The extra variable `temp` temporarily stores one of the two `Card` objects being swapped. The swap cannot be performed with only the two statements

```
deck[first] = deck[second];
deck[second] = deck[first];
```

If `deck[first]` is the "Ace" of "Spades" and `deck[second]` is the "Queen" of "Hearts", after the first assignment, both array elements contain the "Queen" of "Hearts" and the "Ace" of "Spades" is lost—so, the extra variable `temp` is needed. After the `for` loop terminates, the `Card` objects are randomly ordered. A total of only 52 swaps are made in a single pass of the entire array, and the array of `Card` objects is shuffled!

[*Note:* It's recommended that you use a so-called *unbiased* shuffling algorithm for real card games. Such an algorithm ensures that all possible shuffled card sequences are equally likely to occur. Exercise 7.35 asks you to research the popular unbiased Fisher-Yates shuffling algorithm and use it to reimplement the `DeckOfCards` method `shuffle`.]

DeckOfCards Method dealCard

Method `dealCard` (lines 49–56) deals one `Card` in the array. Recall that `currentCard` indicates the index of the next `Card` to be dealt (i.e., the `Card` at the *top* of the deck). Thus, line 52 compares `currentCard` to the length of the deck array. If the deck is not empty (i.e., `currentCard` is less than 52), line 53 returns the "top" `Card` and postincrements `currentCard` to prepare for the next call to `dealCard`—otherwise, `null` is returned. Recall from Chapter 3 that `null` represents a "reference to nothing."

Shuffling and Dealing Cards

Figure 7.11 demonstrates class `DeckOfCards`. Line 9 creates a `DeckOfCards` object named `myDeckOfCards`. The `DeckOfCards` constructor creates the deck with the 52 `Card` objects

in order by suit and face. Line 10 invokes `myDeckOfCards`'s `shuffle` method to rearrange the `Card` objects. Lines 13–20 deal all 52 `Card`s and print them in four columns of 13 `Card`s each. Line 16 deals one `Card` object by invoking `myDeckOfCards`'s `dealCard` method, then displays the `Card` left justified in a field of 19 characters. When a `Card` is output as a `String`, the `Card`'s `toString` method (lines 17–20 of Fig. 7.9) is implicitly invoked. Lines 18–19 start a new line after every four `Card`s.

```

1  // Fig. 7.11: DeckOfCardsTest.java
2  // Card shuffling and dealing.
3
4  public class DeckOfCardsTest
5  {
6      // execute application
7      public static void main(String[] args)
8      {
9          DeckOfCards myDeckOfCards = new DeckOfCards();
10         myDeckOfCards.shuffle(); // place Cards in random order
11
12         // print all 52 Cards in the order in which they are dealt
13         for (int i = 1; i <= 52; i++)
14         {
15             // deal and display a Card
16             System.out.printf("%-19s", myDeckOfCards.dealCard());
17
18             if (i % 4 == 0) // output a newline after every fourth card
19                 System.out.println();
20         }
21     }
22 } // end class DeckOfCardsTest

```

Six of Spades	Eight of Spades	Six of Clubs	Nine of Hearts
Queen of Hearts	Seven of Clubs	Nine of Spades	King of Hearts
Three of Diamonds	Deuce of Clubs	Ace of Hearts	Ten of Spades
Four of Spades	Ace of Clubs	Seven of Diamonds	Four of Hearts
Three of Clubs	Deuce of Hearts	Five of Spades	Jack of Diamonds
King of Clubs	Ten of Hearts	Three of Hearts	Six of Diamonds
Queen of Clubs	Eight of Diamonds	Deuce of Diamonds	Ten of Diamonds
Three of Spades	King of Diamonds	Nine of Clubs	Six of Hearts
Ace of Spades	Four of Diamonds	Seven of Hearts	Eight of Clubs
Deuce of Spades	Eight of Hearts	Five of Hearts	Queen of Spades
Jack of Hearts	Seven of Spades	Four of Clubs	Nine of Diamonds
Ace of Diamonds	Queen of Diamonds	Five of Clubs	King of Spades
Five of Diamonds	Ten of Clubs	Jack of Spades	Jack of Clubs

Fig. 7.11 | Card shuffling and dealing.

Preventing NullPointerExceptions

In Fig. 7.10, we created a deck array of 52 `Card` references—each element of every reference-type array created with `new` is default initialized to `null`. Reference-type variables which are fields of a class are also initialized to `null` by default. A `NullPointerException` occurs when you try to call a method on a `null` reference. In industrial-strength code, ensuring that references are not `null` before you use them to call methods prevents `NullPointerExceptions`.

7.7 Enhanced for Statement

The **enhanced for statement** iterates through the elements of an array *without* using a counter, thus avoiding the possibility of “stepping outside” the array. We show how to use the enhanced for statement with the Java API’s prebuilt data structures (called collections) in Section 7.16. The syntax of an enhanced for statement is:

```
for (parameter : arrayName)
    statement
```

where *parameter* has a *type* and an *identifier* (e.g., `int number`), and *arrayName* is the array through which to iterate. The type of the parameter must be *consistent* with the type of the elements in the array. As the next example illustrates, the identifier represents successive element values in the array on successive iterations of the loop.

Figure 7.12 uses the enhanced for statement (lines 12–13) to sum the integers in an array of student grades. The enhanced for’s parameter is of type `int`, because array contains `int` values—the loop selects one `int` value from the array during each iteration. The enhanced for statement iterates through successive values in the array one by one. The statement’s header can be read as “for each iteration, assign the next element of array to `int` variable `number`, then execute the following statement.” Thus, for each iteration, identifier `number` represents an `int` value in array. Lines 12–13 are equivalent to the following counter-controlled repetition used in lines 12–13 of Fig. 7.5 to total the integers in array, except that counter *cannot be accessed* in the enhanced for statement:

```
for (int counter = 0; counter < array.length; counter++)
    total += array[counter];
```

```
1 // Fig. 7.12: EnhancedForTest.java
2 // Using the enhanced for statement to total integers in an array.
3
4 public class EnhancedForTest
5 {
6     public static void main(String[] args)
7     {
8         int[] array = { 87, 68, 94, 100, 83, 78, 85, 91, 76, 87 };
9         int total = 0;
10
11         // add each element's value to total
12         for (int number : array)
13             total += number;
14
15         System.out.printf("Total of array elements: %d\n", total);
16     }
17 } // end class EnhancedForTest
```

```
Total of array elements: 849
```

Fig. 7.12 | Using the enhanced for statement to total integers in an array.

The enhanced for statement can be used *only* to obtain array elements—it *cannot* be used to *modify* elements. If your program needs to modify elements, use the traditional counter-controlled for statement.

The enhanced for statement can be used in place of the counter-controlled for statement whenever code looping through an array does *not* require access to the counter indicating the index of the current array element. For example, totaling the integers in an array requires access only to the element values—the index of each element is irrelevant. However, if a program must use a counter for some reason other than simply to loop through an array (e.g., to print an index number next to each array element value, as in the examples earlier in this chapter), use the counter-controlled for statement.



Error-Prevention Tip 7.2

The enhanced for statement simplifies the code for iterating through an array making the code more readable and eliminating several error possibilities, such as improperly specifying the control variable's initial value, the loop-continuation test and the increment expression.

Java SE 8

The for statement and the enhanced for statement each iterate sequentially from a starting value to an ending value. In Chapter 17, Java SE 8 Lambdas and Streams, you'll learn about class Stream and its `forEach` method. Working together, these provide an elegant, more concise and less error prone means for iterating through collections so that some of the iterations may occur in parallel with others to achieve better multi-core system performance.

7.8 Passing Arrays to Methods

This section demonstrates how to pass arrays and individual array elements as arguments to methods. To pass an array argument to a method, specify the name of the array *without any brackets*. For example, if array `hourlyTemperatures` is declared as

```
double[] hourlyTemperatures = new double[24];
```

then the method call

```
modifyArray(hourlyTemperatures);
```

passes the reference of array `hourlyTemperatures` to method `modifyArray`. Every array object “knows” its own length. Thus, when we pass an array object's reference into a method, we need not pass the array length as an additional argument.

For a method to receive an array reference through a method call, the method's parameter list must specify an *array parameter*. For example, the method header for method `modifyArray` might be written as

```
void modifyArray(double[] b)
```

indicating that `modifyArray` receives the reference of a `double` array in parameter `b`. The method call passes array `hourlyTemperature`'s reference, so when the called method uses the array variable `b`, it *refers to* the same array object as `hourlyTemperatures` in the caller.

When an argument to a method is an entire array or an individual array element of a reference type, the called method receives a *copy* of the reference. However, when an argument to a method is an individual array element of a primitive type, the called method receives a copy of the element's *value*. Such primitive values are called **scalars** or **scalar quantities**. To pass an individual array element to a method, use the indexed name of the array element as an argument in the method call.

Figure 7.13 demonstrates the difference between passing an entire array and passing a primitive-type array element to a method. Notice that `main` invokes static methods `modifyArray` (line 19) and `modifyElement` (line 30) directly. Recall from Section 6.4 that a static method of a class can invoke other static methods of the same class directly.

The enhanced `for` statement at lines 16–17 outputs the elements of array. Line 19 invokes method `modifyArray`, passing `array` as an argument. The method (lines 36–40) receives a copy of `array`'s reference and uses it to multiply each of `array`'s elements by 2. To prove that `array`'s elements were modified, lines 23–24 output `array`'s elements again. As the output shows, method `modifyArray` doubled the value of each element. We could *not* use the enhanced `for` statement in lines 38–39 because we're modifying the array's elements.

```

1  // Fig. 7.13: PassArray.java
2  // Passing arrays and individual array elements to methods.
3
4  public class PassArray
5  {
6      // main creates array and calls modifyArray and modifyElement
7      public static void main(String[] args)
8      {
9          int[] array = { 1, 2, 3, 4, 5 };
10
11         System.out.printf(
12             "Effects of passing reference to entire array:%n" +
13             "The values of the original array are:%n");
14
15         // output original array elements
16         for (int value : array)
17             System.out.printf("    %d", value);
18
19         modifyArray(array); // pass array reference
20         System.out.printf("%n\nThe values of the modified array are:%n");
21
22         // output modified array elements
23         for (int value : array)
24             System.out.printf("    %d", value);
25
26         System.out.printf(
27             "%n\nEffects of passing array element value:%n" +
28             "array[3] before modifyElement: %d%n", array[3]);
29
30         modifyElement(array[3]); // attempt to modify array[3]
31         System.out.printf(
32             "array[3] after modifyElement: %d%n", array[3]);
33     }
34
35     // multiply each element of an array by 2
36     public static void modifyArray(int[] array2)
37     {
38         for (int counter = 0; counter < array2.length; counter++)
39             array2[counter] *= 2;
40     }

```

Fig. 7.13 | Passing arrays and individual array elements to methods. (Part I of 2.)

```

41
42 // multiply argument by 2
43 public static void modifyElement(int element)
44 {
45     element *= 2;
46     System.out.printf(
47         "Value of element in modifyElement: %d\n", element);
48 }
49 } // end class PassArray

```

Effects of passing reference to entire array:

The values of the original array are:

1 2 3 4 5

The values of the modified array are:

2 4 6 8 10

Effects of passing array element value:

array[3] before modifyElement: 8

Value of element in modifyElement: 16

array[3] after modifyElement: 8

Fig. 7.13 | Passing arrays and individual array elements to methods. (Part 2 of 2.)

Figure 7.13 next demonstrates that when a copy of an individual primitive-type array element is passed to a method, modifying the *copy* in the called method does *not* affect the original value of that element in the calling method's array. Lines 26–28 output the value of array[3] *before* invoking method `modifyElement`. Remember that the value of this element is now 8 after it was modified in the call to `modifyArray`. Line 30 calls method `modifyElement` and passes array[3] as an argument. Remember that array[3] is actually one `int` value (8) in array. Therefore, the program passes a copy of the value of array[3]. Method `modifyElement` (lines 43–48) multiplies the value received as an argument by 2, stores the result in its parameter `element`, then outputs the value of `element` (16). Since method parameters, like local variables, cease to exist when the method in which they're declared completes execution, the method parameter `element` is destroyed when method `modifyElement` terminates. When the program returns control to main, lines 31–32 output the *unmodified* value of array[3] (i.e., 8).

7.9 Pass-By-Value vs. Pass-By-Reference

The preceding example demonstrated how arrays and primitive-type array elements are passed as arguments to methods. We now take a closer look at how arguments in general are passed to methods. Two ways to pass arguments in method calls in many programming languages are **pass-by-value** and **pass-by-reference** (sometimes called **call-by-value** and **call-by-reference**). When an argument is passed by value, a *copy* of the argument's *value* is passed to the called method. The called method works exclusively with the copy. Changes to the called method's copy do *not* affect the original variable's value in the caller.

When an argument is passed by reference, the called method can access the argument's value in the caller directly and modify that data, if necessary. Pass-by-reference improves performance by eliminating the need to copy possibly large amounts of data.

Unlike some other languages, Java does *not* allow you to choose pass-by-value or pass-by-reference—*all arguments are passed by value*. A method call can pass two types of values to a method—copies of primitive values (e.g., values of type `int` and `double`) and copies of references to objects. Objects themselves cannot be passed to methods. When a method modifies a primitive-type parameter, changes to the parameter have no effect on the original argument value in the calling method. For example, when line 30 in `main` of Fig. 7.13 passes `array[3]` to method `modifyElement`, the statement in line 45 that doubles the value of parameter `element` has *no* effect on the value of `array[3]` in `main`. This is also true for reference-type parameters. If you modify a reference-type parameter so that it refers to another object, only the parameter refers to the new object—the reference stored in the caller’s variable still refers to the original object.

Although an object’s reference is passed by value, a method can still interact with the referenced object by calling its `public` methods using the copy of the object’s reference. Since the reference stored in the parameter is a copy of the reference that was passed as an argument, the parameter in the called method and the argument in the calling method refer to the *same* object in memory. For example, in Fig. 7.13, both parameter `array2` in method `modifyArray` and variable `array` in `main` refer to the *same* array object in memory. Any changes made using the parameter `array2` are carried out on the object that `array` references in the calling method. In Fig. 7.13, the changes made in `modifyArray` using `array2` affect the contents of the array object referenced by `array` in `main`. Thus, with a reference to an object, the called method *can* manipulate the caller’s object directly.



Performance Tip 7.1

Passing references to arrays, instead of the array objects themselves, makes sense for performance reasons. Because everything in Java is passed by value, if array objects were passed, a copy of each element would be passed. For large arrays, this would waste time and consume considerable storage for the copies of the elements.

7.10 Case Study: Class `GradeBook` Using an Array to Store Grades

We now present the first part of our case study on developing a `GradeBook` class that instructors can use to maintain students’ grades on an exam and display a grade report that includes the grades, class average, lowest grade, highest grade and a grade distribution bar chart. The version of class `GradeBook` presented in this section stores the grades for one exam in a one-dimensional array. In Section 7.12, we present a version of class `GradeBook` that uses a two-dimensional array to store students’ grades for *several* exams.

Storing Student Grades in an Array in Class `GradeBook`

Class `GradeBook` (Fig. 7.14) uses an array of `ints` to store several students’ grades on a single exam. Array `grades` is declared as an instance variable (line 7), so each `GradeBook` object maintains its *own* set of grades. The constructor (lines 10–14) has two parameters—the name of the course and an array of grades. When an application (e.g., class `GradeBookTest` in Fig. 7.15) creates a `GradeBook` object, the application passes an existing `int` array to the constructor, which assigns the array’s reference to instance variable `grades` (line 13). The `grades` array’s *size* is determined by the `length` instance variable of the constructor’s

array parameter. Thus, a GradeBook object can process a *variable* number of grades. The grade values in the argument could have been input from a user, read from a file on disk (as discussed in Chapter 15) or come from a variety of other sources. In class GradeBook-Test, we initialize an array with grade values (Fig. 7.15, line 10). Once the grades are stored in *instance variable* grades of class GradeBook, all the class's methods can access the elements of grades.

```

1  // Fig. 7.14: GradeBook.java
2  // GradeBook class using an array to store test grades.
3
4  public class GradeBook
5  {
6      private String courseName; // name of course this GradeBook represents
7      private int[] grades; // array of student grades
8
9      // constructor
10     public GradeBook(String courseName, int[] grades)
11     {
12         this.courseName = courseName;
13         this.grades = grades;
14     }
15
16     // method to set the course name
17     public void setCourseName(String courseName)
18     {
19         this.courseName = courseName;
20     }
21
22     // method to retrieve the course name
23     public String getCourseName()
24     {
25         return courseName;
26     }
27
28     // perform various operations on the data
29     public void processGrades()
30     {
31         // output grades array
32         outputGrades();
33
34         // call method getAverage to calculate the average grade
35         System.out.printf("\nClass average is %.2f\n", getAverage());
36
37         // call methods getMinimum and getMaximum
38         System.out.printf("Lowest grade is %d\nHighest grade is %d\n\n",
39             getMinimum(), getMaximum());
40
41         // call outputBarChart to print grade distribution chart
42         outputBarChart();
43     }
44

```

Fig. 7.14 | GradeBook class using an array to store test grades. (Part I of 3.)

```
45 // find minimum grade
46 public int getMinimum()
47 {
48     int lowGrade = grades[0]; // assume grades[0] is smallest
49
50     // loop through grades array
51     for (int grade : grades)
52     {
53         // if grade lower than lowGrade, assign it to lowGrade
54         if (grade < lowGrade)
55             lowGrade = grade; // new lowest grade
56     }
57
58     return lowGrade;
59 }
60
61 // find maximum grade
62 public int getMaximum()
63 {
64     int highGrade = grades[0]; // assume grades[0] is largest
65
66     // loop through grades array
67     for (int grade : grades)
68     {
69         // if grade greater than highGrade, assign it to highGrade
70         if (grade > highGrade)
71             highGrade = grade; // new highest grade
72     }
73
74     return highGrade;
75 }
76
77 // determine average grade for test
78 public double getAverage()
79 {
80     int total = 0;
81
82     // sum grades for one student
83     for (int grade : grades)
84         total += grade;
85
86     // return average of grades
87     return (double) total / grades.length;
88 }
89
90 // output bar chart displaying grade distribution
91 public void outputBarChart()
92 {
93     System.out.println("Grade distribution:");
94
95     // stores frequency of grades in each range of 10 grades
96     int[] frequency = new int[11];
```

Fig. 7.14 | GradeBook class using an array to store test grades. (Part 2 of 3.)


```

97
98 // for each grade, increment the appropriate frequency
99 for (int grade : grades)
100     ++frequency[grade / 10];
101
102 // for each grade frequency, print bar in chart
103 for (int count = 0; count < frequency.length; count++)
104 {
105     // output bar label ("00-09: ", ..., "90-99: ", "100: ")
106     if (count == 10)
107         System.out.printf("%5d: ", 100);
108     else
109         System.out.printf("%02d-%02d: ",
110             count * 10, count * 10 + 9);
111
112     // print bar of asterisks
113     for (int stars = 0; stars < frequency[count]; stars++)
114         System.out.print("*");
115
116     System.out.println();
117 }
118 }
119
120 // output the contents of the grades array
121 public void outputGrades()
122 {
123     System.out.printf("The grades are:%n%n");
124
125     // output each student's grade
126     for (int student = 0; student < grades.length; student++)
127         System.out.printf("Student %2d: %3d%n",
128             student + 1, grades[student]);
129 }
130 } // end class GradeBook

```

Fig. 7.14 | GradeBook class using an array to store test grades. (Part 3 of 3.)

Method `processGrades` (lines 29–43) contains a series of method calls that output a report summarizing the grades. Line 32 calls method `outputGrades` to print the contents of the array `grades`. Lines 126–128 in method `outputGrades` output the students' grades. A counter-controlled `for` statement *must* be used in this case, because lines 127–128 use counter variable `student`'s value to output each grade next to a particular student number (see the output in Fig. 7.15). Although array indices start at 0, a professor might typically number students starting at 1. Thus, lines 127–128 output `student + 1` as the student number to produce grade labels "Student 1: ", "Student 2: ", and so on.

Method `processGrades` next calls method `getAverage` (line 35) to obtain the average of the grades in the array. Method `getAverage` (lines 78–88) uses an enhanced `for` statement to total the values in array `grades` before calculating the average. The parameter in the enhanced `for`'s header (e.g., `int grade`) indicates that for each iteration, the `int` variable `grade` takes on a value in the array `grades`. The averaging calculation in line 87 uses `grades.length` to determine the number of grades being averaged.

Lines 38–39 in method `processGrades` call methods `getMinimum` and `getMaximum` to determine the lowest and highest grades of any student on the exam, respectively. Each of these methods uses an enhanced `for` statement to loop through array `grades`. Lines 51–56 in method `getMinimum` loop through the array. Lines 54–55 compare each grade to `lowGrade`; if a grade is less than `lowGrade`, `lowGrade` is set to that grade. When line 58 executes, `lowGrade` contains the lowest grade in the array. Method `getMaximum` (lines 62–75) works similarly to method `getMinimum`.

Finally, line 42 in method `processGrades` calls `outputBarChart` to print a grade distribution chart using a technique similar to that in Fig. 7.6. In that example, we manually calculated the number of grades in each category (i.e., 0–9, 10–19, ..., 90–99 and 100) by simply looking at a set of grades. Here, lines 99–100 use a technique similar to that in Figs. 7.7–7.8 to calculate the frequency of grades in each category. Line 96 declares and creates array `frequency` of 11 `ints` to store the frequency of grades in each category. For each grade in array `grades`, lines 99–100 increment the appropriate frequency array element. To determine which one to increment, line 100 divides the current grade by 10 using *integer division*—e.g., if grade is 85, line 100 increments `frequency[8]` to update the count of grades in the range 80–89. Lines 103–117 print the bar chart (as shown in Fig. 7.15) based on the values in array `frequency`. Like lines 23–24 of Fig. 7.6, lines 113–116 of Fig. 7.14 use a value in array `frequency` to determine the number of asterisks to display in each bar.

Class `GradeBookTest` That Demonstrates Class `GradeBook`

The application of Fig. 7.15 creates an object of class `GradeBook` (Fig. 7.14) using the `int` array `gradesArray` (declared and initialized in line 10). Lines 12–13 pass a course name and `gradesArray` to the `GradeBook` constructor. Lines 14–15 display a welcome message that includes the course name stored in the `GradeBook` object. Line 16 invokes the `GradeBook` object's `processGrades` method. The output summarizes the 10 grades in `myGradeBook`.



Software Engineering Observation 7.2

A test harness (or test application) is responsible for creating an object of the class being tested and providing it with data. This data could come from any of several sources. Test data can be placed directly into an array with an array initializer, it can come from the user at the keyboard, from a file (as you'll see in Chapter 15), from a database (as you'll see in Chapter 24) or from a network (as you'll see in online Chapter 28). After passing this data to the class's constructor to instantiate the object, the test harness should call upon the object to test its methods and manipulate its data. Gathering data in the test harness like this allows the class to be more reusable, able to manipulate data from several sources.

```

1 // Fig. 7.15: GradeBookTest.java
2 // GradeBookTest creates a GradeBook object using an array of grades,
3 // then invokes method processGrades to analyze them.
4 public class GradeBookTest
5 {
6     // main method begins program execution
7     public static void main(String[] args)
8     {

```

Fig. 7.15 | `GradeBookTest` creates a `GradeBook` object using an array of grades, then invokes method `processGrades` to analyze them. (Part 1 of 2.)

```
9      // array of student grades
10     int[] gradesArray = { 87, 68, 94, 100, 83, 78, 85, 91, 76, 87 };
11
12     GradeBook myGradeBook = new GradeBook(
13         "CS101 Introduction to Java Programming", gradesArray);
14     System.out.printf("Welcome to the grade book for%n%s%n%n",
15         myGradeBook.getCourseName());
16     myGradeBook.processGrades();
17 }
18 } // end class GradeBookTest
```

```
Welcome to the grade book for
CS101 Introduction to Java Programming
```

```
The grades are:
```

```
Student 1: 87
Student 2: 68
Student 3: 94
Student 4: 100
Student 5: 83
Student 6: 78
Student 7: 85
Student 8: 91
Student 9: 76
Student 10: 87
```

```
Class average is 84.90
Lowest grade is 68
Highest grade is 100
```

```
Grade distribution:
```

```
00-09:
10-19:
20-29:
30-39:
40-49:
50-59:
60-69: *
70-79: **
80-89: ****
90-99: **
100: *
```

Fig. 7.15 | GradeBookTest creates a GradeBook object using an array of grades, then invokes method processGrades to analyze them. (Part 2 of 2.)

Java SE 8

In Chapter 17, Java SE 8 Lambdas and Streams, the example of Fig. 17.5 uses stream methods min, max, count and average to process the elements of an int array elegantly and concisely without having to write repetition statements. In Chapter 23, Concurrency, the example of Fig. 23.29 uses stream method summaryStatistics to perform all of these operations in one method call.

7.11 Multidimensional Arrays

Multidimensional arrays with two dimensions are often used to represent *tables* of values with data arranged in *rows* and *columns*. To identify a particular table element, you specify *two* indices. *By convention*, the first identifies the element's row and the second its column. Arrays that require two indices to identify each element are called **two-dimensional arrays**. (Multidimensional arrays can have more than two dimensions.) Java does not support multidimensional arrays directly, but it allows you to specify one-dimensional arrays whose elements are also one-dimensional arrays, thus achieving the same effect. Figure 7.16 illustrates a two-dimensional array named *a* with three rows and four columns (i.e., a three-by-four array). In general, an array with *m* rows and *n* columns is called an ***m*-by-*n* array**.

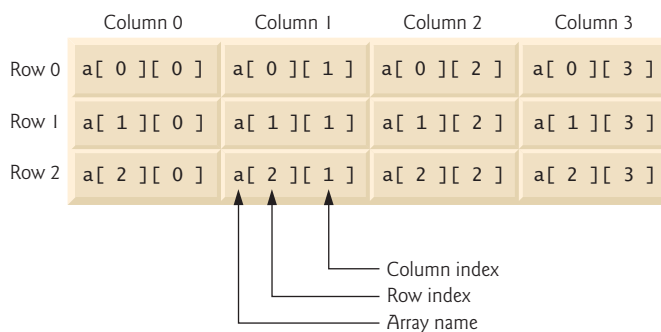


Fig. 7.16 | Two-dimensional array with three rows and four columns.

Every element in array *a* is identified in Fig. 7.16 by an *array-access expression* of the form *a*[*row*][*column*]; *a* is the name of the array, and *row* and *column* are the indices that uniquely identify each element by row and column index. The names of the elements in *row* 0 all have a *first* index of 0, and the names of the elements in *column* 3 all have a *second* index of 3.

Arrays of One-Dimensional Arrays

Like one-dimensional arrays, multidimensional arrays can be initialized with array initializers in declarations. A two-dimensional array *b* with two rows and two columns could be declared and initialized with **nested array initializers** as follows:

```
int[][] b = {{1, 2}, {3, 4}};
```

The initial values are *grouped by row* in braces. So 1 and 2 initialize *b*[0][0] and *b*[0][1], respectively, and 3 and 4 initialize *b*[1][0] and *b*[1][1], respectively. The compiler counts the number of nested array initializers (represented by sets of braces within the outer braces) to determine the number of *rows* in array *b*. The compiler counts the initializer values in the nested array initializer for a row to determine the number of *columns* in that row. As we'll see momentarily, this means that *rows can have different lengths*.

Multidimensional arrays are maintained as *arrays of one-dimensional arrays*. Therefore array *b* in the preceding declaration is actually composed of two separate one-dimensional arrays—one containing the values in the first nested initializer list {1, 2} and one containing the values in the second nested initializer list {3, 4}. Thus, array *b* itself is an array of two elements, each a one-dimensional array of *int* values.

Two-Dimensional Arrays with Rows of Different Lengths

The manner in which multidimensional arrays are represented makes them quite flexible. In fact, the lengths of the rows in array *b* are *not* required to be the same. For example,

```
int[][] b = {{1, 2}, {3, 4, 5}};
```

creates integer array *b* with two elements (determined by the number of nested array initializers) that represent the rows of the two-dimensional array. Each element of *b* is a *reference* to a one-dimensional array of *int* variables. The *int* array for row 0 is a one-dimensional array with *two* elements (1 and 2), and the *int* array for row 1 is a one-dimensional array with *three* elements (3, 4 and 5).

Creating Two-Dimensional Arrays with Array-Creation Expressions

A multidimensional array with the *same* number of columns in every row can be created with an array-creation expression. For example, the following line declares array *b* and assign it a reference to a three-by-four array:

```
int[][] b = new int[3][4];
```

In this case, we use the literal values 3 and 4 to specify the number of rows and number of columns, respectively, but this is *not* required. Programs can also use variables to specify array dimensions, because *new* creates arrays at execution time—not at compile time. The elements of a multidimensional array are initialized when the array object is created.

A multidimensional array in which each row has a *different* number of columns can be created as follows:

```
int[][] b = new int[2][]; // create 2 rows
b[0] = new int[5]; // create 5 columns for row 0
b[1] = new int[3]; // create 3 columns for row 1
```

The preceding statements create a two-dimensional array with two rows. Row 0 has *five* columns, and row 1 has *three* columns.

Two-Dimensional Array Example: Displaying Element Values

Figure 7.17 demonstrates initializing two-dimensional arrays with array initializers, and using nested for loops to *traverse* the arrays (i.e., manipulate *every* element of each array). Class *InitArray*'s *main* declares two arrays. The declaration of *array1* (line 9) uses nested array initializers of the *same* length to initialize the first row to the values 1, 2 and 3, and the second row to the values 4, 5 and 6. The declaration of *array2* (line 10) uses nested initializers of *different* lengths. In this case, the first row is initialized to two elements with the values 1 and 2, respectively. The second row is initialized to one element with the value 3. The third row is initialized to three elements with the values 4, 5 and 6, respectively.

```
1 // Fig. 7.17: InitArray.java
2 // Initializing two-dimensional arrays.
3
4 public class InitArray
5 {
```

Fig. 7.17 | Initializing two-dimensional arrays. (Part 1 of 2.)

```

6 // create and output two-dimensional arrays
7 public static void main(String[] args)
8 {
9     int[][] array1 = {{1, 2, 3}, {4, 5, 6}};
10    int[][] array2 = {{1, 2}, {3}, {4, 5, 6}};
11
12    System.out.println("Values in array1 by row are");
13    outputArray(array1); // displays array1 by row
14
15    System.out.printf("%nValues in array2 by row are%n");
16    outputArray(array2); // displays array2 by row
17 }
18
19 // output rows and columns of a two-dimensional array
20 public static void outputArray(int[][] array)
21 {
22     // loop through array's rows
23     for (int row = 0; row < array.length; row++)
24     {
25         // loop through columns of current row
26         for (int column = 0; column < array[row].length; column++)
27             System.out.printf("%d ", array[row][column]);
28
29         System.out.println();
30     }
31 }
32 } // end class InitArray

```

```

Values in array1 by row are
1 2 3
4 5 6

```

```

Values in array2 by row are
1 2
3
4 5 6

```

Fig. 7.17 | Initializing two-dimensional arrays. (Part 2 of 2.)

Lines 13 and 16 call method `outputArray` (lines 20–31) to output the elements of `array1` and `array2`, respectively. Method `outputArray`'s parameter—`int[][] array`—indicates that the method receives a two-dimensional array. The nested `for` statement (lines 23–30) outputs the rows of a two-dimensional array. In the loop-continuation condition of the outer `for` statement, the expression `array.length` determines the number of rows in the array. In the inner `for` statement, the expression `array[row].length` determines the number of columns in the current row of the array. The inner `for` statement's condition enables the loop to determine the exact number of columns in each row. We demonstrate nested enhanced `for` statements in Fig. 7.18.

Common Multidimensional-Array Manipulations Performed with `for` Statements

Many common array manipulations use `for` statements. As an example, the following `for` statement sets all the elements in row 2 of array `a` in Fig. 7.16 to zero:

```
for (int column = 0; column < a[2].length; column++)
    a[2][column] = 0;
```

We specified row 2; therefore, we know that the *first* index is always 2 (0 is the first row, and 1 is the second row). This for loop varies only the *second* index (i.e., the column index). If row 2 of array `a` contains four elements, then the preceding for statement is equivalent to the assignment statements

```
a[2][0] = 0;
a[2][1] = 0;
a[2][2] = 0;
a[2][3] = 0;
```

The following nested for statement totals the values of all the elements in array `a`:

```
int total = 0;
for (int row = 0; row < a.length; row++)
{
    for (int column = 0; column < a[row].length; column++)
        total += a[row][column];
}
```

These nested for statements total the array elements *one row at a time*. The outer for statement begins by setting the row index to 0 so that the first row's elements can be totaled by the inner for statement. The outer for then increments row to 1 so that the second row can be totaled. Then, the outer for increments row to 2 so that the third row can be totaled. The variable `total` can be displayed when the outer for statement terminates. In the next example, we show how to process a two-dimensional array in a similar manner using nested enhanced for statements.

7.12 Case Study: Class GradeBook Using a Two-Dimensional Array

In Section 7.10, we presented class `GradeBook` (Fig. 7.14), which used a one-dimensional array to store student grades on a single exam. In most semesters, students take several exams. Professors are likely to want to analyze grades across the entire semester, both for a single student and for the class as a whole.

Storing Student Grades in a Two-Dimensional Array in Class GradeBook

Figure 7.18 contains a `GradeBook` class that uses a two-dimensional array grades to store the grades of *several* students on *multiple* exams. Each *row* of the array represents a *single* student's grades for the entire course, and each *column* represents the grades of *all* the students who took a particular exam. Class `GradeBookTest` (Fig. 7.19) passes the array as an argument to the `GradeBook` constructor. In this example, we use a ten-by-three array for ten students' grades on three exams. Five methods perform array manipulations to process the grades. Each method is similar to its counterpart in the earlier one-dimensional array version of `GradeBook` (Fig. 7.14). Method `getMinimum` (lines 46–62) determines the lowest grade of any student for the semester. Method `getMaximum` (lines 65–83) determines the highest grade of any student for the semester. Method `getAverage` (lines 86–96) determines a particular student's semester average. Method `outputBarChart` (lines 99–129) out-

puts a grade bar chart for the entire semester's student grades. Method `outputGrades` (lines 132–156) outputs the array in a tabular format, along with each student's semester average.

```

1  // Fig. 7.18: GradeBook.java
2  // GradeBook class using a two-dimensional array to store grades.
3
4  public class GradeBook
5  {
6      private String courseName; // name of course this grade book represents
7      private int[][] grades; // two-dimensional array of student grades
8
9      // two-argument constructor initializes courseName and grades array
10     public GradeBook(String courseName, int[][] grades)
11     {
12         this.courseName = courseName;
13         this.grades = grades;
14     }
15
16     // method to set the course name
17     public void setCourseName(String courseName)
18     {
19         this.courseName = courseName;
20     }
21
22     // method to retrieve the course name
23     public String getCourseName()
24     {
25         return courseName;
26     }
27
28     // perform various operations on the data
29     public void processGrades()
30     {
31         // output grades array
32         outputGrades();
33
34         // call methods getMinimum and getMaximum
35         System.out.printf("%n%s %d%n%s %d%n%n",
36             "Lowest grade in the grade book is", getMinimum(),
37             "Highest grade in the grade book is", getMaximum());
38
39         // output grade distribution chart of all grades on all tests
40         outputBarChart();
41     }
42
43     // find minimum grade
44     public int getMinimum()
45     {
46         // assume first element of grades array is smallest
47         int lowGrade = grades[0][0];
48

```

Fig. 7.18 | GradeBook class using a two-dimensional array to store grades. (Part I of 4.)


```
49     // loop through rows of grades array
50     for (int[] studentGrades : grades)
51     {
52         // loop through columns of current row
53         for (int grade : studentGrades)
54         {
55             // if grade less than lowGrade, assign it to lowGrade
56             if (grade < lowGrade)
57                 lowGrade = grade;
58         }
59     }
60
61     return lowGrade;
62 }
63
64 // find maximum grade
65 public int getMaximum()
66 {
67     // assume first element of grades array is largest
68     int highGrade = grades[0][0];
69
70     // loop through rows of grades array
71     for (int[] studentGrades : grades)
72     {
73         // loop through columns of current row
74         for (int grade : studentGrades)
75         {
76             // if grade greater than highGrade, assign it to highGrade
77             if (grade > highGrade)
78                 highGrade = grade;
79         }
80     }
81
82     return highGrade;
83 }
84
85 // determine average grade for particular set of grades
86 public double getAverage(int[] setOfGrades)
87 {
88     int total = 0;
89
90     // sum grades for one student
91     for (int grade : setOfGrades)
92         total += grade;
93
94     // return average of grades
95     return (double) total / setOfGrades.length;
96 }
97
98 // output bar chart displaying overall grade distribution
99 public void outputBarChart()
100 {
```

Fig. 7.18 | GradeBook class using a two-dimensional array to store grades. (Part 2 of 4.)

```

101     System.out.println("Overall grade distribution:");
102
103     // stores frequency of grades in each range of 10 grades
104     int[] frequency = new int[11];
105
106     // for each grade in GradeBook, increment the appropriate frequency
107     for (int[] studentGrades : grades)
108     {
109         for (int grade : studentGrades)
110             ++frequency[grade / 10];
111     }
112
113     // for each grade frequency, print bar in chart
114     for (int count = 0; count < frequency.length; count++)
115     {
116         // output bar label ("00-09: ", ..., "90-99: ", "100: ")
117         if (count == 10)
118             System.out.printf("%5d: ", 100);
119         else
120             System.out.printf("%02d-%02d: ",
121                               count * 10, count * 10 + 9);
122
123         // print bar of asterisks
124         for (int stars = 0; stars < frequency[count]; stars++)
125             System.out.print("*");
126
127         System.out.println();
128     }
129 }
130
131 // output the contents of the grades array
132 public void outputGrades()
133 {
134     System.out.printf("The grades are:%n%n");
135     System.out.print(" "); // align column heads
136
137     // create a column heading for each of the tests
138     for (int test = 0; test < grades[0].length; test++)
139         System.out.printf("Test %d ", test + 1);
140
141     System.out.println("Average"); // student average column heading
142
143     // create rows/columns of text representing array grades
144     for (int student = 0; student < grades.length; student++)
145     {
146         System.out.printf("Student %2d", student + 1);
147
148         for (int test : grades[student]) // output student's grades
149             System.out.printf("%8d", test);
150
151         // call method getAverage to calculate student's average grade;
152         // pass row of grades as the argument to getAverage
153         double average = getAverage(grades[student]);

```

Fig. 7.18 | GradeBook class using a two-dimensional array to store grades. (Part 3 of 4.)

```

154         System.out.printf("%.2f\n", average);
155     }
156 }
157 } // end class GradeBook

```

Fig. 7.18 | GradeBook class using a two-dimensional array to store grades. (Part 4 of 4.)

Methods getMinimum and getMaximum

Methods `getMinimum`, `getMaximum`, `outputBarChart` and `outputGrades` each loop through array `grades` by using nested `for` statements—for example, the nested enhanced `for` statement (lines 50–59) from the declaration of method `getMinimum`. The outer enhanced `for` statement iterates through the two-dimensional array `grades`, assigning successive rows to parameter `studentGrades` on successive iterations. The square brackets following the parameter name indicate that `studentGrades` refers to a one-dimensional `int` array—namely, a row in array `grades` containing one student’s grades. To find the lowest overall grade, the inner `for` statement compares the elements of the current one-dimensional array `studentGrades` to variable `lowGrade`. For example, on the first iteration of the outer `for`, row 0 of `grades` is assigned to parameter `studentGrades`. The inner enhanced `for` statement then loops through `studentGrades` and compares each grade value with `lowGrade`. If a grade is less than `lowGrade`, `lowGrade` is set to that grade. On the second iteration of the outer enhanced `for` statement, row 1 of `grades` is assigned to `studentGrades`, and the elements of this row are compared with variable `lowGrade`. This repeats until all rows of `grades` have been traversed. When execution of the nested statement is complete, `lowGrade` contains the lowest grade in the two-dimensional array. Method `getMaximum` works similarly to method `getMinimum`.

Method outputBarChart

Method `outputBarChart` in Fig. 7.18 is nearly identical to the one in Fig. 7.14. However, to output the overall grade distribution for a whole semester, the method here uses nested enhanced `for` statements (lines 107–111) to create the one-dimensional array `frequency` based on all the grades in the two-dimensional array. The rest of the code in each of the two `outputBarChart` methods that displays the chart is identical.

Method outputGrades

Method `outputGrades` (lines 132–156) uses nested `for` statements to output values of the array `grades` and each student’s semester average. The output (Fig. 7.19) shows the result, which resembles the tabular format of a professor’s physical grade book. Lines 138–139 print the column headings for each test. We use a counter-controlled `for` statement here so that we can identify each test with a number. Similarly, the `for` statement in lines 144–155 first outputs a row label using a counter variable to identify each student (line 146). Although array indices start at 0, lines 139 and 146 output `test + 1` and `student + 1`, respectively, to produce test and student numbers starting at 1 (see the output in Fig. 7.19). The inner `for` statement (lines 148–149) uses the outer `for` statement’s counter variable `student` to loop through a specific row of array `grades` and output each student’s test grade. An enhanced `for` statement can be nested in a counter-controlled `for` statement, and vice versa. Finally, line 153 obtains each student’s semester average by passing the current row of `grades` (i.e., `grades[student]`) to method `getAverage`.

Method `getAverage`

Method `getAverage` (lines 86–96) takes one argument—a one-dimensional array of test results for a particular student. When line 153 calls `getAverage`, the argument is `grades[student]`, which specifies that a particular row of the two-dimensional array `grades` should be passed to `getAverage`. For example, based on the array created in Fig. 7.19, the argument `grades[1]` represents the three values (a one-dimensional array of grades) stored in row 1 of the two-dimensional array `grades`. Recall that a two-dimensional array is one whose elements are one-dimensional arrays. Method `getAverage` calculates the sum of the array elements, divides the total by the number of test results and returns the floating-point result as a `double` value (line 95).

Class `GradeBookTest` That Demonstrates Class `GradeBook`

Figure 7.19 creates an object of class `GradeBook` (Fig. 7.18) using the two-dimensional array of ints named `gradesArray` (declared and initialized in lines 10–19). Lines 21–22 pass a course name and `gradesArray` to the `GradeBook` constructor. Lines 23–24 display a welcome message containing the course name, then line 25 invokes `myGradeBook`'s `processGrades` method to display a report summarizing the students' grades for the semester.

```

1  // Fig. 7.19: GradeBookTest.java
2  // GradeBookTest creates GradeBook object using a two-dimensional array
3  // of grades, then invokes method processGrades to analyze them.
4  public class GradeBookTest
5  {
6      // main method begins program execution
7      public static void main(String[] args)
8      {
9          // two-dimensional array of student grades
10         int[][] gradesArray = {{87, 96, 70},
11                                {68, 87, 90},
12                                {94, 100, 90},
13                                {100, 81, 82},
14                                {83, 65, 85},
15                                {78, 87, 65},
16                                {85, 75, 83},
17                                {91, 94, 100},
18                                {76, 72, 84},
19                                {87, 93, 73}};
20
21         GradeBook myGradeBook = new GradeBook(
22             "CS101 Introduction to Java Programming", gradesArray);
23         System.out.printf("Welcome to the grade book for%n%s%n%n",
24             myGradeBook.getCourseName());
25         myGradeBook.processGrades();
26     }
27 } // end class GradeBookTest

```

Fig. 7.19 | `GradeBookTest` creates `GradeBook` object using a two-dimensional array of grades, then invokes method `processGrades` to analyze them. (Part 1 of 2.)

```

Welcome to the grade book for
CS101 Introduction to Java Programming

The grades are:

      Test 1  Test 2  Test 3  Average
Student 1      87      96      70    84.33
Student 2      68      87      90    81.67
Student 3      94     100      90    94.67
Student 4     100      81      82    87.67
Student 5      83      65      85    77.67
Student 6      78      87      65    76.67
Student 7      85      75      83    81.00
Student 8      91      94     100    95.00
Student 9      76      72      84    77.33
Student 10     87      93      73    84.33

Lowest grade in the grade book is 65
Highest grade in the grade book is 100

Overall grade distribution:
00-09:
10-19:
20-29:
30-39:
40-49:
50-59:
60-69: ***
70-79: *****
80-89: *****
90-99: *****
100: ***

```

Fig. 7.19 | GradeBookTest creates GradeBook object using a two-dimensional array of grades, then invokes method processGrades to analyze them. (Part 2 of 2.)

7.13 Variable-Length Argument Lists

With **variable-length argument lists**, you can create methods that receive an unspecified number of arguments. A type followed by an **ellipsis** (`...`) in a method's parameter list indicates that the method receives a variable number of arguments of that particular type. This use of the ellipsis can occur only *once* in a parameter list, and the ellipsis, together with its type and the parameter name, must be placed at the *end* of the parameter list. While you can use method overloading and array passing to accomplish much of what is accomplished with variable-length argument lists, using an ellipsis in a method's parameter list is more concise.

Figure 7.20 demonstrates method average (lines 7–16), which receives a variable-length sequence of `doubles`. Java treats the variable-length argument list as an array whose elements are all of the same type. So, the method body can manipulate the parameter numbers as an array of `doubles`. Lines 12–13 use the enhanced for loop to walk through the array and calculate the total of the `doubles` in the array. Line 15 accesses `numbers.length` to obtain the size of the `numbers` array for use in the averaging calculation. Lines 29, 31 and 33 in `main` call method average with two, three and four arguments, respectively. Method average has a variable-length argument list (line 7), so it can average as many

double arguments as the caller passes. The output shows that each call to method average returns the correct value.



Common Programming Error 7.5

Placing an ellipsis indicating a variable-length argument list in the middle of a parameter list is a syntax error. An ellipsis may be placed only at the end of the parameter list.

```

1 // Fig. 7.20: VarargsTest.java
2 // Using variable-length argument lists.
3
4 public class VarargsTest
5 {
6     // calculate average
7     public static double average(double... numbers)
8     {
9         double total = 0.0;
10
11         // calculate total using the enhanced for statement
12         for (double d : numbers)
13             total += d;
14
15         return total / numbers.length;
16     }
17
18     public static void main(String[] args)
19     {
20         double d1 = 10.0;
21         double d2 = 20.0;
22         double d3 = 30.0;
23         double d4 = 40.0;
24
25         System.out.printf("d1 = %.1f%d2 = %.1f%d3 = %.1f%d4 = %.1f%n",
26             d1, d2, d3, d4);
27
28         System.out.printf("Average of d1 and d2 is %.1f%n",
29             average(d1, d2) );
30         System.out.printf("Average of d1, d2 and d3 is %.1f%n",
31             average(d1, d2, d3) );
32         System.out.printf("Average of d1, d2, d3 and d4 is %.1f%n",
33             average(d1, d2, d3, d4) );
34     }
35 } // end class VarargsTest

```

```

d1 = 10.0
d2 = 20.0
d3 = 30.0
d4 = 40.0

```

```

Average of d1 and d2 is 15.0
Average of d1, d2 and d3 is 20.0
Average of d1, d2, d3 and d4 is 25.0

```

Fig. 7.20 | Using variable-length argument lists.

7.14 Using Command-Line Arguments

It's possible to pass arguments from the command line to an application via method `main`'s `String[]` parameter, which receives an array of `Strings`. By convention, this parameter is named `args`. When an application is executed using the `java` command, Java passes the **command-line arguments** that appear after the class name in the `java` command to the application's `main` method as `Strings` in the array `args`. The number of command-line arguments is obtained by accessing the array's `length` attribute. Common uses of command-line arguments include passing options and filenames to applications.

Our next example uses command-line arguments to determine the size of an array, the value of its first element and the increment used to calculate the values of the array's remaining elements. The command

```
java InitArray 5 0 4
```

passes three arguments, 5, 0 and 4, to the application `InitArray`. Command-line arguments are separated by white space, *not* commas. When this command executes, `InitArray`'s `main` method receives the three-element array `args` (i.e., `args.length` is 3) in which `args[0]` contains the `String` "5", `args[1]` contains the `String` "0" and `args[2]` contains the `String` "4". The program determines how to use these arguments—in Fig. 7.21 we convert the three command-line arguments to `int` values and use them to initialize an array. When the program executes, if `args.length` is not 3, the program prints an error message and terminates (lines 9–12). Otherwise, lines 14–32 initialize and display the array based on the values of the command-line arguments.

Line 16 gets `args[0]`—a `String` that specifies the array size—and converts it to an `int` value that the program uses to create the array in line 17. The static method `parseInt` of class `Integer` converts its `String` argument to an `int`.

Lines 20–21 convert the `args[1]` and `args[2]` command-line arguments to `int` values and store them in `initialValue` and `increment`, respectively. Lines 24–25 calculate the value for each array element.

```

1  // Fig. 7.21: InitArray.java
2  // Initializing an array using command-line arguments.
3
4  public class InitArray
5  {
6      public static void main(String[] args)
7      {
8          // check number of command-line arguments
9          if (args.length != 3)
10             System.out.printf(
11                 "Error: Please re-enter the entire command, including%n" +
12                 "an array size, initial value and increment.%n");
13         else
14         {
15             // get array size from first command-line argument
16             int arrayLength = Integer.parseInt(args[0]);
17             int[] array = new int[arrayLength];

```

Fig. 7.21 | Initializing an array using command-line arguments. (Part 1 of 2.)

```

18
19         // get initial value and increment from command-line arguments
20         int initialValue = Integer.parseInt(args[1]);
21         int increment = Integer.parseInt(args[2]);
22
23         // calculate value for each array element
24         for (int counter = 0; counter < array.length; counter++)
25             array[counter] = initialValue + increment * counter;
26
27         System.out.printf("%s%8s%n", "Index", "Value");
28
29         // display array index and value
30         for (int counter = 0; counter < array.length; counter++)
31             System.out.printf("%5d%8d%n", counter, array[counter]);
32     }
33 }
34 } // end class InitArray

```

```
java InitArray
```

Error: Please re-enter the entire command, including an array size, initial value and increment.

```
java InitArray 5 0 4
```

Index	Value
0	0
1	4
2	8
3	12
4	16

```
java InitArray 8 1 2
```

Index	Value
0	1
1	3
2	5
3	7
4	9
5	11
6	13
7	15

Fig. 7.21 | Initializing an array using command-line arguments. (Part 2 of 2.)

The output of the first execution shows that the application received an insufficient number of command-line arguments. The second execution uses command-line arguments 5, 0 and 4 to specify the size of the array (5), the value of the first element (0) and the increment of each value in the array (4), respectively. The corresponding output shows that these values create an array containing the integers 0, 4, 8, 12 and 16. The output from the third execution shows that the command-line arguments 8, 1 and 2 produce an array whose 8 elements are the nonnegative odd integers from 1 to 15.

7.15 Class Arrays

Class **Arrays** helps you avoid reinventing the wheel by providing static methods for common array manipulations. These methods include **sort** for *sorting* an array (i.e., arranging elements into ascending order), **binarySearch** for *searching* a *sorted* array (i.e., determining whether an array contains a specific value and, if so, where the value is located), **equals** for *comparing* arrays and **fill** for *placing values into an array*. These methods are overloaded for primitive-type arrays and for arrays of objects. Our focus in this section is on using the built-in capabilities provided by the Java API. Chapter 19, Searching, Sorting and Big O, shows how to implement your own sorting and searching algorithms, a subject of great interest to computer-science researchers and students.

Figure 7.22 uses Arrays methods **sort**, **binarySearch**, **equals** and **fill**, and shows how to *copy* arrays with class System's static **arraycopy** method. In main, line 11 sorts the elements of array `doubleArray`. The static method **sort** of class Arrays orders the array's elements in *ascending* order by default. We discuss how to sort in *descending* order later in the chapter. Overloaded versions of **sort** allow you to sort a specific range of elements within the array. Lines 12–15 output the sorted array.

```

1  // Fig. 7.22: ArrayManipulations.java
2  // Arrays class methods and System.arraycopy.
3  import java.util.Arrays;
4
5  public class ArrayManipulations
6  {
7      public static void main(String[] args)
8      {
9          // sort doubleArray into ascending order
10         double[] doubleArray = { 8.4, 9.3, 0.2, 7.9, 3.4 };
11         Arrays.sort(doubleArray);
12         System.out.printf("%ndoubleArray: ");
13
14         for (double value : doubleArray)
15             System.out.printf("%.1f ", value);
16
17         // fill 10-element array with 7s
18         int[] filledIntArray = new int[10];
19         Arrays.fill(filledIntArray, 7);
20         displayArray(filledIntArray, "filledIntArray");
21
22         // copy array intArray into array intArrayCopy
23         int[] intArray = { 1, 2, 3, 4, 5, 6 };
24         int[] intArrayCopy = new int[intArray.length];
25         System.arraycopy(intArray, 0, intArrayCopy, 0, intArray.length);
26         displayArray(intArray, "intArray");
27         displayArray(intArrayCopy, "intArrayCopy");
28
29         // compare intArray and intArrayCopy for equality
30         boolean b = Arrays.equals(intArray, intArrayCopy);
31         System.out.printf("%n%nintArray %s intArrayCopy%n",
32             (b ? "==" : "!="));

```

Fig. 7.22 | Arrays class methods and System.arraycopy. (Part I of 2.)

```

33
34 // compare intArray and filledIntArray for equality
35 b = Arrays.equals(intArray, filledIntArray);
36 System.out.printf("intArray %s filledIntArray%n",
37     (b ? "==" : "!="));
38
39 // search intArray for the value 5
40 int location = Arrays.binarySearch(intArray, 5);
41
42 if (location >= 0)
43     System.out.printf(
44         "Found 5 at element %d in intArray%n", location);
45 else
46     System.out.println("5 not found in intArray");
47
48 // search intArray for the value 8763
49 location = Arrays.binarySearch(intArray, 8763);
50
51 if (location >= 0)
52     System.out.printf(
53         "Found 8763 at element %d in intArray%n", location);
54 else
55     System.out.println("8763 not found in intArray");
56 }
57
58 // output values in each array
59 public static void displayArray(int[] array, String description)
60 {
61     System.out.printf("%n%s: ", description);
62
63     for (int value : array)
64         System.out.printf("%d ", value);
65 }
66 } // end class ArrayManipulations

```

```

doubleArray: 0.2 3.4 7.9 8.4 9.3
filledIntArray: 7 7 7 7 7 7 7 7 7
intArray: 1 2 3 4 5 6
intArrayCopy: 1 2 3 4 5 6

intArray == intArrayCopy
intArray != filledIntArray
Found 5 at element 4 in intArray
8763 not found in intArray

```

Fig. 7.22 | Arrays class methods and System.arraycopy. (Part 2 of 2.)

Line 19 calls static method `fill` of class `Arrays` to populate all 10 elements of `filledIntArray` with 7s. Overloaded versions of `fill` allow you to populate a specific range of elements with the same value. Line 20 calls our class's `displayArray` method (declared at lines 59–65) to output the contents of `filledIntArray`.

Line 25 copies the elements of `intArray` into `intArrayCopy`. The first argument (`intArray`) passed to `System` method `arraycopy` is the array from which elements are to

be copied. The second argument (0) is the index that specifies the *starting point* in the range of elements to copy from the array. This value can be any valid array index. The third argument (`intArrayCopy`) specifies the *destination array* that will store the copy. The fourth argument (0) specifies the index in the destination array *where the first copied element should be stored*. The last argument specifies the *number of elements to copy* from the array in the first argument. In this case, we copy all the elements in the array.

Lines 30 and 35 call static method `equals` of class `Arrays` to determine whether all the elements of two arrays are equivalent. If the arrays contain the same elements in the same order, the method returns `true`; otherwise, it returns `false`.



Error-Prevention Tip 7.3

When comparing array contents, always use `Arrays.equals(array1, array2)`, which compares the two arrays' contents, rather than `array1.equals(array2)`, which compares whether `array1` and `array2` refer to the same array object.

Lines 40 and 49 call static method `binarySearch` of class `Arrays` to perform a binary search on `intArray`, using the second argument (5 and 8763, respectively) as the key. If value is found, `binarySearch` returns the index of the element; otherwise, `binarySearch` returns a negative value. The negative value returned is based on the search key's *insertion point*—the index where the key would be inserted in the array if we were performing an insert operation. After `binarySearch` determines the insertion point, it changes its sign to negative and subtracts 1 to obtain the return value. For example, in Fig. 7.22, the insertion point for the value 8763 is the element with index 6 in the array. Method `binarySearch` changes the insertion point to -6, subtracts 1 from it and returns the value -7. Subtracting 1 from the insertion point guarantees that method `binarySearch` returns positive values (≥ 0) if and only if the key is found. This return value is useful for inserting elements in a sorted array. Chapter 19 discusses binary searching in detail.



Common Programming Error 7.6

Passing an unsorted array to `binarySearch` is a logic error—the value returned is undefined.

Java SE 8—Class `Arrays` Method `parallelSort`

The `Arrays` class now has several new “parallel” methods that take advantage of multi-core hardware. `Arrays` method `parallelSort` can sort large arrays more efficiently on multi-core systems. In Section 23.12, we create a very large array and use features of the Java SE 8 Date/Time API to compare how long it takes to sort the array with methods `sort` and `parallelSort`.

7.16 Introduction to Collections and Class `ArrayList`

The Java API provides several predefined data structures, called **collections**, used to store groups of related objects in memory. These classes provide efficient methods that organize, store and retrieve your data *without* requiring knowledge of how the data is being *stored*. This reduces application-development time.

You've used arrays to store sequences of objects. Arrays do not automatically change their size at execution time to accommodate additional elements. The collection class

`ArrayList<T>` (package `java.util`) provides a convenient solution to this problem—it can *dynamically* change its size to accommodate more elements. The `T` (by convention) is a *placeholder*—when declaring a new `ArrayList`, replace it with the type of elements that you want the `ArrayList` to hold. For example,

```
ArrayList<String> list;
```

declares `list` as an `ArrayList` collection that can store only `Strings`. Classes with this kind of placeholder that can be used with any type are called **generic classes**. *Only nonprimitive types can be used to declare variables and create objects of generic classes.* However, Java provides a mechanism—known as *boxing*—that allows primitive values to be wrapped as objects for use with generic classes. So, for example,

```
ArrayList<Integer> integers;
```

declares `integers` as an `ArrayList` that can store only `Integers`. When you place an `int` value into an `ArrayList<Integer>`, the `int` value is *boxed* (wrapped) as an `Integer` object, and when you get an `Integer` object from an `ArrayList<Integer>`, then assign the object to an `int` variable, the `int` value inside the object is *unboxed* (unwrapped).

Additional generic collection classes and generics are discussed in Chapters 16 and 20, respectively. Figure 7.23 shows some common methods of class `ArrayList<T>`.

Method	Description
<code>add</code>	Adds an element to the <i>end</i> of the <code>ArrayList</code> .
<code>clear</code>	Removes all the elements from the <code>ArrayList</code> .
<code>contains</code>	Returns <code>true</code> if the <code>ArrayList</code> contains the specified element; otherwise, returns <code>false</code> .
<code>get</code>	Returns the element at the specified index.
<code>indexOf</code>	Returns the index of the first occurrence of the specified element in the <code>ArrayList</code> .
<code>remove</code>	Overloaded. Removes the first occurrence of the specified value or the element at the specified index.
<code>size</code>	Returns the number of elements stored in the <code>ArrayList</code> .
<code>trimToSize</code>	Trims the capacity of the <code>ArrayList</code> to the current number of elements.

Fig. 7.23 | Some methods and properties of class `ArrayList<T>`.

Demonstrating an `ArrayList<String>`

Figure 7.24 demonstrates some common `ArrayList` capabilities. Line 10 creates a new empty `ArrayList` of `Strings` with a default initial capacity of 10 elements. The capacity indicates how many items the `ArrayList` can hold *without growing*. `ArrayList` is implemented using a conventional array behind the scenes. When the `ArrayList` grows, it must create a larger internal array and *copy* each element to the new array. This is a time-consuming operation. It would be inefficient for the `ArrayList` to grow each time an element is added. Instead, it grows only when an element is added *and* the number of elements is equal to the capacity—i.e., there’s no space for the new element.

```

1 // Fig. 7.24: ArrayListCollection.java
2 // Generic ArrayList<T> collection demonstration.
3 import java.util.ArrayList;
4
5 public class ArrayListCollection
6 {
7     public static void main(String[] args)
8     {
9         // create a new ArrayList of Strings with an initial capacity of 10
10        ArrayList<String> items = new ArrayList<String>();
11
12        items.add("red"); // append an item to the list
13        items.add(0, "yellow"); // insert "yellow" at index 0
14
15        // header
16        System.out.print(
17            "Display list contents with counter-controlled loop:");
18
19        // display the colors in the list
20        for (int i = 0; i < items.size(); i++)
21            System.out.printf(" %s", items.get(i));
22
23        // display colors using enhanced for in the display method
24        display(items,
25            "\nDisplay list contents with enhanced for statement:");
26
27        items.add("green"); // add "green" to the end of the list
28        items.add("yellow"); // add "yellow" to the end of the list
29        display(items, "List with two new elements:");
30
31        items.remove("yellow"); // remove the first "yellow"
32        display(items, "Remove first instance of yellow:");
33
34        items.remove(1); // remove item at index 1
35        display(items, "Remove second list element (green):");
36
37        // check if a value is in the List
38        System.out.printf("\n\"red\" is %sin the list\n",
39            items.contains("red") ? "" : "not ");
40
41        // display number of elements in the List
42        System.out.printf("Size: %s\n", items.size());
43    }
44
45    // display the ArrayList's elements on the console
46    public static void display(ArrayList<String> items, String header)
47    {
48        System.out.printf(header); // display header
49
50        // display each element in items
51        for (String item : items)
52            System.out.printf(" %s", item);
53    }

```

Fig. 7.24 | Generic ArrayList<T> collection demonstration. (Part I of 2.)

```

54         System.out.println();
55     }
56 } // end class ArrayListCollection

```

```

Display list contents with counter-controlled loop: yellow red
Display list contents with enhanced for statement: yellow red
List with two new elements: yellow red green yellow
Remove first instance of yellow: red green yellow
Remove second list element (green): red yellow
"red" is in the list
Size: 2

```

Fig. 7.24 | Generic `ArrayList<T>` collection demonstration. (Part 2 of 2.)

The **add** method adds elements to the `ArrayList` (lines 12–13). The add method with *one* argument *appends* its argument to the *end* of the `ArrayList`. The add method with *two* arguments *inserts* a new element at the specified *position*. The first argument is an index. As with arrays, collection indices start at zero. The second argument is the *value* to insert at that *index*. The indices of all subsequent elements are incremented by one. Inserting an element is usually slower than adding an element to the end of the `ArrayList`.

Lines 20–21 display the items in the `ArrayList`. Method **size** returns the number of elements currently in the `ArrayList`. Method **get** (line 21) obtains the element at a specified index. Lines 24–25 display the elements again by invoking method `display` (defined at lines 46–55). Lines 27–28 add two more elements to the `ArrayList`, then line 29 displays the elements again to confirm that the two elements were added to the *end* of the collection.

The **remove** method is used to remove an element with a specific value (line 31). It removes only the first such element. If no such element is in the `ArrayList`, `remove` does nothing. An overloaded version of the method removes the element at the specified index (line 34). When an element is removed, the indices of any elements after the removed element decrease by one.

Line 39 uses the **contains** method to check if an item is in the `ArrayList`. The `contains` method returns `true` if the element is found in the `ArrayList`, and `false` otherwise. The method compares its argument to each element of the `ArrayList` in order, so using `contains` on a large `ArrayList` can be *inefficient*. Line 42 displays the `ArrayList`'s size.

Java SE 7—Diamond (<>) Notation for Creating an Object of a Generic Class

Consider line 10 of Fig. 7.24:

```
ArrayList<String> items = new ArrayList<String>();
```

Notice that `ArrayList<String>` appears in the variable declaration *and* in the class instance creation expression. Java SE 7 introduced the **diamond (<>) notation** to simplify statements like this. Using `<>` in a class instance creation expression for an object of a *generic* class tells the compiler to determine what belongs in the angle brackets. In Java SE 7 and higher, the preceding statement can be written as:

```
ArrayList<String> items = new ArrayList<>();
```

When the compiler encounters the diamond (<>) in the class instance creation expression, it uses the declaration of variable `items` to determine the `ArrayList`'s element type (`String`)—this is known as *inferring the element type*.

7.17 (Optional) GUI and Graphics Case Study: Drawing Arcs

Using Java's graphics features, we can create complex drawings that would be more tedious to code line by line. In Figs. 7.25–7.26, we use arrays and repetition statements to draw a rainbow by using Graphics method `fillArc`. Drawing arcs in Java is similar to drawing ovals—an arc is simply a section of an oval.

In Fig. 7.25, lines 10–11 declare and create two new color constants—VIOLET and INDIGO. As you may know, the colors of a rainbow are red, orange, yellow, green, blue, indigo and violet. Java has predefined constants only for the first five colors. Lines 15–17 initialize an array with the colors of the rainbow, starting with the innermost arcs first. The array begins with two `Color.WHITE` elements, which, as you'll soon see, are for drawing the empty arcs at the center of the rainbow. The instance variables can be initialized when they're declared, as shown in lines 10–17. The constructor (lines 20–23) contains a single statement that calls method `setBackground` (which is inherited from class `JPanel`) with the parameter `Color.WHITE`. Method `setBackground` takes a single `Color` argument and sets the background of the component to that color.

```

1  // Fig. 7.25: DrawRainbow.java
2  // Drawing a rainbow using arcs and an array of colors.
3  import java.awt.Color;
4  import java.awt.Graphics;
5  import javax.swing.JPanel;
6
7  public class DrawRainbow extends JPanel
8  {
9      // define indigo and violet
10     private final static Color VIOLET = new Color(128, 0, 128);
11     private final static Color INDIGO = new Color(75, 0, 130);
12
13     // colors to use in the rainbow, starting from the innermost
14     // The two white entries result in an empty arc in the center
15     private Color[] colors =
16         { Color.WHITE, Color.WHITE, VIOLET, INDIGO, Color.BLUE,
17           Color.GREEN, Color.YELLOW, Color.ORANGE, Color.RED };
18
19     // constructor
20     public DrawRainbow()
21     {
22         setBackground(Color.WHITE); // set the background to white
23     }
24
25     // draws a rainbow using concentric arcs
26     public void paintComponent(Graphics g)
27     {
28         super.paintComponent(g);
29
30         int radius = 20; // radius of an arc
31

```

Fig. 7.25 | Drawing a rainbow using arcs and an array of colors. (Part I of 2.)

```

32     // draw the rainbow near the bottom-center
33     int centerX = getWidth() / 2;
34     int centerY = getHeight() - 10;
35
36     // draws filled arcs starting with the outermost
37     for (int counter = colors.length; counter > 0; counter--)
38     {
39         // set the color for the current arc
40         g.setColor(colors[counter - 1]);
41
42         // fill the arc from 0 to 180 degrees
43         g.fillArc(centerX - counter * radius,
44                 centerY - counter * radius,
45                 counter * radius * 2, counter * radius * 2, 0, 180);
46     }
47 }
48 } // end class DrawRainbow

```

Fig. 7.25 | Drawing a rainbow using arcs and an array of colors. (Part 2 of 2.)

Line 30 in `paintComponent` declares local variable `radius`, which determines the radius of each arc. Local variables `centerX` and `centerY` (lines 33–34) determine the location of the midpoint on the base of the rainbow. The loop at lines 37–46 uses control variable `counter` to count backward from the end of the array, drawing the largest arcs first and placing each successive smaller arc on top of the previous one. Line 40 sets the color to draw the current arc from the array. The reason we have `Color.WHITE` entries at the beginning of the array is to create the empty arc in the center. Otherwise, the center of the rainbow would just be a solid violet semicircle. You can change the individual colors and the number of entries in the array to create new designs.

The `fillArc` method call at lines 43–45 draws a filled semicircle. Method `fillArc` requires six parameters. The first four represent the bounding rectangle in which the arc will be drawn. The first two of these specify the coordinates for the *upper-left* corner of the bounding rectangle, and the next two specify its width and height. The fifth parameter is the starting angle on the oval, and the sixth specifies the *sweep*, or the amount of arc to cover. The starting angle and sweep are measured in degrees, with zero degrees pointing right. A *positive* sweep draws the arc *counterclockwise*, while a *negative* sweep draws the arc *clockwise*. A method similar to `fillArc` is `drawArc`—it requires the same parameters as `fillArc`, but draws the edge of the arc rather than filling it.

Class `DrawRainbowTest` (Fig. 7.26) creates and sets up a `JFrame` to display the rainbow. Once the program makes the `JFrame` visible, the system calls the `paintComponent` method in class `DrawRainbow` to draw the rainbow on the screen.

```

1  // Fig. 7.26: DrawRainbowTest.java
2  // Test application to display a rainbow.
3  import javax.swing.JFrame;
4
5  public class DrawRainbowTest
6  {

```

Fig. 7.26 | Test application to display a rainbow. (Part 1 of 2.)


```

7   public static void main(String[] args)
8   {
9       DrawRainbow panel = new DrawRainbow();
10      JFrame application = new JFrame();
11
12      application.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
13      application.add(panel);
14      application.setSize(400, 250);
15      application.setVisible(true);
16  }
17 } // end class DrawRainbowTest

```

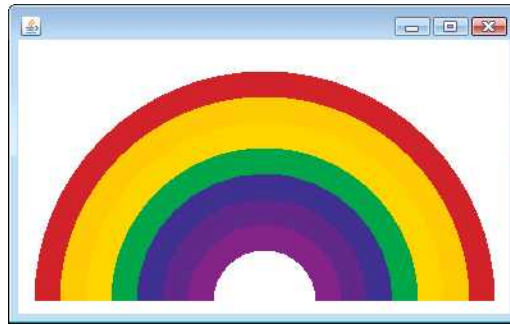


Fig. 7.26 | Test application to display a rainbow. (Part 2 of 2.)

GUI and Graphics Case Study Exercise

- 7.1** (*Drawing Spirals*) In this exercise, you'll draw spirals with methods `drawLine` and `drawArc`.
- Draw a square-shaped spiral (as in the left screen capture of Fig. 7.27), centered on the panel, using method `drawLine`. One technique is to use a loop that increases the line length after drawing every second line. The direction in which to draw the next line should follow a distinct pattern, such as down, left, up, right.
 - Draw a circular spiral (as in the right screen capture of Fig. 7.27), using method `drawArc` to draw one semicircle at a time. Each successive semicircle should have a larger radius (as specified by the bounding rectangle's width) and should continue drawing where the previous semicircle finished.

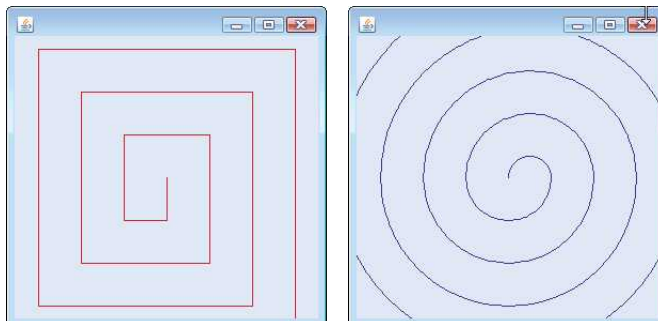


Fig. 7.27 | Drawing a spiral using `drawLine` (left) and `drawArc` (right).

7.18 Wrap-Up

This chapter began our introduction to data structures, exploring the use of arrays to store data in and retrieve data from lists and tables of values. The chapter examples demonstrated how to declare an array, initialize an array and refer to individual elements of an array. The chapter introduced the enhanced for statement to iterate through arrays. We used exception handling to test for `ArrayIndexOutOfBoundsException`s that occur when a program attempts to access an array element outside the bounds of an array. We also illustrated how to pass arrays to methods and how to declare and manipulate multidimensional arrays. Finally, the chapter showed how to write methods that use variable-length argument lists and how to read arguments passed to a program from the command line.

We introduced the `ArrayList<T>` generic collection, which provides all the functionality and performance of arrays, along with other useful capabilities such as dynamic resizing. We used the `add` methods to add new items to the end of an `ArrayList` and to insert items in an `ArrayList`. The `remove` method was used to remove the first occurrence of a specified item, and an overloaded version of `remove` was used to remove an item at a specified index. We used the `size` method to obtain number of items in the `ArrayList`.

We continue our coverage of data structures in Chapter 16, *Generic Collections*. Chapter 16 introduces the Java Collections Framework, which uses generics to allow you to specify the exact types of objects that a particular data structure will store. Chapter 16 also introduces Java's other predefined data structures. Chapter 16 covers additional methods of class `Arrays`. You'll be able to use some of the `Arrays` methods discussed in Chapter 16 after reading the current chapter, but some of the `Arrays` methods require knowledge of concepts presented later in the book. Chapter 20 discusses generics, which enable you to create general models of methods and classes that can be declared once, but used with many different data types. Chapter 21, *Custom Generic Data Structures*, shows how to build dynamic data structures, such as lists, queues, stacks and trees, that can grow and shrink as programs execute.

We've now introduced the basic concepts of classes, objects, control statements, methods, arrays and collections. In Chapter 8, we take a deeper look at classes and objects.

Summary

Section 7.1 Introduction

- Arrays (p. 244) are fixed-length data structures consisting of related data items of the same type.

Section 7.2 Arrays

- An array is a group of variables (called elements or components; p. 245) containing values that all have the same type. Arrays are objects, so they're considered reference types.
- A program refers to any one of an array's elements with an array-access expression (p. 245) that includes the name of the array followed by the index of the particular element in square brackets (`[]`; p. 245).
- The first element in every array has index zero (p. 245) and is sometimes called the zeroth element.
- An index must be a nonnegative integer. A program can use an expression as an index.
- An array object knows its own length and stores this information in a `length` instance variable (p. 246).

Section 7.3 Declaring and Creating Arrays

- To create an array object, specify the array's element type and the number of elements as part of an array-creation expression (p. 246) that uses keyword `new`.
- When an array is created, each element receives a default value—zero for numeric primitive-type elements, `false` for boolean elements and `null` for references.
- In an array declaration, the type and the square brackets can be combined at the beginning of the declaration to indicate that all the identifiers in the declaration are array variables.
- Every element of a primitive-type array contains a variable of the array's declared type. Every element of a reference-type array is a reference to an object of the array's declared type.

Section 7.4 Examples Using Arrays

- A program can create an array and initialize its elements with an array initializer (p. 248).
- Constant variables (p. 250) are declared with keyword `final`, must be initialized before they're used and cannot be modified thereafter.

Section 7.5 Exception Handling: Processing the Incorrect Response

- An exception indicates a problem that occurs while a program executes. The name “exception” suggests that the problem occurs infrequently—if the “rule” is that a statement normally executes correctly, then the problem represents the “exception to the rule.”
- Exception handling (p. 256) enables you to create fault-tolerant programs.
- When a Java program executes, the JVM checks array indices to ensure that they're greater than or equal to 0 and less than the array's length. If a program uses an invalid index, Java generates an exception (p. 256) to indicate that an error occurred in the program at execution time.
- To handle an exception, place any code that might throw an exception (p. 256) in a `try` statement.
- The `try` block (p. 256) contains the code that might throw an exception, and the `catch` block (p. 256) contains the code that handles the exception if one occurs.
- You can have many `catch` blocks to handle different types of exceptions that might be thrown in the corresponding `try` block.
- When a `try` block terminates, any variables declared in the `try` block go out of scope.
- A `catch` block declares a type and an exception parameter. Inside the `catch` block, you can use the parameter's identifier to interact with a caught exception object.
- When a program is executed, array element indices are checked for validity—all indices must be greater than or equal to 0 and less than the length of the array. If an attempt is made to use an invalid index to access an element, an `ArrayIndexOutOfBoundsException` (p. 257) exception occurs.
- An exception object's `toString` method returns the exception's error message.

Section 7.6 Case Study: Card Shuffling and Dealing Simulation

- The `toString` method of an object is called implicitly when the object is used where a `String` is expected (e.g., when `printf` outputs the object as a `String` using the `%s` format specifier or when the object is concatenated to a `String` using the `+` operator).

Section 7.7 Enhanced `for` Statement

- The enhanced `for` statement (p. 262) allows you to iterate through the elements of an array or a collection without using a counter. The syntax of an enhanced `for` statement is:

```
for (parameter : arrayName)  
    statement
```

where *parameter* has a type and an identifier (e.g., `int number`), and *arrayName* is the array through which to iterate.

- The enhanced `for` statement cannot be used to modify elements in an array. If a program needs to modify elements, use the traditional counter-controlled `for` statement.

Section 7.8 Passing Arrays to Methods

- When an argument is passed by value, a copy of the argument's value is made and passed to the called method. The called method works exclusively with the copy.

Section 7.9 Pass-By-Value vs. Pass-By-Reference

- When an argument is passed by reference (p. 265), the called method can access the argument's value in the caller directly and possibly modify it.
- All arguments in Java are passed by value. A method call can pass two types of values to a method—copies of primitive values and copies of references to objects. Although an object's reference is passed by value (p. 265), a method can still interact with the referenced object by calling its `public` methods using the copy of the object's reference.
- To pass an object reference to a method, simply specify in the method call the name of the variable that refers to the object.
- When you pass an array or an individual array element of a reference type to a method, the called method receives a copy of the array or element's reference. When you pass an individual element of a primitive type, the called method receives a copy of the element's value.
- To pass an individual array element to a method, use the indexed name of the array.

Section 7.11 Multidimensional Arrays

- Multidimensional arrays with two dimensions are often used to represent tables of values consisting of information arranged in rows and columns.
- A two-dimensional array (p. 272) with *m* rows and *n* columns is called an *m*-by-*n* array. Such an array can be initialized with an array initializer of the form

```
arrayType[][] arrayName = { {row1 initializer}, {row2 initializer}, ...};
```

- Multidimensional arrays are maintained as arrays of separate one-dimensional arrays. As a result, the lengths of the rows in a two-dimensional array are not required to be the same.
- A multidimensional array with the same number of columns in every row can be created with an array-creation expression of the form

```
arrayType[][] arrayName = new arrayType[numRows][numColumns];
```

Section 7.13 Variable-Length Argument Lists

- An argument type followed by an ellipsis (`...`; p. 281) in a method's parameter list indicates that the method receives a variable number of arguments of that particular type. The ellipsis can occur only once in a method's parameter list. It must be at the end of the parameter list.
- A variable-length argument list (p. 281) is treated as an array within the method body. The number of arguments in the array can be obtained using the array's `length` field.

Section 7.14 Using Command-Line Arguments

- Passing arguments to `main` (p. 283) from the command line is achieved by including a parameter of type `String[]` in the parameter list of `main`. By convention, `main`'s parameter is named `args`.
- Java passes command-line arguments that appear after the class name in the `java` command to the application's `main` method as `Strings` in the array `args`.

Section 7.15 Class Arrays

- Class `Arrays` (p. 285) provides static methods that perform common array manipulations, including `sort` to sort an array, `binarySearch` to search a sorted array, `equals` to compare arrays and `fill` to place items in an array.
- Class `System`'s `arraycopy` method (p. 285) enables you to copy the elements of one array into another.

Section 7.16 Introduction to Collections and Class `ArrayList`

- The Java API's collection classes provide efficient methods that organize, store and retrieve data without requiring knowledge of how the data is being stored.
- An `ArrayList<T>` (p. 288) is similar to an array but can be dynamically resized.
- The `add` method (p. 290) with one argument appends an element to the end of an `ArrayList`.
- The `add` method with two arguments inserts a new element at a specified position in an `ArrayList`.
- The `size` method (p. 290) returns the number of elements currently in an `ArrayList`.
- The `remove` method with a reference to an object as an argument removes the first element that matches the argument's value.
- The `remove` method with an integer argument removes the element at the specified index, and all elements above that index are shifted down by one.
- The `contains` method returns `true` if the element is found in the `ArrayList`, and `false` otherwise.

Self-Review Exercises

- 7.1** Fill in the blank(s) in each of the following statements:
- Lists and tables of values can be stored in _____ and _____.
 - An array is a group of _____ (called elements or components) containing values that all have the same _____.
 - The _____ allows you to iterate through an array's elements without using a counter.
 - The number used to refer to a particular array element is called the element's _____.
 - An array that uses two indices is referred to as a(n) _____ array.
 - Use the enhanced for statement _____ to walk through double array numbers.
 - Command-line arguments are stored in _____.
 - Use the expression _____ to receive the total number of arguments in a command line. Assume that command-line arguments are stored in `String[] args`.
 - Given the command `java MyClass test`, the first command-line argument is _____.
 - A(n) _____ in the parameter list of a method indicates that the method can receive a variable number of arguments.
- 7.2** Determine whether each of the following is *true* or *false*. If *false*, explain why.
- An array can store many different types of values.
 - An array index should normally be of type `float`.
 - An individual array element that's passed to a method and modified in that method will contain the modified value when the called method completes execution.
 - Command-line arguments are separated by commas.
- 7.3** Perform the following tasks for an array called `fractions`:
- Declare a constant `ARRAY_SIZE` that's initialized to 10.
 - Declare an array with `ARRAY_SIZE` elements of type `double`, and initialize the elements to 0.
 - Refer to array element 4.
 - Assign the value 1.667 to array element 9.
 - Assign the value 3.333 to array element 6.

- f) Sum all the elements of the array, using a for statement. Declare the integer variable `x` as a control variable for the loop.
- 7.4** Perform the following tasks for an array called `table`:
- Declare and create the array as an integer array that has three rows and three columns. Assume that the constant `ARRAY_SIZE` has been declared to be 3.
 - How many elements does the array contain?
 - Use a for statement to initialize each element of the array to the sum of its indices. Assume that the integer variables `x` and `y` are declared as control variables.
- 7.5** Find and correct the error in each of the following program segments:
- `final int ARRAY_SIZE = 5;`
`ARRAY_SIZE = 10;`
 - Assume `int[] b = new int[10];`
`for (int i = 0; i <= b.length; i++)`
`b[i] = 1;`
 - Assume `int[][] a = {{1, 2}, {3, 4}};`
`a[1, 1] = 5;`

Answers to Self-Review Exercises

- 7.1** a) arrays, collections. b) variables, type. c) enhanced for statement. d) index (or subscript or position number). e) two-dimensional. f) for (double `d` : numbers). g) an array of Strings, called `args` by convention. h) `args.length`. i) test. j) ellipsis (...).
- 7.2** a) False. An array can store only values of the same type.
b) False. An array index must be an integer or an integer expression.
c) For individual primitive-type elements of an array: False. A called method receives and manipulates a copy of the value of such an element, so modifications do not affect the original value. If the reference of an array is passed to a method, however, modifications to the array elements made in the called method are indeed reflected in the original. For individual elements of a reference type: True. A called method receives a copy of the reference of such an element, and changes to the referenced object will be reflected in the original array element.
d) False. Command-line arguments are separated by white space.
- 7.3** a) `final int ARRAY_SIZE = 10;`
b) `double[] fractions = new double[ARRAY_SIZE];`
c) `fractions[4]`
d) `fractions[9] = 1.667;`
e) `fractions[6] = 3.333;`
f) `double total = 0.0;`
`for (int x = 0; x < fractions.length; x++)`
`total += fractions[x];`
- 7.4** a) `int[][] table = new int[ARRAY_SIZE][ARRAY_SIZE];`
b) Nine.
c) `for (int x = 0; x < table.length; x++)`
`for (int y = 0; y < table[x].length; y++)`
`table[x][y] = x + y;`
- 7.5** a) Error: Assigning a value to a constant after it has been initialized.
Correction: Assign the correct value to the constant in a `final int ARRAY_SIZE` declaration or declare another variable.

- b) Error: Referencing an array element outside the bounds of the array (b[10]).
Correction: Change the `<=` operator to `<`.
- c) Error: Array indexing is performed incorrectly.
Correction: Change the statement to `a[1][1] = 5;`.

Exercises

- 7.6** Fill in the blanks in each of the following statements:
- a) One-dimensional array `p` contains four elements. The names of those elements are _____, _____, _____ and _____.
 - b) Naming an array, stating its type and specifying the number of dimensions in the array is called _____ the array.
 - c) In a two-dimensional array, the first index identifies the _____ of an element and the second index identifies the _____ of an element.
 - d) An *m*-by-*n* array contains _____ rows, _____ columns and _____ elements.
 - e) The name of the element in row 3 and column 5 of array `d` is _____.
- 7.7** Determine whether each of the following is *true* or *false*. If *false*, explain why.
- a) To refer to a particular location or element within an array, we specify the name of the array and the value of the particular element.
 - b) An array declaration reserves space for the array.
 - c) To indicate that 100 locations should be reserved for integer array `p`, you write the declaration
`p[100];`
 - d) An application that initializes the elements of a 15-element array to zero must contain at least one `for` statement.
 - e) An application that totals the elements of a two-dimensional array must contain nested `for` statements.
- 7.8** Write Java statements to accomplish each of the following tasks:
- a) Display the value of element 6 of array `f`.
 - b) Initialize each of the five elements of one-dimensional integer array `g` to 8.
 - c) Total the 100 elements of floating-point array `c`.
 - d) Copy 11-element array `a` into the first portion of array `b`, which contains 34 elements.
 - e) Determine and display the smallest and largest values contained in 99-element floating-point array `w`.
- 7.9** Consider a two-by-three integer array `t`.
- a) Write a statement that declares and creates `t`.
 - b) How many rows does `t` have?
 - c) How many columns does `t` have?
 - d) How many elements does `t` have?
 - e) Write access expressions for all the elements in row 1 of `t`.
 - f) Write access expressions for all the elements in column 2 of `t`.
 - g) Write a single statement that sets the element of `t` in row 0 and column 1 to zero.
 - h) Write individual statements to initialize each element of `t` to zero.
 - i) Write a nested `for` statement that initializes each element of `t` to zero.
 - j) Write a nested `for` statement that inputs the values for the elements of `t` from the user.
 - k) Write a series of statements that determines and displays the smallest value in `t`.
 - l) Write a single `printf` statement that displays the elements of the first row of `t`.
 - m) Write a statement that totals the elements of the third column of `t`. Do not use repetition.
 - n) Write a series of statements that displays the contents of `t` in tabular format. List the column indices as headings across the top, and list the row indices at the left of each row.

7.10 (*Sales Commissions*) Use a one-dimensional array to solve the following problem: A company pays its salespeople on a commission basis. The salespeople receive \$200 per week plus 9% of their gross sales for that week. For example, a salesperson who grosses \$5,000 in sales in a week receives \$200 plus 9% of \$5,000, or a total of \$650. Write an application (using an array of counters) that determines how many of the salespeople earned salaries in each of the following ranges (assume that each salesperson's salary is truncated to an integer amount):

- a) \$200–299
- b) \$300–399
- c) \$400–499
- d) \$500–599
- e) \$600–699
- f) \$700–799
- g) \$800–899
- h) \$900–999
- i) \$1,000 and over

Summarize the results in tabular format.

7.11 Write statements that perform the following one-dimensional-array operations:

- a) Set the 10 elements of integer array `counts` to zero.
- b) Add one to each of the 15 elements of integer array `bonus`.
- c) Display the five values of integer array `bestScores` in column format.

7.12 (*Duplicate Elimination*) Use a one-dimensional array to solve the following problem: Write an application that inputs five numbers, each between 10 and 100, inclusive. As each number is read, display it only if it's not a duplicate of a number already read. Provide for the "worst case," in which all five numbers are different. Use the smallest possible array to solve this problem. Display the complete set of unique values input after the user enters each new value.

7.13 Label the elements of three-by-five two-dimensional array `sales` to indicate the order in which they're set to zero by the following program segment:

```
for (int row = 0; row < sales.length; row++)
{
    for (int col = 0; col < sales[row].length; col++)
    {
        sales[row][col] = 0;
    }
}
```

7.14 (*Variable-Length Argument List*) Write an application that calculates the product of a series of integers that are passed to method `product` using a variable-length argument list. Test your method with several calls, each with a different number of arguments.

7.15 (*Command-Line Arguments*) Rewrite Fig. 7.2 so that the size of the array is specified by the first command-line argument. If no command-line argument is supplied, use 10 as the default size of the array.

7.16 (*Using the Enhanced for Statement*) Write an application that uses an enhanced for statement to sum the double values passed by the command-line arguments. [Hint: Use the static method `parseDouble` of class `Double` to convert a `String` to a double value.]

7.17 (*Dice Rolling*) Write an application to simulate the rolling of two dice. The application should use an object of class `Random` once to roll the first die and again to roll the second die. The sum of the two values should then be calculated. Each die can show an integer value from 1 to 6, so the sum of the values will vary from 2 to 12, with 7 being the most frequent sum, and 2 and 12 the least frequent. Figure 7.28 shows the 36 possible combinations of the two dice. Your application

should roll the dice 36,000,000 times. Use a one-dimensional array to tally the number of times each possible sum appears. Display the results in tabular format.

	1	2	3	4	5	6
1	2	3	4	5	6	7
2	3	4	5	6	7	8
3	4	5	6	7	8	9
4	5	6	7	8	9	10
5	6	7	8	9	10	11
6	7	8	9	10	11	12

Fig. 7.28 | The 36 possible sums of two dice.

7.18 (*Game of Craps*) Write an application that runs 1,000,000 games of craps (Fig. 6.8) and answers the following questions:

- How many games are won on the first roll, second roll, ..., twentieth roll and after the twentieth roll?
- How many games are lost on the first roll, second roll, ..., twentieth roll and after the twentieth roll?
- What are the chances of winning at craps? [Note: You should discover that craps is one of the fairest casino games. What do you suppose this means?]
- What is the average length of a game of craps?
- Do the chances of winning improve with the length of the game?

7.19 (*Airline Reservations System*) A small airline has just purchased a computer for its new automated reservations system. You've been asked to develop the new system. You're to write an application to assign seats on each flight of the airline's only plane (capacity: 10 seats).

Your application should display the following alternatives: Please type 1 for First Class and Please type 2 for Economy. If the user types 1, your application should assign a seat in the first-class section (seats 1–5). If the user types 2, your application should assign a seat in the economy section (seats 6–10). Your application should then display a boarding pass indicating the person's seat number and whether it's in the first-class or economy section of the plane.

Use a one-dimensional array of primitive type `boolean` to represent the seating chart of the plane. Initialize all the elements of the array to `false` to indicate that all the seats are empty. As each seat is assigned, set the corresponding element of the array to `true` to indicate that the seat is no longer available.

Your application should never assign a seat that has already been assigned. When the economy section is full, your application should ask the person if it's acceptable to be placed in the first-class section (and vice versa). If yes, make the appropriate seat assignment. If no, display the message "Next flight leaves in 3 hours."

7.20 (*Total Sales*) Use a two-dimensional array to solve the following problem: A company has four salespeople (1 to 4) who sell five different products (1 to 5). Once a day, each salesperson passes in a slip for each type of product sold. Each slip contains the following:

- The salesperson number
- The product number
- The total dollar value of that product sold that day

Thus, each salesperson passes in between 0 and 5 sales slips per day. Assume that the information from all the slips for last month is available. Write an application that will read all this information for

last month's sales and summarize the total sales by salesperson and by product. All totals should be stored in the two-dimensional array `sales`. After processing all the information for last month, display the results in tabular format, with each column representing a salesperson and each row representing a particular product. Cross-total each row to get the total sales of each product for last month. Cross-total each column to get the total sales by salesperson for last month. Your output should include these cross-totals to the right of the totaled rows and to the bottom of the totaled columns.

7.21 (Turtle Graphics) The Logo language made the concept of *turtle graphics* famous. Imagine a mechanical turtle that walks around the room under the control of a Java application. The turtle holds a pen in one of two positions, up or down. While the pen is down, the turtle traces out shapes as it moves, and while the pen is up, the turtle moves about freely without writing anything. In this problem, you'll simulate the operation of the turtle and create a computerized sketchpad.

Use a 20-by-20 array `floor` that's initialized to zeros. Read commands from an array that contains them. Keep track of the current position of the turtle at all times and whether the pen is currently up or down. Assume that the turtle always starts at position (0, 0) of the floor with its pen up. The set of turtle commands your application must process are shown in Fig. 7.29.

Command	Meaning
1	Pen up
2	Pen down
3	Turn right
4	Turn left
5, 10	Move forward 10 spaces (replace 10 for a different number of spaces)
6	Display the 20-by-20 array
9	End of data (sentinel)

Fig. 7.29 | Turtle graphics commands.

Suppose that the turtle is somewhere near the center of the floor. The following “program” would draw and display a 12-by-12 square, leaving the pen in the up position:

```

2
5, 12
3
5, 12
3
5, 12
3
5, 12
1
6
9

```

As the turtle moves with the pen down, set the appropriate elements of array `floor` to 1s. When the 6 command (display the array) is given, wherever there's a 1 in the array, display an asterisk or any character you choose. Wherever there's a 0, display a blank.

Write an application to implement the turtle graphics capabilities discussed here. Write several turtle graphics programs to draw interesting shapes. Add other commands to increase the power of your turtle graphics language.

7.22 (Knight's Tour) An interesting puzzler for chess buffs is the Knight's Tour problem, originally proposed by the mathematician Euler. Can the knight piece move around an empty chessboard and touch each of the 64 squares once and only once? We study this intriguing problem in depth here.

The knight makes only L-shaped moves (two spaces in one direction and one space in a perpendicular direction). Thus, as shown in Fig. 7.30, from a square near the middle of an empty chessboard, the knight (labeled K) can make eight different moves (numbered 0 through 7).

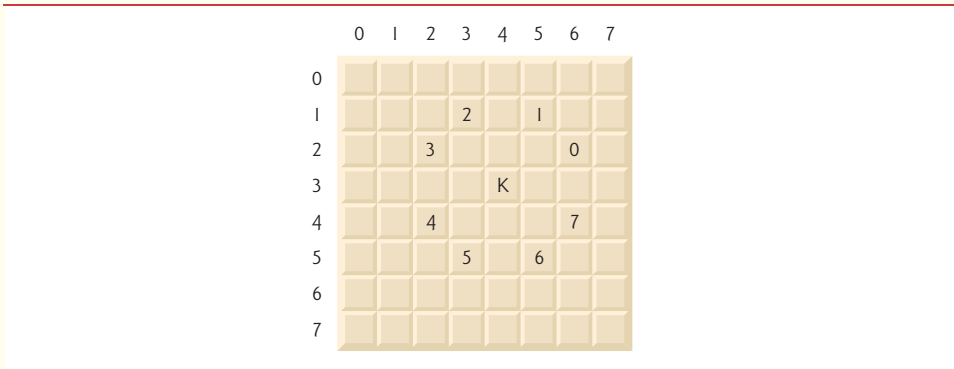


Fig. 7.30 | The eight possible moves of the knight.

- Draw an eight-by-eight chessboard on a sheet of paper, and attempt a Knight's Tour by hand. Put a 1 in the starting square, a 2 in the second square, a 3 in the third, and so on. Before starting the tour, estimate how far you think you'll get, remembering that a full tour consists of 64 moves. How far did you get? Was this close to your estimate?
- Now let's develop an application that will move the knight around a chessboard. The board is represented by an eight-by-eight two-dimensional array board. Each square is initialized to zero. We describe each of the eight possible moves in terms of its horizontal and vertical components. For example, a move of type 0, as shown in Fig. 7.30, consists of moving two squares horizontally to the right and one square vertically upward. A move of type 2 consists of moving one square horizontally to the left and two squares vertically upward. Horizontal moves to the left and vertical moves upward are indicated with negative numbers. The eight moves may be described by two one-dimensional arrays, `horizontal` and `vertical`, as follows:

```
horizontal[0] = 2      vertical[0] = -1
horizontal[1] = 1      vertical[1] = -2
horizontal[2] = -1     vertical[2] = -2
horizontal[3] = -2     vertical[3] = -1
horizontal[4] = -2     vertical[4] = 1
horizontal[5] = -1     vertical[5] = 2
horizontal[6] = 1      vertical[6] = 2
horizontal[7] = 2      vertical[7] = 1
```

Let the variables `currentRow` and `currentColumn` indicate the row and column, respectively, of the knight's current position. To make a move of type `moveNumber`, where `moveNumber` is between 0 and 7, your application should use the statements

```
currentRow += vertical[moveNumber];
currentColumn += horizontal[moveNumber];
```

Write an application to move the knight around the chessboard. Keep a counter that varies from 1 to 64. Record the latest count in each square the knight moves to.

Test each potential move to see if the knight has already visited that square. Test every potential move to ensure that the knight does not land off the chessboard. Run the application. How many moves did the knight make?

- c) After attempting to write and run a Knight's Tour application, you've probably developed some valuable insights. We'll use these insights to develop a *heuristic* (i.e., a common-sense rule) for moving the knight. Heuristics do not guarantee success, but a carefully developed heuristic greatly improves the chance of success. You may have observed that the outer squares are more troublesome than the squares nearer the center of the board. In fact, the most troublesome or inaccessible squares are the four corners.

Intuition may suggest that you should attempt to move the knight to the most troublesome squares first and leave open those that are easiest to get to, so that when the board gets congested near the end of the tour, there will be a greater chance of success.

We could develop an "accessibility heuristic" by classifying each of the squares according to how accessible it is and always moving the knight (using the knight's L-shaped moves) to the most inaccessible square. We label a two-dimensional array *accessibility* with numbers indicating from how many squares each particular square is accessible. On a blank chessboard, each of the 16 squares nearest the center is rated as 8, each corner square is rated as 2, and the other squares have accessibility numbers of 3, 4 or 6 as follows:

2	3	4	4	4	4	3	2
3	4	6	6	6	6	4	3
4	6	8	8	8	8	6	4
4	6	8	8	8	8	6	4
4	6	8	8	8	8	6	4
4	6	8	8	8	8	6	4
3	4	6	6	6	6	4	3
2	3	4	4	4	4	3	2

Write a new version of the Knight's Tour, using the accessibility heuristic. The knight should always move to the square with the lowest accessibility number. In case of a tie, the knight may move to any of the tied squares. Therefore, the tour may begin in any of the four corners. [Note: As the knight moves around the chessboard, your application should reduce the accessibility numbers as more squares become occupied. In this way, at any given time during the tour, each available square's accessibility number will remain equal to precisely the number of squares from which that square may be reached.] Run this version of your application. Did you get a full tour? Modify the application to run 64 tours, one starting from each square of the chessboard. How many full tours did you get?

- d) Write a version of the Knight's Tour application that, when encountering a tie between two or more squares, decides what square to choose by looking ahead to those squares reachable from the "tied" squares. Your application should move to the tied square for which the next move would arrive at a square with the lowest accessibility number.

7.23 (Knight's Tour: Brute-Force Approaches) In part (c) of Exercise 7.22, we developed a solution to the Knight's Tour problem. The approach used, called the "accessibility heuristic," generates many solutions and executes efficiently.

As computers continue to increase in power, we'll be able to solve more problems with sheer computer power and relatively unsophisticated algorithms. Let's call this approach "brute-force" problem solving.

- a) Use random-number generation to enable the knight to walk around the chessboard (in its legitimate L-shaped moves) at random. Your application should run one tour and display the final chessboard. How far did the knight get?

- b) Most likely, the application in part (a) produced a relatively short tour. Now modify your application to attempt 1,000 tours. Use a one-dimensional array to keep track of the number of tours of each length. When your application finishes attempting the 1,000 tours, it should display this information in neat tabular format. What was the best result?
- c) Most likely, the application in part (b) gave you some “respectable” tours, but no full tours. Now let your application run until it produces a full tour. [Caution: This version of the application could run for hours on a powerful computer.] Once again, keep a table of the number of tours of each length, and display this table when the first full tour is found. How many tours did your application attempt before producing a full tour? How much time did it take?
- d) Compare the brute-force version of the Knight’s Tour with the accessibility-heuristic version. Which required a more careful study of the problem? Which algorithm was more difficult to develop? Which required more computer power? Could we be certain (in advance) of obtaining a full tour with the accessibility-heuristic approach? Could we be certain (in advance) of obtaining a full tour with the brute-force approach? Argue the pros and cons of brute-force problem solving in general.

7.24 (Eight Queens) Another puzzler for chess buffs is the Eight Queens problem, which asks the following: Is it possible to place eight queens on an empty chessboard so that no queen is “attacking” any other (i.e., no two queens are in the same row, in the same column or along the same diagonal)? Use the thinking developed in Exercise 7.22 to formulate a heuristic for solving the Eight Queens problem. Run your application. [Hint: It’s possible to assign a value to each square of the chessboard to indicate how many squares of an empty chessboard are “eliminated” if a queen is placed in that square. Each of the corners would be assigned the value 22, as demonstrated by Fig. 7.31. Once these “elimination numbers” are placed in all 64 squares, an appropriate heuristic might be as follows: Place the next queen in the square with the smallest elimination number. Why is this strategy intuitively appealing?]

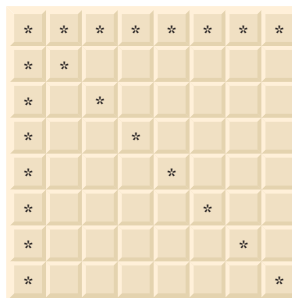


Fig. 7.31 | The 22 squares eliminated by placing a queen in the upper left corner.

7.25 (Eight Queens: Brute-Force Approaches) In this exercise, you’ll develop several brute-force approaches to solving the Eight Queens problem introduced in Exercise 7.24.

- a) Use the random brute-force technique developed in Exercise 7.23 to solve the Eight Queens problem.
- b) Use an exhaustive technique (i.e., try all possible combinations of eight queens on the chessboard) to solve the Eight Queens problem.
- c) Why might the exhaustive brute-force approach not be appropriate for solving the Knight’s Tour problem?
- d) Compare and contrast the random brute-force and exhaustive brute-force approaches.

7.26 (Knight's Tour: Closed-Tour Test) In the Knight's Tour (Exercise 7.22), a full tour occurs when the knight makes 64 moves, touching each square of the chessboard once and only once. A closed tour occurs when the 64th move is one move away from the square in which the knight started the tour. Modify the application you wrote in Exercise 7.22 to test for a closed tour if a full tour has occurred.

7.27 (Sieve of Eratosthenes) A prime number is any integer greater than 1 that's evenly divisible only by itself and 1. The Sieve of Eratosthenes is a method of finding prime numbers. It operates as follows:

- Create a primitive-type `boolean` array with all elements initialized to `true`. Array elements with prime indices will remain `true`. All other array elements will eventually be set to `false`.
- Starting with array index 2, determine whether a given element is `true`. If so, loop through the remainder of the array and set to `false` every element whose index is a multiple of the index for the element with value `true`. Then continue the process with the next element with value `true`. For array index 2, all elements beyond element 2 in the array that have indices which are multiples of 2 (indices 4, 6, 8, 10, etc.) will be set to `false`; for array index 3, all elements beyond element 3 in the array that have indices which are multiples of 3 (indices 6, 9, 12, 15, etc.) will be set to `false`; and so on.

When this process completes, the array elements that are still `true` indicate that the index is a prime number. These indices can be displayed. Write an application that uses an array of 1,000 elements to determine and display the prime numbers between 2 and 999. Ignore array elements 0 and 1.

7.28 (Simulation: The Tortoise and the Hare) In this problem, you'll re-create the classic race of the tortoise and the hare. You'll use random-number generation to develop a simulation of this memorable event.

Our contenders begin the race at square 1 of 70 squares. Each square represents a possible position along the race course. The finish line is at square 70. The first contender to reach or pass square 70 is rewarded with a pail of fresh carrots and lettuce. The course weaves its way up the side of a slippery mountain, so occasionally the contenders lose ground.

A clock ticks once per second. With each tick of the clock, your application should adjust the position of the animals according to the rules in Fig. 7.32. Use variables to keep track of the positions of the animals (i.e., position numbers are 1–70). Start each animal at position 1 (the "starting gate"). If an animal slips left before square 1, move it back to square 1.

Animal	Move type	Percentage of the time	Actual move
Tortoise	Fast plod	50%	3 squares to the right
	Slip	20%	6 squares to the left
	Slow plod	30%	1 square to the right
Hare	Sleep	20%	No move at all
	Big hop	20%	9 squares to the right
	Big slip	10%	12 squares to the left
	Small hop	30%	1 square to the right
	Small slip	20%	2 squares to the left

Fig. 7.32 | Rules for adjusting the positions of the tortoise and the hare.

Generate the percentages in Fig. 7.32 by producing a random integer i in the range $1 \leq i \leq 10$. For the tortoise, perform a “fast plod” when $1 \leq i \leq 5$, a “slip” when $6 \leq i \leq 7$ or a “slow plod” when $8 \leq i \leq 10$. Use a similar technique to move the hare.

Begin the race by displaying

```
BANG !!!!!
AND THEY'RE OFF !!!!!
```

Then, for each tick of the clock (i.e., each repetition of a loop), display a 70-position line showing the letter T in the position of the tortoise and the letter H in the position of the hare. Occasionally, the contenders will land on the same square. In this case, the tortoise bites the hare, and your application should display OUCH!!! beginning at that position. All output positions other than the T, the H or the OUCH!!! (in case of a tie) should be blank.

After each line is displayed, test for whether either animal has reached or passed square 70. If so, display the winner and terminate the simulation. If the tortoise wins, display TORTOISE WINS!!! YAY!!! If the hare wins, display Hare wins. Yuch. If both animals win on the same tick of the clock, you may want to favor the tortoise (the “underdog”), or you may want to display It's a tie. If neither animal wins, perform the loop again to simulate the next tick of the clock. When you're ready to run your application, assemble a group of fans to watch the race. You'll be amazed at how involved your audience gets!

Later in the book, we introduce a number of Java capabilities, such as graphics, images, animation, sound and multithreading. As you study those features, you might enjoy enhancing your tortoise-and-hare contest simulation.

7.29 (*Fibonacci Series*) The Fibonacci series

0, 1, 1, 2, 3, 5, 8, 13, 21, ...

begins with the terms 0 and 1 and has the property that each succeeding term is the sum of the two preceding terms.

- Write a method `fibonacci(n)` that calculates the n th Fibonacci number. Incorporate this method into an application that enables the user to enter the value of n .
- Determine the largest Fibonacci number that can be displayed on your system.
- Modify the application you wrote in part (a) to use `double` instead of `int` to calculate and return Fibonacci numbers, and use this modified application to repeat part (b).

Exercises 7.30–7.34 are reasonably challenging. Once you've done them, you ought to be able to implement most popular card games easily.

7.30 (*Card Shuffling and Dealing*) Modify the application of Fig. 7.11 to deal a five-card poker hand. Then modify class `DeckOfCards` of Fig. 7.10 to include methods that determine whether a hand contains

- a pair
- two pairs
- three of a kind (e.g., three jacks)
- four of a kind (e.g., four aces)
- a flush (i.e., all five cards of the same suit)
- a straight (i.e., five cards of consecutive face values)
- a full house (i.e., two cards of one face value and three cards of another face value)

[Hint: Add methods `getFace` and `getSuit` to class `Card` of Fig. 7.9.]

7.31 (*Card Shuffling and Dealing*) Use the methods developed in Exercise 7.30 to write an application that deals two five-card poker hands, evaluates each hand and determines which is better.

7.32 (*Project: Card Shuffling and Dealing*) Modify the application developed in Exercise 7.31 so that it can simulate the dealer. The dealer's five-card hand is dealt "face down," so the player cannot see it. The application should then evaluate the dealer's hand, and, based on the quality of the hand, the dealer should draw one, two or three more cards to replace the corresponding number of unneeded cards in the original hand. The application should then reevaluate the dealer's hand. [Caution: This is a difficult problem!]

7.33 (*Project: Card Shuffling and Dealing*) Modify the application developed in Exercise 7.32 so that it can handle the dealer's hand automatically, but the player is allowed to decide which cards of the player's hand to replace. The application should then evaluate both hands and determine who wins. Now use this new application to play 20 games against the computer. Who wins more games, you or the computer? Have a friend play 20 games against the computer. Who wins more games? Based on the results of these games, refine your poker-playing application. (This, too, is a difficult problem.) Play 20 more games. Does your modified application play a better game?

7.34 (*Project: Card Shuffling and Dealing*) Modify the application of Figs. 7.9–7.11 to use `Face` and `Suit` enum types to represent the faces and suits of the cards. Declare each of these enum types as a `public` type in its own source-code file. Each `Card` should have a `Face` and a `Suit` instance variable. These should be initialized by the `Card` constructor. In class `DeckOfCards`, create an array of `Faces` that's initialized with the names of the constants in the `Face` enum type and an array of `Suits` that's initialized with the names of the constants in the `Suit` enum type. [Note: When you output an enum constant as a `String`, the name of the constant is displayed.]

7.35 (*Fisher-Yates Shuffling Algorithm*) Research the Fisher-Yates shuffling algorithm online, then use it to reimplement the `shuffle` method in Fig. 7.10.

Special Section: Building Your Own Computer

In the next several problems, we take a temporary diversion from the world of high-level language programming to "peel open" a computer and look at its internal structure. We introduce machine-language programming and write several machine-language programs. To make this an especially valuable experience, we then build a computer (through the technique of software-based *simulation*) on which you can execute your machine-language programs.

7.36 (*Machine-Language Programming*) Let's create a computer called the Simpletron. As its name implies, it's a simple machine, but powerful. The Simpletron runs programs written in the only language it directly understands: Simpletron Machine Language (SML).

The Simpletron contains an *accumulator*—a special register in which information is put before the Simpletron uses that information in calculations or examines it in various ways. All the information in the Simpletron is handled in terms of *words*. A word is a signed four-digit decimal number, such as +3364, -1293, +0007 and -0001. The Simpletron is equipped with a 100-word memory, and these words are referenced by their location numbers 00, 01, ..., 99.

Before running an SML program, we must *load*, or *place*, the program into memory. The first instruction (or statement) of every SML program is always placed in location 00. The simulator will start executing at this location.

Each instruction written in SML occupies one word of the Simpletron's memory (so instructions are signed four-digit decimal numbers). We shall assume that the sign of an SML instruction is always plus, but the sign of a data word may be either plus or minus. Each location in the Simpletron's memory may contain an instruction, a data value used by a program or an unused (and so undefined) area of memory. The first two digits of each SML instruction are the *operation code* specifying the operation to be performed. SML operation codes are summarized in Fig. 7.33.

The last two digits of an SML instruction are the *operand*—the address of the memory location containing the word to which the operation applies. Let's consider several simple SML programs.

Operation code	Meaning
<i>Input/output operations:</i>	
<code>final int READ = 10;</code>	Read a word from the keyboard into a specific location in memory.
<code>final int WRITE = 11;</code>	Write a word from a specific location in memory to the screen.
<i>Load/store operations:</i>	
<code>final int LOAD = 20;</code>	Load a word from a specific location in memory into the accumulator.
<code>final int STORE = 21;</code>	Store a word from the accumulator into a specific location in memory.
<i>Arithmetic operations:</i>	
<code>final int ADD = 30;</code>	Add a word from a specific location in memory to the word in the accumulator (leave the result in the accumulator).
<code>final int SUBTRACT = 31;</code>	Subtract a word from a specific location in memory from the word in the accumulator (leave the result in the accumulator).
<code>final int DIVIDE = 32;</code>	Divide a word from a specific location in memory into the word in the accumulator (leave result in the accumulator).
<code>final int MULTIPLY = 33;</code>	Multiply a word from a specific location in memory by the word in the accumulator (leave the result in the accumulator).
<i>Transfer-of-control operations:</i>	
<code>final int BRANCH = 40;</code>	Branch to a specific location in memory.
<code>final int BRANCHNEG = 41;</code>	Branch to a specific location in memory if the accumulator is negative.
<code>final int BRANCHZERO = 42;</code>	Branch to a specific location in memory if the accumulator is zero.
<code>final int HALT = 43;</code>	Halt. The program has completed its task.

Fig. 7.33 | Simpletron Machine Language (SML) operation codes.

The first SML program (Fig. 7.34) reads two numbers from the keyboard and computes and displays their sum. The instruction +1007 reads the first number from the keyboard and places it into location 07 (which has been initialized to 0). Then instruction +1008 reads the next number into location 08. The *load* instruction, +2007, puts the first number into the accumulator, and the *add* instruction, +3008, adds the second number to the number in the accumulator. *All SML arithmetic instructions leave their results in the accumulator.* The *store* instruction, +2109, places the result back into memory location 09, from which the *write* instruction, +1109, takes the number and displays it (as a signed four-digit decimal number). The *halt* instruction, +4300, terminates execution.

Location	Number	Instruction
00	+1007	(Read A)
01	+1008	(Read B)
02	+2007	(Load A)
03	+3008	(Add B)
04	+2109	(Store C)

Fig. 7.34 | SML program that reads two integers and computes their sum. (Part I of 2.)

Location	Number	Instruction
05	+1109	(Write C)
06	+4300	(Halt)
07	+0000	(Variable A)
08	+0000	(Variable B)
09	+0000	(Result C)

Fig. 7.34 | SML program that reads two integers and computes their sum. (Part 2 of 2.)

The second SML program (Fig. 7.35) reads two numbers from the keyboard and determines and displays the larger value. Note the use of the instruction +4107 as a conditional transfer of control, much the same as Java's `if` statement.

Location	Number	Instruction
00	+1009	(Read A)
01	+1010	(Read B)
02	+2009	(Load A)
03	+3110	(Subtract B)
04	+4107	(Branch negative to 07)
05	+1109	(Write A)
06	+4300	(Halt)
07	+1110	(Write B)
08	+4300	(Halt)
09	+0000	(Variable A)
10	+0000	(Variable B)

Fig. 7.35 | SML program that reads two integers and determines the larger.

Now write SML programs to accomplish each of the following tasks:

- Use a sentinel-controlled loop to read 10 positive numbers. Compute and display their sum.
- Use a counter-controlled loop to read seven numbers, some positive and some negative, and compute and display their average.
- Read a series of numbers, and determine and display the largest number. The first number read indicates how many numbers should be processed.

7.37 (Computer Simulator) In this problem, you're going to build your own computer. No, you'll not be soldering components together. Rather, you'll use the powerful technique of *software-based simulation* to create an object-oriented *software model* of the Simpletron of Exercise 7.36. Your Simpletron simulator will turn the computer you're using into a Simpletron, and you'll actually be able to run, test and debug the SML programs you wrote in Exercise 7.36.

When you run your Simpletron simulator, it should begin by displaying:

```
*** Welcome to Simpletron! ***
*** Please enter your program one instruction ***
*** (or data word) at a time. I will display ***
*** the location number and a question mark (?). ***
*** You then type the word for that location. ***
*** Type -99999 to stop entering your program. ***
```

Your application should simulate the memory of the Simpletron with a one-dimensional array `memory` that has 100 elements. Now assume that the simulator is running, and let's examine the dialog as we enter the program of Fig. 7.35 (Exercise 7.36):

```
00 ? +1009
01 ? +1010
02 ? +2009
03 ? +3110
04 ? +4107
05 ? +1109
06 ? +4300
07 ? +1110
08 ? +4300
09 ? +0000
10 ? +0000
11 ? -99999
```

Your program should display the memory location followed by a question mark. Each value to the right of a question mark is input by the user. When the sentinel value -99999 is input, the program should display the following:

```
*** Program loading completed ***
*** Program execution begins ***
```

The SML program has now been placed (or loaded) in array memory. Now the Simpletron executes the SML program. Execution begins with the instruction in location 00 and, as in Java, continues sequentially, unless directed to some other part of the program by a transfer of control.

Use the variable `accumulator` to represent the accumulator register. Use the variable `instructionCounter` to keep track of the location in memory that contains the instruction being performed. Use the variable `operationCode` to indicate the operation currently being performed (i.e., the left two digits of the instruction word). Use the variable `operand` to indicate the memory location on which the current instruction operates. Thus, `operand` is the rightmost two digits of the instruction currently being performed. Do not execute instructions directly from memory. Rather, transfer the next instruction to be performed from memory to a variable called `instructionRegister`. Then pick off the left two digits and place them in `operationCode`, and pick off the right two digits and place them in `operand`. When the Simpletron begins execution, the special registers are all initialized to zero.

Now, let's walk through execution of the first SML instruction, +1009 in memory location 00. This procedure is called an *instruction-execution cycle*.

The `instructionCounter` tells us the location of the next instruction to be performed. We *fetch* the contents of that location from memory by using the Java statement

```
instructionRegister = memory[instructionCounter];
```

The operation code and the operand are extracted from the instruction register by the statements

```
operationCode = instructionRegister / 100;
operand = instructionRegister % 100;
```

Now the Simpletron must determine that the operation code is actually a *read* (versus a *write*, a *load*, and so on). A switch differentiates among the 12 operations of SML. In the switch statement, the behavior of various SML instructions is simulated as shown in Fig. 7.36. We discuss branch instructions shortly and leave the others to you.

When the SML program completes execution, the name and contents of each register as well as the complete contents of memory should be displayed. Such a printout is often called a computer dump (no, a computer dump is not a place where old computers go). To help you program your dump method, a sample dump format is shown in Fig. 7.37. A dump after executing a Simpletron program would show the actual values of instructions and data values at the moment execution terminated.

Instruction	Description
<i>read:</i>	Display the prompt "Enter an integer", then input the integer and store it in location memory[operand].
<i>load:</i>	accumulator = memory[operand];
<i>add:</i>	accumulator += memory[operand];
<i>halt:</i>	This instruction displays the message *** Simpletron execution terminated ***

Fig. 7.36 | Behavior of several SML instructions in the Simpletron.

```

REGISTERS:
accumulator      +0000
instructionCounter 00
instructionRegister +0000
operationCode     00
operand          00

MEMORY:
0 +0000 +0000 +0000 +0000 +0000 +0000 +0000 +0000 +0000
10 +0000 +0000 +0000 +0000 +0000 +0000 +0000 +0000 +0000
20 +0000 +0000 +0000 +0000 +0000 +0000 +0000 +0000 +0000
30 +0000 +0000 +0000 +0000 +0000 +0000 +0000 +0000 +0000
40 +0000 +0000 +0000 +0000 +0000 +0000 +0000 +0000 +0000
50 +0000 +0000 +0000 +0000 +0000 +0000 +0000 +0000 +0000
60 +0000 +0000 +0000 +0000 +0000 +0000 +0000 +0000 +0000
70 +0000 +0000 +0000 +0000 +0000 +0000 +0000 +0000 +0000
80 +0000 +0000 +0000 +0000 +0000 +0000 +0000 +0000 +0000
90 +0000 +0000 +0000 +0000 +0000 +0000 +0000 +0000 +0000

```

Fig. 7.37 | A sample dump.

Let's proceed with the execution of our program's first instruction—namely, the +1009 in location 00. As we've indicated, the `switch` statement simulates this task by prompting the user to enter a value, reading the value and storing it in memory location `memory[operand]`. The value is then read into location 09.

At this point, simulation of the first instruction is completed. All that remains is to prepare the Simpletron to execute the next instruction. Since the instruction just performed was not a transfer of control, we need merely increment the instruction-counter register as follows:

```
instructionCounter++;
```

This action completes the simulated execution of the first instruction. The entire process (i.e., the instruction-execution cycle) begins anew with the fetch of the next instruction to execute.

Now let's consider how the branching instructions—the transfers of control—are simulated. All we need to do is adjust the value in the instruction counter appropriately. Therefore, the unconditional branch instruction (40) is simulated within the `switch` as

```
instructionCounter = operand;
```

The conditional "branch if accumulator is zero" instruction is simulated as

```
if (accumulator == 0)
    instructionCounter = operand;
```

At this point, you should implement your Simpletron simulator and run each of the SML programs you wrote in Exercise 7.36. If you desire, you may embellish SML with additional features and provide for these features in your simulator.

Your simulator should check for various types of errors. During the program-loading phase, for example, each number the user types into the Simpletron's memory must be in the range -9999 to +9999. Your simulator should test that each number entered is in this range and, if not, keep prompting the user to re-enter the number until the user enters a correct number.

During the execution phase, your simulator should check for various serious errors, such as attempts to divide by zero, attempts to execute invalid operation codes, and accumulator overflows (i.e., arithmetic operations resulting in values larger than +9999 or smaller than -9999). Such serious errors are called *fatal errors*. When a fatal error is detected, your simulator should display an error message, such as

```
*** Attempt to divide by zero ***
*** Simpletron execution abnormally terminated ***
```

and should display a full computer dump in the format we discussed previously. This treatment will help the user locate the error in the program.

7.38 (Simpletron Simulator Modifications) In Exercise 7.37, you wrote a software simulation of a computer that executes programs written in Simpletron Machine Language (SML). In this exercise, we propose several modifications and enhancements to the Simpletron Simulator. In the exercises of Chapter 21, we propose building a compiler that converts programs written in a high-level programming language (a variation of Basic) to Simpletron Machine Language. Some of the following modifications and enhancements may be required to execute the programs produced by the compiler:

- a) Extend the Simpletron Simulator's memory to contain 1,000 memory locations to enable the Simpletron to handle larger programs.
- b) Allow the simulator to perform remainder calculations. This modification requires an additional SML instruction.
- c) Allow the simulator to perform exponentiation calculations. This modification requires an additional SML instruction.
- d) Modify the simulator to use hexadecimal rather than integer values to represent SML instructions.
- e) Modify the simulator to allow output of a newline. This modification requires an additional SML instruction.
- f) Modify the simulator to process floating-point values in addition to integer values.
- g) Modify the simulator to handle string input. [*Hint:* Each Simpletron word can be divided into two groups, each holding a two-digit integer. Each two-digit integer represents the ASCII (see Appendix B) decimal equivalent of a character. Add a machine-language instruction that will input a string and store the string, beginning at a specific Simpletron memory location. The first half of the word at that location will be a count of the number of characters in the string (i.e., the length of the string). Each succeeding half-word contains one ASCII character expressed as two decimal digits. The machine-language instruction converts each character into its ASCII equivalent and assigns it to a half-word.]
- h) Modify the simulator to handle output of strings stored in the format of part (g). [*Hint:* Add a machine-language instruction that will display a string, beginning at a certain Simpletron memory location. The first half of the word at that location is a count of the number of characters in the string (i.e., the length of the string). Each succeeding half-word contains one ASCII character expressed as two decimal digits. The machine-language instruction checks the length and displays the string by translating each two-digit number into its equivalent character.]

7.39 (*Enhanced GradeBook*) Modify the `GradeBook` class of Fig. 7.18 so that the constructor accepts as parameters the number of students and the number of exams, then builds an appropriately sized two-dimensional array, rather than receiving a preinitialized two-dimensional array as it does now. Set each element of the new two-dimensional array to -1 to indicate that no grade has been entered for that element. Add a `setGrade` method that sets one grade for a particular student on a particular exam. Modify class `GradeBookTest` of Fig. 7.19 to input the number of students and number of exams for the `GradeBook` and to allow the instructor to enter one grade at a time.

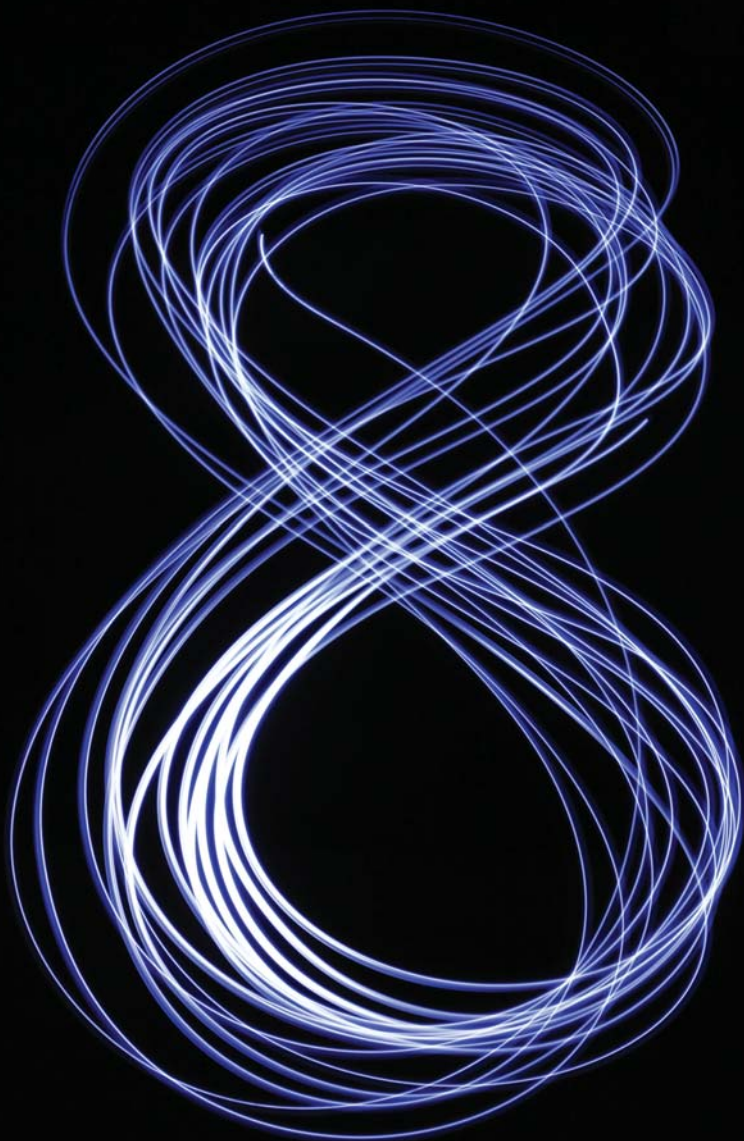
Making a Difference

7.40 (*Polling*) The Internet and the web are enabling more people to network, join a cause, voice opinions, and so on. Recent presidential candidates have used the Internet intensively to get out their messages and raise money for their campaigns. In this exercise, you'll write a simple polling program that allows users to rate five social-consciousness issues from 1 (least important) to 10 (most important). Pick five causes that are important to you (e.g., political issues, global environmental issues). Use a one-dimensional array `topics` (of type `String`) to store the five causes. To summarize the survey responses, use a 5-row, 10-column two-dimensional array `responses` (of type `int`), each row corresponding to an element in the `topics` array. When the program runs, it should ask the user to rate each issue. Have your friends and family respond to the survey. Then have the program display a summary of the results, including:

- a) A tabular report with the five topics down the left side and the 10 ratings across the top, listing in each column the number of ratings received for each topic.
- b) To the right of each row, show the average of the ratings for that issue.
- c) Which issue received the highest point total? Display both the issue and the point total.
- d) Which issue received the lowest point total? Display both the issue and the point total.

Classes and Objects: A Deeper Look

8



Is it a world to hide virtues in?

—William Shakespeare

*But what, to serve
our private ends,
Forbids the cheating
of our friends?*

—Charles Churchill

*This above all: to thine own self
be true.*

—William Shakespeare.

Objectives

In this chapter you'll:

- Use the `throw` statement to indicate that a problem has occurred.
- Use keyword `this` in a constructor to call another constructor in the same class.
- Use `static` variables and methods.
- Import `static` members of a class.
- Use the `enum` type to create sets of constants with unique identifiers.
- Declare `enum` constants with parameters.
- Use `BigDecimal` for precise monetary calculations.

- | | |
|---|---|
| 8.1 Introduction | 8.11 <code>static</code> Class Members |
| 8.2 Time Class Case Study | 8.12 <code>static</code> Import |
| 8.3 Controlling Access to Members | 8.13 <code>final</code> Instance Variables |
| 8.4 Referring to the Current Object's Members with the <code>this</code> Reference | 8.14 Package Access |
| 8.5 Time Class Case Study: Overloaded Constructors | 8.15 Using <code>BigDecimal</code> for Precise Monetary Calculations |
| 8.6 Default and No-Argument Constructors | 8.16 (Optional) GUI and Graphics Case Study: Using Objects with Graphics |
| 8.7 Notes on <code>Set</code> and <code>Get</code> Methods | 8.17 Wrap-Up |
| 8.8 Composition | |
| 8.9 <code>enum</code> Types | |
| 8.10 Garbage Collection | |

Summary | Self-Review Exercise | Answers to Self-Review Exercise | Exercises | Making a Difference

8.1 Introduction

We now take a deeper look at building classes, controlling access to members of a class and creating constructors. We show how to throw an exception to indicate that a problem has occurred (Section 7.5 discussed catching exceptions). We use the `this` keyword to enable one constructor to conveniently call another constructor of the same class. We discuss *composition*—a capability that allows a class to have references to objects of other classes as members. We reexamine the use of *set* and *get* methods. Recall that Section 6.10 introduced the basic `enum` type to declare a set of constants. In this chapter, we discuss the relationship between `enum` types and classes, demonstrating that an `enum` type, like a class, can be declared in its own file with constructors, methods and fields. The chapter also discusses `static` class members and `final` instance variables in detail. We show a special relationship between classes in the same package. Finally, we demonstrate how to use class `BigDecimal` to perform precise monetary calculations. Two additional types of classes—nested classes and anonymous inner classes—are discussed in Chapter 12.

8.2 Time Class Case Study

Our first example consists of two classes—`Time1` (Fig. 8.1) and `Time1Test` (Fig. 8.2). Class `Time1` represents the time of day. Class `Time1Test`'s `main` method creates one object of class `Time1` and invokes its methods. The output of this program appears in Fig. 8.2.

Time1 Class Declaration

Class `Time1`'s private `int` instance variables `hour`, `minute` and `second` (lines 6–8) represent the time in universal-time format (24-hour clock format in which hours are in the range 0–23, and minutes and seconds are each in the range 0–59). `Time1` contains public methods `setTime` (lines 12–25), `toUniversalString` (lines 28–31) and `toString` (lines 34–39). These methods are also called the **public services** or the **public interface** that the class provides to its clients.

```

1  // Fig. 8.1: Time1.java
2  // Time1 class declaration maintains the time in 24-hour format.
3
4  public class Time1
5  {
6      private int hour; // 0 - 23
7      private int minute; // 0 - 59
8      private int second; // 0 - 59
9
10     // set a new time value using universal time; throw an
11     // exception if the hour, minute or second is invalid
12     public void setTime(int hour, int minute, int second)
13     {
14         // validate hour, minute and second
15         if (hour < 0 || hour >= 24 || minute < 0 || minute >= 60 ||
16             second < 0 || second >= 60)
17         {
18             throw new IllegalArgumentException(
19                 "hour, minute and/or second was out of range");
20         }
21
22         this.hour = hour;
23         this.minute = minute;
24         this.second = second;
25     }
26
27     // convert to String in universal-time format (HH:MM:SS)
28     public String toUniversalString()
29     {
30         return String.format("%02d:%02d:%02d", hour, minute, second);
31     }
32
33     // convert to String in standard-time format (H:MM:SS AM or PM)
34     public String toString()
35     {
36         return String.format("%d:%02d:%02d %s",
37             ((hour == 0 || hour == 12) ? 12 : hour % 12),
38             minute, second, (hour < 12 ? "AM" : "PM"));
39     }
40 } // end class Time1

```

Fig. 8.1 | Time1 class declaration maintains the time in 24-hour format.

Default Constructor

In this example, class `Time1` does *not* declare a constructor, so the compiler supplies a default constructor. Each instance variable implicitly receives the default `int` value. Instance variables also can be initialized when they're declared in the class body, using the same initialization syntax as with a local variable.

Method `setTime` and Throwing Exceptions

Method `setTime` (lines 12–25) is a `public` method that declares three `int` parameters and uses them to set the time. Lines 15–16 test each argument to determine whether the value is outside the proper range. The hour value must be greater than or equal to 0 and less than

24, because universal-time format represents hours as integers from 0 to 23 (e.g., 1 PM is hour 13 and 11 PM is hour 23; midnight is hour 0 and noon is hour 12). Similarly, both minute and second values must be greater than or equal to 0 and less than 60. For values outside these ranges, `setTime` **throws an exception** of type `IllegalArgumentException` (lines 18–19), which notifies the client code that an invalid argument was passed to the method. As you learned in Section 7.5, you can use `try...catch` to catch exceptions and attempt to recover from them, which we'll do in Fig. 8.2. The class instance creation expression in the **throw statement** (Fig. 8.1; line 18) creates a new object of type `IllegalArgumentException`. The parentheses following the class name indicate a call to the `IllegalArgumentException` constructor. In this case, we call the constructor that allows us to specify a custom error message. After the exception object is created, the throw statement immediately terminates method `setTime` and the exception is returned to the calling method that attempted to set the time. If the argument values are all valid, lines 22–24 assign them to the hour, minute and second instance variables.



Software Engineering Observation 8.1

For a method like `setTime` in Fig. 8.1, validate all of the method's arguments before using them to set instance variable values to ensure that the object's data is modified only if all the arguments are valid.

Method `toUniversalString`

Method `toUniversalString` (lines 28–31) takes no arguments and returns a `String` in *universal-time format*, consisting of two digits each for the hour, minute and second—recall that you can use the 0 flag in a `printf` format specification (e.g., `"%02d"`) to display leading zeros for a value that doesn't use all the character positions in the specified field width. For example, if the time were 1:30:07 PM, the method would return 13:30:07. Line 30 uses static method `format` of class `String` to return a `String` containing the formatted hour, minute and second values, each with two digits and possibly a leading 0 (specified with the 0 flag). Method `format` is similar to method `System.out.printf` except that `format` *returns* a formatted `String` rather than displaying it in a command window. The formatted `String` is returned by method `toUniversalString`.

Method `toString`

Method `toString` (lines 34–39) takes no arguments and returns a `String` in *standard-time format*, consisting of the hour, minute and second values separated by colons and followed by AM or PM (e.g., 11:30:17 AM or 1:27:06 PM). Like method `toUniversalString`, method `toString` uses static `String` method `format` to format the minute and second as two-digit values, with leading zeros if necessary. Line 37 uses a conditional operator (`?:`) to determine the value for hour in the `String`—if the hour is 0 or 12 (AM or PM), it appears as 12; otherwise, it appears as a value from 1 to 11. The conditional operator in line 30 determines whether AM or PM will be returned as part of the `String`.

Recall all objects in Java have a `toString` method that returns a `String` representation of the object. We chose to return a `String` containing the time in standard-time format. Method `toString` is called *implicitly* whenever a `Time1` object appears in the code where a `String` is needed, such as the value to output with a `%s` format specifier in a call to `System.out.printf`. You may also call `toString` *explicitly* to obtain a `String` representation of a `Time` object.

Using Class Time1

Class `Time1Test` (Fig. 8.2) uses class `Time1`. Line 9 declares and creates a `Time1` object `time`. Operator `new` implicitly invokes class `Time1`'s default constructor, because `Time1` does not declare any constructors. To confirm that the `Time1` object was initialized properly, line 12 calls the private method `displayTime` (lines 35–39), which, in turn, calls the `Time1` object's `toUniversalString` and `toString` methods to output the time in universal-time format and standard-time format, respectively. Note that `toString` could have been called implicitly here rather than explicitly. Next, line 16 invokes method `setTime` of the time object to change the time. Then line 17 calls `displayTime` again to output the time in both formats to confirm that it was set correctly.

**Software Engineering Observation 8.2**

Recall from Chapter 3 that methods declared with access modifier `private` can be called only by other methods of the class in which the `private` methods are declared. Such methods are commonly referred to as *utility methods* or *helper methods* because they're typically used to support the operation of the class's other methods.

```

1  // Fig. 8.2: Time1Test.java
2  // Time1 object used in an app.
3
4  public class Time1Test
5  {
6      public static void main(String[] args)
7      {
8          // create and initialize a Time1 object
9          Time1 time = new Time1(); // invokes Time1 constructor
10
11         // output string representations of the time
12         displayTime("After time object is created", time);
13         System.out.println();
14
15         // change time and output updated time
16         time.setTime(13, 27, 6);
17         displayTime("After calling setTime", time);
18         System.out.println();
19
20         // attempt to set time with invalid values
21         try
22         {
23             time.setTime(99, 99, 99); // all values out of range
24         }
25         catch (IllegalArgumentException e)
26         {
27             System.out.printf("Exception: %s%n%n", e.getMessage());
28         }
29
30         // display time after attempt to set invalid values
31         displayTime("After calling setTime with invalid values", time);
32     }

```

Fig. 8.2 | `Time1` object used in an app. (Part I of 2.)

```

33
34 // displays a Time1 object in 24-hour and 12-hour formats
35 private static void displayTime(String header, Time1 t)
36 {
37     System.out.printf("%s\nUniversal time: %s\nStandard time: %s\n",
38         header, t.toUniversalString(), t.toString());
39 }
40 } // end class Time1Test

```

After time object is created
 Universal time: 00:00:00
 Standard time: 12:00:00 AM

After calling setTime
 Universal time: 13:27:06
 Standard time: 1:27:06 PM

Exception: hour, minute and/or second was out of range

After calling setTime with invalid values
 Universal time: 13:27:06
 Standard time: 1:27:06 PM

Fig. 8.2 | Time1 object used in an app. (Part 2 of 2.)

Calling Time1 Method setTime with Invalid Values

To illustrate that method setTime *validates* its arguments, line 23 calls method setTime with *invalid* arguments of 99 for the hour, minute and second. This statement is placed in a try block (lines 21–24) in case setTime throws an IllegalArgumentException, which it will do since the arguments are all invalid. When this occurs, the exception is caught at lines 25–28, and line 27 displays the exception’s error message by calling its getMessage method. Line 31 outputs the time again in both formats to confirm that setTime did *not* change the time when invalid arguments were supplied.

Software Engineering of the Time1 Class Declaration

Consider several issues of class design with respect to class Time1. The instance variables hour, minute and second are each declared private. The actual data representation used within the class is of no concern to the class’s clients. For example, it would be perfectly reasonable for Time1 to represent the time internally as the number of seconds since midnight or the number of minutes and seconds since midnight. Clients could use the same public methods and get the same results without being aware of this. (Exercise 8.5 asks you to represent the time in class Time2 of Fig. 8.5 as the number of seconds since midnight and show that indeed no change is visible to the clients of the class.)



Software Engineering Observation 8.3

Classes simplify programming, because the client can use only a class’s public methods. Such methods are usually client oriented rather than implementation oriented. Clients are neither aware of, nor involved in, a class’s implementation. Clients generally care about what the class does but not how the class does it.



Software Engineering Observation 8.4

Interfaces change less frequently than implementations. When an implementation changes, implementation-dependent code must change accordingly. Hiding the implementation reduces the possibility that other program parts will become dependent on class implementation details.

Java SE 8—Date/Time API

This section's example and several of this chapter's later examples demonstrate various class-implementation concepts in classes that represent dates and times. In professional Java development, rather than building your own date and time classes, you'll typically reuse the ones provided by the Java API. Though Java has always had classes for manipulating dates and times, Java SE 8 introduces a new **Date/Time API**—defined by the classes in the package `java.time`—applications built with Java SE 8 should use the Date/Time API's capabilities, rather than those in earlier Java versions. The new API fixes various issues with the older classes and provides more robust, easier-to-use capabilities for manipulating dates, times, time zones, calendars and more. We use some Date/Time API features in Chapter 23. You can learn more about the Date/Time API's classes at:

download.java.net/jdk8/docs/api/java/time/package-summary.html

8.3 Controlling Access to Members

The access modifiers `public` and `private` control access to a class's variables and methods. In Chapter 9, we'll introduce the additional access modifier `protected`. The primary purpose of `public` methods is to present to the class's clients a view of the services the class provides (i.e., the class's `public` interface). Clients need not be concerned with how the class accomplishes its tasks. For this reason, the class's `private` variables and `private` methods (i.e., its *implementation details*) are *not* accessible to its clients.

Figure 8.3 demonstrates that `private` class members are *not* accessible outside the class. Lines 9–11 attempt to access the `private` instance variables `hour`, `minute` and `second` of the `Time1` object `time`. When this program is compiled, the compiler generates error messages that these `private` members are not accessible. This program assumes that the `Time1` class from Fig. 8.1 is used.

```

1  // Fig. 8.3: MemberAccessTest.java
2  // Private members of class Time1 are not accessible.
3  public class MemberAccessTest
4  {
5      public static void main(String[] args)
6      {
7          Time1 time = new Time1(); // create and initialize Time1 object
8
9          time.hour = 7; // error: hour has private access in Time1
10         time.minute = 15; // error: minute has private access in Time1
11         time.second = 30; // error: second has private access in Time1
12     }
13 } // end class MemberAccessTest

```

Fig. 8.3 | Private members of class `Time1` are not accessible. (Part I of 2.)

```

MemberAccessTest.java:9: hour has private access in Time1
    time.hour = 7; // error: hour has private access in Time1
    ^
MemberAccessTest.java:10: minute has private access in Time1
    time.minute = 15; // error: minute has private access in Time1
    ^
MemberAccessTest.java:11: second has private access in Time1
    time.second = 30; // error: second has private access in Time1
    ^
3 errors

```

Fig. 8.3 | Private members of class `Time1` are not accessible. (Part 2 of 2.)



Common Programming Error 8.1

An attempt by a method that's not a member of a class to access a private member of that class generates a compilation error.

8.4 Referring to the Current Object's Members with the `this` Reference

Every object can access a *reference to itself* with keyword **this** (sometimes called the **this reference**). When an instance method is called for a particular object, the method's body *implicitly* uses keyword `this` to refer to the object's instance variables and other methods. This enables the class's code to know which object should be manipulated. As you'll see in Fig. 8.4, you can also use keyword `this` *explicitly* in an instance method's body. Section 8.5 shows another interesting use of keyword `this`. Section 8.11 explains why keyword `this` cannot be used in a static method.

We now demonstrate implicit and explicit use of the `this` reference (Fig. 8.4). This example is the first in which we declare *two* classes in one file—class `ThisTest` is declared in lines 4–11, and class `SimpleTime` in lines 14–47. We do this to demonstrate that when you compile a `.java` file containing more than one class, the compiler produces a separate class file with the `.class` extension for every compiled class. In this case, two separate files are produced—`SimpleTime.class` and `ThisTest.class`. When one source-code (`.java`) file contains multiple class declarations, the compiler places both class files for those classes in the *same* directory. Note also in Fig. 8.4 that only class `ThisTest` is declared `public`. A source-code file can contain only *one* `public` class—otherwise, a compilation error occurs. *Non-public classes* can be used only by other classes in the *same package*. So, in this example, class `SimpleTime` can be used only by class `ThisTest`.

```

1  // Fig. 8.4: ThisTest.java
2  // this used implicitly and explicitly to refer to members of an object.
3
4  public class ThisTest
5  {
6      public static void main(String[] args)
7      {

```

Fig. 8.4 | `this` used implicitly and explicitly to refer to members of an object. (Part 1 of 2.)

```
8      SimpleTime time = new SimpleTime(15, 30, 19);
9      System.out.println(time.buildString());
10  }
11 } // end class ThisTest
12
13 // class SimpleTime demonstrates the "this" reference
14 class SimpleTime
15 {
16     private int hour; // 0-23
17     private int minute; // 0-59
18     private int second; // 0-59
19
20     // if the constructor uses parameter names identical to
21     // instance variable names the "this" reference is
22     // required to distinguish between the names
23     public SimpleTime(int hour, int minute, int second)
24     {
25         this.hour = hour; // set "this" object's hour
26         this.minute = minute; // set "this" object's minute
27         this.second = second; // set "this" object's second
28     }
29
30     // use explicit and implicit "this" to call toUniversalString
31     public String buildString()
32     {
33         return String.format("%24s: %s%n%24s: %s",
34             "this.toUniversalString()", this.toUniversalString(),
35             "toUniversalString()", toUniversalString());
36     }
37
38     // convert to String in universal-time format (HH:MM:SS)
39     public String toUniversalString()
40     {
41         // "this" is not required here to access instance variables,
42         // because method does not have local variables with same
43         // names as instance variables
44         return String.format("%02d:%02d:%02d",
45             this.hour, this.minute, this.second);
46     }
47 } // end class SimpleTime
```

```
this.toUniversalString(): 15:30:19
toUniversalString(): 15:30:19
```

Fig. 8.4 | `this` used implicitly and explicitly to refer to members of an object. (Part 2 of 2.)

Class `SimpleTime` (lines 14–47) declares three private instance variables—`hour`, `minute` and `second` (lines 16–18). The class's constructor (lines 23–28) receives three `int` arguments to initialize a `SimpleTime` object. Once again, we used parameter names for the constructor (line 23) that are *identical* to the class's instance-variable names (lines 16–18), so we use the `this` reference to refer to the instance variables in lines 25–27.

**Error-Prevention Tip 8.1**

Most IDEs will issue a warning if you say `x = x;` instead of `this.x = x;`. The statement `x = x;` is often called a no-op (no operation).

Method `buildString` (lines 31–36) returns a `String` created by a statement that uses the `this` reference explicitly and implicitly. Line 34 uses it *explicitly* to call method `toUniversalString`. Line 35 uses it *implicitly* to call the same method. Both lines perform the same task. You typically will not use `this` explicitly to reference other methods within the current object. Also, line 45 in method `toUniversalString` explicitly uses the `this` reference to access each instance variable. This is *not* necessary here, because the method does *not* have any local variables that shadow the instance variables of the class.

**Performance Tip 8.1**

Java conserves storage by maintaining only one copy of each method per class—this method is invoked by every object of the class. Each object, on the other hand, has its own copy of the class's instance variables. Each method of the class implicitly uses `this` to determine the specific object of the class to manipulate.

Class `ThisTest`'s main method (lines 6–10) demonstrates class `SimpleTime`. Line 8 creates an instance of class `SimpleTime` and invokes its constructor. Line 9 invokes the object's `buildString` method, then displays the results.

8.5 Time Class Case Study: Overloaded Constructors

As you know, you can declare your own constructor to specify how objects of a class should be initialized. Next, we demonstrate a class with several **overloaded constructors** that enable objects of that class to be initialized in different ways. To overload constructors, simply provide multiple constructor declarations with different signatures.

Class Time2 with Overloaded Constructors

The `Time1` class's default constructor in Fig. 8.1 initialized `hour`, `minute` and `second` to their default 0 values (i.e., midnight in universal time). The default constructor does not enable the class's clients to initialize the time with nonzero values. Class `Time2` (Fig. 8.5) contains five overloaded constructors that provide convenient ways to initialize objects. In this program, four of the constructors invoke a fifth, which in turn ensures that the value supplied for `hour` is in the range 0 to 23, and the values for `minute` and `second` are each in the range 0 to 59. The compiler invokes the appropriate constructor by matching the number, types and order of the types of the arguments specified in the constructor call with the number, types and order of the types of the parameters specified in each constructor declaration. Class `Time2` also provides *set* and *get* methods for each instance variable.

```

1 // Fig. 8.5: Time2.java
2 // Time2 class declaration with overloaded constructors.
3
4 public class Time2
5 {

```

Fig. 8.5 | `Time2` class with overloaded constructors. (Part 1 of 4.)


```
6     private int hour; // 0 - 23
7     private int minute; // 0 - 59
8     private int second; // 0 - 59
9
10    // Time2 no-argument constructor:
11    // initializes each instance variable to zero
12    public Time2()
13    {
14        this(0, 0, 0); // invoke constructor with three arguments
15    }
16
17    // Time2 constructor: hour supplied, minute and second defaulted to 0
18    public Time2(int hour)
19    {
20        this(hour, 0, 0); // invoke constructor with three arguments
21    }
22
23    // Time2 constructor: hour and minute supplied, second defaulted to 0
24    public Time2(int hour, int minute)
25    {
26        this(hour, minute, 0); // invoke constructor with three arguments
27    }
28
29    // Time2 constructor: hour, minute and second supplied
30    public Time2(int hour, int minute, int second)
31    {
32        if (hour < 0 || hour >= 24)
33            throw new IllegalArgumentException("hour must be 0-23");
34
35        if (minute < 0 || minute >= 60)
36            throw new IllegalArgumentException("minute must be 0-59");
37
38        if (second < 0 || second >= 60)
39            throw new IllegalArgumentException("second must be 0-59");
40
41        this.hour = hour;
42        this.minute = minute;
43        this.second = second;
44    }
45
46    // Time2 constructor: another Time2 object supplied
47    public Time2(Time2 time)
48    {
49        // invoke constructor with three arguments
50        this(time.getHour(), time.getMinute(), time.getSecond());
51    }
52
53    // Set Methods
54    // set a new time value using universal time;
55    // validate the data
56    public void setTime(int hour, int minute, int second)
57    {
```

Fig. 8.5 | Time2 class with overloaded constructors. (Part 2 of 4.)

```
58     if (hour < 0 || hour >= 24)
59         throw new IllegalArgumentException("hour must be 0-23");
60
61     if (minute < 0 || minute >= 60)
62         throw new IllegalArgumentException("minute must be 0-59");
63
64     if (second < 0 || second >= 60)
65         throw new IllegalArgumentException("second must be 0-59");
66
67     this.hour = hour;
68     this.minute = minute;
69     this.second = second;
70 }
71
72 // validate and set hour
73 public void setHour(int hour)
74 {
75     if (hour < 0 || hour >= 24)
76         throw new IllegalArgumentException("hour must be 0-23");
77
78     this.hour = hour;
79 }
80
81 // validate and set minute
82 public void setMinute(int minute)
83 {
84     if (minute < 0 && minute >= 60)
85         throw new IllegalArgumentException("minute must be 0-59");
86
87     this.minute = minute;
88 }
89
90 // validate and set second
91 public void setSecond(int second)
92 {
93     if (second >= 0 && second < 60)
94         throw new IllegalArgumentException("second must be 0-59");
95
96     this.second = second;
97 }
98
99 // Get Methods
100 // get hour value
101 public int getHour()
102 {
103     return hour;
104 }
105
106 // get minute value
107 public int getMinute()
108 {
109     return minute;
110 }
```

Fig. 8.5 | Time2 class with overloaded constructors. (Part 3 of 4.)

```

111
112     // get second value
113     public int getSecond()
114     {
115         return second;
116     }
117
118     // convert to String in universal-time format (HH:MM:SS)
119     public String toUniversalString()
120     {
121         return String.format(
122             "%02d:%02d:%02d", getHour(), getMinute(), getSecond());
123     }
124
125     // convert to String in standard-time format (H:MM:SS AM or PM)
126     public String toString()
127     {
128         return String.format("%d:%02d:%02d %s",
129             ((getHour() == 0 || getHour() == 12) ? 12 : getHour() % 12),
130             getMinute(), getSecond(), (getHour() < 12 ? "AM" : "PM"));
131     }
132 } // end class Time2

```

Fig. 8.5 | Time2 class with overloaded constructors. (Part 4 of 4.)

Class Time2's Constructors—Calling One Constructor from Another via this

Lines 12–15 declare a so-called **no-argument constructor** that's invoked without arguments. Once you declare any constructors in a class, the compiler will *not* provide a *default constructor*. This no-argument constructor ensures that class Time2's clients can create Time2 objects with default values. Such a constructor simply initializes the object as specified in the constructor's body. In the body, we introduce a use of `this` that's allowed only as the *first* statement in a constructor's body. Line 14 uses `this` in method-call syntax to invoke the Time2 constructor that takes three parameters (lines 30–44) with values of 0 for the hour, minute and second. Using `this` as shown here is a popular way to *reuse* initialization code provided by another of the class's constructors rather than defining similar code in the no-argument constructor's body. We use this syntax in four of the five Time2 constructors to make the class easier to maintain and modify. If we need to change how objects of class Time2 are initialized, only the constructor that the class's other constructors call will need to be modified.



Common Programming Error 8.2

It's a compilation error when `this` is used in a constructor's body to call another of the class's constructors if that call is not the first statement in the constructor. It's also a compilation error when a method attempts to invoke a constructor directly via `this`.

Lines 18–21 declare a Time2 constructor with a single `int` parameter representing the hour, which is passed with 0 for the minute and second to the constructor at lines 30–44. Lines 24–27 declare a Time2 constructor that receives two `int` parameters representing the hour and minute, which are passed with 0 for the second to the constructor at lines 30–44. Like the no-argument constructor, each of these constructors invokes the constructor at lines 30–44 to minimize code duplication. Lines 30–44 declare the Time2 constructor

that receives three `int` parameters representing the hour, minute and second. This constructor validates and initializes the instance variables.

Lines 47–51 declare a `Time2` constructor that receives a reference to another `Time2` object. The values from the `Time2` argument are passed to the three-argument constructor at lines 30–44 to initialize the hour, minute and second. Line 50 could have directly accessed the hour, minute and second values of the argument `time` with the expressions `time.hour`, `time.minute` and `time.second`—even though hour, minute and second are declared as private variables of class `Time2`. This is due to a special relationship between objects of the same class. We’ll see in a moment why it’s preferable to use the *get* methods.



Software Engineering Observation 8.5

When one object of a class has a reference to another object of the same class, the first object can access all the second object’s data and methods (including those that are private).

Class `Time2`’s *setTime* Method

Method `setTime` (lines 56–70) throws an `IllegalArgumentException` (lines 59, 62 and 65) if any the method’s arguments is out of range. Otherwise, it sets `Time2`’s instance variables to the argument values (lines 67–69).

Notes Regarding Class `Time2`’s *set* and *get* Methods and Constructors

`Time2`’s *get* methods are called throughout the class. In particular, methods `toUniversalString` and `toString` call methods `getHour`, `getMinute` and `getSecond` in line 122 and lines 129–130, respectively. In each case, these methods could have accessed the class’s private data directly without calling the *get* methods. However, consider changing the representation of the time from three `int` values (requiring 12 bytes of memory) to a single `int` value representing the total number of seconds that have elapsed since midnight (requiring only four bytes of memory). If we made such a change, only the bodies of the methods that access the private data directly would need to change—in particular, the three-argument constructor, the `setTime` method and the individual *set* and *get* methods for the hour, minute and second. There would be no need to modify the bodies of methods `toUniversalString` or `toString` because they do *not* access the data directly. Designing the class in this manner reduces the likelihood of programming errors when altering the class’s implementation.

Similarly, each `Time2` constructor could include a copy of the appropriate statements from the three-argument constructor. Doing so may be slightly more efficient, because the extra constructor calls are eliminated. But, *duplicating* statements makes changing the class’s internal data representation more difficult. Having the `Time2` constructors call the constructor with three arguments requires that any changes to the implementation of the three-argument constructor be made only once. Also, the compiler can optimize programs by removing calls to simple methods and replacing them with the expanded code of their declarations—a technique known as **inlining the code**, which improves program performance.

Using Class `Time2`’s Overloaded Constructors

Class `Time2Test` (Fig. 8.6) invokes the overloaded `Time2` constructors (lines 8–12 and 24). Line 8 invokes the `Time2` no-argument constructor. Lines 9–12 demonstrate passing arguments to the other `Time2` constructors. Line 9 invokes the single-argument constructor that receives an `int` at lines 18–21 of Fig. 8.5. Line 10 invokes the two-argument constructor at lines 24–27 of Fig. 8.5. Line 11 invokes the three-argument constructor at lines 30–44 of

Fig. 8.5. Line 12 invokes the single-argument constructor that takes a `Time2` at lines 47–51 of Fig. 8.5. Next, the app displays the `String` representations of each `Time2` object to confirm that it was initialized properly (lines 15–19). Line 24 attempts to initialize `t6` by creating a new `Time2` object and passing three *invalid* values to the constructor. When the constructor attempts to use the invalid hour value to initialize the object's hour, an `IllegalArgumentException` occurs. We catch this exception at line 26 and display its error message, which results in the last line of the output.

```

1 // Fig. 8.6: Time2Test.java
2 // Overloaded constructors used to initialize Time2 objects.
3
4 public class Time2Test
5 {
6     public static void main(String[] args)
7     {
8         Time2 t1 = new Time2(); // 00:00:00
9         Time2 t2 = new Time2(2); // 02:00:00
10        Time2 t3 = new Time2(21, 34); // 21:34:00
11        Time2 t4 = new Time2(12, 25, 42); // 12:25:42
12        Time2 t5 = new Time2(t4); // 12:25:42
13
14        System.out.println("Constructed with:");
15        displayTime("t1: all default arguments", t1);
16        displayTime("t2: hour specified; default minute and second", t2);
17        displayTime("t3: hour and minute specified; default second", t3);
18        displayTime("t4: hour, minute and second specified", t4);
19        displayTime("t5: Time2 object t4 specified", t5);
20
21        // attempt to initialize t6 with invalid values
22        try
23        {
24            Time2 t6 = new Time2(27, 74, 99); // invalid values
25        }
26        catch (IllegalArgumentException e)
27        {
28            System.out.printf("%nException while initializing t6: %s%n",
29                               e.getMessage());
30        }
31    }
32
33    // displays a Time2 object in 24-hour and 12-hour formats
34    private static void displayTime(String header, Time2 t)
35    {
36        System.out.printf("%s%n   %s%n   %s%n",
37                           header, t.toUniversalString(), t.toString());
38    }
39 } // end class Time2Test

```

```

Constructed with:
t1: all default arguments
    00:00:00
    12:00:00 AM

```

Fig. 8.6 | Overloaded constructors used to initialize `Time2` objects. (Part I of 2.)

```

t2: hour specified; default minute and second
    02:00:00
    2:00:00 AM
t3: hour and minute specified; default second
    21:34:00
    9:34:00 PM
t4: hour, minute and second specified
    12:25:42
    12:25:42 PM
t5: Time2 object t4 specified
    12:25:42
    12:25:42 PM
Exception while initializing t6: hour must be 0-23

```

Fig. 8.6 | Overloaded constructors used to initialize Time2 objects. (Part 2 of 2.)

8.6 Default and No-Argument Constructors

Every class *must* have at least *one* constructor. If you do not provide any in a class's declaration, the compiler creates a *default constructor* that takes *no* arguments when it's invoked. The default constructor initializes the instance variables to the initial values specified in their declarations or to their default values (zero for primitive numeric types, `false` for boolean values and `null` for references). In Section 9.4.1, you'll learn that the default constructor also performs another task.

Recall that if your class declares constructors, the compiler will *not* create a default constructor. In this case, you must declare a no-argument constructor if default initialization is required. Like a default constructor, a no-argument constructor is invoked with empty parentheses. The `Time2` no-argument constructor (lines 12–15 of Fig. 8.5) explicitly initializes a `Time2` object by passing to the three-argument constructor 0 for each parameter. Since 0 is the default value for `int` instance variables, the no-argument constructor in this example could actually be declared with an empty body. In this case, each instance variable would receive its default value when the no-argument constructor is called. If we omit the no-argument constructor, clients of this class would not be able to create a `Time2` object with the expression `new Time2()`.



Error-Prevention Tip 8.2

Ensure that you do not include a return type in a constructor definition. Java allows other methods of the class besides its constructors to have the same name as the class and to specify return types. Such methods are not constructors and will not be called when an object of the class is instantiated.



Common Programming Error 8.3

A compilation error occurs if a program attempts to initialize an object of a class by passing the wrong number or types of arguments to the class's constructor.

8.7 Notes on Set and Get Methods

As you know, a class's private fields can be manipulated *only* by its methods. A typical manipulation might be the adjustment of a customer's bank balance (e.g., a private in-

stance variable of a class `BankAccount`) by a method `computeInterest`. *Set* methods are also commonly called **mutator methods**, because they typically *change* an object's state—i.e., *modify* the values of instance variables. *Get* methods are also commonly called **accessor methods** or **query methods**.

Set and Get Methods vs. public Data

It would seem that providing *set* and *get* capabilities is essentially the same as making a class's instance variables `public`. This is one of the subtleties that makes Java so desirable for software engineering. A `public` instance variable can be read or written by any method that has a reference to an object containing that variable. If an instance variable is declared `private`, a `public` *get* method certainly allows other methods to access it, but the *get* method can *control* how the client can access it. For example, a *get* method might control the format of the data it returns, shielding the client code from the actual data representation. A `public` *set* method can—and should—carefully scrutinize attempts to modify the variable's value and throw an exception if necessary. For example, attempts to *set* the day of the month to 37 or a person's weight to a negative value should be rejected. Thus, although *set* and *get* methods provide access to `private` data, the access is restricted by the implementation of the methods. This helps promote good software engineering.



Software Engineering Observation 8.6

Classes should never have public nonconstant data, but declaring data `public static final` enables you to make constants available to clients of your class. For example, class `Math` offers `public static final` constants `Math.E` and `Math.PI`.



Error-Prevention Tip 8.3

Do not provide `public static final` constants if the constants' values are likely to change in future versions of your software.

Validity Checking in Set Methods

The benefits of data integrity do not follow automatically simply because instance variables are declared `private`—you must provide validity checking. A class's *set* methods could determine that attempts were made to assign invalid data to objects of the class. Typically *set* methods have `void` return type and use exception handling to indicate attempts to assign invalid data. We discuss exception handling in detail in Chapter 11.



Software Engineering Observation 8.7

When appropriate, provide public methods to change and retrieve the values of private instance variables. This architecture helps hide the implementation of a class from its clients, which improves program modifiability.



Error-Prevention Tip 8.4

*Using *set* and *get* methods helps you create classes that are easier to debug and maintain. If only one method performs a particular task, such as setting an instance variable in an object, it's easier to debug and maintain the class. If the instance variable is not being set properly, the code that actually modifies instance variable is localized to one *set* method. Your debugging efforts can be focused on that one method.*

Predicate Methods

Another common use for accessor methods is to test whether a condition is *true* or *false*—such methods are often called **predicate methods**. An example would be class `ArrayList`'s `isEmpty` method, which returns `true` if the `ArrayList` is empty and `false` otherwise. A program might test `isEmpty` before attempting to read another item from an `ArrayList`.

8.8 Composition

A class can have references to objects of other classes as members. This is called **composition** and is sometimes referred to as a *has-a relationship*. For example, an `AlarmClock` object needs to know the current time *and* the time when it's supposed to sound its alarm, so it's reasonable to include *two* references to `Time` objects in an `AlarmClock` object. A car *has-a* steering wheel, a break pedal and an accelerator pedal.

Class Date

This composition example contains classes `Date` (Fig. 8.7), `Employee` (Fig. 8.8) and `EmployeeTest` (Fig. 8.9). Class `Date` (Fig. 8.7) declares instance variables `month`, `day` and `year` (lines 6–8) to represent a date. The constructor receives three `int` parameters. Lines 17–19 validate the month—if it's out-of-range, lines 18–19 throw an exception. Lines 22–25 validate the day. If the day is incorrect based on the number of days in the particular month (except February 29th which requires special testing for leap years), lines 24–25 throw an exception. Lines 28–31 perform the leap year testing for February. If the month is February and the day is 29 and the year is not a leap year, lines 30–31 throw an exception. If no exceptions are thrown, then lines 33–35 initialize the `Date`'s instance variables and lines 38–38 output the `this` reference as a `String`. Since `this` is a reference to the current `Date` object, the object's `toString` method (lines 42–45) is called *implicitly* to obtain the object's `String` representation. In this example, we assume that the value for year is correct—an industrial-strength `Date` class should also validate the year.

```

1  // Fig. 8.7: Date.java
2  // Date class declaration.
3
4  public class Date
5  {
6      private int month; // 1-12
7      private int day; // 1-31 based on month
8      private int year; // any year
9
10     private static final int[] daysPerMonth =
11         { 0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31 };
12
13     // constructor: confirm proper value for month and day given the year
14     public Date(int month, int day, int year)
15     {
16         // check if month in range
17         if (month <= 0 || month > 12)
18             throw new IllegalArgumentException(
19                 "month (" + month + ") must be 1-12");

```

Fig. 8.7 | Date class declaration. (Part 1 of 2.)

```

20
21 // check if day in range for month
22 if (day <= 0 ||
23     (day > daysPerMonth[month] && !(month == 2 && day == 29)))
24     throw new IllegalArgumentException("day (" + day +
25         ") out-of-range for the specified month and year");
26
27 // check for leap year if month is 2 and day is 29
28 if (month == 2 && day == 29 && !(year % 400 == 0 ||
29     (year % 4 == 0 && year % 100 != 0)))
30     throw new IllegalArgumentException("day (" + day +
31         ") out-of-range for the specified month and year");
32
33 this.month = month;
34 this.day = day;
35 this.year = year;
36
37 System.out.printf(
38     "Date object constructor for date %s%n", this);
39 }
40
41 // return a String of the form month/day/year
42 public String toString()
43 {
44     return String.format("%d/%d/%d", month, day, year);
45 }
46 } // end class Date

```

Fig. 8.7 | Date class declaration. (Part 2 of 2.)

Class Employee

Class *Employee* (Fig. 8.8) has instance variables *firstName*, *lastName*, *birthDate* and *hireDate*. Members *firstName* and *lastName* are references to *String* objects. Members *birthDate* and *hireDate* are references to *Date* objects. This demonstrates that a class can have as instance variables references to objects of other classes. The *Employee* constructor (lines 12–19) takes four parameters representing the first name, last name, birth date and hire date. The objects referenced by the parameters are assigned to the *Employee* object's instance variables. When class *Employee*'s *toString* method is called, it returns a *String* containing the employee's name and the *String* representations of the two *Date* objects. Each of these *Strings* is obtained with an *implicit* call to the *Date* class's *toString* method.

```

1 // Fig. 8.8: Employee.java
2 // Employee class with references to other objects.
3
4 public class Employee
5 {
6     private String firstName;
7     private String lastName;
8     private Date birthDate;
9     private Date hireDate;

```

Fig. 8.8 | *Employee* class with references to other objects. (Part 1 of 2.)

```

10
11 // constructor to initialize name, birth date and hire date
12 public Employee(String firstName, String lastName, Date birthDate,
13                 Date hireDate)
14 {
15     this.firstName = firstName;
16     this.lastName = lastName;
17     this.birthDate = birthDate;
18     this.hireDate = hireDate;
19 }
20
21 // convert Employee to String format
22 public String toString()
23 {
24     return String.format("%s, %s Hired: %s Birthday: %s",
25                           lastName, firstName, hireDate, birthDate);
26 }
27 } // end class Employee

```

Fig. 8.8 | Employee class with references to other objects. (Part 2 of 2.)

Class EmployeeTest

Class `EmployeeTest` (Fig. 8.9) creates two `Date` objects to represent an `Employee`'s birthday and hire date, respectively. Line 10 creates an `Employee` and initializes its instance variables by passing to the constructor two `Strings` (representing the `Employee`'s first and last names) and two `Date` objects (representing the birthday and hire date). Line 12 *implicitly* invokes the `Employee`'s `toString` method to display the values of its instance variables and demonstrate that the object was initialized properly.

```

1 // Fig. 8.9: EmployeeTest.java
2 // Composition demonstration.
3
4 public class EmployeeTest
5 {
6     public static void main(String[] args)
7     {
8         Date birth = new Date(7, 24, 1949);
9         Date hire = new Date(3, 12, 1988);
10        Employee employee = new Employee("Bob", "Blue", birth, hire);
11
12        System.out.println(employee);
13    }
14 } // end class EmployeeTest

```

```

Date object constructor for date 7/24/1949
Date object constructor for date 3/12/1988
Blue, Bob Hired: 3/12/1988 Birthday: 7/24/1949

```

Fig. 8.9 | Composition demonstration.

8.9 enum Types

In Fig. 6.8, we introduced the basic enum type, which defines a set of constants represented as unique identifiers. In that program the enum constants represented the game's status. In this section we discuss the relationship between enum types and classes. Like classes, all enum types are *reference* types. An enum type is declared with an **enum declaration**, which is a comma-separated list of *enum constants*—the declaration may optionally include other components of traditional classes, such as constructors, fields and methods (as you'll see momentarily). Each enum declaration declares an enum class with the following restrictions:

1. enum constants are *implicitly* final.
2. enum constants are *implicitly* static.
3. Any attempt to create an object of an enum type with operator new results in a compilation error.

The enum constants can be used anywhere constants can be used, such as in the case labels of switch statements and to control enhanced for statements.

Declaring Instance Variables, a Constructor and Methods in an enum Type

Figure 8.10 demonstrates instance variables, a constructor and methods in an enum type. The enum declaration (lines 5–37) contains two parts—the enum constants and the other members of the enum type. The first part (lines 8–13) declares six constants. Each is optionally followed by arguments that are passed to the **enum constructor** (lines 20–24). Like the constructors you've seen in classes, an enum constructor can specify any number of parameters and can be overloaded. In this example, the enum constructor requires two String parameters. To properly initialize each enum constant, we follow it with parentheses containing two String arguments. The second part (lines 16–36) declares the other members of the enum type—two instance variables (lines 16–17), a constructor (lines 20–24) and two methods (lines 27–30 and 33–36).

Lines 16–17 declare the instance variables `title` and `copyrightYear`. Each enum constant in enum type `Book` is actually an object of enum type `Book` that has its own copy of instance variables `title` and `copyrightYear`. The constructor (lines 20–24) takes two String parameters, one that specifies the book's title and one that specifies its copyright year. Lines 22–23 assign these parameters to the instance variables. Lines 27–36 declare methods, which return the book title and copyright year, respectively.

```

1  // Fig. 8.10: Book.java
2  // Declaring an enum type with a constructor and explicit instance fields
3  // and accessors for these fields
4
5  public enum Book
6  {
7      // declare constants of enum type
8      JHTP("Java How to Program", "2015"),
9      CHTP("C How to Program", "2013"),
10     IW3HTP("Internet & World Wide Web How to Program", "2012"),

```

Fig. 8.10 | Declaring an enum type with a constructor and explicit instance fields and accessors for these fields. (Part 1 of 2.)

```

11  CPPHTP("C++ How to Program", "2014"),
12  VBHTP("Visual Basic How to Program", "2014"),
13  CSHPHTP("Visual C# How to Program", "2014");
14
15  // instance fields
16  private final String title; // book title
17  private final String copyrightYear; // copyright year
18
19  // enum constructor
20  Book(String title, String copyrightYear)
21  {
22      this.title = title;
23      this.copyrightYear = copyrightYear;
24  }
25
26  // accessor for field title
27  public String getTitle()
28  {
29      return title;
30  }
31
32  // accessor for field copyrightYear
33  public String getCopyrightYear()
34  {
35      return copyrightYear;
36  }
37  } // end enum Book

```

Fig. 8.10 | Declaring an enum type with a constructor and explicit instance fields and accessors for these fields. (Part 2 of 2.)

Using enum type Book

Figure 8.11 tests the enum type `Book` and illustrates how to iterate through a range of enum constants. For every enum, the compiler generates the static method `values` (called in line 12) that returns an array of the enum's constants in the order they were declared. Lines 12–14 use the enhanced for statement to display all the constants declared in the enum `Book`. Line 14 invokes the enum `Book`'s `getTitle` and `getCopyrightYear` methods to get the title and copyright year associated with the constant. When an enum constant is converted to a `String` (e.g., `book` in line 13), the constant's identifier is used as the `String` representation (e.g., `JHTP` for the first enum constant).

```

1  // Fig. 8.11: EnumTest.java
2  // Testing enum type Book.
3  import java.util.EnumSet;
4
5  public class EnumTest
6  {
7      public static void main(String[] args)
8      {

```

Fig. 8.11 | Testing enum type `Book`. (Part 1 of 2.)

```

9      System.out.println("All books:");
10
11     // print all books in enum Book
12     for (Book book : Book.values())
13         System.out.printf("%-10s%-45s%s%n", book,
14                             book.getTitle(), book.getCopyrightYear());
15
16     System.out.printf("\nDisplay a range of enum constants:%n");
17
18     // print first four books
19     for (Book book : EnumSet.range(Book.JHTP, Book.CPPHTP))
20         System.out.printf("%-10s%-45s%s%n", book,
21                             book.getTitle(), book.getCopyrightYear());
22     }
23 } // end class EnumTest

```

```

All books:
JHTP      Java How to Program          2015
CHTP      C How to Program              2013
IW3HTP    Internet & World Wide Web How to Program 2012
CPPHTP    C++ How to Program                 2014
VBHTP     Visual Basic How to Program       2014
CSHARPHTP Visual C# How to Program            2014

Display a range of enum constants:
JHTP      Java How to Program          2015
CHTP      C How to Program              2013
IW3HTP    Internet & World Wide Web How to Program 2012
CPPHTP    C++ How to Program                 2014

```

Fig. 8.11 | Testing enum type Book. (Part 2 of 2.)

Lines 19–21 use the static method **range** of class **EnumSet** (declared in package `java.util`) to display a range of the enum **Book**’s constants. Method **range** takes two parameters—the first and the last enum constants in the range—and returns an **EnumSet** that contains all the constants between these two constants, inclusive. For example, the expression `EnumSet.range(Book.JHTP, Book.CPPHTP)` returns an **EnumSet** containing `Book.JHTP`, `Book.CHTP`, `Book.IW3HTP` and `Book.CPPHTP`. The enhanced **for** statement can be used with an **EnumSet** just as it can with an array, so lines 12–14 use it to display the title and copyright year of every book in the **EnumSet**. Class **EnumSet** provides several other static methods for creating sets of enum constants from the same enum type.



Common Programming Error 8.4

In an enum declaration, it’s a syntax error to declare enum constants after the enum type’s constructors, fields and methods.

8.10 Garbage Collection

Every object uses system resources, such as memory. We need a disciplined way to give resources back to the system when they’re no longer needed; otherwise, “resource leaks” might occur that would prevent resources from being reused by your program or possibly

by other programs. The JVM performs automatic **garbage collection** to reclaim the *memory* occupied by objects that are no longer used. When there are *no more references* to an object, the object is *eligible* to be collected. Collection typically occurs when the JVM executes its **garbage collector**, which may not happen for a while, or even at all before a program terminates. So, memory leaks that are common in other languages like C and C++ (because memory is *not* automatically reclaimed in those languages) are *less* likely in Java, but some can still happen in subtle ways. Resource leaks other than memory leaks can also occur. For example, an app may open a file on disk to modify its contents—if the app does not close the file, it must terminate before any other app can use the file.

A Note about Class Object's finalize Method

Every class in Java has the methods of class `Object` (package `java.lang`), one of which is method **finalize**. (You'll learn more about class `Object` in Chapter 9.) You should *never* use method **finalize**, because it can cause many problems and there's uncertainty as to whether it will *ever* get called before a program terminates.

The original intent of **finalize** was to allow the garbage collector to perform **termination housekeeping** on an object just before reclaiming the object's memory. Now, it's considered better practice for any class that uses system resources—such as files on disk—to provide a method that programmers can call to release resources when they're no longer needed in a program. `AutoCloseable` objects reduce the likelihood of resource leaks when you use them with the try-with-resources statement. As its name implies, an `AutoCloseable` object is closed automatically, once a try-with-resources statement finishes using the object. We discuss this in more detail in Section 11.12.



Software Engineering Observation 8.8

Many Java API classes (e.g., class `Scanner` and classes that read files from or write files to disk) provide `close` or `dispose` methods that programmers can call to release resources when they're no longer needed in a program.

8.11 static Class Members

Every object has its own copy of all the instance variables of the class. In certain cases, only one copy of a particular variable should be *shared* by all objects of a class. A **static field**—called a **class variable**—is used in such cases. A static variable represents **classwide information**—all objects of the class share the *same* piece of data. The declaration of a static variable begins with the keyword `static`.

Motivating static

Let's motivate static data with an example. Suppose that we have a video game with Martians and other space creatures. Each Martian tends to be brave and willing to attack other space creatures when the Martian is aware that at least four other Martians are present. If fewer than five Martians are present, each of them becomes cowardly. Thus, each Martian needs to know the `martianCount`. We could endow class `Martian` with `martianCount` as an *instance variable*. If we do this, then every Martian will have a *separate copy* of the instance variable, and every time we create a new Martian, we'll have to update the instance variable `martianCount` in every Martian object. This wastes space with the redundant copies, wastes time in updating the separate copies and is error prone. Instead, we declare `mar-`

tianCount to be static, making martianCount classwide data. Every Martian can see the martianCount as if it were an instance variable of class Martian, but only *one* copy of the static martianCount is maintained. This saves space. We save time by having the Martian constructor increment the static martianCount—there's only one copy, so we do not have to increment separate copies for each Martian object.



Software Engineering Observation 8.9

Use a static variable when all objects of a class must use the same copy of the variable.

Class Scope

Static variables have *class scope*—they can be used in all of the class's methods. We can access a class's public static members through a reference to any object of the class, or by qualifying the member name with the class name and a dot (`.`), as in `Math.random()`. A class's private static class members can be accessed by client code only through methods of the class. Actually, *static class members exist even when no objects of the class exist*—they're available as soon as the class is loaded into memory at execution time. To access a public static member when no objects of the class exist (and even when they do), prefix the class name and a dot (`.`) to the static member, as in `Math.PI`. To access a private static member when no objects of the class exist, provide a public static method and call it by qualifying its name with the class name and a dot.



Software Engineering Observation 8.10

Static class variables and methods exist, and can be used, even if no objects of that class have been instantiated.

static Methods Cannot Directly Access Instance Variables and Instance Methods

A static method *cannot* access a class's instance variables and instance methods, because a static method can be called even when no objects of the class have been instantiated. For the same reason, the `this` reference *cannot* be used in a static method. The `this` reference must refer to a specific object of the class, and when a static method is called, there might not be any objects of its class in memory.



Common Programming Error 8.5

A compilation error occurs if a static method calls an instance method in the same class by using only the method name. Similarly, a compilation error occurs if a static method attempts to access an instance variable in the same class by using only the variable name.



Common Programming Error 8.6

Referring to `this` in a static method is a compilation error.

Tracking the Number of Employee Objects That Have Been Created

Our next program declares two classes—`Employee` (Fig. 8.12) and `EmployeeTest` (Fig. 8.13). Class `Employee` declares private static variable `count` (Fig. 8.12, line 7) and public static method `getCount` (lines 36–39). The static variable `count` maintains a count of the number of objects of class `Employee` that have been created so far. This class

variable is initialized to zero in line 7. If a static variable is *not* initialized, the compiler assigns it a default value—in this case 0, the default value for type `int`.

```

1  // Fig. 8.12: Employee.java
2  // static variable used to maintain a count of the number of
3  // Employee objects in memory.
4
5  public class Employee
6  {
7      private static int count = 0; // number of Employees created
8      private String firstName;
9      private String lastName;
10
11     // initialize Employee, add 1 to static count and
12     // output String indicating that constructor was called
13     public Employee(String firstName, String lastName)
14     {
15         this.firstName = firstName;
16         this.lastName = lastName;
17
18         ++count; // increment static count of employees
19         System.out.printf("Employee constructor: %s %s; count = %d\n",
20             firstName, lastName, count);
21     }
22
23     // get first name
24     public String getFirstName()
25     {
26         return firstName;
27     }
28
29     // get last name
30     public String getLastName()
31     {
32         return lastName;
33     }
34
35     // static method to get static count value
36     public static int getCount()
37     {
38         return count;
39     }
40 } // end class Employee

```

Fig. 8.12 | static variable used to maintain a count of the number of `Employee` objects in memory.

When `Employee` objects exist, variable `count` can be used in any method of an `Employee` object—this example increments `count` in the constructor (line 18). The `public` static method `getCount` (lines 36–39) returns the number of `Employee` objects that have been created so far. When no objects of class `Employee` exist, client code can access variable `count` by calling method `getCount` via the class name, as in `Employee.getCount()`. When objects exist, method `getCount` can also be called via any reference to an `Employee` object.



Good Programming Practice 8.1

Invoke every static method by using the class name and a dot (.) to emphasize that the method being called is a static method.

Class *EmployeeTest*

EmployeeTest method *main* (Fig. 8.13) instantiates two *Employee* objects (lines 13–14). When each *Employee* object's constructor is invoked, lines 15–16 of Fig. 8.12 assign the *Employee*'s first name and last name to instance variables *firstName* and *lastName*. These two statements do *not* make copies of the original *String* arguments. Actually, *String* objects in Java are **immutable**—they cannot be modified after they're created. Therefore, it's safe to have *many* references to one *String* object. This is not normally the case for objects of most other classes in Java. If *String* objects are immutable, you might wonder why we're able to use operators *+* and *+=* to concatenate *String* objects. *String*-concatenation actually results in a *new* *String* object containing the concatenated values. The original *String* objects are *not* modified.

```

1  // Fig. 8.13: EmployeeTest.java
2  // static member demonstration.
3
4  public class EmployeeTest
5  {
6      public static void main(String[] args)
7      {
8          // show that count is 0 before creating Employees
9          System.out.printf("Employees before instantiation: %d\n",
10                           Employee.getCount());
11
12         // create two Employees; count should be 2
13         Employee e1 = new Employee("Susan", "Baker");
14         Employee e2 = new Employee("Bob", "Blue");
15
16         // show that count is 2 after creating two Employees
17         System.out.printf("\nEmployees after instantiation:\n");
18         System.out.printf("via e1.getCount(): %d\n", e1.getCount());
19         System.out.printf("via e2.getCount(): %d\n", e2.getCount());
20         System.out.printf("via Employee.getCount(): %d\n",
21                            Employee.getCount());
22
23         // get names of Employees
24         System.out.printf("\nEmployee 1: %s %s\nEmployee 2: %s %s\n",
25                            e1.getFirstName(), e1.getLastName(),
26                            e2.getFirstName(), e2.getLastName());
27     }
28 } // end class EmployeeTest

```

```

Employees before instantiation: 0
Employee constructor: Susan Baker; count = 1
Employee constructor: Bob Blue; count = 2

```

Fig. 8.13 | static member demonstration. (Part I of 2.)

```

Employees after instantiation:
via e1.getCount(): 2
via e2.getCount(): 2
via Employee.getCount(): 2

Employee 1: Susan Baker
Employee 2: Bob Blue

```

Fig. 8.13 | static member demonstration. (Part 2 of 2.)

When `main` terminates, local variables `e1` and `e2` are discarded—remember that a local variable exists *only* until the block in which it's declared completes execution. Because `e1` and `e2` were the only references to the `Employee` objects created in lines 13–14 (Fig. 8.13), these objects become “eligible for garbage collection” as `main` terminates.

In a typical app, the garbage collector *might* eventually reclaim the memory for any objects that are eligible for collection. If any objects are not reclaimed before the program terminates, the operating system will reclaim the memory used by the program. The JVM does *not* guarantee when, or even whether, the garbage collector will execute. When it does, it's possible that no objects or only a subset of the eligible objects will be collected.

8.12 static Import

In Section 6.3, you learned about the static fields and methods of class `Math`. We access class `Math`'s static fields and *methods* by preceding each with the class name `Math` and a dot (`.`). A **static import** declaration enables you to import the static members of a class or interface so you can access them via their *unqualified names* in your class—that is, the class name and a dot (`.`) are *not* required when using an imported static member.

static Import Forms

A static import declaration has two forms—one that imports a particular static member (which is known as **single static import**) and one that imports *all* static members of a class (known as **static import on demand**). The following syntax imports a particular static member:

```
import static packageName.ClassName.staticMemberName;
```

where *packageName* is the package of the class (e.g., `java.lang`), *ClassName* is the name of the class (e.g., `Math`) and *staticMemberName* is the name of the static field or method (e.g., `PI` or `abs`). The following syntax imports *all* static members of a class:

```
import static packageName.ClassName.*;
```

The asterisk (`*`) indicates that *all* static members of the specified class should be available for use in the file. static import declarations import *only* static class members. Regular import statements should be used to specify the classes used in a program.

Demonstrating static Import

Figure 8.14 demonstrates a static import. Line 3 is a static import declaration, which imports *all* static fields and methods of class `Math` from package `java.lang`. Lines 9–12 access the `Math` class's static fields `E` (line 11) and `PI` (line 12) and the static methods

`sqrt` (line 9) and `ceil` (line 10) *without* preceding the field names or method names with class name `Math` and a dot.



Common Programming Error 8.7

A compilation error occurs if a program attempts to import two or more classes' static methods that have the same signature or static fields that have the same name.

```

1 // Fig. 8.14: StaticImportTest.java
2 // Static import of Math class methods.
3 import static java.lang.Math.*;
4
5 public class StaticImportTest
6 {
7     public static void main(String[] args)
8     {
9         System.out.printf("sqrt(900.0) = %.1f\n", sqrt(900.0));
10        System.out.printf("ceil(-9.8) = %.1f\n", ceil(-9.8));
11        System.out.printf("E = %f\n", E);
12        System.out.printf("PI = %f\n", PI);
13    }
14 } // end class StaticImportTest

```

```

sqrt(900.0) = 30.0
ceil(-9.8) = -9.0
E = 2.718282
PI = 3.141593

```

Fig. 8.14 | static import of Math class methods.

8.13 final Instance Variables

The **principle of least privilege** is fundamental to good software engineering. In the context of an app's code, it states that code should be granted only the amount of privilege and access that it needs to accomplish its designated task, but no more. This makes your programs more robust by preventing code from accidentally (or maliciously) modifying variable values and calling methods that should *not* be accessible.

Let's see how this principle applies to instance variables. Some of them need to be *modifiable* and some do not. You can use the keyword `final` to specify that a variable is *not* modifiable (i.e., it's a *constant*) and that any attempt to modify it is an error. For example,

```
private final int INCREMENT;
```

declares a `final` (constant) instance variable `INCREMENT` of type `int`. Such variables can be initialized when they're declared. If they're not, they *must* be initialized in every constructor of the class. Initializing constants in constructors enables each object of the class to have a different value for the constant. If a `final` variable is *not* initialized in its declaration or in every constructor, a compilation error occurs.



Software Engineering Observation 8.11

Declaring an instance variable as `final` helps enforce the principle of least privilege. If an instance variable should not be modified, declare it to be `final` to prevent modification. For example, in Fig. 8.8, the instance variables `firstName`, `lastName`, `birthDate` and `hireDate` are never modified after they're initialized, so they should be declared `final`. We'll enforce this practice in all programs going forward. You'll see additional benefits of `final` in Chapter 23, *Concurrency*.



Common Programming Error 8.8

Attempting to modify a `final` instance variable after it's initialized is a compilation error.



Error-Prevention Tip 8.5

Attempts to modify a `final` instance variable are caught at compilation time rather than causing execution-time errors. It's always preferable to get bugs out at compilation time, if possible, rather than allow them to slip through to execution time (where experience has found that repair is often many times more expensive).



Software Engineering Observation 8.12

A `final` field should also be declared `static` if it's initialized in its declaration to a value that's the same for all objects of the class. After this initialization, its value can never change. Therefore, we don't need a separate copy of the field for every object of the class. Making the field `static` enables all objects of the class to share the `final` field.

8.14 Package Access

If no access modifier (`public`, `protected` or `private`—we discuss `protected` in Chapter 9) is specified for a method or variable when it's declared in a class, the method or variable is considered to have **package access**. In a program that consists of one class declaration, this has no specific effect. However, if a program uses *multiple* classes from the *same* package (i.e., a group of related classes), these classes can access each other's package-access members directly through references to objects of the appropriate classes, or in the case of static members through the class name. Package access is rarely used.

Figure 8.15 demonstrates package access. The app contains two classes in one source-code file—the `PackageDataTest` class containing `main` (lines 5–21) and the `PackageData` class (lines 24–41). Classes in the same source file are part of the same package. Consequently, class `PackageDataTest` is allowed to modify the package-access data of `PackageData` objects. When you compile this program, the compiler produces two separate `.class` files—`PackageDataTest.class` and `PackageData.class`. The compiler places the two `.class` files in the same directory. You can also place class `PackageData` (lines 24–41) in a separate source-code file.

In the `PackageData` class declaration, lines 26–27 declare the instance variables `number` and `string` with no access modifiers—therefore, these are package-access instance variables. Class `PackageDataTest`'s `main` method creates an instance of the `PackageData` class (line 9) to demonstrate the ability to modify the `PackageData` instance variables directly (as shown in lines 15–16). The results of the modification can be seen in the output window.

```
1 // Fig. 8.15: PackageDataTest.java
2 // Package-access members of a class are accessible by other classes
3 // in the same package.
4
5 public class PackageDataTest
6 {
7     public static void main(String[] args)
8     {
9         PackageData packageData = new PackageData();
10
11         // output String representation of packageData
12         System.out.printf("After instantiation:%n%s%n", packageData);
13
14         // change package access data in packageData object
15         packageData.number = 77;
16         packageData.string = "Goodbye";
17
18         // output String representation of packageData
19         System.out.printf("%nAfter changing values:%n%s%n", packageData);
20     }
21 } // end class PackageDataTest
22
23 // class with package access instance variables
24 class PackageData
25 {
26     int number; // package-access instance variable
27     String string; // package-access instance variable
28
29     // constructor
30     public PackageData()
31     {
32         number = 0;
33         string = "Hello";
34     }
35
36     // return PackageData object String representation
37     public String toString()
38     {
39         return String.format("number: %d; string: %s", number, string);
40     }
41 } // end class PackageData
```

```
After instantiation:
number: 0; string: Hello

After changing values:
number: 77; string: Goodbye
```

Fig. 8.15 | Package-access members of a class are accessible by other classes in the same package.

8.15 Using BigDecimal for Precise Monetary Calculations

In earlier chapters, we demonstrated monetary calculations using values of type `double`. In Chapter 5, we discussed the fact that some `double` values are represented *approximately*.

Any application that requires precise floating-point calculations—such as those in financial applications—should instead use class **BigDecimal** (from package **java.math**).

Interest Calculations Using **BigDecimal**

Figure 8.16 reimplements the interest calculation example of Fig. 5.6 using objects of class **BigDecimal** to perform the calculations. We also introduce class **NumberFormat** (package **java.text**) for formatting numeric values as *locale-specific* Strings—for example, in the U.S. locale, the value 1234.56, would be formatted as "1,234.56", whereas in many European locales it would be formatted as "1.234,56".

```

1  // Interest.java
2  // Compound-interest calculations with BigDecimal.
3  import java.math.BigDecimal;
4  import java.text.NumberFormat;
5
6  public class Interest
7  {
8      public static void main(String args[])
9      {
10         // initial principal amount before interest
11         BigDecimal principal = BigDecimal.valueOf(1000.0);
12         BigDecimal rate = BigDecimal.valueOf(0.05); // interest rate
13
14         // display headers
15         System.out.printf("%s%20s%n", "Year", "Amount on deposit");
16
17         // calculate amount on deposit for each of ten years
18         for (int year = 1; year <= 10; year++)
19         {
20             // calculate new amount for specified year
21             BigDecimal amount =
22                 principal.multiply(rate.add(BigDecimal.ONE).pow(year));
23
24             // display the year and the amount
25             System.out.printf("%4d%20s%n", year,
26                 NumberFormat.getCurrencyInstance().format(amount));
27         }
28     }
29 } // end class Interest

```

Year	Amount on deposit
1	\$1,050.00
2	\$1,102.50
3	\$1,157.62
4	\$1,215.51
5	\$1,276.28
6	\$1,340.10
7	\$1,407.10
8	\$1,477.46
9	\$1,551.33
10	\$1,628.89

Fig. 8.16 | Compound-interest calculations with **BigDecimal**.

Creating `BigDecimal` Objects

Lines 11–12 declare and initialize `BigDecimal` variables `principal` and `rate` using the `BigDecimal` static method `valueOf` that receives a `double` argument and returns a `BigDecimal` object that represents the *exact* value specified.

Performing the Interest Calculations with `BigDecimal`

Lines 21–22 perform the interest calculation using `BigDecimal` methods `multiply`, `add` and `pow`. The expression in line 22 evaluates as follows:

1. First, the expression `rate.add(BigDecimal.ONE)` adds 1 to the `rate` to produce a `BigDecimal` containing 1.05—this is equivalent to `1.0 + rate` in line 19 of Fig. 5.6. The `BigDecimal` constant `ONE` represents the value 1. Class `BigDecimal` also provides the commonly used constants `ZERO` (0) and `TEN` (10).
2. Next, `BigDecimal` method `pow` is called on the preceding result to raise 1.05 to the power `year`—this is equivalent to passing `1.0 + rate` and `year` to method `Math.pow` in line 19 of Fig. 5.6.
3. Finally, we call `BigDecimal` method `multiply` on the `principal` object, passing the preceding result as the argument. This returns a `BigDecimal` representing the amount on deposit at the end of the specified year.

Since the expression `rate.add(BigDecimal.ONE)` produces the same value in each iteration of the loop, we could have simply initialized `rate` to 1.05 in line 12; however, we chose to mimic the precise calculations we used in line 19 of Fig. 5.6.

Formatting Currency Values with `NumberFormat`

During each iteration of the loop, line 26

```
NumberFormat.getCurrencyInstance().format(amount)
```

evaluates as follows:

1. First, the expression uses `NumberFormat`'s static method `getCurrencyInstance` to get a `NumberFormat` that's pre-configured to format numeric values as locale-specific currency Strings—for example, in the U.S. locale, the numeric value 1628.89 is formatted as \$1,628.89. Locale-specific formatting is an important part of **internationalization**—the process of customizing your applications for users' various locales and spoken languages.
2. Next, the expression invokes method `NumberFormat` method `format` (on the object returned by `getCurrencyInstance`) to perform the formatting of the amount value. Method `format` then returns the locale-specific String representation, rounded to two-digits to the right of the decimal point.

Rounding `BigDecimal` Values

In addition to precise calculations, `BigDecimal` also gives you control over how values are rounded—by default all calculations are exact and *no* rounding occurs. If you do not specify how to round `BigDecimal` values and a given value cannot be represented exactly—such as the result of 1 divided by 3, which is 0.3333333...—an `ArithmeticException` occurs.

Though we do not do so in this example, you can specify the *rounding mode* for `BigDecimal` by supplying a `MathContext` object (package `java.math`) to class `BigDecimal`'s

constructor when you create a `BigDecimal`. You may also provide a `MathContext` to various `BigDecimal` methods that perform calculations. Class `MathContext` contains several pre-configured `MathContext` objects that you can learn about at

<http://docs.oracle.com/javase/7/docs/api/java/math/MathContext.html>

By default, each pre-configured `MathContext` uses so called “bankers rounding” as explained for the `RoundingMode` constant `HALF_EVEN` at:

http://docs.oracle.com/javase/7/docs/api/java/math/RoundingMode.html#HALF_EVEN

Scaling `BigDecimal` Values

A `BigDecimal`’s scale is the number of digits to the right of its decimal point. If you need a `BigDecimal` rounded to a specific digit, you can call `BigDecimal` method `setScale`. For example, the following expression returns a `BigDecimal` with two digits to the right of the decimal point and using bankers rounding:

```
amount.setScale(2, RoundingMode.HALF_EVEN)
```

8.16 (Optional) GUI and Graphics Case Study: Using Objects with Graphics

Most of the graphics you’ve seen to this point did not vary with each program execution. Exercise 6.2 in Section 6.13 asked you to create a program that generated shapes and colors at random. In that exercise, the drawing changed every time the system called `paintComponent`. To create a more consistent drawing that remains the same each time it’s drawn, we must store information about the displayed shapes so that we can reproduce them each time the system calls `paintComponent`. To do this, we’ll create a set of shape classes that store information about each shape. We’ll make these classes “smart” by allowing objects of these classes to draw themselves by using a `Graphics` object.

Class `MyLine`

Figure 8.17 declares class `MyLine`, which has all these capabilities. Class `MyLine` imports classes `Color` and `Graphics` (lines 3–4). Lines 8–11 declare instance variables for the coordinates of the endpoints needed to draw a line, and line 12 declares the instance variable that stores the color of the line. The constructor at lines 15–22 takes five parameters, one for each instance variable that it initializes. Method `draw` at lines 25–29 requires a `Graphics` object and uses it to draw the line in the proper color and between the endpoints.

```
1 // Fig. 8.17: MyLine.java
2 // MyLine class represents a line.
3 import java.awt.Color;
4 import java.awt.Graphics;
5
6 public class MyLine
7 {
```

Fig. 8.17 | `MyLine` class represents a line. (Part 1 of 2.)

```

 8     private int x1; // x-coordinate of first endpoint
 9     private int y1; // y-coordinate of first endpoint
10     private int x2; // x-coordinate of second endpoint
11     private int y2; // y-coordinate of second endpoint
12     private Color color; // color of this line
13
14     // constructor with input values
15     public MyLine(int x1, int y1, int x2, int y2, Color color)
16     {
17         this.x1 = x1;
18         this.y1 = y1;
19         this.x2 = x2;
20         this.y2 = y2;
21         this.color = color;
22     }
23
24     // Draw the line in the specified color
25     public void draw(Graphics g)
26     {
27         g.setColor(color);
28         g.drawLine(x1, y1, x2, y2);
29     }
30 } // end class MyLine

```

Fig. 8.17 | MyLine class represents a line. (Part 2 of 2.)

Class DrawPanel

In Fig. 8.18, we declare class `DrawPanel`, which will generate random objects of class `MyLine`. Line 12 declares the `MyLine` array `lines` to store the lines to draw. Inside the constructor (lines 15–37), line 17 sets the background color to `Color.WHITE`. Line 19 creates the array with a random length between 5 and 9. The loop at lines 22–36 creates a new `MyLine` for every element in the array. Lines 25–28 generate random coordinates for each line's endpoints, and lines 31–32 generate a random color for the line. Line 35 creates a new `MyLine` object with the randomly generated values and stores it in the array. Method `paintComponent` iterates through the `MyLine` objects in array `lines` using an enhanced for statement (lines 45–46). Each iteration calls the `draw` method of the current `MyLine` object and passes it the `Graphics` object for drawing on the panel.

```

1 // Fig. 8.18: DrawPanel.java
2 // Program that uses class MyLine
3 // to draw random lines.
4 import java.awt.Color;
5 import java.awt.Graphics;
6 import java.security.SecureRandom;
7 import javax.swing.JPanel;
8
9 public class DrawPanel extends JPanel
10 {
11     private SecureRandom randomNumbers = new SecureRandom();
12     private MyLine[] lines; // array of lines

```

Fig. 8.18 | Program that uses class `MyLine` to draw random lines. (Part 1 of 2.)

```

13
14 // constructor, creates a panel with random shapes
15 public DrawPanel()
16 {
17     setBackground(Color.WHITE);
18
19     lines = new MyLine[5 + randomNumbers.nextInt(5)];
20
21     // create lines
22     for (int count = 0; count < lines.length; count++)
23     {
24         // generate random coordinates
25         int x1 = randomNumbers.nextInt(300);
26         int y1 = randomNumbers.nextInt(300);
27         int x2 = randomNumbers.nextInt(300);
28         int y2 = randomNumbers.nextInt(300);
29
30         // generate a random color
31         Color color = new Color(randomNumbers.nextInt(256),
32                                 randomNumbers.nextInt(256), randomNumbers.nextInt(256));
33
34         // add the line to the list of lines to be displayed
35         lines[count] = new MyLine(x1, y1, x2, y2, color);
36     }
37 }
38
39 // for each shape array, draw the individual shapes
40 public void paintComponent(Graphics g)
41 {
42     super.paintComponent(g);
43
44     // draw the lines
45     for (MyLine line : lines)
46         line.draw(g);
47 }
48 } // end class DrawPanel

```

Fig. 8.18 | Program that uses class `MyLine` to draw random lines. (Part 2 of 2.)

Class TestDraw

Class `TestDraw` in Fig. 8.19 sets up a new window to display our drawing. Since we're setting the coordinates for the lines only once in the constructor, the drawing does not change if `paintComponent` is called to refresh the drawing on the screen.

```

1 // Fig. 8.19: TestDraw.java
2 // Creating a JFrame to display a DrawPanel.
3 import javax.swing.JFrame;
4
5 public class TestDraw
6 {

```

Fig. 8.19 | Creating a `JFrame` to display a `DrawPanel`. (Part 1 of 2.)

```

7   public static void main(String[] args)
8   {
9       DrawPanel panel = new DrawPanel();
10      JFrame app = new JFrame();
11
12      app.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
13      app.add(panel);
14      app.setSize(300, 300);
15      app.setVisible(true);
16  }
17 } // end class TestDraw

```

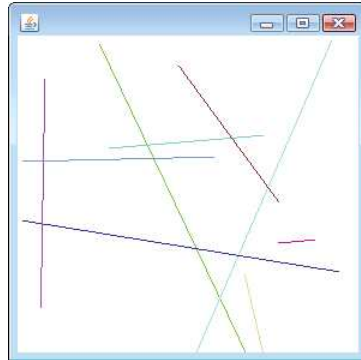


Fig. 8.19 | Creating a JFrame to display a DrawPanel. (Part 2 of 2.)

GUI and Graphics Case Study Exercise

8.1 Extend the program in Figs. 8.17–8.19 to randomly draw rectangles and ovals. Create classes `MyRectangle` and `MyOval`. Both of these classes should include $x1$, $y1$, $x2$, $y2$ coordinates, a color and a boolean flag to determine whether the shape is filled. Declare a constructor in each class with arguments for initializing all the instance variables. To help draw rectangles and ovals, each class should provide methods `getUpperLeftX`, `getUpperLeftY`, `getWidth` and `getHeight` that calculate the upper-left x -coordinate, upper-left y -coordinate, width and height, respectively. The upper-left x -coordinate is the smaller of the two x -coordinate values, the upper-left y -coordinate is the smaller of the two y -coordinate values, the width is the absolute value of the difference between the two x -coordinate values, and the height is the absolute value of the difference between the two y -coordinate values.

Class `DrawPanel`, which extends `JPanel` and handles the creation of the shapes, should declare three arrays, one for each shape type. The length of each array should be a random number between 1 and 5. The constructor of class `DrawPanel` will fill each array with shapes of random position, size, color and fill.

In addition, modify all three shape classes to include the following:

- A constructor with no arguments that sets the shape's coordinates to 0, the color of the shape to `Color.BLACK`, and the filled property to `false` (`MyRectangle` and `MyOval` only).
- Set* methods for the instance variables in each class. The methods that set a coordinate value should verify that the argument is greater than or equal to zero before setting the coordinate—if it's not, they should set the coordinate to zero. The constructor should call the *set* methods rather than initialize the local variables directly.
- Get* methods for the instance variables in each class. Method `draw` should reference the coordinates by the *get* methods rather than access them directly.

8.17 Wrap-Up

In this chapter, we presented additional class concepts. The `Time` class case study showed a complete class declaration consisting of `private` data, overloaded `public` constructors for initialization flexibility, `set` and `get` methods for manipulating the class's data, and methods that returned `String` representations of a `Time` object in two different formats. You also learned that every class can declare a `toString` method that returns a `String` representation of an object of the class and that method `toString` can be called implicitly whenever an object of a class appears in the code where a `String` is expected. We showed how to throw an exception to indicate that a problem has occurred.

You learned that the `this` reference is used implicitly in a class's instance methods to access the class's instance variables and other instance methods. You also saw explicit uses of the `this` reference to access the class's members (including shadowed fields) and how to use keyword `this` in a constructor to call another constructor of the class.

We discussed the differences between default constructors provided by the compiler and no-argument constructors provided by the programmer. You learned that a class can have references to objects of other classes as members—a concept known as composition. You learned more about `enum` types and how they can be used to create a set of constants for use in a program. You learned about Java's garbage-collection capability and how it (unpredictably) reclaims the memory of objects that are no longer used. The chapter explained the motivation for `static` fields in a class and demonstrated how to declare and use `static` fields and methods in your own classes. You also learned how to declare and initialize `final` variables.

You learned that fields declared without an access modifier are given package access by default. You saw the relationship between classes in the same package that allows each class in a package to access the package-access members of other classes in the package. Finally, we demonstrated how to use class `BigDecimal` to perform precise monetary calculations.

In the next chapter, you'll learn about an important aspect of object-oriented programming in Java—inheritance. You'll see that all classes in Java are related by inheritance, directly or indirectly, to the class called `Object`. You'll also begin to understand how the relationships between classes enable you to build more powerful apps.

Summary

Section 8.2 Time Class Case Study

- The `public` methods of a class are also known as the class's `public` services or `public` interface (p. 316). They present to the class's clients a view of the services the class provides.
- A class's `private` members are not accessible to its clients.
- `String` class static method `format` (p. 318) is similar to method `System.out.printf` except that `format` returns a formatted `String` rather than displaying it in a command window.
- All objects in Java have a `toString` method that returns a `String` representation of the object. Method `toString` is called implicitly when an object appears in code where a `String` is needed.

Section 8.3 Controlling Access to Members

- The access modifiers `public` and `private` control access to a class's variables and methods.

- The primary purpose of `public` methods is to present to the class's clients a view of the services the class provides. Clients need not be concerned with how the class accomplishes its tasks.
- A class's private variables and private methods (i.e., its implementation details) are not accessible to its clients.

Section 8.4 Referring to the Current Object's Members with the `this` Reference

- An instance method of an object implicitly uses keyword `this` (p. 322) to refer to the object's instance variables and other methods. Keyword `this` can also be used explicitly.
- The compiler produces a separate file with the `.class` extension for every compiled class.
- If a local variable has the same name as a class's field, the local variable shadows the field. You can use the `this` reference in a method to refer to the shadowed field explicitly.

Section 8.5 Time Class Case Study: Overloaded Constructors

- Overloaded constructors enable objects of a class to be initialized in different ways. The compiler differentiates overloaded constructors (p. 324) by their signatures.
- To call one constructor of a class from another of the same class, you can use the `this` keyword followed by parentheses containing the constructor arguments. If used, such a constructor call must appear as the first statement in the constructor's body.

Section 8.6 Default and No-Argument Constructors

- If no constructors are provided in a class, the compiler creates a default constructor.
- If a class declares constructors, the compiler will not create a default constructor. In this case, you must declare a no-argument constructor (p. 327) if default initialization is required.

Section 8.7 Notes on Set and Get Methods

- *Set* methods are commonly called mutator methods (p. 331) because they typically change a value. *Get* methods are commonly called accessor methods (p. 331) or query methods. A predicate method (p. 332) tests whether a condition is true or false.

Section 8.8 Composition

- A class can have references to objects of other classes as members. This is called composition (p. 332) and is sometimes referred to as a *has-a* relationship.

Section 8.9 enum Types

- All `enum` types (p. 335) are reference types. An `enum` type is declared with an `enum` declaration, which is a comma-separated list of `enum` constants. The declaration may optionally include other components of traditional classes, such as constructors, fields and methods.
- `enum` constants are implicitly `final`, because they declare constants that should not be modified.
- `enum` constants are implicitly `static`.
- Any attempt to create an object of an `enum` type with operator `new` results in a compilation error.
- `enum` constants can be used anywhere constants can be used, such as in the case labels of `switch` statements and to control enhanced `for` statements.
- Each `enum` constant in an `enum` declaration is optionally followed by arguments which are passed to the `enum` constructor.
- For every `enum`, the compiler generates a `static` method called `values` (p. 336) that returns an array of the `enum`'s constants in the order in which they were declared.
- `EnumSet` `static` method `range` (p. 337) receives the first and last `enum` constants in a range and returns an `EnumSet` that contains all the constants between these two constants, inclusive.

Section 8.10 Garbage Collection

- The Java Virtual Machine (JVM) performs automatic garbage collection (p. 338) to reclaim the memory occupied by objects that are no longer in use. When there are no more references to an object, the object is eligible for garbage collection. The memory for such an object can be reclaimed when the JVM executes its garbage collector.

Section 8.11 static Class Members

- A static variable (p. 338) represents classwide information that's shared among the class's objects.
- static variables have class scope. A class's public static members can be accessed through a reference to any object of the class, or they can be accessed by qualifying the member name with the class name and a dot (.). Client code can access a class's private static class members only through methods of the class.
- static class members exist as soon as the class is loaded into memory.
- A method declared static cannot access a class's instance variables and instance methods, because a static method can be called even when no objects of the class have been instantiated.
- The this reference cannot be used in a static method.

Section 8.12 static Import

- A static import declaration (p. 342) enables you to refer to imported static members without the class name and a dot (.). A single static import declaration imports one static member, and a static import on demand imports all static members of a class.

Section 8.13 final Instance Variables

- In the context of an app's code, the principle of least privilege (p. 343) states that code should be granted only the amount of privilege and access that it needs to accomplish its designated task.
- Keyword final specifies that a variable is not modifiable. Such variables must be initialized when they're declared or by each of a class's constructors.

Section 8.14 Package Access

- If no access modifier is specified for a method or variable when it's declared in a class, the method or variable is considered to have package access (p. 344).

Section 8.15 Using BigDecimal for Precise Monetary Calculations

- Any application that requires precise floating-point calculations without rounding errors—such as those in financial applications—should instead use class BigDecimal (package java.math; p. 346).
- BigDecimal static method valueOf (p. 347) with a double argument returns a BigDecimal that represents the exact value specified.
- BigDecimal method add (p. 347) adds its argument BigDecimal to the BigDecimal on which the method is called and returns the result.
- BigDecimal provides the constants ONE (1), ZERO (0) and TEN (10).
- BigDecimal method pow (p. 347) raises its first argument to the power specified in its second argument.
- BigDecimal method multiply (p. 347) multiplies its argument BigDecimal with the BigDecimal on which the method is called and returns the result.
- Class NumberFormat (package java.text; p. 347) provides capabilities for formatting numeric values as locale-specific Strings. The class's static method getCurrencyInstance returns a pre-configured NumberFormat for local-specific currency values. NumberFormat method format performs the formatting.

- Locale-specific formatting is an important part of internationalization—the process of customizing your applications for users’ various locales and spoken languages.
- `BigDecimal` gives you control over how values are rounded—by default all calculations are exact and no rounding occurs. If you do not specify how to round `BigDecimal` values and a given value cannot be represented exactly an `ArithmeticException` occurs.
- You can specify the rounding mode for `BigDecimal` by supplying a `MathContext` object (package `java.math`) to class `BigDecimal`’s constructor when you create a `BigDecimal`. You may also provide a `MathContext` to various `BigDecimal` methods that perform calculations. By default, each pre-configured `MathContext` uses so called “bankers rounding.”
- A `BigDecimal`’s scale is the number of digits to the right of its decimal point. If you need a `BigDecimal` rounded to a specific digit, you can call `BigDecimal` method `setScale`.

Self-Review Exercise

8.1 Fill in the blanks in each of the following statements:

- A(n) _____ imports all static members of a class.
- `String` class static method _____ is similar to method `System.out.printf`, but returns a formatted `String` rather than displaying a `String` in a command window.
- If a method contains a local variable with the same name as one of its class’s fields, the local variable _____ the field in that method’s scope.
- The `public` methods of a class are also known as the class’s _____ or _____.
- A(n) _____ declaration specifies one class to import.
- If a class declares constructors, the compiler will not create a(n) _____.
- An object’s _____ method is called implicitly when an object appears in code where a `String` is needed.
- `Get` methods are commonly called _____ or _____.
- A(n) _____ method tests whether a condition is true or false.
- For every `enum`, the compiler generates a static method called _____ that returns an array of the `enum`’s constants in the order in which they were declared.
- Composition is sometimes referred to as a(n) _____ relationship.
- A(n) _____ declaration contains a comma-separated list of constants.
- A(n) _____ variable represents classwide information that’s shared by all the objects of the class.
- A(n) _____ declaration imports one static member.
- The _____ states that code should be granted only the amount of privilege and access that it needs to accomplish its designated task.
- Keyword _____ specifies that a variable is not modifiable after initialization in a declaration or constructor.
- A(n) _____ declaration imports only the classes that the program uses from a particular package.
- `Set` methods are commonly called _____ because they typically change a value.
- Use class _____ to perform precise monetary calculations.
- Use the _____ statement to indicate that a problem has occurred.

Answers to Self-Review Exercise

- 8.1** a) static import on demand. b) `format`. c) shadows. d) `public` services, `public` interface. e) single-type-import. f) default constructor. g) `toString`. h) accessor methods, query methods. i) predicate. j) values. k) *has-a*. l) `enum`. m) static. n) single static import. o) principle of least privilege. p) `final`. q) type-import-on-demand. r) mutator methods. s) `BigDecimal`. t) `throw`.

Exercises

8.2 (*Based on Section 8.14*) Explain the notion of package access in Java. Explain the negative aspects of package access.

8.3 What happens when a return type, even `void`, is specified for a constructor?

8.4 (*Rectangle Class*) Create a class `Rectangle` with attributes `length` and `width`, each of which defaults to 1. Provide methods that calculate the rectangle's perimeter and area. It has *set* and *get* methods for both `length` and `width`. The *set* methods should verify that `length` and `width` are each floating-point numbers larger than 0.0 and less than 20.0. Write a program to test class `Rectangle`.

8.5 (*Modifying the Internal Data Representation of a Class*) It would be perfectly reasonable for the `Time2` class of Fig. 8.5 to represent the time internally as the number of seconds since midnight rather than the three integer values `hour`, `minute` and `second`. Clients could use the same public methods and get the same results. Modify the `Time2` class of Fig. 8.5 to implement the time as the number of seconds since midnight and show that no change is visible to the clients of the class.

8.6 (*Savings Account Class*) Create class `SavingsAccount`. Use a static variable `annualInterestRate` to store the annual interest rate for all account holders. Each object of the class contains a private instance variable `savingsBalance` indicating the amount the saver currently has on deposit. Provide method `calculateMonthlyInterest` to calculate the monthly interest by multiplying the `savingsBalance` by `annualInterestRate` divided by 12—this interest should be added to `savingsBalance`. Provide a static method `modifyInterestRate` that sets the `annualInterestRate` to a new value. Write a program to test class `SavingsAccount`. Instantiate two `savingsAccount` objects, `saver1` and `saver2`, with balances of \$2000.00 and \$3000.00, respectively. Set `annualInterestRate` to 4%, then calculate the monthly interest for each of 12 months and print the new balances for both savers. Next, set the `annualInterestRate` to 5%, calculate the next month's interest and print the new balances for both savers.

8.7 (*Enhancing Class Time2*) Modify class `Time2` of Fig. 8.5 to include a `tick` method that increments the time stored in a `Time2` object by one second. Provide method `incrementMinute` to increment the minute by one and method `incrementHour` to increment the hour by one. Write a program that tests the `tick` method, the `incrementMinute` method and the `incrementHour` method to ensure that they work correctly. Be sure to test the following cases:

- a) incrementing into the next minute,
- b) incrementing into the next hour and
- c) incrementing into the next day (i.e., 11:59:59 PM to 12:00:00 AM).

8.8 (*Enhancing Class Date*) Modify class `Date` of Fig. 8.7 to perform error checking on the initializer values for instance variables `month`, `day` and `year` (currently it validates only the month and day). Provide a method `nextDay` to increment the day by one. Write a program that tests method `nextDay` in a loop that prints the date during each iteration to illustrate that the method works correctly. Test the following cases:

- a) incrementing into the next month and
- b) incrementing into the next year.

8.9 Rewrite the code in Fig. 8.14 to use a separate `import` declaration for each static member of class `Math` that's used in the example.

8.10 Write an enum type `TrafficLight`, whose constants (`RED`, `GREEN`, `YELLOW`) take one parameter—the duration of the light. Write a program to test the `TrafficLight` enum so that it displays the enum constants and their durations.

8.11 (*Complex Numbers*) Create a class called `Complex` for performing arithmetic with complex numbers. Complex numbers have the form

$$\text{realPart} + \text{imaginaryPart} * i$$

where i is

$$\sqrt{-1}$$

Write a program to test your class. Use floating-point variables to represent the private data of the class. Provide a constructor that enables an object of this class to be initialized when it's declared. Provide a no-argument constructor with default values in case no initializers are provided. Provide public methods that perform the following operations:

- a) Add two Complex numbers: The real parts are added together and the imaginary parts are added together.
- b) Subtract two Complex numbers: The real part of the right operand is subtracted from the real part of the left operand, and the imaginary part of the right operand is subtracted from the imaginary part of the left operand.
- c) Print Complex numbers in the form (*realPart*, *imaginaryPart*).

8.12 (Date and Time Class) Create class `DateAndTime` that combines the modified `Time2` class of Exercise 8.7 and the modified `Date` class of Exercise 8.8. Modify method `incrementHour` to call method `nextDay` if the time is incremented into the next day. Modify methods `toString` and `toUniversalString` to output the date in addition to the time. Write a program to test the new class `DateAndTime`. Specifically, test incrementing the time to the next day.

8.13 (Set of Integers) Create class `IntegerSet`. Each `IntegerSet` object can hold integers in the range 0–100. The set is represented by an array of `boolean`s. Array element `a[i]` is `true` if integer i is in the set. Array element `a[j]` is `false` if integer j is not in the set. The no-argument constructor initializes the array to the “empty set” (i.e., all `false` values).

Provide the following methods: The static method `union` creates a set that's the set-theoretic union of two existing sets (i.e., an element of the new set's array is set to `true` if that element is `true` in either or both of the existing sets—otherwise, the new set's element is set to `false`). The static method `intersection` creates a set which is the set-theoretic intersection of two existing sets (i.e., an element of the new set's array is set to `false` if that element is `false` in either or both of the existing sets—otherwise, the new set's element is set to `true`). Method `insertElement` inserts a new integer k into a set (by setting `a[k]` to `true`). Method `deleteElement` deletes integer m (by setting `a[m]` to `false`). Method `toString` returns a `String` containing a set as a list of numbers separated by spaces. Include only those elements that are present in the set. Use `---` to represent an empty set. Method `isEqualTo` determines whether two sets are equal. Write a program to test class `IntegerSet`. Instantiate several `IntegerSet` objects. Test that all your methods work properly.

8.14 (Date Class) Create class `Date` with the following capabilities:

- a) Output the date in multiple formats, such as

```
MM/DD/YYYY
June 14, 1992
DDD YYYY
```

- b) Use overloaded constructors to create `Date` objects initialized with dates of the formats in part (a). In the first case the constructor should receive three integer values. In the second case it should receive a `String` and two integer values. In the third case it should receive two integer values, the first of which represents the day number in the year. [Hint: To convert the `String` representation of the month to a numeric value, compare `Strings` using the `equals` method. For example, if `s1` and `s2` are `Strings`, the method call `s1.equals(s2)` returns `true` if the `Strings` are identical and otherwise returns `false`.]

8.15 (Rational Numbers) Create a class called `Rational` for performing arithmetic with fractions. Write a program to test your class. Use integer variables to represent the private instance variables of the class—the numerator and the denominator. Provide a constructor that enables an object of

this class to be initialized when it's declared. The constructor should store the fraction in reduced form. The fraction

$$2/4$$

is equivalent to $1/2$ and would be stored in the object as 1 in the numerator and 2 in the denominator. Provide a no-argument constructor with default values in case no initializers are provided. Provide public methods that perform each of the following operations:

- a) Add two `Rational` numbers: The result of the addition should be stored in reduced form. Implement this as a static method.
- b) Subtract two `Rational` numbers: The result of the subtraction should be stored in reduced form. Implement this as a static method.
- c) Multiply two `Rational` numbers: The result of the multiplication should be stored in reduced form. Implement this as a static method.
- d) Divide two `Rational` numbers: The result of the division should be stored in reduced form. Implement this as a static method.
- e) Return a `String` representation of a `Rational` number in the form a/b , where a is the numerator and b is the denominator.
- f) Return a `String` representation of a `Rational` number in floating-point format. (Consider providing formatting capabilities that enable the user of the class to specify the number of digits of precision to the right of the decimal point.)

8.16 (*Huge Integer Class*) Create a class `HugeInteger` which uses a 40-element array of digits to store integers as large as 40 digits each. Provide methods `parse`, `toString`, `add` and `subtract`. Method `parse` should receive a `String`, extract each digit using method `charAt` and place the integer equivalent of each digit into the integer array. For comparing `HugeInteger` objects, provide the following methods: `isEqualTo`, `isNotEqualTo`, `isGreaterThan`, `isLessThan`, `isGreaterThanOrEqualTo` and `isLessThanOrEqualTo`. Each of these is a predicate method that returns `true` if the relationship holds between the two `HugeInteger` objects and returns `false` if the relationship does not hold. Provide a predicate method `isZero`. If you feel ambitious, also provide methods `multiply`, `divide` and `remainder`. [Note: Primitive `boolean` values can be output as the word “true” or the word “false” with format specifier `%b`.]

8.17 (*Tic-Tac-Toe*) Create a class `TicTacToe` that will enable you to write a program to play Tic-Tac-Toe. The class contains a private 3-by-3 two-dimensional array. Use an `enum` type to represent the value in each cell of the array. The `enum`'s constants should be named `X`, `O` and `EMPTY` (for a position that does not contain an `X` or an `O`). The constructor should initialize the board elements to `EMPTY`. Allow two human players. Wherever the first player moves, place an `X` in the specified square, and place an `O` wherever the second player moves. Each move must be to an empty square. After each move, determine whether the game has been won and whether it's a draw. If you feel ambitious, modify your program so that the computer makes the moves for one of the players. Also, allow the player to specify whether he or she wants to go first or second. If you feel exceptionally ambitious, develop a program that will play three-dimensional Tic-Tac-Toe on a 4-by-4-by-4 board [Note: This is an extremely challenging project!].

8.18 (*Account Class with BigDecimal balance*) Rewrite the `Account` class of Section 3.5 to store the balance as a `BigDecimal` object and to perform all calculations using `BigDecimal`s.

Making a Difference

8.19 (*Project: Emergency Response Class*) The North American emergency response service, 9-1-1, connects callers to a local Public Service Answering Point (PSAP). Traditionally, the PSAP would ask the caller for identification information—including the caller's address, phone number and the nature of the emergency, then dispatch the appropriate emergency responders (such as the police, an ambu-

lance or the fire department). *Enhanced 9-1-1 (or E9-1-1)* uses computers and databases to determine the caller's physical address, directs the call to the nearest PSAP, and displays the caller's phone number and address to the call taker. *Wireless Enhanced 9-1-1* provides call takers with identification information for wireless calls. Rolled out in two phases, the first phase required carriers to provide the wireless phone number and the location of the cell site or base station transmitting the call. The second phase required carriers to provide the location of the caller (using technologies such as GPS). To learn more about 9-1-1, visit <http://www.fcc.gov/pshs/services/911-services/Welcome.html> and <http://people.howstuffworks.com/9-1-1.htm>.

An important part of creating a class is determining the class's attributes (instance variables). For this class design exercise, research 9-1-1 services on the Internet. Then, design a class called *Emergency* that might be used in an object-oriented 9-1-1 emergency response system. List the attributes that an object of this class might use to represent the emergency. For example, the class might include information on who reported the emergency (including their phone number), the location of the emergency, the time of the report, the nature of the emergency, the type of response and the status of the response. The class attributes should completely describe the nature of the problem and what's happening to resolve that problem.

9

Object-Oriented Programming: Inheritance

*Say not you know another
entirely,
till you have divided an
inheritance with him.*

—Johann Kasper Lavater

*This method is to define as the
number of a class the class of all
classes similar to the given class.*

—Bertrand Russell

Objectives

In this chapter you'll:

- Understand inheritance and how to use it to develop new classes based on existing classes.
- Learn the notions of superclasses and subclasses and the relationship between them.
- Use keyword **extends** to create a class that inherits attributes and behaviors from another class.
- Use access modifier **protected** in a superclass to give subclass methods access to these superclass members.
- Access superclass members with **super** from a subclass.
- Learn how constructors are used in inheritance hierarchies.
- Learn about the methods of class **Object**, the direct or indirect superclass of all classes.



- 9.1 Introduction
- 9.2 Superclasses and Subclasses
- 9.3 protected Members
- 9.4 Relationship Between Superclasses and Subclasses
 - 9.4.1 Creating and Using a `CommissionEmployee` Class
 - 9.4.2 Creating and Using a `BasePlusCommissionEmployee` Class
 - 9.4.3 Creating a `CommissionEmployee`–`BasePlusCommissionEmployee` Inheritance Hierarchy
 - 9.4.4 `CommissionEmployee`–`BasePlusCommissionEmployee` Inheritance Hierarchy Using `protected` Instance Variables
 - 9.4.5 `CommissionEmployee`–`BasePlusCommissionEmployee` Inheritance Hierarchy Using `private` Instance Variables
- 9.5 Constructors in Subclasses
- 9.6 Class Object
- 9.7 (Optional) GUI and Graphics Case Study: Displaying Text and Images Using `Labels`
- 9.8 Wrap-Up

Summary | Self-Review Exercises | Answers to Self-Review Exercises | Exercises

9.1 Introduction

This chapter continues our discussion of object-oriented programming (OOP) by introducing **inheritance**, in which a new class is created by acquiring an existing class's members and possibly embellishing them with new or modified capabilities. With inheritance, you can save time during program development by basing new classes on existing proven and debugged high-quality software. This also increases the likelihood that a system will be implemented and maintained effectively.

When creating a class, rather than declaring completely new members, you can designate that the new class should *inherit* the members of an existing class. The existing class is called the **superclass**, and the new class is the **subclass**. (The C++ programming language refers to the superclass as the **base class** and the subclass as the **derived class**.) A subclass can become a superclass for future subclasses.

A subclass can add its own fields and methods. Therefore, a subclass is *more specific* than its superclass and represents a more specialized group of objects. The subclass exhibits the behaviors of its superclass and can modify those behaviors so that they operate appropriately for the subclass. This is why inheritance is sometimes referred to as **specialization**.

The **direct superclass** is the superclass from which the subclass explicitly inherits. An **indirect superclass** is any class above the direct superclass in the **class hierarchy**, which defines the inheritance relationships among classes—as you'll see in Section 9.2, diagrams help you understand these relationships. In Java, the class hierarchy begins with class `Object` (in package `java.lang`), which *every* class in Java directly or indirectly **extends** (or “inherits from”). Section 9.6 lists the methods of class `Object` that are inherited by all other Java classes. Java supports only **single inheritance**, in which each class is derived from exactly *one* direct superclass. Unlike C++, Java does *not* support multiple inheritance (which occurs when a class is derived from more than one direct superclass). Chapter 10, Object-Oriented Programming: Polymorphism and Interfaces, explains how to use Java *interfaces* to realize many of the benefits of multiple inheritance while avoiding the associated problems.

We distinguish between the *is-a relationship* and the *has-a relationship*. *Is-a* represents inheritance. In an *is-a* relationship, *an object of a subclass can also be treated as an object of its superclass*—e.g., a car *is a* vehicle. By contrast, *has-a* represents composition (see Chapter 8). In a *has-a* relationship, *an object contains as members references to other objects*—e.g., a car *has a* steering wheel (and a car object has a reference to a steering-wheel object).

New classes can inherit from classes in **class libraries**. Organizations develop their own class libraries and can take advantage of others available worldwide. Some day, most new software likely will be constructed from **standardized reusable components**, just as automobiles and most computer hardware are constructed today. This will facilitate the rapid development of more powerful, abundant and economical software.

9.2 Superclasses and Subclasses

Often, an object of one class *is an* object of another class as well. For example, a CarLoan *is a* Loan as are HomeImprovementLoans and MortgageLoans. Thus, in Java, class CarLoan can be said to inherit from class Loan. In this context, class Loan is a superclass and class CarLoan is a subclass. A CarLoan *is a* specific type of Loan, but it's incorrect to claim that every Loan *is a* CarLoan—the Loan could be any type of loan. Figure 9.1 lists several simple examples of superclasses and subclasses—superclasses tend to be “more general” and subclasses “more specific.”

Superclass	Subclasses
Student	GraduateStudent, UndergraduateStudent
Shape	Circle, Triangle, Rectangle, Sphere, Cube
Loan	CarLoan, HomeImprovementLoan, MortgageLoan
Employee	Faculty, Staff
BankAccount	CheckingAccount, SavingsAccount

Fig. 9.1 | Inheritance examples.

Because every subclass object *is an* object of its superclass, and one superclass can have many subclasses, the set of objects represented by a superclass is often larger than the set of objects represented by any of its subclasses. For example, the superclass Vehicle represents *all* vehicles, including cars, trucks, boats, bicycles and so on. By contrast, subclass Car represents a smaller, more specific subset of vehicles.

University Community Member Hierarchy

Inheritance relationships form treelike *hierarchical* structures. A superclass exists in a hierarchical relationship with its subclasses. Let's develop a sample class hierarchy (Fig. 9.2), also called an **inheritance hierarchy**. A university community has thousands of members, including employees, students and alumni. Employees are either faculty or staff members. Faculty members are either administrators (e.g., deans and department chairpersons) or teachers. The hierarchy could contain many other classes. For example, students can be graduate or undergraduate students. Undergraduate students can be freshmen, sophomores, juniors or seniors.

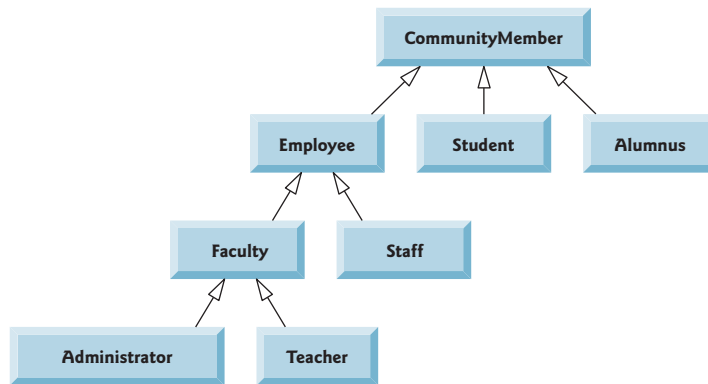


Fig. 9.2 | Inheritance hierarchy UML class diagram for university `CommunityMembers`.

Each arrow in the hierarchy represents an *is-a* relationship. As we follow the arrows upward in this class hierarchy, we can state, for example, that “an `Employee` *is a* `CommunityMember`” and “a `Teacher` *is a* `Faculty` member.” `CommunityMember` is the direct superclass of `Employee`, `Student` and `Alumnus` and is an indirect superclass of all the other classes in the diagram. Starting from the bottom, you can follow the arrows and apply the *is-a* relationship up to the topmost superclass. For example, an `Administrator` *is a* `Faculty` member, *is an* `Employee`, *is a* `CommunityMember` and, of course, *is an* `Object`.

Shape Hierarchy

Now consider the Shape inheritance hierarchy in Fig. 9.3. This hierarchy begins with superclass `Shape`, which is extended by subclasses `TwoDimensionalShape` and `ThreeDimensionalShape`—Shapes are either `TwoDimensionalShapes` or `ThreeDimensionalShapes`. The third level of this hierarchy contains *specific* types of `TwoDimensionalShapes` and `ThreeDimensionalShapes`. As in Fig. 9.2, we can follow the arrows from the bottom of the diagram to the topmost superclass in this class hierarchy to identify several *is-a* relationships. For example, a `Triangle` *is a* `TwoDimensionalShape` and *is a* `Shape`, while a `Sphere` *is a* `ThreeDimensionalShape` and *is a* `Shape`. This hierarchy could contain many other classes. For example, ellipses and trapezoids also are `TwoDimensionalShapes`.

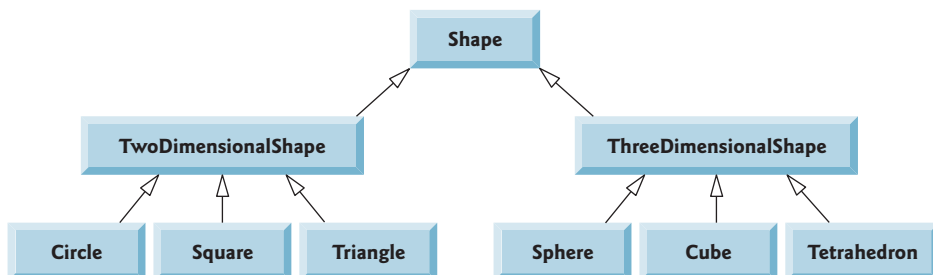


Fig. 9.3 | Inheritance hierarchy UML class diagram for Shapes.

Not every class relationship is an inheritance relationship. In Chapter 8, we discussed the *has-a* relationship, in which classes have members that are references to objects of other classes. Such relationships create classes by *composition* of existing classes. For example, given the classes `Employee`, `BirthDate` and `TelephoneNumber`, it's improper to say that an `Employee` *is a* `BirthDate` or that an `Employee` *is a* `TelephoneNumber`. However, an `Employee` *has a* `BirthDate`, and an `Employee` *has a* `TelephoneNumber`.

It's possible to treat superclass objects and subclass objects similarly—their commonalities are expressed in the superclass's members. Objects of all classes that extend a common superclass can be treated as objects of that superclass—such objects have an *is-a* relationship with the superclass. Later in this chapter and in Chapter 10, we consider many examples that take advantage of the *is-a* relationship.

A subclass can customize methods that it inherits from its superclass. To do this, the subclass **overrides** (*redefines*) the superclass method with an appropriate implementation, as we'll see in the chapter's code examples.

9.3 protected Members

Chapter 8 discussed access modifiers `public` and `private`. A class's `public` members are accessible wherever the program has a *reference* to an *object* of that class or one of its *subclasses*. A class's `private` members are accessible only within the class itself. In this section, we introduce the access modifier **protected**. Using `protected` access offers an intermediate level of access between `public` and `private`. A superclass's `protected` members can be accessed by members of that superclass, by members of its subclasses and by members of other classes in the *same package*—`protected` members also have *package access*.

All `public` and `protected` superclass members retain their original access modifier when they become members of the subclass—`public` members of the superclass become `public` members of the subclass, and `protected` members of the superclass become `protected` members of the subclass. A superclass's `private` members are *not* accessible outside the class itself. Rather, they're *hidden* from its subclasses and can be accessed only through the `public` or `protected` methods inherited from the superclass.

Subclass methods can refer to `public` and `protected` members inherited from the superclass simply by using the member names. When a subclass method *overrides* an inherited superclass method, the *superclass* version of the method can be accessed from the *subclass* by preceding the superclass method name with keyword **super** and a dot (`.`) separator. We discuss accessing overridden members of the superclass in Section 9.4.



Software Engineering Observation 9.1

Methods of a subclass cannot directly access private members of their superclass. A subclass can change the state of private superclass instance variables only through non-private methods provided in the superclass and inherited by the subclass.



Software Engineering Observation 9.2

Declaring private instance variables helps you test, debug and correctly modify systems. If a subclass could access its superclass's private instance variables, classes that inherit from that subclass could access the instance variables as well. This would propagate access to what should be private instance variables, and the benefits of information hiding would be lost.

9.4 Relationship Between Superclasses and Subclasses

We now use an inheritance hierarchy containing types of *employees* in a company's payroll application to discuss the relationship between a superclass and its subclass. In this company, *commission employees* (who will be represented as objects of a superclass) are paid a percentage of their sales, while *base-salaried commission employees* (who will be represented as objects of a subclass) receive a base salary *plus* a percentage of their sales.

We divide our discussion of the relationship between these classes into five examples. The first declares class `CommissionEmployee`, which directly inherits from class `Object` and declares as `private` instance variables a first name, last name, social security number, commission rate and gross (i.e., total) sales amount.

The second example declares class `BasePlusCommissionEmployee`, which also directly inherits from class `Object` and declares as `private` instance variables a first name, last name, social security number, commission rate, gross sales amount *and* base salary. We create this class by *writing every line of code* the class requires—we'll soon see that it's much more efficient to create it by inheriting from class `CommissionEmployee`.

The third example declares a new `BasePlusCommissionEmployee` class that *extends* class `CommissionEmployee` (i.e., a `BasePlusCommissionEmployee` *is a* `CommissionEmployee` who also has a base salary). This *software reuse lets us write much less code* when developing the new subclass. In this example, class `BasePlusCommissionEmployee` attempts to access class `CommissionEmployee`'s private members—this results in compilation errors, because the subclass *cannot* access the superclass's private instance variables.

The fourth example shows that if `CommissionEmployee`'s instance variables are declared as `protected`, the `BasePlusCommissionEmployee` subclass *can* access that data directly. Both `BasePlusCommissionEmployee` classes contain identical functionality, but we show how the inherited version is easier to create and manage.

After we discuss the convenience of using `protected` instance variables, we create the fifth example, which sets the `CommissionEmployee` instance variables back to `private` to enforce good software engineering. Then we show how the `BasePlusCommissionEmployee` subclass can use `CommissionEmployee`'s public methods to manipulate (in a controlled manner) the private instance variables inherited from `CommissionEmployee`.

9.4.1 Creating and Using a `CommissionEmployee` Class

We begin by declaring class `CommissionEmployee` (Fig. 9.4). Line 4 begins the class declaration and indicates that class `CommissionEmployee` **extends** (i.e., *inherits from*) class **Object** (from package `java.lang`). This causes class `CommissionEmployee` to inherit the class `Object`'s methods—class `Object` does not have any fields. If you don't explicitly specify which class a new class extends, the class extends `Object` implicitly. For this reason, you typically will not include “`extends Object`” in your code—we do so in this one example only for demonstration purposes.

Overview of Class `CommissionEmployee`'s Methods and Instance Variables

Class `CommissionEmployee`'s public services include a constructor (lines 13–34) and methods `earnings` (lines 87–90) and `toString` (lines 93–101). Lines 37–52 declare public *get* methods for the class's `final` instance variables (declared in lines 6–8) `firstName`, `lastName` and `socialSecurityNumber`. These three instance variables are declared `final` because they do not need to be modified after they're initialized—this is also why we do

not provide corresponding *set* methods. Lines 55–84 declare public *set* and *get* methods for the class's *grossSales* and *commissionRate* instance variables (declared in lines 9–10). The class declares its instance variables as *private*, so objects of other classes cannot directly access these variables.

```

1  // Fig. 9.4: CommissionEmployee.java
2  // CommissionEmployee class represents an employee paid a
3  // percentage of gross sales.
4  public class CommissionEmployee extends Object
5  {
6      private final String firstName;
7      private final String lastName;
8      private final String socialSecurityNumber;
9      private double grossSales; // gross weekly sales
10     private double commissionRate; // commission percentage
11
12     // five-argument constructor
13     public CommissionEmployee(String firstName, String lastName,
14         String socialSecurityNumber, double grossSales,
15         double commissionRate)
16     {
17         // implicit call to Object's default constructor occurs here
18
19         // if grossSales is invalid throw exception
20         if (grossSales < 0.0)
21             throw new IllegalArgumentException(
22                 "Gross sales must be >= 0.0");
23
24         // if commissionRate is invalid throw exception
25         if (commissionRate <= 0.0 || commissionRate >= 1.0)
26             throw new IllegalArgumentException(
27                 "Commission rate must be > 0.0 and < 1.0");
28
29         this.firstName = firstName;
30         this.lastName = lastName;
31         this.socialSecurityNumber = socialSecurityNumber;
32         this.grossSales = grossSales;
33         this.commissionRate = commissionRate;
34     } // end constructor
35
36     // return first name
37     public String getFirstName()
38     {
39         return firstName;
40     }
41
42     // return last name
43     public String getLastName()
44     {
45         return lastName;
46     }

```

Fig. 9.4 | CommissionEmployee class represents an employee paid a percentage of gross sales.
(Part I of 3.)

```
47
48 // return social security number
49 public String getSocialSecurityNumber()
50 {
51     return socialSecurityNumber;
52 }
53
54 // set gross sales amount
55 public void setGrossSales(double grossSales)
56 {
57     if (grossSales < 0.0)
58         throw new IllegalArgumentException(
59             "Gross sales must be >= 0.0");
60
61     this.grossSales = grossSales;
62 }
63
64 // return gross sales amount
65 public double getGrossSales()
66 {
67     return grossSales;
68 }
69
70 // set commission rate
71 public void setCommissionRate(double commissionRate)
72 {
73     if (commissionRate <= 0.0 || commissionRate >= 1.0)
74         throw new IllegalArgumentException(
75             "Commission rate must be > 0.0 and < 1.0");
76
77     this.commissionRate = commissionRate;
78 }
79
80 // return commission rate
81 public double getCommissionRate()
82 {
83     return commissionRate;
84 }
85
86 // calculate earnings
87 public double earnings()
88 {
89     return commissionRate * grossSales;
90 }
91
92 // return String representation of CommissionEmployee object
93 @Override // indicates that this method overrides a superclass method
94 public String toString()
95 {
96     return String.format("%s: %s %s\n%s: %s\n%s: %.2f\n%s: %.2f",
97         "commission employee", firstName, lastName,
98         "social security number", socialSecurityNumber,
```

Fig. 9.4 | CommissionEmployee class represents an employee paid a percentage of gross sales.
(Part 2 of 3.)

```

99         "gross sales", grossSales,
100         "commission rate", commissionRate);
101     }
102 } // end class CommissionEmployee

```

Fig. 9.4 | `CommissionEmployee` class represents an employee paid a percentage of gross sales. (Part 3 of 3.)

Class `CommissionEmployee`'s Constructor

Constructors are *not* inherited, so class `CommissionEmployee` does not inherit class `Object`'s constructor. However, a superclass's constructors are still available to be called by subclasses. In fact, Java requires that *the first task of any subclass constructor is to call its direct superclass's constructor*, either explicitly or implicitly (if no constructor call is specified), to ensure that the instance variables inherited from the superclass are initialized properly. The syntax for calling a superclass constructor explicitly is discussed in Section 9.4.3. In this example, class `CommissionEmployee`'s constructor calls class `Object`'s constructor implicitly. If the code does not include an explicit call to the superclass constructor, Java *implicitly* calls the superclass's default or *no-argument* constructor. The comment in line 17 of Fig. 9.4 indicates where the implicit call to the superclass `Object`'s default constructor is made (you do *not* write the code for this call). `Object`'s default constructor does nothing. Even if a class does not have constructors, the default constructor that the compiler implicitly declares for the class will call the superclass's default or no-argument constructor.

After the implicit call to `Object`'s constructor, lines 20–22 and 25–27 validate the `grossSales` and `commissionRate` arguments. If these are valid (that is, the constructor does not throw an `IllegalArgumentException`), lines 29–33 assign the constructor's arguments to the class's instance variables.

We did not validate the values of arguments `firstName`, `lastName` and `socialSecurityNumber` before assigning them to the corresponding instance variables. We could validate the first and last names—perhaps to ensure that they're of a reasonable length. Similarly, a social security number could be validated using regular expressions (Section 14.7) to ensure that it contains nine digits, with or without dashes (e.g., 123-45-6789 or 123456789).

Class `CommissionEmployee`'s earnings Method

Method `earnings` (lines 87–90) calculates a `CommissionEmployee`'s earnings. Line 89 multiplies the `commissionRate` by the `grossSales` and returns the result.

Class `CommissionEmployee`'s toString Method and the @Override Annotation

Method `toString` (lines 93–101) is special—it's one of the methods that *every* class inherits directly or indirectly from class `Object` (summarized in Section 9.6). Method `toString` returns a `String` representing an object. It's called implicitly whenever an object must be converted to a `String` representation, such as when an object is output by `printf` or output by `String` method `format` via the `%s` format specifier. Class `Object`'s `toString` method returns a `String` that includes the name of the object's class. It's primarily a placeholder that can be *overridden* by a subclass to specify an appropriate `String` representation of the data in a subclass object. Method `toString` of class `CommissionEmployee` overrides (redefines) class `Object`'s `toString` method. When invoked, `CommissionEmployee`'s `toString`

method uses `String` method format to return a `String` containing information about the `CommissionEmployee`. To override a superclass method, a subclass must declare a method with the *same signature* (method name, number of parameters, parameter types and order of parameter types) as the superclass method—`Object`'s `toString` method takes no parameters, so `CommissionEmployee` declares `toString` with no parameters.

Line 93 uses the optional **@Override annotation** to indicate that the following method declaration (i.e., `toString`) should *override* an *existing* superclass method. This annotation helps the compiler catch a few common errors. For example, in this case, you intend to override superclass method `toString`, which is spelled with a lowercase “t” and an uppercase “S.” If you inadvertently use a lowercase “s,” the compiler will flag this as an error because the superclass does not contain a method named `toString`. If you didn't use the `@Override` annotation, `toString` would be an entirely different method that would *not* be called if a `CommissionEmployee` were used where a `String` was needed.

Another common overriding error is declaring the wrong number or types of parameters in the parameter list. This creates an *unintentional overload* of the superclass method, rather than overriding the existing method. If you then attempt to call the method (with the correct number and types of parameters) on a subclass object, the superclass's version is invoked—potentially leading to subtle logic errors. When the compiler encounters a method declared with `@Override`, it compares the method's signature with the superclass's method signatures. If there isn't an exact match, the compiler issues an error message, such as “method does not override or implement a method from a supertype.” You would then correct your method's signature so that it matches one in the superclass.



Error-Prevention Tip 9.1

Though the `@Override` annotation is optional, declare overridden methods with it to ensure at compilation time that you defined their signatures correctly. It's always better to find errors at compile time rather than at runtime. For this reason, the `toString` methods in Fig. 7.9 and in Chapter 8's examples should have been declared with `@Override`.



Common Programming Error 9.1

It's a compilation error to override a method with a more restricted access modifier—a public superclass method cannot become a protected or private subclass method; a protected superclass method cannot become a private subclass method. Doing so would break the is-a relationship, which requires that all subclass objects be able to respond to method calls made to public methods declared in the superclass. If a public method, could be overridden as a protected or private method, the subclass objects would not be able to respond to the same method calls as superclass objects. Once a method is declared public in a superclass, the method remains public for all that class's direct and indirect subclasses.

Class `CommissionEmployeeTest`

Figure 9.5 tests class `CommissionEmployee`. Lines 9–10 instantiate a `CommissionEmployee` object and invoke `CommissionEmployee`'s constructor (lines 13–34 of Fig. 9.4) to initialize it with “Sue” as the first name, “Jones” as the last name, “222-22-2222” as the social security number, 10000 as the gross sales amount (\$10,000) and .06 as the commission rate (i.e., 6%). Lines 15–24 use `CommissionEmployee`'s `get` methods to retrieve the object's instance-variable values for output. Lines 26–27 invoke the object's `setGrossSales` and `setCommissionRate` methods to change the values of instance variables `grossSales` and `commission`.

Rate. Lines 29–30 output the String representation of the updated `CommissionEmployee`. When an object is output using the `%s` format specifier, the object's `toString` method is invoked *implicitly* to obtain the object's String representation. [Note: In this chapter, we do not use the `earnings` method in each class, but it's used extensively in Chapter 10.]

```

1  // Fig. 9.5: CommissionEmployeeTest.java
2  // CommissionEmployee class test program.
3
4  public class CommissionEmployeeTest
5  {
6      public static void main(String[] args)
7      {
8          // instantiate CommissionEmployee object
9          CommissionEmployee employee = new CommissionEmployee(
10             "Sue", "Jones", "222-22-2222", 10000, .06);
11
12         // get commission employee data
13         System.out.println(
14             "Employee information obtained by get methods:");
15         System.out.printf("%n%s %s%n", "First name is",
16             employee.getFirstName());
17         System.out.printf("%s %s%n", "Last name is",
18             employee.getLastName());
19         System.out.printf("%s %s%n", "Social security number is",
20             employee.getSocialSecurityNumber());
21         System.out.printf("%s %.2f%n", "Gross sales is",
22             employee.getGrossSales());
23         System.out.printf("%s %.2f%n", "Commission rate is",
24             employee.getCommissionRate());
25
26         employee.setGrossSales(5000);
27         employee.setCommissionRate(.1);
28
29         System.out.printf("%n%s:%n%n%s%n",
30             "Updated employee information obtained by toString", employee);
31     } // end main
32 } // end class CommissionEmployeeTest

```

Employee information obtained by get methods:

```

First name is Sue
Last name is Jones
Social security number is 222-22-2222
Gross sales is 10000.00
Commission rate is 0.06

```

Updated employee information obtained by toString:

```

commission employee: Sue Jones
social security number: 222-22-2222
gross sales: 5000.00
commission rate: 0.10

```

Fig. 9.5 | CommissionEmployee class test program.

9.4.2 Creating and Using a BasePlusCommissionEmployee Class

We now discuss the second part of our introduction to inheritance by declaring and testing (a completely new and independent) class `BasePlusCommissionEmployee` (Fig. 9.6), which contains a first name, last name, social security number, gross sales amount, commission rate *and* base salary. Class `BasePlusCommissionEmployee`'s public services include a `BasePlusCommissionEmployee` constructor (lines 15–42) and methods `earnings` (lines 111–114) and `toString` (lines 117–126). Lines 45–108 declare public *get* and *set* methods for the class's private instance variables (declared in lines 7–12) `firstName`, `lastName`, `socialSecurityNumber`, `grossSales`, `commissionRate` and `baseSalary`. These variables and methods encapsulate all the necessary features of a base-salaried commission employee. Note the *similarity* between this class and class `CommissionEmployee` (Fig. 9.4)—in this example, we'll not yet exploit that similarity.

```

1  // Fig. 9.6: BasePlusCommissionEmployee.java
2  // BasePlusCommissionEmployee class represents an employee who receives
3  // a base salary in addition to commission.
4
5  public class BasePlusCommissionEmployee
6  {
7      private final String firstName;
8      private final String lastName;
9      private final String socialSecurityNumber;
10     private double grossSales; // gross weekly sales
11     private double commissionRate; // commission percentage
12     private double baseSalary; // base salary per week
13
14     // six-argument constructor
15     public BasePlusCommissionEmployee(String firstName, String lastName,
16         String socialSecurityNumber, double grossSales,
17         double commissionRate, double baseSalary)
18     {
19         // implicit call to Object's default constructor occurs here
20
21         // if grossSales is invalid throw exception
22         if (grossSales < 0.0)
23             throw new IllegalArgumentException(
24                 "Gross sales must be >= 0.0");
25
26         // if commissionRate is invalid throw exception
27         if (commissionRate <= 0.0 || commissionRate >= 1.0)
28             throw new IllegalArgumentException(
29                 "Commission rate must be > 0.0 and < 1.0");
30
31         // if baseSalary is invalid throw exception
32         if (baseSalary < 0.0)
33             throw new IllegalArgumentException(
34                 "Base salary must be >= 0.0");
35

```

Fig. 9.6 | `BasePlusCommissionEmployee` class represents an employee who receives a base salary in addition to a commission. (Part I of 3.)

```
36     this.firstName = firstName;
37     this.lastName = lastName;
38     this.socialSecurityNumber = socialSecurityNumber;
39     this.grossSales = grossSales;
40     this.commissionRate = commissionRate;
41     this.baseSalary = baseSalary;
42 } // end constructor
43
44 // return first name
45 public String getFirstName()
46 {
47     return firstName;
48 }
49
50 // return last name
51 public String getLastName()
52 {
53     return lastName;
54 }
55
56 // return social security number
57 public String getSocialSecurityNumber()
58 {
59     return socialSecurityNumber;
60 }
61
62 // set gross sales amount
63 public void setGrossSales(double grossSales)
64 {
65     if (grossSales < 0.0)
66         throw new IllegalArgumentException(
67             "Gross sales must be >= 0.0");
68
69     this.grossSales = grossSales;
70 }
71
72 // return gross sales amount
73 public double getGrossSales()
74 {
75     return grossSales;
76 }
77
78 // set commission rate
79 public void setCommissionRate(double commissionRate)
80 {
81     if (commissionRate <= 0.0 || commissionRate >= 1.0)
82         throw new IllegalArgumentException(
83             "Commission rate must be > 0.0 and < 1.0");
84
85     this.commissionRate = commissionRate;
86 }
87
```

Fig. 9.6 | BasePlusCommissionEmployee class represents an employee who receives a base salary in addition to a commission. (Part 2 of 3.)

```

88     // return commission rate
89     public double getCommissionRate()
90     {
91         return commissionRate;
92     }
93
94     // set base salary
95     public void setBaseSalary(double baseSalary)
96     {
97         if (baseSalary < 0.0)
98             throw new IllegalArgumentException(
99                 "Base salary must be >= 0.0");
100
101         this.baseSalary = baseSalary;
102     }
103
104     // return base salary
105     public double getBaseSalary()
106     {
107         return baseSalary;
108     }
109
110     // calculate earnings
111     public double earnings()
112     {
113         return baseSalary + (commissionRate * grossSales);
114     }
115
116     // return String representation of BasePlusCommissionEmployee
117     @Override
118     public String toString()
119     {
120         return String.format(
121             "%s: %s %s%n%s: %s%n%s: %.2f%n%s: %.2f%n%s: %.2f",
122             "base-salaried commission employee", firstName, lastName,
123             "social security number", socialSecurityNumber,
124             "gross sales", grossSales, "commission rate", commissionRate,
125             "base salary", baseSalary);
126     }
127 } // end class BasePlusCommissionEmployee

```

Fig. 9.6 | BasePlusCommissionEmployee class represents an employee who receives a base salary in addition to a commission. (Part 3 of 3.)

Class BasePlusCommissionEmployee does *not* specify “extends Object” in line 5, so the class *implicitly* extends Object. Also, like class CommissionEmployee’s constructor (lines 13–34 of Fig. 9.4), class BasePlusCommissionEmployee’s constructor invokes class Object’s default constructor *implicitly*, as noted in the comment in line 19.

Class BasePlusCommissionEmployee’s earnings method (lines 111–114) returns the result of adding the BasePlusCommissionEmployee’s base salary to the product of the commission rate and the employee’s gross sales.

Class BasePlusCommissionEmployee overrides Object method toString to return a String containing the BasePlusCommissionEmployee’s information. Once again, we use

format specifier `%.2f` to format the gross sales, commission rate and base salary with two digits of precision to the right of the decimal point (line 121).

Testing Class *BasePlusCommissionEmployee*

Figure 9.7 tests class *BasePlusCommissionEmployee*. Lines 9–11 create a *BasePlusCommissionEmployee* object and pass "Bob", "Lewis", "333-33-3333", 5000, .04 and 300 to the constructor as the first name, last name, social security number, gross sales, commission rate and base salary, respectively. Lines 16–27 use *BasePlusCommissionEmployee*'s *get* methods to retrieve the values of the object's instance variables for output. Line 29 invokes the object's *setBaseSalary* method to change the base salary. Method *setBaseSalary* (Fig. 9.6, lines 95–102) ensures that instance variable *baseSalary* is not assigned a negative value. Line 33 of Fig. 9.7 invokes method *toString* *explicitly* to get the object's String representation.

```

1  // Fig. 9.7: BasePlusCommissionEmployeeTest.java
2  // BasePlusCommissionEmployee test program.
3
4  public class BasePlusCommissionEmployeeTest
5  {
6      public static void main(String[] args)
7      {
8          // instantiate BasePlusCommissionEmployee object
9          BasePlusCommissionEmployee employee =
10             new BasePlusCommissionEmployee(
11                 "Bob", "Lewis", "333-33-3333", 5000, .04, 300);
12
13         // get base-salaried commission employee data
14         System.out.println(
15             "Employee information obtained by get methods:\n");
16         System.out.printf("%s %s\n", "First name is",
17             employee.getFirstName());
18         System.out.printf("%s %s\n", "Last name is",
19             employee.getLastName());
20         System.out.printf("%s %s\n", "Social security number is",
21             employee.getSocialSecurityNumber());
22         System.out.printf("%s %.2f\n", "Gross sales is",
23             employee.getGrossSales());
24         System.out.printf("%s %.2f\n", "Commission rate is",
25             employee.getCommissionRate());
26         System.out.printf("%s %.2f\n", "Base salary is",
27             employee.getBaseSalary());
28
29         employee.setBaseSalary(1000);
30
31         System.out.printf("\n%s:\n\n%s\n",
32             "Updated employee information obtained by toString",
33             employee.toString());
34     } // end main
35 } // end class BasePlusCommissionEmployeeTest

```

Fig. 9.7 | *BasePlusCommissionEmployee* test program. (Part I of 2.)

```
Employee information obtained by get methods:

First name is Bob
Last name is Lewis
Social security number is 333-33-3333
Gross sales is 5000.00
Commission rate is 0.04
Base salary is 300.00

Updated employee information obtained by toString:

base-salaried commission employee: Bob Lewis
social security number: 333-33-3333
gross sales: 5000.00
commission rate: 0.04
base salary: 1000.00
```

Fig. 9.7 | BasePlusCommissionEmployee test program. (Part 2 of 2.)

Notes on Class BasePlusCommissionEmployee

Much of class `BasePlusCommissionEmployee`'s code (Fig. 9.6) is *similar*, or *identical*, to that of class `CommissionEmployee` (Fig. 9.4). For example, private instance variables `firstName` and `lastName` and methods `setFirstName`, `getFirstName`, `setLastName` and `getLastName` are identical to those of class `CommissionEmployee`. The classes also both contain private instance variables `socialSecurityNumber`, `commissionRate` and `grossSales`, and corresponding *get* and *set* methods. In addition, the `BasePlusCommissionEmployee` constructor is *almost* identical to that of class `CommissionEmployee`, except that `BasePlusCommissionEmployee`'s constructor also sets the `baseSalary`. The other additions to class `BasePlusCommissionEmployee` are private instance variable `baseSalary` and methods `setBaseSalary` and `getBaseSalary`. Class `BasePlusCommissionEmployee`'s `toString` method is *almost* identical to that of class `CommissionEmployee` except that it also outputs instance variable `baseSalary` with two digits of precision to the right of the decimal point.

We literally *copied* code from class `CommissionEmployee` and *pasted* it into class `BasePlusCommissionEmployee`, then modified class `BasePlusCommissionEmployee` to include a base salary and methods that manipulate the base salary. This “*copy-and-paste*” approach is often error prone and time consuming. Worse yet, it spreads copies of the same code throughout a system, creating code-maintenance problems—changes to the code would need to be made in multiple classes. Is there a way to “acquire” the instance variables and methods of one class in a way that makes them part of other classes *without duplicating code*? Next we answer this question, using a more elegant approach to building classes that emphasizes the benefits of inheritance.



Software Engineering Observation 9.3

With inheritance, the instance variables and methods that are the same for all the classes in the hierarchy are declared in a superclass. Changes made to these common features in the superclass are inherited by the subclass. Without inheritance, changes would need to be made to all the source-code files that contain a copy of the code in question.

9.4.3 Creating a CommissionEmployee–BasePlusCommissionEmployee Inheritance Hierarchy

Now we declare class `BasePlusCommissionEmployee` (Fig. 9.8) to *extend* class `CommissionEmployee` (Fig. 9.4). A `BasePlusCommissionEmployee` object *is a* `CommissionEmployee`, because inheritance passes on class `CommissionEmployee`'s capabilities. Class `BasePlusCommissionEmployee` also has instance variable `baseSalary` (Fig. 9.8, line 6). Keyword `extends` (line 4) indicates inheritance. `BasePlusCommissionEmployee` *inherits* `CommissionEmployee`'s instance variables and methods.



Software Engineering Observation 9.4

At the design stage in an object-oriented system, you'll often find that certain classes are closely related. You should "factor out" common instance variables and methods and place them in a superclass. Then use inheritance to develop subclasses, specializing them with capabilities beyond those inherited from the superclass.



Software Engineering Observation 9.5

Declaring a subclass does not affect its superclass's source code. Inheritance preserves the integrity of the superclass.

Only `CommissionEmployee`'s public and protected members are directly accessible in the subclass. The `CommissionEmployee` constructor is *not* inherited. So, the public `BasePlusCommissionEmployee` services include its constructor (lines 9–23), public methods inherited from `CommissionEmployee`, and methods `setBaseSalary` (lines 26–33), `getBaseSalary` (lines 36–39), `earnings` (lines 42–47) and `toString` (lines 50–60). Methods `earnings` and `toString` *override* the corresponding methods in class `CommissionEmployee` because their superclass versions do not properly calculate a `BasePlusCommissionEmployee`'s earnings or return an appropriate `String` representation, respectively.

```

1 // Fig. 9.8: BasePlusCommissionEmployee.java
2 // private superclass members cannot be accessed in a subclass.
3
4 public class BasePlusCommissionEmployee extends CommissionEmployee
5 {
6     private double baseSalary; // base salary per week
7
8     // six-argument constructor
9     public BasePlusCommissionEmployee(String firstName, String lastName,
10         String socialSecurityNumber, double grossSales,
11         double commissionRate, double baseSalary)
12     {
13         // explicit call to superclass CommissionEmployee constructor
14         super(firstName, lastName, socialSecurityNumber,
15             grossSales, commissionRate);
16
17         // if baseSalary is invalid throw exception
18         if (baseSalary < 0.0)
19             throw new IllegalArgumentException(
20                 "Base salary must be >= 0.0");
21     }

```

Fig. 9.8 | private superclass members cannot be accessed in a subclass. (Part I of 3.)

```

22     this.baseSalary = baseSalary;
23 }
24
25 // set base salary
26 public void setBaseSalary(double baseSalary)
27 {
28     if (baseSalary < 0.0)
29         throw new IllegalArgumentException(
30             "Base salary must be >= 0.0");
31
32     this.baseSalary = baseSalary;
33 }
34
35 // return base salary
36 public double getBaseSalary()
37 {
38     return baseSalary;
39 }
40
41 // calculate earnings
42 @Override
43 public double earnings()
44 {
45     // not allowed: commissionRate and grossSales private in superclass
46     return baseSalary + (commissionRate * grossSales);
47 }
48
49 // return String representation of BasePlusCommissionEmployee
50 @Override
51 public String toString()
52 {
53     // not allowed: attempts to access private superclass members
54     return String.format(
55         "%s: %s %s%n%s: %s%n%s: %.2f%n%s: %.2f%n%s: %.2f",
56         "base-salaried commission employee", firstName, lastName,
57         "social security number", socialSecurityNumber,
58         "gross sales", grossSales, "commission rate", commissionRate,
59         "base salary", baseSalary);
60 }
61 } // end class BasePlusCommissionEmployee

```

BasePlusCommissionEmployee.java:46: error: commissionRate has private access in CommissionEmployee

```

    return baseSalary + (commissionRate * grossSales);
                        ^

```

BasePlusCommissionEmployee.java:46: error: grossSales has private access in CommissionEmployee

```

    return baseSalary + (commissionRate * grossSales);
                                   ^

```

BasePlusCommissionEmployee.java:56: error: firstName has private access in CommissionEmployee

```

    "base-salaried commission employee", firstName, lastName,
                                         ^

```

Fig. 9.8 | private superclass members cannot be accessed in a subclass. (Part 2 of 3.)

```

BasePlusCommissionEmployee.java:56: error: lastName has private access in
CommissionEmployee
    "base-salaried commission employee", firstName, lastName,
                                                    ^
BasePlusCommissionEmployee.java:57: error: socialSecurityNumber has private
access in CommissionEmployee
    "social security number", socialSecurityNumber,
                                ^
BasePlusCommissionEmployee.java:58: error: grossSales has private access in
CommissionEmployee
    "gross sales", grossSales, "commission rate", commissionRate,
                    ^
BasePlusCommissionEmployee.java:58: error: commissionRate has private access
inCommissionEmployee
    "gross sales", grossSales, "commission rate", commissionRate,
                                                    ^

```

Fig. 9.8 | private superclass members cannot be accessed in a subclass. (Part 3 of 3.)

A Subclass's Constructor Must Call Its Superclass's Constructor

Each subclass constructor must implicitly or explicitly call one of its superclass's constructors to initialize the instance variables inherited from the superclass. Lines 14–15 in `BasePlusCommissionEmployee`'s six-argument constructor (lines 9–23) explicitly call class `CommissionEmployee`'s five-argument constructor (declared at lines 13–34 of Fig. 9.4) to initialize the superclass portion of a `BasePlusCommissionEmployee` object (i.e., variables `firstName`, `lastName`, `socialSecurityNumber`, `grossSales` and `commissionRate`). We do this by using the **superclass constructor call syntax**—keyword `super`, followed by a set of parentheses containing the superclass constructor arguments, which are used to initialize the superclass instance variables `firstName`, `lastName`, `socialSecurityNumber`, `grossSales` and `commissionRate`, respectively. If `BasePlusCommissionEmployee`'s constructor did not invoke the superclass's constructor explicitly, the compiler would attempt to insert a call to the superclass's default or no-argument constructor. Class `CommissionEmployee` does not have such a constructor, so the compiler would issue an error. The explicit superclass constructor call in lines 14–15 of Fig. 9.8 must be the *first* statement in the constructor's body. When a superclass contains a no-argument constructor, you can use `super()` to call that constructor explicitly, but this is rarely done.



Software Engineering Observation 9.6

You learned previously that you should not call a class's instance methods from its constructors and that we'll say why in Chapter 10. Calling a superclass constructor from a subclass constructor does not contradict this advice.

BasePlusCommissionEmployee Methods Earnings and toString

The compiler generates errors for line 46 (Fig. 9.8) because `CommissionEmployee`'s instance variables `commissionRate` and `grossSales` are private—subclass `BasePlusCommissionEmployee`'s methods are *not* allowed to access superclass `CommissionEmployee`'s private instance variables. We used **red text** in Fig. 9.8 to indicate erroneous code. The compiler issues additional errors at lines 56–58 of `BasePlusCommissionEmployee`'s `toString` method for the same reason. The errors in `BasePlusCommissionEmployee` could have been prevented by using the *get* methods inherited from class `CommissionEmployee`.

For example, line 46 could have called `getCommissionRate` and `getGrossSales` to access `CommissionEmployee`'s private instance variables `commissionRate` and `grossSales`, respectively. Lines 56–58 also could have used appropriate *get* methods to retrieve the values of the superclass's instance variables.

9.4.4 `CommissionEmployee`–`BasePlusCommissionEmployee` Inheritance Hierarchy Using protected Instance Variables

To enable class `BasePlusCommissionEmployee` to directly access superclass instance variables `firstName`, `lastName`, `socialSecurityNumber`, `grossSales` and `commissionRate`, we can declare those members as `protected` in the superclass. As we discussed in Section 9.3, a superclass's protected members are accessible by all subclasses of that superclass. In the new `CommissionEmployee` class, we modified only lines 6–10 of Fig. 9.4 to declare the instance variables with the `protected` access modifier as follows:

```
protected final String firstName;
protected final String lastName;
protected final String socialSecurityNumber;
protected double grossSales; // gross weekly sales
protected double commissionRate; // commission percentage
```

The rest of the class declaration (which is not shown here) is identical to that of Fig. 9.4.

We could have declared `CommissionEmployee`'s instance variables `public` to enable subclass `BasePlusCommissionEmployee` to access them. However, declaring `public` instance variables is poor software engineering because it allows unrestricted access to these variables from any class, greatly increasing the chance of errors. With `protected` instance variables, the subclass gets access to the instance variables, but classes that are not subclasses and classes that are not in the same package cannot access these variables directly—recall that `protected` class members are also visible to other classes in the same package.

Class `BasePlusCommissionEmployee`

Class `BasePlusCommissionEmployee` (Fig. 9.9) extends the new version of class `CommissionEmployee` with `protected` instance variables. `BasePlusCommissionEmployee` objects inherit `CommissionEmployee`'s protected instance variables `firstName`, `lastName`, `socialSecurityNumber`, `grossSales` and `commissionRate`—all these variables are now protected members of `BasePlusCommissionEmployee`. As a result, the compiler does not generate errors when compiling line 45 of method `earnings` and lines 54–56 of method `toString`. If another class extends this version of class `BasePlusCommissionEmployee`, the new subclass also can access the protected members.

```
1 // Fig. 9.9: BasePlusCommissionEmployee.java
2 // BasePlusCommissionEmployee inherits protected instance
3 // variables from CommissionEmployee.
4
5 public class BasePlusCommissionEmployee extends CommissionEmployee
6 {
7     private double baseSalary; // base salary per week
```

Fig. 9.9 | `BasePlusCommissionEmployee` inherits protected instance variables from `CommissionEmployee`. (Part I of 2.)

```

8
9 // six-argument constructor
10 public BasePlusCommissionEmployee(String firstName, String lastName,
11     String socialSecurityNumber, double grossSales,
12     double commissionRate, double baseSalary)
13 {
14     super(firstName, lastName, socialSecurityNumber,
15         grossSales, commissionRate);
16
17     // if baseSalary is invalid throw exception
18     if (baseSalary < 0.0)
19         throw new IllegalArgumentException(
20             "Base salary must be >= 0.0");
21
22     this.baseSalary = baseSalary;
23 }
24
25 // set base salary
26 public void setBaseSalary(double baseSalary)
27 {
28     if (baseSalary < 0.0)
29         throw new IllegalArgumentException(
30             "Base salary must be >= 0.0");
31
32     this.baseSalary = baseSalary;
33 }
34
35 // return base salary
36 public double getBaseSalary()
37 {
38     return baseSalary;
39 }
40
41 // calculate earnings
42 @Override // indicates that this method overrides a superclass method
43 public double earnings()
44 {
45     return baseSalary + (commissionRate * grossSales);
46 }
47
48 // return String representation of BasePlusCommissionEmployee
49 @Override
50 public String toString()
51 {
52     return String.format(
53         "%s: %s %s%n%s: %s%n%s: %.2f%n%s: %.2f%n%s: %.2f",
54         "base-salaried commission employee", firstName, lastName,
55         "social security number", socialSecurityNumber,
56         "gross sales", grossSales, "commission rate", commissionRate,
57         "base salary", baseSalary);
58 }
59 } // end class BasePlusCommissionEmployee

```

Fig. 9.9 | BasePlusCommissionEmployee inherits protected instance variables from CommissionEmployee. (Part 2 of 2.)

A Subclass Object Contains the Instance Variables of All of Its Superclasses

When you create a `BasePlusCommissionEmployee` object, it contains all instance variables declared in the class hierarchy to that point—that is, those from classes `Object` (which does not have instance variables), `CommissionEmployee` and `BasePlusCommissionEmployee`. Class `BasePlusCommissionEmployee` does *not* inherit `CommissionEmployee`'s five-argument constructor, but *explicitly invokes* it (lines 14–15) to initialize the instance variables that `BasePlusCommissionEmployee` inherited from `CommissionEmployee`. Similarly, `CommissionEmployee`'s constructor *implicitly* calls class `Object`'s constructor. `BasePlusCommissionEmployee`'s constructor must *explicitly* call `CommissionEmployee`'s constructor because `CommissionEmployee` does *not* have a no-argument constructor that could be invoked implicitly.

Testing Class `BasePlusCommissionEmployee`

The `BasePlusCommissionEmployeeTest` class for this example is identical to that of Fig. 9.7 and produces the same output, so we do not show it here. Although the version of class `BasePlusCommissionEmployee` in Fig. 9.6 does not use inheritance and the version in Fig. 9.9 does, both classes provide the *same* functionality. The source code in Fig. 9.9 (59 lines) is considerably shorter than that in Fig. 9.6 (127 lines), because most of the class's functionality is now inherited from `CommissionEmployee`—there's now only one copy of the `CommissionEmployee` functionality. This makes the code easier to maintain, modify and debug, because the code related to a `CommissionEmployee` exists only in that class.

*Notes on Using **protected** Instance Variables*

In this example, we declared superclass instance variables as **protected** so that subclasses could access them. Inheriting **protected** instance variables enables direct access to the variables by subclasses. In most cases, however, it's better to use **private** instance variables to encourage proper software engineering. Your code will be easier to maintain, modify and debug.

Using **protected** instance variables creates several potential problems. First, the subclass object can set an inherited variable's value directly without using a *set* method. Therefore, a subclass object can assign an invalid value to the variable, possibly leaving the object in an inconsistent state. For example, if we were to declare `CommissionEmployee`'s instance variable `grossSales` as **protected**, a subclass object (e.g., `BasePlusCommissionEmployee`) could then assign a negative value to `grossSales`. Another problem with using **protected** instance variables is that subclass methods are more likely to be written so that they depend on the superclass's data implementation. In practice, subclasses should depend only on the superclass services (i.e., non-private methods) and not on the superclass data implementation. With **protected** instance variables in the superclass, we may need to modify all the subclasses of the superclass if the superclass implementation changes. For example, if for some reason we were to change the names of instance variables `firstName` and `lastName` to `first` and `last`, then we would have to do so for all occurrences in which a subclass directly references superclass instance variables `firstName` and `lastName`. Such a class is said to be **fragile** or **brittle**, because a small change in the superclass can “break” subclass implementation. You should be able to change the superclass implementation while still providing the same services to the subclasses. Of course, if the superclass services change, we must reimplement our subclasses. A third problem is that a class's **protected** members

are visible to all classes in the same package as the class containing the protected members—this is not always desirable.



Software Engineering Observation 9.7

Use the protected access modifier when a superclass should provide a method only to its subclasses and other classes in the same package, but not to other clients.



Software Engineering Observation 9.8

Declaring superclass instance variables private (as opposed to protected) enables the superclass implementation of these instance variables to change without affecting subclass implementations.



Error-Prevention Tip 9.2

When possible, do not include protected instance variables in a superclass. Instead, include non-private methods that access private instance variables. This will help ensure that objects of the class maintain consistent states.

9.4.5 CommissionEmployee–BasePlusCommissionEmployee Inheritance Hierarchy Using private Instance Variables

Let's reexamine our hierarchy once more, this time using good software engineering practices.

Class *CommissionEmployee*

Class *CommissionEmployee* (Fig. 9.10) declares instance variables *firstName*, *lastName*, *socialSecurityNumber*, *grossSales* and *commissionRate* as *private* (lines 6–10) and provides public methods *getFirstName*, *getLastName*, *getSocialSecurityNumber*, *setGrossSales*, *getGrossSales*, *setCommissionRate*, *getCommissionRate*, *earnings* and *toString* for manipulating these values. Methods *earnings* (lines 87–90) and *toString* (lines 93–101) use the class's *get* methods to obtain the values of its instance variables. If we decide to change the names of the instance variables, the *earnings* and *toString* declarations will *not* require modification—only the bodies of the *get* and *set* methods that directly manipulate the instance variables will need to change. These changes occur solely within the superclass—no changes to the subclass are needed. *Localizing the effects of changes* like this is a good software engineering practice.

```

1 // Fig. 9.10: CommissionEmployee.java
2 // CommissionEmployee class uses methods to manipulate its
3 // private instance variables.
4 public class CommissionEmployee
5 {
6     private final String firstName;
7     private final String lastName;
8     private final String socialSecurityNumber;
9     private double grossSales; // gross weekly sales
10    private double commissionRate; // commission percentage

```

Fig. 9.10 | *CommissionEmployee* class uses methods to manipulate its private instance variables. (Part I of 3.)

```
11
12 // five-argument constructor
13 public CommissionEmployee(String firstName, String lastName,
14     String socialSecurityNumber, double grossSales,
15     double commissionRate)
16 {
17     // implicit call to Object constructor occurs here
18
19     // if grossSales is invalid throw exception
20     if (grossSales < 0.0)
21         throw new IllegalArgumentException(
22             "Gross sales must be >= 0.0");
23
24     // if commissionRate is invalid throw exception
25     if (commissionRate <= 0.0 || commissionRate >= 1.0)
26         throw new IllegalArgumentException(
27             "Commission rate must be > 0.0 and < 1.0");
28
29     this.firstName = firstName;
30     this.lastName = lastName;
31     this.socialSecurityNumber = socialSecurityNumber;
32     this.grossSales = grossSales;
33     this.commissionRate = commissionRate;
34 } // end constructor
35
36 // return first name
37 public String getFirstName()
38 {
39     return firstName;
40 }
41
42 // return last name
43 public String getLastName()
44 {
45     return lastName;
46 }
47
48 // return social security number
49 public String getSocialSecurityNumber()
50 {
51     return socialSecurityNumber;
52 }
53
54 // set gross sales amount
55 public void setGrossSales(double grossSales)
56 {
57     if (grossSales < 0.0)
58         throw new IllegalArgumentException(
59             "Gross sales must be >= 0.0");
60
61     this.grossSales = grossSales;
62 }
```

Fig. 9.10 | CommissionEmployee class uses methods to manipulate its private instance variables. (Part 2 of 3.)

```

63
64 // return gross sales amount
65 public double getGrossSales()
66 {
67     return grossSales;
68 }
69
70 // set commission rate
71 public void setCommissionRate(double commissionRate)
72 {
73     if (commissionRate <= 0.0 || commissionRate >= 1.0)
74         throw new IllegalArgumentException(
75             "Commission rate must be > 0.0 and < 1.0");
76
77     this.commissionRate = commissionRate;
78 }
79
80 // return commission rate
81 public double getCommissionRate()
82 {
83     return commissionRate;
84 }
85
86 // calculate earnings
87 public double earnings()
88 {
89     return getCommissionRate() * getGrossSales();
90 }
91
92 // return String representation of CommissionEmployee object
93 @Override
94 public String toString()
95 {
96     return String.format("%s: %s %s\n%s: %s\n%s: %.2f\n%s: %.2f",
97         "commission employee", getFirstName(), getLastName(),
98         "social security number", getSocialSecurityNumber(),
99         "gross sales", getGrossSales(),
100         "commission rate", getCommissionRate());
101 }
102 } // end class CommissionEmployee

```

Fig. 9.10 | CommissionEmployee class uses methods to manipulate its private instance variables. (Part 3 of 3.)

Class BasePlusCommissionEmployee

Subclass *BasePlusCommissionEmployee* (Fig. 9.11) inherits *CommissionEmployee*'s non-private methods and can access (in a controlled way) the private superclass members via those methods. Class *BasePlusCommissionEmployee* has several changes that distinguish it from Fig. 9.9. Methods *earnings* (lines 43–47) and *toString* (lines 50–55) each invoke method *getBaseSalary* to obtain the base salary value, rather than accessing *baseSalary* directly. If we decide to rename instance variable *baseSalary*, only the bodies of method *setBaseSalary* and *getBaseSalary* will need to change.

```
1 // Fig. 9.11: BasePlusCommissionEmployee.java
2 // BasePlusCommissionEmployee class inherits from CommissionEmployee
3 // and accesses the superclass's private data via inherited
4 // public methods.
5
6 public class BasePlusCommissionEmployee extends CommissionEmployee
7 {
8     private double baseSalary; // base salary per week
9
10    // six-argument constructor
11    public BasePlusCommissionEmployee(String firstName, String lastName,
12        String socialSecurityNumber, double grossSales,
13        double commissionRate, double baseSalary)
14    {
15        super(firstName, lastName, socialSecurityNumber,
16            grossSales, commissionRate);
17
18        // if baseSalary is invalid throw exception
19        if (baseSalary < 0.0)
20            throw new IllegalArgumentException(
21                "Base salary must be >= 0.0");
22
23        this.baseSalary = baseSalary;
24    }
25
26    // set base salary
27    public void setBaseSalary(double baseSalary)
28    {
29        if (baseSalary < 0.0)
30            throw new IllegalArgumentException(
31                "Base salary must be >= 0.0");
32
33        this.baseSalary = baseSalary;
34    }
35
36    // return base salary
37    public double getBaseSalary()
38    {
39        return baseSalary;
40    }
41
42    // calculate earnings
43    @Override
44    public double earnings()
45    {
46        return getBaseSalary() + super.earnings();
47    }
48
49    // return String representation of BasePlusCommissionEmployee
50    @Override
51    public String toString()
52    {
```

Fig. 9.11 | BasePlusCommissionEmployee class inherits from CommissionEmployee and accesses the superclass's private data via inherited public methods. (Part 1 of 2.)

```

53         return String.format("%s %s\n%s: %.2f", "base-salaried",
54                               super.toString(), "base salary", getBaseSalary());
55     }
56 } // end class BasePlusCommissionEmployee

```

Fig. 9.11 | BasePlusCommissionEmployee class inherits from CommissionEmployee and accesses the superclass's private data via inherited public methods. (Part 2 of 2.)

Class BasePlusCommissionEmployee's earnings Method

Method `earnings` (lines 43–47) overrides class `CommissionEmployee`'s `earnings` method (Fig. 9.10, lines 87–90) to calculate a base-salaried commission employee's earnings. The new version obtains the portion of the earnings based on commission alone by calling `CommissionEmployee`'s `earnings` method with `super.earnings()` (line 46), then adds the base salary to this value to calculate the total earnings. Note the syntax used to invoke an *overridden* superclass method from a subclass—place the keyword `super` and a dot (`.`) separator before the superclass method name. This method invocation is a good software engineering practice—if a method performs all or some of the actions needed by another method, call that method rather than duplicate its code. By having `BasePlusCommissionEmployee`'s `earnings` method invoke `CommissionEmployee`'s `earnings` method to calculate part of a `BasePlusCommissionEmployee` object's earnings, we *avoid duplicating the code and reduce code-maintenance problems*.



Common Programming Error 9.2

When a superclass method is overridden in a subclass, the subclass version often calls the superclass version to do a portion of the work. Failure to prefix the superclass method name with the keyword `super` and the dot (`.`) separator when calling the superclass's method causes the subclass method to call itself, potentially creating an error called infinite recursion, which would eventually cause the method-call stack to overflow—a fatal runtime error. Recursion, used correctly, is a powerful capability discussed in Chapter 18.

Class BasePlusCommissionEmployee's toString Method

Similarly, `BasePlusCommissionEmployee`'s `toString` method (Fig. 9.11, lines 50–55) overrides `CommissionEmployee`'s `toString` method (Fig. 9.10, lines 93–101) to return a `String` representation that's appropriate for a base-salaried commission employee. The new version creates part of a `BasePlusCommissionEmployee` object's `String` representation (i.e., the `String` "commission employee" and the values of class `CommissionEmployee`'s private instance variables) by calling `CommissionEmployee`'s `toString` method with the expression `super.toString()` (Fig. 9.11, line 54). `BasePlusCommissionEmployee`'s `toString` method then completes the remainder of a `BasePlusCommissionEmployee` object's `String` representation (i.e., the value of class `BasePlusCommissionEmployee`'s base salary).

Testing Class BasePlusCommissionEmployee

Class `BasePlusCommissionEmployeeTest` performs the same manipulations on a `BasePlusCommissionEmployee` object as in Fig. 9.7 and produces the same output, so we do not show it here. Although each `BasePlusCommissionEmployee` class you've seen behaves identically, the version in Fig. 9.11 is the best engineered. By using inheritance and by calling methods that hide the data and ensure consistency, we've efficiently and effectively constructed a well-engineered class.

9.5 Constructors in Subclasses

As we explained, instantiating a subclass object begins a chain of constructor calls in which the subclass constructor, before performing its own tasks, explicitly uses `super` to call one of the constructors in its direct superclass or implicitly calls the superclass's default or no-argument constructor. Similarly, if the superclass is derived from another class—true of every class except `Object`—the superclass constructor invokes the constructor of the next class up the hierarchy, and so on. The last constructor called in the chain is *always* `Object`'s constructor. The original subclass constructor's body finishes executing *last*. Each superclass's constructor manipulates the superclass instance variables that the subclass object inherits. For example, consider again the `CommissionEmployee`–`BasePlusCommissionEmployee` hierarchy from Figs. 9.10–9.11. When an app creates a `BasePlusCommissionEmployee` object, its constructor is called. That constructor calls `CommissionEmployee`'s constructor, which in turn calls `Object`'s constructor. Class `Object`'s constructor has an *empty body*, so it immediately returns control to `CommissionEmployee`'s constructor, which then initializes the `CommissionEmployee` instance variables that are part of the `BasePlusCommissionEmployee` object. When `CommissionEmployee`'s constructor completes execution, it returns control to `BasePlusCommissionEmployee`'s constructor, which initializes the `baseSalary`.



Software Engineering Observation 9.9

Java ensures that even if a constructor does not assign a value to an instance variable, the variable is still initialized to its default value (e.g., 0 for primitive numeric types, false for booleans, null for references).

9.6 Class Object

As we discussed earlier in this chapter, all classes in Java inherit directly or indirectly from class `Object` (package `java.lang`), so its 11 methods (some are overloaded) are inherited by all other classes. Figure 9.12 summarizes `Object`'s methods. We discuss several `Object` methods throughout this book (as indicated in Fig. 9.12).

Method	Description
<code>equals</code>	This method compares two objects for equality and returns <code>true</code> if they're equal and <code>false</code> otherwise. The method takes any <code>Object</code> as an argument. When objects of a particular class must be compared for equality, the class should override method <code>equals</code> to compare the <i>contents</i> of the two objects. For the requirements of implementing this method (which include also overriding method <code>hashCode</code>), refer to the method's documentation at docs.oracle.com/javase/7/docs/api/java/lang/Object.html#equals(java.lang.Object) . The default <code>equals</code> implementation uses operator <code>==</code> to determine whether two references <i>refer to the same object</i> in memory. Section 14.3.3 demonstrates class <code>String</code> 's <code>equals</code> method and differentiates between comparing <code>String</code> objects with <code>==</code> and with <code>equals</code> .
<code>hashCode</code>	Hashcodes are <code>int</code> values used for high-speed storage and retrieval of information stored in a data structure that's known as a hashtable (see Section 16.11). This method is also called as part of <code>Object</code> 's default <code>toString</code> method implementation.

Fig. 9.12 | `Object` methods. (Part 1 of 2.)

Method	Description
<code>toString</code>	This method (introduced in Section 9.4.1) returns a <code>String</code> representation of an object. The default implementation of this method returns the package name and class name of the object's class typically followed by a hexadecimal representation of the value returned by the object's <code>hashCode</code> method.
<code>wait</code> , <code>notify</code> , <code>notifyAll</code>	Methods <code>notify</code> , <code>notifyAll</code> and the three overloaded versions of <code>wait</code> are related to multithreading, which is discussed in Chapter 23.
<code>getClass</code>	Every object in Java knows its own type at execution time. Method <code>getClass</code> (used in Sections 10.5 and 12.5) returns an object of class <code>Class</code> (package <code>java.lang</code>) that contains information about the object's type, such as its class name (returned by <code>Class</code> method <code>getName</code>).
<code>finalize</code>	This protected method is called by the garbage collector to perform termination housekeeping on an object just before the garbage collector reclaims the object's memory. Recall from Section 8.10 that it's unclear whether, or when, <code>finalize</code> will be called. For this reason, most programmers should avoid method <code>finalize</code> .
<code>clone</code>	This protected method, which takes no arguments and returns an <code>Object</code> reference, makes a copy of the object on which it's called. The default implementation performs a so-called shallow copy —instance-variable values in one object are copied into another object of the same type. For reference types, only the references are copied. A typical overridden <code>clone</code> method's implementation would perform a deep copy that creates a new object for each reference-type instance variable. <i>Implementing <code>clone</code> correctly is difficult. For this reason, its use is discouraged.</i> Some industry experts suggest that object serialization should be used instead. We discuss object serialization in Chapter 15. Recall from Chapter 7 that arrays are objects. As a result, like all other objects, arrays inherit the members of class <code>Object</code> . Every array has an overridden <code>clone</code> method that copies the array. However, if the array stores references to objects, the objects are not copied—a shallow copy is performed.

Fig. 9.12 | Object methods. (Part 2 of 2.)

9.7 (Optional) GUI and Graphics Case Study: Displaying Text and Images Using Labels

Programs often use labels when they need to display information or instructions to the user in a graphical user interface. **Labels** are a convenient way of identifying GUI components on the screen and keeping the user informed about the current state of the program. In Java, an object of class `JLabel` (from package `javax.swing`) can display text, an image or both. The example in Fig. 9.13 demonstrates several `JLabel` features, including a plain text label, an image label and a label with both text and an image.

```

1 // Fig 9.13: LabelDemo.java
2 // Demonstrates the use of labels.
3 import java.awt.BorderLayout;
4 import javax.swing.ImageIcon;

```

Fig. 9.13 | `JLabel` with text and with images. (Part 1 of 2.)


```

5  import javax.swing.JLabel;
6  import javax.swing.JFrame;
7
8  public class LabelDemo
9  {
10     public static void main(String[] args)
11     {
12         // Create a label with plain text
13         JLabel northLabel = new JLabel("North");
14
15         // create an icon from an image so we can put it on a JLabel
16         ImageIcon labelIcon = new ImageIcon("GUIitip.gif");
17
18         // create a label with an Icon instead of text
19         JLabel centerLabel = new JLabel(labelIcon);
20
21         // create another label with an Icon
22         JLabel southLabel = new JLabel(labelIcon);
23
24         // set the label to display text (as well as an icon)
25         southLabel.setText("South");
26
27         // create a frame to hold the labels
28         JFrame application = new JFrame();
29
30         application.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
31
32         // add the labels to the frame; the second argument specifies
33         // where on the frame to add the label
34         application.add(northLabel, BorderLayout.NORTH);
35         application.add(centerLabel, BorderLayout.CENTER);
36         application.add(southLabel, BorderLayout.SOUTH);
37
38         application.setSize(300, 300);
39         application.setVisible(true);
40     } // end main
41 } // end class LabelDemo

```



Fig. 9.13 | JLabel with text and with images. (Part 2 of 2.)

Lines 3–6 import the classes we need to display JLabels. BorderLayout from package java.awt contains constants that specify where we can place GUI components in the

JFrame. Class **ImageIcon** represents an image that can be displayed on a JLabel, and class JFrame represents the window that will contain all the labels.

Line 13 creates a JLabel that displays its constructor argument—the string "North". Line 16 declares local variable `labelIcon` and assigns it a new **ImageIcon**. The constructor for **ImageIcon** receives a **String** that specifies the path to the image. Since we specify only a filename, Java assumes that it's in the same directory as class `LabelDemo`. **ImageIcon** can load images in GIF, JPEG and PNG image formats. Line 19 declares and initializes local variable `centerLabel` with a JLabel that displays the `labelIcon`. Line 22 declares and initializes local variable `southLabel` with a JLabel similar to the one in line 19. However, line 25 calls method **setText** to change the text the label displays. Method `setText` can be called on any JLabel to change its text. This JLabel displays both the icon and the text.

Line 28 creates the JFrame that displays the JLabels, and line 30 indicates that the program should terminate when the JFrame is closed. We attach the labels to the JFrame in lines 34–36 by calling an overloaded version of method `add` that takes two parameters. The first parameter is the component we want to attach, and the second is the region in which it should be placed. Each JFrame has an associated **layout** that helps the JFrame position the GUI components that are attached to it. The default layout for a JFrame is known as a **BorderLayout** and has five regions—NORTH (top), SOUTH (bottom), EAST (right side), WEST (left side) and CENTER. Each of these is declared as a constant in class **BorderLayout**. When calling method `add` with one argument, the JFrame places the component in the CENTER automatically. If a position already contains a component, then the new component takes its place. Lines 38 and 39 set the size of the JFrame and make it visible on screen.

GUI and Graphics Case Study Exercise

9.1 Modify GUI and Graphics Case Study Exercise 8.1 to include a JLabel as a status bar that displays counts representing the number of each shape displayed. Class `DrawPanel` should declare a method that returns a **String** containing the status text. In `main`, first create the `DrawPanel`, then create the JLabel with the status text as an argument to the JLabel's constructor. Attach the JLabel to the SOUTH region of the JFrame, as shown in Fig. 9.14.

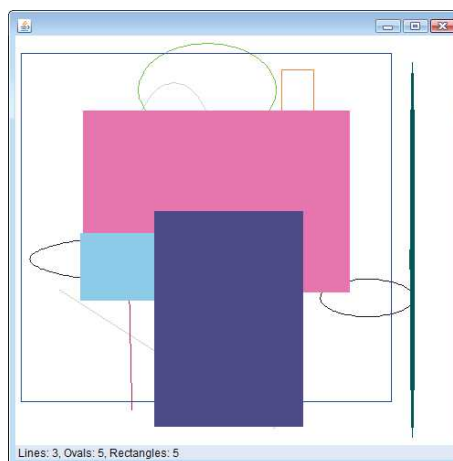


Fig. 9.14 | JLabel displaying shape statistics.

9.8 Wrap-Up

This chapter introduced inheritance—the ability to create classes by acquiring an existing class’s members (without copying and pasting the code) and having the ability to embellish them with new capabilities. You learned the notions of superclasses and subclasses and used keyword `extends` to create a subclass that inherits members from a superclass. We showed how to use the `@Override` annotation to prevent unintended overloading by indicating that a method overrides a superclass method. We introduced the access modifier `protected`; subclass methods can directly access `protected` superclass members. You learned how to use `super` to access overridden superclass members. You also saw how constructors are used in inheritance hierarchies. Finally, you learned about the methods of class `Object`, the direct or indirect superclass of all Java classes.

In Chapter 10, Object-Oriented Programming: Polymorphism and Interfaces, we build on our discussion of inheritance by introducing *polymorphism*—an object-oriented concept that enables us to write programs that conveniently handle, in a more general and convenient manner, objects of a wide variety of classes related by inheritance. After studying Chapter 10, you’ll be familiar with classes, objects, encapsulation, inheritance and polymorphism—the key technologies of object-oriented programming.

Summary

Section 9.1 Introduction

- Inheritance (p. 361) reduces program-development time.
- The direct superclass (p. 361) of a subclass is the one from which the subclass inherits. An indirect superclass (p. 361) of a subclass is two or more levels up the class hierarchy from that subclass.
- In single inheritance (p. 361), a class is derived from one superclass. In multiple inheritance, a class is derived from more than one direct superclass. Java does not support multiple inheritance.
- A subclass is more specific than its superclass and represents a smaller group of objects (p. 361).
- Every object of a subclass is also an object of that class’s superclass. However, a superclass object is not an object of its class’s subclasses.
- An *is-a* relationship (p. 362) represents inheritance. In an *is-a* relationship, an object of a subclass also can be treated as an object of its superclass.
- A *has-a* relationship (p. 362) represents composition. In a *has-a* relationship, a class object contains references to objects of other classes.

Section 9.2 Superclasses and Subclasses

- Single-inheritance relationships form treelike hierarchical structures—a superclass exists in a hierarchical relationship with its subclasses.

Section 9.3 `protected` Members

- A superclass’s `public` members are accessible wherever the program has a reference to an object of that superclass or one of its subclasses.
- A superclass’s `private` members can be accessed directly only within the superclass’s declaration.
- A superclass’s `protected` members (p. 364) have an intermediate level of protection between `public` and `private` access. They can be accessed by members of the superclass, by members of its subclasses and by members of other classes in the same package.

- A superclass's private members are hidden in its subclasses and can be accessed only through the `public` or `protected` methods inherited from the superclass.
- An overridden superclass method can be accessed from a subclass if the superclass method name is preceded by `super` (p. 364) and a dot (`.`) separator.

Section 9.4 Relationship Between Superclasses and Subclasses

- A subclass cannot access the private members of its superclass, but it can access the non-private members.
- A subclass can invoke a constructor of its superclass by using the keyword `super`, followed by a set of parentheses containing the superclass constructor arguments. This must appear as the first statement in the subclass constructor's body.
- A superclass method can be overridden in a subclass to declare an appropriate implementation for the subclass.
- The `@Override` annotation (p. 369) indicates that a method should override a superclass method. When the compiler encounters a method declared with `@Override`, it compares the method's signature with the superclass's method signatures. If there isn't an exact match, the compiler issues an error message, such as "method does not override or implement a method from a supertype."
- Method `toString` takes no arguments and returns a `String`. The `Object` class's `toString` method is normally overridden by a subclass.
- When an object is output using the `%s` format specifier, the object's `toString` method is called implicitly to obtain its `String` representation.

Section 9.5 Constructors in Subclasses

- The first task of a subclass constructor is to call its direct superclass's constructor (p. 378) to ensure that the instance variables inherited from the superclass are initialized.

Section 9.6 Class `Object`

- See the table of class `Object`'s methods in Fig. 9.12.

Self-Review Exercises

- 9.1** Fill in the blanks in each of the following statements:
- _____ is a form of software reusability in which new classes acquire the members of existing classes and embellish those classes with new capabilities.
 - A superclass's _____ members can be accessed in the superclass declaration *and in* subclass declarations.
 - In a(n) _____ relationship, an object of a subclass can also be treated as an object of its superclass.
 - In a(n) _____ relationship, a class object has references to objects of other classes as members.
 - In single inheritance, a class exists in a(n) _____ relationship with its subclasses.
 - A superclass's _____ members are accessible anywhere that the program has a reference to an object of that superclass or to an object of one of its subclasses.
 - When an object of a subclass is instantiated, a superclass _____ is called implicitly or explicitly.
 - Subclass constructors can call superclass constructors via the _____ keyword.
- 9.2** State whether each of the following is *true* or *false*. If a statement is *false*, explain why.
- Superclass constructors are not inherited by subclasses.
 - A *has-a* relationship is implemented via inheritance.

- c) A Car class has an *is-a* relationship with the SteeringWheel and Brakes classes.
- d) When a subclass redefines a superclass method by using the same signature, the subclass is said to overload that superclass method.

Answers to Self-Review Exercises

9.1 a) Inheritance. b) public and protected. c) *is-a* or inheritance. d) *has-a* or composition. e) hierarchical. f) public. g) constructor. h) super.

9.2 a) True. b) False. A *has-a* relationship is implemented via composition. An *is-a* relationship is implemented via inheritance. c) False. This is an example of a *has-a* relationship. Class Car has an *is-a* relationship with class Vehicle. d) False. This is known as overriding, not overloading—an overloaded method has the same name, but a different signature.

Exercises

9.3 (*Using Composition Rather Than Inheritance*) Many programs written with inheritance could be written with composition instead, and vice versa. Rewrite class BasePlusCommissionEmployee (Fig. 9.11) of the CommissionEmployee–BasePlusCommissionEmployee hierarchy to use composition rather than inheritance.

9.4 (*Software Reuse*) Discuss the ways in which inheritance promotes software reuse, saves time during program development and helps prevent errors.

9.5 (*Student Inheritance Hierarchy*) Draw an inheritance hierarchy for students at a university similar to the hierarchy shown in Fig. 9.2. Use Student as the superclass of the hierarchy, then extend Student with classes UndergraduateStudent and GraduateStudent. Continue to extend the hierarchy as deep (i.e., as many levels) as possible. For example, Freshman, Sophomore, Junior and Senior might extend UndergraduateStudent, and DoctoralStudent and MastersStudent might be subclasses of GraduateStudent. After drawing the hierarchy, discuss the relationships that exist between the classes. [Note: You do not need to write any code for this exercise.]

9.6 (*Shape Inheritance Hierarchy*) The world of shapes is much richer than the shapes included in the inheritance hierarchy of Fig. 9.3. Write down all the shapes you can think of—both two-dimensional and three-dimensional—and form them into a more complete Shape hierarchy with as many levels as possible. Your hierarchy should have class Shape at the top. Classes TwoDimensionalShape and ThreeDimensionalShape should extend Shape. Add additional subclasses, such as Quadrilateral and Sphere, at their correct locations in the hierarchy as necessary.

9.7 (*protected vs. private*) Some programmers prefer not to use protected access, because they believe it breaks the encapsulation of the superclass. Discuss the relative merits of using protected access vs. using private access in superclasses.

9.8 (*Quadrilateral Inheritance Hierarchy*) Write an inheritance hierarchy for classes Quadrilateral, Trapezoid, Parallelogram, Rectangle and Square. Use Quadrilateral as the superclass of the hierarchy. Create and use a Point class to represent the points in each shape. Make the hierarchy as deep (i.e., as many levels) as possible. Specify the instance variables and methods for each class. The private instance variables of Quadrilateral should be the *x-y* coordinate pairs for the four endpoints of the Quadrilateral. Write a program that instantiates objects of your classes and outputs each object's area (except Quadrilateral).

9.9 (*What Does Each Code Snippet Do?*)

- a) Assume that the following method call is located in an overridden earnings method in a subclass:

```
super.earnings()
```

- b) Assume that the following line of code appears before a method declaration:

```
@Override
```

- c) Assume that the following line of code appears as the first statement in a constructor's body:

```
super(firstArgument, secondArgument);
```

9.10 (*Write a Line of Code*) Write a line of code that performs each of the following tasks:

- Specify that class `PieceWorker` inherits from class `Employee`.
- Call superclass `Employee`'s `toString` method from subclass `PieceWorker`'s `toString` method.
- Call superclass `Employee`'s constructor from subclass `PieceWorker`'s constructor—assume that the superclass constructor receives three `Strings` representing the first name, last name and social security number.

9.11 (*Using `super` in a Constructor's Body*) Explain why you would use `super` in the first statement of a subclass constructor's body.

9.12 (*Using `super` in an Instance Method's Body*) Explain why you would use `super` in the body of a subclass's instance method.

9.13 (*Calling `get` Methods in a Class's Body*) In Figs. 9.10–9.11 methods `earnings` and `toString` each call various *get* methods within the same class. Explain the benefits of calling these *get* methods within the classes.

9.14 (*Employee Hierarchy*) In this chapter, you studied an inheritance hierarchy in which class `BasePlusCommissionEmployee` inherited from class `CommissionEmployee`. However, not all types of employees are `CommissionEmployee`s. In this exercise, you'll create a more general `Employee` superclass that *factors out* the attributes and behaviors in class `CommissionEmployee` that are common to all `Employees`. The common attributes and behaviors for all `Employees` are `firstName`, `lastName`, `socialSecurityNumber`, `getFirstName`, `getLastName`, `getSocialSecurityNumber` and a portion of method `toString`. Create a new superclass `Employee` that contains these instance variables and methods and a constructor. Next, rewrite class `CommissionEmployee` from Section 9.4.5 as a subclass of `Employee`. Class `CommissionEmployee` should contain only the instance variables and methods that are not declared in superclass `Employee`. Class `CommissionEmployee`'s constructor should invoke class `Employee`'s constructor and `CommissionEmployee`'s `toString` method should invoke `Employee`'s `toString` method. Once you've completed these modifications, run the `CommissionEmployeeTest` and `BasePlusCommissionEmployeeTest` apps using these new classes to ensure that the apps still display the same results for a `CommissionEmployee` object and `BasePlusCommissionEmployee` object, respectively.

9.15 (*Creating a New Subclass of `Employee`*) Other types of `Employees` might include `SalariedEmployees` who get paid a fixed weekly salary, `PieceWorkers` who get paid by the number of pieces they produce or `HourlyEmployees` who get paid an hourly wage with time-and-a-half—1.5 times the hourly wage—for hours worked over 40 hours.

Create class `HourlyEmployee` that inherits from class `Employee` (Exercise 9.14) and has instance variable `hours` (a `double`) that represents the hours worked, instance variable `wage` (a `double`) that represents the wages per hour, a constructor that takes as arguments a first name, a last name, a social security number, an hourly wage and the number of hours worked, *set* and *get* methods for manipulating the hours and wage, an `earnings` method to calculate an `HourlyEmployee`'s earnings based on the hours worked and a `toString` method that returns the `HourlyEmployee`'s `String` representation. Method `setWage` should ensure that wage is nonnegative, and `setHours` should ensure that the value of hours is between 0 and 168 (the total number of hours in a week). Use class `HourlyEmployee` in a test program that's similar to the one in Fig. 9.5.

Object-Oriented Programming: Polymorphism and Interfaces

10



*General propositions do not
decide concrete cases.*

—Oliver Wendell Holmes

*A philosopher of imposing
stature doesn't think in a
vacuum. Even his most abstract
ideas are, to some extent,
conditioned by what is or is not
known in the time when he
lives.*

—Alfred North Whitehead

Objectives

In this chapter you'll:

- Learn the concept of polymorphism.
- Use overridden methods to effect polymorphism.
- Distinguish between abstract and concrete classes.
- Declare abstract methods to create abstract classes.
- Learn how polymorphism makes systems extensible and maintainable.
- Determine an object's type at execution time.
- Declare and implement interfaces, and become familiar with the Java SE 8 interface enhancements.

- 10.1** Introduction
- 10.2** Polymorphism Examples
- 10.3** Demonstrating Polymorphic Behavior
- 10.4** Abstract Classes and Methods
- 10.5** Case Study: Payroll System Using Polymorphism
 - 10.5.1 Abstract Superclass `Employee`
 - 10.5.2 Concrete Subclass `SalariedEmployee`
 - 10.5.3 Concrete Subclass `HourlyEmployee`
 - 10.5.4 Concrete Subclass `CommissionEmployee`
 - 10.5.5 Indirect Concrete Subclass `BasePlusCommissionEmployee`
 - 10.5.6 Polymorphic Processing, Operator `instanceof` and Downcasting
- 10.6** Allowed Assignments Between Superclass and Subclass Variables
- 10.7** `final` Methods and Classes
- 10.8** A Deeper Explanation of Issues with Calling Methods from Constructors
- 10.9** Creating and Using Interfaces
 - 10.9.1 Developing a `Payable` Hierarchy
 - 10.9.2 Interface `Payable`
 - 10.9.3 Class `Invoice`
 - 10.9.4 Modifying Class `Employee` to Implement Interface `Payable`
 - 10.9.5 Modifying Class `SalariedEmployee` for Use in the `Payable` Hierarchy
 - 10.9.6 Using Interface `Payable` to Process Invoices and Employees Polymorphically
 - 10.9.7 Some Common Interfaces of the Java API
- 10.10** Java SE 8 Interface Enhancements
 - 10.10.1 `default` Interface Methods
 - 10.10.2 `static` Interface Methods
 - 10.10.3 Functional Interfaces
- 10.11** (Optional) GUI and Graphics Case Study: Drawing with Polymorphism
- 10.12** Wrap-Up

Summary | Self-Review Exercises | Answers to Self-Review Exercises |
Exercises | Making a Difference

10.1 Introduction

We continue our study of object-oriented programming by explaining and demonstrating **polymorphism** with inheritance hierarchies. Polymorphism enables you to “program in the *general*” rather than “program in the *specific*.” In particular, polymorphism enables you to write programs that process objects that share the same superclass, either directly or indirectly, as if they were all objects of the superclass; this can simplify programming.

Consider the following example of polymorphism. Suppose we create a program that simulates the movement of several types of animals for a biological study. Classes `Fish`, `Frog` and `Bird` represent the types of animals under investigation. Imagine that each class extends superclass `Animal`, which contains a method `move` and maintains an animal’s current location as *x-y* coordinates. Each subclass implements method `move`. Our program maintains an `Animal` array containing references to objects of the various `Animal` subclasses. To simulate the animals’ movements, the program sends each object the *same* message once per second—namely, `move`. Each specific type of `Animal` responds to a `move` message in its own way—a `Fish` might swim three feet, a `Frog` might jump five feet and a `Bird` might fly ten feet. Each object knows how to modify its *x-y* coordinates appropriately for its *specific* type of movement. Relying on each object to know how to “do the right thing” (i.e., do what’s appropriate for that type of object) in response to the *same* method call is the key concept of polymorphism. The *same* message (in this case, `move`) sent to a *variety* of objects has *many forms* of results—hence the term polymorphism.

Implementing for Extensibility

With polymorphism, we can design and implement systems that are easily *extensible*—new classes can be added with little or no modification to the general portions of the program, as long as the new classes are part of the inheritance hierarchy that the program processes generically. The new classes simply “plug right in.” The only parts of a program that must be altered are those that require direct knowledge of the new classes that we add to the hierarchy. For example, if we extend class `Animal` to create class `Tortoise` (which might respond to a move message by crawling one inch), we need to write only the `Tortoise` class and the part of the simulation that instantiates a `Tortoise` object. The portions of the simulation that tell each `Animal` to move generically can remain the same.

Chapter Overview

First, we discuss common examples of polymorphism. We then provide a simple example demonstrating polymorphic behavior. We use superclass references to manipulate *both* superclass objects and subclass objects polymorphically.

We then present a case study that revisits the employee hierarchy of Section 9.4.5. We develop a simple payroll application that polymorphically calculates the weekly pay of several different types of employees using each employee’s `earnings` method. Though the earnings of each type of employee are calculated in a *specific* way, polymorphism allows us to process the employees “in the *general*.” In the case study, we enlarge the hierarchy to include two new classes—`SalariedEmployee` (for people paid a fixed weekly salary) and `HourlyEmployee` (for people paid an hourly salary and “time-and-a-half” for overtime). We declare a common set of functionality for all the classes in the updated hierarchy in an “abstract” class, `Employee`, from which “concrete” classes `SalariedEmployee`, `HourlyEmployee` and `CommissionEmployee` inherit directly and “concrete” class `BasePlusCommissionEmployee` inherits indirectly. As you’ll soon see, *when we invoke each employee’s earnings method off a superclass Employee reference (regardless of the employee’s type), the correct earnings subclass calculation is performed*, due to Java’s polymorphic capabilities.

Programming in the Specific

Occasionally, when performing polymorphic processing, we need to program “in the *specific*.” Our `Employee` case study demonstrates that a program can determine the *type* of an object at *execution time* and act on that object accordingly. In the case study, we’ve decided that `BasePlusCommissionEmployees` should receive 10% raises on their base salaries. So, we use these capabilities to determine whether a particular employee object *is a* `BasePlusCommissionEmployee`. If so, we increase that employee’s base salary by 10%.

Interfaces

The chapter continues with an introduction to Java *interfaces*, which are particularly useful for assigning *common* functionality to possibly *unrelated* classes. This allows objects of these classes to be processed polymorphically—objects of classes that **implement** the *same* interface can respond to all of the interface method calls. To demonstrate creating and using interfaces, we modify our payroll application to create a generalized accounts payable application that can calculate payments due for company employees *and* invoice amounts to be billed for purchased goods.

10.2 Polymorphism Examples

Let's consider several additional examples of polymorphism.

Quadrilaterals

If class `Rectangle` is derived from class `Quadrilateral`, then a `Rectangle` object *is a* more *specific* version of a `Quadrilateral`. Any operation (e.g., calculating the perimeter or the area) that can be performed on a `Quadrilateral` can also be performed on a `Rectangle`. These operations can also be performed on other `Quadrilaterals`, such as `Squares`, `Parallelograms` and `Trapezoids`. The polymorphism occurs when a program invokes a method through a superclass `Quadrilateral` variable—at execution time, the correct subclass version of the method is called, based on the type of the reference stored in the superclass variable. You'll see a simple code example that illustrates this process in Section 10.3.

Space Objects in a Video Game

Suppose we design a video game that manipulates objects of classes `Martian`, `Venusian`, `Plutonian`, `SpaceShip` and `LaserBeam`. Imagine that each class inherits from the superclass `SpaceObject`, which contains method `draw`. Each subclass implements this method. A screen manager maintains a collection (e.g., a `SpaceObject` array) of references to objects of the various classes. To refresh the screen, the screen manager periodically sends each object the *same* message—namely, `draw`. However, each object responds its *own* way, based on its class. For example, a `Martian` object might draw itself in red with green eyes and the appropriate number of antennae. A `SpaceShip` object might draw itself as a bright silver flying saucer. A `LaserBeam` object might draw itself as a bright red beam across the screen. Again, the *same* message (in this case, `draw`) sent to a *variety* of objects has “many forms” of results.

A screen manager might use polymorphism to facilitate adding new classes to a system with minimal modifications to the system's code. Suppose that we want to add `Mercurian` objects to our video game. To do so, we'd build a class `Mercurian` that extends `SpaceObject` and provides its own `draw` method implementation. When `Mercurian` objects appear in the `SpaceObject` collection, the screen-manager code *invokes method `draw`, exactly as it does for every other object in the collection, regardless of its type*. So the new `Mercurian` objects simply “plug right in” without any modification of the screen manager code by the programmer. Thus, without modifying the system (other than to build new classes and modify the code that creates new objects), you can use polymorphism to conveniently include additional types that were not even considered when the system was created.



Software Engineering Observation 10.1

Polymorphism enables you to deal in generalities and let the execution-time environment handle the specifics. You can tell objects to behave in manners appropriate to those objects, without knowing their specific types, as long as they belong to the same inheritance hierarchy.



Software Engineering Observation 10.2

Polymorphism promotes extensibility: Software that invokes polymorphic behavior is independent of the object types to which messages are sent. New object types that can respond to existing method calls can be incorporated into a system without modifying the base system. Only client code that instantiates new objects must be modified to accommodate new types.

10.3 Demonstrating Polymorphic Behavior

Section 9.4 created a class hierarchy, in which class `BasePlusCommissionEmployee` inherited from `CommissionEmployee`. The examples in that section manipulated `CommissionEmployee` and `BasePlusCommissionEmployee` objects by using references to them to invoke their methods—we aimed superclass variables at superclass objects and subclass variables at subclass objects. These assignments are natural and straightforward—superclass variables are *intended* to refer to superclass objects, and subclass variables are *intended* to refer to subclass objects. However, as you’ll soon see, other assignments are possible.

In the next example, we aim a *superclass* reference at a *subclass* object. We then show how invoking a method on a subclass object via a superclass reference invokes the *subclass* functionality—the type of the *referenced object*, *not* the type of the *variable*, determines which method is called. This example demonstrates that *an object of a subclass can be treated as an object of its superclass*, enabling various interesting manipulations. A program can create an array of superclass variables that refer to objects of many subclass types. This is allowed because each subclass object *is an* object of its superclass. For instance, we can assign the reference of a `BasePlusCommissionEmployee` object to a superclass `CommissionEmployee` variable, because a `BasePlusCommissionEmployee` *is a* `CommissionEmployee`—so we can treat a `BasePlusCommissionEmployee` as a `CommissionEmployee`.

As you’ll learn later in the chapter, you *cannot treat a superclass object as a subclass object*, because a superclass object is *not* an object of any of its subclasses. For example, we cannot assign the reference of a `CommissionEmployee` object to a subclass `BasePlusCommissionEmployee` variable, because a `CommissionEmployee` is *not* a `BasePlusCommissionEmployee`—a `CommissionEmployee` does *not* have a `baseSalary` instance variable and does *not* have methods `setBaseSalary` and `getBaseSalary`. The *is-a* relationship applies only *up the hierarchy* from a subclass to its direct (and indirect) superclasses, and *not* vice versa (i.e., not down the hierarchy from a superclass to its subclasses or indirect subclasses).

The Java compiler *does* allow the assignment of a superclass reference to a subclass variable if we explicitly *cast* the superclass reference to the subclass type. Why would we ever want to perform such an assignment? A superclass reference can be used to invoke *only* the methods declared in the superclass—attempting to invoke *subclass-only* methods through a superclass reference results in compilation errors. If a program needs to perform a subclass-specific operation on a subclass object referenced by a superclass variable, the program must first cast the superclass reference to a subclass reference through a technique known as **downcasting**. This enables the program to invoke subclass methods that are *not* in the superclass. We demonstrate the mechanics of downcasting in Section 10.5.



Software Engineering Observation 10.3

Although it’s allowed, you should generally avoid downcasting.

The example in Fig. 10.1 demonstrates three ways to use superclass and subclass variables to store references to superclass and subclass objects. The first two are straightforward—as in Section 9.4, we assign a superclass reference to a superclass variable, and a subclass reference to a subclass variable. Then we demonstrate the relationship between subclasses and superclasses (i.e., the *is-a* relationship) by assigning a subclass reference to a superclass variable. This program uses classes `CommissionEmployee` and `BasePlusCommissionEmployee` from Fig. 9.10 and Fig. 9.11, respectively.

```

1 // Fig. 10.1: PolymorphismTest.java
2 // Assigning superclass and subclass references to superclass and
3 // subclass variables.
4
5 public class PolymorphismTest
6 {
7     public static void main(String[] args)
8     {
9         // assign superclass reference to superclass variable
10        CommissionEmployee commissionEmployee = new CommissionEmployee(
11            "Sue", "Jones", "222-22-2222", 10000, .06);
12
13        // assign subclass reference to subclass variable
14        BasePlusCommissionEmployee basePlusCommissionEmployee =
15            new BasePlusCommissionEmployee(
16                "Bob", "Lewis", "333-33-3333", 5000, .04, 300);
17
18        // invoke toString on superclass object using superclass variable
19        System.out.printf("%s %s:%n%n%s%n",
20            "Call CommissionEmployee's toString with superclass reference ",
21            "to superclass object", commissionEmployee.toString());
22
23        // invoke toString on subclass object using subclass variable
24        System.out.printf("%s %s:%n%n%s%n",
25            "Call BasePlusCommissionEmployee's toString with subclass",
26            "reference to subclass object",
27            basePlusCommissionEmployee.toString());
28
29        // invoke toString on subclass object using superclass variable
30        CommissionEmployee commissionEmployee2 =
31            basePlusCommissionEmployee;
32        System.out.printf("%s %s:%n%n%s%n",
33            "Call BasePlusCommissionEmployee's toString with superclass",
34            "reference to subclass object", commissionEmployee2.toString());
35    } // end main
36 } // end class PolymorphismTest

```

Call CommissionEmployee's toString with superclass reference to superclass object:

```

commission employee: Sue Jones
social security number: 222-22-2222
gross sales: 10000.00
commission rate: 0.06

```

Call BasePlusCommissionEmployee's toString with subclass reference to subclass object:

```

base-salaried commission employee: Bob Lewis
social security number: 333-33-3333
gross sales: 5000.00
commission rate: 0.04
base salary: 300.00

```

Fig. 10.1 | Assigning superclass and subclass references to superclass and subclass variables.
(Part I of 2.)

Call `BasePlusCommissionEmployee`'s `toString` with superclass reference to subclass object:

```
base-salaried commission employee: Bob Lewis
social security number: 333-33-3333
gross sales: 5000.00
commission rate: 0.04
base salary: 300.00
```

Fig. 10.1 | Assigning superclass and subclass references to superclass and subclass variables.
(Part 2 of 2.)

In Fig. 10.1, lines 10–11 create a `CommissionEmployee` object and assign its reference to a `CommissionEmployee` variable. Lines 14–16 create a `BasePlusCommissionEmployee` object and assign its reference to a `BasePlusCommissionEmployee` variable. These assignments are natural—for example, a `CommissionEmployee` variable’s primary purpose is to hold a reference to a `CommissionEmployee` object. Lines 19–21 use `commissionEmployee` to invoke `toString` *explicitly*. Because `commissionEmployee` refers to a `CommissionEmployee` object, superclass `CommissionEmployee`’s version of `toString` is called. Similarly, lines 24–27 use `basePlusCommissionEmployee` to invoke `toString` *explicitly* on the `BasePlusCommissionEmployee` object. This invokes subclass `BasePlusCommissionEmployee`’s version of `toString`.

Lines 30–31 then assign the reference of subclass object `basePlusCommissionEmployee` to a superclass `CommissionEmployee` variable, which lines 32–34 use to invoke method `toString`. *When a superclass variable contains a reference to a subclass object, and that reference is used to call a method, the subclass version of the method is called.* Hence, `commissionEmployee2.toString()` in line 34 actually calls class `BasePlusCommissionEmployee`’s `toString` method. The Java compiler allows this “crossover” because an object of a subclass *is* an object of its superclass (but *not* vice versa). When the compiler encounters a method call made through a variable, the compiler determines if the method can be called by checking the variable’s class type. If that class contains the proper method declaration (or inherits one), the call is compiled. At execution time, the type of the object to which the variable refers determines the actual method to use. This process, called *dynamic binding*, is discussed in detail in Section 10.5.

10.4 Abstract Classes and Methods

When we think of a class, we assume that programs will create objects of that type. Sometimes it’s useful to declare classes—called **abstract classes**—for which you *never* intend to create objects. Because they’re used only as superclasses in inheritance hierarchies, we refer to them as **abstract superclasses**. These classes cannot be used to instantiate objects, because, as we’ll soon see, abstract classes are *incomplete*. Subclasses must declare the “missing pieces” to become “concrete” classes, from which you can instantiate objects. Otherwise, these subclasses, too, will be abstract. We demonstrate abstract classes in Section 10.5.

Purpose of Abstract Classes

An abstract class’s purpose is to provide an appropriate superclass from which other classes can inherit and thus share a common design. In the Shape hierarchy of Fig. 9.3, for exam-

ple, subclasses inherit the notion of what it means to be a *Shape*—perhaps common attributes such as `location`, `color` and `borderThickness`, and behaviors such as `draw`, `move`, `resize` and `changeColor`. Classes that can be used to instantiate objects are called **concrete classes**. Such classes provide implementations of *every* method they declare (some of the implementations can be inherited). For example, we could derive concrete classes `Circle`, `Square` and `Triangle` from abstract superclass `TwoDimensionalShape`. Similarly, we could derive concrete classes `Sphere`, `Cube` and `Tetrahedron` from abstract superclass `ThreeDimensionalShape`. Abstract superclasses are *too general* to create real objects—they specify only what is common among subclasses. We need to be more *specific* before we can create objects. For example, if you send the `draw` message to abstract class `TwoDimensionalShape`, the class knows that two-dimensional shapes should be *drawable*, but it does not know what *specific* shape to draw, so it cannot implement a real `draw` method. Concrete classes provide the specifics that make it reasonable to instantiate objects.

Not all hierarchies contain abstract classes. However, you'll often write client code that uses only abstract superclass types to reduce the client code's dependencies on a range of subclass types. For example, you can write a method with a parameter of an abstract superclass type. When called, such a method can receive an object of *any* concrete class that directly or indirectly extends the superclass specified as the parameter's type.

Abstract classes sometimes constitute several levels of a hierarchy. For example, the *Shape* hierarchy of Fig. 9.3 begins with abstract class *Shape*. On the next level of the hierarchy are *abstract* classes `TwoDimensionalShape` and `ThreeDimensionalShape`. The next level of the hierarchy declares *concrete* classes for `TwoDimensionalShapes` (`Circle`, `Square` and `Triangle`) and for `ThreeDimensionalShapes` (`Sphere`, `Cube` and `Tetrahedron`).

Declaring an Abstract Class and Abstract Methods

You make a class abstract by declaring it with keyword **abstract**. An abstract class normally contains one or more **abstract methods**. An abstract method is an *instance method* with keyword `abstract` in its declaration, as in

```
public abstract void draw(); // abstract method
```

Abstract methods do *not* provide implementations. A class that contains *any* abstract methods must be explicitly declared abstract even if that class contains some concrete (nonabstract) methods. Each concrete subclass of an abstract superclass also must provide concrete implementations of each of the superclass's abstract methods. Constructors and static methods cannot be declared abstract. Constructors are *not* inherited, so an abstract constructor could never be implemented. Though non-private static methods are inherited, they cannot be overridden. Since abstract methods are meant to be overridden so that they can process objects based on their types, it would not make sense to declare a static method as abstract.



Software Engineering Observation 10.4

An abstract class declares common attributes and behaviors (both abstract and concrete) of the various classes in a class hierarchy. An abstract class typically contains one or more abstract methods that subclasses must override if they are to be concrete. The instance variables and concrete methods of an abstract class are subject to the normal rules of inheritance.



Common Programming Error 10.1

Attempting to instantiate an object of an abstract class is a compilation error.



Common Programming Error 10.2

Failure to implement a superclass's abstract methods in a subclass is a compilation error unless the subclass is also declared abstract.

Using Abstract Classes to Declare Variables

Although we cannot instantiate objects of abstract superclasses, you'll soon see that we *can* use abstract superclasses to declare variables that can hold references to objects of *any* concrete class *derived from* those abstract superclasses. We'll use such variables to manipulate subclass objects *polymorphically*. You also can use abstract superclass names to invoke static methods declared in those abstract superclasses.

Consider another application of polymorphism. A drawing program needs to display many shapes, including types of new shapes that you'll *add* to the system *after* writing the drawing program. The drawing program might need to display shapes, such as `Circles`, `Triangles`, `Rectangles` or others, that derive from abstract class `Shape`. The drawing program uses `Shape` variables to manage the objects that are displayed. To draw any object in this inheritance hierarchy, the drawing program uses a superclass `Shape` variable containing a reference to the subclass object to invoke the object's `draw` method. This method is declared abstract in superclass `Shape`, so each concrete subclass *must* implement method `draw` in a manner specific to that shape—each object in the `Shape` inheritance hierarchy *knows how to draw itself*. The drawing program does not have to worry about the type of each object or whether the program has ever encountered objects of that type.

Layered Software Systems

Polymorphism is particularly effective for implementing so-called *layered software systems*. In operating systems, for example, each type of physical device could operate quite differently from the others. Even so, commands to read or write data from and to devices may have a certain uniformity. For each device, the operating system uses a piece of software called a *device driver* to control all communication between the system and the device. The write message sent to a device-driver object needs to be interpreted specifically in the context of that driver and how it manipulates devices of a specific type. However, the write call itself really is no different from the write to any other device in the system—place some number of bytes from memory onto that device. An object-oriented operating system might use an abstract superclass to provide an “interface” appropriate for all device drivers. Then, through inheritance from that abstract superclass, subclasses are formed that all behave similarly. The device-driver methods are declared as abstract methods in the abstract superclass. The implementations of these abstract methods are provided in the concrete subclasses that correspond to the specific types of device drivers. New devices are always being developed, often long after the operating system has been released. When you buy a new device, it comes with a device driver provided by the device vendor. The device is immediately operational after you connect it to your computer and install the driver. This is another elegant example of how polymorphism makes systems *extensible*.

10.5 Case Study: Payroll System Using Polymorphism

This section reexamines the `CommissionEmployee-BasePlusCommissionEmployee` hierarchy that we explored throughout Section 9.4. Now we use an abstract method and polymorphism to perform payroll calculations based on an enhanced employee inheritance hierarchy that meets the following requirements:

A company pays its employees on a weekly basis. The employees are of four types: Salaried employees are paid a fixed weekly salary regardless of the number of hours worked, hourly employees are paid by the hour and receive overtime pay (i.e., 1.5 times their hourly salary rate) for all hours worked in excess of 40 hours, commission employees are paid a percentage of their sales and base-salaried commission employees receive a base salary plus a percentage of their sales. For the current pay period, the company has decided to reward salaried-commission employees by adding 10% to their base salaries. The company wants you to write an application that performs its payroll calculations polymorphically.

We use abstract class `Employee` to represent the general concept of an employee. The classes that extend `Employee` are `SalariedEmployee`, `CommissionEmployee` and `HourlyEmployee`. Class `BasePlusCommissionEmployee`—which extends `CommissionEmployee`—represents the last employee type. The UML class diagram in Fig. 10.2 shows the inheritance hierarchy for our polymorphic employee-payroll application. Abstract class name `Employee` is italicized—a convention of the UML.

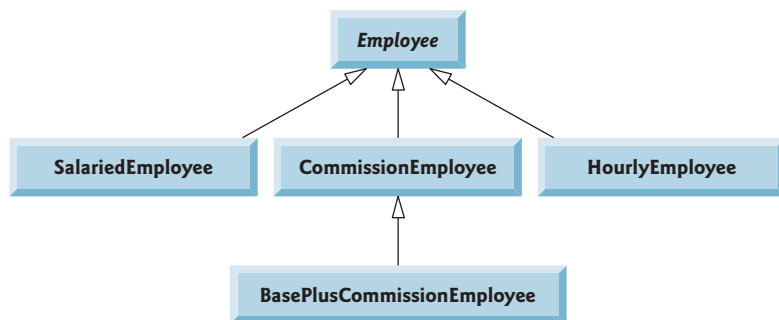


Fig. 10.2 | `Employee` hierarchy UML class diagram.

Abstract superclass `Employee` declares the “interface” to the hierarchy—that is, the set of methods that a program can invoke on all `Employee` objects. We use the term “interface” here in a *general* sense to refer to the various ways programs can communicate with objects of *any* `Employee` subclass. Be careful not to confuse the general notion of an “interface” with the formal notion of a Java interface, the subject of Section 10.9. Each employee, regardless of the way his or her earnings are calculated, has a first name, a last name and a social security number, so private instance variables `firstName`, `lastName` and `socialSecurityNumber` appear in abstract superclass `Employee`.

The diagram in Fig. 10.3 shows each of the five classes in the hierarchy down the left side and methods `earnings` and `toString` across the top. For each class, the diagram shows the desired results of each method. We do not list superclass `Employee`’s `get` methods because they’re *not* overridden in any of the subclasses—each of these methods is inherited and used “as is” by each subclass.

	earnings	toString
Employee	abstract	<i>firstName</i> <i>lastName</i> social security number: <i>SSN</i>
Salaried- Employee	weeklySalary	salaried employee: <i>firstName</i> <i>lastName</i> social security number: <i>SSN</i> weekly salary: <i>weeklySalary</i>
Hourly- Employee	<pre> if (hours <= 40) wage * hours else if (hours > 40) { 40 * wage + (hours - 40) * wage * 1.5 } </pre>	hourly employee: <i>firstName</i> <i>lastName</i> social security number: <i>SSN</i> hourly wage: <i>wage</i> ; hours worked: <i>hours</i>
Commission- Employee	commissionRate * grossSales	commission employee: <i>firstName</i> <i>lastName</i> social security number: <i>SSN</i> gross sales: <i>grossSales</i> ; commission rate: <i>commissionRate</i>
BasePlus- Commission- Employee	(commissionRate * grossSales) + baseSalary	base salaried commission employee: <i>firstName</i> <i>lastName</i> social security number: <i>SSN</i> gross sales: <i>grossSales</i> ; commission rate: <i>commissionRate</i> ; base salary: <i>baseSalary</i>

Fig. 10.3 | Polymorphic interface for the Employee hierarchy classes.

The following sections implement the Employee class hierarchy of Fig. 10.2. The first section implements *abstract superclass* Employee. The next four sections each implement one of the *concrete* classes. The last section implements a test program that builds objects of all these classes and processes those objects polymorphically.

10.5.1 Abstract Superclass Employee

Class Employee (Fig. 10.4) provides methods earnings and toString, in addition to the *get* methods that return the values of Employee's instance variables. An earnings method certainly applies *generically* to all employees. But each earnings calculation depends on the employee's particular class. So we declare earnings as abstract in superclass Employee because a *specific* default implementation does not make sense for that method—there isn't enough information to determine what amount earnings should return.

Each subclass overrides earnings with an appropriate implementation. To calculate an employee's earnings, the program assigns to a superclass Employee variable a reference to the employee's object, then invokes the earnings method on that variable. We maintain an array of Employee variables, each holding a reference to an Employee object. You *cannot* use class Employee directly to create Employee *objects*, because Employee is an *abstract* class. Due to inheritance, however, all objects of all Employee subclasses may be thought of as Employee

objects. The program will iterate through the array and call method `earnings` for each `Employee` object. Java processes these method calls *polymorphically*. Declaring `earnings` as an abstract method in `Employee` enables the calls to `earnings` through `Employee` variables to compile and forces every direct *concrete* subclass of `Employee` to *override* `earnings`.

Method `toString` in class `Employee` returns a `String` containing the first name, last name and social security number of the employee. As we'll see, each subclass of `Employee` *overrides* method `toString` to create a `String` representation of an object of that class that contains the employee's type (e.g., "salaried employee:") followed by the rest of the employee's information.

Let's consider class `Employee`'s declaration (Fig. 10.4). The class includes a constructor that receives the first name, last name and social security number (lines 11–17); *get* methods that return the first name, last name and social security number (lines 20–23, 26–29 and 32–35, respectively); method `toString` (lines 38–43), which returns the `String` representation of an `Employee`; and abstract method `earnings` (line 46), which will be implemented by each of the *concrete* subclasses. The `Employee` constructor does *not* validate its parameters in this example; normally, such validation should be provided.

```

1  // Fig. 10.4: Employee.java
2  // Employee abstract superclass.
3
4  public abstract class Employee
5  {
6      private final String firstName;
7      private final String lastName;
8      private final String socialSecurityNumber;
9
10     // constructor
11     public Employee(String firstName, String lastName,
12                     String socialSecurityNumber)
13     {
14         this.firstName = firstName;
15         this.lastName = lastName;
16         this.socialSecurityNumber = socialSecurityNumber;
17     }
18
19     // return first name
20     public String getFirstName()
21     {
22         return firstName;
23     }
24
25     // return last name
26     public String getLastName()
27     {
28         return lastName;
29     }
30
31     // return social security number
32     public String getSocialSecurityNumber()
33     {

```

Fig. 10.4 | `Employee` abstract superclass. (Part 1 of 2.)

```

34     return socialSecurityNumber;
35 }
36
37 // return String representation of Employee object
38 @Override
39 public String toString()
40 {
41     return String.format("%s %s%social security number: %s",
42         getFirstName(), getLastName(), getSocialSecurityNumber());
43 }
44
45 // abstract method must be overridden by concrete subclasses
46 public abstract double earnings(); // no implementation here
47 } // end abstract class Employee

```

Fig. 10.4 | Employee abstract superclass. (Part 2 of 2.)

Why did we decide to declare `earnings` as an abstract method? It simply does not make sense to provide a *specific* implementation of this method in class `Employee`. We cannot calculate the earnings for a *general* `Employee`—we first must know the *specific* type of `Employee` to determine the appropriate earnings calculation. By declaring this method abstract, we indicate that each concrete subclass *must* provide an appropriate earnings implementation and that a program will be able to use superclass `Employee` variables to invoke method `earnings` *polymorphically* for any type of `Employee`.

10.5.2 Concrete Subclass `SalariedEmployee`

Class `SalariedEmployee` (Fig. 10.5) extends class `Employee` (line 4) and overrides *abstract* method `earnings` (lines 38–42), which makes `SalariedEmployee` a *concrete* class. The class includes a constructor (lines 9–19) that receives a first name, a last name, a social security number and a weekly salary; a *set* method to assign a new *nonnegative* value to instance variable `weeklySalary` (lines 22–29); a *get* method to return `weeklySalary`'s value (lines 32–35); a method `earnings` (lines 38–42) to calculate a `SalariedEmployee`'s earnings; and a method `toString` (lines 45–50), which returns a `String` including "salaried employee: " followed by employee-specific information produced by superclass `Employee`'s `toString` method and `SalariedEmployee`'s `getWeeklySalary` method. Class `SalariedEmployee`'s constructor passes the first name, last name and social security number to the `Employee` constructor (line 12) to initialize the private instance variables of the superclass. Once again, we've duplicated the `weeklySalary` validation code in the constructor and the `setWeeklySalary` method. Recall that more complex validation could be placed in a static class method that's called from the constructor and the *set* method.



Error-Prevention Tip 10.1

We've said that you should not call a class's instance methods from its constructors—you can call static class methods and make the required call to one of the superclass's constructors. If you follow this advice, you'll avoid the problem of calling the class's overridable methods either directly or indirectly, which can lead to runtime errors.

Method `earnings` overrides `Employee`'s *abstract* method `earnings` to provide a *concrete* implementation that returns the `SalariedEmployee`'s weekly salary. If we do not

```

1  // Fig. 10.5: SalariedEmployee.java
2  // SalariedEmployee concrete class extends abstract class Employee.
3
4  public class SalariedEmployee extends Employee
5  {
6      private double weeklySalary;
7
8      // constructor
9      public SalariedEmployee(String firstName, String lastName,
10         String socialSecurityNumber, double weeklySalary)
11      {
12          super(firstName, lastName, socialSecurityNumber);
13
14          if (weeklySalary < 0.0)
15              throw new IllegalArgumentException(
16                  "Weekly salary must be >= 0.0");
17
18          this.weeklySalary = weeklySalary;
19      }
20
21      // set salary
22      public void setWeeklySalary(double weeklySalary)
23      {
24          if (weeklySalary < 0.0)
25              throw new IllegalArgumentException(
26                  "Weekly salary must be >= 0.0");
27
28          this.weeklySalary = weeklySalary;
29      }
30
31      // return salary
32      public double getWeeklySalary()
33      {
34          return weeklySalary;
35      }
36
37      // calculate earnings; override abstract method earnings in Employee
38      @Override
39      public double earnings()
40      {
41          return getWeeklySalary();
42      }
43
44      // return String representation of SalariedEmployee object
45      @Override
46      public String toString()
47      {
48          return String.format("salaried employee: %s\n%s: $%,.2f",
49              super.toString(), "weekly salary", getWeeklySalary());
50      }
51  } // end class SalariedEmployee

```

Fig. 10.5 | SalariedEmployee concrete class extends abstract class Employee.

implement earnings, class `SalariedEmployee` must be declared abstract—otherwise, class `SalariedEmployee` will not compile. Of course, we want `SalariedEmployee` to be a *concrete* class in this example.

Method `toString` (lines 45–50) overrides `Employee` method `toString`. If class `SalariedEmployee` did *not* override `toString`, `SalariedEmployee` would have inherited the `Employee` version of `toString`. In that case, `SalariedEmployee`'s `toString` method would simply return the employee's full name and social security number, which does *not* adequately represent a `SalariedEmployee`. To produce a complete `String` representation of a `SalariedEmployee`, the subclass's `toString` method returns "salaried employee: " followed by the superclass `Employee`-specific information (i.e., first name, last name and social security number) obtained by invoking the *superclass*'s `toString` method (line 49)—this is a nice example of *code reuse*. The `String` representation of a `SalariedEmployee` also contains the employee's weekly salary obtained by invoking the class's `getWeeklySalary` method.

10.5.3 Concrete Subclass `HourlyEmployee`

Class `HourlyEmployee` (Fig. 10.6) also extends `Employee` (line 4). The class includes a constructor (lines 10–25) that receives a first name, a last name, a social security number, an hourly wage and the number of hours worked. Lines 28–35 and 44–51 declare *set* methods that assign new values to instance variables `wage` and `hours`, respectively. Method `setWage` (lines 28–35) ensures that `wage` is *nonnegative*, and method `setHours` (lines 44–51) ensures that the value of `hours` is between 0 and 168 (the total number of hours in a week) inclusive. Class `HourlyEmployee` also includes *get* methods (lines 38–41 and 54–57) to return the values of `wage` and `hours`, respectively; a method `earnings` (lines 60–67) to calculate an `HourlyEmployee`'s earnings; and a method `toString` (lines 70–76), which returns a `String` containing the employee's type ("hourly employee: ") and the employee-specific information. The `HourlyEmployee` constructor, like the `SalariedEmployee` constructor, passes the first name, last name and social security number to the superclass `Employee` constructor (line 13) to initialize the private instance variables. In addition, method `toString` calls *superclass* method `toString` (line 74) to obtain the `Employee`-specific information (i.e., first name, last name and social security number)—this is another nice example of *code reuse*.

```

1  // Fig. 10.6: HourlyEmployee.java
2  // HourlyEmployee class extends Employee.
3
4  public class HourlyEmployee extends Employee
5  {
6      private double wage; // wage per hour
7      private double hours; // hours worked for week
8
9      // constructor
10     public HourlyEmployee(String firstName, String lastName,
11         String socialSecurityNumber, double wage, double hours)
12     {
13         super(firstName, lastName, socialSecurityNumber);
14

```

Fig. 10.6 | `HourlyEmployee` class extends `Employee`. (Part 1 of 3.)

```

15     if (wage < 0.0) // validate wage
16         throw new IllegalArgumentException(
17             "Hourly wage must be >= 0.0");
18
19     if ((hours < 0.0) || (hours > 168.0)) // validate hours
20         throw new IllegalArgumentException(
21             "Hours worked must be >= 0.0 and <= 168.0");
22
23     this.wage = wage;
24     this.hours = hours;
25 }
26
27 // set wage
28 public void setWage(double wage)
29 {
30     if (wage < 0.0) // validate wage
31         throw new IllegalArgumentException(
32             "Hourly wage must be >= 0.0");
33
34     this.wage = wage;
35 }
36
37 // return wage
38 public double getWage()
39 {
40     return wage;
41 }
42
43 // set hours worked
44 public void setHours(double hours)
45 {
46     if ((hours < 0.0) || (hours > 168.0)) // validate hours
47         throw new IllegalArgumentException(
48             "Hours worked must be >= 0.0 and <= 168.0");
49
50     this.hours = hours;
51 }
52
53 // return hours worked
54 public double getHours()
55 {
56     return hours;
57 }
58
59 // calculate earnings; override abstract method earnings in Employee
60 @Override
61 public double earnings()
62 {
63     if (getHours() <= 40) // no overtime
64         return getWage() * getHours();
65     else
66         return 40 * getWage() + (getHours() - 40) * getWage() * 1.5;
67 }

```

Fig. 10.6 | HourlyEmployee class extends Employee. (Part 2 of 3.)

```

68
69 // return String representation of HourlyEmployee object
70 @Override
71 public String toString()
72 {
73     return String.format("hourly employee: %s\n%s: $%,.2f; %s: %%,.2f",
74         super.toString(), "hourly wage", getWage(),
75         "hours worked", getHours());
76 }
77 } // end class HourlyEmployee

```

Fig. 10.6 | HourlyEmployee class extends Employee. (Part 3 of 3.)

10.5.4 Concrete Subclass CommissionEmployee

Class `CommissionEmployee` (Fig. 10.7) extends class `Employee` (line 4). The class includes a constructor (lines 10–25) that takes a first name, a last name, a social security number, a sales amount and a commission rate; *set* methods (lines 28–34 and 43–50) to assign valid new values to instance variables `commissionRate` and `grossSales`, respectively; *get* methods (lines 37–40 and 53–56) that retrieve the values of these instance variables; method `earnings` (lines 59–63) to calculate a `CommissionEmployee`'s earnings; and method `toString` (lines 66–73), which returns the employee's type, namely, "commission employee: " and employee-specific information. The constructor also passes the first name, last name and social security number to *superclass* `Employee`'s constructor (line 14) to initialize `Employee`'s private instance variables. Method `toString` calls *superclass* method `toString` (line 70) to obtain the `Employee`-specific information (i.e., first name, last name and social security number).

```

1 // Fig. 10.7: CommissionEmployee.java
2 // CommissionEmployee class extends Employee.
3
4 public class CommissionEmployee extends Employee
5 {
6     private double grossSales; // gross weekly sales
7     private double commissionRate; // commission percentage
8
9     // constructor
10    public CommissionEmployee(String firstName, String lastName,
11        String socialSecurityNumber, double grossSales,
12        double commissionRate)
13    {
14        super(firstName, lastName, socialSecurityNumber);
15
16        if (commissionRate <= 0.0 || commissionRate >= 1.0) // validate
17            throw new IllegalArgumentException(
18                "Commission rate must be > 0.0 and < 1.0");
19
20        if (grossSales < 0.0) // validate
21            throw new IllegalArgumentException("Gross sales must be >= 0.0");
22    }

```

Fig. 10.7 | CommissionEmployee class extends Employee. (Part 1 of 2.)

```

23     this.grossSales = grossSales;
24     this.commissionRate = commissionRate;
25 }
26
27 // set gross sales amount
28 public void setGrossSales(double grossSales)
29 {
30     if (grossSales < 0.0) // validate
31         throw new IllegalArgumentException("Gross sales must be >= 0.0");
32
33     this.grossSales = grossSales;
34 }
35
36 // return gross sales amount
37 public double getGrossSales()
38 {
39     return grossSales;
40 }
41
42 // set commission rate
43 public void setCommissionRate(double commissionRate)
44 {
45     if (commissionRate <= 0.0 || commissionRate >= 1.0) // validate
46         throw new IllegalArgumentException(
47             "Commission rate must be > 0.0 and < 1.0");
48
49     this.commissionRate = commissionRate;
50 }
51
52 // return commission rate
53 public double getCommissionRate()
54 {
55     return commissionRate;
56 }
57
58 // calculate earnings; override abstract method earnings in Employee
59 @Override
60 public double earnings()
61 {
62     return getCommissionRate() * getGrossSales();
63 }
64
65 // return String representation of CommissionEmployee object
66 @Override
67 public String toString()
68 {
69     return String.format("%s: %s\n%s: $%,.2f; %s: %.2f",
70         "commission employee", super.toString(),
71         "gross sales", getGrossSales(),
72         "commission rate", getCommissionRate());
73 }
74 } // end class CommissionEmployee

```

Fig. 10.7 | CommissionEmployee class extends Employee. (Part 2 of 2.)

10.5.5 Indirect Concrete Subclass BasePlusCommissionEmployee

Class `BasePlusCommissionEmployee` (Fig. 10.8) extends class `CommissionEmployee` (line 4) and therefore is an *indirect* subclass of class `Employee`. Class `BasePlusCommissionEmployee` has a constructor (lines 9–20) that receives a first name, a last name, a social security number, a sales amount, a commission rate and a base salary. It then passes all of these except the base salary to the `CommissionEmployee` constructor (lines 13–14) to initialize the superclass instance variables. `BasePlusCommissionEmployee` also contains a *set* method (lines 23–29) to assign a new value to instance variable `baseSalary` and a *get* method (lines 32–35) to return `baseSalary`'s value. Method `earnings` (lines 38–42) calculates a `BasePlusCommissionEmployee`'s earnings. Line 41 in method `earnings` calls *superclass* `CommissionEmployee`'s `earnings` method to calculate the commission-based portion of the employee's earnings—this is another nice example of *code reuse*. `BasePlusCommissionEmployee`'s `toString` method (lines 45–51) creates a `String` representation of a `BasePlusCommissionEmployee` that contains "base-salaried", followed by the `String` obtained by invoking *superclass* `CommissionEmployee`'s `toString` method (line 49), then the base salary. The result is a `String` beginning with "base-salaried commission employee" followed by the rest of the `BasePlusCommissionEmployee`'s information. Recall that `CommissionEmployee`'s `toString` obtains the employee's first name, last name and social security number by invoking the `toString` method of its *superclass* (i.e., `Employee`)—yet another example of *code reuse*. `BasePlusCommissionEmployee`'s `toString` initiates a *chain of method calls* that span all three levels of the `Employee` hierarchy.

```

1  // Fig. 10.8: BasePlusCommissionEmployee.java
2  // BasePlusCommissionEmployee class extends CommissionEmployee.
3
4  public class BasePlusCommissionEmployee extends CommissionEmployee
5  {
6      private double baseSalary; // base salary per week
7
8      // constructor
9      public BasePlusCommissionEmployee(String firstName, String lastName,
10         String socialSecurityNumber, double grossSales,
11         double commissionRate, double baseSalary)
12     {
13         super(firstName, lastName, socialSecurityNumber,
14             grossSales, commissionRate);
15
16         if (baseSalary < 0.0) // validate baseSalary
17             throw new IllegalArgumentException("Base salary must be >= 0.0");
18
19         this.baseSalary = baseSalary;
20     }
21
22     // set base salary
23     public void setBaseSalary(double baseSalary)
24     {
25         if (baseSalary < 0.0) // validate baseSalary
26             throw new IllegalArgumentException("Base salary must be >= 0.0");

```

Fig. 10.8 | `BasePlusCommissionEmployee` class extends `CommissionEmployee`. (Part I of 2.)

```

27
28     this.baseSalary = baseSalary;
29 }
30
31 // return base salary
32 public double getBaseSalary()
33 {
34     return baseSalary;
35 }
36
37 // calculate earnings; override method earnings in CommissionEmployee
38 @Override
39 public double earnings()
40 {
41     return getBaseSalary() + super.earnings();
42 }
43
44 // return String representation of BasePlusCommissionEmployee object
45 @Override
46 public String toString()
47 {
48     return String.format("%s %s; %s: $%,.2f",
49         "base-salaried", super.toString(),
50         "base salary", getBaseSalary());
51 }
52 } // end class BasePlusCommissionEmployee

```

Fig. 10.8 | BasePlusCommissionEmployee class extends CommissionEmployee. (Part 2 of 2.)

10.5.6 Polymorphic Processing, Operator instanceof and Downcasting

To test our Employee hierarchy, the application in Fig. 10.9 creates an object of each of the four *concrete* classes SalariedEmployee, HourlyEmployee, CommissionEmployee and BasePlusCommissionEmployee. The program manipulates these objects *nonpolymorphically*, via variables of each object's own type, then *polymorphically*, using an array of Employee variables. While processing the objects polymorphically, the program increases the base salary of each BasePlusCommissionEmployee by 10%—this requires *determining the object's type at execution time*. Finally, the program polymorphically determines and outputs the *type* of each object in the Employee array. Lines 9–18 create objects of each of the four concrete Employee subclasses. Lines 22–30 output the String representation and earnings of each of these objects *nonpolymorphically*. Each object's toString method is called *implicitly* by printf when the object is output as a String with the %s format specifier.

```

1 // Fig. 10.9: PayrollSystemTest.java
2 // Employee hierarchy test program.
3
4 public class PayrollSystemTest
5 {

```

Fig. 10.9 | Employee hierarchy test program. (Part 1 of 4.)

```

6 public static void main(String[] args)
7 {
8     // create subclass objects
9     SalariedEmployee salariedEmployee =
10     new SalariedEmployee("John", "Smith", "111-11-1111", 800.00);
11     HourlyEmployee hourlyEmployee =
12     new HourlyEmployee("Karen", "Price", "222-22-2222", 16.75, 40);
13     CommissionEmployee commissionEmployee =
14     new CommissionEmployee(
15     "Sue", "Jones", "333-33-3333", 10000, .06);
16     BasePlusCommissionEmployee basePlusCommissionEmployee =
17     new BasePlusCommissionEmployee(
18     "Bob", "Lewis", "444-44-4444", 5000, .04, 300);
19
20     System.out.println("Employees processed individually:");
21
22     System.out.printf("%n%s%n%s: $%,.2f%n%n",
23     salariedEmployee, "earned", salariedEmployee.earnings());
24     System.out.printf("%s%n%s: $%,.2f%n%n",
25     hourlyEmployee, "earned", hourlyEmployee.earnings());
26     System.out.printf("%s%n%s: $%,.2f%n%n",
27     commissionEmployee, "earned", commissionEmployee.earnings());
28     System.out.printf("%s%n%s: $%,.2f%n%n",
29     basePlusCommissionEmployee,
30     "earned", basePlusCommissionEmployee.earnings());
31
32     // create four-element Employee array
33     Employee[] employees = new Employee[4];
34
35     // initialize array with Employees
36     employees[0] = salariedEmployee;
37     employees[1] = hourlyEmployee;
38     employees[2] = commissionEmployee;
39     employees[3] = basePlusCommissionEmployee;
40
41     System.out.printf("Employees processed polymorphically:%n%n");
42
43     // generically process each element in array employees
44     for (Employee currentEmployee : employees)
45     {
46         System.out.println(currentEmployee); // invokes toString
47
48         // determine whether element is a BasePlusCommissionEmployee
49         if (currentEmployee instanceof BasePlusCommissionEmployee())
50         {
51             // downcast Employee reference to
52             // BasePlusCommissionEmployee reference
53             BasePlusCommissionEmployee employee =
54             (BasePlusCommissionEmployee) currentEmployee;
55
56             employee.setBaseSalary(1.10 * employee.getBaseSalary());
57

```

Fig. 10.9 | Employee hierarchy test program. (Part 2 of 4.)

```

58         System.out.printf(
59             "new base salary with 10%% increase is: $%,.2f%n",
60             employee.getBaseSalary());
61     } // end if
62
63     System.out.printf(
64         "earned $%,.2f%n%n", currentEmployee.earnings());
65 } // end for
66
67 // get type name of each object in employees array
68 for (int j = 0; j < employees.length; j++)
69     System.out.printf("Employee %d is a %s%n", j,
70         employees[j].getClass().getName());
71 } // end main
72 } // end class PayrollSystemTest

```

Employees processed individually:

salaried employee: John Smith
social security number: 111-11-1111
weekly salary: \$800.00
earned: \$800.00

hourly employee: Karen Price
social security number: 222-22-2222
hourly wage: \$16.75; hours worked: 40.00
earned: \$670.00

commission employee: Sue Jones
social security number: 333-33-3333
gross sales: \$10,000.00; commission rate: 0.06
earned: \$600.00

base-salaried commission employee: Bob Lewis
social security number: 444-44-4444
gross sales: \$5,000.00; commission rate: 0.04; base salary: \$300.00
earned: \$500.00

Employees processed polymorphically:

salaried employee: John Smith
social security number: 111-11-1111
weekly salary: \$800.00
earned \$800.00

hourly employee: Karen Price
social security number: 222-22-2222
hourly wage: \$16.75; hours worked: 40.00
earned \$670.00

commission employee: Sue Jones
social security number: 333-33-3333
gross sales: \$10,000.00; commission rate: 0.06
earned \$600.00

Fig. 10.9 | Employee hierarchy test program. (Part 3 of 4.)

```
base-salaried commission employee: Bob Lewis
social security number: 444-44-4444
gross sales: $5,000.00; commission rate: 0.04; base salary: $300.00
new base salary with 10% increase is: $330.00
earned $530.00

Employee 0 is a SalariedEmployee
Employee 1 is a HourlyEmployee
Employee 2 is a CommissionEmployee
Employee 3 is a BasePlusCommissionEmployee
```

Fig. 10.9 | Employee hierarchy test program. (Part 4 of 4.)

Creating the Array of Employees

Line 33 declares `employees` and assigns it an array of four `Employee` variables. Line 36 assigns to `employees[0]` a reference to a `SalariedEmployee` object. Line 37 assigns to `employees[1]` a reference to an `HourlyEmployee` object. Line 38 assigns to `employees[2]` a reference to a `CommissionEmployee` object. Line 39 assigns to `employees[3]` a reference to a `BasePlusCommissionEmployee` object. These assignments are allowed, because a `SalariedEmployee` *is an* `Employee`, an `HourlyEmployee` *is an* `Employee`, a `CommissionEmployee` *is an* `Employee` and a `BasePlusCommissionEmployee` *is an* `Employee`. Therefore, we can assign the references of `SalariedEmployee`, `HourlyEmployee`, `CommissionEmployee` and `BasePlusCommissionEmployee` objects to *superclass* `Employee` variables, *even though Employee is an abstract class*.

Polymorphically Processing Employees

Lines 44–65 iterate through array `employees` and invoke methods `toString` and `earnings` with `Employee` variable `currentEmployee`, which is assigned the reference to a different `Employee` in the array on each iteration. The output illustrates that the specific methods for each class are indeed invoked. All calls to method `toString` and `earnings` are resolved at *execution* time, based on the *type* of the object to which `currentEmployee` refers. This process is known as **dynamic binding** or **late binding**. For example, line 46 *implicitly* invokes method `toString` of the object to which `currentEmployee` refers. As a result of *dynamic binding*, Java decides which class's `toString` method to call *at execution time rather than at compile time*. Only the methods of class `Employee` can be called via an `Employee` variable (and `Employee`, of course, includes the methods of class `Object`). A superclass reference can be used to invoke only methods of the *superclass*—the *subclass* method implementations are invoked *polymorphically*.

Performing Type-Specific Operations on BasePlusCommissionEmployees

We perform special processing on `BasePlusCommissionEmployee` objects—as we encounter these objects at execution time, we increase their base salary by 10%. When processing objects *polymorphically*, we typically do not need to worry about the *specifics*, but to adjust the base salary, we *do* have to determine the *specific* type of `Employee` object at *execution time*. Line 49 uses the **instanceof** operator to determine whether a particular `Employee` object's type is `BasePlusCommissionEmployee`. The condition in line 49 is *true* if the object referenced by `currentEmployee` *is a* `BasePlusCommissionEmployee`. This would also be *true* for any object of a `BasePlusCommissionEmployee` subclass because of the *is-a* rela-

tionship a subclass has with its superclass. Lines 53–54 *downcast* `currentEmployee` from type `Employee` to type `BasePlusCommissionEmployee`—this cast is allowed only if the object has an *is-a* relationship with `BasePlusCommissionEmployee`. The condition at line 49 ensures that this is the case. This cast is required if we’re to invoke subclass `BasePlusCommissionEmployee` methods `getBaseSalary` and `setBaseSalary` on the current `Employee` object—as you’ll see momentarily, *attempting to invoke a subclass-only method directly on a superclass reference is a compilation error*.



Common Programming Error 10.3

Assigning a superclass variable to a subclass variable is a compilation error.



Common Programming Error 10.4

*When downcasting a reference, a `ClassCastException` occurs if the referenced object at execution time does not have an *is-a* relationship with the type specified in the cast operator.*

If the `instanceof` expression in line 49 is true, lines 53–60 perform the special processing required for the `BasePlusCommissionEmployee` object. Using `BasePlusCommissionEmployee` variable `employee`, line 56 invokes subclass-only methods `getBaseSalary` and `setBaseSalary` to retrieve and update the employee’s base salary with the 10% raise.

Calling earnings Polymorphically

Lines 63–64 invoke method `earnings` on `currentEmployee`, which polymorphically calls the appropriate subclass object’s `earnings` method. Obtaining the earnings of the `SalaryedEmployee`, `HourlyEmployee` and `CommissionEmployee` polymorphically in lines 63–64 produces the same results as obtaining these employees’ earnings individually in lines 22–27. The earnings amount obtained for the `BasePlusCommissionEmployee` in lines 63–64 is higher than that obtained in lines 28–30, due to the 10% increase in its base salary.

Getting Each Employee’s Class Name

Lines 68–70 display each employee’s type as a `String`. Every object *knows its own class* and can access this information through the `getClass` method, which all classes inherit from class `Object`. Method `getClass` returns an object of type `Class` (from package `java.lang`), which contains information about the object’s type, including its class name. Line 70 invokes `getClass` on the current object to get its class. The result of the `getClass` call is used to invoke `getName` to get the object’s class name.

Avoiding Compilation Errors with Downcasting

In the previous example, we avoided several compilation errors by *downcasting* an `Employee` variable to a `BasePlusCommissionEmployee` variable in lines 53–54. If you remove the cast operator (`BasePlusCommissionEmployee`) from line 54 and attempt to assign `Employee` variable `currentEmployee` directly to `BasePlusCommissionEmployee` variable `employee`, you’ll receive an “incompatible types” compilation error. This error indicates that the attempt to assign the reference of superclass object `currentEmployee` to subclass variable `employee` is *not* allowed. The compiler prevents this assignment because a `CommissionEmployee` is *not* a `BasePlusCommissionEmployee`—*the is-a relationship applies only between the subclass and its superclasses, not vice versa*.

Similarly, if lines 56 and 60 used superclass variable `currentEmployee` to invoke subclass-only methods `getBaseSalary` and `setBaseSalary`, we'd receive "cannot find symbol" compilation errors at these lines. Attempting to invoke subclass-only methods via a superclass variable is *not* allowed—even though lines 56 and 60 execute only if `instanceof` in line 49 returns `true` to indicate that `currentEmployee` holds a reference to a `BasePlusCommissionEmployee` object. Using a superclass `Employee` variable, we can invoke only methods found in class `Employee`—`earnings`, `toString` and `Employee`'s *get* and *set* methods.



Software Engineering Observation 10.5

Although the actual method that's called depends on the runtime type of the object to which a variable refers, a variable can be used to invoke only those methods that are members of that variable's type, which the compiler verifies.

10.6 Allowed Assignments Between Superclass and Subclass Variables

Now that you've seen a complete application that processes diverse subclass objects *polymorphically*, we summarize what you can and cannot do with superclass and subclass objects and variables. Although a subclass object also *is a* superclass object, the two classes are nevertheless different. As discussed previously, subclass objects can be treated as objects of their superclass. But because the subclass can have additional subclass-only members, assigning a superclass reference to a subclass variable is not allowed without an *explicit cast*—such an assignment would leave the subclass members undefined for the superclass object.

We've discussed three proper ways to assign superclass and subclass references to variables of superclass and subclass types:

1. Assigning a superclass reference to a superclass variable is straightforward.
2. Assigning a subclass reference to a subclass variable is straightforward.
3. Assigning a subclass reference to a superclass variable is safe, because the subclass object *is an* object of its superclass. However, the superclass variable can be used to refer *only* to superclass members. If this code refers to subclass-only members through the superclass variable, the compiler reports errors.

10.7 final Methods and Classes

We saw in Sections 6.3 and 6.10 that variables can be declared `final` to indicate that they cannot be modified *after* they're initialized—such variables represent constant values. It's also possible to declare methods, method parameters and classes with the `final` modifier.

Final Methods Cannot Be Overridden

A **final method** in a superclass *cannot* be overridden in a subclass—this guarantees that the `final` method implementation will be used by all direct and indirect subclasses in the hierarchy. Methods that are declared `private` are implicitly `final`, because it's not possible to override them in a subclass. Methods that are declared `static` are also implicitly `final`. A `final` method's declaration can never change, so all subclasses use the same method

implementation, and calls to `final` methods are resolved at compile time—this is known as **static binding**.

Final Classes Cannot Be Superclasses

A **final class** cannot be extended to create a subclass. All methods in a `final` class are implicitly `final`. Class `String` is an example of a `final` class. If you were allowed to create a subclass of `String`, objects of that subclass could be used wherever `Strings` are expected. Since class `String` cannot be extended, programs that use `Strings` can rely on the functionality of `String` objects as specified in the Java API. Making the class `final` also prevents programmers from creating subclasses that might bypass security restrictions.

We’ve now discussed declaring variables, methods and classes `final`, and we’ve emphasized that if something *can* be `final` it *should* be `final`. Compilers can perform various optimizations when they know something is `final`. When we study concurrency in Chapter 23, you’ll see that `final` variables make it much easier to parallelize your programs for use on today’s multi-core processors. For more insights on the use of `final`, visit

<http://docs.oracle.com/javase/tutorial/java/IandI/final.html>



Common Programming Error 10.5

Attempting to declare a subclass of a `final` class is a compilation error.



Software Engineering Observation 10.6

In the Java API, the vast majority of classes are not declared `final`. This enables inheritance and polymorphism. However, in some cases, it’s important to declare classes `final`—typically for security reasons. Also, unless you carefully design a class for extension, you should declare the class as `final` to avoid (often subtle) errors.

10.8 A Deeper Explanation of Issues with Calling Methods from Constructors

Do not call overridable methods from constructors. When creating a *subclass* object, this could lead to an overridden method being called before the *subclass* object is fully initialized.

Recall that when you construct a *subclass* object, its constructor first calls one of the direct *superclass*’s constructors. If the *superclass* constructor calls an overridable method, the *subclass*’s version of that method will be called by the *superclass* constructor—before the *subclass* constructor’s body has a chance to execute. This could lead to subtle, difficult-to-detect errors if the *subclass* method that was called depends on initialization that has not yet been performed in the *subclass* constructor’s body.

It’s acceptable to call a `static` method from a constructor. For example, a constructor and a *set* method often perform the same validation for a particular instance variable. If the validation code is brief, it’s acceptable to duplicate it in the constructor and the *set* method. If lengthier validation is required, define a `static` validation method (typically a private helper method) then call it from the constructor and the *set* method. It’s also acceptable for a constructor to call a `final` instance method, provided that the method does not directly or indirectly call an overridable instance method.

10.9 Creating and Using Interfaces

[Note: As written, this section and its code example apply through Java SE 7. Java SE 8's interface enhancements are introduced in Section 10.10 and discussed in more detail in Chapter 17.]

Our next example (Figs. 10.11–10.15) reexamines the payroll system of Section 10.5. Suppose that the company involved wishes to perform several accounting operations in a single accounts payable application—in addition to calculating the earnings that must be paid to each employee, the company must also calculate the payment due on each of several invoices (i.e., bills for goods purchased). Though applied to *unrelated* things (i.e., employees and invoices), both operations have to do with obtaining some kind of payment amount. For an employee, the payment refers to the employee's earnings. For an invoice, the payment refers to the total cost of the goods listed on the invoice. Can we calculate such *different* things as the payments due for employees and invoices in a *single* application *polymorphically*? Does Java offer a capability requiring that *unrelated* classes implement a set of *common* methods (e.g., a method that calculates a payment amount)? Java **interfaces** offer exactly this capability.

Standardizing Interactions

Interfaces define and standardize the ways in which things such as people and systems can interact with one another. For example, the controls on a radio serve as an interface between radio users and a radio's internal components. The controls allow users to perform only a limited set of operations (e.g., change the station, adjust the volume, choose between AM and FM), and different radios may implement the controls in different ways (e.g., using push buttons, dials, voice commands). The interface specifies *what* operations a radio must permit users to perform but does not specify *how* the operations are performed.

Software Objects Communicate Via Interfaces

Software objects also communicate via interfaces. A Java interface describes a set of methods that can be called on an object to tell it, for example, to perform some task or return some piece of information. The next example introduces an interface named `Payable` to describe the functionality of any object that must be “capable of being paid” and thus must offer a method to determine the proper payment amount due. An **interface declaration** begins with the keyword **interface** and contains *only* constants and abstract methods. Unlike classes, all interface members *must* be `public`, and *interfaces may not specify any implementation details*, such as concrete method declarations and instance variables. All methods declared in an interface are implicitly `public` abstract methods, and all fields are implicitly `public`, `static` and `final`.



Good Programming Practice 10.1

According to the Java Language Specification, it's proper style to declare an interface's abstract methods without keywords `public` and `abstract`, because they're redundant in interface-method declarations. Similarly, an interface's constants should be declared without keywords `public`, `static` and `final`, because they, too, are redundant.

Using an Interface

To use an interface, a concrete class must specify that it **implements** the interface and must declare each method in the interface with the signature specified in the interface declaration. To specify that a class implements an interface, add the `implements` keyword and the

name of the interface to the end of your class declaration's first line. A class that does not implement *all* the methods of the interface is an *abstract* class and must be declared `abstract`. Implementing an interface is like signing a *contract* with the compiler that states, "I will declare all the methods specified by the interface or I will declare my class `abstract`."



Common Programming Error 10.6

Failing to implement any method of an interface in a concrete class that implements the interface results in a compilation error indicating that the class must be declared `abstract`.

Relating Disparate Types

An interface is often used when *disparate* classes—i.e., classes that are not related by a class hierarchy—need to share common methods and constants. This allows objects of *unrelated* classes to be processed *polymorphically*—objects of classes that implement the *same* interface can respond to the *same* method calls. You can create an interface that describes the desired functionality, then implement this interface in any classes that require that functionality. For example, in the accounts payable application developed in this section, we implement interface `Payable` in any class that must be able to calculate a payment amount (e.g., `Employee`, `Invoice`).

Interfaces vs. Abstract Classes

An interface is often used in place of an abstract class when there's no default implementation to inherit—that is, no fields and no default method implementations. Like `public abstract` classes, interfaces are typically `public` types. Like a `public` class, a `public` interface must be declared in a file with the same name as the interface and the `.java` filename extension.



Software Engineering Observation 10.7

Many developers feel that interfaces are an even more important modeling technology than classes, especially with the new interface enhancements in Java SE 8 (see Section 10.10).

Tagging Interfaces

We'll see in Chapter 15, Files, Streams and Object Serialization, the notion of *tagging interfaces* (also called *marker interfaces*)—empty interfaces that have *no* methods or constant values. They're used to add *is-a* relationships to classes. For example, in Chapter 15 we'll discuss a mechanism called *object serialization*, which can convert objects to byte representations and can convert those byte representations back to objects. To enable this mechanism to work with your objects, you simply have to mark them as `Serializable` by adding `implements Serializable` to the end of your class declaration's first line. Then, all the objects of your class have the *is-a* relationship with `Serializable`.

10.9.1 Developing a `Payable` Hierarchy

To build an application that can determine payments for employees and invoices alike, we first create interface `Payable`, which contains method `getPaymentAmount` that returns a `double` amount that must be paid for an object of any class that implements the interface. Method `getPaymentAmount` is a general-purpose version of method `earnings` of the `Employee` hierarchy—method `earnings` calculates a payment amount specifically for an `Employee`, while `getPaymentAmount` can be applied to a broad range of possibly unrelated objects. After declaring interface `Payable`, we introduce class `Invoice`, which implements

interface `Payable`. We then modify class `Employee` such that it also implements interface `Payable`. Finally, we update `Employee` subclass `SalariedEmployee` to “fit” into the `Payable` hierarchy by renaming `SalariedEmployee` method `earnings` as `getPaymentAmount`.



Good Programming Practice 10.2

When declaring a method in an interface, choose a method name that describes the method’s purpose in a general manner, because the method may be implemented by many unrelated classes.

Classes `Invoice` and `Employee` both represent things for which the company must be able to calculate a payment amount. Both classes implement the `Payable` interface, so a program can invoke method `getPaymentAmount` on `Invoice` objects and `Employee` objects alike. As we’ll soon see, this enables the *polymorphic* processing of `Invoices` and `Employees` required for the company’s accounts payable application.

The UML class diagram in Fig. 10.10 shows the interface and class hierarchy used in our accounts payable application. The hierarchy begins with interface `Payable`. The UML distinguishes an interface from other classes by placing the word “interface” in guillemets (« and ») above the interface name. The UML expresses the relationship between a class and an interface through a relationship known as **realization**. A class is said to *realize*, or *implement*, the methods of an interface. A class diagram models a realization as a dashed arrow with a hollow arrowhead pointing from the implementing class to the interface. The diagram in Fig. 10.10 indicates that classes `Invoice` and `Employee` each realize interface `Payable`. As in the class diagram of Fig. 10.2, class `Employee` appears in *italics*, indicating that it’s an *abstract class*. Concrete class `SalariedEmployee` extends `Employee`, *inheriting its superclass’s realization relationship* with interface `Payable`.

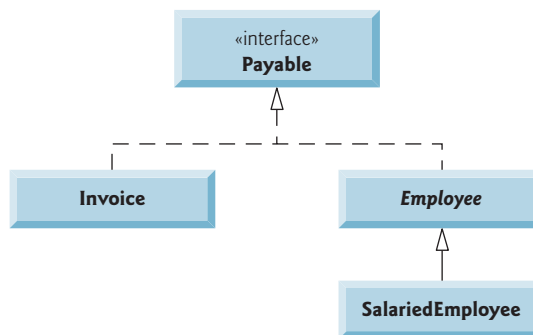


Fig. 10.10 | `Payable` hierarchy UML class diagram.

10.9.2 Interface `Payable`

The declaration of interface `Payable` begins in Fig. 10.11 at line 4. Interface `Payable` contains public abstract method `getPaymentAmount`. Interface methods are always public and abstract, so they do not need to be declared as such. Interface `Payable` has only one method, but interfaces can have *any* number of methods. In addition, method `getPaymentAmount` has no parameters, but interface methods *can* have parameters. Interfaces may also contain `final` static constants.

```

1 // Fig. 10.11: Payable.java
2 // Payable interface declaration.
3
4 public interface Payable
5 {
6     double getPaymentAmount(); // calculate payment; no implementation
7 }

```

Fig. 10.11 | Payable interface declaration.

10.9.3 Class Invoice

We now create class `Invoice` (Fig. 10.12) to represent a simple invoice that contains billing information for only one kind of part. The class declares private instance variables `partNumber`, `partDescription`, `quantity` and `pricePerItem` (in lines 6–9) that indicate the part number, a description of the part, the quantity of the part ordered and the price per item. Class `Invoice` also contains a constructor (lines 12–26), *get* and *set* methods (lines 29–69) that manipulate the class’s instance variables and a `toString` method (lines 72–78) that returns a `String` representation of an `Invoice` object. Methods `setQuantity` (lines 41–47) and `setPricePerItem` (lines 56–63) ensure that `quantity` and `pricePerItem` obtain only nonnegative values.

```

1 // Fig. 10.12: Invoice.java
2 // Invoice class that implements Payable.
3
4 public class Invoice implements Payable
5 {
6     private final String partNumber;
7     private final String partDescription;
8     private int quantity;
9     private double pricePerItem;
10
11     // constructor
12     public Invoice(String partNumber, String partDescription, int quantity,
13         double pricePerItem)
14     {
15         if (quantity < 0) // validate quantity
16             throw new IllegalArgumentException("Quantity must be >= 0");
17
18         if (pricePerItem < 0.0) // validate pricePerItem
19             throw new IllegalArgumentException(
20                 "Price per item must be >= 0");
21
22         this.quantity = quantity;
23         this.partNumber = partNumber;
24         this.partDescription = partDescription;
25         this.pricePerItem = pricePerItem;
26     } // end constructor
27

```

Fig. 10.12 | Invoice class that implements `Payable`. (Part I of 3.)

```

28     // get part number
29     public String getPartNumber()
30     {
31         return partNumber; // should validate
32     }
33
34     // get description
35     public String getPartDescription()
36     {
37         return partDescription;
38     }
39
40     // set quantity
41     public void setQuantity(int quantity)
42     {
43         if (quantity < 0) // validate quantity
44             throw new IllegalArgumentException("Quantity must be >= 0");
45
46         this.quantity = quantity;
47     }
48
49     // get quantity
50     public int getQuantity()
51     {
52         return quantity;
53     }
54
55     // set price per item
56     public void setPricePerItem(double pricePerItem)
57     {
58         if (pricePerItem < 0.0) // validate pricePerItem
59             throw new IllegalArgumentException(
60                 "Price per item must be >= 0");
61
62         this.pricePerItem = pricePerItem;
63     }
64
65     // get price per item
66     public double getPricePerItem()
67     {
68         return pricePerItem;
69     }
70
71     // return String representation of Invoice object
72     @Override
73     public String toString()
74     {
75         return String.format("%s: %n%s: %s (%s) %n%s: %d %n%s: $%,.2f",
76             "invoice", "part number", getPartNumber(), getPartDescription(),
77             "quantity", getQuantity(), "price per item", getPricePerItem());
78     }
79

```

Fig. 10.12 | Invoice class that implements Payable. (Part 2 of 3.)

```

80 // method required to carry out contract with interface Payable
81 @Override
82 public double getPaymentAmount()
83 {
84     return getQuantity() * getPricePerItem(); // calculate total cost
85 }
86 } // end class Invoice

```

Fig. 10.12 | Invoice class that implements Payable. (Part 3 of 3.)

A Class Can Extend Only One Other Class But Can Implement Many Interfaces

Line 4 indicates that class `Invoice` implements interface `Payable`. Like all classes, class `Invoice` also *implicitly* extends `Object`. Java does not allow subclasses to inherit from more than one superclass, but it allows a class to *inherit* from one *superclass* and *implement* as many *interfaces* as it needs. To implement more than one interface, use a comma-separated list of interface names after keyword `implements` in the class declaration, as in:

```
public class ClassName extends SuperclassName implements FirstInterface,
    SecondInterface, ...
```



Software Engineering Observation 10.8

All objects of a class that implements multiple interfaces have the is-a relationship with each implemented interface type.

Class `Invoice` implements the one abstract method in interface `Payable`—method `getPaymentAmount` is declared in lines 81–85. The method calculates the total payment required to pay the invoice. The method multiplies the values of `quantity` and `pricePerItem` (obtained through the appropriate *get* methods) and returns the result. This method satisfies the implementation requirement for this method in interface `Payable`—we’ve *fulfilled* the *interface contract* with the compiler.

10.9.4 Modifying Class `Employee` to Implement Interface `Payable`

We now modify class `Employee` such that it implements interface `Payable`. Figure 10.13 contains the modified class, which is identical to that of Fig. 10.4 with two exceptions. First, line 4 of Fig. 10.13 indicates that class `Employee` now implements interface `Payable`. For this example, we renamed method `earnings` to `getPaymentAmount` throughout the `Employee` hierarchy. As with method `earnings` in the version of class `Employee` in Fig. 10.4, however, it does not make sense to *implement* method `getPaymentAmount` in class `Employee` because we cannot calculate the earnings payment owed to a *general* `Employee`—we must first know the *specific* type of `Employee`. In Fig. 10.4, we declared method `earnings` as abstract for this reason, so class `Employee` had to be declared abstract. This forced each `Employee` *concrete* subclass to *override* `earnings` with an implementation.

```

1 // Fig. 10.13: Employee.java
2 // Employee abstract superclass that implements Payable.
3

```

Fig. 10.13 | Employee abstract superclass that implements Payable. (Part 1 of 2.)

```

4 public abstract class Employee implements Payable
5 {
6     private final String firstName;
7     private final String lastName;
8     private final String socialSecurityNumber;
9
10    // constructor
11    public Employee(String firstName, String lastName,
12        String socialSecurityNumber)
13    {
14        this.firstName = firstName;
15        this.lastName = lastName;
16        this.socialSecurityNumber = socialSecurityNumber;
17    }
18
19    // return first name
20    public String getFirstName()
21    {
22        return firstName;
23    }
24
25    // return last name
26    public String getLastName()
27    {
28        return lastName;
29    }
30
31    // return social security number
32    public String getSocialSecurityNumber()
33    {
34        return socialSecurityNumber;
35    }
36
37    // return String representation of Employee object
38    @Override
39    public String toString()
40    {
41        return String.format("%s %s\nsocial security number: %s",
42            getFirstName(), getLastName(), getSocialSecurityNumber());
43    }
44
45    // Note: We do not implement Payable method getPaymentAmount here so
46    // this class must be declared abstract to avoid a compilation error.
47 } // end abstract class Employee

```

Fig. 10.13 | Employee abstract superclass that implements Payable. (Part 2 of 2.)

In Fig. 10.13, we handle this situation differently. Recall that when a class implements an interface, it makes a *contract* with the compiler stating either that the class will implement *each* method in the interface or the class will be declared abstract. Because class `Employee` does not provide a `getPaymentAmount` method, the class must be declared abstract. Any concrete subclass of the abstract class *must* implement the interface methods to fulfill the superclass's contract with the compiler. If the subclass does *not* do so, it too *must* be declared

abstract. As indicated by the comments in lines 45–46, class `Employee` of Fig. 10.13 does *not* implement method `getPaymentAmount`, so the class is declared abstract. Each direct `Employee` subclass *inherits the superclass's contract* to implement method `getPaymentAmount` and thus must implement this method to become a concrete class for which objects can be instantiated. A class that extends one of `Employee`'s concrete subclasses will inherit an implementation of `getPaymentAmount` and thus will also be a concrete class.

10.9.5 Modifying Class `SalariedEmployee` for Use in the Payable Hierarchy

Figure 10.14 contains a modified `SalariedEmployee` class that extends `Employee` and fulfills superclass `Employee`'s contract to implement Payable method `getPaymentAmount`. This version of `SalariedEmployee` is identical to that of Fig. 10.5, but it replaces method earnings with method `getPaymentAmount` (lines 39–43). Recall that the Payable version of the method has a more *general* name to be applicable to possibly *disparate* classes. (If we included the remaining `Employee` subclasses from Section 10.5—`HourlyEmployee`, `CommissionEmployee` and `BasePlusCommissionEmployee`—in this example, their earnings methods should also be renamed `getPaymentAmount`. We leave these modifications to Exercise 10.15 and use only `SalariedEmployee` in our test program here. Exercise 10.16 asks you to implement interface Payable in the entire `Employee` class hierarchy of Figs. 10.4–10.9 *without* modifying the `Employee` subclasses.)

```

1  // Fig. 10.14: SalariedEmployee.java
2  // SalariedEmployee class that implements interface Payable.
3  // method getPaymentAmount.
4  public class SalariedEmployee extends Employee
5  {
6      private double weeklySalary;
7
8      // constructor
9      public SalariedEmployee(String firstName, String lastName,
10         String socialSecurityNumber, double weeklySalary)
11     {
12         super(firstName, lastName, socialSecurityNumber);
13
14         if (weeklySalary < 0.0)
15             throw new IllegalArgumentException(
16                 "Weekly salary must be >= 0.0");
17
18         this.weeklySalary = weeklySalary;
19     }
20
21     // set salary
22     public void setWeeklySalary(double weeklySalary)
23     {
24         if (weeklySalary < 0.0)
25             throw new IllegalArgumentException(
26                 "Weekly salary must be >= 0.0");

```

Fig. 10.14 | `SalariedEmployee` class that implements interface Payable method `getPaymentAmount`. (Part 1 of 2.)


```

27
28     this.weeklySalary = weeklySalary;
29 }
30
31 // return salary
32 public double getWeeklySalary()
33 {
34     return weeklySalary;
35 }
36
37 // calculate earnings; implement interface Payable method that was
38 // abstract in superclass Employee
39 @Override
40 public double getPaymentAmount()
41 {
42     return getWeeklySalary();
43 }
44
45 // return String representation of SalariedEmployee object
46 @Override
47 public String toString()
48 {
49     return String.format("salaried employee: %s\n%s: $%,.2f",
50         super.toString(), "weekly salary", getWeeklySalary());
51 }
52 } // end class SalariedEmployee

```

Fig. 10.14 | SalariedEmployee class that implements interface Payable method getPaymentAmount. (Part 2 of 2.)

When a class implements an interface, the same *is-a* relationship provided by inheritance applies. Class Employee implements Payable, so we can say that an Employee *is a* Payable. In fact, objects of any classes that extend Employee are also Payable objects. SalariedEmployee objects, for instance, are Payable objects. Objects of any subclasses of the class that implements the interface can also be thought of as objects of the interface type. Thus, just as we can assign the reference of a SalariedEmployee object to a superclass Employee variable, we can assign the reference of a SalariedEmployee object to an interface Payable variable. Invoice implements Payable, so an Invoice object also *is a* Payable object, and we can assign the reference of an Invoice object to a Payable variable.



Software Engineering Observation 10.9

When a method parameter is declared with a superclass or interface type, the method processes the object passed as an argument polymorphically.



Software Engineering Observation 10.10

Using a superclass reference, we can polymorphically invoke any method declared in the superclass and its superclasses (e.g., class Object). Using an interface reference, we can polymorphically invoke any method declared in the interface, its superinterfaces (one interface can extend another) and in class Object—a variable of an interface type must refer to an object to call methods, and all objects have the methods of class Object.

10.9.6 Using Interface Payable to Process Invoices and Employees Polymorphically

PayableInterfaceTest (Fig. 10.15) illustrates that interface Payable can be used to process a set of Invoices and Employees *polymorphically* in a single application. Line 9 declares payableObjects and assigns it an array of four Payable variables. Lines 12–13 assign the references of Invoice objects to the first two elements of payableObjects. Lines 14–17 then assign the references of SalariedEmployee objects to the remaining two elements of payableObjects. These assignments are allowed because an Invoice *is a* Payable, a SalariedEmployee *is an* Employee and an Employee *is a* Payable. Lines 23–29 use the enhanced for statement to *polymorphically* process each Payable object in payableObjects, printing the object as a String, along with the payment amount due. Line 27 invokes method toString via a Payable interface reference, even though toString is not declared in interface Payable—all references (including those of interface types) refer to objects that extend Object and therefore have a toString method. (Method toString also can be invoked implicitly here.) Line 28 invokes Payable method getPaymentAmount to obtain the payment amount for each object in payableObjects, regardless of the actual type of the object. The output reveals that each of the method calls in lines 27–28 invokes the appropriate class's implementation of methods toString and getPaymentAmount. For instance, when currentPayable refers to an Invoice during the first iteration of the for loop, class Invoice's toString and getPaymentAmount execute.

```

1  // Fig. 10.15: PayableInterfaceTest.java
2  // Payable interface test program processing Invoices and
3  // Employees polymorphically.
4  public class PayableInterfaceTest
5  {
6      public static void main(String[] args)
7      {
8          // create four-element Payable array
9          Payable[] payableObjects = new Payable[4];
10
11         // populate array with objects that implement Payable
12         payableObjects[0] = new Invoice("01234", "seat", 2, 375.00);
13         payableObjects[1] = new Invoice("56789", "tire", 4, 79.95);
14         payableObjects[2] =
15             new SalariedEmployee("John", "Smith", "111-11-1111", 800.00);
16         payableObjects[3] =
17             new SalariedEmployee("Lisa", "Barnes", "888-88-8888", 1200.00);
18
19         System.out.println(
20             "Invoices and Employees processed polymorphically:");
21
22         // generically process each element in array payableObjects
23         for (Payable currentPayable : payableObjects)
24         {

```

Fig. 10.15 | Payable interface test program processing Invoices and Employees polymorphically. (Part I of 2.)

```

25         // output currentPayable and its appropriate payment amount
26         System.out.printf("%n%s %n%s: $%,.2f%n",
27             currentPayable.toString(), // could invoke implicitly
28             "payment due", currentPayable.getPaymentAmount());
29     }
30 } // end main
31 } // end class PayableInterfaceTest

```

Invoices and Employees processed polymorphically:

```

invoice:
part number: 01234 (seat)
quantity: 2
price per item: $375.00
payment due: $750.00

invoice:
part number: 56789 (tire)
quantity: 4
price per item: $79.95
payment due: $319.80

salaried employee: John Smith
social security number: 111-11-1111
weekly salary: $800.00
payment due: $800.00

salaried employee: Lisa Barnes
social security number: 888-88-8888
weekly salary: $1,200.00
payment due: $1,200.00

```

Fig. 10.15 | Payable interface test program processing Invoices and Employees polymorphically. (Part 2 of 2.)

10.9.7 Some Common Interfaces of the Java API

You'll use interfaces extensively when developing Java applications. The Java API contains numerous interfaces, and many of the Java API methods take interface arguments and return interface values. Figure 10.16 overviews a few of the more popular interfaces of the Java API that we use in later chapters.

Interface	Description
Comparable	Java contains several comparison operators (e.g., <, <=, >, >=, ==, !=) that allow you to compare primitive values. However, these operators <i>cannot</i> be used to compare objects. Interface Comparable is used to allow objects of a class that implements the interface to be compared to one another. Interface Comparable is commonly used for ordering objects in a collection such as an array. We use Comparable in Chapter 16, Generic Collections, and Chapter 20, Generic Classes and Methods.

Fig. 10.16 | Common interfaces of the Java API. (Part 1 of 2.)

Interface	Description
<code>Serializable</code>	An interface used to identify classes whose objects can be written to (i.e., serialized) or read from (i.e., deserialized) some type of storage (e.g., file on disk, database field) or transmitted across a network. We use <code>Serializable</code> in Chapter 15, Files, Streams and Object Serialization, and Chapter 28, Networking.
<code>Runnable</code>	Implemented by any class that represents a task to perform. Objects of such as class are often executed in parallel using a technique called <i>multithreading</i> (discussed in Chapter 23, Concurrency). The interface contains one method, <code>run</code> , which specifies the behavior of an object when executed.
GUI event-listener interfaces	You work with graphical user interfaces (GUIs) every day. In your web browser, you might type the address of a website to visit, or you might click a button to return to a previous site. The browser responds to your interaction and performs the desired task. Your interaction is known as an <i>event</i> , and the code that the browser uses to respond to an event is known as an <i>event handler</i> . In Chapter 12, GUI Components: Part 1, and Chapter 22, GUI Components: Part 2, you'll learn how to build GUIs and event handlers that respond to user interactions. Event handlers are declared in classes that implement an appropriate <i>event-listener interface</i> . Each event-listener interface specifies one or more methods that must be implemented to respond to user interactions.
<code>AutoCloseable</code>	Implemented by classes that can be used with the try-with-resources statement (Chapter 11, Exception Handling: A Deeper Look) to help prevent resource leaks.

Fig. 10.16 | Common interfaces of the Java API. (Part 2 of 2.)

10.10 Java SE 8 Interface Enhancements

This section introduces Java SE 8's new interface features. We discuss these in more detail in later chapters.

10.10.1 default Interface Methods

Prior to Java SE 8, interface methods could be *only* public abstract methods. This meant that an interface specified *what* operations an implementing class must perform but not *how* the class should perform them.

In Java SE 8, interfaces also may contain public **default methods** with *concrete* default implementations that specify *how* operations are performed when an implementing class does not override the methods. If a class implements such an interface, the class also receives the interface's default implementations (if any). To declare a default method, place the keyword `default` before the method's return type and provide a concrete method implementation.

Adding Methods to Existing Interfaces

Prior to Java SE 8, adding methods to an interface would break any implementing classes that did not implement the new methods. Recall that if you didn't implement each of an interface's methods, you had to declare your class abstract.

Any class that implements the original interface will *not* break when a default method is added—the class simply receives the new default method. When a class imple-

ments a Java SE 8 interface, the class “signs a contract” with the compiler that says, “I will declare all the *abstract* methods specified by the interface or I will declare my class *abstract*”—the implementing class is not required to override the interface’s default methods, but it can if necessary.



Software Engineering Observation 10.11

Java SE 8 default methods enable you to evolve existing interfaces by adding new methods to those interfaces without breaking code that uses them.

Interfaces vs. *abstract* Classes

Prior to Java SE 8, an interface was typically used (rather than an abstract class) when there were no implementation details to inherit—no fields and no method implementations. With default methods, you can instead declare common method implementations in interfaces, which gives you more flexibility in designing your classes.

10.10.2 static Interface Methods

Prior to Java SE 8, it was common to associate with an interface a class containing static helper methods for working with objects that implemented the interface. In Chapter 16, you’ll learn about class `Collections` which contains many static helper methods for working with objects that implement interfaces `Collection`, `List`, `Set` and more. For example, `Collections` method `sort` can sort objects of *any* class that implements interface `List`. With static interface methods, such helper methods can now be declared directly in interfaces rather than in separate classes.

10.10.3 Functional Interfaces

As of Java SE 8, any interface containing only one abstract method is known as a **functional interface**. There are many such interfaces throughout the Java SE 7 APIs, and many new ones in Java SE 8. Some functional interfaces that you’ll use in this book include:

- `ActionListener` (Chapter 12)—You’ll implement this interface to define a method that’s called when the user clicks a button.
- `Comparator` (Chapter 16)—You’ll implement this interface to define a method that can compare two objects of a given type to determine whether the first object is less than, equal to or greater than the second.
- `Runnable` (Chapter 23)—You’ll implement this interface to define a task that may be run in parallel with other parts of your program.

Functional interfaces are used extensively with Java SE 8’s new lambda capabilities that we introduce in Chapter 17. In Chapter 12, you’ll often implement an interface by creating a so-called anonymous inner class that implements the interface’s method(s). In Java SE 8, lambdas provide a shorthand notation for creating *anonymous methods* that the compiler automatically translates into anonymous inner classes for you.

10.11 (Optional) GUI and Graphics Case Study: Drawing with Polymorphism

You may have noticed in the drawing program created in GUI and Graphics Case Study Exercise 8.1 (and modified in GUI and Graphics Case Study Exercise 9.1) that shape

classes have many similarities. Using inheritance, we can “factor out” the common features from all three classes and place them in a single shape *superclass*. Then, using variables of the superclass type, we can manipulate shape objects *polymorphically*. Removing the redundant code will result in a smaller, more flexible program that’s easier to maintain.

GUI and Graphics Case Study Exercises

10.1 Modify the `MyLine`, `MyOval` and `MyRectangle` classes of GUI and Graphics Case Study Exercise 8.1 and Exercise 9.1 to create the class hierarchy in Fig. 10.17. Classes of the `MyShape` hierarchy should be “smart” shape classes that know how to draw themselves (if provided with a `Graphics` object that tells them where to draw). Once the program creates an object from this hierarchy, it can manipulate it *polymorphically* for the rest of its lifetime as a `MyShape`.

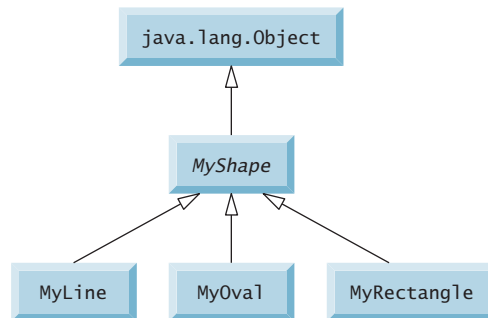


Fig. 10.17 | `MyShape` hierarchy.

In your solution, class `MyShape` in Fig. 10.17 *must* be abstract. Since `MyShape` represents any shape in general, you cannot implement a `draw` method without knowing *specifically* what shape it is. The data representing the coordinates and color of the shapes in the hierarchy should be declared as private members of class `MyShape`. In addition to the common data, class `MyShape` should declare the following methods:

- A *no-argument constructor* that sets all the coordinates of the shape to 0 and the color to `Color.BLACK`.
- A constructor that initializes the coordinates and color to the values of the arguments supplied.
- Set* methods for the individual coordinates and color that allow the programmer to set any piece of data independently for a shape in the hierarchy.
- Get* methods for the individual coordinates and color that allow the programmer to retrieve any piece of data independently for a shape in the hierarchy.
- The abstract method

```
public abstract void draw(Graphics g);
```

which the program’s `paintComponent` method will call to draw a shape on the screen.

To ensure proper encapsulation, all data in class `MyShape` must be private. This requires declaring proper *set* and *get* methods to manipulate the data. Class `MyLine` should provide a no-argument constructor and a constructor with arguments for the coordinates and color. Classes `MyOval` and `MyRectangle` should provide a no-argument constructor and a constructor with argu-

ments for the coordinates, color and determining whether the shape is filled. The no-argument constructor should, in addition to setting the default values, set the shape to be an unfilled shape.

You can draw lines, rectangles and ovals if you know two points in space. Lines require $x1$, $y1$, $x2$ and $y2$ coordinates. The `drawLine` method of the `Graphics` class will connect the two points supplied with a line. If you have the same four coordinate values ($x1$, $y1$, $x2$ and $y2$) for ovals and rectangles, you can calculate the four arguments needed to draw them. Each requires an upper-left x -coordinate value (the smaller of the two x -coordinate values), an upper-left y -coordinate value (the smaller of the two y -coordinate values), a *width* (the absolute value of the difference between the two x -coordinate values) and a *height* (the absolute value of the difference between the two y -coordinate values). Rectangles and ovals should also have a `filled` flag that determines whether to draw the shape as a filled shape.

There should be no `MyLine`, `MyOval` or `MyRectangle` variables in the program—only `MyShape` variables that contain references to `MyLine`, `MyOval` and `MyRectangle` objects. The program should generate random shapes and store them in an array of type `MyShape`. Method `paintComponent` should walk through the `MyShape` array and draw every shape, by polymorphically calling every shape's `draw` method.

Allow the user to specify (via an input dialog) the number of shapes to generate. The program will then generate and display the shapes along with a status bar that informs the user how many of each shape were created.

10.2 (Drawing Application Modification) In the preceding exercise, you created a `MyShape` hierarchy in which classes `MyLine`, `MyOval` and `MyRectangle` extend `MyShape` directly. If your hierarchy was properly designed, you should be able to see the similarities between the `MyOval` and `MyRectangle` classes. Redesign and reimplement the code for the `MyOval` and `MyRectangle` classes to “factor out” the common features into the abstract class `MyBoundedShape` to produce the hierarchy in Fig. 10.18.

Class `MyBoundedShape` should declare two constructors that mimic those of class `MyShape`, only with an added parameter to specify whether the shape is filled. Class `MyBoundedShape` should also declare *get* and *set* methods for manipulating the filled flag and methods that calculate the upper-left x -coordinate, upper-left y -coordinate, width and height. Remember, the values needed to draw an oval or a rectangle can be calculated from two (x, y) coordinates. If designed properly, the new `MyOval` and `MyRectangle` classes should each have two constructors and a `draw` method.

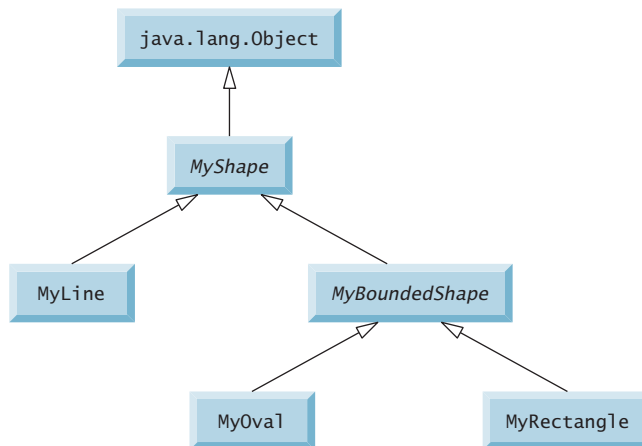


Fig. 10.18 | `MyShape` hierarchy with `MyBoundedShape`.

10.12 Wrap-Up

This chapter introduced polymorphism—the ability to process objects that share the same superclass in a class hierarchy as if they were all objects of the superclass. We discussed how polymorphism makes systems extensible and maintainable, then demonstrated how to use overridden methods to effect polymorphic behavior. We introduced abstract classes, which allow you to provide an appropriate superclass from which other classes can inherit. You learned that an abstract class can declare abstract methods that each subclass must implement to become a concrete class and that a program can use variables of an abstract class to invoke the subclasses’ implementations of abstract methods polymorphically. You also learned how to determine an object’s type at execution time. We explained the notions of final methods and classes. Finally, the chapter discussed declaring and implementing an interface as a way for possibly disparate classes to implement common functionality, enabling objects of those classes to be processed polymorphically.

You should now be familiar with classes, objects, encapsulation, inheritance, interfaces and polymorphism—the most essential aspects of object-oriented programming.

In the next chapter, you’ll learn about exceptions, useful for handling errors during a program’s execution. Exception handling provides for more robust programs.

Summary

Section 10.1 Introduction

- Polymorphism (p. 396) enables us to write programs that process objects that share the same superclass as if they were all objects of the superclass; this can simplify programming.
- With polymorphism, we can design and implement systems that are easily extensible. The only parts of a program that must be altered to accommodate new classes are those that require direct knowledge of the new classes that you add to the hierarchy.

Section 10.3 Demonstrating Polymorphic Behavior

- When the compiler encounters a method call made through a variable, it determines if the method can be called by checking the variable’s class type. If that class contains the proper method declaration (or inherits one), the call is compiled. At execution time, the type of the object to which the variable refers determines the actual method to use.

Section 10.4 Abstract Classes and Methods

- Abstract classes (p. 401) cannot be used to instantiate objects, because they’re incomplete.
- The primary purpose of an abstract class is to provide an appropriate superclass from which other classes can inherit and thus share a common design.
- Classes that can be used to instantiate objects are called concrete classes (p. 402). They provide implementations of every method they declare (some of the implementations can be inherited).
- Programmers often write client code that uses only abstract superclasses (p. 402) to reduce client code’s dependencies on specific subclass types.
- Abstract classes sometimes constitute several levels of a hierarchy.
- An abstract class normally contains one or more abstract methods (p. 402).
- Abstract methods do not provide implementations.
- A class that contains any abstract methods must be declared as an abstract class (p. 402). Each concrete subclass must provide implementations of each of the superclass’s abstract methods.

- Constructors and static methods cannot be declared abstract.
- Abstract superclass variables can hold references to objects of any concrete class derived from the superclass. Programs typically use such variables to manipulate subclass objects polymorphically.
- Polymorphism is particularly effective for implementing layered software systems.

Section 10.5 Case Study: Payroll System Using Polymorphism

- A hierarchy designer can demand that each concrete subclass provide an appropriate method implementation by including an abstract method in a superclass.
- Most method calls are resolved at execution time, based on the type of the object being manipulated. This process is known as dynamic binding (p. 417) or late binding.
- A superclass variable can be used to invoke only methods declared in the superclass.
- Operator `instanceof` (p. 417) determines if an object has the *is-a* relationship with a specific type.
- Every object in Java knows its own class and can access it through Object method `getClass` (p. 418), which returns an object of type `Class` (package `java.lang`).
- The *is-a* relationship applies only between the subclass and its superclasses, not vice versa.

Section 10.7 `final` Methods and Classes

- A method that's declared `final` (p. 419) in a superclass cannot be overridden in a subclass.
- Methods declared `private` are implicitly `final`, because you can't override them in a subclass.
- Methods that are declared `static` are implicitly `final`.
- A `final` method's declaration can never change, so all subclasses use the same implementation, and calls to `final` methods are resolved at compile time—this is known as static binding (p. 420).
- The compiler can optimize programs by removing calls to `final` methods and inlining their expanded code at each method-call location.
- A class that's declared `final` cannot be extended (p. 420).
- All methods in a `final` class are implicitly `final`.

Section 10.9 Creating and Using Interfaces

- An interface (p. 421) specifies *what* operations are allowed but not *how* they're performed.
- A Java interface describes a set of methods that can be called on an object.
- An interface declaration begins with the keyword `interface` (p. 421).
- All interface members must be `public`, and interfaces may not specify any implementation details, such as concrete method declarations and instance variables.
- All methods declared in an interface are implicitly `public` abstract methods and all fields are implicitly `public`, `static` and `final`.
- To use an interface, a concrete class must specify that it implements (p. 421) the interface and must declare each interface method with the signature specified in the interface declaration. A class that does not implement all the interface's methods must be declared `abstract`.
- Implementing an interface is like signing a contract with the compiler that states, "I will declare all the methods specified by the interface or I will declare my class `abstract`."
- An interface is typically used when disparate (i.e., unrelated) classes need to share common methods and constants. This allows objects of unrelated classes to be processed polymorphically—objects of classes that implement the same interface can respond to the same method calls.
- You can create an interface that describes the desired functionality, then implement the interface in any classes that require that functionality.

- An interface is often used in place of an abstract class when there's no default implementation to inherit—that is, no instance variables and no default method implementations.
- Like `public` abstract classes, interfaces are typically `public` types, so they're normally declared in files by themselves with the same name as the interface and the `.java` filename extension.
- Java does not allow subclasses to inherit from more than one superclass, but it does allow a class to inherit from a superclass and implement more than one interface.
- All objects of a class that implement multiple interfaces have the *is-a* relationship with each implemented interface type.
- An interface can declare constants. The constants are implicitly `public`, `static` and `final`.

Section 10.10 Java SE 8 Interface Enhancements

- In Java SE 8, an interface may declare `default` methods—that is, `public` methods with concrete implementations that specify how an operation should be performed.
- When a class implements an interface, the class receives the interface's `default` concrete implementations if it does not override them.
- To declare a `default` method in an interface, you simply place the keyword `default` before the method's return type and provide a complete method body.
- When you enhance an existing interface with `default` methods—any classes that implemented the original interface will not break—it'll simply receive the `default` method implementations.
- With `default` methods, you can declare common method implementations in interfaces (rather than abstract classes), which gives you more flexibility in designing your classes.
- As of Java SE 8, interfaces may now include `public static` methods.
- As of Java SE 8, any interface containing only one method is known as a functional interface. There are many such interfaces throughout the Java APIs.
- Functional interfaces are used extensively with Java SE 8's new lambda capabilities. As you'll see, lambdas provide a shorthand notation for creating anonymous methods.

Self-Review Exercises

- 10.1** Fill in the blanks in each of the following statements:
- If a class contains at least one abstract method, it's a(n) _____ class.
 - Classes from which objects can be instantiated are called _____ classes.
 - _____ involves using a superclass variable to invoke methods on superclass and subclass objects, enabling you to “program in the general.”
 - Methods that are not interface methods and that do not provide implementations must be declared using keyword _____.
 - Casting a reference stored in a superclass variable to a subclass type is called _____.
- 10.2** State whether each of the statements that follows is *true* or *false*. If *false*, explain why.
- All methods in an abstract class must be declared as abstract methods.
 - Invoking a subclass-only method through a subclass variable is not allowed.
 - If a superclass declares an abstract method, a subclass must implement that method.
 - An object of a class that implements an interface may be thought of as an object of that interface type.
- 10.3** (*Java SE 8 interfaces*) Fill in the blanks in each of the following statements:
- In Java SE 8, an interface may declare _____—that is, `public` methods with concrete implementations that specify how an operation should be performed.
 - As of Java SE 8, interfaces can now include _____ helper methods.
 - As of Java SE 8, any interface containing only one method is known as a(n) _____.

Answers to Self-Review Exercises

- 10.1** a) abstract. b) concrete. c) Polymorphism. d) abstract. e) downcasting.
- 10.2** a) False. An abstract class can include methods with implementations and abstract methods. b) False. Trying to invoke a subclass-only method with a superclass variable is not allowed. c) False. Only a concrete subclass must implement the method. d) True.
- 10.3** a) default methods. b) static. c) functional interface.

Exercises

- 10.4** How does polymorphism enable you to program “in the general” rather than “in the specific”? Discuss the key advantages of programming “in the general.”
- 10.5** What are abstract methods? Describe the circumstances in which an abstract method would be appropriate.
- 10.6** How does polymorphism promote extensibility?
- 10.7** Discuss three proper ways in which you can assign superclass and subclass references to variables of superclass and subclass types.
- 10.8** Compare and contrast abstract classes and interfaces. Why would you use an abstract class? Why would you use an interface?
- 10.9** (*Java SE 8 Interfaces*) Explain how default methods enable you to add new methods to an existing interface without breaking the classes that implemented the original interface.
- 10.10** (*Java SE 8 Interfaces*) What is a functional interface?
- 10.11** (*Java SE 8 Interfaces*) Why is it useful to be able to add static methods to interfaces?
- 10.12** (*Payroll System Modification*) Modify the payroll system of Figs. 10.4–10.9 to include private instance variable `birthDate` in class `Employee`. Use class `Date` of Fig. 8.7 to represent an employee’s birthday. Add *get* methods to class `Date`. Assume that payroll is processed once per month. Create an array of `Employee` variables to store references to the various employee objects. In a loop, calculate the payroll for each `Employee` (polymorphically), and add a \$100.00 bonus to the person’s payroll amount if the current month is the one in which the `Employee`’s birthday occurs.
- 10.13** (*Project: Shape Hierarchy*) Implement the Shape hierarchy shown in Fig. 9.3. Each `TwoDimensionalShape` should contain method `getArea` to calculate the area of the two-dimensional shape. Each `ThreeDimensionalShape` should have methods `getArea` and `getVolume` to calculate the surface area and volume, respectively, of the three-dimensional shape. Create a program that uses an array of `Shape` references to objects of each concrete class in the hierarchy. The program should print a text description of the object to which each array element refers. Also, in the loop that processes all the shapes in the array, determine whether each shape is a `TwoDimensionalShape` or a `ThreeDimensionalShape`. If it’s a `TwoDimensionalShape`, display its area. If it’s a `ThreeDimensionalShape`, display its area and volume.
- 10.14** (*Payroll System Modification*) Modify the payroll system of Figs. 10.4–10.9 to include an additional `Employee` subclass `PieceWorker` that represents an employee whose pay is based on the number of pieces of merchandise produced. Class `PieceWorker` should contain private instance variables `wage` (to store the employee’s wage per piece) and `pieces` (to store the number of pieces produced). Provide a concrete implementation of method `earnings` in class `PieceWorker` that calculates the employee’s earnings by multiplying the number of pieces produced by the wage per piece. Create an array of `Employee` variables to store references to objects of each concrete class in the new `Employee` hierarchy. For each `Employee`, display its `String` representation and earnings.

10.15 (Accounts Payable System Modification) In this exercise, we modify the accounts payable application of Figs. 10.11–10.15 to include the complete functionality of the payroll application of Figs. 10.4–10.9. The application should still process two `Invoice` objects, but now should process one object of each of the four `Employee` subclasses. If the object currently being processed is a `BasePlusCommissionEmployee`, the application should increase the `BasePlusCommissionEmployee`'s base salary by 10%. Finally, the application should output the payment amount for each object. Complete the following steps to create the new application:

- Modify classes `HourlyEmployee` (Fig. 10.6) and `CommissionEmployee` (Fig. 10.7) to place them in the `Payable` hierarchy as subclasses of the version of `Employee` (Fig. 10.13) that implements `Payable`. [*Hint: Change the name of method `earnings` to `getPaymentAmount` in each subclass so that the class satisfies its inherited contract with interface `Payable`.*]
- Modify class `BasePlusCommissionEmployee` (Fig. 10.8) such that it extends the version of class `CommissionEmployee` created in part (a).
- Modify `PayableInterfaceTest` (Fig. 10.15) to polymorphically process two `Invoices`, one `SalariedEmployee`, one `HourlyEmployee`, one `CommissionEmployee` and one `BasePlusCommissionEmployee`. First output a `String` representation of each `Payable` object. Next, if an object is a `BasePlusCommissionEmployee`, increase its base salary by 10%. Finally, output the payment amount for each `Payable` object.

10.16 (Accounts Payable System Modification) It's possible to include the functionality of the payroll application (Figs. 10.4–10.9) in the accounts payable application without modifying `Employee` subclasses `SalariedEmployee`, `HourlyEmployee`, `CommissionEmployee` or `BasePlusCommissionEmployee`. To do so, you can modify class `Employee` (Fig. 10.4) to implement interface `Payable` and declare method `getPaymentAmount` to invoke method `earnings`. Method `getPaymentAmount` would then be inherited by the subclasses in the `Employee` hierarchy. When `getPaymentAmount` is called for a particular subclass object, it polymorphically invokes the appropriate `earnings` method for that subclass. Reimplement Exercise 10.15 using the original `Employee` hierarchy from the payroll application of Figs. 10.4–10.9. Modify class `Employee` as described in this exercise, and *do not* modify any of class `Employee`'s subclasses.

Making a Difference

10.17 (CarbonFootprint Interface: Polymorphism) Using interfaces, as you learned in this chapter, you can specify similar behaviors for possibly disparate classes. Governments and companies worldwide are becoming increasingly concerned with carbon footprints (annual releases of carbon dioxide into the atmosphere) from buildings burning various types of fuels for heat, vehicles burning fuels for power, and the like. Many scientists blame these greenhouse gases for the phenomenon called global warming. Create three small classes unrelated by inheritance—classes `Building`, `Car` and `Bicycle`. Give each class some unique appropriate attributes and behaviors that it does not have in common with other classes. Write an interface `CarbonFootprint` with a `getCarbonFootprint` method. Have each of your classes implement that interface, so that its `getCarbonFootprint` method calculates an appropriate carbon footprint for that class (check out a few websites that explain how to calculate carbon footprints). Write an application that creates objects of each of the three classes, places references to those objects in `ArrayList<CarbonFootprint>`, then iterates through the `ArrayList`, polymorphically invoking each object's `getCarbonFootprint` method. For each object, print some identifying information and the object's carbon footprint.

Exception Handling: A Deeper Look

11



It is common sense to take a method and try it. If it fails, admit it frankly and try another. But above all, try something.

—Franklin Delano Roosevelt

*O! throw away the
worse part of it,
And live the purer
with the other half.*

—William Shakespeare

*If they're running and they don't
look where they're going
I have to come out from
somewhere and catch them.*

—Jerome David Salinger

Objectives

In this chapter you'll:

- Learn what exceptions are and how they're handled.
- Learn when to use exception handling.
- Use `try` blocks to delimit code in which exceptions might occur.
- Use `throw` to indicate a problem.
- Use `catch` blocks to specify exception handlers.
- Use the `finally` block to release resources.
- Understand the exception class hierarchy.
- Create user-defined exceptions.

- 11.1 Introduction
- 11.2 Example: Divide by Zero without Exception Handling
- 11.3 Example: Handling `ArithmeticExceptions` and `InputMismatchExceptions`
- 11.4 When to Use Exception Handling
- 11.5 Java Exception Hierarchy
- 11.6 `finally` Block
- 11.7 Stack Unwinding and Obtaining Information from an Exception Object
- 11.8 Chained Exceptions
- 11.9 Declaring New Exception Types
- 11.10 Preconditions and Postconditions
- 11.11 Assertions
- 11.12 `try-with-Resources`: Automatic Resource Deallocation
- 11.13 Wrap-Up

Summary | Self-Review Exercises | Answers to Self-Review Exercises | Exercises

11.1 Introduction

As you know from Chapter 7, an exception is an indication of a problem that occurs during a program's execution. Exception handling enables you to create applications that can resolve (or handle) exceptions. In many cases, handling an exception allows a program to continue executing as if no problem had been encountered. The features presented in this chapter help you write *robust* and *fault-tolerant* programs that can deal with problems and continue executing or *terminate gracefully*. Java exception handling is based in part on the work of Andrew Koenig and Bjarne Stroustrup.¹

First, we demonstrate basic exception-handling techniques by handling an exception that occurs when a method attempts to divide an integer by zero. Next, we introduce several classes at the top of Java's exception-handling class hierarchy. As you'll see, only classes that extend `Throwable` (package `java.lang`) directly or indirectly can be used with exception handling. We then show how to use *chained exceptions*—when you invoke a method that indicates an exception, you can throw another exception and chain the original one to the new one. This enables you to add application-specific information to the original exception. Next, we introduce *preconditions* and *postconditions*, which must be true when your methods are called and when they return, respectively. We then present *assertions*, which you can use at development time to help debug your code. We also discuss two exception-handling features that were introduced in Java SE 7—catching multiple exceptions with one catch handler and the new `try-with-resources` statement that automatically releases a resource after it's used in the `try` block.

This chapter focuses on the exception-handling concepts and presents several mechanical examples that demonstrate various features. As you'll see in later chapters, many Java APIs methods throw exceptions that we handle in our code. Figure 11.1 shows some of the exception types you've already seen and others you'll learn about.

1. A. Koenig and B. Stroustrup, "Exception Handling for C++ (revised)," *Proceedings of the Usenix C++ Conference*, pp. 149–176, San Francisco, April 1990.

Chapter	Sample of exceptions used
Chapter 7	<code>ArrayIndexOutOfBoundsException</code>
Chapters 8–10	<code>IllegalArgumentException</code>
Chapter 11	<code>ArithmeticException</code> , <code>InputMismatchException</code>
Chapter 15	<code>SecurityException</code> , <code>FileNotFoundException</code> , <code>IOException</code> , <code>ClassNotFoundException</code> , <code>IllegalStateException</code> , <code>FormatterClosedException</code> , <code>NoSuchElementException</code>
Chapter 16	<code>ClassCastException</code> , <code>UnsupportedOperationException</code> , <code>NullPointerException</code> , custom exception types
Chapter 20	<code>ClassCastException</code> , custom exception types
Chapter 21	<code>IllegalArgumentException</code> , custom exception types
Chapter 23	<code>InterruptedException</code> , <code>IllegalMonitorStateException</code> , <code>ExecutionException</code> , <code>CancellationException</code>
Chapter 28	<code>MalformedURLException</code> , <code>EOFException</code> , <code>SocketException</code> , <code>InterruptedException</code> , <code>UnknownHostException</code>
Chapter 24	<code>SQLException</code> , <code>IllegalStateException</code> , <code>PatternSyntaxException</code>
Chapter 31	<code>SQLException</code>

Fig. 11.1 | Various exception types that you'll see throughout this book

11.2 Example: Divide by Zero without Exception Handling

First we demonstrate what happens when errors arise in an application that does not use exception handling. Figure 11.2 prompts the user for two integers and passes them to method `quotient`, which calculates the integer quotient and returns an `int` result. In this example, you'll see that exceptions are **thrown** (i.e., the exception occurs) by a method when it detects a problem and is unable to handle it.

```

1  // Fig. 11.2: DivideByZeroNoExceptionHandling.java
2  // Integer division without exception handling.
3  import java.util.Scanner;
4
5  public class DivideByZeroNoExceptionHandling
6  {
7      // demonstrates throwing an exception when a divide-by-zero occurs
8      public static int quotient(int numerator, int denominator)
9      {
10         return numerator / denominator; // possible division by zero
11     }
12
13     public static void main(String[] args)
14     {
15         Scanner scanner = new Scanner(System.in);

```

Fig. 11.2 | Integer division without exception handling. (Part 1 of 2.)

```

16
17     System.out.print("Please enter an integer numerator: ");
18     int numerator = scanner.nextInt();
19     System.out.print("Please enter an integer denominator: ");
20     int denominator = scanner.nextInt();
21
22     int result = quotient(numerator, denominator);
23     System.out.printf(
24         "%nResult: %d / %d = %d%n", numerator, denominator, result);
25 }
26 } // end class DivideByZeroNoExceptionHandling

```

```

Please enter an integer numerator: 100
Please enter an integer denominator: 7

Result: 100 / 7 = 14

```

```

Please enter an integer numerator: 100
Please enter an integer denominator: 0
Exception in thread "main" java.lang.ArithmeticException: / by zero
    at DivideByZeroNoExceptionHandling.quotient(
        DivideByZeroNoExceptionHandling.java:10)
    at DivideByZeroNoExceptionHandling.main(
        DivideByZeroNoExceptionHandling.java:22)

```

```

Please enter an integer numerator: 100
Please enter an integer denominator: hello
Exception in thread "main" java.util.InputMismatchException
    at java.util.Scanner.throwFor(Unknown Source)
    at java.util.Scanner.next(Unknown Source)
    at java.util.Scanner.nextInt(Unknown Source)
    at java.util.Scanner.nextInt(Unknown Source)
    at DivideByZeroNoExceptionHandling.main(
        DivideByZeroNoExceptionHandling.java:20)

```

Fig. 11.2 | Integer division without exception handling. (Part 2 of 2.)

Stack Trace

The first sample execution in Fig. 11.2 shows a successful division. In the second execution, the user enters the value 0 as the denominator. Several lines of information are displayed in response to this invalid input. This information is known as a **stack trace**, which includes the name of the exception (`java.lang.ArithmeticException`) in a descriptive message that indicates the problem that occurred and the method-call stack (i.e., the call chain) at the time it occurred. The stack trace includes the path of execution that led to the exception method by method. This helps you debug the program.

Stack Trace for an `ArithmeticException`

The first line specifies that an `ArithmeticException` has occurred. The text after the name of the exception ("`/ by zero`") indicates that this exception occurred as a result of an attempt to divide by zero. Java does not allow division by zero in integer arithmetic. When this occurs, Java throws an **`ArithmeticException`**. `ArithmeticExceptions` can arise from

a number of different problems, so the extra data (“/ by zero”) provides more specific information. Java *does* allow division by zero with floating-point values. Such a calculation results in the value positive or negative infinity, which is represented in Java as a floating-point value (but displays as the string `Infinity` or `-Infinity`). If 0.0 is divided by 0.0, the result is NaN (not a number), which is also represented in Java as a floating-point value (but displays as `NaN`). If you need to compare a floating-point value to NaN, use the method `isNaN` of class `Float` (for `float` values) or of class `Double` (for `double` values). Classes `Float` and `Double` are in package `java.lang`.

Starting from the last line of the stack trace, we see that the exception was detected in line 22 of method `main`. Each line of the stack trace contains the class name and method (e.g., `DivideByZeroNoExceptionHandling.main`) followed by the filename and line number (e.g., `DivideByZeroNoExceptionHandling.java:22`). Moving up the stack trace, we see that the exception occurs in line 10, in method `quotient`. The top row of the call chain indicates the **throw point**—the initial point at which the exception occurred. The throw point of this exception is in line 10 of method `quotient`.

Stack Trace for an InputMismatchException

In the third execution, the user enters the string “hello” as the denominator. Notice again that a stack trace is displayed. This informs us that an `InputMismatchException` has occurred (package `java.util`). Our prior examples that input numeric values assumed that the user would input a proper integer value. However, users sometimes make mistakes and input noninteger values. An **`InputMismatchException`** occurs when `Scanner` method `nextInt` receives a string that does not represent a valid integer. Starting from the end of the stack trace, we see that the exception was detected in line 20 of method `main`. Moving up the stack trace, we see that the exception occurred in method `nextInt`. Notice that in place of the filename and line number, we’re provided with the text `Unknown Source`. This means that the so-called *debugging symbols* that provide the filename and line number information for that method’s class were not available to the JVM—this is typically the case for the classes of the Java API. Many IDEs have access to the Java API source code and will display filenames and line numbers in stack traces.

Program Termination

In the sample executions of Fig. 11.2 when exceptions occur and stack traces are displayed, the program also *exits*. This does not always occur in Java. Sometimes a program may continue even though an exception has occurred and a stack trace has been printed. In such cases, the application may produce unexpected results. For example, a graphical user interface (GUI) application will often continue executing. In Fig. 11.2 both types of exceptions were detected in method `main`. In the next example, we’ll see how to *handle* these exceptions so that you can enable the program to run to normal completion.

11.3 Example: Handling ArithmeticExceptions and InputMismatchExceptions

The application in Fig. 11.3, which is based on Fig. 11.2, uses *exception handling* to process any `ArithmeticExceptions` and `InputMismatchExceptions` that arise. The application still prompts the user for two integers and passes them to method `quotient`, which calculates the quotient and returns an `int` result. This version of the application uses ex-

ception handling so that if the user makes a mistake, the program catches and handles (i.e., deals with) the exception—in this case, allowing the user to re-enter the input.

```

1  // Fig. 11.3: DivideByZeroWithExceptionHandling.java
2  // Handling ArithmeticExceptions and InputMismatchExceptions.
3  import java.util.InputMismatchException;
4  import java.util.Scanner;
5
6  public class DivideByZeroWithExceptionHandling
7  {
8      // demonstrates throwing an exception when a divide-by-zero occurs
9      public static int quotient(int numerator, int denominator)
10     {
11         throws ArithmeticException
12         {
13             return numerator / denominator; // possible division by zero
14         }
15
16     public static void main(String[] args)
17     {
18         Scanner scanner = new Scanner(System.in);
19         boolean continueLoop = true; // determines if more input is needed
20
21         do
22         {
23             try // read two numbers and calculate quotient
24             {
25                 System.out.print("Please enter an integer numerator: ");
26                 int numerator = scanner.nextInt();
27                 System.out.print("Please enter an integer denominator: ");
28                 int denominator = scanner.nextInt();
29
30                 int result = quotient(numerator, denominator);
31                 System.out.printf("%nResult: %d / %d = %d%n", numerator,
32                                     denominator, result);
33                 continueLoop = false; // input successful; end looping
34             }
35             catch (InputMismatchException inputMismatchException)
36             {
37                 System.err.printf("%nException: %s%n",
38                                     inputMismatchException);
39                 scanner.nextLine(); // discard input so user can try again
40                 System.out.printf(
41                     "You must enter integers. Please try again.%n%n");
42             }
43             catch (ArithmeticException arithmeticException)
44             {
45                 System.err.printf("%nException: %s%n", arithmeticException);
46                 System.out.printf(
47                     "Zero is an invalid denominator. Please try again.%n%n");
48             }
49         } while (continueLoop);
50     } // end class DivideByZeroWithExceptionHandling

```

Fig. 11.3 | Handling ArithmeticExceptions and InputMismatchExceptions. (Part 1 of 2.)

```
Please enter an integer numerator: 100  
Please enter an integer denominator: 7
```

```
Result: 100 / 7 = 14
```

```
Please enter an integer numerator: 100  
Please enter an integer denominator: 0
```

```
Exception: java.lang.ArithmeticException: / by zero  
Zero is an invalid denominator. Please try again.
```

```
Please enter an integer numerator: 100  
Please enter an integer denominator: 7
```

```
Result: 100 / 7 = 14
```

```
Please enter an integer numerator: 100  
Please enter an integer denominator: hello
```

```
Exception: java.util.InputMismatchException  
You must enter integers. Please try again.
```

```
Please enter an integer numerator: 100  
Please enter an integer denominator: 7
```

```
Result: 100 / 7 = 14
```

Fig. 11.3 | Handling ArithmeticExceptions and InputMismatchExceptions. (Part 2 of 2.)

The first sample execution in Fig. 11.3 does *not* encounter any problems. In the second execution the user enters a *zero denominator*, and an `ArithmeticException` exception occurs. In the third execution the user enters the string "hello" as the denominator, and an `InputMismatchException` occurs. For each exception, the user is informed of the mistake and asked to try again, then is prompted for two new integers. In each sample execution, the program runs to completion successfully.

Class `InputMismatchException` is imported in line 3. Class `ArithmeticException` does not need to be imported because it's in package `java.lang`. Line 18 creates the `boolean` variable `continueLoop`, which is `true` if the user has *not* yet entered valid input. Lines 20–48 repeatedly ask users for input until a *valid* input is received.

Enclosing Code in a try Block

Lines 22–33 contain a **try block**, which encloses the code that *might* throw an exception and the code that should *not* execute if an exception occurs (i.e., if an exception occurs, the remaining code in the try block will be skipped). A try block consists of the keyword `try` followed by a block of code enclosed in curly braces. [Note: The term "try block" sometimes refers only to the block of code that follows the `try` keyword (not including the `try` keyword itself). For simplicity, we use the term "try block" to refer to the block of

code that follows the `try` keyword, as well as the `try` keyword.] The statements that read the integers from the keyboard (lines 25 and 27) each use method `nextInt` to read an `int` value. Method `nextInt` throws an `InputMismatchException` if the value read in is *not* an integer.

The division that can cause an `ArithmeticException` is not performed in the `try` block. Rather, the call to method `quotient` (line 29) invokes the code that attempts the division (line 12); the JVM *throws* an `ArithmeticException` object when the denominator is zero.



Software Engineering Observation 11.1

Exceptions may surface through explicitly mentioned code in a `try` block, through deeply nested method calls initiated by code in a `try` block or from the Java Virtual Machine as it executes Java bytecodes.

Catching Exceptions

The `try` block in this example is followed by two catch blocks—one that handles an `InputMismatchException` (lines 34–41) and one that handles an `ArithmeticException` (lines 42–47). A **catch block** (also called a **catch clause** or **exception handler**) *catches* (i.e., receives) and *handles* an exception. A catch block begins with the keyword `catch` followed by a parameter in parentheses (called the *exception parameter*, discussed shortly) and a block of code enclosed in curly braces. [Note: The term “catch clause” is sometimes used to refer to the keyword `catch` followed by a block of code, whereas the term “catch block” refers to only the block of code following the `catch` keyword, but not including it. For simplicity, we use the term “catch block” to refer to the block of code following the `catch` keyword, as well as the keyword itself.]

At least one catch block or a **finally block** (discussed in Section 11.6) *must* immediately follow the `try` block. Each catch block specifies in parentheses an **exception parameter** that identifies the exception type the handler can process. When an exception occurs in a `try` block, the catch block that executes is the *first* one whose type matches the type of the exception that occurred (i.e., the type in the catch block matches the thrown exception type exactly or is a direct or indirect superclass of it). The exception parameter’s name enables the catch block to interact with a caught exception object—e.g., to implicitly invoke the caught exception’s `toString` method (as in lines 37 and 44), which displays basic information about the exception. Notice that we use the **System.err (standard error stream) object** to output error messages. By default, `System.err`’s print methods, like those of `System.out`, display data to the *command prompt*.

Line 38 of the first catch block calls `Scanner` method `nextLine`. Because an `InputMismatchException` occurred, the call to method `nextInt` never successfully read in the user’s data—so we read that input with a call to method `nextLine`. We do not do anything with the input at this point, because we know that it’s *invalid*. Each catch block displays an error message and asks the user to try again. After either catch block terminates, the user is prompted for input. We’ll soon take a deeper look at how this flow of control works in exception handling.



Common Programming Error 11.1

It’s a syntax error to place code between a `try` block and its corresponding catch blocks.

Multi-catch

It's relatively common for a try block to be followed by several catch blocks to handle various types of exceptions. If the bodies of several catch blocks are identical, you can use the **multi-catch** feature (introduced in Java SE 7) to catch those exception types in a *single* catch handler and perform the same task. The syntax for a *multi-catch* is:

```
catch (Type1 | Type2 | Type3 e)
```

Each exception type is separated from the next with a vertical bar (|). The preceding line of code indicates that *any* of the types (or their subclasses) can be caught in the exception handler. Any number of Throwable types can be specified in a multi-catch.

Uncaught Exceptions

An **uncaught exception** is one for which there are no matching catch blocks. You saw uncaught exceptions in the second and third outputs of Fig. 11.2. Recall that when exceptions occurred in that example, the application terminated early (after displaying the exception's *stack trace*). This does not always occur as a result of uncaught exceptions. Java uses a "multithreaded" model of program execution—each **thread** is a *concurrent activity*. One program can have many threads. If a program has only *one* thread, an uncaught exception will cause the program to terminate. If a program has *multiple* threads, an uncaught exception will terminate *only* the thread in which the exception occurred. In such programs, however, certain threads may rely on others, and if one thread terminates due to an uncaught exception, there may be adverse effects on the rest of the program. Chapter 23, Concurrency, discusses these issues in depth.

Termination Model of Exception Handling

If an exception occurs in a try block (such as an InputMismatchException being thrown as a result of the code at line 25 of Fig. 11.3), the try block *terminates* immediately and program control transfers to the *first* of the following catch blocks in which the exception parameter's type matches the thrown exception's type. In Fig. 11.3, the first catch block catches InputMismatchExceptions (which occur if invalid input is entered) and the second catch block catches ArithmeticExceptions (which occur if an attempt is made to divide by zero). After the exception is handled, program control does *not* return to the throw point, because the try block has *expired* (and its *local variables* have been *lost*). Rather, control resumes after the last catch block. This is known as the **termination model of exception handling**. Some languages use the **resumption model of exception handling**, in which, after an exception is handled, control resumes just after the *throw point*.

Notice that we name our exception parameters (inputMismatchException and arithmeticException) based on their type. Java programmers often simply use the letter *e* as the name of their exception parameters.

After executing a catch block, this program's flow of control proceeds to the first statement after the last catch block (line 48 in this case). The condition in the `do...while` statement is `true` (variable `continueLoop` contains its initial value of `true`), so control returns to the beginning of the loop and the user is again prompted for input. This control statement will loop until *valid* input is entered. At that point, program control reaches line 32, which assigns `false` to variable `continueLoop`. The try block then *terminates*. If no exceptions are thrown in the try block, the catch blocks are *skipped* and control continues with the first statement after the catch blocks (we'll learn about another possibility when

we discuss the `finally` block in Section 11.6). Now the condition for the `do...while` loop is false, and method `main` ends.

The `try` block and its corresponding `catch` and/or `finally` blocks form a **try statement**. Do not confuse the terms “try block” and “try statement”—the latter includes the try block as well as the following catch blocks and/or `finally` block.

As with any other block of code, when a `try` block terminates, *local variables* declared in the block *go out of scope* and are no longer accessible; thus, the local variables of a `try` block are not accessible in the corresponding catch blocks. When a catch block *terminates*, *local variables* declared within the catch block (including the exception parameter of that catch block) also *go out of scope* and are *destroyed*. Any remaining catch blocks in the `try` statement are *ignored*, and execution resumes at the first line of code after the `try...catch` sequence—this will be a `finally` block, if one is present.

Using the **throws** Clause

In method `quotient` (Fig. 11.3, lines 9–13), line 10 is known as a **throws clause**. It specifies the exceptions the method *might* throw if problems occur. This clause, which must appear after the method’s parameter list and before the body, contains a comma-separated list of the exception types. Such exceptions may be thrown by statements in the method’s body or by methods called from there. We’ve added the `throws` clause to this application to indicate that this method might throw an `ArithmeticException`. Method `quotient`’s callers are thus informed that the method might throw an `ArithmeticException`. Some exception types, such as `ArithmeticException`, are not required to be listed in the `throws` clause. For those that are, the method can throw exceptions that have the *is-a* relationship with the classes listed in the `throws` clause. You’ll learn more about this in Section 11.5.



Error-Prevention Tip 11.1

Read the online API documentation for a method before using it in a program. The documentation specifies the exceptions thrown by the method (if any) and indicates reasons why such exceptions may occur. Next, read the online API documentation for the specified exception classes. The documentation for an exception class typically contains potential reasons that such exceptions occur. Finally, provide for handling those exceptions in your program.

When line 12 executes, if the denominator is zero, the JVM throws an `ArithmeticException` object. This object will be caught by the catch block at lines 42–47, which displays basic information about the exception by *implicitly* invoking the exception’s `toString` method, then asks the user to try again.

If the denominator is not zero, method `quotient` performs the division and returns the result to the point of invocation of method `quotient` in the `try` block (line 29). Lines 30–31 display the result of the calculation and line 32 sets `continueLoop` to false. In this case, the `try` block completes successfully, so the program skips the catch blocks and fails the condition at line 48, and method `main` completes execution normally.

When `quotient` throws an `ArithmeticException`, `quotient` *terminates* and does *not* return a value, and `quotient`’s *local variables go out of scope* (and are destroyed). If `quotient` contained local variables that were references to objects and there were no other references to those objects, the objects would be marked for *garbage collection*. Also, when an exception occurs, the `try` block from which `quotient` was called *terminates* before lines 30–32 can execute. Here, too, if local variables were created in the `try` block prior to the exception’s being thrown, these variables would go out of scope.

If an `InputMismatchException` is generated by lines 25 or 27, the try block *terminates* and execution *continues* with the catch block at lines 34–41. In this case, method `quotient` is not called. Then method `main` continues after the last catch block (line 48).

11.4 When to Use Exception Handling

Exception handling is designed to process **synchronous errors**, which occur when a statement executes. Common examples we'll see throughout the book are *out-of-range array indices*, *arithmetic overflow* (i.e., a value outside the representable range of values), *division by zero*, *invalid method parameters* and *thread interruption* (as we'll see in Chapter 23). Exception handling is not designed to process problems associated with **asynchronous events** (e.g., disk I/O completions, network message arrivals, mouse clicks and keystrokes), which occur in parallel with, and *independent of*, the program's flow of control.



Software Engineering Observation 11.2

Incorporate your exception-handling and error-recovery strategy into your system from the inception of the design process—including these after a system has been implemented can be difficult.



Software Engineering Observation 11.3

Exception handling provides a single, uniform technique for documenting, detecting and recovering from errors. This helps programmers working on large projects understand each other's error-processing code.



Software Engineering Observation 11.4

There's a great variety of situations that generate exceptions—some exceptions are easier to recover from than others.

11.5 Java Exception Hierarchy

All Java exception classes inherit directly or indirectly from class **Exception**, forming an *inheritance hierarchy*. You can extend this hierarchy with your own exception classes.

Figure 11.4 shows a small portion of the inheritance hierarchy for class **Throwable** (a subclass of `Object`), which is the superclass of class `Exception`. Only `Throwable` objects can be used with the exception-handling mechanism. Class `Throwable` has two direct subclasses: `Exception` and `Error`. Class `Exception` and its subclasses—for instance, `RuntimeException` (package `java.lang`) and `IOException` (package `java.io`)—represent exceptional situations that can occur in a Java program and that can be caught by the application. Class **Error** and its subclasses represent *abnormal situations* that happen in the JVM. Most *Errors* happen infrequently and should not be caught by applications—it's usually not possible for applications to recover from *Errors*.

The Java exception hierarchy contains hundreds of classes. Information about Java's exception classes can be found throughout the Java API. You can view `Throwable`'s documentation at docs.oracle.com/javase/7/docs/api/java/lang/Throwable.html. From there, you can look at this class's subclasses to get more information about Java's `Exceptions` and `Errors`.

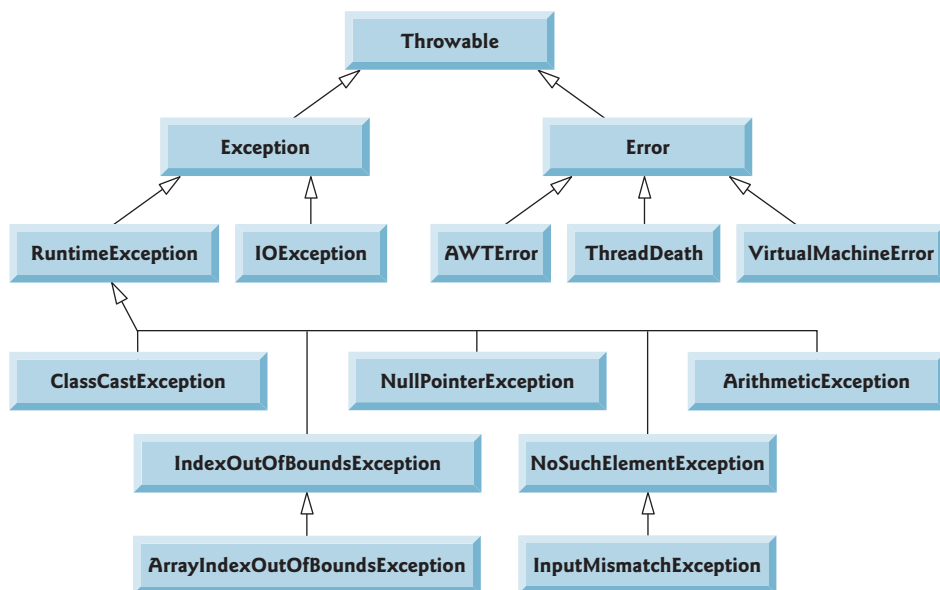


Fig. 11.4 | Portion of class `Throwable`'s inheritance hierarchy.

Checked vs. Unchecked Exceptions

Java distinguishes between **checked exceptions** and **unchecked exceptions**. This distinction is important, because the Java compiler enforces special requirements for *checked* exceptions (discussed momentarily). An exception's type determines whether it's checked or unchecked.

RuntimeExceptions Are Unchecked Exceptions

All exception types that are direct or indirect subclasses of **RuntimeException** (package `java.lang`) are *unchecked* exceptions. These are typically caused by defects in your program's code. Examples of unchecked exceptions include:

- `ArrayIndexOutOfBoundsException` (discussed in Chapter 7)—You can avoid these by ensuring that your array indices are always greater than or equal to 0 and less than the array's length.
- `ArithmeticException` (shown in Fig. 11.3)—You can avoid the `ArithmeticException` that occurs when you divide by zero by checking the denominator to determine whether it's 0 *before* performing the calculation.

Classes that inherit directly or indirectly from class `Error` (Fig. 11.4) are *unchecked*, because Errors are such serious problems that your program should not even attempt to deal with them.

Checked Exceptions

All classes that inherit from class `Exception` but *not* directly or indirectly from class `RuntimeException` are considered to be *checked* exceptions. Such exceptions are typically caused by conditions that are not under the control of the program—for example, in file processing, the program can't open a file if it does not exist.

The Compiler and Checked Exceptions

The compiler checks each method call and method declaration to determine whether the method throws a checked exception. If so, the compiler verifies that the checked exception is *caught* or is *declared* in a throws clause—this is known as the **catch-or-declare requirement**. We show how to catch or declare checked exceptions in the next several examples. Recall from Section 11.3 that the throws clause specifies the exceptions a method throws. Such exceptions are not caught in the method's body. To satisfy the *catch* part of the *catch-or-declare requirement*, the code that generates the exception must be wrapped in a try block and must provide a catch handler for the checked-exception type (or one of its superclasses). To satisfy the *declare* part of the catch-or-declare requirement, the method containing the code that generates the exception must provide a throws clause containing the checked-exception type after its parameter list and before its method body. If the catch-or-declare requirement is not satisfied, the compiler will issue an error message. This forces you to think about the problems that may occur when a method that throws checked exceptions is called.



Error-Prevention Tip 11.2

You must deal with checked exceptions. This results in more robust code than would be created if you were able to simply ignore them.



Common Programming Error 11.2

If a subclass method overrides a superclass method, it's an error for the subclass method to list more exceptions in its throws clause than the superclass method does. However, a subclass's throws clause can contain a subset of a superclass's throws clause.



Software Engineering Observation 11.5

If your method calls other methods that throw checked exceptions, those exceptions must be caught or declared. If an exception can be handled meaningfully in a method, the method should catch the exception rather than declare it.

The Compiler and Unchecked Exceptions

Unlike checked exceptions, the Java compiler does *not* examine the code to determine whether an unchecked exception is caught or declared. Unchecked exceptions typically can be *prevented* by proper coding. For example, the unchecked `ArithmeticException` thrown by method `quotient` (lines 9–13) in Fig. 11.3 can be avoided if the method ensures that the denominator is not zero *before* performing the division. Unchecked exceptions are *not* required to be listed in a method's throws clause—even if they are, it's *not* required that such exceptions be caught by an application.



Software Engineering Observation 11.6

Although the compiler does not enforce the catch-or-declare requirement for unchecked exceptions, provide appropriate exception-handling code when it's known that such exceptions might occur. For example, a program should process the `NumberFormatException` from `Integer` method `parseInt`, even though `NumberFormatException` is an indirect subclass of `RuntimeException` (and thus an unchecked exception). This makes your programs more robust.

Catching Subclass Exceptions

If a catch handler is written to catch *superclass* exception objects, it can also catch all objects of that class's *subclasses*. This enables catch to handle related exceptions *polymorphically*. You can catch each subclass individually if those exceptions require different processing.

Only the First Matching catch Executes

If *multiple* catch blocks match a particular exception type, only the *first* matching catch block executes when an exception of that type occurs. It's a compilation error to catch the *exact same type* in two different catch blocks associated with a particular try block. However, there can be several catch blocks that match an exception—i.e., several catch blocks whose types are the same as the exception type or a superclass of that type. For instance, we could follow a catch block for type `ArithmeticException` with a catch block for type `Exception`—both would match `ArithmeticExceptions`, but only the first matching catch block would execute.



Common Programming Error 11.3

Placing a catch block for a superclass exception type before other catch blocks that catch subclass exception types would prevent those catch blocks from executing, so a compilation error occurs.



Error-Prevention Tip 11.3

Catching subclass types individually is subject to error if you forget to test for one or more of the subclass types explicitly; catching the superclass guarantees that objects of all subclasses will be caught. Positioning a catch block for the superclass type after all other subclass catch blocks ensures that all subclass exceptions are eventually caught.



Software Engineering Observation 11.7

In industry, throwing or catching type `Exception` is discouraged—we use it here simply to demonstrate exception-handling mechanics. In subsequent chapters, we generally throw and catch more specific exception types.

11.6 finally Block

Programs that obtain certain resources must return them to the system to avoid so-called **resource leaks**. In programming languages such as C and C++, the most common resource leak is a *memory leak*. Java performs automatic *garbage collection* of memory no longer used by programs, thus avoiding most memory leaks. However, other types of resource leaks can occur. For example, files, database connections and network connections that are not closed properly after they're no longer needed might not be available for use in other programs.



Error-Prevention Tip 11.4

A subtle issue is that Java does not entirely eliminate memory leaks. Java will not garbage-collect an object until there are no remaining references to it. Thus, if you erroneously keep references to unwanted objects, memory leaks can occur.

The `finally` block (which consists of the `finally` keyword, followed by code enclosed in curly braces), sometimes referred to as the **finally clause**, is optional. If it's

present, it's placed after the last catch block. If there are no catch blocks, the `finally` block, if present, immediately follows the `try` block.

When the finally Block Executes

The `finally` block will execute *whether or not* an exception is thrown in the corresponding `try` block. The `finally` block also will execute if a `try` block exits by using a `return`, `break` or `continue` statement or simply by reaching its closing right brace. The one case in which the `finally` block will *not* execute is if the application *exits early* from a `try` block by calling method `System.exit`. This method, which we demonstrate in Chapter 15, *immediately* terminates an application.

If an exception that occurs in a `try` block cannot be caught by one of that `try` block's catch handlers, the program skips the rest of the `try` block and control proceeds to the `finally` block. Then the program passes the exception to the next outer `try` block—normally in the calling method—where an associated catch block might catch it. This process can occur through many levels of `try` blocks. Also, the exception could go *uncaught* (as we discussed in Section 11.3).

If a catch block throws an exception, the `finally` block still executes. Then the exception is passed to the next outer `try` block—again, normally in the calling method.

Releasing Resources in a finally Block

Because a `finally` block always executes, it typically contains *resource-release code*. Suppose a resource is allocated in a `try` block. If no exception occurs, the catch blocks are *skipped* and control proceeds to the `finally` block, which frees the resource. Control then proceeds to the first statement after the `finally` block. If an exception occurs in the `try` block, the `try` block *terminates*. If the program catches the exception in one of the corresponding catch blocks, it processes the exception, then the `finally` block *releases the resource* and control proceeds to the first statement after the `finally` block. If the program doesn't catch the exception, the `finally` block *still* releases the resource and an attempt is made to catch the exception in a calling method.



Error-Prevention Tip 11.5

The finally block is an ideal place to release resources acquired in a try block (such as opened files), which helps eliminate resource leaks.



Performance Tip 11.1

Always release a resource explicitly and at the earliest possible moment at which it's no longer needed. This makes resources available for reuse as early as possible, thus improving resource utilization and program performance.

Demonstrating the finally Block

Figure 11.5 demonstrates that the `finally` block executes even if an exception is *not* thrown in the corresponding `try` block. The program contains static methods `main` (lines 6–18), `throwException` (lines 21–44) and `doesNotThrowException` (lines 47–64). Methods `throwException` and `doesNotThrowException` are declared `static`, so `main` can call them directly without instantiating a `UsingExceptions` object.

```

1  // Fig. 11.5: UsingExceptions.java
2  // try...catch...finally exception handling mechanism.
3
4  public class UsingExceptions
5  {
6      public static void main(String[] args)
7      {
8          try
9          {
10             throwException();
11         }
12         catch (Exception exception) // exception thrown by throwException
13         {
14             System.err.println("Exception handled in main");
15         }
16
17         doesNotThrowException();
18     }
19
20     // demonstrate try...catch...finally
21     public static void throwException() throws Exception
22     {
23         try // throw an exception and immediately catch it
24         {
25             System.out.println("Method throwException");
26             throw new Exception(); // generate exception
27         }
28         catch (Exception exception) // catch exception thrown in try
29         {
30             System.err.println(
31                 "Exception handled in method throwException");
32             throw exception; // rethrow for further processing
33
34             // code here would not be reached; would cause compilation errors
35         }
36
37         finally // executes regardless of what occurs in try...catch
38         {
39             System.err.println("Finally executed in throwException");
40         }
41
42         // code here would not be reached; would cause compilation errors
43     }
44 }
45
46 // demonstrate finally when no exception occurs
47 public static void doesNotThrowException()
48 {
49     try // try block does not throw an exception
50     {
51         System.out.println("Method doesNotThrowException");
52     }

```

Fig. 11.5 | try...catch...finally exception-handling mechanism. (Part I of 2.)